



面向 21 世纪 课 程 教 材
Textbook Series for 21st Century

高级语言 C++ 程序设计 编程范例与精解

周玉龙 刘 璟

高等教育出版社

前 言

计算机编程技术一直受到国内外各高等学府以及计算机界的高度重视。以面向对象技术为基础的 C++ 程序设计方法与技术代表了目前世界上最先进的计算机编程方法与技术。

学过和正在学习 C++ 程序设计方法与技术的人们可能都会有这样的感受,除了掌握语言的特定语法与使用规则外,学习与研究程序范例的作用也非常重要。通过学习、研究、模仿、进而想方设法改进与扩充程序范例的功能这一过程,也正是自己加深理解、学会使用并进一步提高编程能力的过程。为了对 C++ 语言进行有效使用尽快提高 C++ 程序设计方面的编程技术,能够顺利编制出各种具有较高水平的应用程序,学习研究并参照模仿一套 C++ 程序设计编程范例中所蕴涵的程序设计技术、技巧与经验,无疑是一条迅速提高自己编程技术与水平的捷径。基于上述想法,笔者编写了这本《高级语言 C++ 程序设计编程范例与精解》,以奉献给学习计算机编程技术的广大高校师生、科技人员以及计算机爱好者。

本书是《高级语言 C++ 程序设计》一书的配套教材。是专门针对《高级语言 C++ 程序设计》一书的各章节内容所精选而配制的 213 个编程范例。这些范例隶属于如下的十一个单元:第一章 C++ 语言初步;第二章 基本数据类型与基本运算;第三章 分支与循环处理;第四章 数组及其应用;第五章 函数的设计与应用;第六章 指针、结构体及引用;第七章 类与对象;第八章 继承与派生;第九章 运算符重载;第十章 模板及其应用;第十一章 流类库与输入/输出。

这是一套完整而又具有一定广度与深度的精选编程范例,这些范例以及对它们的求解方法与过程,非常适合于刚刚开始学习 C++ 程序设计的学生、科技工作者以及自学者来参照模仿,具有很高的实用参考价值,可启发读者在程序设计方面的思维积极性,提高面向对象程序设计方面的实用编程技术与技巧。例如,本书具有如下几方面的特色及其相关内容:

(1) 精选了一批具有一定深度的以及通过自定义类(类型)来实现面向对象编程的应用范例,既用于提高编程能力,又为今后开发面向对象的实用性软件奠定了基础。例如,八皇后问题,骑士巡游问题,万年历问题, n 元线性方程组求解,矩阵求逆等具有一定深度的问题;以及一批自定义的类(类型)及其对它们的具体使用——自定义的时间类、日期类、日期时间类、分数类、复数类、矩阵类、栈类、队列类、链表类、计算器类、学生类,等等。这些众多的自定义类中所涉及到的面向对象编程方法与技术是学习 C++ 编程技术的重点与难点,通过较多的编程范例来学习来模仿,进而掌握这些方法与技术是很有必要的,而这些内容在教科书或其他的同类书中则很少见或只能见到其中的某一些。

(2) 对待同一个问题,要设法采用尽可能多的求解处理方法与实现技术,并对各种不同实现方法与技术的特点进行优劣性评价与比较。

学习研究 C++ 语言,灵活掌握与使用 C++ 语言为用户提供的各种数据类型以及用户自定义数据类型,并熟悉对各种不同数据的存储与处理方法和手段,是学好、用好 C++ 语言

的关键所在。为启发学生把学习重点放在编程技术与技巧方面,往往要求对同一问题尽可能采用多种不同解法,这样既可以提高对程序设计语言应用能力及其使用灵活性的理解,同时又能启发学生的思维积极性与创造性。例如对下述的同一个简单问题,本书就设计并给出了十种不同的求解方法(对应于如下编号的范例:1.5、4.1、4.14、5.23、6.7、6.17、6.18、7.4、11.17、11.18)。所要求解的问题为:从键盘输入 10 个 `int` 型数,而后反序输出它们。

十种实现方法的本质区别在于存放输入数据的结构与位置的不同,从而导致不同的程序处理方式。存放数据的位置与处理方式依次为: 1. 使用 10 个 `int` 型变量; 2. 使用 `int` 型数组; 3. 使用 `char` 型字符数组(即字符串); 4. 使用递归函数中的局部变量; 5. 使用通过运算符 `new` 动态申请到的数据空间; 6. 使用指针与链表的栈式存储方式; 7. 使用指针与链表的队列式存储方式; 8. 使用自定义类中的私有数据成员; 9. 使用 `text` 文件; 10. 使用 `binary` 文件。实际上,213 个范例中还包含有许多一题二解、一题三四解的情况。

(3) 通过分析探讨多种不同类型问题的具体实例,来提高对递归函数以及对问题的递归求解方法的理解,从而提高使用技术水平。例如,本书对 15 个不同问题采用了递归求解的方法,形成了 15 个不同的精选范例(5.19~5.25、5.30~5.34、5.40、5.47、5.48)。

递归函数以及对问题的递归求解方法是学习程序设计语言时的一个难点,本书着重从如何进行问题分解入手,通过分析探讨多种不同类型问题的具体实例,来学习这种较难掌握的递归求解问题的方法与技术。即是说,主要是想办法将欲处理的“度为 n 的问题”分解化简为一个或若干个相同性质的“度小于等于 $n-1$ 的问题”以及某些“本原问题”(不必再进行递归处理的“已解问题”)。另外程序员还应注意并保障递归函数在执行若干次后能够退出递归(不再进行递归调用,也即能够实现递归出口)。

(4) 含有若干个与绘图有关的范例及其相关的实现程序。在不同的操作系统与编译实现平台下,涉及到的绘图程序的编制方法可能有所不同。本书所有程序都是在 VC6.0 环境下实现的,绘图程序也不例外。实际上,在 VC6.0 环境下,可首先借助于 AppWizard 来自动生成一个典型的“Hello World”程序框架,而后在所生成的程序框架基础上,增加相关的“绘图”处理类以及处理 `WM_PAINT` 消息的“重画”处理函数等,进而实现所需的与绘图有关的程序功能。

需要注意的是,执行所生成的框架程序后进入的是 Windows 窗口模式(与 Dos 窗口模式不同,该模式支持鼠标操作,是以消息进行驱动执行的),而且框架程序还自动为用户提供了可用的“File”和“Help”菜单以及一些简单的菜单命令项以及可用于进行窗口极大化、极小化、和关闭的鼠标按钮。这正是 VC6.0 之 AppWizard 的诱人之处,它为开发 Windows 程序提供了方便,使程序员不必再去为每一应用编写那些基本相同的框架式底层支撑程序(也即上述自动生成的程序框架),而把注意力放在进行逻辑处理的所谓上层程序部分的编写上,最后再将它们衔接交融成同一个实现绘图功能的程序。

另外,对每一个范例都进行了精解,这意味着,并不是一步给出结果程序,而是把重点放在解释说明如何构造出这一程序,比如要重点解释说明程序编制中使用的数据结构、变量和语句的使用含义与相互关联性、实现方式以及此种实现方式的特点与优缺点评价等方面。实际上,许多看来相互无关的变量、语句、数据结构及实现方式,其实都是相互有关联的,只有深刻理解与体会到这种相互关联性,并能对它们进行合理与灵活的使用,才

可将程序设计技术与水平提升到一个新的高度。

书中所有范例所对应的程序都在 VC6.0 集成环境下调试通过，并附有可供参考的程序执行结果。

本书内容的选取与定夺是作者多年教学与科研中使用 C++ 经验的总结积累与结晶。我们期待并相信，本书的编程范例中所蕴涵的程序设计技术、技巧与经验必定会对广大读者起到较大的帮助与启发作用。

书中可能存在不少的错误与不足，敬请阅读本书的老师 and 同学们予以批评指正。

编者

2002 年于南开大学

内容提要

本书是《高级语言 C++程序设计》一书的配套教材。是专门针对《高级语言 C++程序设计》一书的各章节内容所精选而配制的 213 个编程范例。这些具有一定广度与深度的精选范例以及对它们的求解方法与过程，非常适合刚刚开始学习 C++程序设计的学生、科技工作者以及自学者来参照模仿，具有很高的实用参考价值，可启发读者在程序设计方面的思维积极性，提高面向对象程序设计方面的实用编程技术与技巧。

出版发行	高等教育出版社	购书热线	010-64054588
社 址	北京市东城区沙滩后街 55 号	免费咨询	800-810-0598
邮政编码	100009	网 址	http://www.hep.edu.cn
传 真	010-64014048		http://www.hep.com.cn
经 销	新华书店北京发行所		
印 刷			
开 本	787×1092 1/16	版 次	年 月第 版
印 张	36		年 月第 版
字 数	820 000	印 次	年 月第 次印刷
定 价	39.00 元		

本书如有缺页、倒页、脱页等质量问题，请到所购图书销售部门联系调换。

版权所有 侵权必究

目 录

第一章 C++语言初步	1
1.1 运行最简单程序	3
1.2 在屏幕上显示3行字符	3
1.3 对两整数进行四则运算	4
1.4 计算圆的面积与周长	5
1.5 输入10个整数而后反序输出的实现方式一	6
第二章 基本数据类型与基本运算	8
2.1 算术运算符功能的理解与使用	10
2.2 关系运算符和逻辑运算符功能的理解与使用	12
2.3 位运算符功能的理解与使用	13
2.4 表达式加括号使运算结果不变	15
2.5 使用自定义枚举类型	18
2.6 求两个时间(时刻)的时间差	19
2.7 计算x的平方、平方根及倒数等	21
2.8 一元二次方程求根	21
第三章 分支与循环处理	23
3.1 求三个实数中的最大数与最小数	24
3.2 设计一个计算商品价格的程序	25
3.3 switch语句的执行效果	27
3.4 将考试成绩等级字母变换为相应的百分制分数段	28
3.5 求实数a的n次方	30
3.6 求出正整数n的所有因子	30
3.7 求正整数n的所有因子的平方根之和	31
3.8 n个实数的整数部分和以及小数部分和	32
3.9 求最小公倍数	33
3.10 求最大公约数	34
3.11 改写程序使其中不出现switch与for语句	35
3.12 反序输出某正整数的各数位值	37
3.13 求正整数n之反序数的两倍值	38
3.14 验证奇数的指定特性	39
3.15 按近似公式求近似圆周率	40
3.16 三种控制语句的使用	41

3.17	求三种不同序列的累加和	43
3.18	求满足 eps 精度的累加和	45
3.19	处理从键盘输入的一批字符	46
3.20	将读入字符“拼装成”实数并显示出来	48
3.21	求 1 000 以内的所有“完全数”	52
3.22	求 100 ~999 内的所有“回文数”	53
3.23	编循环程序显示指定图形	54
3.24	理解二重循环程序的执行过程	55
3.25	根据要求显示具有不同特征的图形	56
3.26	循环累加不同的项数求出多个不同近似程度的 E 值	58
3.27	求满足给定两条件的最小自然数 n	59
第四章 数组及其应用		61
4.1	输入 10 个整数而后反序输出的实现方式二	62
4.2	二进制数转化为十进制数	63
4.3	十进制正整数转化为二进制	64
4.4	求字符串长度	65
4.5	两个字符串连接	66
4.6	字符串比较	67
4.7	求 1 000 以内的所有“完全数”并将结果按指定格式输出	69
4.8	求数组中的最大数和最小数	71
4.9	三种控制语句与数组的配合使用	72
4.10	对数组中的 10 个元素进行排序	74
4.11	合并两个有序数组	75
4.12	罗马数据转化为十进制数据	76
4.13	十进制正整数转化为罗马数据	78
4.14	使用最少数人民币纸币方案的求取	81
4.15	输入 10 个整数而后反序输出的实现方式三	83
4.16	计算某一天是星期几(方式一)	85
4.17	屏幕显示某年某指定月的月历(方式一)	86
4.18	求取正整数“圈”中连续四数之和最大者	89
4.19	将正整数转换为“英文表示字符串”	90
4.20	二维整型数组按行按列求出各累加和	92
4.21	二维数组的简单应用	93
4.22	二矩阵相加	95
4.23	处理二维字符数组	97
4.24	统计各行“单词”个数的处理方案一	99
4.25	统计各行“单词”个数的处理方案二	101
4.26	二矩阵相乘	103
4.27	在屏幕上“画出”最近 15 天的气温值柱形图	105
4.28	统计每一正整数的出现频率	107

第五章 函数的设计与应用	110
5.1 自定义函数及其调用	112
5.2 一般自定义函数及其调用	113
5.3 设计无参函数实现固定显示功能	114
5.4 设计一参函数使显示行数可变化	115
5.5 设计二参函数使变化更灵活	116
5.6 全局变量与无参函数配合使用	117
5.7 将十进制数转化为二进制数的自定义函数	118
5.8 将十进制数转化为八进制数的自定义函数	119
5.9 返回布尔值的函数	121
5.10 判断整数 x 是否仅由奇数字组成的自定义函数	122
5.11 判断正整数 n 是否为“回文数”的自定义函数	123
5.12 判断整数 n 是否为素数的自定义函数	124
5.13 返回整数 x 某一指定位之数值的函数	126
5.14 三种控制语句与数组及函数配合使用	127
5.15 计算两个日期之间的连续天数	130
5.16 计算出两个日期的间隔天数	133
5.17 计算某一天是星期几(方式二)	137
5.18 屏幕显示某年某指定月的月历(方式二)	139
5.19 使用三种方法求 Fibonacci 数列的某个项值	141
5.20 函数的递归调用	143
5.21 使用递归解法求实数 a 的 n 次方	145
5.22 使用递归方法打印一批数据	146
5.23 输入 10 个整数而后反序输出的实现方式四	147
5.24 按递归方法求最大公约数	148
5.25 读非递归函数后添加具有相同功能的递归函数	149
5.26 求字符串长度的函数	151
5.27 二进制数转化为十进制数的函数	152
5.28 查看数组中满足某条件之数据的个数	153
5.29 在升序数组中折半查找某元素序号的函数	154
5.30 显示数组某下标范围内所有元素的递归函数	156
5.31 判断两数组前 n 个元素值是否完全相同的递归与非递归函数	157
5.32 判断某数组前 n 个元素是否完全有序的递归与非递归函数	159
5.33 判断 s 字符串是否为“回文”的递归函数	160
5.34 函数的间接递归	162
5.35 两个二进制正整数的四则运算	164
5.36 两个罗马数据的四则运算	168
5.37 将 ASCII 码数字字符串转化为整数的函数	172
5.38 将整数转化为 ASCII 码数字字符串的函数	174
5.39 实现插入排序的 InsertionSort 函数	176
5.40 分析递归函数的执行结果	177

5.41	对两个字符数组实现某些统计功能的函数	179
5.42	不同作用域中同名标识符的处理	181
5.43	文件级作用域以及函数级作用域的变量	183
5.44	全局变量以及具有 static 存储类别变量的使用	185
5.45	extern 型外部变量的使用	186
5.46	与全局变量以及静态局部变量相关的使用	189
5.47	求解骑士巡游问题——为骑士求解巡游“路线图”	190
5.48	求解八皇后问题	195
第六章 指针、结构体及引用		199
6.1	与字符指针相关的使用	203
6.2	指针及其相关运算	204
6.3	判断字符串是否为合法标识符	205
6.4	在字符串数组中查找给定串 string 的出现位置	207
6.5	通过指针参数来实现“双向传值”	209
6.6	字符指针与其他类型指针的使用区别	211
6.7	输入 10 个整数而后反序输出的实现方式五	213
6.8	测试并运行含有 new 与 delete 的指定程序	214
6.9	实现具有动态大小的矩阵乘法	216
6.10	求解线性方程组	219
6.11	赋值参数、指针参数以及引用参数的使用区别	225
6.12	求满秩矩阵的逆矩阵	227
6.13	有关指针数组(各分量均为指针)的使用	232
6.14	指向一维数组的指针及其使用	234
6.15	结构体与数组两种数据结构的结合使用	236
6.16	添加函数完成与结构体有关的对复数的某些指定运算	239
6.17	输入十个整数而后反序输出的实现方式六	242
6.18	输入十个整数而后反序输出的实现方式七	244
6.19	使用由结构体形成的链表来实现数据排序	246
6.20	将某个有序链表合并(插入)到另一个有序链表之中	248
6.21	合并两个有序链表形成一个新的有序结果链表	252
6.22	利用链表存储字符串并实现相关的字符串处理	257
6.23	对函数指针的使用	261
6.24	使用返回指针值的函数	263
6.25	求取源串中首次出现某子串的头字符位置	264
6.26	通过带参数的 main 函数实现指定的四则运算	267
6.27	引用的建立及简单使用	269
第七章 类与对象		271
7.1	自定义类(类型)及其相关的构造与析构函数	274
7.2	动态对象的构造与析构	276

7.3 自定义的 Circle 类(类型)以及对它的使用	278
7.4 输入十个整数而后反序输出的实现方式八	280
7.5 自定义类及其有关类成员函数的使用	282
7.6 一个简单的复数类型 complex	285
7.7 学生类(类型)student	288
7.8 自定义的 queue(队列) 类型	291
7.9 在平面上用直线、矩形和圆画出一个粗略的汽车图形	295
7.10 含有静态数据成员及静态函数成员的自定义类	303
7.11 自定义类中含有常量数据成员	305
7.12 友元函数的使用	306
7.13 在复数类型 complex 中使用多个友元函数	309
7.14 简单的分数类(类型)fraction	312
7.15 “ 后缀式简单单步计算器 ” 类(类型)Calculator	319
7.16 自定义时间类(类型)TimeType	329
7.17 在自定义时间类(类型)TimeType 中使用多个友元函数	336
7.18 自定义日期类(类型)DateType	343
7.19 自定义的计算器类(类型)compu	350
7.20 求满秩矩阵逆矩阵的自定义类 Matrix	360
第八章 继承与派生	366
8.1 继承与派生关系中相关成员的存取权限	368
8.2 学生教师档案的管理	372
8.3 派生类的构造函数及其对象的初始化	378
8.4 派生类的构造与析构函数及其对象的初始化	380
8.5 使用类定义方式求解线性方程组	383
8.6 通过组合方式自定义日期时间类(类型)DateTimeType	389
8.7 通过继承方式自定义日期时间类(类型)DateTimeType	394
8.8 虚函数的使用	398
8.9 通过虚函数手段按不同方法来计算定积分	401
8.10 增加梯形图元类后在平面上画出简单图形	407
8.11 派生关系中的赋值兼容性以及重名成员处理等	418
第九章 运算符重载	422
9.1 通过友元方式重载运算符	425
9.2 通过类成员方式重载运算符	428
9.3 自定义简单分数类(类型)fraction 中的运算符重载	431
9.4 在处理链表的 link 类中重载下标运算符	439
9.5 重载全局运算符 new 与 delete	442
9.6 对自定义类重载运算符 new 与 delete	444
9.7 全权控制运算符 new 与 delete 的分配与释放细节	447
9.8 自定义的类型转换运算符(函数)	450

9.9 成员指针运算符的使用.....	451
9.10 自定义矩阵 <code>matrix</code> 类(类型)中的运算符重载	453
第十章 模板及其应用	467
10.1 对一维数组进行排序的函数模板	469
10.2 二维数组中找最大元和最小元的函数模板	471
10.3 类模板的使用.....	474
10.4 既使用类型参数又使用普通参数的类模板	477
10.5 为链表类模板 <code>list</code> 补充成员函数	479
10.6 实现队列类模板 <code>queue</code>	484
10.7 实现数组类模板 <code>array</code>	488
10.8 使用了栈类模板的计算器类(类型) <code>compu</code>	493
10.9 类模板作基类派生出类模板	502
第十一章 流类库与输入/输出	506
11.1 插入与提取运算符的两种使用方式	509
11.2 对自定义 <code>fraction</code> 类重载输入运算符“ <code>>></code> ”与输出运算符“ <code><<</code> ”.....	511
11.3 使用格式控制符将数据输出到屏幕上	514
11.4 使用格式控制符将数据输出到 <code>text</code> 型磁盘文件中	515
11.5 从文件 <code>cin</code> 以抽取算符读入各字符的方式一	517
11.6 从文件 <code>cin</code> 以抽取算符读入各字符的方式二	519
11.7 从文件 <code>cin</code> 以 <code>get</code> 函数读入各字符	520
11.8 从磁盘文件 <code>fin</code> 以抽取算符读入各字符	521
11.9 使用 <code>get</code> 与 <code>put</code> 对指定文件进行拷贝	522
11.10 对磁盘文件的读/写处理(数值性数据)	523
11.11 对磁盘文件的读/写处理(文本性数据).....	528
11.12 使用 <code>getline</code> 读出 <code>text</code> 文件中的“一篇文章”进行处理	531
11.13 使用 <code>getline</code> 读出 <code>text</code> 文件中的 C++ 源程序并删除注释	533
11.14 读出两个指定文件的各行内容进行比较	536
11.15 使用算符“ <code>>></code> ”与“ <code><<</code> ”对自定义的 <code>text</code> 型文件进行读/写	539
11.16 使用 <code>write</code> 与 <code>read</code> 对自定义的 <code>binary</code> 型文件进行读/写	542
11.17 输入 10 整数而后反序输出的实现方式九	545
11.18 输入 10 个整数而后反序输出的实现方式十	547
11.19 <code>binary</code> 型学生数据文件的简单管理	548
11.20 对 <code>binary</code> 型学生数据文件管理程序进一步扩充	552
参考文献	562

第一章 C++语言初步

[语法要点]

详细内容可参阅教材第二章。

(1) 每一个 C++ 程序都是一个基本符号的序列，基本符号包括大小写英文字母、数字(0~9)和特殊符号(33 个)。

(2) 注释可以提高程序的可读性，在程序的执行中不起作用，在编译时被跳过。符号//表示该行中其右的所有内容为注释。符号“/*”和“*/”表示在其间的一行或多行内容为注释。

(3) C++ 程序中的基本语义单元是单词，由空格或分割符划分；若干单词按一定语法规则组成语句，例如，int a,b=3; 是一个说明语句，C++ 程序是一个语句序列。

C++ 的单词分为五类：关键字(key word)，标识符(identifier)，字面常量(literal constant)，运算符(operator)和分割符(separator)。

关键字 由系统指定，有固定含义，如 include、int、const、float 等。

标识符 是由用户指定的以字母或下横线开头的字母、数字、下横线序列，用来给变量、常量、函数、类、对象等语法元素起名，不可与关键字相同，区分字母的大小写(即 Abc、abc 是两个不同的标识符)，有一定的有效长度。

字面常量 包括十进制、八进制、十六进制整数、浮点数、字符常量(如“ A ”、“ 3 ”、“ = ”)和字符串常量(如“ Hello world!”)。

运算符 一般由一个或多个特殊符号组成。如+、%、==、*=；也有些关键字如 sizeof、new、delete 也同时分列入运算符。

分割符 用来界定或分割其他语法成分。包括：空格、“ ”、#、()、/*、*/、//、’、;、{ } 等。

(4) 主函数 **main** 是 C++ 程序中惟一必不可少的成分，最短的 C++ 程序为 main (){ }，它是合法的，但什么也不做。C++ 程序中的主函数是整个程序的主控模块；又是程序的入口；还是程序与其运行环境的接口。

主函数名 main 是系统指定的标识符，主函数的返回类型为 void 或 int (可缺省)，一般主函数没有参数，其首部为 void main(void)。

(5) 预处理命令 用来完成 C++ 程序编译前的“预处理”工作。预处理命令必须由“#”符号开头。预处理命令是从 C 语言中继承下来的，由于 C++ 语言本身和编程环境的发展，目前采用最多的是文件嵌入命令(# include)。

include 命令 用来把另一个称为头文件的内容在编译前嵌入到 # include 所在位置，与下面的程序一同作为源程序参加编译。

define 命令 称为宏定义命令，例如，# define PI 3.14，相当于把 PI 说明为常量 3.14，由于 C++ 中主张使用常量说明及内联函数，所以在 C++ 语言中 #define 命令的使

用比 C 语言中少得多。

if 和 **# endif** 等条件编译命令, 用来根据条件决定源程序中哪些参加编译, 哪些不参加编译。

(6) 输入/输出(I/O) C++语言的标准 I/O 语句简明方便。

为了使用 **cout** 语句和 **cin** 语句进行输出输入, 要使用文件嵌入命令“**# include <iostream.h>**”, 用来把系统提供的标准 I/O 流类的说明包含到源程序中。

cout 是系统指定的标识符, 用来表示输出的目标, 这里指屏幕, “**<<**”称为插入运算符。

cin 则表示输入的来源, 这里指键盘, “**>>**”称为提取运算符。

endl 标准标识符, 表示“回车”, 它与字符“**\n**”起相同作用, **cout << endl; cout << “\n”**, 都使打印头或光标移到下一行的左端。

C++语言保留了 C 语言中用于 I/O 操作的标准函数库, 其说明在头文件 **stdio.h** 中, 与上述 **cout <<**、**cin >>** 对应的 I/O 函数为 **printf()** 和 **scanf()**, 读者可参见教材附录 B。

(7) C++程序的运行

从编辑到运行 过程为编辑—预处理—编译—链接—运行。

在实际操作过程中, 编辑是由人工在计算机上完成的, 其余几个步骤则可由计算机完成, 不过如果出现故障, 调试的过程当然应由人工在计算机上完成。

Visual C++的集成环境 可以支持采用 C++语言进行的各种软件开发, 把开发过程按工程项目来管理, 因此其功能齐全, 系统复杂。与本书相关的上机实习题目一般比较简单, 只需要把从编辑到运行一个程序的基本过程搞清楚就可以了, 下面是最基本的操作步骤:

①在“程序”表中选择 **Microsoft Visual C++ 6.0**, 进入到 **Visual C++ 6.0** 的集成环境窗口;

②在“**File**”菜单下选择“**new**”项, 出现“**new**”对话框;

③在“**new**”对话框, 点击“**Projects**”标签, 在对话框中, 选择“**win32 console Application**”(win32 控制台应用); 在“**Projects name**”位置下输入项目名称(例如: **demo**); 在“**Location**”位置下, 选择工作区空间的位置, 就是确定新建的以 **demo** 为名的工作区目录的地址(例如: **d:\mydir1**), 完成后按“**OK**”按钮;

④在下一对话框中, 由用户选择项目类型, 例如, 可选择“**An empty project**”, 完成后按“**Finish**”按钮, 并在下一“**报告**”窗口选择表示接受的“**OK**”按钮;

⑤进入编程环境窗口, 窗口分为四部分。上部: 菜单和工具条; 中左: 工作区(workspace)显示窗口, 这里显示工作中与项目相关的各种文件种类等信息; 中右: 视图区, 是显示和编辑程序文件的操作区; 下部: 输出区, 程序调试过程中进行编译、链接、运行时输出的相关信息在这里显示。

⑥选择“**Project/Add To Project/new**”, 出现对话框, 点击“**Files**”并在出现的文件类型表中选择“**C++ Source File**”, 确认使右上角的项目名文本框中为“**demo**”, 其下的“**File**”文本框中添入一个源文件名, 如“**main**”, 并确认使下面的“**Location**”文本框中为工作区目录的位置路径“**d:\mydir1**”, 按“**OK**”回到主环境窗口。

⑦在屏幕中右的视图区(也即显示和编辑程序文件的操作区)中输入源程序。

⑧ 在“ Build”菜单下，选“ compile main.cpp”，进行编译；查看屏幕下部的输出区即“ output”区的输出信息，如有出错信息，修改源程序重新编译。

⑨ 编译通过后，选“ Build”菜单的“ Build demo.exe”进行链接，如有出错信息，进行修改。

⑩ 选“ Build”菜单的“ Execute demo.exe”运行程序。

上述给出的是最基本的操作步骤，实际上，也可通过某些其他的选择或操作步骤来实现运行一个程序的目的，随着对 VC6.0 集成环境的逐步熟悉，将会一步步提高对它的使用技术与应用技巧。

1.1 运行最简单程序

在计算机上对教材 2.1.1 节的 program2_1.cpp 进行编辑、编译、连接、运行。

[分析提示]

参照教材附录 A，对程序进行编辑、编译、连接、运行。

[解答程序]

```
//program2_1.cpp
#include <iostream.h>
void main(void)
{
    cout<<"Let's learn to write a C++ program.";
}
```

[输出结果]

Let's learn to write a C++ program.

1.2 在屏幕上显示 3 行字符

编程序，执行后在屏幕上显示出如下的 3 行字符：

Hello, everyone!

How are you!

Good-by!

[分析提示]

通过使用如下形式的语句来显示 1 行字符：cout<<".....\n";其中的省略号代表要显示的字符串本身，最后的“ \n”为转义字符，表示换行。

[解答程序]

```
#include <iostream.h>

//要往标准流对象 cout(即屏幕)上输出信息, 必须包含头文件
// " iostream.h"

//以双斜杠(" //")开头直到行末的全体(符号)称为注释。
//在编译时注释将被当做空格处理

void main() //任何一个 C++程序都含有一个必须叫做 main 的主函数
{
    cout<<"Hello, everyone!\n"; //显示第一行, 靠最后的" \n" 实现换行
    cout<<"How are you!"<<"\n"; //显示第二行
    cout<<"Good-bye!"<<endl; //显示第三行, endl 的功能与" \n" 相同,
    //实现换行
}
```

[输出结果]

```
Hello, everyone!
How are you!
Good-bye!
```

1.3 对两整数进行四则运算

编程序, 从键盘输入两个 int 型(整型)数据, 而后通过使用运算符+、-、*、/、%来计算这两个数的和、差、积、商以及余数, 并将结果显示在屏幕上。

[分析提示]

(1) 要从键盘为变量输入数据, 可使用如下形式的语句来完成:

```
cin >> <变量> >> ... >> <变量>;
```

(2) cout 的输出项可以是表达式, 此时由 cout 负责将计算出的表达式结果值显示出来。如“ cout<<a+b;”。

[解答程序]

```
#include <iostream.h>

void main(void)
{
    int a,b; //说明 a、b 为两个 int 型(整型)变量
    cout<<"Input 2 integers:"; //屏幕提示进行输入
    cin>>a>>b; //从键盘输入两个数赋值给变量 a 与 b(两数据以空格分割)
    cout<<a<<"+"<<b<<"="<<a+b<<endl;
```

```
                                //在屏幕上显示出两数的和(和由计算机负责算出)
cout<<a<<"-"<<b<<"="<<a-b<<endl;    //在屏幕上显示出两数的差
cout<<a<<"*"<<b<<"="<<a*b<<endl;    //两数的积
cout<<a<<"/"<<b<<"="<<a/b<<endl;    //两数的商
cout<<a<<"%"<<b<<"="<<a%b<<endl;    //两数的余数
}
```

[输出结果]

Input 2 integers:11 3

11+3=14

11-3=8

11*3=33

11/3=3

11%3=2

1.4 计算圆的面积与周长

编程序实现：用户从键盘输入圆的半径(任一个浮点数 **radius**)，由程序负责计算出其相应的圆的面积与周长，并将计算结果显示出来。

[分析提示]

(1) 说明 **float** 类型的变量 **radius**、**area** 以及 **circum**，分别用来表示圆的半径、面积与周长；并说明一个 **float** 型的有名常量 **pai**，且将它的值设定为字面常量 3.14。

(2) 通过 **cin** 输入半径的值，而后通过“**area=pai*radius*radius;**”以及“**circum=2*pai*radius;**”来计算出相应的圆的面积与周长。

[解答程序]

```
#include <iostream.h>

/*
    若要往标准流对象 cout(即屏幕)上输出信息，或者要从标准流对象 cin(即键盘)上输入
    信息，必须包含头文件“ iostream.h”
*/

//以有序双符号“ /*” 开始(往后)，直到第一次遇见有序双符号“ */” 为止的全体(符号)
//也称为注释。此种方式给出的注释可以占一行或多行

void main()
{
    const float pai=3.14; //说明一个 float 型(浮点型)有名常量 pai
                          //并将它的值设定为字面常量 3.14，该有名常量 pai 的值在程序中保持不变
}
```

```

float radius; //说明一个 float 型(浮点型)变量 radius, 用于表示圆的半径
//并说明一个 float 型(浮点型)变量 circum, 用于表示圆的周长
cout<<"Enter radius:";
//在屏幕上显示“ Enter radius:” 符号串, 提示用户输入半径值
cin>>radius;
//等待用户从键盘输入一个浮点数, 并将其赋值给变量 radius
float area=pai*radius*radius;
//说明一个浮点型变量 area, 并将三个变量的乘积赋值给它
cout<<"\nThe area of circle is "; //在屏幕上先换行(“ \n” 的功能)
//而后显示出: “ The area of circle is” 符号串
cout<<area<<endl;
//在屏幕上紧接着显示 area 变量的值并换行
float circum=2*pai*radius;
//说明一个浮点型变量 circum, 并将三个变量的乘积赋值给它
cout<<"The circum of circle is "<<circum<<endl;
//在屏幕上先显示出: “ The circum of circle is” 符号串
//而后紧接着显示 circum 变量的值并换行
}

```

[输出结果]

```

Enter radius:2.5
The area of circle is 19.625
The circum of circle is 15.7

```

1.5 输入十个整数而后反序输出的实现方式一

读下述程序, 并上机进行验证, 看是否实现了输入十个整数而后反序输出的预期目标。注意: 本程序使用十个 int 型变量存放输入数据。

```

#include <iostream.h>
void main(void)
{
    int a1,a2,a3,a4,a5,a6,a7,a8,a9,a10;           //注 1
    cout<<"Input 10 integers:"<<endl;             //注 2
    cin>>a1>>a2>>a3>>a4>>a5>>a6>>a7>>a8>>a9>>a10;    //注 3
    cout<<"---- The result ----"<<endl;
    cout<<a10<<" "<<a9<<" "<<a8<<" "<<a7<<" "<<a6<<" "; //注 4
    cout<<a5<<" "<<a4<<" "<<a3<<" "<<a2<<" "<<a1<<endl;
}

```

[分析提示]

- (1) 说明十个 `int` 型变量，并通过 `cin` 依次为各变量输入数据。
- (2) 通过 `cout` 对各变量进行输出，但输出变量的顺序与 `cin` 中的使用顺序正好相反。

[解答程序]

具体程序见题目处。

注 1：使用 10 个 `int` 型变量存放数据。

注 2：提示输入 10 个整数。

注 3：输入 10 个数据放入各变量。

注 4：按输入的相反顺序输出它们，数间以空格相分割。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```


第二章 基本数据类型与基本运算

[语法要点]

详细内容可参阅教材第三章。

(1) C++语言对于数据的类型划分是严格的,它所支持的数据类型很多,特别是通过类(class)的机制,可实现用户任意定义数据类型,其支持的数据类型可分为如下三类:

① 基本类型及派生类型 这是完全由系统提供的,是所有类型数据构造的基础,包括 int、float、char、void 类型(新的 C++版本把 bool 类型由自定义类型归为基本类型)。

基本类型的派生类型主要是在值的集合上有些变化,常用的有 unsigned char、unsigned int、long int 等。

② 用户定义类型 由用户以类(class)形式定义。

在 C++语言中,结构(struct)和联合(union)已与 C 语言不同,其功能类似于类,也可定义新的类型。枚举(enum)类型,实际上是 int 类型的子集,只是一种由用户参与定义的类型。

③ 导出类型 是可通过任何其他类型来定义的类型,包括数组(array)、指针(pointer)和引用(reference)。

例如, int 类型数组, pointer(指针)数组,用户定义的 set(集合)类型的数组,指针、引用同样可由各种类型导出。值得注意的是: C++语言之前的 C 语言中有指针类型,无引用类型; C++语言之后的 Java 语言中无指针类型,有引用类型。

(2) 基本类型的数据在程序中的主要形式为变量和常量。

变量必须说明: 赋名(标识符), 指定类型(分配一定空间); 一般要赋初值。

常量分为有名常量和字面常量。

有名常量要说明: 赋名; 指定类型; 赋一个不可改变的值。

字面常量可有四种类型(int, float, char, string), 字面常量不需说明,可直接出现在程序中。其中的 string 表示字符串类型(注意, string 并不是 C++预定义的类型名)。

在程序中被说明的变量(和有名常量)有一定的生存期和作用域,应遵循以下规则:

① 先说明后使用;

② 在所有程序块{ }外说明的为全局量,生存期从说明到整个任务结束;

③ 在一个变量(或有名常量)作用域中间,同一名称若有另一说明,则原变量的作用域应减去后者的作用域。

(3) 语句 C++程序由一个个语句组成,每个语句完成一种功能或动作。

C++程序中包括:

① 说明语句 包括变量说明、常量说明以及类、对象、类型、函数及其参数的说明,说明语句不仅仅是指出语法成分的名字和类型,也包括创建和初始化工作。

② 表达式语句 是程序中对数据进行处理和操作的语句,例如,赋值语句,函数调

用语句，输入输出语句。

③ 控制语句 是程序中用来控制语句执行次序的语句，例如，if 语句、for 语句、break 语句等等、它使得程序中的语句可以按一定的条件但不一定按书写的顺序去执行。执行语句中往往包含说明语句、表达式语句、复合语句。

④ 复合语句 将若干语句 $S_1; S_2; \dots; S_n$ 用花括号括起来； $\{ S_1; S_2; \dots; S_n \}$ 就组合成了一个复合语句，所以 C++ 程序中的语句概念是嵌套的。

⑤ 空语句 是什么都不做的语句，这是 C++ 语言中的重要概念，例如， $\{ \text{int } a=3; ; \}$ ，这是一个由两个语句组成的复合语句，第二个语句是空语句。

(4) 运算符 C++ 语言提供了最丰富的运算，每种运算符支持一种或多种运算。

C++ 语言中由系统提供的数据类型称为基本数据类型及其派生类型。C++ 语言中的运算符代表了系统提供的对各种数据的处理和操作。C++ 语言允许用户定义新的类型，它也允许用户通过函数和运算符重载来重新定义新的运算符操作。

按运算分量个数分类：单目运算符，如， $-$ 、 $\&$ 、 $++$ 、 $--$ 、 $!$ 等；双目运算符有， $+$ 、 $*$ 、 $\&\&$ 、 $=$ 、 $<=$ 等；多目运算符只有 $?:$ 、 $;$ ；函数调用符 $()$ 实际上是一种多层运算符。

C++ 语言中的运算符可以分为下面几种情形：

① 人们常见且习惯使用的，如 $+$ 、 $-$ 、 $*$ 、 $/$ 、 $>$ 、 $<$ 等，与平时使用一致，最易掌握。

② 人们平时习惯使用，但不用作运算的，如 $()$ 、 $=$ 、 $[]$ ，要注意， $()$ 是函数调用符， $=$ 是赋值运算符， $[]$ 是下标运算符。

③ C++ 程序中常用但与其他地方用法不同的运算符，如 $++$ 、 $--$ 、 $!=$ 、 $==$ 、 $*=$ 、 $\&\&$ 等，应掌握它们的用法。

④ 还有一些运算符由关键字表示，如 `sizeof` 的 `new`、`delete` 等，由此也可以进一步体会在 C++ 语言中，运算符概念与一般数学中运算符的概念的不同。

(5) 表达式 这是 C++ 语言中的又一个重要概念，它与上面提到的表达式语句在概念上不完全相同。例如， $a+b$ 是表达式，但不是表达式语句，表达式的定义可以描述为：

① 单个的变量或常量是表达式；

② 由表达式作运算分量与运算符，按规则组成的运算式是表达式。

例如， a 、 3.5 、 pai 是表达式(设 a 、 pai 已说明)，则 $a*b+c$ 也是表达式，这是因为 a 、 b 、 c 是表达式， $a*b$ 也是表达式，故 $a*b+c$ 是表达式。因此表达式可以由许多变量、常量和运算符组成。

(6) 表达式的二义性避免

$a+b*c$ 的计算可有两种次序 $(a+b)*c$ 和 $a+b*c$ ，为避免这种不确定性，C++ 语言提供三种确定计算的先后次序的方法：

① 括号 $()$ 优先：表达式中任意的子表达式可以加括号，计算时括号内的部分优先；如 $a*(b+c)$ ，先做 $b+c$ ，然后与 a 相乘。

② 没有括号的地方，不同的运算按运算符的优先级的顺序进行计算。优先级高的先计算，如 $a+b*c$ ，先做 $b*c$ ，然后与 a 相加。

③ 具有相同优先级的运算，按左结合或右结合两种规则顺序计算。

左结合：自左向右顺序，如 $a+b-c$ ，等价于 $(a+b)-c$ 。

右结合：自右向左顺序，如 $*\&a$ 等价于 $*(\&a)$ ，又如 $b=c=4$ 等价于 $b=(c=4)$ 。

在教材的 3.4.10 一节中给出了 C++运算符的优先级表。注意，位运算的运算符&、^、| 应为三个不同的优先等级(由高向低)，而逻辑运算的运算符&&与|| 为两个不同的优先级，&&优先级较高。

2.1 算术运算符功能的理解与使用

下述程序执行后，会输出什么结果？

```
#include <iostream.h>
void main(void)
{
    int i=100, j, k;
    char c1='E', c2;
    double x=2.23, y=4.567, z;
    cout<<"int i=100, j, k;"<<endl;           //注 1
    cout<<"char c1='E', c2;"<<endl;
    cout<<"double x=2.23, y=4.567, z;"<<endl;
    cout<<"-----"<<endl;
    cout<<"A'+12="<<'A'+12<<endl;           //注 2
    cout<<"a'-'A'="<<'a'-'A'<<endl;         //注 3
    cout<<"2*'A'+B/5="<<2*'A'+B/5<<endl;     //注 4
    cout<<"11/3="<<11/3<<endl;               //注 5
    cout<<"11%3="<<11%3<<endl;               //注 6
    cout<<"11/3.0="<<11/3.0<<endl;           //注 7
    j=k--i+c1+x;                             //注 8
    cout<<"After 'j=k--i+c1+x; i="<<i;
    cout<<" j="<<j<<" k="<<k<<endl;
    c2=c1++;                                  //注 9
    cout<<"After 'c2=c1++; c2="<<c2<<" c1="<<c1<<endl;
    y+=i*2;                                   //注 10
    cout<<"After 'y+=i*2; y="<<y<<endl;
    z=x*=(k=100)+1;                           //注 11
    cout<<"After 'z=x*=(k=100)+1; k="<<k;
    cout<<" x="<<x<<" z="<<z<<endl;
    j=5;
    cout<<"After 'j=5; ++j="<<++j<<endl;     //注 12
    j=5;
    cout<<"After 'j=5; --j+="<<--j++<<endl;   //注 13
}
```

[分析提示]

本题对 `int`、`char` 及 `double` 类型的数据进行多种算术运算，并通过使用 `cout` 来输出各计算表达式的结果值。

[解答程序]

具体程序见题目处。

注 1: 输出三行提示，是各变量的类型说明信息。

注 2: 字符“ A ”参加运算时，使用其 ASCII 码值 65。

注 3: 字符“ a ”的 ASCII 码值为 97。

注 4: 整数除以整数“ B ” /5 的结果仍为整数(甩掉小数部分)。

注 5: 整数除以整数 11/3 的结果仍为整数(甩掉小数部分)。

注 6: 求 11 除以 3 的余数。

注 7: 整数除以实数 11/3.0 的结果为实数。

注 8: i 先减 1, 再去参加运算。

注 9: c1 先参加运算, 而后再加 1。

注 10: 等同于 `y=y+i*2;`

注 11: 等同于 `z=(x=x*((k=100)+1));` 但并不等同于 `z=(x=x*(k=100)+1);`

注 12: `-++j` 等同于 `-(++j)`。

注 13: `-j++` 等同于 `-(j++)`。

[输出结果]

```
int i=100, j, k;
```

```
char c1='E', c2;
```

```
double x=2.23, y=4.567, z;
```

```
-----
```

```
'A'+12=77
```

```
'a'-'A'=32
```

```
2*'A'+B/5=143
```

```
11/3=3
```

```
11%3=2
```

```
11/3.0=3.66667
```

```
After 'j=k--i+c1+x;' i=99 j=170 k=170
```

```
After 'c2=c1++;' c2=E c1=F
```

```
After 'y+=i*2;' y=202.567
```

```
After 'z=x*=(k=100)+1;' k=100 x=225.23 z=225.23
```

```
After 'j=5;' -++j=-6
```

```
After 'j=5;' -j++=-5
```

2.2 关系运算符和逻辑运算符功能的理解与使用

设有如下变量说明：

```
int i=32, j=1, k=3;
```

请写出下述 12 个表达式各自的结果值，并编出程序对结果进行验证。

```
2!=3;      (7>5)&&(5>2);    7>5&&5>2;    7>5>2;    !!!123;
!(true || false);    !(true && false) || (true || false);
false || (false && false) || true;    34-I || k;    j!=k && i!=k || j==0;
i && j && k;    !i || (j-k) && i && !(k-3 || i*k);
```

[分析提示]

- (1) 通过使用 `cout` 来输出各表达式的结果值。
- (2) 注意，关系运算和逻辑运算的结果都为非“0”即“1”的布尔值。

[解答程序]

```
#include <iostream.h>
void main(void) {
    cout<<"(2!=3) ==>"<<(2!=3)<<endl;        //必须加括号!
        //注意, cout<<"2!=3 ==>"<<2!=3<<endl; 会出现语法错误!
        //因为算符“ != ”(关系或称比较运算符)
        //比算符“ << ”(被重载了的移位运算符)的优先级低!
    cout<<"(7>5) && (5>2) ==>"<<((7>5) && (5>2))<<endl;
        //必须加括号。算符“ && ”(逻辑运算符)
        //比算符“ << ”(被重载了的移位运算符)的优先级低!
    cout<<"7>5 && 5>2 ==>"<<(7>5 && 5>2)<<endl;
        //比较运算符的优先级高于逻辑运算符“ && ”
    cout<<"(7>5>2) ==>"<<(7>5>2)<<endl;
        //等同于(7>5)>2, 其结果值等同于 1>2, 得结果值 0
    cout<<"!!!123 ==>"<<!!!123<<endl;
        //!!!123 等同于 !(!(123)), 结果为 0;
    cout<<"!(true||false) ==>"<<!(true||false)<<endl;
    cout<<"!(true && false)||true==>";
        cout<<"!(true && false)||true==>";
    cout<<"false||(false && false)||true ==>";
        cout<<"false||(false && false)||true==>";
    int i=32, j=1, k=3;
    cout<<"-----"<<endl;
    cout<<"int i=32, j=1, k=3;"<<endl;        //输出提示信息
```

```

cout<<"-----"<<endl;
cout<<"34-illk == "<<(34-illk)<<endl;           //非 0 数等同于逻辑值 true
cout<<"j!=k && i!=k || j==0 == "<<(j!=k && i!=k || j==0)<<endl;
cout<<"i && j && k == "<<(i && j && k)<<endl; //非 0 数等同于逻辑值 true
cout<<"!ll(j-k) && i && !(k-3lli*k) == ";
cout<<( !ll(j-k) && i && !(k-3lli*k) )<<endl;
}

```

[输出结果]

```

(2!=3) ==>1
(7>5) && (5>2) ==>1
7>5 && 5>2 ==>1
(7>5>2) ==>0
!!!123 ==>0
!(true||false) ==>1
!(true && false)||true ==>1
false||false && false||true ==>1
-----
int i=32, j=1, k=3;
-----
34-illk ==>1
j!=k && i!=k || j==0 ==>1
i && j && k ==>1
!ll(j-k) && i && !(k-3lli*k) ==>0

```

2.3 位运算符功能的理解与使用

读下述程序，写出执行结果，以加深对各种位运算符使用功能的理解。

```

#include <iostream.h>

void main()
{
    unsigned char d1=69, d2=37, d3;           //注 1
    cout<<"unsigned char d1=69, d2=37, d3;"<<endl;
    cout<<"d1="<<d1<<"          d2="<<d2<<endl;
    cout<<"int(d1)="<<int(d1)<<"   int(d2)="<<int(d2)<<endl;
    cout<<"sizeof(unsigned char)="<<sizeof(unsigned char)<<endl;
    cout<<"sizeof(d1)="<<sizeof(d1)<<endl;
    cout<<"-----"<<endl;
}

```

```

        d3=d1&d2;                                     //注 2
        cout<<"After 'd3=d1&d2;'  int(d3)="<<int(d3)<<endl;    //注 3
        d3=d1ld2;
        cout<<"After 'd3=d1ld2;'  int(d3)="<<int(d3)<<endl;
        d3=d1^d2;
        cout<<"After 'd3=d1^d2;'  int(d3)="<<int(d3)<<endl;
        d3=d1<<1;
        cout<<"After 'd3=d1<<1;'  int(d3)="<<int(d3)<<endl;
        d3=d1>>1;
        cout<<"After 'd3=d1>>1;'  int(d3)="<<int(d3)<<endl;
        d3=~d1;
        cout<<"After 'd3=~d1;'    int(d3)="<<int(d3)<<endl;
        cout<<"-----"<<endl;
        unsigned char x=3, y=1, z=32, k;
        cout<<"unsigned char x=3, y=1, z=32, k;"<<endl;        //注 4
        cout<<"-----"<<endl;
        cout<<"(x<<y)&z="<<((x<<y)&z)<<endl;
        cout<<"(z>>y)x="<<((z>>y)x)<<endl;
        cout<<"(x^y&~z)="<<((x^y&~z))<<endl;
        k=~(z>>y)x);
        cout<<"k=~(z>>y)x), int(k)="<<int(k)<<endl;
    }

```

[分析提示]

- (1) 本题对各种位运算符进行使用，并通过 `cout` 来输出运算结果。
- (2) 注意，位运算符对运算对象的每一位进行运算处理，其运算结果并非一个非“0”即“1”的布尔值。

[解答程序]

具体程序见题目处。

注 1：每一变量占一个字节，具有八个二进制位。

注 2：变量 `d1` 与 `d2` 按位做“与”运算(仅在一个字节范围内进行)。

注 3：按整数形式输出结果。

注 4：输出提示信息后，按当前变量的值进行表达式计算。

[输出结果]

```

unsigned char d1=69, d2=37, d3;
d1=E      d2=%
int(d1)=69  int(d2)=37

```

```

sizeof(unsigned char)=1
sizeof(d1)=1
-----
After 'd3=d1&d2;'  int(d3)=5
After 'd3=d1ld2;'  int(d3)=101
After 'd3=d1^d2;'  int(d3)=96
After 'd3=d1<<1;'  int(d3)=138
After 'd3=d1>>1;'  int(d3)=34
After 'd3=~d1;'    int(d3)=186
-----
unsigned char x=3, y=1, z=32, k;
-----
(x<<y)&z=0
(z>>y|x)=19
(x^y&~z)=2
k=~(z>>y|x), int(k)=236

```

2.4 表达式加括号使运算结果不变

假设变量具有如下初值：int i=1, j=2, k=3;

将下面的表达式加上括号，使得其计算顺序以及运算结果(包括整个表达式结果以及对各变量值产生的改变)都与原来相同。并编出程序，使用上述变量初值对加括号前后的各表达式的运算结果进行验证。

- (1) i=j=k++-2
- (2) i++>j--+k||4.3*i==j*2+k
- (3) i=j++, j=k++
- (4) i&j+k<2||i++==4
- (5) !i<0 && j>1 || j<0
- (6) !i&& j+1>k*k||i+j!=0

[分析提示]

(1) 本题测试对运算符优先级的理解与掌握情况。

(2) 表达式计算(处理)结束后，除产生一个表达式结果值以外，可能表达式中的某些相关变量的值也被改变了。例如，i=j=k++-2，它等同于：i=(j=((k++)-2))，计算处理后，整个表达式产生一个值，而其中的三个变量(i 与 j 及 k)的值也都进行了改变。要通过 cout 输出上述 4 个值。

[解答程序]

```

#include <iostream.h>
#include <conio.h>           //要使用“ getch( );” ——无回显地读入一个任意字符
                             //(输出多于一屏幕, 实现暂停功能, 任按一键继续)

void main(void)
{
    int i=1, j=2, k=3, n;
    cout<<"      int i=1, j=2, k=3, n;"<<endl;
    cout<<"----- (1) i=j=k++-2 -----"<<endl;
    //(1) i=j=k++-2    等同于: i=(j=((k++)-2))
    n=(i=j=k++-2);
    cout<<"After `n=(i=j=k++-2);"<<endl;
    cout<<"      i="<<i<<",   j="<<j<<",   k="<<k<<",   n="<<n<<endl;
    i=1, j=2, k=3;
    n=(i=(j=((k++)-2)));
    cout<<"After `n=(i=(j=((k++)-2)));"<<endl;
    cout<<"      i="<<i<<",   j="<<j<<",   k="<<k<<",   n="<<n<<endl;
    //(2) i++>j-->k||4.3*i==j*2+k
    //等同于: ( (i++) > (j-- + k) ) || ( (4.3*i) == (j*2+k) )
    cout<<"----- (2) i++>j-->k||4.3*i==j*2+k -----"<<endl;
    i=1, j=2, k=3;
    n=(i++>j-->k||4.3*i==j*2+k);
    cout<<"After `n=(i++>j-->k||4.3*i==j*2+k);"<<endl;
    cout<<"      i="<<i<<",   j="<<j<<",   k="<<k<<",   n="<<n<<endl;
    i=1, j=2, k=3;
    n=( ( (i++) > (j-- + k) ) || ( (4.3*i) == (j*2+k) ) );
    cout<<"After `n=( ( (i++) > (j-- + k) ) || ( (4.3*i) == (j*2+k) ) );"<<endl;
    cout<<"      i="<<i<<",   j="<<j<<",   k="<<k<<",   n="<<n<<endl;
    //(3) i=j++, j=k++    等同于: (i=(j++)), j=(k++)
    cout<<"----- (3) i=j++, j=k++ -----"<<endl;
    i=1, j=2, k=3;
    n=(i=j++, j=k++);
    cout<<"After `n=(i=j++, j=k++);"<<endl;
    cout<<"      i="<<i<<",   j="<<j<<",   k="<<k<<",   n="<<n<<endl;
    i=1, j=2, k=3;
    n=( (i=(j++)), j=(k++) );
    cout<<"After `n=( (i=(j++)), j=(k++) );"<<endl;
    cout<<"      i="<<i<<",   j="<<j<<",   k="<<k<<",   n="<<n<<endl;
    //(4) i&j+k<2||i++==4    等同于: (i && ((j+k)<2)) || ((i++)==4)

```

```

cout<<"----- (4) i&j+k<2||i++==4 -----"<<endl;
i=1, j=2, k=3;
n=(i&& j+k<2||i++==4);
cout<<"After `n=(i&& j+k<2||i++==4);" <<endl;
cout<<"      i="<<i<<" , j="<<j<<" , k="<<k<<" , n="<<n<<endl;
i=1, j=2, k=3;
n=( (i && ((j+k)<2)) || ((i++)==4) );
cout<<"After `n=( (i && ((j+k)<2)) || ((i++)==4) );" <<endl;
cout<<"      i="<<i<<" , j="<<j<<" , k="<<k<<" , n="<<n<<endl;
getch();          //等待输入一个任意字符(并不回显),
                  //而后继续执行后续程序部分(实现暂停功能)

//(5) !i<0 && j>1 || j<0  等同于: (((!i)<0) && (j>1)) || (j<0)
cout<<"----- (5) !i<0 && j>1 || j<0 -----"<<endl;
i=1, j=2, k=3;
n=!i<0 && j>1 || j<0;
cout<<"After `n=!i<0 && j>1 || j<0);" <<endl;
cout<<"      i="<<i<<" , j="<<j<<" , k="<<k<<" , n="<<n<<endl;
i=1, j=2, k=3;
n=(((!i)<0) && (j>1)) || (j<0) );
cout<<"After `n=(((!i)<0) && (j>1)) || (j<0) );" <<endl;
cout<<"      i="<<i<<" , j="<<j<<" , k="<<k<<" , n="<<n<<endl;
//(6) !i&& j+1>k*k||i+j!=0  等同于: ((!i)&&((j+1)>(k*k)))||((i+j)!=0)
cout<<"----- (6) !i&& j+1>k*k||i+j!=0 -----"<<endl;
i=1, j=2, k=3;
n=!i&& j+1>k*k||i+j!=0;
cout<<"After `n=!i&& j+1>k*k||i+j!=0);" <<endl;
cout<<"      i="<<i<<" , j="<<j<<" , k="<<k<<" , n="<<n<<endl;
i=1, j=2, k=3;
n=(((!i)&&((j+1)>(k*k)))||((i+j)!=0) );
cout<<"After `n=(((!i)&&((j+1)>(k*k)))||((i+j)!=0) );" <<endl;
cout<<"      i="<<i<<" , j="<<j<<" , k="<<k<<" , n="<<n<<endl;
cout<<"-----"<<endl;
}

```

[输出结果]

```

int i=1, j=2, k=3, n;
----- (1) i=j=k++-2 -----
After `n=(i=j=k++-2);'
i=1, j=1, k=4, n=1

```



```
cout<<RED<<' '<<YELLOW<<' '<<WHITE<<endl;           //注 3
a=d;                                                       //注 4
c2=c;                                                      //注 5
b=myColor(BLACK+1);                                       //注 6
cout<<"c1="<<c1<<"  c2="<<c2<<endl;                     //注 7
cout<<"a="<<a<<"  b="<<b<<"  c="<<c<<"  d="<<d<<endl;   //注 8
}
```

[分析提示]

通过 `enum` 来说明自定义的枚举类型，本题定义 `myColor` 为枚举类型，并对其枚举常量值进行了使用。

[解答程序]

具体程序见题目处。

注 1：定义了枚举类型名 `myColor`，同时又说明了两个枚举变量 `c1` 与 `c2`，且给变量 `c1` 赋了初值。

注 2：说明了四个 `myColor` 类型的变量 `a`、`b`、`c`、`d`，且给变量 `c` 与 `d` 赋了初值。

注 3：`RED` 值为 1，`YELLOW` 被赋成了 5，其后的 `WHITE` 将代表 $5+2=7$ 。

注 4：同类型变量间的赋值，使 `a` 值为 `BLUE` 即 0。注意，直接使用“`a=0;`”将产生类型不符的语法错误。当然，若通过强制类型转换，使用“`a=myColor(0);`”也是可以的。

注 5：同类型变量间的赋值，使 `c2` 值为 `BLACK` 即 6。

注 6：`BLACK+1` 值为 7，而后经过强制类型转换后赋值给 `b`。

注 7：`c1` 为 `RED=1`，`c2` 为 `BLACK` 值为 6。

注 8：输出结果“`a=0 b=7 c=6 d=0`”。

[输出结果]

```
1 5 7
c1=1 c2=6
a=0 b=7 c=6 d=0
```

2.6 求两个时间(时刻)的时间差

编程序，接收用户从键盘输入的一天内的两个时间(时— 0~23、分— 0~59、秒—— 0~59)，求出这两个时刻的时间差(按秒计算)，并将结果显示在屏幕上(用到求绝对值的标准库函数 `abs` 时，要在程序开头处使用“`#include <math.h>`”)。

[分析提示]

- (1) 通过 cin 输入两个时、分、秒数据。
- (2) 计算出每一时、分、秒距离 0 时 0 分 0 秒的总秒数，而后求两个总秒数差的绝对值。

[解答程序]

```
#include <iostream.h>
#include <math.h>
void main(void)
{
    int h1,m1,s1, h2,m2,s2;    //h1、m1、s1 表示时刻 1 的时、分、秒,
                                //h2、m2、s2 表示时刻 2 的时、分、秒
    long sum1, sum2, dif;      //sum1 中存放时刻 1 距 0 时 0 分 0 秒的总秒数
                                //sum2 中存放时刻 2 距 0 时 0 分 0 秒的总秒数
                                //dif 为时刻 1 与时刻 2 的秒数差(sum2-sum1 后取绝对值)
    cout<<"Input time1(hour1[0~23] minute1[0~59] second1[0~59]):";
    cin>>h1>>m1>>s1;        //输入时刻 1 的时、分、秒
    cout<<"Input time2(hour2[0~23] minute2[0~59] second2[0~59]):";
    cin>>h2>>m2>>s2;        //输入时刻 2 的时、分、秒
    sum1=h1*3600+m1*60+s1;    //sum1 为时刻 1 距 0 时 0 分 0 秒的总秒数
    sum2=h2*3600+m2*60+s2;    //sum2 为时刻 2 距 0 时 0 分 0 秒的总秒数
    dif=abs(sum2-sum1);      //两个时刻的时间差(按秒计算)
    cout<<"dif="<<dif<<" seconds"<<endl;    //输出结果
}
```

[输出结果]

```
Input time1(hour1[0~23] minute1[0~59] second1[0~59]):8 15 22
Input time2(hour2[0~23] minute2[0~59] second2[0~59]):13 0 59
dif=17137 seconds
```

又一次执行:

```
Input time1(hour1[0~23] minute1[0~59] second1[0~59]):0 0 0
Input time2(hour2[0~23] minute2[0~59] second2[0~59]):23 59 59
dif=86399 seconds
```

再一次执行:

```
Input time1(hour1[0~23] minute1[0~59] second1[0~59]):13 0 1
Input time2(hour2[0~23] minute2[0~59] second2[0~59]):12 59 59
dif=2 seconds
```

2.7 计算 x 的平方、平方根及倒数等

编程序，从键盘输入 `float` 型数据 x ，而后计算 x 的平方、 x 的平方根、 x 的倒数以及 $2x-1$ 的值，并将结果显示在屏幕上(注：用到求平方根的标准库函数 `sqrt` 时，要在程序开头处使用“`#include <math.h>`”)。

[分析提示]

(1) 通过 `cin` 输入 `float` 型变量 x 的数据。

(2) 在包含了“`math.h`”头文件的情况下，可以直接调用标准库函数 `sqrt` 来求出平方根。

[解答程序]

```
#include <iostream.h>
#include <math.h>
void main(void)
{
    float x;                //说明 x 为 float 型(浮点型)变量
    cout<<"Input a real number:"; //屏幕提示进行输入
    cin>>x;                 //从键盘输入一个实数赋值给 x 变量
    cout<<"x*x="<<x*x<<endl; //在屏幕上显示出 x 的平方值
    cout<<"sqrt(x)="<<sqrt(x)<<endl; //计算并显示出 x 的平方根之值
    cout<<"1/x="<<1/x<<endl; //显示出 x 的倒数值
    cout<<"2*x-1="<<2*x-1<<endl; //显示出 2*x-1 之值
}
```

[输出结果]

```
Input a real number:123.456
x*x=15241.4
sqrt(x)=11.1111
1/x=0.00810005
2*x-1=245.912
```

2.8 一元二次方程求根

输入三个实数 a 、 b 、 c (满足 $b^2-4ac>0$)，求出方程 $a*x^2+b*x+c=0$ 的两个实根，并显示在屏幕上。

[分析提示]

通过 cin 输入方程系数 a、b、c 的值，而后根据求根公式计算，并输出它的两个根。

[解答程序]

```
#include <iostream.h>
#include <math.h>
void main(void)
{
    double a,b,c,x1,x2;           //说明 a、b、c、x1、x2 为 double 型变量
    cout<<"Input a,b,c ( b*b-4*a*c>0 ):"; //屏幕提示进行输入
    cin>>a>>b>>c;                //从键盘输入 a、b、c 的值(方程的三个系数)
    double sqrtVal=sqrt(b*b-4*a*c); //先求出判别式的值
    x1=(-b+sqrtVal)/(2*a);          //第一个实根 x1
    x2=(-b-sqrtVal)/(2*a);          //第二个实根 x2
    cout<<"x1="<<x1<<endl;        //显示出 x1 的值
    cout<<"x2="<<x2<<endl;        //显示 x2 的值
}
```

[输出结果]

```
Input a,b,c ( b*b-4*a*c>0 ):2.2  24.5  -8.15
x1=0.323269
x2=-11.4596
```

第三章 分支与循环处理

[语法要点]

详细内容可参阅教材第四章

(1) 控制语句是用来控制程序中各语句执行的次序，更确切地说，控制语句是用来改变或打破程序中语句按书写的先后顺序执行的规律。

分支语句：if 语句，switch 语句。

循环语句：for 语句，while 语句，do_while 语句。

无条件控制语句：break 语句，continue 语句，return 语句，goto 语句。

① if 语句：亦称条件语句，如果语句，它有两种形式：

if(<表达式 E>)<语句 S>

if(<表达式 E>)<语句 S₁> else <语句 S₂>

表达式 E 应为 int 型或 char 型。其值等于 0 为假，非 0 为真。

在 if 语句嵌套时，每个 else 只与它前面的最近的那个未配对的 if 配对。

② switch 语句 又称开关语句或分情况语句，它是一种多分支语句。其基本格式为：

switch(<表达式 E>)

{

case <情况常量 C₁>: <语句序列 S₁>

case <情况常量 C₂>: <语句序列 S₂>

...

case <情况常量 C_n>: <语句序列 S_n>

default: < 语句序列 S_{n+1}>

}

③ for 语句 是 C++ 程序中功能最强的循环语句，其格式为

for(<表达式 E₁>; <表达式 E₂>; <表达式 E₃>)<语句 S>

④ while 语句 实际上是 for 语句在表达式 E₁ 和 E₃ 为空的特殊情形，其格式为

while(< 表达式 >)<语句 S>

⑤ do-while 语句 类似于 while 语句，只是把循环的判定移至循环体之后，其格式为：

do <语句 S> while(<表达式 E>);

⑥ break 语句 又称跳出语句。其功能是把程序的运行流程跳转到它所在的循环语句或 switch 语句的出口。其格式为：

break;

⑦ continue 语句 又称继续语句，其功能是把程序运行流程跳转到该循环体(不是该循环语句)的出口点，结束循环体的一次重复，其格式为

continue;

⑧ return 语句 又称返回语句。只用于函数定义，其功能为

- i 把程序运行的流程跳转到该函数的调用点。
- ii 计算返回表达式 E，并把其值作为该函数的返回值。其格式为
return; 或 return< 表达式 E>;

⑨ goto 语句 又称转向语句，其功能是令程序跳转到程序指定的某标号语句处。

由于 goto 语句的跳转控制有较大的随意性，建议不用或少用 goto 语句。其格式为

goto< 标号>;

(2) 复合语句：又称为块语句(block)或程序块，其格式为：

{ <语句 S₁> <语句 S₂> ... <语句 S_n> }

(3) 空语句，就是什么都不做。其格式为：

;

3.1 求三个实数中的最大数与最小数

输入任意三个实数，求出其中的最大者与最小者，并在屏幕上显示出来。

[分析提示]

- (1) 说明 float 型变量 a、b、c，并通过 cin 为它们输入任意三个实数。
- (2) 本题通过三目运算符“ ? 、 # ”以及 if 语句来进行比较求出其中的大数和小数。

[解答程序]

```
#include <iostream.h>
void main()
{
    float a,b,c,max,min;
    cout<<"Please input 3 floating numbers:";
    cin>>a>>b>>c;           //从键盘输入三个实数放入 a、b、c
    max=(b>a)?b:a;             //a、b 中的大者放 max
    min=(b<a)?b:a;             //a、b 中的小者放 min
    if(c>max) max=c;           //若 c>max，则将 max 改为 c
    if(c<min) min=c;           //若 c<min，则将 min 改为 c
    cout<<"max="<<max<<endl;   //显示结果
    cout<<"min="<<min<<endl;
}
```

[输出结果]

Please input 3 floating numbers:2.3 56.3 -6.7

```
max=56.3  
min=-6.7
```

3.2 设计一个计算商品价格的程序

某商店出售四种商品：A 商品每公斤 3.25 元；B 商品每个 16.5 元；C 商品每米 22.5 元；D 商品每台 368 元，超过 3 台优惠 10%，超过 8 台优惠 15%。为其设计一个计算价格的程序(输入购买四种商品的数量，计算出应付款金额，并显示付款金额明细单)。

[分析提示]

(1) 通过 cin 依次输入购买四种商品的数量，而后按商品价格以及优惠条件进行计算，并显示所需的结果数据。

(2) 从输入到输出的数据以及信息格式也需要精心设计。例如，本题可通过“先提示后输入”的方式依次输入四种商品数量，而后由程序负责计算，并按如下格式输出结果信息：

```
input A-Num:12  
input B-Num:5  
input C-Num:8  
input D-Num:10  
  
----- The price sheet -----  
A: 3.25(yuan)*12(Kilo)=39(yuan)  
B: 16.5(yuan)*5(Ge)=82.5(yuan)  
C: 22.5(yuan)*8(Mi)=180(yuan)  
D: 368(yuan)*10(Tai)*0.85=3128(yuan)  
  
You should pay => : 3429.5(yuan)
```

[解答程序]

```
#include <iostream.h>  
#include <math.h>  
void main()  
{  
    int ANum,BNum,CNum,DNum;  
        //四个变量代表购买 A、B、C、D 四种商品的数量  
    float APrice=3.25, BPrice=16.5, CPrice=22.5, DPrice=368;  
        //A、B、C、D 四种商品的单价  
    float ASum, BSum, CSum, DSum, TotalSum;
```

```

//ASum 为购买 A 商品所需的付款金额(元数),
//BSum、CSum、DSum 为购买 B、C、D 商品所需的付款金额
//TotalSum 为购买 A、B、C、D 四种商品所需的总付款金额
cout<<"input A-Num:";      //提示输入购买 A 商品的数量
cin>>ANum;                 //键盘输入 ANum
cout<<"input B-Num:";      //提示输入购买 B 商品的数量
cin>>BNum;                 //键盘输入 BNum
cout<<"input C-Num:";      //提示输入购买 C 商品的数量
cin>>CNum;                 //键盘输入 CNum
cout<<"input D-Num:";      //提示输入购买 D 商品的数量
cin>>DNum;                 //键盘输入 DNum
ASum=APrice*ANum;          //算出购买 A 商品所需的付款金额(元数)放 ASum
BSum=BPrice*BNum;          //购买 B 商品所需的付款金额(元数)
CSum=CPrice*CNum;          //C 商品所需的付款金额(元数)
DSum=DPrice*DNum;          //D 商品的付款金额(元数)
if(DNum>8)
    DSum*=0.85;             //购买 D 商品超过 8 台优惠 15%
if(DNum>3 && DNum<=8)
    DSum*=0.9;              //购买 D 商品超过 3 台优惠 10%
TotalSum=ASum+BSum+CSum+DSum;
//TotalSum 为购买 A、B、C、D 四种商品所需的总付款金额
cout<<endl;
cout<<"----- The price sheet -----"<<endl; //显示出付款金额明细单
cout<<"A: "<<APrice<<"(yuan)*"<<ANum<<"(Kilo)="<<ASum<<"(yuan)"<<endl;
//A 商品明细
cout<<"B: "<<BPrice<<"(yuan)*"<<BNum<<"(Ge)="<<BSum<<"(yuan)"<<endl;
//B 商品明细
cout<<"C: "<<CPrice<<"(yuan)*"<<CNum<<"(Mi)="<<CSum<<"(yuan)"<<endl;
//C 商品明细
if(DNum<=3)                 //D 商品明细
    cout<<"D: "<<DPrice<<"(yuan)*"<<DNum<<"(Tai)="<<DSum<<"(yuan)"<<endl;
else
    if(DNum<=8)
        cout<<"D: "<<DPrice<<"(yuan)*"<<DNum<<"(Tai)*0.9="<<DSum<<"(yuan)"<<endl;
    else
        cout<<"D: "<<DPrice<<"(yuan)*"<<DNum<<"(Tai)*0.85="<<DSum<<"(yuan)"<<endl;
cout<<endl;
cout<<"You should pay => : "<<TotalSum<<"(yuan)"<<endl;
//所需的总付款金额

```

```
}
```

[输出结果]

见分析提示处，略。

3.3 switch 语句的执行效果

如下程序执行时，当为变量 *i* 输入 1 (或 2 或 3 或 4) 后，会在屏幕上输出什么结果呢？

```
#include <iostream.h>

void main( )
{
    int i;
    cout<<"i=?";
    cin>>i;
    switch (i)                //注 1
    {
        case 1:                //注 2
            cout<<"i="<<i<<endl;    //注 3
        case 2:                //注 4
            cout<<"i="<<i<<endl;    //注 5
        case 3:                //注 6
            cout<<"i="<<i<<endl;    //注 7
    }
}
```

[分析提示]

注意，switch 语句中的“ case <常量>:”只起一个语句标号的作用，它是一个分支入口点，系统从某个匹配上的入口点开始执行，直到遇到了 break 语句或者遇到了 switch 语句体的右花括号时，方才结束 switch 语句。

[解答程序]

具体程序见题目处。

注 1：根据变量 *i* 的值来进行多分支处理。

注 2：当 *i* 值为 1 时的处理分支入口点。

注 3：输出 "i=1"; 由于没遇到 break 也没遇到 switch 语句体的右花括号，接着往下执行。

注 4：当 *i* 值为 2 时的处理分支入口点。

注 5：输出变量 *i* 的当前值(当从“ case 2:”入口”时，*i* 值为 2，当从“ case 1:”入

口”接下来时, i 值为 1); 由于没遇到 `break` 也没遇到 `switch` 语句体的右花括号, 接着往下执行。

注 6: 当 i 值为 3 时的处理分支入口点。

注 7: 输出变量 i 的当前值; 下面将遇到 `switch` 语句体的右花括号而结束本 `switch` 句。

[输出结果]

i=?1

i=1

i=1

i=1

又一次执行:

i=?3

i=3

再一次执行:

i=?4

3.4 将考试成绩等级字母变换为相应的百分制分数段

编程序, 从键盘输入一个表示考试成绩等级的字母, 而后根据该字母值显示出相应的百分制分数段。对应关系为: “A” 或 “a” 对应于 90—100; “B” 或 “b” 对应于 80—89; “C” 或 “c” 对应于 70—79; “D” 或 “d” 对应于 60—69; “E” 或 “e” 对应于 0—59; 输入其他字母时, 提示输入错误信息。

[分析提示]

(1) 说明 `char` 型的变量 `grade`, 并通过 `cin` 为其输入一个等级字母。而后通过使用 “`switch (grade)`” 来进行多分支处理, 且该 `switch` 语句中设立 “`default:`” 分支, 用于处理输入了其他字母时的情况。

(2) 每一分支处理结束后, 均通过一个 `break` 语句结束本分支并结束 `switch` 语句。

[解答程序]

```
#include <iostream.h>

void main( )
{
    char grade;
    cout<<"Input a grade('A'— 'E'):"<<endl;
```

```
cin>>grade;           //从键盘输入一个表示考试成绩等级的字母
switch (grade)         //根据变量 grade 的值来进行多分支处理
{
case 'A':              //当 grade 为大、小写的 A 字母时的处理分支
case 'a':
    cout<<"90—100"<<endl; //具体处理部分, 显示出相应的百分制分数段
    break;               //结束本分支, 进而结束本 switch 语句
case 'B':              //当 grade 为大、小写的 B 字母时的处理分支
case 'b':
    cout<<"80—89"<<endl;  //具体处理部分, 显示出相应的百分制分数段
    break;               //结束本 switch 句
case 'C':              //当 grade 为大、小写的 C 字母时的处理分支
case 'c':
    cout<<"70 —79"<<endl; //显示出相应的百分制分数段
    break;               //结束本 switch 句
case 'D':              //当 grade 为大、小写的 D 字母时的处理分支
case 'd':
    cout<<"60—69"<<endl;  //显示相应的百分制分数段
    break;               //结束本 switch 句
case 'E':              //当 grade 为大、小写的 E 字母时的处理分支
case 'e':
    cout<<"0—59"<<endl;   //显示相应的百分制分数段
    break;               //结束本 switch 句
default:                //当 grade 不为上述任一字母时的处理
    cout<<"Input ERROR! ('A'—'E' or 'a'—'e?')"<<endl;
                        //提示输入了错误信息
}
}
```

[输出结果]

Input a grade('A'—'E'):

A

90—100

再一次执行:

Input a grade('A'—'E'):

f

Input ERROR! ('A'—'E' or 'a'—'e?')

3.5 求实数 a 的 n 次方

求实数 a 的 n 次方(double a ; int n ; 且 $n > 0$, a 与 n 的值通过键盘输入)。

[分析提示]

(1) 这是一个求“累乘积”的问题。可设一个 double 型变量 res , 初值赋为 1, 而后执行 n 次“ $res *= a$;”, 从而使 res 的结果变为 a 的 n 次方。

(2) 执行 n 次“ $res *= a$;”的过程可通过循环来实现, 如本题使用了 while 语句(当然也可通过使用 for 语句或 do-while 语句)。

[解答程序]

```
#include <iostream.h>
void main(void)
{
    int n;
    double a, res;
    cout<<"Input a double A, and a positive integer N:";
                                                    //提示输入实数 a 及正整数 n
    cin>>a>>n;
                                                    //输入两个数据
    res=1;
    while (n-->0)
                                                    //在 res 上(其初值为 1)累乘 n 个 a
        res*=a;
    cout<<"res="<<res<<endl;
                                                    //输出累乘结果 res
}
```

[输出结果]

```
Input a double A, and a positive integer N:5.5 3
res=166.375
```

3.6 求出正整数 n 的所有因子

求出正整数 n 的所有因子(包括 1 与 n 本身), 其中的 n 值通过键盘输入。

[分析提示]

(1) 条件“ $n \% k == 0$ ”意味着 k 是 n 的因子。

(2) 如下的 for 语句可找 n 的所有因子 k (包括 1 与 n 本身):

```
for ( k=1; k<=n; k++ )
```

```
if ( n % k == 0 )
    <k 为 n 的因子>;
```

[解答程序]

```
#include <iostream.h>
void main(void)
{
    int n;
    cout<<"Enter a positive integer: ";           //提示输入正整数
    cin>>n;                                         //输入数据放 n 中
    cout <<"Number "<<n<<" Factors: ";
    for (int k=1; k<=n; k++)
        //看 1 到 n 中的哪些数可以整除 n(可整除者，都为 n 的因子)
        if (n % k == 0)
            cout << k << " ";                   //输出各因子
    cout<<endl;
}
```

[输出结果]

```
Enter a positive integer: 100
Number 100 Factors: 1  2  4  5  10  20  25  50  100
```

3.7 求正整数 n 的所有因子的平方根之和

求正整数 n 的所有因子(包括 1 与 n 本身)的平方根之和，其中的 n 值通过键盘输入。

[分析提示]

在上一题的基础上，每当找到 n 的一个因子 k 后，就立即将 k 的平方根累加到一个 `double` 型变量 `sum` 上，而 `sum` 的初值在说明时就置为 0 (“滚雪球”式地要从 0 开始累加起)。

[解答程序]

```
#include <iostream.h>
#include <math.h>
void main(void)
{
    int n;
    double sum=0;                                //sum 中放累加和
```



```

cout<<"Enter a positive integer: ";           //提示输入正整数
cin>>n;                                       //输入数据放 n 中
cout <<"Number "<<n<<" Factors: ";
for (int k=1; k<=n; k++)
{
    //看 1 到 n 中的哪些数可以整除 n(可整除者, 都为 n 的因子)
    if (n % k == 0)                           //k 为 n 的因子时
    {
        cout << k << " ";                   //输出 k
        sum+=sqrt(k);                        //将 k 的平方根累加到 sum 中
    }
}
cout<<"\nsum="<<sum<<endl;                 //输出累加和
}

```

[输出结果]

```

Enter a positive integer: 100
Number 100 Factors: 1  2  4  5  10  20  25  50  100
sum=36.3558

```

3.8 n 个实数的整数部分和以及小数部分和

任意输入 n 个实数(其中的 n 以及 n 个实数都通过键盘输入), 求出它们的和 sum、它们的整数部分的和 sum1 以及它们的小数部分的和 sum2。

[分析提示]

- (1) 说明变量 sum、sum1 以及 sum2, 并将它们的初值置为 0(用来存放三个累加和)。
- (2) 编制一个能够执行 n 次的循环, 每次输入并处理一个实数(按要求进行累加)。
- (3) 通过“int(x)”可以得到 x 的整数部分, 通过“x-int(x)”可得到 x 的小数部分。

[解答程序]

```

#include <iostream.h>
void main( )
{
    int n, sum1=0;                             //n 以及整数部分的和 sum1 为 int 型数据
    double sum2=0, sum=0, x;
    //n 个实数的和 sum 以及它们的小数部分的和 sum2 为 double 型数据
    cout<<"n=? ";                             //提示输入 n
}

```

```
cin>>n;                //输入 n
cout<<"Input n="<<n<<" float point numbers:"<<endl; //提示输入 n 个实数
for(int i=0; i<n; i++) //共处理 n 个数
{
    cin>>x;            //输入一个实数 x
    sum+=x;            //将 x 累加到 sum 上
    sum1+=int(x);      //将 x 的整数部分累加到 sum1 上
    sum2+=x-int(x);    //将 x 的小数部分累加到 sum2 上
}
cout<<"sum="<<sum<<endl; //输出三个累加和
cout<<"sum1="<<sum1<<endl;
cout<<"sum2="<<sum2<<endl;
}
```

[输出结果]

```
n=? 5
Input n=5 float point numbers:
2.2 3.3 4.45 6.89 0.1257
sum=16.9657
sum1=15
sum2=1.9657
```

3.9 求最小公倍数

输入两个正整数 a、b, 求出它们的最小公倍数 c。

[分析提示]

求解方法之一为先求出 a、b 中的大者 max 及小者 min, 而后找出 max 的 1 倍、2 倍、...、n 倍中的满足下述条件的最小者 c: 该 c 恰好可以被 min 除尽。本题采用此种方法。

[解答程序]

```
#include <iostream.h>
void main(void)
{
    int a,b,c,max,min; //说明五个 int 型变量
    cout<<"Input 2 positive integers:"; //屏幕提示输入两个正整数
    cin>>a>>b; //从键盘输入两个正整数, 给变量 a 与 b 赋值 (两数据以空格分割)
```

```

if(a>b)           //先求出 a、b 中的大者 max 及小者 min
{
    max=a; min=b;
}
else
{
    max=b; min=a;
}

//而后找出 max 的 1 倍、2 倍、...、n 倍中的满足下述条件的最小者 c:
//该 c 恰好可以被 min 除尽

c=max;           //max 的 1 倍赋给 c
while (c%min)    //当前的 c 不可被 min 除尽时继续循环
    c+=max;       //将 c“长”为 max 的 2 倍、3 倍、...、n 倍
cout<<"c="<<c<<endl; //输出恰好可以被 min 除尽的 c
}

```

[输出结果]

Input 2 integers:3 5
c=15

再一次执行:

Input 2 integers:26 16
c=208

3.10 求最大公约数

输入两个正整数 a、b, 求出它们的最大公约数 c。

[分析提示]

(1) 可按如下的方法来求解:

gcd(m,n)=n;	若 m%n 等于 0;
gcd(m,n)=gcd(n,m%n);	若 m%n 不等于 0。

(2) 当 m%n 不等于 0 时, 由于 $\text{gcd}(m,n) = \text{gcd}(n, m\%n)$, 即是说, 将 n 当做新 m, 将 m%n 当做新的 n 后, 接着去判断 m%n 是否等于 0, 从而形成一个 while (m%n!=0) ...” 的循环。

[解答程序]

```
#include <iostream.h>
```

```
void main(void)
{
    int a,b,c,m,n;           //说明五个 int 型变量
    cout<<"Input 2 positive integers:";    //屏幕提示输入两个正整数
    cin>>a>>b;              //从键盘输入两个数赋值给 a 与 b 变量(两数据以空格分割)
    m=a; n=b;                //m 与 n 的初值为 a 和 b
    while (m%n)               //当 m%n 不等于 0 时继续循环
    {
        int tmp=m; m=n;
        n = tmp % n;         //注意, 若 m%n 不等于 0 时, 将 n 当做新 m,
                               //将 m%n 当做新的 n 后, 接着去判断 m%n 是否等于 0
    }
    c=n;                      //此时 m%n 等于 0, 所求结果 c 等于 n
    cout<<"gcd("<<a<<","<<b<<")="<<c<<endl;
                               //在屏幕上显示出 a 与 b 的最大公约数 c
}
```

[输出结果]

```
Input 2 positive integers:20 8
gcd (20,8) = 4
```

再一次执行:

```
Input 2 positive integers:17 5
gcd (17,5) = 1
```

3.11 改写程序使其中不出现 switch 与 for 语句

叙述程序功能(执行后可以做什么事情); 并改写该程序, 使其中不出现 switch 与 for 语句, 但要使程序的执行结果与改写前完全相同。

```
#include <iostream.h>
void main( )
{
    int i, n, sum=0, sum6=0, sum9=0, s9=0;
    for(i=1; i<=5; i++)
    {
        cin>>n;
        switch(n/10)
        {
```

```

        case 9:
            { s9+=n; sum9++; break; }
        case 6:
            { sum6++; break; }
        default:
            sum++;
        }
    }
    cout<<"sum6="<<sum6<<endl;
    cout<<"sum9="<<sum9<<endl;
    cout<<"sum="<<sum<<endl;
    cout<<"ave9="<<float(s9)/sum9<<endl;
}

```

[分析提示]

(1) for 循环执行五次，可用 while 来替代 for。

(2) switch 语句根据条件“一分为三”，可用“if (...) s1; else if (...) s2; else s3;”形式的语句来取代。

[解答程序]

(1) 程序功能(执行后可以做什么事情):

从键盘任意输入五个整数，统计出此五个数中处于 60~69 范围的数的个数 sum6、处于 90~99 范围的数的个数 sum9、不处于上述两范围的数的个数 sum 以及处于 90~99 范围的数的平均值 ave9，并将这些统计结果输出到屏幕上。

(2) 改写后的程序:

```

#include <iostream.h>

void main( )
{
    int i=1, n, sum=0, sum6=0, sum9=0, s9=0;
    while(i<=5)                //共输入五个整数并处理
    {
        cin>>n;
        if (n/10==9)           //n 处于 90~99 范围内
        {
            s9+=n;              //处于 90~99 范围内的数的累加和
            sum9++;              //处于 90~99 范围内的数的个数 sum9
        }
        else
            if (n/10==6)         //n 处于 60~69 范围内

```

```
        sum6++;           //处于 60~69 范围内的数的个数 sum6
    else
        sum++;           //不处于上述二范围内的数的个数 sum
    i++;
}
cout<<"sum6="<<sum6<<endl;
cout<<"sum9="<<sum9<<endl;
cout<<"sum="<<sum<<endl;
cout<<"ave9="<<float(s9)/sum9<<endl;    //处于 90~99 范围内的数的平均值 ave9
}
```

[输出结果]

```
123 67 90 60 95
sum6=2
sum9=2
sum=1
ave9=92.5
```

3.12 反序输出某正整数的各数位值

反序输出某正整数的各数位值，如输入 123，要输出 321。

[分析提示]

(1) 通过使用“ $n\%10$ ”可获得当前正整数 n 的个位数字，它正是要反序输出的首位数字；而后通过“ $n=n/10$ ；”可将当前 n 除以 10 (取整)，也即甩掉当前的个位而形成一个新数仍放 n 中；接着再通过使用“ $n\%10$ ”获得当前 n 的个位数字，它正是初始 n 的倒数第二位即十位的那一数字，如此等等，直到所谓的当前 n 变成了 0。

(2) 将上述的处理思想转换为 C++ 的循环，本程序使用了 do-while 语句。

[解答程序]

```
#include <iostream.h>
#include <process.h>
void main(void)
{
    int n;
    cout<<"Input a positive number: ";    //提示输入正整数
    cin>>n;                                //输入的正整数放 n 中
    if (n<0)                               //若 n 为负数，提示信息后退出
```

```
{
    cout<<"Please input positive number!"<<endl;
    exit(0);
}
cout<<"The number in reverse order is: ";
do
{
    cout<<n%10;                //截下并输出当前 n 的个位
    n/=10;
    //将当前 n 除以 10(取整), 也即甩掉当前的个位而形成一个新数仍放 n 中
} while (n!=0);
cout<<endl;
}
```

[输出结果]

Enter the number: 1582

The number in reverse order is: 2851

再一次执行:

Input a positive number: -5316

Please input positive number!

3.13 求正整数 n 之反序数的两倍值

键盘输入正整数 n, 求 n 之反序数的 2 倍值, 如: 输入 158, 输出应为 $res=851*2=1702$ 。

[分析提示]

在上一题的基础上, 设 res 变量用于累加出 n 的“反序数”。res 初值置为 0, 每当截下当前 n 的个位数字 digit 时, 通过“ $res=res*10+digit$ ”的执行(使原 res 各位均左移 1 位, 留出个位累加上 digit), 可在 res 中逐步累加出 n 的“反序数”。

[解答程序]

```
#include <iostream.h>
void main(void)
{
    int n, digit, res=0;                //在 res 中累加出 n 的“反序数”
    cout<<"Input a integer:";          //提示输入数据
    cin>>n;                             //输入的数据放 n 中
```

```
do
{
    digit=n%10;           //截下当前 n 的个位放 digit
    res=res*10+digit;     //在 res 中累加出 n 的“反序数”
                           //(原 res 各位均左移 1 位，留出个位累加上 digit)
    n/=10;               //将当前 n 除以 10(取整)，也即甩掉当前的个位
                           //而形成一个新数仍放 n 中
} while (n!=0);
cout<<"res="<<res<<"*2="<<2*res<<endl;    //输出结果
}
```

[输出结果]

```
Input a integer:158
res=851*2=1702
```

3.14 验证奇数的指定特性

编程序，验证 100 以内的奇数，它们的平方被 8 除余数均为 1。

[分析提示]

- (1) 使用 while 循环依次对奇数 i 进行验证(i 初值为 1，每循环一次增长 2，在“ while (i<100)”范围内进行)。
- (2) 通过条件“ i*i%8==1”来判断奇数 i 的平方除以 8 的余数恰为 1。

[解答程序]

```
#include <iostream.h>
void main()
{
    int i=1, OkNum=0, NotOkNum=0, mod;    //i 为第一个奇数
    //OkNum 中累加满足条件的数据个数, NotOkNum 中累加不满足条件的数据个数
    while (i<100)
    {
        mod=i*i%8;           //奇数 i 的平方除以 8 的余数放入 mod
        if(mod==1) OkNum++;   //余数为 1 时，OkNum 加 1
        else NotOkNum++;     //余数非 1 时，NotOkNum 加 1
        i+=2;                //使 i 成为下一个奇数
    }
    cout<<"Total OK number="<<OkNum<<endl;
```



```

        cout<<"Total NOT_OK number="<<NotOkNum<<endl;
    }

```

[输出结果]

```

Total OK number=50
Total NOT_OK number=0

```

3.15 按近似公式求近似圆周率

按照如下的与计算圆周率 PAI 有关的公式，求出累加 500 项以及累加 3 000 项的两个近似圆周率 PAI 值。

$$\text{PAI}/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + \dots$$

[分析提示]

(1) 使用“ while (i<3000)”形式的循环依次对公式中的各累加项 item 进行累加。当循环累加到 500 次的时候，输出与当时前 500 项累加和相关的一个值。

(2) 循环中的累加项 item 通过下式来形成：“ item=1.0/(2*i+1)*sign;”，而 sign 为当前累加项的符号，初值为 1，每循环一次“ sign=-sign;”，使之变为相反的符号。

[解答程序]

```

#include <iostream.h>
#include <math.h>
void main( )
{
    int i=0, sign=1;                //i 记录已累加的项数, sign 为当前累加项的符号
    double pai4=0, item;            //pai4 放 PAI/4 的累加值, item 为当前累加项
    while (i<3000)
    {
        item=1.0/(2*i+1)*sign;      //形成当前累加项 item
        pai4+=item;                 //往 pai4 上累加 item
        i++;                        //记录累加项数的 i 加 1
        if(i==500)                  //累加至 500 项时要输出一个近似值
            cout<<"PAI500="<<pai4*4<<endl;
        sign=-sign;                 //下一累加项的符号与本项相反(正负相间)
    }
    cout<<"PAI3000="<<pai4*4<<endl; //累加至 3 000 项时的近似值
}

```

[输出结果]

PAI500=3.13959

PAI3000=3.14126

3.16 三种控制语句的使用

读下面的程序，判断当为变量 x 输入 -2 时，执行后会显示什么结果。若为变量 x 输入 -1(或 1)时，显示结果又如何。

```
#include <iostream.h>

void main( )
{
    int x, i, j=-2, s=0;
    cout<<"Input x:";
    cin>>x;                                //注 1
    i=x;                                    //注 2
    if(i<0)                                 //注 3
    {
        cout<<"i="<<i<<" ";
        i=i+1;
    }
    cout<<"--if_OK!\n";
    i=x;                                    //注 4
    while(i<0)                             //注 5
    {
        cout<<"i="<<i<<" ";
        i+=1;
    }
    cout<<"--while_OK!\n";
    i=x;                                    //注 6
    do                                      //注 7
    {
        cout<<"i="<<i<<" ";
        i++;
    }
    while(i<0);                             //注 8
    cout<<"--do_OK!\n";
    i=x;                                    //注 9
    for(;;)                                 //注 10
```

```

    {
        s+=j+5;                //注 11
        j++;
        if(j>i)                //注 12
            break;
    }
    cout<<"s="<<s<<"  --for_OK!\n";
}

```

[分析提示]

本题中的主要语句为 if、while、do-while、以及 for。它们都是“复合型”的语句，在条件成立的情况下，都要去执行各自的“子句”句体，但执行的次数相互有区别。编程者要十分清楚每种语句各自的句体究竟要执行多少次！

[解答程序]

具体程序见题目处。

注 1：为变量 x 输入一个值。

注 2：将初始输入的 x 值赋给变量 i。

注 3：条件成立，一次性执行“子句”句体。

注 4：将初始输入的 x 值赋给变量 i。

注 5：只要条件成立，就再一次执行“子句”句体。

注 6：将初始输入的 x 值赋给变量 i。

注 7：至少执行一次“子句”句体。

注 8：只要条件成立，就再一次去执行“子句”句体。

注 9：将初始输入的 x 值赋给变量 i。

注 10：三个表达式全省略的 for 语句，认为“条件”恒真。

注 11：往 s 上累加数据。

注 12：若条件成立，靠 break 跳出循环。

[输出结果]

Input x: -2

i=-2 --if_OK!

i=-2 i=-1 --while_OK!

i=-2 i=-1 --do_OK!

s=3 --for_OK!

又一次执行:

Input x: -1

i=-1 --if_OK!

```
i=-1  --while_OK!
i=-1  --do_OK!
s=7   --for_OK!
```

再一次执行:

```
Input x:1
--if_OK!
--while_OK!
i=1  --do_OK!
s=18 --for_OK!
```

3.17 求三种不同序列的累加和

求下述三个序列的累加和(每一序列各累加 n 项, 正整数 n 通过 `cin` 从键盘输入)。

$$s1 = 1/2 + 1/5 + 1/8 + 1/11 + 1/14 + 1/17 + \dots$$

$$s2 = 1/2 - 1/9 + 1/16 - 1/23 + 1/30 - 1/37 + \dots$$

$$s3 = 2/1 + 3/2 + 5/3 + 8/5 + 13/8 + 21/13 + \dots$$

[分析提示]

(1) 说明: `double s1=0, s2=0, s3=0;` //要在 $s1$ 、 $s2$ 、 $s3$ 中存放累加和。

(2) 使用“`for(int i=1; i<=n; i++)`”形式的循环依次对三个公式中的各累加项 `item` 进行累加。循环 n 次, 各累加 n 个 `item` 项。

(3) 注意第一序列中后一 `item` 累加项的分母比前一项的分母大 3; 第二序列中后一 `item` 累加项的分母比前一项的分母大 7 而且符号互异; 第三序列中后一 `item` 累加项的分子为前一项的分子与分母之和, 而后一 `item` 累加项的分母恰为前一项的分子。这些关系是形成下一累加项的基础。

[解答程序]

```
#include <iostream.h>

void main(void)
{
    int n;
    double down1=2, down2=2, sign2=1, up3=2, down3=1;
    //n 为累加项数; down1 为第一累加序列中累加项的分母(初值为 2)
    //down2 为第二累加序列中累加项的分母, sign2 为该累加项的符号(1 或 -1)
    //up3 为第三累加序列中累加项的分子, down3 为第三累加序列中累加项的分母
    double s1=0, s2=0, s3=0;        //s1、s2、s3 中存放累加和
    double item1, item2, item3;     //item1、item2、item3 中存放累加项的值
```

```

cout<<"Input n: ";           //提示输入累加项数 n
cin>>n;                       //输入 n
for (int i=1; i<=n; i++)      //循环 n 次, 各累加 n 项
{
    item1=1/down1;            //形成第一累加序列中的当前累加项 item1
    item2=sign2*1/down2;      //形成第二累加序列中的当前累加项 item2
    item3=up3/down3;          //形成第三累加序列中的当前累加项 item3
    s1+=item1;                 //往 s1 上累加 item1
    s2+=item2;                 //往 s2 上累加 item2
    s3+=item3;                 //往 s3 上累加 item3
    down1+=3;
    //准备第一累加序列中下一累加项的分母(为当前累加项的分母加 3)
    down2+=7;
    //准备第二累加序列中下一累加项的分母(为当前累加项的分母加 7)
    sign2*=-1;
    //准备第二累加序列中下一累加项的符号(为当前累加项的符号乘-1)
    double tmp=up3;
    up3=up3+down3;
    //准备第三累加序列中下一累加项的分子(为当前累加项的分子+分母之值)
    down3=tmp;
    //准备第三累加序列中下一累加项的分母(为当前累加项的分子)
}
cout<<"s1="<<s1<<endl;      //输出第一累加序列之累加和 s1
cout<<"s2="<<s2<<endl;      //输出第二累加序列之累加和 s2
cout<<"s3="<<s3<<endl;      //输出第三累加序列之累加和 s3
}

```

[输出结果]

```

Input n: 2
s1=0.7
s2=0.388889
s3=3.5

```

再一次执行:

```

Input n: 100
s1=1.97502
s2=0.425758
s3=162.103

```

3.18 求满足 eps 精度的累加和

按如下的公式求累加和 s ，要求每一累加项的值都要大于等于 eps 精度，而实数 eps 的值(为小于 1 的正数)是通过 `cin` 从键盘输入的。即是说，累加项数与 eps 大小有关(不再累加其值小于 eps 精度的项)。

$$s = 1/1 + 2/(1+2) + 3/(1+2+3) + 4/(1+2+3+4) + 5/(1+2+3+4+5) + \dots$$

最后要将累加和 s 、所累加的项数 count 、最后那一累加项 item (其值大于等于 eps)以及刚小于 eps 尚未进行累加的那一项的具体值均一并显示在屏幕上。

例如，程序执行后可显示如下形式的结果：

Input eps (0<eps<1) : 0.00123

s = 13.9428

count = 1625

item2 (<eps) = 0.00122926

item1 (>=eps) = 0.00123001

[分析提示]

(1) 说明：`double eps, item, s=0.0;` //要在 s 中存放累加和，而要求每一累加项 item 的值都要大于等于 eps 精度值。

(2) 使用“`while (item>=eps)`”形式的循环依次对公式中的各累加项 item 进行累加，直到累加到某项的值小于 eps 精度时为止。

(3) 序列中后一累加项 item 是这样形成的：其分子为当前所累加到的项数 count 、其分母则为上一累加项的分母加上 count 。

[解答程序]

```
#include <iostream.h>
void main(void)
{
    int count=1;                //count 记录所累加的项数
    double eps, down=1.0, item, s=0.0;
        //eps 表示精度，其值(为小于 1 的正数)是通过 cin 从键盘输入的
        //down 为累加项的分母(初值为 1，以后靠逐次累加上 count 而得)
        //item 中存放累加项的值(总通过当前的 count/down 来形成)
        //s 中存放所有项的累加和(初值为 0)
    do
    {
        cout<<"Input eps ( 0<eps<1 ) : "; //提示输入表示精度的 eps
        cin>>eps;                          //输入 eps
    } while (eps>=1 || eps<=0);             //实数 eps 的值必须为小于 1 的正数
```

```

item=count/down; //item 中存放累加项的值(总通过当前的 count/down 来形成)
while (item>=eps) //若 item 的当前值大于等于 eps 精度时要继续累加
{
    //直到累加到某项的值小于 eps 精度时为止(不再累加其值小于 eps 精度的项)
    s+=item; //往 s 上累加 item
    count++;
    //准备下一累加项的分子(循环出口后, 其值减 1 即为所累加的项数)
    down+=count;
    //准备下一累加项的分母(为当前累加项的分母加上 count)
    item=count/down; //形成下一累加项 item
}
cout<<"s = "<<s<<endl; //输出累加和 s
cout<<"count = "<<count-1<<endl; //输出所累加的项数
cout<<"item2 ( <eps ) = "<<item<<endl;
    //输出刚小于 eps 尚未进行累加的那一项的具体值
down-=count; //最后那一累加项的分母为 down
count--; //最后那一累加项的分子为 count
item=count/down; //最后那一累加项的具体值为 count/down
cout<<"item1 ( >=eps ) = "<<item<<endl;
    //输出最后那一累加项的具体值(其值将大于等于 eps)
}

```

[输出结果]

```

Input eps ( 0<eps<1 ) : 0.00123
s = 13.9428
count = 1625
item2 ( <eps ) = 0.00122926
item1 ( >=eps ) = 0.00123001

```

3.19 处理从键盘输入的一批字符

从键盘输入以“ #”符号作为结束符的一批字符, 将其中的数字字符按其相应整型数值累加到一个初值为 0 的变量 sum 中, 并判断该批字符中出现了运算符(+、-、*、/)中的哪几个, 最后将结果显示到屏幕上。

例如, 程序执行后, 可在屏幕上显示如下形式的结果:

```

Input a series of characters (ended with '#'):
I am a student.#

```

----- The result -----

sum=0

No operator found!

又一次执行程序后, 输入另一批数据, 则屏幕显示结果为:

Input a series of characters (ended with #):

OK?? C++ 9a8b7c ---010#

----- The result -----

sum=25

Operators found! (+ -)

[分析提示]

(1) 可按如下方法实现: 构造一个循环, 循环中通过“`cin>>ch;`”每次读入一个字符并对 `ch` 进行处理, 直到读入了“`#`”符号后结束程序。

(2) “对字符 `ch` 进行处理”要做的是: 将数字字符按其相应整型数值累加到一个初值为 0 的变量 `sum` 中, 并判断记录出现了哪几个运算符。

[解答程序]

```
#include <iostream.h>
void main()
{
    char ch;                //说明一个 char 型变量 ch
    int sum=0, op1=0, op2=0, op3=0, op4=0;
    //变量 sum 放累加和, 要将所有的数字字符按其相应整型数值累加到 sum 中
    //op1 标记“ + ”符号出现与否(0——无, 1——出现)
    //op2 标记“ - ”符号出现与否(0——无, 1——出现)
    //op3 标记“ * ”符号出现与否(0——无, 1——出现)
    //op4 标记“ / ”符号出现与否(0——无, 1——出现)
    cout<<"Input a series of character (ended with #):"<<endl;
    //提示输入 1 批字符
    cin>>ch;                //读入一个字符放入 ch
    while (ch!= '#')        //若读入的字符非“ # ”时, 循环处理每一字符
    {
        if ( ch>='0' && ch<='9') //若 ch 为数字字符
            sum+=ch-'0';        //将所有的数字字符按其相应整型数值累加到 sum 中
        if ( ch=='+' ) op1=1;    //若 ch 为“ + ”字符, 将 op1 变量置为 1
        if ( ch=='-' ) op2=1;    //若 ch 为“ - ”字符, 将 op2 变量置为 1
        if ( ch=='*' ) op3=1;    //若 ch 为“ * ”字符, 将 op3 变量置为 1
        if ( ch=='/' ) op4=1;    //若 ch 为“ / ”字符, 将 op4 变量置为 1
        cin>>ch;                //读入下一个字符放入 ch
    }
```



```

    }
    cout<<"----- The result -----"<<endl;
    cout<<"sum="<<sum<<endl;    //输出数字字符按其相应整型数值的累加和 sum
    if(op1||op2||op3||op4)        //当出现有运算符时，输出提示信息及各运算符
    {
        cout<<"Operators found! ( ";
        if(op1) cout<<" + ";
        if(op2) cout<<" - ";
        if(op3) cout<<" * ";
        if(op4) cout<<" / ";
        cout<<")"<<endl;
    }
    else                            //没出现运算符时，输出提示信息
        cout<<"No operator found!"<<endl;
}

```

[输出结果]

Input a series of character (ended with #):

I am a student.#

----- The result -----

sum=0

No operator found!

再一次执行:

Input a series of character (ended with #):

OK?? C++ 9a8b7c ---010#

----- The result -----

sum=25

Operators found! (+ -)

3.20 将读入字符“ 拼装成” 实数并显示出来

从键盘输入以“ #” 符号作为结束符的一批字符，但要求该批输入字符将能构成一个实数(可以带正负号)，例如可输入：123#，或者 123456.78921#，或者 -5683.96#，或者 +568.72#等。程序负责逐一读入这批字符，并将它们“ 拼装成” 一个实数并显示出该实数。而且要求程序进行严格的实数格式的语法检查，一旦发现语法错误则提示出错信息并退出程序。构成所输入实数的语法(包括结束符“ #”)可设定为

<正、负号><整数部分><小数点><小数部分>#

其中的<整数部分>以及<小数部分>均由 1 到任意有限多个数字字符构成;而且允许省略掉最后的<小数点>及<小数部分>。

例如, 程序执行后的屏幕显示形式可设计为

Input a series of characters (ended with '#') that will form a real number:

- 123.45#

----- The result -----

num=- 123.45

又一次执行程序:

Input a series of characters (ended with '#') that will form a real number:

586#

----- The result -----

num=586

再一次执行程序:

Input a series of characters (ended with '#') that will form a real number:

+35.6f82#

Wrong_char -- 'f', must be digit ('0'~'9') or '#'

以下给出进一步的输入/输出对应关系实例(包含各种出错信息):

输入	输出
+568.72#	num=568.72
123456.78921#	num=123456.78921
12.5#	Wrong_char -- '', must be sign ('+', '-') or digit ('0'~'9')
+t28.96#	Wrong_char -- 't', must be digit ('0'~'9')
- 12B3.4#	Wrong_char -- 'B', must be '.' or '#' or digit ('0'~'9')
+35.6f82#	Wrong_char -- 'f', must be digit ('0'~'9') or '#'
86.+12#	Wrong_char -- '+', after '.', must be digit ('0'~'9')

[分析提示]

(1) 首先读入并处理第一个符号: 它应该是数的符号 (“+”, “-”) 或某一个数字字符 (“0” —— “9”)。

(2) 接着读入并处理“<整数部分>”——由 1 到任意有限多个数字字符构成。通过如下的公式来“拼装”起整数 num(该变量的初值为 0): $num=num*10+(ch-'0')$;

(3) 若出现了小数点的话, 要接着读入并处理<小数部分>——也由 1 到任意有限多个数字字符构成。通过如下的公式来“拼装”起小数 num2(该变量的初值为 0): $num2+=fac*(ch-'0')$; 其中的 fac 表示某一小数位的权值——初值为 0.1(表示十分位), 以后逐次除以 10(fac/=10;)用来表示点后的百分位、千分位、..., 各自的权值。

[解答程序]

```
#include <iostream.h>
#include <process.h>
```

```

void main()
{
    char ch;                //说明一个 char 型变量 ch
    cout<<"Input a series of characters (ended with '#')";
    cout<<" that will form a real number:"<<endl;    //提示输入 1 批字符
    cin>>ch;                //读入一个字符放入 ch
    double num=0;           //将在 num 中“ 拼装” 运算对象
    char sign='+';          //数的正、负号将记录在 sign 中，无符号时认为是正数
    bool hasSign=false;     //标记是否已出现过正、负号
    if(ch=='-' || ch=='+')   //是否为正、负号
    {
        sign=ch;            //将数的正、负号记录在 sign 中
        hasSign=true;       //标记已出现过正、负号
        cin>>ch;            //读入下一个字符放入 ch
    }
    if(ch<'0' || ch>'9')    //字符 ch 非数字字符
    {
        if(hasSign)         //已出现过正、负号，提示出错—输入的非数字字符
            cout<<"Wrong_char -- \"<<ch<<"\", must be digit ('0'~'9')"<<endl;
        else                //没出现过正、负号时，提示出错—输入的非数字字符或正、负号
            cout<<"Wrong_char -- \"<<ch;
            cout<<"\", must be sign ('+', '-') or digit ('0'~'9')"<<endl;
            exit(0);
    }
    while (ch>='0' && ch<='9')    //处理连续的各数字字符(小数点前的整数部分)
    {
        num=num*10+(ch-'0');    //在 num 中“ 拼装” 运算对象的整数部分
        cin>>ch;                //读入下一个字符放入 ch
    }
    if(ch=='#')              //若整数部分后面不跟小数点而为“ #”，返回“ 拼装” 好的整数
    {
        if(sign=='-')         //若有负号
            num=-num;
        cout<<"----- The result -----"<<endl;
        cout.precision(15);    //设置浮点数有效数字为 15 位
        cout<<"num="<<num<<endl; //输出整数值的累加和 sum
        return;                //结束程序
    }
    if(ch!='.')              //连续的整数部分数字字符后面没跟小数点，也不为“ #” 符号

```

```

{
cout<<"Wrong_char -- \"<<ch<<"\', must be '.' or '#' or digit (0~9)"<<endl;
    //提示出错，表示输入的非“.”或“#”或数字字符

    exit(0);
}
double num2=0.0, fac=0.1;    //处理点后部分时，在 num2 中拼装小数部分值
                             //fac 表示点后十分位、百分位、千分位、... 的因子值

cin>>ch;
if(ch<'0' || ch>'9')        //第一个字符应为数字字符
{
cout<<"Wrong_char -- \"<<ch<<"\', after '.', must be digit (0~9)"<<endl;
    //提示出错，表示点后输入的非数字字符

    exit(0);
}
while (ch>='0' && ch<='9')    //处理点后数据
{
    num2+=fac*(ch-'0');        //在 num2 中“拼装”小数部分值
    cin>>ch;                  //读入下一个字符放入 ch
    fac/=10;                  //fac 表示点后十分位、百分位、千分位、... 的因子值
}
num+=num2;                   //整数部分加上小数部分值
if(sign=='-')                 //若有负号
    num=-num;
if(ch!='#')                   //当结束符非“#”符号时出错
{
    cout<<"Wrong_char -- \"<<ch<<"\', must be digit (0~9) or '#'<<endl;
    //提示出错，表示当前符号非“#”符号也非数字字符

    exit(0);
}
cout<<"----- The result -----"<<endl;
cout.precision(15);           //设置浮点数有效数字为 15 位
cout<<"num="<<num<<endl;      //输出累加和 sum
}

```

[输出结果]

Input a series of characters (ended with '#') that will form a real number:

— 123.45#

----- The result -----

num=— 123.45

再一次执行:

Input a series of characters (ended with '#') that will form a real number:

+35.6f82#

Wrong_char -- 'f', must be digit ('0'~'9') or '#'

3.21 求 1 000 以内的所有“完全数”

求 1 000 以内的所有“完全数”(若 n 的包括 1 但不包括 n 本身的所有因子之和等于 n , 则 n 为“完全数”。如, $6=1+2+3$; $28=1+2+4+7+14$)。

[分析提示]

(1) 通过“for (int $n=1$; $n \leq 1000$; $n++$)”的循环来依次判断每一个 n 是否为“完全数”。

(2) 求出 n 的包括 1 但不包括 n 本身的所有因子, 并将它们累加到 sum 上(sum 初值为 0), 而后看 sum 是否等于 n 。

[解答程序]

```
#include <iostream.h>

void main(void)
{
    for (int n=1; n<=1000; n++)          //求 1 000 以内的所有“完全数”
    {
        int sum=0;                      //sum 中放因子的累加和
        for (int k=1; k<n; k++)
            //看 1 到 n-1 中的哪些数可以整除 n(可整除者, 都为 n 的因子)
            if (n % k == 0)              //k 为 n 的因子时
                sum+=k;                  //将 k 累加到 sum 中
        if(sum==n)                      //因子和等于 n 时即为“完全数”
            cout<<"n="<<n<<endl;      //输出该“完全数”
    }
}
```

[输出结果]

n=6

n=28

n=496

3.22 求 100~999 内的所有“回文数”

求出 100~999 内的所有“回文数”(即正读与反读大小相同的数)。如 101、111、121、...、989、999 都是“回文数”。

[分析提示]

(1) 通过“for(int n=100; n<=999; n++)”的循环来依次判断每一个 n 是否为“回文数”。

(2) 判断某数 n 是否为“回文数”，可先求出 n 的“反序数”res(为之前的范例题，已做过)，而后判断 res 是否等于 n。

[解答程序]

```
#include <iostream.h>

void main()
{
    int res, tmp, num=0;
    //在 res 中累加出“反序数”，在 num 中累加“回文数”个数
    for(int n=100; n<=999; n++)    //逐一判断 n(从 100~999)是否为“回文数”
    {
        tmp=n; res=0;            //累加“反序数”的 res 每次置为 0
        do                      //求出 n 的“反序数”放 res 中
        {
            int digit=tmp%10;    //截下当前 tmp(其初值为 n)的个位放 digit
            res=res*10+digit;    //在 res 中累加出 n 的“反序数”
                                // (原 res 各位均左移 1 位，留出个位累加上 digit)
            tmp/=10;             //将当前 tmp 除以 10(取整)，
                                //也即“甩掉”当前的个位而形成一个新数仍放 tmp 中
        } while (tmp != 0);
        if (res==n)
            //判断 n 的“反序数”res 是否等于 n(即判断 n 是否为“回文数”)
            {
                num++;           //在 num 中累加“回文数”个数
                cout<<n<<" ";   //输出“回文数”n
                if(num%12==0)    //每 12 个“回文数”占 1 行
                    cout<<endl;
            }
    }
    cout<<endl;
}
```

[输出结果]

```
101 111 121 131 141 151 161 171 181 191 202 212
222 232 242 252 262 272 282 292 303 313 323 333
343 353 363 373 383 393 404 414 424 434 444 454
464 474 484 494 505 515 525 535 545 555 565 575
585 595 606 616 626 636 646 656 666 676 686 696
707 717 727 737 747 757 767 777 787 797 808 818
828 838 848 858 868 878 888 898 909 919 929 939
949 959 969 979 989 999
```

3.23 编循环程序显示指定图形

编循环程序，负责显示出如下的图形(由 10 行符号组成)。

```
      +
    +++
  +++++
+++++++
+++++++
**
****
*****
*****
*****
```

[分析提示]

(1) 通过“for(i=1; i<=5; i++)”的循环，负责输出前 5 行的“+”号图案。注意，输出第 i 行的“+”号前，要先空 2*(5-i)格，而后连续输出 2*i-1 个“+”号。

(2) 通过第二个“for(i=1; i<=5; i++)”的循环，负责输出后 5 行的“*”号图案。注意，第 i 行要连续输出 2*i 个“*”号(前面没有空格)。

[解答程序]

```
#include<iostream.h>
void main( )
{
    int i,j,n=5;
    for(i=1; i<=n; i++)          //输出前 n 行的“+”号图案
    {
        for(j=1; j<=2*(n-i); j++)
```

```
        cout<<' ';      //输出第i行的“+”号前,先空2*(n-i)格(其中的n为常量5)
    for(j=1; j<=2*i-1; j++)
        cout<<'+';      //连续输出2*i-1个“+”号
    cout<<endl;
}
for(i=1; i<=n; i++)      //输出后n行的“*”号图案
{
    for(j=1; j<=2*i; j++)
        cout<<'*';      //第i行要连续输出2*i个“*”号(前面没空格)
    cout<<endl;
}
}
```

[输出结果]

见题目处,略。

3.24 理解二重循环程序的执行过程

读下述程序,写出执行结果。

```
#include <iostream.h>
void main(void)
{
    const int n=9;
    int i, j;
    for(i=1; i<=n; i++)      //注 1
    {
        for(j=i; j>=1; j--)    //注 2
            cout<<j<<" ";
        cout<<endl;
    }
    for(i=1; i<=9; i++)      //注 3
    {
        for(j=i; j<=9; j++)    //注 4
            cout<<j<<" ";
        cout<<endl;
    }
}
```


[分析提示]

(1) 第一个二重循环控制显示 9 行, 而第 i 行($i=1, 2, \dots, 9$)共显示 i 个以空格分割的数。

(2) 第二个二重循环也控制显示 9 行, 而第 i 行($i=1, 2, \dots, 9$)共显示 $9-i+1$ 个以空格分割的数。

[解答程序]

具体程序见题目处。

注 1: 控制显示 9 行, “行号” $i=1, 2, \dots, 8, 9$ 。

注 2: 第 i 行显示如下 i 个以空格分割的数: $i, i-1, \dots, 1$ 。

注 3: 控制显示 9 行, “行号” $i=1, 2, \dots, 8, 9$ 。

注 4: 第 i 行显示如下 $9-i+1$ 个以空格分割的数: $i, i+1, \dots, 8, 9$ 。

[输出结果]

```
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
6 5 4 3 2 1
7 6 5 4 3 2 1
8 7 6 5 4 3 2 1
9 8 7 6 5 4 3 2 1
1 2 3 4 5 6 7 8 9
2 3 4 5 6 7 8 9
3 4 5 6 7 8 9
4 5 6 7 8 9
5 6 7 8 9
6 7 8 9
7 8 9
8 9
9
```

3.25 根据要求显示具有不同特征的图形

从键盘读入一个 1~9 的数字字符(或一个 A~I 的字母)endCh, 而后显示出具有如下特征的图形: 该图形的第一行为数字字符 1(或字母 A), 第二行为从 1(或 A)符号开头、连续递增到 2(或 B)、而后连续递减且由三个符号构成的“回文字”, ..., 最后一行为从 1(或

A)符号开头、连续递增到 endCh、而后连续递减且由 $2*(endCh-1)$ 个数字字符(或由 $2*(endCh-A)$ 个字母)构成的“回文字”。

例如,当输入数字字符 4 时,显示图形如下:

```
1
121
12321
1234321
```

当输入字母 D 时,显示图形如下:

```
A
ABA
ABCBA
ABCD CBA
```

[分析提示]

(1) 说明: char endCh, startCh; //表示“结束”字符 endCh,“开始”字符 startCh。若输入的 endCh 为字母,则将“开始”字符 startCh 置为“ A”;若输入的 endCh 为数字(字符),则将“开始”字符 startCh 置为“ 0”。

(2) 通过“ for(i=startCh; i<=endCh; i++)”的循环,用来控制共显示 endCh-startCh+1 行的图形。

(3) 显示第 i 行的“数据”前,先显示若干个空格,而后显示第 i 行“前半”的“数据”:从 1(或 A)符号开头、连续递增到 i;紧接着显示第 i 行“后半”的“数据”:从 i-1 符号开始、连续递减到 1(或 A)。

[解答程序]

```
#include <iostream.h>
void main(void)
{
    char endCh, startCh;           //“结束”字符 endCh,“开始”字符 startCh
    char i, j;
    do //提示并保障用户输入了一个处于“ 1” —“ 9” 或“ A” —“ I” 的字符放在 endCh
    {
        cout<<"Input a char (1->9, A->I)";
        cin>>endCh;
    } while (!(endCh>='1' && endCh<='9') && !(endCh>='A' && endCh<='I'));
    if(endCh<'A') startCh='1';      //“开始”字符 startCh 为“ 1”
    else startCh='A';              //“开始”字符 startCh 为“ A”
    for(i=startCh; i<=endCh; i++) //控制共显示 endCh-startCh+1 行
    {
        for(j=startCh; j<(endCh-i+startCh); j++)
```

```

        cout<<" ";           //显示第 i 行的“ 数据” 前，先显示若干个空格
        for(j=startCh; j<=i; j++)
            cout<<j;
        //显示第 i 行“ 前半” 的“ 数据”：从 1(或 A)符号开头、连续递增到 i
        for(j=i-1; j>=startCh; j--)
            cout<<j;
        //显示第 i 行“ 后半” 的“ 数据”：从 i-1 符号开始、连续递减到 1(或 A)
        cout<<endl;
    }
}

```

[输出结果]

Input a char (1->9, A->I)4

```

1
121
12321
1234321

```

再一次执行：

Input a char (1->9, A->I)D

```

A
ABA
ABCBA
ABCD CBA

```

3.26 循环累加不同的项数求出多个不同近似程度的 E 值

对于 $N=2, 4, 6, 8, \dots, 18, 20$ ，分别求出 $E(N)=1/1! + 1/2! + \dots + 1/N!$ 之值并输出它们。

[分析提示]

(1) 说明：double E=0.0, item=1; E 中放累加和，item 为累加项。并用如下形式一重循环“ for(int n=1; n<=20; n++)” 配合使用“ E+=item;” 来求出所需的 10 个累加和。

(2) 当累加至偶数项时，如 2 项、4 项、6 项、...、18 项、20 项时，输出当时的累加结果 E 值。

(3) 注意后项的 item 恰为前项的 item 除以 n。

[解答程序]

```
#include <iostream.h>

//用一重循环实现，效率高、执行速度快。当然也可用二重循环或其他方法来实现。

void main(void)
{
    double E=0.0, item=1;           //E 中放累加和，item 为累加项，
    for(int n=1; n<=20; n++)
    {
        //从第 1 项开始，一共(最多)累加 20 项(算出 10 个所需结果)
        item/=n;                    //使 item 的值依次为 1/1!、1/2!、...、1/20! (后项等于前项除以 n)
        E+=item;                    //将当前项累加到 E 上
        if (n%2==0) cout<<"E("<<n<<")="<<E<<endl;
        //累加至偶数项时，如 2 项、4 项、6 项、...、18 项、20 项时，
        //输出当时的累加结果 E 值
    }
}
```

[输出结果]

```
E(2)=1.5
E(4)=1.70833
E(6)=1.71806
E(8)=1.71828
E(10)=1.71828
E(12)=1.71828
E(14)=1.71828
E(16)=1.71828
E(18)=1.71828
E(20)=1.71828
```

3.27 求满足给定两条件的最小自然数 n

求满足下述两个条件的最小自然数 n:

(1) n 的个位数字为 8;

(2) 将 n 的个位数字 8 移到最高位数字之前(如 128 移过后为 812)，所得的新数恰为 n 的 4 倍。

[分析提示]

(1) 可使用“ while (1)”形式的循环来寻找满足条件的 n: n 从 18 找起，逐次加上

10(下一个个位为 8 的数), 而后经“移动个位”后看是否满足条件, 找到所求数以后靠 break 跳出循环。

(2) 为形成“移动个位”之后的新数 n2, 可先形成原 n 的最高位 n2Most, 它的实际值为“8***”, 其位数与 n 的位数相同, 但首位数字为 8, 其余位数字全为 0。

(3) 通过“ $n2=n/10+n2Most$,”则可求出“移动个位”以后的新数 n2(实际上 n/10 为 n 甩掉个位 8 后的那一部分, 而后在它前面“拼上”了一个 8)。

[解答程序]

```
#include <iostream.h>
void main(void)  main()
{
    long n=18, n2, n2Most;
        //n 从 18 开始找起, n2 存放移动后的 n 值, n2Most 为 n2 的最高位
        //(n2Most 的实际值为“8***”, 位数与 n 的位数相同,
        //首位数字为 8, 其余位数字全为 0)
    while (1)
    {
        //n 从 18 找起, 逐次加上 10(下一个个位为 8 的数), 找到所求数后靠 break
        //跳出循环
        n2Most=1;
        while (n2Most<=n)          //通过循环为形成“最高位”做准备
            n2Most*=10;
        n2Most*=0.8;
        //此时的 n2Most 即为求出的最高位, 它的值为“80...0”,
        //位数与 n 的位数相同, 首位数字为 8, 其余位数字全为 0
        n2=n/10+n2Most;
        //n/10 为 n 甩掉个位 8 后的那一部分, 在它前面“拼上”一个 8
        if(n2==4*n)                //判断所得新数 n2 是否恰为 n 的 4 倍
        {
            cout<<"n="<<n<<endl;    //输出找到的结果
            cout<<"(n*4="<<n*4<<)"<<endl;
            break;                    //靠 break 跳出循环
        }
        n+=10;    //没找到结果时, n 加上 10 后(注意其个位仍为 8)继续找
    }
}
```

[输出结果]

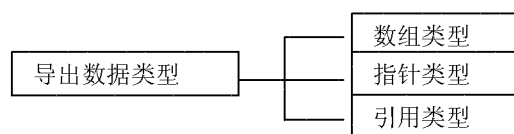
```
n=205128
(n*4=820512)
```

第四章 数组及其应用

[语法要点]

详细内容可参阅教材第四章。

(1) 导出数据类型 这种类型是在其他已定义类型的基础上定义的, 而且其运算也是确定的。在 C++ 语言中, 主要有三种, 即:



其中的数组类型代表若干同一种类型数据 (或对象) 的组合; 指针类型代表某一种类型数据 (或对象) 的首地址; 而引用类型则代表某一种类型数据 (或对象) 的引用。

(2) 数组类型 数组并不是一种新的数据类型, 而是某一特定类型 (可以是基本类型及其派生类型, 也可以是用户定义的类型或导出类型) 的若干变量 (或对象) 成组说明的一种形式, 因此, 把数组及后面介绍的指针和引用类型称为导出类型。例如:

```
int array[ 4];
```

这个说明语句说明了四个整型变量, 这四个变量有一个共同的名字 `array`, 它们之间则由“下标”来区分, 即 `array[0]`, `array[1]`, `array[2]`, `array[3]`。

(3) 一维数组的说明 任何一种已知类型数据都可说明为数组, 数组说明的格式为
<类型名><数组名>[<元素数>]={<初值表>} ; (注, 初值表部分可省略)

例如: `int list[100]`, `A[10][10]`, `B[]={4,3,2}`;

```
char ch[26];
```

```
complex com[4]={ (3.2, 4.7), (0.0, 0.0) };
```

其中, 数组 `list[100]` 未赋初值。数组 `A[10][10]` 为一个二维数组 (它也可以视为由一维数组 `int A[10]` 组成的数组), 未赋初值。数组 `B[ ]` 已赋初值, 元素数为 3。数组 `com[4]` 是用户定义的 `complex` 类型的数组, 数组有四个元素, 其中前两个元素已赋初值, 初值分别为 (3.2, 4.7), (0.0, 0.0)。

(4) 下标变量 C++ 语言未提供对于数组整体进行操作的运算符和运算。对于数组的操作是通过对于其元素即下标变量进行的 (注, 数组元素在内存中连续存放, 数组名本身代表这批连续数据在内存中的首地址, 在以后的与指针相关的章节中, 还允许利用数组名进行一些其他的运算与操作)。对于下标变量可进行下列操作:

- 赋值 (可按以下三种方式进行):

- ① 初始化: `int A[ 4] = {1, 2, 3};`

- ② 赋值语句: `A[3]= 4* A[1] ;`

③ 输入语句: 如 `cin >> A[0]` ; 或 `for(int i=0; i<4; i++) cin >> A[i];`

• 一般运算: 下标变量可与同一类型的一般变量一样参加它所允许的运算。如,
`A[0] += A[2]++; cout << A[0] << " " << A[1] * A[2];` 等等。

下标也可以是一个表达式, 如, `int A[4] = {1, 2, 3, 4}; A[3] = A[A[2] - A[1]] * 4;`
 另外注意, 下标表达式的值应保持在 $0 \sim n-1$ 范围之内(其中的 n 为数组具有的元素个数)。

(5) 多维数组的说明(以二维为例)

`<类型名><数组名>[<行数>][<列数>]`

在二维数组中, 为数组赋初值的方式有

```
int a[2][3] = {{1,2,3},{4,5,6}};
```

亦可写为

```
int a[2][3] = {1, 2, 3, 4, 5, 6};
```

二维数组的操作, 同样是通过对其元素即下标变量的操作来进行的, 二维数组的元素即下标变量均含有两个下标, 第一个下标通常称为行下标, 而第二个下标被称为列下标。

(6) 数组与字符串 从表面上看, 一个字符串就是一个字符数组, 但在 C++ 语言中, 二者并不完全相同, 读者在涉及到字符串处理时, 必须注意。

字符串是一个以串尾符“`\0`”结尾的字符型数组, 但任一个字符型数组不见得必为字符串(因为其中可以不包含“`\0`”结尾字符)。从下面的例子可看出其区别:

```
char string1[7] = "China";
char string2[] = "China";
char string3[7] = {'c', 'h', 'i', 'n', 'a'};
char string4[7] = {'c', 'h', 'i', 'n', 'a', '\0'};
```

`string1` 的长度为 7, 赋初值后其前 6 个元素已被赋值(第 6 个元素为“`\0`”);

`string2` 的长度为 6 (第 6 个元素为“`\0`”);

`string3` 的长度为 7, 最后没有串尾符“`\0`”;

`string4` 的长度为 7, 赋初值后与 `string1` 相同。

4.1 输入 10 个整数而后反序输出的实现方式二

读下述程序, 并上机进行验证, 看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意, 本程序使用 `int` 型数组存放输入数据。

```
#include <iostream.h>

void main()
{
    int b[10], i; //注 1
    cout << "Input 10 integers: " << endl; //注 2
    for (i=0; i<10; i++) //注 3
```

```
        cin>>b[i];
    cout<<"---- The result ----"<<endl;
    for (i=9; i>=0; I--)                //注 4
        cout<<b[i]<<" ";
    cout<<endl;
}
```

[分析提示]

程序说明一个具有 10 个 int 型分量的数组, 首先将数据通过 cin 依次输入到数组各分量中(按下标从小到大存放), 而后再通过下标的变化(从大到小)将这些数据输出来。

[解答程序]

具体程序见题目处。

注 1: 说明了一个 int 型的一维数组 b, 及一个整型变量 i。

注 2: 提示输入 10 个整数。

注 3: 输入 10 个整数, 依次放入各下标变量中, 下标 i 从 0 起, 递增变化到 9。

注 4: 按反序输出 b 数组中的各元素, 下标 i 从 9 起, 递减变化到 0。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```

4.2 二进制数转化为十进制数

将输入的二进制数(一个非“0”即“1”的字符串)转化为十进制数。

[分析提示]

用字符数组 a 存放刚输入的二进制数(非“0”即“1”的字符串); 而后从后往前逐一计算每一位的“位权”w (2 的 0 次方、2 的 1 次方、...), 再计算“位权”乘以“位值”的值并将其累加到一个初值为 0 的变量 value 上, 最后输出该 value。

[解答程序]

```
#include <iostream.h>
#include <string.h>
#include <process.h>
void main(void)
```



```

{
    int i, value=0, w=1, bitv;
        //value 放累加和, w 代表“位权” (2 的 0 次方、2 的 1 次方、...)
    char a[20];
    cout<<"Input a bit binary number (less than 20 bit): "; //提示输入
    cin>>a; //用字符数组 a 存放刚输入的二进制数(非“0”即“1”的字符串);
    for (i=strlen(a)-1; i>=0; i--) //从后往前逐一处理每一个二进制位
    {
        bitv=a[i];
        if (bitv == '1')
            value += w;
            //计算“位权”乘以“位值”并累加到一个初值为 0 的变量 value 上
        else
            if (bitv != '0') //输入的某位既非“0”又非“1”时, 出错退出
            {
                cout<<"Input error!"<<endl;
                exit(0);
            }
            w*=2; //使 w 变为下一位的权值
    }
    cout<<a<<"(2) = "<<value<<"(10)"<<endl; //最后输出该 value
}

```

[输出结果]

Enter a bit binary number (less than 20 bit): 10111

10111(2) = 23(10)

再一次执行:

Enter a bit binary number (less than 20 bit): 11111111

11111111(2) = 255(10)

4.3 十进制正整数转化为二进制

将输入的十进制正整数转化为二进制数。

[分析提示]

采用的十进制数转化为二进制数方法为:“逐次除以 2, 倒着排余数”。即是说, 使用一个 int 型数组 a 存放逐次得到的各余数, 最后将数组 a 中的各余数倒着输出来, 此结

果恰为所需的二进制数。

[解答程序]

```
#include <iostream.h>
void main(void)
{
    int ind=0, bitv, n;           //ind 初值为 0(从 0 下标开始往 a 中放各余数)
    int a[20];                   //用 int 型数组 a 存放逐次得到的各余数
    cout<<"Input a positive integer: "; //提示输入正整数
    cin>>n;                       //输入的正整数放 n 中
    cout<<n<<"(10)=";           //输出 10 进制形式的 n 值
    while (n>=1)                 //逐次除以 2, 将余数放在数组 a 中(按下标 ind 位置放)
    {
        bitv=n%2;                //余数在 bitv
        a[ind]=bitv;              //将余数放在数组 a 中(按 ind 下标位置放)
        ind++;                    //下推下标 ind 位置
        n=n/2;                    //将 n 除以 2(取整)仍放回 n
    }
    for(int i=ind-1; i>=0; i--) //将数组 a 中的余数倒着输出来(即为所求结果)
        cout <<a[i];
    cout<<"(2)"<<endl;
}
```

[输出结果]

Input a positive integer: 13

13(10)=1101(2)

再一次执行:

Input a positive integer: 255

255(10)=11111111(2)

4.4 求字符串长度

输入一个字符串放入字符数组中,而后统计并输出该串的长度(但不能使用标准库函数 strlen)。

[分析提示]

(1) 说明字符数组 s, 并通过 "cin.getline(s, 100);" 从键盘 cin 往 s 中输入一个字符串。

getline 的第二参数 100 用来限制输入串的长度最大为 100。

(2) 字符串的结束符号为“ \0”，可从 s[0]依次往后直到找到“ \0”字符处，从而确定出字符串的长度。

[解答程序]

```
#include <iostream.h>
void main()
{
    char s[101];    //字符数组 s 用于存放字符串
    cout<<"Input a string (Ended with ENTER):"<<endl;    //提示输入
    cin.getline(s, 100);    //输入串 s
    int i=0;
    while(s[i])    //逐一下推 i (i 初值为 0)，直到遇见“ \0” (注意“ \0” 的值等于 0)
        i++;
    cout<<"The string length="<<i<<endl;    //s 串的长度为当前的 i 值
}
```

[输出结果]

```
Input a string (Ended with ENTER):
abc 12345 999!
The string length=14
```

4.5 两个字符串连接

输入两个长度不超过 100 的字符串放入两个不同的字符数组中，而后将两个输入串连接起来(放入第一个字符数组中)，并输出结果串以及各串的长度(但不能使用标准库函数 strcat 及 strlen)。

[分析提示]

(1) 说明：char s1[202], s2[101]; 并从键盘读入两个字符串放入此二数组中。

(2) 将 s2 字符串连接到 s1 字符串的“后面”(“压着 s1 的“ \0”字符也即串结束符”往后连接)。连接结束后，一定要往 s1 串的尾部再放置一个“ \0”字符，以表示新的 s1 串的结束。

[解答程序]

```
#include <iostream.h>
void main()
{
```

```

char s1[202], s2[101];           //两个字符数组用于存放字符串
                                //将 s2 串连接到 s1 串的后面(“ 压着 ‘ \0 ’ 字符” 往后接)
cout<<"Input the first string (Ended with ENTER):"<<endl;
cin.getline(s1, 100);           //输入第一个串 s1
cout<<"Input the second string (Ended with ENTER):"<<endl;
cin.getline(s2, 100);           //输入第二个串 s2
cout<<"-----"<<endl;
int i=0, j=0, s1Len, s2Len, catLen;
while(s1[i])//逐一下推 i (i 初值为 0), 直到遇见“ \0” (注意“ \0” 的值等于 0)
    i++;
s1Len=i;                        //s1 串的长度为当前的 i 值
while(s2[j])                    //逐一下推 j (j 初值为 0), 直到遇见“ \0”
    s1[i++] = s2[j++];          //将 s2 的诸字符连接到 s1 的尾部
s1[i]='\0';                     //存放一个结束符
s2Len=j;                        //s2 串的长度为当前的 j 值
catLen=s1Len+s2Len;
cout<<"The result cat_string is:"<<endl;
cout<<s1<<endl;                //当前的 s1 已为连接后的串
cout<<"First string length="<<s1Len<<endl;
cout<<"Second string length="<<s2Len<<endl;
cout<<"result cat_string length="<<catLen<<endl;
}

```

[输出结果]

```

Input the first string (Ended with ENTER):
I am a student.
Input the second string (Ended with ENTER):
aaa bcd12345 !!! ok!
-----
The result cat_string is:
I am a student.aaa bcd12345 !!! ok!
First string length=15
Second string length=20
result cat_string length=35

```

4.6 字符串比较

输入两个长度不超过 100 的字符串放入两个不同的字符数组中，而后比较两个输入

串的大小(按字典序), 并输出比较结果(但不能使用标准库函数 `strcmp`)。

[分析提示]

(1) 通过使用“`while(s1[i] && s2[i])`”形式的循环(下标 `i` 从 0 开始, 每循环一次 `i` 增长 1), 在 `s1` 串与 `s2` 串都没结束时, 来逐一比较两个串的相应字符, 一旦比较出大小, 显示出比较结果信息后立刻使用 `exit` 来结束整个程序。

(2) 上述 `while` 没比较出大小时, 此时 `s1` 串与 `s2` 串中至少有一个已经结束, 而且二串的前半部分完全相同。若 `s1` 串尚未结束(而 `s2` 串已结束)则 `s1` 串大。若 `s2` 串尚未结束(而 `s1` 串已结束)则 `s2` 串大。

(3) 若二串同时结束, 且所有字符完全相同, 则二字符串相等。

[解答程序]

```
#include <iostream.h>
#include <process.h>

void main( )
{
    char s1[101], s2[101];           //两个字符数组用于存放字符串
    cout<<"Input the first string (Ended with ENTER):"<<endl;
    cin.getline(s1, 100);           //输入第一个串 s1
    cout<<"Input the second string (Ended with ENTER):"<<endl;
    cin.getline(s2, 100);           //输入第二个串 s2
    cout<<"-----"<<endl;
    int i=0;
    while(s1[i] && s2[i])           //s1 串与 s2 串都没结束时, 逐一比较相应字符
    {
        //逐一下推 i (i 初值为 0), 直到遇见某一个串的“\0”时停止
        if(s1[i]>s2[i])               //s1 串的当前字符大于 s2 串的当前字符时
        {
            cout<<"str1>str2"<<endl; //已比较出大小, 显示信息后退出
            exit(0);
        }
        else
        {
            if(s1[i]<s2[i])           //s1 串的当前字符小于 s2 串的当前字符时
            {
                cout<<"str1<str2"<<endl; //已比较出大小, 显示信息后退出
                exit(1);
            }
            i++;                     //当前还决不出大小, 继续比较下一个字符
        }
    }
}
```

```

//注意，此时 s1 串与 s2 串中至少有一个已经结束，而且二串的前半部分
//完全相同
if(s1[i])                                //若 s1 串尚未结束(而 s2 串已结束)则 s1 串大
{
    cout<<"str1>str2"<<endl;           //已比较出大小，显示信息后退出
    exit(2);
}
if(s2[i])                                //若 s2 串尚未结束(而 s1 串已结束)则 s2 串大
{
    cout<<"str1<str2"<<endl;           //已比较出大小，显示信息后退出
    exit(3);
}
cout<<"str1=str2"<<endl;                //二串同时结束，且所有字符完全相同
}

```

[输出结果]

Input the first string (Ended with ENTER):

abc 12345

Input the second string (Ended with ENTER):

abc 1234

str1>str2

再一次执行:

Input the first string (Ended with ENTER):

12 china OK!

Input the second string (Ended with ENTER):

12 china ok!!!

str1<str2

4.7 求 1 000 以内的所有“完全数”并将结果

按指定格式输出

求 1 000 以内的所有“完全数”(若 n 的包括 1 但不包括 n 本身的所有因子之和等于 n ，则 n 为“完全数”。如： $6=1+2+3$ ； $28=1+2+4+7+14$)。并按如下的格式输出结果：

$n=6=1+2+3$

$n=28=1+2+4+7+14$

...

[分析提示]

- (1) 该题是对 34 题的改进。要求输出完全数的同时，按格式输出该完全数的所有因子。
- (2) 通过“for (int n=1; n<=1000; n++)”的循环来寻找 1 000 以内的所有“完全数”。
- (3) 设置 factor 数组，用于存放数 n 的每一个因子。

[解答程序]

```
#include <iostream.h>

void main(void)
{
    for (int n=1; n<=1000; n++)          //求 1 000 以内的所有“完全数”
    {
        int sum=0;                      //sum 存放因子的累加和
        int factor[50], ind=0;          //factor 数组中存放各因子, ind 记录因子个数
        for (int k=1; k<n; k++)
            //看 1 到 n-1 中的哪些数可以整除 n(可整除者, 都为 n 的因子)
        {
            if (n % k == 0)              //k 为 n 的因子时
            {
                factor[ind++]=k;          //将 k 放入 factor 数组中, ind 下推 1
                sum+=k;                  //将 k 累加到 sum 中
            }
        }
        if(sum==n)                      //因子和 sum 等于 n 时即为“完全数”
        {
            cout<<"n="<<n<<"=";        //输出该“完全数”
            for(int i=0; i<ind; i++)      //输出该“完全数”的各因子(按要求的累加式格式)
                if(i==0) cout<<factor[i];
                else      cout<<'+'<<factor[i];
            cout<<endl;
        }
    }
}
```

[输出结果]

$n=6=1+2+3$

$n=28=1+2+4+7+14$

$n=496=1+2+4+8+16+31+62+124+248$

4.8 求数组中的最大数和最小数

任意输入 10 个整数放于 A 数组中,而后求出该数组中的最大数 **max** 以及最小数 **min**。

[分析提示]

- (1) 先认为 A[0]既为最大数又为最小数(通过: **max=min=A[0];**)。
- (2) 若后面的 A[i](i=1, 2, ..., 9)大于 **max**, 则将 **max** 改为 A[i]。
- (3) 若后面的 A[i](i=1, 2, ..., 9)小于 **min**, 则将 **min** 改为 A[i]。

[解答程序]

```
#include <iostream.h>
void main( )
{
    int A[10];
    int i, max, min;
    cout<<"input A's 10 numbers:"<<endl;
    for(i=0; i<10; i++)          //任意输入 10 个整数放于 A 数组中
        cin>>A[i];
    max=min=A[0];                //先认为 A[0]既为最大数又为最小数
    for(i=1; i<10; i++)
    {
        if(A[i]>max)              //若后面的 A[i]大于 max, 则将 max 改为 A[i]
            max=A[i];
        if(A[i]<min)              //若后面的 A[i]小于 min, 则将 min 改为 A[i]
            min=A[i];
    }
    cout<<"max="<<max<<endl;
    cout<<"min="<<min<<endl;
}
```

[输出结果]

```
input A's 10 numbers:
22 59 -4 123 -82 0 66 10 -29 88
max=123
min=-82
```


4.9 三种控制语句与数组的配合使用

读下面的程序，看执行后会显示出什么结果(注意，该范例与范例 3.16 有关联)。

```
#include <iostream.h>

void main( )
{
    int i, k;
    int j=-2, s=0;
    int n[3]={ -2, -1, 1 };           //注 1
    for(k=0; k<3; k++)                //注 2
    {
        i=n[k];                       //注 3
        if(i<0)                       //注 4
        {
            cout<<"i="<<i<<" ";
            i=i+1;
        }
        cout<<"--if_OK!\n";
        i=n[k];
        while (i<0)                   //注 5
        {
            cout<<"i="<<i<<" ";
            i+=1;
        }
        cout<<"--while_OK!\n";
        i=n[k];
        do                            //注 6
        {
            cout<<"i="<<i<<" ";
            i++;
        }
        while(i<0);                   //注 7
        cout<<"--do_OK!\n";
        i=n[k];
        for(;;)                       //注 8
        {
            s+=j+5;                   //注 9
            j++;
        }
    }
}
```

```

        if(j>i) break;           //注 10
    }
    cout<<"s="<<s<<"  --for_OK!\n";
    cout<<"-----\n";
}
}

```

[分析提示]

(1) 本范例与范例 3.16 相比,多使用了一个 `n` 数组,并在最外层增加了一个“`for(k=0; k<3; k++)`”的循环,在循环体中逐次取出 `n` 数组中的元素进行使用。

(2) 本题中的主要语句仍为 `if`、`while`、`do-while`、以及 `for`。目的仍是要搞清楚每种语句各自的使用语义。

[解答程序]

具体程序见题目处。

注 1: 说明具有三个元素的 `n` 数组,并赋予初始值。

注 2: 三次循环,依次使用 `n` 数组中的三个数。

注 3: 将 `n` 数组第 `k` 个分量之值赋给 `I`。

注 4: 若条件成立,一次性执行“子句”句体。

注 5: 只要条件成立,就再一次执行“子句”句体。

注 6: 至少执行一次“子句”句体。

注 7: 只要条件成立,就再一次去执行“子句”句体。

注 8: 三个表达式全省略的 `for` 语句,认为“条件”恒真。

注 9: 往 `s` 上累加数据。

注 10: 若条件成立,靠 `break` 跳出循环。

[输出结果]

```

i=-2  --if_OK!
i=-2  i=-1  --while_OK!
i=-2  i=-1  --do_OK!
s=3  --for_OK!
-----
i=-1  --if_OK!
i=-1  --while_OK!
i=-1  --do_OK!
s=7  --for_OK!
-----
--if_OK!
--while_OK!

```

```

i=1  --do_OK!
s=18  --for_OK!
-----

```

4.10 对数组中的 10 个元素进行排序

任意输入 10 个整数放于 A 数组中,而后使用某种方法对数组 A 的 10 个元素进行排序(升序排列),并将排序结果显示在屏幕上。

[分析提示]

(1) 使用“ **for (i=0; i<9; i++)**”的循环,使 A 数组中的 10 个数据按升序排列。要保障: **i=0** 的循环使 A[0]处的元素交换为最小, **i=1** 的循环使 A[1]处的元素交换为次小, ...。

(2) 每遍 **i** 循环中,总找出从 **a[i]**到数组末这些元素中的最小者,而后将最小者调换到 **a[i]**的位置上。

[解答程序]

```

#include <iostream.h>
void main( )
{
    const int n=10;
    int i, j, smallIndex, A[n];    //smallIndex 为每一趟中选出的最小元素之下标
    cout<<"input A's 10 numbers:"<<endl;
    for(i=0; i<10; i++)          //任意输入 10 个整数放于数组 A 中
        cin>>A[i];
    for (i=0; i<n-1; i++)
    {
        //i=0 的循环使 A[0]处交换为最小, i=1 的循环使 A[1]处交换为次小, ...
        smallIndex=i;           //先认为最小元素之下标为 i (即认为 A[i]最小)
        //在元素 A[i+1]到 A[n-1]中逐个比较找出最小者, 并记录
        //其下标到 smallIndex
        for (j=i+1; j<n; j++)
            if (A[j] < A[smallIndex])
                smallIndex = j;
        //smallIndex 始终记录当前找到的最小值的下标将这一趟找到的最小
        //元素与 A[i]交换(也即交换 A[i]与 A[smallIndex]之值)
        int temp;
        temp = A[i];
        A[i] = A[smallIndex];
        A[smallIndex] = temp;
    }
}

```

```

    }
    for (i=0; i<10; i++)        //输出排序后的 A 数组各元素值(已有序!)
        cout<<A[i]<<" ";
    cout<<endl;
}

```

[输出结果]

input A's 10 numbers:

22 11 39 0 -34 100 -3 66 39 -99

-99 -34 -3 0 11 22 39 39 66 100

4.11 合并两个有序数组

设有如下对数组 A、B 的说明,并假设两个数组的元素都已经有序(升序排列)。编程,合并数组 A、B 形成一个新数组 C,但要使 C 的元素仍有序(升序排列)。

int A[10]={-5, -2, 0, 6, 18, 39, 66, 66, 68, 100};

int B[10]={3, 4, 10, 18, 19, 28, 50, 59, 62, 66};

[分析提示]

(1) 由于数组 A、B 都已有序,可从数组 A 与 B 的 0 下标处开始逐一往后取元素进行比较,将其中的小者放入 C 数组。而后看小者属于哪一数组(A 或 B)就再从那一数组中取来下一元素再继续进行比较,如此等等,直到 A 或 B 某一数组的所有元素都被取光时为止。

(2) 将另一个尚未取光其元素的数组中的剩余各元素依次取来放入 C。

[解答程序]

```

#include <iostream.h>
void main( )
{
    int A[10]={-5, -2, 0, 6, 18, 39, 66, 66, 68, 100}; //数组 A 已经有序(升序排列)
    int B[10]={3, 4, 10, 18, 19, 28, 50, 59, 62, 66}; //数组 B 已经有序(升序排列)
    int i, C[20];
    cout<<"----- A (small->large) -----"<<endl;
    for(i=0; i<10; i++)        //输出数组 A
        cout<<A[i]<<" ";
    cout<<endl;
    cout<<endl<<"----- B (small->large) -----"<<endl;
    for(i=0; i<10; i++)        //输出数组 B

```

```

        cout<<B[i]<<" ";
    cout<<endl;
    //将 A 与 B 的各元素按从小到大的顺序合并到 C 中
    int indA=0, indB=0, indC=0;           //从数组 A 与 B 的 0 下标处开始逐一往后取元素
    while(indA<10 && indB<10)           //当数组 A 与 B 都没结束时(二者都没被取光时)
        if(A[indA]<=B[indB])
            C[indC++]=A[indA++];         //数组 A 的元素小, 取出该元素放入 C
        else
            C[indC++]=B[indB++];         //数组 B 的元素小, 取出该元素放入 C
    while(indA<10)                       //当数组 A 还没结束时, 将剩余各元素放入 C
        C[indC++]=A[indA++];
    while(indB<10)                       //当数组 B 还没结束时, 将剩余各元素放入 C
        C[indC++]=B[indB++];
    cout<<endl<<"----- C (small->large) -----"<<endl;
    for(i=0; i<20; i++)                 //显示出 C 的各元素
        cout<<C[i]<<" ";
    cout<<endl;
}

```

[输出结果]

```

----- A (small->large) -----
-5  -2  0  6  18  39  66  66  68  100

----- B (small->large) -----
3  4  10  18  19  28  50  59  62  66

----- C (small->large) -----
-5  -2  0  3  4  6  10  18  18  19  28  39  50  59  62  66  66  66  68  100

```

4.12 罗马数据转化为十进制数据

将输入的罗马数据转化为十进制数。

假设罗马数据中只使用如下七个基值字母: M、D、C、L、X、V、I, 分别用来表示 1 000、500、100、50、10、5、1。例如罗马数据 DLXVII 对应的十进制数为 567, 而罗马数据 MDCCCCLXXVI 对应的十进制数为 1 976。

[分析提示]

可使用如下方法来实现: 用字符数组 a 存放刚输入的罗马数据(使用七个基值字母组

合成的字符串); 而后对 a 数组中的各“基值”字母进行转换, 并累加“拼装”到一个初值为 0 的变量 value 上(通过“ while (a[i])”的循环进行); 最后输出该 value。

[解答程序]

```
#include <iostream.h>

void main(void)
{
    int i=0, value=0;           //value 放累加和
    char a[30], baseCh;        //baseCh 放基值字母
    cout<<"Input a Roman_number(only use--M,D,C,L,X,V,I): ";    //提示输入
    cin>>a;
    //用字符数组 a 存放刚输入的罗马数据(使用七个基值字母组合成的字符串)
    while (a[i])               //从前往后逐一处理每一个基值字母
    {
        baseCh=a[i++];        //baseCh 为基值字母, 使用完 i 后下推 1
        switch(baseCh)
        {
            case 'M':          //基值字母 M 表示 1 000
                value += 1000;
                                //将基值字母所表示的数量累加到变量 value 上
                break;
            case 'D':          //基值字母 D 表示 500
                value += 500;
                                //将基值字母所表示的数量累加到变量 value 上
                break;
            case 'C':          //基值字母 C 表示 100
                value += 100;
                break;
            case 'L':          //基值字母 L 表示 50
                value += 50;
                break;
            case 'X':          //基值字母 X 表示 10
                value += 10;
                break;
            case 'V':          //基值字母 V 表示 5
                value += 5;
                break;
            case 'I':          //基值字母 I 表示 1
                value += 1;
```

```

        break;
    default:          //非七个基值字母时提示出错
        cout<<"Wrong! baseCh="<<baseCh<<endl;
        break;
    }
}
cout<<a<<"(Rom) = "<<value<<"(10)"<<endl; //最后输出该 value
}

```

[输出结果]

Input a Roman_number(only use--M,D,C,L,X,V,I): DCLXXVIII
 DCLXXVIII(Rom) = 678(10)

再一次执行:

Input a Roman_number(only use--M,D,C,L,X,V,I): MDCCCCLXXVI
 MDCCCCLXXVI(Rom) = 1976(10)

4.13 十进制正整数转化为罗马数据

将输入的十进制正整数转换为罗马数据。

假设罗马数据中只使用如下七个基值字母: M、D、C、L、X、V、I, 分别用来表示 1 000、500、100、50、10、5、1。例如十进制数 678 对应的罗马数据为: DCLXXVIII, 而十进制数 1 976 对应的罗马数据为 MDCCCCLXXVI。

[分析提示]

(1) 转换方法一: 转换过程实际上是求出所需每一基值字母的个数, 而且总要尽可能地使用大基值字母。假设用 n_{1000} 记录经转换后对“M”所需的个数, 用 n_{500} 记录经转换后对“D”所需的个数, 并假设要将十进制正整数 n 转换为罗马数据, 则可以使用多个类似于下述形式的语句:

```

if (n>=1000)          //从大到小计算出对每一基值所需的个数(基值 1 000 最大)
{
    n1000=n/1000; //所需基值 1 000 的个数(即转换结果中所需“ M” 的个数)
    n=n%1000;    //不足 1 000 的余数部分仍放入 n
}
if (n>=500)           //从大到小计算出对每一基值所需的个数(基值 500 次大)
{
    n500=1;        //所需基值 500 的个数(即转换结果中所需“ D” 的个数)
}

```

```

n=n-500;        //不足 500 的余数部分仍放入 n
}

```

(2) 转换方法二：该转换方法是对方法一的改进。可使用三个数组，并通过使用它们来简化程序。

```

int base[7]={1000, 500, 100, 50, 10, 5, 1};    //罗马数据的基值
int count[7]={0, 0, 0, 0, 0, 0, 0};           //对每一基值所需的个数
char romCh[7]={ 'M','D','C','L','X','V','I'}; //表示基值的字母
for(int i=0; i<7; i++)                        //从大到小计算出对每一基值所需的个数 count[i]
    if (n>=base[i])                          //其中的 n 为要进行转换的十进制正整数
    {
        count[i]=n/base[i];                  //算出所需基值 base[i]的个数 count[i]
        n=n-...;                             //甩掉大基值个数的数据后所剩的零头
    }

```

[解答程序一]

```

#include <iostream.h>
void main(void)
{
    int n;
    cout<<"Input a positive integer: ";      //提示输入正整数
    cin>>n;                                   //输入的正整数放 n 中
    cout<<n<<"(10) = ";                     //输出十进制形式的 n 值
    int n1000=0, n500=0, n100=0, n50=0, n10=0, n5=0, n1=0;
                                           //记录经转换后，对每一基值所需的个数
    if (n>=1000)                          //从大到小计算出对每一基值所需的个数(基值 1 000 最大)
    {
        n1000=n/1000;                      //所需基值 1 000 的个数(即转换结果中所需“ M” 的个数)
        n=n%1000;                          //不足 1 000 的余数部分
    }
    if (n>=500)                            //从大到小计算出对每一基值所需的个数(基值 500 次大)
    {
        n500=1;                            //所需基值 500 的个数(即转换结果中所需“ D” 的个数)
        n=n-500;                          //不足 500 的余数部分
    }
    if (n>=100)                            //基值 100 第三大
    {
        n100=n/100;                        //所需基值 100 的个数(即转换结果中所需“ C” 的个数)
        n=n%100;                          //不足 100 的余数部分
    }
}

```



```

    if (n>=50)
    {
        n50=n/50;          //所需基值 50 的个数(即转换结果中所需“ L” 的个数)
        n=n-50;            //不足 50 的余数部分
    }
    if (n>=10)
    {
        n10=n/10;          //所需基值 10 的个数(即转换结果中所需“ X” 的个数)
        n=n%10;            //不足 10 的余数部分
    }
    if (n>=5)
    {
        n5=n/5;            //所需基值 5 的个数(即转换结果中所需“ V” 的个数)
        n=n-5;             //不足 5 的余数部分
    }
    n1=n;                  //所需基值 1 的个数(即转换结果中所需“ I” 的个数)
    for(int i=1; i<=n1000; i++) //所需基值 1000 的个数——输出所需的若干个“ M”
        cout<<"M";
    for(i=1; i<=n500; i++)    //所需基值 500 的个数——输出所需的若干个“ D”
        cout<<"D";
    for(i=1; i<=n100; i++)    //所需基值 100 的个数——输出所需的若干个“ C”
        cout<<"C";
    for(i=1; i<=n50; i++)     //所需基值 50 的个数——输出所需的若干个“ L”
        cout<<"L";
    for(i=1; i<=n10; i++)     //所需基值 10 的个数——输出所需的若干个“ X”
        cout<<"X";
    for(i=1; i<=n5; i++)      //所需基值 5 的个数——输出所需的若干个“ V”
        cout<<"V";
    for(i=1; i<=n1; i++)      //所需基值 1 的个数——输出所需的若干个“ I”
        cout<<"I";
    cout<<"(Rom)"<<endl;
}

```

[输出结果]

Input a positive integer: 678
 678(10) = DCLXXVIII(Rom)

再一次执行:

Input a positive integer: 1976

1976(10) = MDCCCCLXXVI(Rom)

[解答程序二]

```
#include <iostream.h>

void main(void)
{
    int n;
    cout<<"Input a positive integer: ";           //提示输入正整数
    cin>>n;                                         //输入的正整数放 n 中
    cout<<n<<"(10) = ";                          //输出十进制形式的 n 值
    int base[7]={ 1000, 500, 100, 50, 10, 5, 1};   //罗马数据的基值
    int count[7]={0, 0, 0, 0, 0, 0, 0};           //对每一基值所需的个数
    char romCh[7]={'M','D','C','L','X','V','I'};   //表示基值的字母
    for(int i=0; i<7; i++)        //“ 从大到小” 计算出对每一基值所需的个数 count[i]
        if (n>=base[i])
        {
            count[i]=n/base[i];                //算出所需基值 base[i]的个数 count[i]
            //其中的 base[i]为基值 1000, 500, 100, 50, 10, 5, 1 之一
            n=n%base[i];                        //甩掉大基值个数的数据后所剩的“ 零头”
        }
    for(i=0; i<7; i++)                        //结果至多由七种基值字母构成
        for(int j=0; j<count[i]; j++)         //count[i]中为每一基值所需的个数
            cout<<romCh[i];                   //输出相应基值字母
    cout<<"(Rom)"<<endl;
}
```

[输出结果]

与解答一处相同，略。

4.14 使用最少数人民币纸币方案的求取

输入一个正整数 m ，它代表一个人民币钱数(元数)。求取这样一个方案，使用最少张数的人民币纸币，凑成上述的钱数 m 。

注，共有七种元以上面值的人民币纸币，分别为：100，50，20，10，5，2，1。假设输入数据为 196 时，则最少要用 6 张人民币纸币： $196=1*100+1*50+2*20+1*5+1*1$ 。当输入数据为 908 时，则最少要用 12 张人民币纸币： $908=9*100+1*5+1*2+1*1$ 。

可设计一个程序，使执行后的屏幕显示结果具有如下形式：

Input a positive integer: 196

196(yuan) = 1*100 + 1*50 + 2*20 + 1*5 + 1*1(yuan)

total_sheet=6

[分析提示]

求取过程实际上是求出所需每一面值人民币的张数(而且总要尽可能地使用大面值钱币)。处理方法可参照上一题的“十进制正整数转换为罗马数据”的方法。

[解答程序]

```
#include <iostream.h>

void main(void)
{
    int n;                                //代表要输入的正整数(元数)
    int base[7]={100, 50, 20, 10, 5, 2, 1}; //人民币纸币七种元以上的面值
    int count[10]={0, 0, 0, 0, 0, 0, 0}; //记录对每一面值的纸币所需的张数
    cout<<"Input a positive integer: "; //提示输入正整数(元数)
    cin>>n;                                //输入的正整数放 n 中
    cout<<n<<"(yuan) = ";
    for(int i=0; i<7; i++) //从大到小计算对每一面值的纸币所需的张数 count[i]
        if (n>=base[i]) //其中的 base[i]为面值数 100、50、20、10、5、2、1 之一
        {
            count[i]=n/base[i]; //所需纸币的张数放 count[i]
            n=n%base[i];        //甩掉大面值张数的钱数后所剩的“零头”
        }
    int total_sheet=0; //纸币总张数 total_sheet
    bool first=true;
    for(i=0; i<7; i++) //结果至多由七种面值的纸币构成
        if(count[i]) //当需要某面值的纸币时
        {
            total_sheet+=count[i]; //增加纸币总张数 total_sheet
            if (first) //显示第一“分解项”前不要加号“+”
            {
                cout<<count[i]<<"*"<<base[i];
                //count[i]中为每一面值纸币所需的张数
                first=false;
            }
            else //显示随后的各“分解项”时前面要加一个“+”
                cout<<" + "<<count[i]<<"*"<<base[i];
                //count[i]中为每一面值纸币所需的张数
        }
}
```

```
cout<<"(yuan)"<<endl;
cout<<"total_sheet="<<total_sheet<<endl; //显示出纸币总张数 total_sheet
}
```

[输出结果]

Input a positive integer: 196

196(yuan) = 1*100 + 1*50 + 2*20 + 1*5 + 1*1(yuan)

total_sheet=6

再一次执行:

Input a positive integer: 908

908(yuan) = 9*100 + 1*5 + 1*2 + 1*1(yuan)

total_sheet=12

4.15 输入 10 个整数而后反序输出的实现方式三

读下述程序,并上机进行验证,看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意,程序使用 `char` 型字符数组(即字符串)存放数据,而后从后往前逐一“拼装”出所需的 10 个整数并输出。

```
#include <iostream.h>
#include <stdio.h> //注 1
#include <string.h> //注 2
void main( )
{
    char s[80]; //注 3
    cout<<"Input 10 integers:"<<endl;
    gets(s); //注 4
    cout<<"---- The result ----"<<endl;
    int lastInd=strlen(s)-1; //注 5
    while (lastInd>=0) //注 6
    {
        int dat=0, weight=1; //注 7
        while(lastInd>=0 && s[lastInd]>='0' && s[lastInd]<='9') //注 8
        {
            dat+=(s[lastInd]-'0')*weight; //注 9
            lastInd--; //注 10
            weight*=10; //注 11
        }
    }
}
```

```

        if( lastInd>=0 )                                //注 12
        {
            char ch=s[lastInd];                        //注 13
            if( ch=='-' || ch=='+' )                    //注 14
            {
                lastInd--;
                if(ch=='-') dat*=-1;                    //注 15
            }
        }
        cout<<dat<<" ";                                //注 16
        if(lastInd<0)                                    //注 17
            break;
        while(lastInd>=0 && s[lastInd]!=' ')            //注 18
            lastInd--;
    }
    cout<<endl;
}

```

[分析提示]

(1) 说明一个分量为 `char` 型的字符数组，首先将数据(一个字符串)输入到字符数组中，而后从后往前逐一“拼装”出所需的 10 个整数并输出它们(注意，“拼装”过程是一个由 `char` 数字字符序列转化为 `int` 数据的过程)。

(2) 处理主体是一个二重循环，外层的“`while (lastInd>=0)`”循环负责处理完字符数组中的所有字符(从后往前处理)，而内层的“`while(lastInd>=0 && s[lastInd]>= '0' && s[lastInd]<= '9')`”则负责“拼装”起一个整数放入 `dat` 中。

[解答程序]

具体程序见题目处。

注 1: 要使用“`gets`”来读入字符串。

注 2: 要使用“`strlen`”。

注 3: 说明一个分量为 `char` 型的字符数组，用来存放读入的字符串。

注 4: 输入 10 个数据(一个字符串)放入字符数组 `s` 中。

注 5: `lastInd` 为当前被“拼装”数的最末一个数字字符的位置(下标值)。

注 6: 尚未“拼装”处理完所有字符时要继续处理。

注 7: “拼装”起一个整数放入 `dat` 中，`weight` 为位权(依次取值 1、10、100、...，表示个位、十位、百位、...)。

注 8: 将从 `lastInd` 往前的那“一段”数字字符“拼装”为一个整数放入 `dat` 中。

注 9: 注意 `s[lastInd]`—‘0’ 为数字字符对应的整数值。

注 10: 准备处理前一字符。

- 注 11: 前一数字字符的位权增大 10 倍。
注 12: 数字字符前带有其他字符(尚未“拼装”处理完所有字符时)。
注 13: 数前的那一字符。
注 14: 是否为正负号。
注 15: ch 为负号时 dat 取负值。
注 16: 将“拼装”好的 1 个整数 dat 显示到屏幕上。
注 17: 处理完所有字符时跳出循环结束处理。
注 18: 甩掉数据间的空格。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```

4.16 计算某一天是星期几(方式一)

由用户输入一个年份(如 2002)以及该年元旦是星期几(如 2), 而后再输入该年的任意一个月份日期(如输入 12 和 31, 表示 12 月 31 日), 由程序计算出这一天是星期几(如应输出 2, 表示 2002 年 12 月 31 日是星期二)。

注意, 闰年 2 月为 29 天, 非闰年 2 月为 28 天。可被 4 整除而不可被 100 整除的年份、或者可被 400 整除的年份都是闰年。

例如, 程序执行后, 可在屏幕上显示如下形式的结果:

```
input YEAR and what WEEKDAY of 1.1 (0~6) in that year:2002 2
input MONTH (1~12) and DAY (1~28|29|30|31) of that year:12 31
2002.12.31 => weekday: 2
```

[分析提示]

(1) 假设要计算 year 年 month 月 day 日那一天是星期几, 首先计算出 year 年的 1 到 month-1 那几个整月的累加天数, 而后再累加上 month 那一月(即当前月份)的不足一个整月的“零散”天数。并将该累加放到变量 DaysFrom11 中, 该数据为 month(月)day(日)那一天距离 1 月 1 日(即元旦)的总天数。

(2) 通过“ans=(WkDay11+DaysFrom11)%7;”来求出所需结果, 其中的 WkDay11 表示该年的元旦是星期几(其中用 0 表示星期日)。

[解答程序]

```
#include <iostream.h>
void main( )
```

```

{
    int year, month, day, WkDay11, DaysFrom11=0;
        //year、month、day -- 年、月、日;
        //WkDay11 -- 该年元旦是星期几;
        //DaysFrom11 -- 某月某日距离 1 月 1 日即元旦的天数, 初值为 0, 靠累加而得。
    int MthDays[12]={0,31,28,31,30,31,30,31,31,30,31,30};
        //MthDays[i]表示月份为 i 的那一个月的天数(若闰年的话, 2 月份要再增加 1 天)
    cout<<"input YEAR and what WEEKDAY of 1.1 (0~6) in that year:";
        //提示输入
    cin>>year>>WkDay11;    //输入年份(如 2001)以及该年元旦是星期几(如 1)
    cout<<"input MONTH (1~12) and DAY (1~28|29|30|31) of that year:";
        //提示输入
    cin>>month>>day;        //输入该年的任意一个月份日期(如输入 5 和 1,
        //表示 5 月 1 日)
    int i, ans;                //ans 放计算结果(星期几, 0 表示星期日)
    for(i=1; i<month; i++)    //累加 1 ~ month-1 月份的整月天数
        DaysFrom11+=MthDays[i];
    if(month>2)                //若月份大于 2 且该年为闰年时, 2 月份要再增加 1 天
        if( (year%4==0) && (year%100 !=0) || (year%400 ==0) )
            DaysFrom11+=1;
    DaysFrom11+=(day-1); //累加当前月份(不足一整月)的“零散”天数
    ans=(WkDay11+DaysFrom11)%7; //ans 为计算结果(星期几, 0 表示星期日)
    cout<<year<<". "<<month<<". "<<day<<" => weekday: "<<ans<<endl;
        //显示结果
}

```

[输出结果]

```

input YEAR and what WEEKDAY of 1.1 (0~6) in that year:2002 2
input MONTH (1~12) and DAY (1~28|29|30|31) of that year:12 31
2002.12.31 => weekday: 2

```

4.17 屏幕显示某年某指定月的月历(方式一)

由用户任意输入一个年份(如 2002)以及该年元旦是星期几(如 2), 而后再输入该年的任意一个月份(如输入 5, 表示 5 月), 由程序负责在屏幕上显示出这一个月月的月历。

例如, 程序执行后, 可在屏幕上显示如下形式的结果(其中输出的“ 2002.10.1 => weekday: 2”表示欲处理月的 1 号是星期几):

```

Input YEAR and what WEEKDAY of 1.1 (0~6) in that year:2002 2

```

Input MONTH (1~12) of that year:10

2002.10.1 => weekday: 2

```

-----
Sun Mon Tue Wed Thu Fri Sat
-----
          1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
-----

```

[分析提示]

(1) 按照上一范例的处理方法, 首先计算出欲处理月的 1 号是星期几(本程序还一并在屏幕上显示出该信息)。

(2) 按照题目处设计的格式将欲处理月的各日期数显示在屏幕上(也即为那一月的月历)。注意, 每处理一天, “星期数”要长 1, 当处理到“周末”时, 下一周的日期数要换行显示, 且“星期数”要从头来。

[解答程序]

```

#include <iostream.h>
#include <iomanip.h>
void main( )
{
    int year, month, WkDay11, DaysFrom11=0;
        //year、month -- 年、月;
        //WkDay11 -- 该年元旦是星期几;
        //DaysFrom11 -- 某月 1 日距离 1 月 1 日即元旦的天数, 初值为 0,
        //靠累加而得。
    int MthDays[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
        //MthDays[i]表示月份为 i 的那一个月的天数(若闰年的话, 2 月份要再增加 1 天)
    cout<<"Input YEAR and what WEEKDAY of 1.1 (0~6) in that year:";
        //提示输入
    cin>>year>>WkDay11;    //输入年份(如 2002)以及该年元旦是星期几(如 2)
    cout<<"Input MONTH (1~12) of that year:";    //提示输入
    cin>>month;    //输入该年的任意一个月份(如输入 5, 表示 5 月)
    int i, ans;    //ans 为计算结果(星期几, 0 表示星期日)
    for(i=1; i<month; i++)    //累加 1 -> month-1 月份的整月天数
        DaysFrom11+=MthDays[i];
}

```



```

if(month>2)                //若月份大于 2 且该年为闰年时, 2 月份要再增加 1 天
    if( (year%4==0) && (year%100 !=0) || (year%400 ==0) )
        DaysFrom11+=1;
ans=(WkDay11+DaysFrom11)%7; //ans 为欲处理月的 1 号是星期几(0 表示星期日)
cout<<year<<". "<<month<<". "<<1<<" => weekday: "<<ans<<endl;
                                //显示欲处理月的 1 号是星期几

cout<<"-----"<<endl;
cout<<" Sun Mon Tue Wed Thu Fri Sat"<<endl;
cout<<"-----"<<endl;
int wday=ans;                //wday 现在为欲处理月的 1 号是星期几
cout<<setw(4*(wday+1))<<1;    //将日期数“ 1” 显示在适当位置
for (i=2; i<=MthDays[month]; i++) //显示那一月的其他日期数 i(i=2,3,4,...)
{
    wday++;                //每处理一天,“ 星期数” 长 1
    if (wday % 7==0)        //处理到“ 周末” 时
    {
        cout<<endl;        //下一周的日期数要换行显示
        wday=0;            //“ 星期数” 从头来
    }
    cout<<setw(4)<<i;        //在适当位置显示出日期数 i
}
cout<<endl;
cout<<"-----"<<endl;
}

```

[输出结果]

Input YEAR and what WEEKDAY of 1.1 (0~6) in that year:2002 2

Input MONTH (1~12) of that year:12

2002.12.1 => weekday: 0

```

-----
Sun  Mon Tue Wed Thu Fri Sat
-----
  1   2   3   4   5   6   7
  8   9  10  11  12  13  14
15  16  17  18  19  20  21
22  23  24  25  26  27  28
29  30  31
-----

```

再一次执行:

Input YEAR and what WEEKDAY of 1.1 (0~6) in that year:2003 3

Input MONTH (1~12) of that year:2

2003.2.1 => weekday: 6

Sun	Mon	Tue	Wed	Thu	Fri	Sat
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	

4.18 求取正整数“圈”中连续四数之和最大者

由 15 个正整数排成一圈，编程序找出连续的四个数，它们的和是最大的(不小于圈中任何其他连续四数之和)。

[分析提示]

(1) 说明一个可存放 15 个 int 型数据的数组 a，而后通过 cin 从键盘输入任意 15 个正整数放入 a 数组中。

(2) 通过“for(i=0; i<15; i++)”的循环，从如下的 15 个和数中(每一和数由 4 个连续的 a[i]相加而得)找出其中的最大者: 第 i 个和数为 a[t1]+a[t2]+a[t3]+a[t4]，其中的 t1=i%15; t2=(i+1)%15; t3=(i+2)%15; t4=(i+3)%15。t1、t2、t3、t4 即为四连数在圈中的位置(数组下标值)。由于将数组看成是“圈状”的，最后一个数的后继应是第一个数，所以要以 15 取模。

[解答程序]

```
#include <iostream.h>

void main()
{
    const int n=15;
    int i, a[n];           //用于存放 n 个正整数的数组 a
    cout<<"Input "<<n<<" positive integers:"<<endl;      //提示输入
    for(i=0; i<n; i++)    //输入 n 个正整数放入 a 数组中
        cin>>a[i];
    int max4=0, s4, start, t1, t2, t3, t4;
```

```

//max4 记录圈中当前已“查过”的四连数的最大值
//s4 为某一个四连数的累加和, start 记录该四连数的“起点”(数组下标值)
//t1、t2、t3、t4 为四连数在圈中的位置(数组下标值)
for(i=0; i<n; i++)
{
    t1=i%n; //由于将数组看成是“圈状”的, 最后一个数的后继应是第一个数,
            //所以要以 n 取模
    t2=(i+1)%n;    t3=(i+2)%n;    t4=(i+3)%n;
    if( (s4=a[t1]+a[t2]+a[t3]+a[t4])>max4 ) //当前四连数累加和大于 max4
    {
        max4=s4; //记录新的 max4
        start=i; //记录此四连数的“起点”(数组下标值)
    }
}
t1=start; //具有最大值的四连数在圈中的位置 t1、t2、t3、t4
t2=(start+1)%n;    t3=(start+2)%n;    t4=(start+3)%n;
cout<<"max4=>(\ ' "<t1<<"\ ' +\ ' "<t2<<"\ ' +\ ' "<t3<<"\ ' +\ ' "<t4<<"\ ' ): ";
//输出结果
cout<<a[t1]<<"+"<a[t2]<<"+"<a[t3]<<"+"<a[t4]<<"="<<max4<<endl;
}

```

[输出结果]

Input 15 positive integers:

88 99 1 2 3 4 5 6 7 8 9 10 55 66 1

max4=>(\ ' 13' +\ ' 14' +\ ' 0' +\ ' 1'): 66+1+88+99=254

再一次执行:

Input 15 positive integers:

1 3 5 7 25 2 76 1 1 80 30 18 8 33 22

max4=>(\ ' 6' +\ ' 7' +\ ' 8' +\ ' 9'): 76+1+1+80=158

4.19 将正整数转换为“英文表示字符串”

由用户输入任意一个正整数 num(如 32 760), 由程序负责将该正整数转换为如下形式的“英文表示字符串”并显示出来: three-two-seven-six-zero。又如, 当输入 14 895 时, 相应的输出应为: one-four-eight-nine-five。

[分析提示]

(1) 说明如下一个二维数组 `eng`, 元素值为“英文表示字符串”中所需的各“数字”:
`char eng[10][6]={“ zero”, “ one”, “ two”, “ three”, “ four”, “ five”, “ six”, “ seven”, “ eight”, “ nine” };`

(2) 对用户输入的那一任意正整数 `num` 进行分解——将 `num` 的各位数字截取下来先放入 `int` 型的数组 `a` 中, 而后通过使用“ `eng[a[i]]`”将 `a` 中各位数字对应的处于 `eng` 数组中的英文数字显示出来。

[解答程序]

```
#include <iostream.h>

void main( )
{
    int num, a[20], ind=0;
    //正整数 num; a 数组存放分解出来的 num 的各位数字(先个位、再十位...)
    //ind 为当前可用来存放数据的 a 数组的下标位置
    char eng[10][6]
    //二维数组 eng, 元素值为“英文表示字符串”中所需的各“数字”
        ={"zero", "one", "two", "three", "four", "five", "six", "seven", "eight", "nine"};
    cout<<"Input a positive integer:";
    cin>>num;        //用户输入任意一个正整数 num
    while (num)      //“分解” num 的各位数字(先个位、再十位、..., 依次放入数组 a 中)
    {
        a[ind++]=num%10;
        //数组 a 存放分解出的各位数字, 放于 ind 下标位置处
        num/=10;
    }
    //倒着处理 a 数组中的各位数字, 也即将它们对应的各英文“数字”显示出来
    cout<<"Eng_exp: "<<eng[a[ind-1]];
    for(int i=ind-2; i>=0; i--)
        cout<<"-"<<eng[a[i]];
    cout<<endl;
}
```

[输出结果]

Input a positive integer:32760

Eng_exp: three-two-seven-six-zero

再一次执行:

Input a positive integer:14895

Eng_exp: one-four-eight-nine-five

4.20 二维整型数组按行按列求出各累加和

对二维整型数组(m 行 n 列)按行按列求出各累加和(共求出 m 个行累加和以及 n 个列累加和)并输出这些累加结果。

[分析提示]

(1) 对二维数组 A 的第 k 行进行累加时, 可按如下的方法实现:

```
s=0;                                //累加和放 s
for(j=0; j<n; j++)                  //每行 n 个元素(共 n 列)
    s += A[k][j];                   //累加到 s 上, 第一下标 k 表示行
```

(2) 对二维数组 A 的第 k 列进行累加时, 可按如下的方法实现:

```
s=0;                                //累加和放 s
for(i=0; i<m; i++)                  //每列 m 个元素(共 m 行)
    s += A[i][k];                   //累加到 s 上, 第二下标 k 表示列
```

[解答程序]

```
#include <iostream.h>
void main(void)
{
    const int m=3;
    const int n=4;
    int i, j, s;
    int A[m][n] = { {1,2,3,4}, {5,6,7,8}, {3,6,9,0} }; //定义并初始化数组
    for (i=0; i<m; i++)                                //输出数组元素, 共 m 行
    {
        for (j=0; j<n; j++)                            //每行 n 个元素(共 n 列)
            cout<<A[i][j]<<" ";
        cout << endl;
    }
    cout<<"-----"<<endl;
    //计算二维数组 A 的每行元素之和并输出
    for (i=0; i<m; i++)                                //数组共 m 行
    {
        s=0;                                            //累加和放 s
        for(j=0; j<n; j++)                            //每行 n 个元素(共 n 列)
            s += A[i][j];                             //累加到 s 上
```

```

        cout<<"Sum of row "<<i<<" is "<<s<< endl; //输出该行的累加和
    }
    cout<<"-----"<<endl;
    //计算二维数组 A 的每列元素之和并输出
    for (j=0; j<n; j++)                //数组共 n 列
    {
        s=0;                            //累加和放 s
        for(i=0; i<m; i++)              //每列 m 个元素(共 m 行)
            s += A[i][j];               //累加到 s 上
        cout<<"Sum of col "<<j<<" is "<<s<< endl; //输出该列的累加和
    }
}

```

[输出结果]

```

1  2  3  4
5  6  7  8
3  6  9  0

```

Sum of row 0 is 10

Sum of row 1 is 26

Sum of row 2 is 18

Sum of col 0 is 9

Sum of col 1 is 14

Sum of col 2 is 19

Sum of col 3 is 12

4.21 二维数组的简单应用

设有 4 行 4 列的数组 b , 其元素 $b[i][j]=i+j+1$ ($i, j=0, 1, 2, 3$)。编程序, 实现:

- (1) 求第二行 4 元素之和;
- (2) 求第三列 4 元素的平均值;
- (3) 求最大数, 最小数及主对角线 4 元素的平方和。

[分析提示]

- (1) 第二行的 4 个元素为 $b[1][j]$ ($j=0, 1, 2, 3$)。
- (2) 第三列的 4 个元素为 $b[i][2]$ ($i=0, 1, 2, 3$)。
- (3) 对角线元素的第一下标 i (行下标)等于其第二下标 j (列下标)。

[解答程序]

```

#include <iostream.h>

void main( )
{
    int b[4][4], i, j; //说明了一个 int 型 4 行 4 列的二维数组 b, 及整型变量 i, j
    //按公式“ b[i][j]=i+j+1 ”为 b 数组各元素赋值, 并将 b 数组的这些初值显示在屏幕上
    for (i=0; i<4; i++) //i 为数组的行下标——从 0 变化到 3
    {
        for (j=0; j<4; j++) //j 为数组的列下标——从 0 变化到 3
        {
            b[i][j]=i+j+1; //按公式“b[i][j]=i+j+1”为 b 数组各元素赋值
            cout<<" "<<b[i][j]; //将 b 数组的这些初值显示在屏幕上
        }
        cout<<endl; //每 4 数占一行
    }
    cout<<"-- The result --"<<endl;
    //1. 求第二行 4 元素之和
    int sum_lin2=0; //sum_lin2 中放累加和
    for (j=0; j<4; j++) //循环 4 次, 求出 4 元素之和
        sum_lin2+=b[1][j]; //第二行元素为 b[1][j] (j=0, 1, 2, 3)
    cout<<"sum_lin2="<<sum_lin2<<endl; //输出所求累加和
    //2. 求第三列 4 元素之平均值:
    int sum_col3=0; //sum_col3 中放累加和
    for (i=0; i<4; i++) //循环 4 次, 求出 4 元素之和
        sum_col3+=b[i][2]; //第三列元素为 b[i][2] (i=0, 1, 2, 3)
    cout<<"ave_col3="<<sum_col3/4.0<<endl;
    //sum_col3/4.0 为平均值。若将 4.0 改为 4 则只能求出平均值的整数部分
    //3. 求最大数, 最小数及主对角线 4 元素的平方和
    int max_elem=b[0][0], min_elem=b[0][0], sum_diag=0; //说明 3 个变量并置初值
    //先认为 b[0][0] 为目前发现的最大数 max_elem, 又为最小数 min_elem
    for (i=0; i<4; i++) //i 为数组的行下标——从 0 变化到 3
        for (j=0; j<4; j++) //j 为数组的列下标——从 0 变化到 3
        {
            if ( b[i][j]>max_elem ) max_elem=b[i][j];
            //若发现 b[i][j] 比当前最大数 max_elem 更大,
            //则将 max_elem 修改为 b[i][j] 之值
            if ( b[i][j]<min_elem ) min_elem=b[i][j];
            //若发现 b[i][j] 比当前最小数 min_elem 更小,
            //则将 min_elem 修改为 b[i][j] 之值
        }
}

```

```

        if ( i==j ) sum_diag+=b[i][j]*b[i][j];
        //当行下标 i 等于列下标 j 时, b[i][j]为主对角线元素,
        //往 sum_diag 上累加其平方和
    }

    cout<<"max_elem="<<max_elem<<endl;    //输出所求最大数
    cout<<"min_elem="<<min_elem<<endl;    //输出所求最小数
    cout<<"sum_diag="<<sum_diag<<endl;    //输出所求主对角线 4 元素的平方和
}

```

[输出结果]

```

1  2  3  4
2  3  4  5
3  4  5  6
4  5  6  7
-- The result --
sum_lin2=14
ave_col3=4.5
max_elem=7
min_elem=1
sum_diag=84

```

4.22 二矩阵相加

说明三个行列数完全相同的二维数组用于表示矩阵:

```
int A[m][n], B[m][n], C[m][n];
```

给 A、B 二矩阵赋初值, 而后计算 $C=A+B$ (矩阵加):

[分析提示]

可用二重循环来实现矩阵加, 也就是说, 可按如下公式依次求出各 $C[i][j]$:
 $C[i][j]=A[i][j]+B[i][j]$; ($i=0, 1, \dots, m-1$; $j=0, 1, \dots, n-1$)

[解答程序]

```

#include <iostream.h>

void main(void)
{
    const int m=3;
    const int n=4;
    int A[m][n] = { { 1,2,3,4}, { 5,6,7,8}, { 0,2,4,6} };

```



```

//定义并初始化数组, 表示矩阵 A
int B[m][n] = { {1,0,2,1}, {2,1,1,0}, {3,6,5,2} };
//定义并初始化数组, 表示矩阵 B

int i, j, C[m][n];
cout<<"---- A ----"<<endl;
for (i=0; i<m; i++) //输出数组 A 的元素, 共 m 行
{
    for (j=0; j<n; j++) //每行 n 个元素(共 n 列)
        cout<<A[i][j]<<" ";
    cout << endl;
}
cout<<"---- B ----"<<endl;
for (i=0; i<m; i++) //输出数组 B 的元素, 共 m 行
{
    for (j=0; j<n; j++) //每行 n 个元素(共 n 列)
        cout<<B[i][j]<<" ";
    cout << endl;
}
//计算 C=A+B
for (i=0; i<m; i++) //数组共 m 行
    for(j=0; j<n; j++) //每行 n 个元素(共 n 列)
        C[i][j]=A[i][j]+B[i][j]; //计算出每一个 C[i][j]
cout<<"-- C=A+B ---"<<endl;
for (i=0; i<m; i++) //输出 C 数组元素, 共 m 行
{
    for (j=0; j<n; j++) //每行 n 个元素(共 n 列)
        cout<<C[i][j]<<" ";
    cout << endl;
}
}

```

[输出结果]

```

----- A -----
1  2  3  4
5  6  7  8
0  2  4  6
----- B -----
1  0  2  1
2  1  1  0

```

```

3  6  5  2
-- C=A+B ---
2  2  5  5
7  7  8  8
3  8  9  8

```

4.23 处理二维字符数组

设有二维字符数组 `dig[10][10]`，其元素值均为数字字符(“0”、“1”，...，“9”)，编程序，首先为 `dig` 数组设置初值，而后统计出其中 10 个数字字符各自出现的次数。

[分析提示]

(1) 可通过多次使用表达式“`rand() % 10 + 48`”来为数组 `dig` 生成其各元素值(均为“0”~“9”字符间的随机数字字符值)。标准库函数“`rand()`”返回 0~32767 间的一个 `int` 型随机值，则“`rand() % 10`”将是 0~9 间的一个随机值，从而“`rand() % 10 + 48`”为一个“0”~“9”字符间的随机数字字符值。

(2) 说明具有 10 个元素的 `int` 型数组 `sum`，使用 `sum[0]` 记录“0”字符出现的次数，`sum[1]` 记录“1”字符出现的次数，...，`sum[9]` 记录“9”字符出现的次数。

[解答程序]

```

#include <iostream.h>
#include <stdlib.h>
#include <time.h>
void main( )
{
    const int m=10;
    const int n=10;
    char dig[m][n];          //二维字符数组 dig[10][10]
    int i, j, tmp, sum[m];
    //sum[0]记录“0”字符出现的次数，sum[1]记录“1”字符出现的次数，...
    srand((unsigned)time(NULL));
    //srand 设置随机数产生器所需的“种子值”，
    //time 函数返回一个随机时间作为它的实参
    for(i=0; i<m; i++)        //为二维字符数组 dig 生成随机数字字符值
        for(j=0; j<n; j++)
        {
            tmp=rand() % 10;
            //rand()返回 0~32767 间的一个随机值，tmp 将是 0~9 间的一个随机值

```

```

        dig[i][j]=48+tmp;
        //48+tmp 将成为“ 0” ~“ 9” 字符间的一个随机数字字符值
    }
    for(i=0; i<m; i++)        //输出随机生成的 dig 数组
    {
        for(j=0; j<n; j++)
            cout<<dig[i][j]<<" ";
        cout<<endl;
    }
    for(i=0; i<m; i++)
        sum[i]=0;            //作为累加和的各 sum[i]均先置为 0
    for(i=0; i<m; i++)
        for(j=0; j<n; j++)
            sum[dig[i][j]-48]++;
        //按 dig[i][j]的字符大小，增值相应的 sum 分量(作累加)注意，
        //sum[0]记录“ 0” 字符出现的次数，sum[1]记录“ 1” 字符出现的次数， ...
    for(i=0; i<m; i++)
        cout<<"sum["<<i<<"]="<<sum[i]<<endl; //输出统计结果
}

```

[输出结果]

```

1  3  4  4  5  4  5  2  4  6
6  9  8  5  6  9  0  8  8  6
2  9  0  1  8  1  8  4  2  2
0  8  3  3  5  8  5  8  5  0
5  5  6  4  7  2  1  9  1  2
8  7  7  4  7  9  3  9  9  4
7  2  2  9  2  8  6  4  1  0
8  8  1  6  1  0  5  2  7  0
2  3  2  2  4  2  0  0  4  9
9  7  2  7  7  6  0  5  4  4

sum[0]=10
sum[1]=8
sum[2]=15
sum[3]=5
sum[4]=13
sum[5]=10
sum[6]=8
sum[7]=9

```

```
sum[8]=12
```

```
sum[9]=10
```

4.24 统计各行“ 单词” 个数的处理方案一

从键盘输入 n 行字符(如, n 个英文句子), 每行不超过 80 个字符, 以回车结束每一行。统计出各行所具有的“ 单词” 个数(“ 单词” 被定义为以字母开头的连续字母串)。

[分析提示]

(1) 本处理方案先读入 n 行字符存放在一个二维字符数组中, 而后逐一处理它们。

(2) 数据部分概要:

```
const int n=3;    //定义常量 n, 共输入并处理 n 行字符
char s[n][81];    //s 为 char 型二维数组, 用于存放输入的 n 行字符
                  //(每行最多放 80 字符另加一结束符)
int count[n];     //使用 count 数组元素记录各行所具有的单词个数, 对应关系为:
                  //将 s[i]那一行所具有的单词个数放于 count[i]之中
```

(3) 处理部分概述:

1) 输入 n 行字符(每行以回车为结束), 依次放于一维字符数组 $s[i]$ 中(注意, $s[i]$ 为二维字符数组 s 的一行), 并将记录各行单词个数的 $count[i]$ 均置为 0(作累加单元用);

2) 依次处理已放在 $s[0]$ 、 $s[1]$ 、...、 $s[n-1]$ 中的那 n 行字符, 将统计的各行单词个数放于 $count[i]$ 之中;

3) 输出统计出的各行所具有的“ 单词” 个数(即各 $count[i]$ 的值)。

[解答程序]

```
#include <iostream.h>
#include <stdio.h>
void main( )
{
    const int n=3;        //定义常量 n, 共输入并处理 n 行字符
    char s[n][81], ch;
                           //s 为 char 型二维数组, 用于存放输入的 n 行字符
                           //(每行最多放 80 字符另加一结束符)

    int count[n], i, j;
                           //使用 count 数组元素记录各行所具有的单词个数
                           //对应关系为: 将 s[i]那一行所具有的单词个数放于 count[i]中

    cout<<"----- Input "<<n<<" strings -----"<<endl;
                           //提示进行输入

    for(i=0; i<n; i++)    //共输入 n 行
```

```

    {
        gets(s[i]);          //等待用户从键盘输入 1 行字符(以回车为结束)
                             //输入字符由系统自动放于一维数组 s[i]中, 且以“ \0” 作为结束符号
        count[i]=0;          //将记录单词个数的 count[i]先置为 0
    }
    for(i=0; i<n; i++)
        //共处理 n 行字符(这些行现已在 s[0], s[1], ..., s[n-1]中)
        int word_begin=0;    //word_begin 为一个标记, 现将它置为“ 单词尚未开始”
        for (j=0; j<81 && s[i][j]!='\0'; j++)
            //从头到尾处理一遍 s[i]数组中的各字符
            ch=s[i][j]; //取出 s[i]数组中的第 j 个字符放 ch 中
            if ( ch>='a' && ch<='z' || ch>='A' && ch<='Z')
                //若 ch 为大小写字母
            {
                if (word_begin==0) //如果目前的标记仍为“ 单词尚未开始”
                {
                    word_begin=1; //将标记改为“ 单词已开始”
                    count[i]++;    //累计第 i 行单词个数的下标变量加 1
                }
            }
            else //若 ch 非字母时, 将标记置为“ 单词尚未开始”
                word_begin=0;
        }
    }
    /*
        注意, 上述二重循环中的内 if 语句处, 如果“ word_begin==0” 条件不满足, 也即目前的
        标记已不为“ 单词尚未开始” 时, 则什么事也不做, 此时内嵌的 if 句结束, 导致外层的
        if-else 语句结束, 导致作为 for-j 循环体的复合语句结束, 从而应转到 for-j 头的“ 表达式 3”
        也即 j++处去执行, 进而去执行 for-j 的下一循环。
    */
    cout<<"----- The result -----"<<endl;
    for(i=0; i<n; i++) //输出统计出的各行所具有的“ 单词” 个数(共 n 行)
        cout<<"count["<<i<<"]="<<count[i]<<endl;
}

```

[输出结果]

```

----- Input 3 strings -----
Hello world!
C++ OK! 123,abc.

```

```
0 11 22 22 11 0 + - * /
----- The result -----
count[0]=2
count[1]=3
count[2]=0
```

4.25 统计各行“单词”个数的处理方案二

从键盘输入 n 行字符(如, n 个英文句子), 每行不超过 80 个符号, 以回车结束每一行。统计出各行所具有的“单词”个数(“单词”被定义为以字母开头的连续字母串)。

[分析提示]

(1) 本处理方案的程序主体为一个执行 n 次的循环, 每次循环读入 1 行字符放在同一个一维字符数组中——下一次覆盖上一次内容, 而后立刻处理该行字符。

(2) 数据部分概要:

```
const int n=3;      //定义常量 n, 共输入并处理 n 行字符
char s[81];         //s 为 char 型一维数组, 用于存放输入的 1 行字符
                    //每行最多放 80 字符另加一个结束符
int count;          //使用 count 记录该行所具有的单词个数
```

(3) 处理部分概述:

循环 n 次, 每次均做如下的事情:

1) 输入 1 行字符(以回车为结束), 放于一维字符数组 s 中, 并将记录该行单词个数的 $count$ 置为 0 (作累加单元用);

2) 处理已放在 s 中的那 1 行字符, 统计其中的单词个数放于 $count$ 之中;

3) 输出统计出的该行所具有的单词个数(即各 $count$ 的值)。

[解答程序]

```
#include <iostream.h>
#include <stdio.h>

void main( )
{
    const int n=3;          //定义常量 n, 共输入并处理 n 行字符
    char s[81], ch;
        //s 为 char 型一维数组, 用于存放输入的一行字符一行最多放 80 字符
        //另加一结束符

    int count, i, j;        //使用 count 记录该行所具有的单词个数
    for(i= 0; i<n; i++)     //共处理 n 行字符(每次循环输入并处理 1 行)
    {
```

```

cout<<"----- Input a string -----"<<endl;    //提示进行输入
gets(s);          //等待用户从键盘输入 1 行字符(以回车为结束)
//输入的各字符由系统自动放于一维数组 s 中, 且以“ \0” 作为结束符号
count=0;          //将记录单词个数的 count 先置为 0
int word_begin=0;  //word_begin 为一个标记, 现将它置为“ 单词尚未开始”
for (j=0; j<81 && s[j]!='\0'; j++)
{
    //从头到尾处理一遍 s 数组中的各字符
    ch=s[j];      //取出 s 数组中的第 j 个字符放 ch 中
    if ( ch>='a' && ch<='z' || ch>='A' && ch<='Z')    //若 ch 为大小写字母
    {
        if (word_begin==0)//如果目前的标记仍为“ 单词尚未开始”
        {
            word_begin=1; //将标记改为“ 单词已开始”
            count++;      //累计单词个数的变量 count 加 1
        }
    }
    else          //若 ch 非字母时, 将标记置为“ 单词尚未开始”
        word_begin=0;
}

/*
    注意, 上述二重循环中的内 if 语句处, 如果“ word_begin==0” 条件不满足,
    也即目前的标记已不为“ 单词尚未开始” 时, 则什么事也不做, 此时内嵌的 if
    语句结束, 导致外层的 if-else 语句结束, 导致作为 for-j 循环体的复合语句结束,
    从而应转到 for-j 头的“ 表达式 3” 也即 j++处去执行, 进而去执行 for-j 的下一
    次循环。
*/

cout<<"count="<<count<<endl;    //输出统计出的该行所具有的单词个数
}
}

```

[输出结果]

```

----- Input a string -----
Hello world!
count=2
----- Input a string -----
C++ OK! 123,abc.
count=3
----- Input a string -----

```

```
0 11 22 22 11 0 + - */
count=0
```

4.26 二矩阵相乘

说明三个行列数有关系的二维数组用于表示矩阵 A、B、C (假设要计算 $C=A*B$, 即矩阵相乘):

```
int A[m][n], B[n][t], C[m][t]; //矩阵 A 的列数要等于矩阵 B 的行数
给 A、B 二矩阵赋初值, 而后计算出乘积矩阵 C 并输出结果。
```

[分析提示]

可按如下公式来计算乘积矩阵 C 的每一个元素 $C[i][j]$

(其中 $i=0, 1, \dots, m-1$; $j=0, 1, \dots, t-1$):

```
{
    s=0; //矩阵 A 的第 i 行与矩阵 B 的第 j 列诸元素相乘后累加到 s 上
    s+=A[i][k]*B[k][j];           //(k=0, 1, ..., n-1)
    C[i][j]=s;                     //s 值即为 C[i][j]
}
```

[解答程序]

```
#include <iostream.h>

void main(void)
{
    const int m=3;
    const int n=4;
    const int t=3;
    int A[m][n] = { {1,2,3,4}, {5,6,7,8}, {0,2,4,6} };
                                     //定义并初始化数组, 表示矩阵 A
    int B[n][t] = { {1,0,2}, {2,1,1}, {5,3,1}, {3,6,5} };
                                     //定义并初始化数组, 表示矩阵 B

    int i, j, k, C[m][t];
    cout<<"---- A ----"<<endl;
    for (i=0; i<m; i++)              //输出数组 A 元素, 共 m 行
    {
        for (j=0; j<n; j++)          //每行 n 个元素(共 n 列)
            cout<<A[i][j]<<" ";
        cout<<endl;
    }
```



```

cout<<"---- B ----"<<endl;
for (i=0; i<n; i++)           //输出数组 B 元素, 共 n 行
{
    for (j=0; j<t; j++)        //每行 t 个元素(共 t 列)
        cout<<B[i][j]<<" ";
    cout << endl;
}
//计算 C=A*B(即矩阵乘)
for (i=0; i<m; i++)           //数组 C 共 m 行
    for(j=0; j<t; j++)        //每行 t 个元素(共 t 列)
    {
        int s=0;    //矩阵 A 的第 i 行与矩阵 B 的第 j 列诸元素相乘后累加到 s 上
        for(k=0; k<n; k++)    //共 n 对元素相乘
            s+=A[i][k]*B[k][j]; //累加到 s 上
        C[i][j]=s;           //s 值即为 C[i][j]
    }
cout<<"-- C=A*B ---"<<endl;
for (i=0; i<m; i++)           //输出 C 数组元素, 共 m 行
{
    for (j=0; j<t; j++)        //每行 t 个元素(共 t 列)
        cout<<C[i][j]<<" ";
    cout << endl;
}
}

```

[输出结果]

```

----- A -----
1  2  3  4
5  6  7  8
0  2  4  6
----- B -----
1  0  2
2  1  1
5  3  1
3  6  5
-- C=A*B ---
32  35  27
76  75  63
42  50  36

```

4.27 在屏幕上“画出”最近 15 天的气温值柱形图

设计一个程序,它可以把最近 15 天的气温值以柱形图的形式显示出来,假设气温在 22 到 35 度之间(取整),温度的高低以“*”组成的“柱”的高低来表示。

[分析提示]

在“文本模式”的“字符屏幕”上“画图”的通常实现方法如下:

1)将“字符屏幕”与程序中的一个二维字符数组建立对应关系。如,本程序的 `scArray` 数组就对应于欲显示的“字符屏幕”,其中的 `scArray[0][0]`表示“字符屏幕”的左上角点,而 `scArray[y][x]`则表示“字符屏幕”的第 $y+1$ 行第 $x+1$ 列的那一个点(y 值即行号由上往下扩展, x 值即列号由左往右扩展);

2)将二维字符数组的各元素均置为“空”(将数组各元素均赋值为空格,使之对应于一个“空白字符屏幕”);

3)按某种计算方法(或计算公式)算出应该在“字符屏幕”的哪些位置处“画点”(通过往对应字符数组的某些元素处置“*”符号来实现,使用一系列的“*”构成气温值柱形图);

4)将已准备好的当前字符数组 `scArray` 各元素的值显示到“字符屏幕”上(在“字符屏幕”上“画”出了所需图形)。

[解答程序]

```
#include <iomanip.h>
#include <iostream.h>
void main()
{
    const int wid_Per_Star=3;
        //每一个“*”所占的宽度(也即“柱”与“柱”之间的距离为该值减 1)
    const int Do_Per_Star=2;
        //柱形图中的一个“*”所代表的度数(用于控制柱形图的高度)
    const int Do_Start=22;
        //柱形图中温度的起始值(用于控制柱形图所能表示的温度范围)
    const int length=80;           //“字符屏幕”每行的字符数
    const int height=7;            //“字符屏幕”具有的行数(柱形图的最大高度)
    char scArray[height][length]; //表示“字符屏幕”的二维字符数组
    const int day=15;              //要显示多少天的气温柱形图
    int temperature[day];          //存放温度值的 temperature 数组
    cout<<"---- Please input "<<day<<" day's temperature (22~35) ----"<<endl;
    for(int i=0; i<day; i++)       //输入共 day 天的温度值放入 temperature 数组
        cin>>temperature[i];
```

```

//将“ 字符屏幕” 置为“ 空” (将对应二维字符数组 scArray 的各元素均赋值为空格)
for(i=0; i<height; i++)
    for(int j=0; j<length; j++)
        scArray[i][j]=' ';
//处理输入的共 day 天的温度值(各 temperature[i]之值)
//往表示“ 字符屏幕” 的二维字符数组 scArray 的相应位置添加“ *”
for(i=0; i<day; i++)
{
    int star_num = (temperature[i]-Do_Start)/Do_Per_Star;
    //temperature[i]的温度值需要用多少个“ *” 来“ 画” 出
    int x, ystart, yend;
    x=wid_Per_Star*i;
    //在第 wid_Per_Star*i 的那一列处, 共“ 画” 出 star_num 个连续的“ *”
    ystart=height-star_num;
    //从 height-star_num 行开始, 一直“ 画” 到 height-1 行
    yend=height-1;
    //在以屏幕左上角为(0,0)点的坐标系下, 往(x, ystart), (x, ystart+1), ...
    //(x, yend)这些点处各“ 画” 一个“ *” (也即在某一列中连续“ 画” 若干个“ *” )
    for(int y=ystart; y<=yend; y++) scArray[y][x]= '*';
    //在以屏幕左上角为(0,0)点的坐标系下, 往(x,y)点处“ 画” 一个“ *”
    //注意, 上述(x,y)点相当于“ 字符屏幕” 二维字符数组 scArray 的 y 行 x 列位置
}
//显示出对应于“ 字符屏幕” 的二维字符数组 scArray 各元素的值
for(i=0; i<height; i++)          //“ 字符屏幕” 共有 height 行
{
    cout.width(wid_Per_Star);
    //每一个“ *” 所占的宽度为 wid_Per_Star(也即“ 柱” 与“ 柱” 之间的距离为该值减 1)
    for(int j=0; j<length; j++) //“ 字符屏幕” 每行具有 length 列
        cout<<scArray[i][j];
    cout<<endl;
}
cout<<"-----"<<endl;
for(i=0; i<day; i++)    //将每天的温度值数据也一并显示在柱形图的下方
    cout<<setw(wid_Per_Star)<<temperature[i];
cout<<endl<<"-----"<<endl;
}

```

[输出结果]

---- Please input 15 day's temperature (22--35) ----

22 24 26 29 27 33 31 35 30 28 29 33 34 30 29

```

                *               *

            *       *           *   *

        *   *   *   *           *   *   *

    *       *   *   *   *   *   *   *   *   *   *

*   *   *   *   *   *   *   *   *   *   *   *

*   *   *   *   *   *   *   *   *   *   *   *

```

 22 24 26 29 27 33 31 35 30 28 29 33 34 30 29

4.28 统计每一正整数的出现频率

任意输入 n 个正整数，求出其中互不相同的数的个数 num ，以及此 num 个互不相同的数在原有 n 个数中各自出现的频率(即出现次数)，并将结果显示出来。

[分析提示]

(1) 说明两个数组： $\text{int } a[n], b[n][2]$;

a 数组用于存放刚输入的 n 个正整数， b 数组具有 2 列，第 1 列存放那些互不相同的数本身，第 2 列存放数的出现频率(次数)。

(2) 可通过从头到尾遍历一遍 a 数组的各元素，来形成 b 数组(具有 2 列的表)中的各元素值。具体地说，若元素 $a[i]$ 非 0 (为互不相同的某一个数) 时，除了将 $a[i]$ “登记”到 b 数组中之外，还要将 b 数组中记录该数出现频率的变量值加 1。另外，要将 a 数组中后续的与 $a[i]$ 相同的各数均“消掉”(赋值为 0，因为有前提“输入的均为正整数”)。

[解答程序]

```

#include <iomanip.h>

void main(void)
{
    const int n=10;

    int a[n], b[n][2], i, num=0; //输入的 n 个正整数先放于数组 a 中
    //数组 b 的第 1 列存放那些互不相同的数，第 2 列存放数的出现频率(次数)

```

```

cout<<"Input "<<n<<" positive INTEGERS:"<<endl;           //提示输入
for(i=0; i<n; i++)           //输入 n 个正整数，放于数组 a 中
{
    cin>>a[i];
    b[i][1]=0;               //各数的出现频率(次数)初值为 0
}
for(i=0; i<n; i++)
    //通过从头到尾遍历一遍数组 a 的各元素，
    //来形成 b 数组(具有 2 列的表)中的各元素值
    if(a[i])                 //若元素非 0(为互不相同的某一个数)
    {
        b[num][0]=a[i]; //第 1 列存放那些互不相同的数(本身)
        b[num][1]++;    //第 2 列存放数的出现频率(次数)，遇见一个数时次数加 1
        for(int j=i+1; j<n; j++)
            //将 a 中后续的与 a[i]相同的各数均“消掉”(赋值为 0)
            if (a[j]==a[i])
            {
                b[num][1]++; //遇见相同的数后出现次数要加 1
                a[j]=0;      //将相同的数“消掉”(赋值为 0)
            }
        num++;               //互不相同的数的个数 num 增加 1
    }
}

cout<<"---- The result is ----"<<endl; //输出结果
cout<<"    num="<<num<<endl;           //互不相同的数的个数 num
cout<<setw(10)<<"Number"<<setw(10)<<"Times"<<endl;
for(i=0; i<num; i++)           //输出 b 数组(具有 2 列的表)中的各元素值
    cout<<setw(8)<<b[i][0]<<setw(10)<<b[i][1]<<endl;
cout<<endl;
}

```

[输出结果]

Input 10 positive INTEGERS:

1 2 4 2 1 8 6 4 2 35

----- The result is -----

num=6

Number	Times
--------	-------

1	2
---	---

2	3
---	---

4	2
8	1
6	1
35	1

第五章 函数的设计与应用

[语法要点]

详细内容可参阅原教材第五章。

(1) 函数说明语句 C++程序允许两种函数说明语句的形式,把它们分别称为函数原型(或函数声明)和函数定义。

函数原型(亦称函数声明)用来指出函数的名称、类型和参数,其格式为

[< 属性说明>]< 类型>< 函数名>(< 参数表>);

例如:

```
int add (int a,int b);
```

```
inline void swap (float &, float &);
```

```
void print (char *) ;
```

函数定义与函数原型的主要区别是它还包括函数体,其格式为

[< 属性说明>]< 类型>< 函数名>(< 参数表>) {< 函数体> }

(2) 函数调用 是已定义函数的一次实际运行,其具体的调用实施过程如下:

① 按照调用语句中的函数名,在整个程序中搜索同名函数定义;

② 对实参的参数个数、类型、顺序进行核对,判定是否与函数定义中的形参表对应一致;

③ 根据参数的类型(值参数或引用参数)进行值参数的值传递或引用参数的换名;

④ 运行函数体代码;

⑤ 返回调用点,并返回所要求的函数值,即返回计算结果。

(3) 函数的返回 完成两项任务:

① 把运行控制从函数体返回到函数调用点。

② 根据返回值要求,返回所需要的数据值,有三种情形:

- 返回 void 类型
- 返回数值类型
- 返回引用类型

值返回方式是 C 和 Pascal 语言中惟一的返回方式,C++语言提供的引用返回概念是其特有的一种很强的功能,当函数定义中把该函数说明为某类型的引用类型时,该函数调用后返回的不单是值,而是包含返回值的变量(或对象)。

(4) 函数的参数

① 无参函数 其函数说明为下列形式:

```
void print( void);
```

```
int getx( );
```

② 一个或多个参数

多数函数有一个或多个具有确定的个数、类型和顺序的参数，例如：

```
void sort( int n, char*array) {...};  
float max( float a, float b, float c) {...};
```

③ 不定个数参数。

(5) 值调用方式和引用调用方式

值调用方式 执行函数调用时，在检查函数名及参数表之后，首先为值参数分配内存，然后计算其对应的实参表达式，并把计算的值赋给刚刚创建的参数变量，进而开始函数体的运行。

引用调用方式

引用形参 函数定义的参数表中，名字前加上符号&的参数为引用形参。引用形参在调用过程中的参数传递机制不同于赋值形参。其要点是：

① 函数的调用语句中对应于引用形参的实参类型必须利用与引用开方参保持一致，非变量的表达式则不允许作为与引用开方参对应的实参作为与引用开方参对应的实参。

② 参数传递的内容不是实参的值，而是地址，其实际的效果是令对应的引用形参在调用过程中，作为一个变量名指向作为实参的这个变量，与赋值形参的不同就在这里体现出来，在引用调用过程中并不创建新的参数变量！

③ 在函数体程序块的运行中，引用形参的每次出现，由于它现在已经就是指向实参变量，因此相当于被实参变量所代替，即起到了所谓的“换名”的作用。

④ 在函数体程序运行结束，控制转回调用点时，该引用形参与实参变量的对应关系也就终止了。但是在调用过程中对于这个实参变量的所有处理和操作的结果，却保留下来。这一点也是区别于赋值调用的。

(6) 函数嵌套

一个函数的函数体中包含一个或多个函数调用语句，即称为函数嵌套。

(7) 函数递归

函数 A 在其函数体中直接包含对自己的调用语句，函数 A 称为（直接）递归函数。

函数 A 在其函数体中间接地包含对自己的调用，例如 A 调用函数 B，但函数 B 又调用函数 A，则函数 A 称为（间接）递归函数。

使用递归函数应注意：

① 无论是直接递归还是间接递归，都必须保证在有限次调用之后能够结束。

② 函数调用时系统要付出时间和空间代价，在机能相同的情形下，总是非递归程序效率较高。

(8) 内联(inline)函数

① 在 C++ 程序中符合下列条件的函数称为内联函数：

- 函数说明前冠以“inline”关键字的函数；
- 类内定义的函数成员。

② 在编译过程中，凡内联函数，系统把它的执行代码插入到该函数的每个调用点，从而使程序执行过程中，每次该函数调用时不需控制转移，但该函数代码可能有多个拷贝出现在目标程序中。

- ③ 一般把函数体短小而又频繁调用的函数说明为内联函数较好。
- ④ 利用编译预处理的宏定义方式，也可以实现类似于内联函数的功能。不过，内联函数的方式更为方便和可靠。

5.1 自定义函数及其调用

读程序，写出执行结果。

```
#include <iostream.h>
int add (int x, int y)                                //注 1
{
    return (x+y);                                     //注 2
}

void main(void)
{
    int x, i;
    cout<<"x1="<<add(4,8)<<endl;                    //注 3
    cout<<"x2="<<add(3, add(1,2))<<endl;              //注 4
    x=0;
    for(i=1; i<=6; i++)
        x=add(x, i);                                  //注 5
    cout<<"x3="<<x<<endl;
    cout<<"x4="<<add(add(add(add(add(1,1), 1), 1), 1), 1 )<<endl;    //注 6
    cout<<"x5="<<add(add(add(add(add(1,2), 3), 4), 5), 6 )<<endl;
}
```

[分析提示]

- (1) 除 main 主函数外，程序还包含了第二个函数——自定义函数 add。
- (2) 对 add 进行调用后的返回结果为一个 int 型数据，从而可进一步作为调用 add 函数的实参来使用。

[解答程序]

具体程序见题目处。

注 1：自定义函数 add，具有两个参数。

注 2：函数返回值为两参数之和。

注 3：对自定义函数 add 进行调用，代入实参 4 与 8。

注 4：将函数调用“ add(1,2)” 的结果值作为第二实参进一步去调用 add 函数。

注 5：两个实参都为变量。

注 6：多层次地将函数调用结果值作为新的实参去递归调用 `add` 函数。

[输出结果]

```
x1=12
x2=6
x3=21
x4=6
x5=21
```

5.2 一般自定义函数及其调用

设 $f(x) = (x * x - x + 1) / 2 - 1.6$; 编程序计算:

1) $b = (f(2.2) + 3 * f(5)) / f(3.1)$;

2) 对任意输入的一个实数 a , 求出 $f(a)$ 。

[分析提示]

编写一个自定义函数 f , 实现上述的 $f(x)$ 函数关系; 而后进行所要求的函数调用。

[解答程序]

```
#include <iostream.h>
double f (double x)
{
    //自定义函数 f, 含有一个 double 型形参 x, 调用时将代入具体的实参
    double y;                      //局部变量 y
    y=(x*x-x+1)/2-1.6;             //y 即为所计算出的 f(x)值
    return (y);
    //对非 void 类型的函数, 必须有一个 return 语句, 由它返回函数值
}

//注意: 若 main 函数书写在前, 而自定义函数 f 在后时,
//则必须在 main 前先列出函数 f 的原型。
void main( )
{
    double b, a;
    b=(f(2.2)+3*f(5))/f(3.1);
    //调用自定义函数 f 算出 b 值, 而后输出
    //注意该语句中以不同的实参对 f 函数调用了三次
    cout<<"b="<<b<<endl;
```

```

        cout<<"Input a=";           //提示用户输入
        cin>>a;                     //任意输入一个实数 a
        cout<<"f(a)="<<f(a)<<endl; //算出 f(a)并输出
    }

```

[输出结果]

```

b=12.4919
Input a=10
f(a)=43.9

```

5.3 设计无参函数实现固定显示功能

设计参函数 `void displayStars()`，它负责完成固定功能：在屏幕的同一行上连续显示 40 个“*”。并编写主函数 `main`，对该函数进行具体调用。

[分析提示]

自定义函数 `displayStars` 没有返回值，所以其函数类型被说明为 `void`。

[解答程序]

```

#include <iostream.h>

void displayStars( )           //自定义无参函数 displayStars
{
    //将 displayStars 定为 void 类型，是因为不需要它返回任何值
    for(int i=1; i<=40; i++)    //连续显示 40 个“*”
        cout<<"*";
    cout<<endl;                //换一行
}

void main( )
{
    cout<<"the result of first call to 'displayStars( )':"<<endl;
    displayStars( );            //对自定义无参函数 displayStars 的第一次调用
    cout<<"the result of second call to 'displayStars( )':"<<endl;
    displayStars( );            //对自定义无参函数 displayStars 的第二次调用
}

```

[输出结果]

```

the result of first call to 'displayStars( )':

```

```

*****
the result of second call to 'displayStars()':
*****

```

5.4 设计一参函数使显示行数可变化

设计一参函数 `void displayStars(int k)`，它负责显示出 `k` 行“*”来，其中每行均显示连续的八个“*”。并编写主函数 `main`，对该函数进行具体调用。

[分析提示]

自定义的 `displayStars` 函数具有一个参数 `k`，它代表着可变化的行数，调用时通过与 `k` 对应的实参来控制具体显示的行数，从而使该自定义函数的使用具有了灵活性。

[解答程序]

```

#include <iostream.h>

void displayStars(int k)           //k 为形参，由调用处的实参提供实际值
{
    for(int i=1; i<=k; i++)        //显示出 k 行，其中每行均显示连续的八个“*”
        cout<<"*****"<<endl;
}

void main( )
{
    int lines_num;                 //欲显示的行数 lines_num
    cout<<"input lines_num:";      //提示用户输入
    cin>>lines_num;                //输入 lines_num
    cout<<"the result of call to 'displayStars("<<lines_num<<")':"<<endl;
    displayStars(lines_num);        //以 lines_num 为实参去调用 displayStars
}

```

[输出结果]

```

input lines_num:3
the result of call to 'displayStars(3)':
*****
*****
*****

```

5.5 设计二参函数使变化更灵活

设计两参函数，void displayStars(int k, int n)，它负责显示出 k 行“*”来，且每行均显示连续的 n 个“*”。并编写主函数 main，对该函数进行具体调用。

[分析提示]

自定义的 displayStars 函数具有两个参数，它们分别代表着可变化的行数以及每行要显示“*”的个数。调用时将传递过来相应的实参值，从而使该自定义函数具有了更大的灵活性。

[解答程序]

```
#include <iostream.h>

void displayStars(int k, int n)
{
    //自定义函数 displayStars，有两个参数
    //它负责显示出 k 行“*”来，且每行均显示连续的 n 个“*”
    for(int i=1; i<=k; i++)           //循环 k 次，显示出 k 行“*”
    {
        for(int j=1; j<=n; j++)       //循环 n 次，显示出 1 行的 n 个“*”
            cout<<"*";
        cout<<endl;
    }
}

void main( )
{
    int lines_num, stars_num;
    //显示出 lines_num 行，每行显示 stars_num 个“*”
    cout<<"input 'lines_num', 'stars_num':";    //提示用户输入
    cin>>lines_num>>stars_num;                //输入 lines_num 及 stars_num
    cout<<"the result of call to 'displayStars(";
    cout<<lines_num<<","<<stars_num<<"):"<<endl;
    displayStars(lines_num, stars_num);
    //以 lines_num 及 stars_num 为实参去调用 displayStars
}
```

[输出结果]

```
input 'lines_num', 'stars_num':4 10
```

the result of call to 'displayStars(4,10)':

```
*****
*****
*****
*****
```

5.6 全局变量与无参函数配合使用

设计无参函数 `void displayStars( )`，并结合使用全局变量 `k` 与 `n`，使每调用一次该函数，总显示出 `k` 行“*”来，且每行均显示连续的 `n` 个“*”。并编写主函数 `main`，对该函数进行具体调用。

[分析提示]

(1) 在所有函数之外(在不属于任一函数定义的外面)说明变量 `k` 与 `n`，它们即为全局变量，其作用域即有效区域为整个文件。

(2) 每次调用函数 `displayStars` 前，要给全局变量 `k` 与 `n` 赋上所需的不同值。被调函数 `displayStars` 将通过全局变量 `k` 与 `n` 的具体值来确定要显示的行数以及每行要显示的“*”的个数。

[解答程序]

```
#include <iostream.h>

int k,n;                                //int 型全局变量 k、n
//在所有函数之外(在不属于任一函数定义的外面)说明了全局变量 k 与 n，
//它们的作用域即有效区域为整个文件。

void displayStars( )                    //自定义无参函数 displayStars
{
    //它负责显示出 k 行“*”来，且每行均显示连续的 n 个“*” (k 与 n 都
    //是全局变量)
    for(int i=1; i<=k; i++)
    {
        //循环 k 次，显示出 k 行“*”——全局变量 k
        for(int j=1; j<=n; j++)        //循环 n 次，显示出 1 行的 n 个“*”——全局变量 n
            cout<<"*";
        cout<<endl;
    }
}
```

```

void main()
{
    cout<<"input k\`'(lines_num), `n\`'(stars_num):";    //提示用户输入
    cin>>k>>n;    //用户输入全局变量 k, 全局变量 n
    cout<<"the result of call to `displayStars( )`:"<<endl;
    displayStars( );    //调用无参函数 displayStars
}

```

[输出结果]

```

input k\`'(lines_num), `n\`'(stars_num):3 20
the result of call to `displayStars( )`:
*****
*****
*****

```

5.7 将十进制数转化为二进制数的自定义函数

编写一个将十进制数，转化为二进制数的函数，具有如下原型：

```
void Chg10To2(int n);
```

参数 n 为一个十进制数，函数功能为：将十进制数 n 转化为二进制数并输出。
并编写主函数，对所编函数进行调用。

[分析提示]

十进制数转化为二进制数的方法提示：“逐次除以 2，倒着排余数”。用 `int` 型数组 `res` 存放逐次得到的各余数，最后将数组 `res` 中的各余数倒着输出来。

[解答程序]

```

#include <iostream.h>
void Chg10To2(int n)    //将十进制数 n 转化为二进制数并输出
{
    int res[20], ind=0, n0=n;    //用 int 型数组 res 存放逐次得到的各余数;
    //ind 初值为 0(从 0 下标开始往 res 中存放各余数)
    while (n)    //逐次除以 2, 将余数放在数组 res 中(按 ind 下标位置存放)
    {
        res[ind++]=n%2;    //每次“下推” ind 下标位置
        n=n/2;    //将 n 除以 2(取整)仍放回 n
    }
    cout<<n0<<"(10)=";
}

```

```
while (--ind>=0)           //将数组 res 中的各余数倒着输出来(即为所求的二进制数)
    cout<<res[ind];
    cout<<"(2)"<<endl;
}

void main(void)
{
    int n;
    cout<<"Input a positive integer ( '-1' will ending!): ";
                                //提示输入正整数( 输入“-1” 结束程序)
    cin>>n;                      //输入的正整数放 n 中
    while ( n != -1 )           //输入的 n 非“ -1” 时继续处理
    {
        Chg10To2(n);           //以 n 为实参调用自定义函数 Chg10To2, 并输出转化结果
        cout<<"Input a positive integer ( '-1' will ending!): ";
        cin>>n;                 //输入下一个正整数放 n 中
    }
}
```

[输出结果]

```
Input a positive integer ( '-1' will ending!): 12
12(10)=1100(2)
Input a positive integer ( '-1' will ending!): 255
255(10)=11111111(2)
Input a positive integer ( '-1' will ending!): -1
```

5.8 将十进制数转化为八进制数的自定义函数

编写一个将十进制数转化为八进制数的函数，具有如下原型：

```
void Chg10To8(int n);
```

参数 **n** 为一个十进制数，函数功能为：将十进制数 **n** 转化为八进制数并输出。
并编写主函数，对所编函数进行调用。

[分析提示]

十进制数转化为八进制数的方法提示：“逐次除以 8，倒着排余数”。用 **int** 型数组 **res** 存放逐次得到的各余数，最后将数组 **res** 中的各余数倒着输出来。

[解答程序]

```
#include <iostream.h>

void Chg10To8(int n)          //将十进制数 n 转化为八进制数并输出
{
    int res[20], ind=0, n0=n;    //用 int 型数组 res 存放逐次得到的各余数;
    //ind 初值为 0(从 0 下标开始往 res 中放各余数)
    while (n)    //逐次除以 8, 将余数存放在数组 res 中(按 ind 下标位置放)
    {
        res[ind++]=n%8;    //每次下推 ind 下标位置
        n=n/8;            //将 n 除以 8(取整)仍放回 n
    }
    cout<<n0<<"(10) = ";
    while (--ind>=0)        //将数组 res 中的各余数倒着输出来(即为所求的 11 进制数)
        cout<<res[ind];
    cout<<"(8)"<<endl;
}

void main(void)
{
    int n;
    cout<<"Input a positive integer ( '—1' will ending!): ";
    cin>>n;                //输入的正整数放 n 中(输入“ —1” 结束程序)
    while ( n != —1 )      //输入的 n 非“ —1” 时继续处理
    {
        Chg10To8(n);        //以 n 为实参调用自定义函数 Chg10To8, 并输出转化结果
        cout<<"Input a positive integer ( '—1' will ending!): ";
        cin>>n;            //输入下一个正整数放 n 中
    }
}
```

[输出结果]

Input a positive integer ('—1' will ending!): 100

100(10) = 144(8)

Input a positive integer ('—1' will ending!): 32

32(10) = 40(8)

Input a positive integer ('—1' will ending!): —1

5.9 返回布尔值的函数

编一个返回布尔值的函数 `p`，其函数原型为：`bool p(int x);`

具有如下功能：若正整数 `x` 的各位数字之积等于 $x*x-10*x-22$ 的话，返回 `true`，否则返 `false`。并编写主函数，通过调用 `p`，找出 100 以内的满足上述关系的所有 `x`。

[分析提示]

(1) 函数 `p` 中通过多次使用“`x%10` 以及 `x/=10`”的操作来获取 `x` 的从个位开始的各位数字，并把这些数字累乘到一个初值为 1 的变量 `res` 中。

(2) 主函数中通过“`for( x=1; x<=100; x++)`”的循环，依次去调用 `p(x)`来找出满足条件的所有 `x`。

[解答程序]

```
#include <iostream.h>

bool p(int x);           //函数 p 的原型

void main( )
{
    for(int x=1; x<=100; x++)    //通过调用 p，找出 100 以内的满足“ p 关系”的所有 x
        if(p(x))                //以 x 为实参调用函数 p，返回一个布尔值
            cout<<"x="<<x<<" OK!"<<endl;
}

bool p(int x)             //返回布尔值的函数
{
    //若正整数 x 的各位数字之积等于  $x*x-10*x-22$  的话，返回 true，否则返回 false
    int tmp=x, res=1;      //res 中存放 x 的各位数字之积
    while(tmp>0)           //"截取" x 的各位数字并累乘到 res 上
    {
        res*=tmp%10;       //"截取" 当前 tmp 的个位，累乘到 res 上
        tmp=tmp/10;        //"甩掉" 当前 tmp 的个位
    }
    return(res==x*x-10*x-22); //返回结果
}
```

[输出结果]

x=12 OK!

5.10 判断整数 x 是否仅由奇数字组成的自定义函数

编写具有如下原型的函数：bool all_oddNum (int x);

其功能为：若整数 x 仅由奇数字(1、3、5、7、9)组成时(如 x=1571)，函数返回 true，否则返回 false(如当 x=310 时)。并编写主函数，对它进行调用。

[分析提示]

(1)函数 all_oddNum 中通过多次使用“ x%10 以及 x/=10”的操作来获取 x 的从个位开始的各位数字，并判断这些数字是否全为奇数。

(2) 主函数中输入任一个整数 x，并使用 x 作为实参去调用 all_oddNum 函数。

[解答程序]

```
#include <iostream.h>

bool all_oddNum (int x)
{
    //若整数 x 仅由奇数字(1、3、5、7、9)组成时返回 true，否则返回 false
    while (x)                //“ 分离” x 的各位数字加以判断
    {
        int tmp=x%10;        //“ 分离” 出 x 的个位
        if (tmp==0||tmp==2||tmp==4||tmp==6||tmp==8)    //若为偶数字
            return (false);    //一旦发现偶数字，则立刻返回 false(不再继续循环了!)
        x/=10;                //甩掉个位，形成新的 x
    }
    return (true);            //若循环结束，意味着仅由奇数字组成，返回 true
}

void main(void)
{
    int x;
    cout<<"Input a integer: x=";    //提示进行输入
    cin>>x;                          //输入 x
    if (all_oddNum(x))                //以 x 为实参调用自定义函数 all_oddNum，若返回 true 时
        cout<<"YES! -- (Only: 1, 3, 5, 7, 9)"<<endl;
    else
        //返回 false 时
        cout<<"NO! -- (Include: 0, 2, 4, 6, 8)"<<endl;
}
```

[输出结果]

```
Input a integer: x=9135
YES! -- (Only: 1, 3, 5, 7, 9)
```

再一次执行:

```
Input a integer: x=1350
NO! -- (Include: 0, 2, 4, 6, 8)
```

5.11 判断正整数 n 是否为“回文数”的自定义函数

编一个返回布尔值的函数 `symm`, 其函数原型为: `bool symm(long n);`

具有如下功能: 若正整数 n 为“回文数”(正读与反读为相同大小的数)的话, 返回 `true`, 否则返回 `false`。

并编写主函数, 通过调用 `symm`, 求出 11~1000 内的所有的满足下述特征的 m : m 和 $2*m$ 以及 $m*m$ 都是“回文数”, 如 11, 22, 101, ... (因为 $m=11$, $2*m=22$, $m*m=121$; $m=22$, $2*m=44$, $m*m=484$; $m=101$, $2*m=202$, $m*m=10201$; ...)。

[分析提示]

(1)在 `symm` 函数中, 按如下方法来判断 n 是否为“回文数”: 先求出 n 的“逆序数” m , 若 m 等于 n , 则 n 为“回文数”(如, $n=132$ 时, 其“逆序数”为 231)。

(2) 主函数中通过使用“`for(m=11; m<1000; m++)`”的循环, 并对逻辑表达式“`( symm(m) && symm(2*m) && symm(m*m) )`”进行判断, 以找出满足特征的所有 m (逻辑表达式中对自定义函数 `symm` 进行了三次调用)。

[解答程序]

```
#include <iostream.h>

bool symm(long n)
{
    //若正整数 n 为“回文数”的话, 返回 true, 否则返回 false
    //实现方法: 先求出 n 的“逆序数” m, 若 m 等于 n, 则 n 为“回文数”
    long tmp=n, m=0;           //在 m 中“合成” n 的“逆序数”
    while(tmp) //从个位往左,“截取” n 的各位数字来“合成” n 的“逆序数” m
    {
        m=m*10+tmp%10; //截取当前 tmp 的个位,“合成”到 m 上
        tmp=tmp/10;    //甩掉当前 tmp 的个位
    }
    return ( m==n );      //返回结果(若 m 等于 n 返回 true, 否则返回 false)
}
```

```

void main( )
{
    long m;                //通过调用 symm, 求出 11~1 000 内的所有的满足
                           //下述特征的 m: m 和 2*m 以及 m*m 都是“回文数”
    for(m=11; m<1000; m++)
    {
        if ( symm(m) && symm(2*m) && symm(m*m) )
            cout<<"m="<<m<<"", 2*m="<<2*m<<"",  m*m="<<m*m<<endl;
            //输出满足条件者
    }
}

```

[输出结果]

```

m=11,  2*m=22,  m*m=121
m=22,  2*m=44,  m*m=484
m=101, 2*m=202, m*m=10201
m=111, 2*m=222, m*m=12321
m=121, 2*m=242, m*m=14641
m=202, 2*m=404, m*m=40804
m=212, 2*m=424, m*m=44944

```

5.12 判断整数 n 是否为素数的自定义函数

编写具有如下原型的函数 isPrime, 用来判断整数 n 是否为素数: `bool isPrime(int n);` 而后编出主函数, 任意输入一个大于 4 的偶数 n, 找出满足 $n=n_1+n_2$ 的所有数对, 其中要求 n_1 与 n_2 均为素数(通过调用 isPrime 来判断素数)。如偶数 14 可以分解为 $14=7+7$ 以及 $14=11+3$ 。而偶数 100 可以分解为: $53+47$ 、 $59+41$ 、 $71+29$ 、 $83+17$ 、 $89+11$ 、 $97+3$ 。

[分析提示]

(1)isPrime 中判断整数 n 是否为素数: 判断从 2 到 n 这些数中是否有 n 的因子。

(2) 主函数中输入任一个偶数 n, 而后通过如下的循环来找出满足条件的“数对” i 与 $n-i$ (注意: i 与 $n-i$ 的和恰为 n, 要在所有这样的“数对”中找):“ for ($i=n/2$; $i \leq n-2$; $i++$)”, 而后在循环中判断“ `isPrime(i) && isPrime(n-i)`”是否成立。

[解答程序]

```

#include <iostream.h>
#include <math.h>

```

```

bool isPrime (int n)                                //判断整数 n 是否为素数的布尔函数
{
    for(int i=2; i<=sqrt(n); i++)                    //判断从 2 到 n 这些数中有没有 n 的因子
        if(n%i==0)                                  //一旦有一个因子，则非素数
            return false;                            //返回 false
    return true;                                     //所有都除不尽时，才为素数
}

void main(void)
{
    int n;
    cout<<"Input a even number!"<<endl; //提示输入一个偶数
    cout<<"n=? ";                        //提示输入偶数 n
    cin>>n;                               //输入 n
    for (int i=n/2; i<=n-2; i++)
        //注意：i 与 n-i 的和恰为 n，要在所有这样的“数对”中找
        if (i%2!=0 && (n-i)%2!=0)
            //i 与 n-i 均为奇数时，才有可能成为所求的“数对”
            if ( isPrime(i) && isPrime(n-i) ) //进一步判断二者均为素数
                cout<<n<<"="<<i<<"+"<<n-i<<endl; //输出满足要求的“数对”
}

```

[输出结果]

Input a even number!

n=? 14

14=7+7

14=11+3

再一次执行：

Input a even number!

n=? 100

100=53+47

100=59+41

100=71+29

100=83+17

100=89+11

100=97+3

5.13 返回整数 x 某一指定位之数值的函数

编写具有如下原型的两个函数：

`int digitR(int x, int n);` //返回整数 x 的从右边数第 n 位的数值(倒数第 n 位)

`int digitL(int x, int n);` //返回整数 x 的从左边数第 n 位的数值(正数第 n 位)

例如: `digitR(12345, 2)` 返回 4; `digitR(12345, 5)` 返回 1; `digitR(12345, 6)` 返回 0;
`digitL(67891, 2)` 返回 7; `digitL(67891, 5)` 返回 1; `digitL(67890, 6)` 返回 0。

[分析提示]

(1) 编写自定义函数 `digitR` 和 `digitL`, 并编写 `main` 函数对它们进行调用。

(2) `digitR` 中可通过多次使用“`x%10` 以及 `x=x/10`”, 从而可截出从 x 的个位开始往左的某一个位的数值(倒数第某一位)。

(3) 在 `digitL` 中说明一个 `int` 型的 `res` 数组, 用来存放分离出来的 x 的各位数值, 而后从数组中“取出”所需的整数 x 的从左边数第 n 位的数值(正数第 n 位)。

[解答程序]

```
#include <iostream.h>

int digitR(int x, int n)          //返回整数 x 的从右边数第 n 位的数值(倒数第 n 位)
{
    int res;
    for (int i=1; i<=n; i++)      //从右边数第 n 位, 共循环 n 次
    {
        res = x % 10;            //当前 x 的个位放 res
        x=x/10;                  //甩掉 x 的个位后形成新的 x
    }
    return (res);                //返回第 n 次取到的个位
}

int digitL(int x, int n)          //返回整数 x 的从左边数第 n 位的数值(正数第 n 位)
{
    int res[20], ind=19;
    //res 数组用于存放分离出来的各位数值, ind 表示下标位置(从后往前放)
    while (x) //将 x 的各位数值分离出来, 依次放入 res 数组的后半部分
    {
        res[ind--]=x%10;          //当前 x 的个位放 res 数组
        x=x/10;                  //甩掉 x 的个位后形成新的 x
    }
    if(ind+n>19) return 0;        //x 的总位数小于 n 时返回 0
}
```

```

        return (res[ind+n]);           //返回 res 数组中已分离出来的某个所要求位的数值
    }

    void main(void)
    {
        int x, n;                      //准备求出整数 x 的倒数第 n 位以及正数第 n 位的数值并输出
        cout<<"x=? n=?"<<endl;        //提示输入整数 x 以及 n
        cin>>x>>n;                     //输入 x 与 n
        int resR=digitR(x,n);           //求出 x 的倒数第 n 位的数值
        cout<<"resR="<<resR<<endl;     //输出结果
        int resL=digitL(x,n);           //求出 x 的正数第 n 位的数值
        cout<<"resL="<<resL<<endl;     //输出结果
    }

```

[输出结果]

```

x=? n=?
12345 2
resR=4
resL=2

```

再一次执行:

```

x=? n=?
12345 6
resR=0
resL=0

```

5.14 三种控制语句与数组及函数配合使用

读下面的程序，写出执行结果(注意，该范例与范例 3.16 以及范例 4.9 有关联)。

```

#include <iostream.h>

void func_if(int i)                      //注 1
{
    if(i<0)                             //注 2
    {
        cout<<"i="<<i<<" ";
        i=i+1;
    }
    cout<<"--if_OK!\n";
}

```



```
    }

    void func_while(int i)                                //注 3
    {
        while(i<0)                                        //注 4
        {
            cout<<"i="<<i<<" ";
            i+=1;
        }
        cout<<"--while_OK!\n";
    }

    void func_do(int i)                                    //注 5
    {
        do                                                //注 6
        {
            cout<<"i="<<i<<" ";
            i++;
        } while(i<0);                                    //注 7
        cout<<"--do_OK!\n";
    }

    void func_for(int i)                                    //注 8
    {
        int j=-2, s=0;
        for(;;)                                           //注 9
        {
            s+=j+5;                                        //注 10
            j++;
            if(j>i)                                        //注 11
                break;
        }
        cout<<"s="<<s<<" --for_OK!\n";
    }

    void main( )
    {
        int k;
        int n[3]={ -2, -1, 1};                            //注 12
```

```
        for(k=0; k<3; k++)                //注 13
        {
            func_if(n[k]);                  //注 14
            func_while(n[k]);
            func_do(n[k]);
            func_for(n[k]);
            cout<<"-----\n";
        }
    }
```

[分析提示]

(1) 本范例与范例 4.9 相比, 除使用 `n` 数组之外, 还使用了多个自定义函数, 但它们实现了完全相同的功能, 并显示出完全相同的执行结果。

(2) 本题中的主要语句仍为 `if`、`while`、`do-while` 以及 `for`, 但与函数配合使用, 实现形参与实参相结合等语法及处理功能。

[解答程序]

具体程序见题目处。

注 1: 自定义的 `func_if` 函数。

注 2: 若条件成立, 一次性执行“子句”句体。

注 3: 自定义的 `func_while` 函数。

注 4: 只要条件成立, 就再一次执行“子句”句体。

注 5: 自定义的 `func_do` 函数。

注 6: 至少执行一次“子句”句体。

注 7: 只要条件成立, 就再一次去执行“子句”句体。

注 8: 自定义的 `func_for` 函数。

注 9: 三个表达式全省略的 `for` 语句, 认为“条件”恒真。

注 10: 往 `s` 上累加数据。

注 11: 若条件成立, 靠 `break` 跳出循环。

注 12: 说明具有三个元素的 `n` 数组, 并赋予初始值。

注 13: 三次循环, 依次使用 `n` 数组中的三个数, 即以数组元素 `n[k]` 为实参去调用各函数。

注 14: 以数组元素 `n[k]` 为实参去调用 `func_if` 函数。

[输出结果]

```
i=-2  --if_OK!
i=-2  i=-1  --while_OK!
i=-2  i=-1  --do_OK!
s=3  --for_OK!
```

```

-----
i=-1  --if_OK!
i=-1  --while_OK!
i=-1  --do_OK!
s=7   --for_OK!
-----

--if_OK!
--while_OK!
i=1   --do_OK!
s=18  --for_OK!
-----

```

5.15 计算两个日期之间的连续天数

由用户任意输入两个日期 如: 2003 1 8——表示 2003 年 1 月 8 日, 2001 12 28——表示 2001 年 12 月 28 日), 而后由程序计算出这两个日期之间的连续天数(包括两头的“ 起始日”与“ 终止日”。如, 2002 年 5 月 1 日到 2002 年 5 月 2 日算做连续 2 天, 2003 年 1 月 8 日与 2001 年 12 月 28 日之间的连续日为 377 天, 而 1999 年 5 月 1 日至 2010 年 2 月 3 日之间的连续日为 3 932 天)。

例如, 程序执行后, 可在屏幕上显示如下形式的结果(第一次执行):

```

Input YEAR1  MONTH1  DAY1:2002 1 28
Input YEAR2  MONTH2  DAY2:2002 3 2
2002.1.1 => 2002.1.28(total_days): 28
2002.1.1 => 2002.3.2(total_days): 61
-----
2002.1.28 => 2002.3.2(total_days)=34
    又一次执行, 屏幕显示结果为:
Input YEAR1  MONTH1  DAY1:2003 1 8
Input YEAR2  MONTH2  DAY2:2001 12 28
2001.1.1 => 2001.12.28(total_days): 362
2001.12.28 => 2001.12.31(total_days): 4
2002(total_days): 365
2003.1.1 => 2003.1.8(total_days): 8
-----
2001.12.28 => 2003.1.8(total_days)=377

```

[分析提示]

可编写具有如下原型的函数 daysFrom1_1, 负责计算非整年的(年份由参数 year 提

供) 从 1 月 1 日到某月某日(月份及日子的值由参数 month 与 day 提供) 的连续天数:

```
int daysFrom1_1 (int year, int month, int day);
```

这样的话,“ 两头” 非整年具有的天数均可通过调用 daysFrom1_1 来算出, 而“ 中间” 的整年具有的天数则可单独直接计算。若年份为同一年的话, 要两次调用 daysFrom1_1)。

[解答程序]

```
#include <iostream.h>
#include <math.h>

int daysFrom1_1 (int year, int month, int day)
{
    //负责计算年份为 year 的那一年从 1 月 1 日到某月某日
    //(月份及日子的值由参数 month 与 day 提供)的连续天数
    int MthDays[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
    //MthDays[i]表示月份为 i 的那一个月的天数(若闰年的话, 2 月份要再增加 1 天)
    int i, sum=0;           //sum 放累加结果(最终结果的连续天数靠累加而得)
    for(i=1; i<month; i++)  //累加 1 -> month-1 月份每月的整月之天数
        sum+=MthDays[i];
    if(month>2)             //若月份大于 2 且该年为闰年时, 2 月份要再增加 1 天
        if( (year%4==0) && (year%100 !=0) || (year%400 ==0) )
            sum+=1;
    //可被 4 整除而不可被 100 整除的年份或者可被 400 整除的年份都是闰年
    sum+=day;               //累加当前月份(不足一整月)的“ 零散” 天数
    cout<<year<<".1.1 => "<<year<<". "<<month;
    cout<<". "<<day<<"(total_days): "<<sum<<endl;
    return sum;             //返回累加和(天数)
}

void swap (int& x1, int& x2)  //交换变量 x1 与 x2 的当前值
{
    int tmp=x1;
    x1=x2;
    x2=tmp;
}

void main( )
{
    int year1, month1, day1, year2, month2, day2, from11a, from11b;
    //year1、month1、day1 —— 第一个日期的年、月、日
    //year2、month2、day2 —— 第二个日期的年、月、日
```

```

//from11a —— 第一个日期的年、月、日距离当年1月1日的天数
//from11b —— 第二个日期的年、月、日距离当年1月1日的天数
long totalDays=0;
//totalDays 放总累加结果天数(最终结果的连续天数靠累加而得)
cout<<"Input YEAR1  MONTH1  DAY1:";
//提示输入第一个日期的年、月、日
cin>>year1>>month1>>day1;
//输入年、月、日
cout<<"Input YEAR2  MONTH2  DAY2:";
//提示输入第二个日期的年、月、日
cin>>year2>>month2>>day2;
//输入年、月、日
if (year1>year2)
{
//如果两个年份不同时,总使 year1、month1、day1 所表示的日期“小”(“靠前”)
swap(year1, year2);
swap(month1, month2);
swap(day1, day2);
}
if(year1==year2) //如果两个年份相同时,
{
//通过两次调用 daysFrom1_1, 而后再算出最终结果
from11a=daysFrom1_1 (year1, month1, day1);
//计算年份为 year1 的那一年从1月1日到 month1 月 day1 日的连续天数
from11b=daysFrom1_1 (year2, month2, day2);
//计算年份为 year2 也即 year1 的那一年从1月1日
//到 month2 月 day2 日的连续天数
totalDays=abs(from11a-from11b)+1;
//两个连续天数之差加1后即为所求
}
else //两个年份不同,而且 year1、month1、day1 所表示的日期“小”(“靠前”)
{
//“两头”非整年具有的天数均可通过调用 daysFrom1_1 来算出
//而“中间”的整年具有的天数则可单独直接计算
from11a=daysFrom1_1 (year1, month1, day1);
//靠前的非整年之前半部分天数 from11a
int total_day1=365-from11a+1;
//靠前的非整年之后半部分天数 total_day1
if(month1<=2)
//若月份小于等于2且该年为闰年时,2月份要再增加1天
if( (year1%4==0) && (year1%100 !=0) || (year1%400 ==0) )

```

```

        total_day1+=1;
    cout<<year1<<". "<<month1<<". "<<day1<<" => ";
    cout<<year1<<".12.31(total_days): "<<total_day1<<endl;
        //显示非整年之后半部分天数
    totalDays+=total_day1;
        //将非整年之后半部分天数累加到 totalDays 中
    int tmpYear=year1;
    while (++tmpYear<year2)
    {
        //“ 中间 ” 的整年(tmpYear 为年份)具有的天数单独直接计算
        int tmpDays=365;           //先认为该年为 365 天
        if((tmpYear%4==0) && (tmpYear%100 !=0) || (tmpYear%400 ==0))
            tmpDays+=1;           //若闰年时再加 1 天
        cout<<tmpYear<<"(total_days): "<<tmpDays<<endl;
            //显示该整年之天数
        totalDays+=tmpDays;       //将该整年之天数累加到 totalDays 中
    }
    from11b=daysFrom1_1 (year2, month2, day2);
        //靠后的非整年之前半部分天数 from11b
    totalDays+=from11b;
        //将非整年之前半部分天数累加到 totalDays 中
    }
    cout<<"-----"<<endl;
    cout<<year1<<". "<<month1<<". "<<day1<<" => ";           //输出最终结果
    cout<<year2<<". "<<month2<<". "<<day2<<"(total_days)=";
    cout<<totalDays<<endl;
}

```

[输出结果]

与题目处相同，略。

5.16 计算出两个日期的间隔天数

参照范例 5.15 的计算两个日期之间连续天数的实现方法，定义如下的 difs2Date 函数：

```
long difs2Date(int year1, int month1, int day1, int year2, int month2, int day2);
```

它负责计算出两个日期的间隔天数并返回——先计算两个日期期间的连续天数，而后减 1 得到间隔天数。其中的 year1、month1、day1 表示第一个日期的年、月、日；而 year2、

month2、day2 表示第二个日期的年、月、日。而后编制主函数，对 difs2Date 进行调用。

注意，将计算出两个日期间隔总天数的 difs2Date 函数放在“ 5.16 difs2Date.h”的头文件中(注意：以后还要多次使用该头文件)，主文件中通过 include 编译预处理指令将该头文件内容包含进来，并在 main 主函数中调用 difs2Date 函数。

例如，程序执行后，可在屏幕上显示如下形式的结果：

```
Input YEAR1  MONTH1  DAY1:2001 12 28
Input YEAR2  MONTH2  DAY2:2003 1 8
2001.12.28 => 2003.1.8 (difs2Date) =376 (days)
```

又一次执行，屏幕显示结果为：

```
Input YEAR1  MONTH1  DAY1:1999 5 1
Input YEAR2  MONTH2  DAY2:2010 2 3
1999.5.1 => 2010.2.3(difs2Date)=3931(days)
```

再一次执行：

```
Input YEAR1  MONTH1  DAY1:1 1 1
Input YEAR2  MONTH2  DAY2:2002 12 31
1.1.1 => 2002.12.31(difs2Date)=731214(days)
```

[分析提示]

把一部分经常用到的程序源码放到一个以“.h”为后缀的用户头文件中(如本题的“difs2Date.h”头文件)，以后凡用到这“块”程序时，就通过 include 编译预处理指令将该头文件内容包含进去。

[解答程序]

difs2Date.h

该头文件中定义能够计算出两个日期间隔总天数的 difs2Date 函数：

```
long difs2Date (int year1, int month1, int day1, int year2, int month2, int day2);
```

它负责计算出两个日期的间隔天数并返回——先计算二日期间的连续天数，而后减 1 得到间隔天数。其中的 year1、month1、day1 表示第一个日期的年、月、日；而 year2、month2、day2 表示第二个日期的年、月、日。计算两个日期间连续天数的方法与习题 5.15 的实现方法完全相同。

difs2Date 函数中要调用 daysFrom1_1 函数以及 swap 函数，它们一并被包含在该头文件之中：

```
int daysFrom1_1 (int year, int month, int day);
```

负责计算年份为 year 的那一年从 1 月 1 日到某月某日(月份及日子的值由参数 month 与 day 提供)的连续天数。

```
void swap (int& x1, int& x2);
```

负责交换变量 x1 与 x2 的当前值。

```
int daysFrom1_1 (int year, int month, int day)
{
```

```

        //负责计算年份为 year 的那一年从 1 月 1 日到某月某日
        //(月份及日子的值由参数 month 与 day 提供)的连续天数
int MthDays[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
        //MthDays[i]表示月份为 i 的那一个月的天数(若闰年的话, 2 月份要再增加 1 天)
int i, sum=0;           //sum 放累加结果(最终结果的连续天数靠累加而得)
for(i=1; i<month; i++)  //累加 1 -> month-1 月份每月的整月之天数
    sum+=MthDays[i];
if(month>2)             //若月份大于 2 且该年为闰年时, 2 月份要再增加 1 天
    if( (year%4==0) && (year%100 !=0) || (year%400 ==0) )
        sum+=1;
        //可被 4 整除而不可被 100 整除的年份或者可被 400 整除的年份都是闰年
sum+=day;               //累加当前月份(不足一整月)的“零散”天数
return sum;             //返回累加和(天数)
}

void swap (int& x1, int& x2)    //交换变量 x1 与 x2 的当前值
{
    int tmp=x1;
    x1=x2;
    x2=tmp;
}

long difs2Date(int year1, int month1, int day1, int year2, int month2, int day2)
{
    //计算出两个日期的间隔天数并返回
    //先计算两个日期间的连续天数, 而后减 1 得到间隔天数
    //year1、month1、day1——第一个日期的年、月、日;
    //year2、month2、day2——第二个日期的年、月、日;
    int from11a, from11b;
    //from11a——第一个日期的年、月、日距离当年 1 月 1 日的天数;
    //from11b——第二个日期的年、月、日距离当年 1 月 1 日的天数;
    long totalDays=0;
    //totalDays 放总累加结果天数(最终结果的连续天数靠累加而得)
    if (year1>year2)
    {
        //如果两个年份不同时, 总使 year1、month1、day1 所表示的日期“小”(“靠前”)
        swap (year1, year2);
        swap (month1, month2);
        swap (day1, day2);
    }
}

```



```

}
if(year1==year2)           //如果两个年份相同时，
{
    //通过两次调用 daysFrom1_1，而后再算出最终结果
    from11a=daysFrom1_1 (year1, month1, day1);
    //计算年份为 year1 的那一年从 1 月 1 日到 month1 月 day1 日的连续天数
    from11b=daysFrom1_1 (year2, month2, day2);
    //计算年份为 year2 也即 year1 的那一年从 1 月 1 日
    //到 month2 月 day2 日的连续天数
    totalDays=abs(from11a-from11b)+1;
    //两个连续天数之差加 1 后即为首求
}
else//两个年份不同，而且 year1、month1、day1 所表示的日期“小”（“靠前”）
{
    //“两头”非整年具有的天数均可通过调用 daysFrom1_1 来算出，
    //而“中间”的整年具有的天数则可单独直接计算。
    from11a=daysFrom1_1 (year1, month1, day1);
    //靠前的非整年之前半部分天数 from11a
    int total_day1=365-from11a+1;
    //靠前的非整年之后半部分天数 total_day1
    if(month1<=2)
        //若月份小于等于 2 且该年为闰年时，2 月份要再增加 1 天
        if( (year1%4==0) && (year1%100 !=0) || (year1%400 ==0) )
            total_day1+=1;
    totalDays+=total_day1;
    //将非整年之“后半段”天数累加到 totalDays 中
    int tmpYear=year1;
    while (++tmpYear<year2)
    {
        //“中间”的整年(tmpYear 为年份)具有的天数单独直接计算
        int tmpDays=365;    //先认为该年为 365 天
        if((tmpYear%4==0) && (tmpYear%100 !=0) || (tmpYear%400 ==0))
            tmpDays += 1;    //若闰年时再加 1 天
        totalDays += tmpDays;
        //将该整年之天数累加到 totalDays 中
    }
    from11b=daysFrom1_1 (year2, month2, day2);
    //靠后的非整年之“前半段”天数 from11b
    totalDays+=from11b;
}

```

```

        //将非整年之“前半段”天数累加到 totalDays 中
    }
    return totalDays-1;           //两个日期间的连续天数减 1 得到间隔天数
}

//=====
//以下为本程序的主文件，通过 include 将用户头文件“difs2Date.h”的内容包含进来
#include <iostream.h>
#include <math.h>
#include "difs2Date.h"
//计算两个日期间隔总天数的 difs2Date 函数放在“difs2Date.h”的头文件中
void main( )
{
    int year1, month1, day1, year2, month2, day2;
        //year1、month1、day1 —— 第一个日期的年、月、日
        //year2、month2、day2 —— 第二个日期的年、月、日

    long difs;
    cout<<"Input YEAR1  MONTH1  DAY1:";    //提示输入第一个日期的年、月、日
    cin>>year1>>month1>>day1;              //输入年、月、日
    cout<<"Input YEAR2  MONTH2  DAY2:";    //提示输入第二个日期的年、月、日
    cin>>year2>>month2>>day2;              //输入年、月、日
    difs=difs2Date(year1, month1, day1, year2, month2, day2);
        //difs2Date 负责计算出两个日期的间隔天数并返回
    cout<<year1<<". "<<month1<<". "<<day1<<" => "; //输出最终结果的间隔天数
    cout<<year2<<". "<<month2<<". "<<day2<<"(difs2Date)=";
    cout<<difs<<"(days)"<<endl;
}

```

[输出结果]

与题目处相同，略。

5.17 计算某一天是星期几(方式二)

由用户输入一个表示日期的年、月、日(如输入: 2002 12 31, 表示 2002 年 12 月 31 日), 由程序计算出这一天是星期几(如应输出 2, 表示 2002 年 12 月 31 日是星期二)。

例如, 程序执行后, 可在屏幕上显示如下形式的结果:

```

Input a date(YEAR MONTH DAY):2002 1 1
2002.1.1 => weekday: 2

```

又一次输入:

Input a date(YEAR MONTH DAY):3000 1 1

3000.1.1 => weekday: 3

再一次输入:

Input a date(YEAR MONTH DAY):1825 12 31

1825.12.31 => weekday: 6

[分析提示]

使用能够计算出两个日期间隔总天数的 `difs2Date` 函数(在“`difs2Date.h`”中定义), 并利用元年元月元日(即 1 年 1 月 1 日)为星期一的已知事实, 首先求出从日期(1,1,1)到日期(y,m,d)的间隔总天数 `difsD`: `long difsD=difs2Date (1, 1, 1, y, m, d);`

那么, $(\text{difsD}+1)\%7$ 即为所求结果的星期几(0 代表星期日)。

[解答程序]

```
#include <iostream.h>
#include <math.h>
#include "difs2Date.h"
//计算两个日期间隔总天数的 difs2Date 函数放在“ difs2Date.h”的头文件中
int whatD(int y, int m, int d)
{
    //计算并返回由(y,m,d)指定的日期为星期几(0 表示星期日)
    //注意: 公元的元年元月元日(即 1 年 1 月 1 日)为星期一
    long difsD=difs2Date(1, 1, 1, y, m, d);
    //算出从日期(1,1,1)到日期(y,m,d)的间隔总天数
    return (difsD+1)%7; //间隔总天数加 1 模 7 即为所求结果
}

void main( )
{
    int year, month, day;
    cout<<"Input a date(YEAR MONTH DAY):";//提示输入一个日期(年、月、日)
    cin>>year>>month>>day; //输入日期(年、月、日)
    int ans; //ans 为计算结果(星期几, 0 表示星期日)
    ans=whatD(year,month,day);
    //调用 whatD 计算并返回由(year,month,day)指定的日期为星期几
    cout<<year<<". "<<month<<". "<<day<<" => weekday: "<<ans<<endl;
    //显示结果
}
```

[输出结果]

与题目处相同，略。

5.18 屏幕显示某年某指定月的月历(方式二)

由用户任意输入一个年份 `year` 及任意一个月份 `month`，由程序负责在屏幕上显示出该年这一个月月的月历。例如，程序执行后，可在屏幕上显示如下形式的结果：

Input YEAR and MONTH (1~12):2008 10

2008.10.1 => weekday: 3

```
-----
Sun Mon Tue Wed Thu Fri Sat
-----
                1  2  3  4
    5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
-----
```

[分析提示]

(1) 参照范例 4.17 的处理方法，并结合使用“`difs2Date.h`”中定义的 `difs2Date` 函数的功能，可首先计算出 `year` 年 `month` 月 1 日（欲处理月的 1 号）是星期几（注意，已知公元的元年元月元日即 1 年 1 月 1 日为星期一）。也即首先通过“`difsD=difs2Date(1, 1, 1, year, month, 1);`”计算出从日期(1,1,1)到日期(year,month,1)的间隔总天数，而后通过 $(difsD+1)\%7$ 即可求出 `year` 年 `month` 月的 1 日是星期几（0 表示星期日）。

(2) 在屏幕上按格式显示出这一个月月的月历。

[解答程序]

```
#include <iostream.h>
#include <iomanip.h>           //use 'setw'
#include <math.h>              //use 'abs'
#include "difs2Date.h"
//计算两个日期间隔总天数的 difs2Date 函数放在“ difs2Date.h”的头文件中

int whatD(int y, int m, int d)
{
    //注意：公元的元年元月元日(即 1 年 1 月 1 日)为星期一
    long difsD=difs2Date(1, 1, 1, y, m, d);
    //算出从日期(1,1,1)到日期(y,m,d)的间隔总天数
```

```

        return (difsD+1)%7;           //间隔总天数加 1 模 7 即为所求结果
    }

void main()
{
    int year, month;                  //year、month——年、月
    cout<<"Input YEAR and MONTH (1~12):";    //提示输入年、月
    cin>>year>>month;                //输入年份及月份
    int ans;                          //ans 为计算结果(星期几, 0 表示星期日)
    int MthDays[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
    //MthDays[i]表示月份为 i 的那一个月的天数(若闰年的话, 2 月份要再增加 1 天)
    ans=whatD (year,month,1);
        //ans 为欲处理月的 1 号是星期几 (0 表示星期日)
    cout<<year<<". "<<month<<". "<<1<<" => weekday: "<<ans<<endl;
        //显示欲处理月的 1 号是星期几
    cout<<"-----"<<endl;
    cout<<" Sun Mon Tue Wed Thu Fri Sat"<<endl;
    cout<<"-----"<<endl;
    int wday=ans;                     //wday 现在为欲处理月的 1 号是星期几
    cout<<setw(4*(wday+1))<<1;        //将日期数“ 1” 显示在适当位置
    for (int i=2; i<=MthDays[month]; i++)
    {
        //显示那一月的其他日期数 i(i=2,3,4,...)
        wday++;                       //每处理一天,“ 星期数” 长 1
        if (wday % 7==0)              //处理到“ 周末” 时
        {
            cout<<endl;               //下一周的日期数要换行显示
            wday=0;                   //“ 星期数” 从头来
        }
        cout<<setw(4)<<i;              //在适当位置显示出日期数 i
    }
    cout<<endl;
    cout<<"-----"<<endl;
}

```

[输出结果]

与题目处相同, 略。

5.19 使用三种方法求 Fibonacci 数列的某个项值

编写三个不同的函数，均用于求出 Fibonacci 数列的某个 100 以内的某一项的值(如，fib(3)返回 2；fib(6)返回 8；fib(29)返回 514229)。并编出主函数，对这三个函数进行调用。

计算 Fibonacci 数列某一项值的公式如下 使用 $F<1>$ 表示数列第一项,...,用 $F<n>$ 表示数列第 n 项):

$$F<1> = 1 \quad (n=1)$$

$$F<2> = 1 \quad (n=2)$$

$$F<n> = F<n-2> + F<n-1> \quad (\text{对任意大于 2 的 } n)$$

提示：三个不同的函数对应于三种不同的求取方法——数组法；简单变量“数据平移”法；递归法。注意，当数据增大时，使用递归法求出结果的时间会明显高于前两种方法。

[分析提示]

(1) 数组法——说明一个 fib 数组并赋初值：int fib[101]={0, 1, 1};

而后通过如下的循环语句求出第 n 个 Fibonacci 数：for (int i=3; i<=n; i++) fib[i]=fib[i-2]+fib[i-1];

(2) 简单变量“数据平移”法——说明并使用如下三个简单变量：int olditem1=1, olditem2=1, newitem; 而后通过使用：newitem = olditem1+olditem2; 来算出“距它最近”的前两个项的和。之后为了计算后面的新 Fibonacci 项 newitem，需要进行一种“数据平移”：olditem1 = olditem2; 将 olditem2 当作新的 olditem1), olditem2 = newitem; 将 newitem 当做新的 olditem2), 然后继续计算新的 newitem，如此等等。

(3) 递归法——最容易编写但执行速度最慢的一种求解方法。

[解答程序]

```
#include <iostream.h>

unsigned long fib1(int n);           //三个函数原型
unsigned long fib2(int n);
unsigned long fib3(int n);

void main( )
{
    int n;
    unsigned long answer;
    cout<<"Enter a positive number (<100): "; //提示输入
    cin>>n;                                   //输入 n
    answer = fib1(n);
    //以 n 为实参，调用 fib1(方法一)，求出第 n 个 Fibonacci 数
```

```

cout<<"---- method 1 ----"<<endl;
cout<<n<<"th Fibonacci number is: "<<answer<<endl;
cout.flush( );
answer = fib2(n);
    //以 n 为实参, 调用 fib2(方法二), 求出第 n 个 Fibonacci 数
cout<<"---- method 2 ----"<<endl;
cout<<n<<"th Fibonacci number is: "<<answer<<endl;
cout.flush( );
answer = fib3(n);
    //以 n 为实参, 调用 fib3(方法三), 求出第 n 个 Fibonacci 数
cout<<"---- method 3 ----"<<endl;
cout<<n<<"th Fibonacci number is: "<<answer<<endl;
}

unsigned long fib1 (int n)                //方法一: 使用数组
{
    int fib[101]={0, 1, 1};
    if (n < 3 ) return (1);
    for (int i=3; i<=n; i++)              //计算出第 n 个 Fibonacci 数
        fib[i]=fib[i-2]+fib[i-1];
    return (fib[n]);                     //返回所求数
}

unsigned long fib2 (int n)
{
    //方法二: 使用三个简单变量, 实现时进行一种“数据平移”
    int olditem1=1, olditem2=1, newitem;
    if (n < 3 ) return (1);
    for (int i=3; i<=n; i++)              //计算出第 n 个 Fibonacci 数
    {
        newitem=olditem1+olditem2;
        //新的 Fibonacci 项 newitem 总是“距它最近”的前两个项的和
        //为了计算后面的新 Fibonacci 项 newitem, 需要进行一种“数据平移”
        olditem1=olditem2;
        //将 olditem2 当做新的 olditem1 (将 olditem2 的值“移入” olditem1)
        olditem2=newitem;
        //将 newitem 当做新的 olditem2 (将 newitem 的值“移入” olditem2)
    }
    return (newitem);
}

```

```

    }

    unsigned long fib3 (int n)           //方法三： 使用递归
    {
        if (n < 3 ) return (1);
        return ( fib3(n-2) + fib3(n-1) );
    }

```

[输出结果]

Enter a positive number (<100): 6

----- method 1 -----

6th Fibonacci number is: 8

----- method 2 -----

6th Fibonacci number is: 8

----- method 3 -----

6th Fibonacci number is: 8

再一次执行:

Enter a positive number (<100): 29

----- method 1 -----

29th Fibonacci number is: 514229

----- method 2 -----

29th Fibonacci number is: 514229

----- method 3 -----

29th Fibonacci number is: 514229

5.20 函数的递归调用

读下述的递归函数 f，分析一下函数调用 f(1357)将输出什么结果。

```

void f(int n)
{
    if(n>0)
    {
        cout<<n%10<<" ";
        f(n/10);
    }
}

```

进一步考虑，若将 if 句体中的“ cout<<n%10<<" ";” 与其后的“ f (n/10);” 的顺序

互掉一下，函数调用 `f(1357)` 又将输出什么结果？

[分析提示]

(1) 先输出 `n` 的个位，再通过“`f(n/10)`”进行递归调用，去处理甩掉个位后的新实参(十位以上构成的新数据)，从而可知，`f` 函数实现的是将 `n` 的各位数字按照反序输出(先个位，再十位，...)。

(2) 若将 `if` 句体中的“`cout<<n%10<<" "`”与其后的“`f(n/10);`”的顺序互换，则先通过递归处理“甩掉”个位后的新实参(十位以上构成的新数据)，最后再输出 `n` 的个位。

[解答程序]

```
#include <iostream.h>

void f(int n);                //函数 f 的原型

void main( )
{
    cout<<"f(1357) =>";
    f(1357);                  //以 1357 为实参，调用函数 f
    cout<<endl;
}

void f(int n)
{
    if(n>0)
    {
        cout<<n%10<<" ";      //先输出 n 的个位
        f(n/10);
        //再以甩掉个位后的新实参(十位以上构成的新数据)，对 f 进行
        //自递归调用
        //注意：执行本句“ f(n/10);”时，仍然先输出其个位，而后才是其各
        //高位数字
        //从而可知，f 函数实现的是将 n 的各位数字按照反序输出(先个位，
        //再十位，...)
    }
}
```

[输出结果]

`f(1357) =>7 5 3 1`

进一步，若将 `if` 句体中的“`cout<<n%10<<" "`”与其后的“`f(n/10);`”的顺序互掉一下，函数调用 `f(1357)` 输出的结果将是：`f(1357) =>1 3 5 7`

```
void f(int n)
{
    if(n>0)
    {
        f(n/10);
        //先以甩掉个位后的新实参(十位以上构成的新数据), 对 f 进行自
        //递归调用
        cout<<n%10<<" ";          //再输出 n 的个位
        //注意: 本句对 n 的个位的输出, 是在执行完上句“ f(n/10);” 之后
        //才进行的
        //即是说, 是在执行上句“ f(n/10);” 之时输出 n 的十位以上各数位数
        //字之后才轮到输出 n 的个位的, 从而可知, 最后输出的是 n 的个位
    }
}
```

5.21 使用递归解法求实数 a 的 n 次方

输入实数 a 及一个正整数 n, 求出 a 的 n 次方。但要求使用递归求解方法。

[分析提示]

可按如下公式来编制递归处理函数:

power(a, n)=a;	若 n 等于 1
power(a, n)=a * power(a, n-1);	若 n 大于 1

[解答程序]

```
#include <iostream.h>

double power(double a, int n)          //递归函数 power, 求出 a 的 n 次方并返回该数
{
    if(n==1) return a;                //power(a,n)=a; 若 n 等于 1
    else return a*power(a, n-1);      //power(a,n)=a*power(a,n-1); 若 n 大于 1
}

void main(void)
{
    double a, res;                    //说明两个 double 型变量
    int n;                            //说明 1 个 int 型变量
    cout<<"Input a double A, and a positive integer N:"; //提示输入
    cin>>a>>n;                       //从键盘输入两个数 a 与 n
}
```

```

    res=power(a,n);           //调用递归函数 power, 求出 a 的 n 次方并返回该数
    cout.precision(15);       //设置输出的有效数字个数为 15
    cout<<"power("<<a<<","<<n<<")="<<res<<endl; //在屏幕上显示出结果
}

```

[输出结果]

```

Input a double A, and a positive integer N:5.5 3
power(5.5, 3)=166.375

```

5.22 使用递归方法打印一批数据

读下述的非递归函数 `myPrint1`, 而后添加一个与 `myPrint1` 具有相同功能的递归函数 `myPrint2`, 并使用下述的主函数对它们进行调用, 以验证其正确性。

```

#include <iostream.h>
#include <math.h>
void myPrint1(int n)
{
    for (int i=n; i>=1; i--)
        cout<<sqrt(i)<<" ";
}

void main(void)
{
    int N;
    cout<<"Input a positive integer N:";           //提示输入正整数 N
    cin>>N;
    cout<<"----- Calling 'myPrint1(N);'-----"<<endl;
    myPrint1(N);                                   //调用非递归函数
    cout<<endl;
    cout<<"----- Calling 'myPrint2(N);'-----"<<endl;
    myPrint2(N);                                   //调用递归函数
    cout<<endl;
}

```

[分析提示]

- (1) 非递归函数 `myPrint1`, 它负责求出 $n, n-1, \dots, 2, 1$ 各自的平方根并显示出来。
- (2) 递归函数 `myPrint2`, 将问题分解处理为: 先求出 n 的平方根并显示出来; 而后进行递归调用 `myPrint2` 求出 $n-1, n-2, \dots, 2, 1$ 各自的平方根并显示出来。

[解答程序]

```

void myPrint2(int n)           //递归函数 myPrint2，参数 n 为正整数
{
    //负责求出 n,n-1,...,2,1 各自的平方根并显示出来
    if(n>=1)                  //若 n 大于等于 1 时进行如下处理
    {
        cout<<sqrt(n)<<" "; //显示出 n 的平方根
        myPrint2(n-1);
        //而后递归调用，负责求出 n-1,n-2,...,2,1 各自的平方根并显示出来
    }
}

```

[输出结果]

```

Input a positive integer N:6
----- Calling 'myPrint1(N);' -----
2.44949  2.23607  2  1.73205  1.41421  1
----- Calling 'myPrint2(N);' -----
2.44949  2.23607  2  1.73205  1.41421  1

```

5.23 输入 10 个整数而后反序输出的实现方式四

读下述程序，并上机进行验证，看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意，本程序使用递归函数中的局部变量存放输入数据。

```

#include <iostream.h>

void func (int n)           //注 1
{
    int i;                  //注 2
    cin>>i;                 //注 3
    if(n>1)                 //注 4
        func(n-1);
    else                    //注 5
        cout<<"---- The result ----"<<endl;
    cout<<i<<" ";         //注 6
}

void main(void)
{
    cout<<"Input 10 integers:"<<endl;
}

```

```
func(10);                                //注 7
cout<<endl;
}
```

[分析提示]

程序说明了一个自递归的函数 `func`，而 `func` 中具有一个局部变量 `i`，输入数据保存在本递归层次中的局部变量 `i` 之中（注意，系统会保障每一递归层次具有一个互不相同的 `i` 变量空间）。而后要使本递归层次局部变量 `i` 的输出是在处理完前 `n-1` 个数据（靠自递归调用语句 `func(n-1)` 完成）之后进行！从而实现了先输入的 `i` 变量之值要在最后才输出的要求。

[解答程序]

具体程序见题目处。

注 1：递归函数 `func`，共处理 `n` 个数。

注 2：局部变量 `i`。

注 3：输入数据保存在本递归层次中的局部变量 `i` 中，注意每一递归层次具有一个互不相同的 `i`。

注 4：若 `n>1`，则先递归处理后 `n-1` 个数据。

注 5：`n` 等于 1 时，输出提示串。

注 6：而后输出本递归层次中的局部变量 `i`。注意，本递归层次局部变量 `i` 的输出是在处理完 `n-1` 个数据（靠调用语句 `func(n-1)` 完成）之后进行的！从而实现了先输入的 `i` 变量之值要在最后才输出的要求。

注 7：调用自递归函数 `func` 来处理 10 个数。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```

5.24 按递归方法求最大公约数

输入两个正整数 `a`、`b`，按递归方法求出它们的最大公约数 `c`。

[分析提示]

求解方法如下：

<code>gcd(m, n) = n;</code>	若 <code>m%n</code> 等于 0
<code>gcd(m, n) = gcd(n, m%n);</code>	若 <code>m%n</code> 不等于 0

[解答程序]

```
#include <iostream.h>

int gcd(int m, int n) //递归函数 gcd,求出正整数 a 与 b 的最大公约数并返回该数
{
    if(m%n==0) return n;           //gcd(m,n)=n;           若 m%n 等于 0
    return gcd(n, m%n);           //gcd(m,n)=gcd(n,m%n);     若 m%n 不等于 0
}

void main(void)
{
    int a,b,c;                     //说明三个 int 型变量
    cout<<"Input 2 positive integers:"; //屏幕提示输入两个正整数
    cin>>a>>b;                     //从键盘输入两个数赋值给 a 与 b 变量(两个数据以空格分割)
    c=gcd(a,b);                     //调用递归函数 gcd, 求出正整数 a 与 b 的最大公约数
    cout<<"gcd("<<a<<","<<b<<")="<<c<<endl;
                                   //在屏幕上显示出 a 与 b 的最大公约数 c
}
```

[输出结果]

```
Input 2 positive integers:20 8
gcd(20,8)=4
```

再一次执行:

```
Input 2 positive integers:17 5
gcd(17,5)=1
```

5.25 读非递归函数后添加具有相同功能的递归函数

读下述的非递归函数 myf1, 而后添加一个与 myf1 具有相同功能的递归函数 myf2, 并使用下述的主函数对它们进行调用, 以验证其正确性。

```
#include <iostream.h>

void myf1(int n1, int n2)
{
    for (int i=n1; i<=n2; i++)
        cout<<"i="<<i<<"," i*i="<<i*i<<"," 1.0/i="<<1.0/i<<endl;
}

void main(void)
```

```

{
    int N1, N2;
    cout<<"Input 2 positive integer N1, N2:";          //提示输入正整数 N1 与 N2
    cin>>N1>>N2;
    cout<<"---- Calling 'myf1(N1, N2);' ----"<<endl;
    myf1(N1, N2);                                       //调用非递归函数 myf1,
    cout<<"---- Calling 'myf2(N1, N2);' ----"<<endl;
    myf2(N1, N2);                                       //调用递归函数 myf2,
}

```

[分析提示]

- (1) 非递归函数 myf1, 负责显示出从 n1 到 n2 这些数本身、其平方以及其倒数。
- (2) 递归函数 myf2, 将问题分解处理为: 先显示出 n1 本身、其平方以及其倒数; 而后进行递归调用 myf2 显示出从 n1+1 到 n2 这些数本身、其平方以及其倒数。

[解答程序]

```

void myf2(int n1, int n2)    //递归函数 myf2, 参数 n1,n2 为正整数
{
    //负责显示出从 n1 到 n2 这些数本身、其平方以及其倒数
    if(n2>=n1)              //若 n2 大于等于 n1 时, 进行如下处理
    {
        cout<<"n1="<<n1<<"", n1*n1="<<n1*n1<<"", 1.0/n1="<<1.0/n1<<endl;
        //显示出 n1 本身、其平方以及其倒数
        myf2(n1+1,n2); //而后递归调用, 负责显示出
        //从 n1+1 到 n2 这些数本身、其平方以及其倒数
    }
}

```

[输出结果]

```

Input 2 positive integer N1, N2:3 6
----- Calling 'myf1(N1, N2);' -----
i=3, i*i=9, 1.0/i=0.333333
i=4, i*i=16, 1.0/i=0.25
i=5, i*i=25, 1.0/i=0.2
i=6, i*i=36, 1.0/i=0.166667
----- Calling 'myf2(N1, N2);' -----
n1=3, n1*n1=9, 1.0/n1=0.333333
n1=4, n1*n1=16, 1.0/n1=0.25
n1=5, n1*n1=25, 1.0/n1=0.2

```

$n1=6$, $n1*n1=36$, $1.0/n1=0.166667$

5.26 求字符串长度的函数

编写一个求字符串长度的函数(不可调用标准库函数 `strlen`)，具有如下原型：

```
int MyStrlen(char s[ ]);
```

参数为字符串 `s`，函数返回值为串 `s` 的长度。并编写主函数，对所编函数进行调用。

[分析提示]

(1) `MyStrlen` 函数中，从头开始逐字符地检验字符数组 `s` 的各分量 `s[i]`，直到遇到了“`\0`”结束字符时，“`\0`”字符以前的字符个数即为所求。

(2) `MyStrlen` 函数的形参为字符型数组，数组作形参时，其大小即分量个数可以省略。

[解答程序]

```
#include <iostream.h>

int MyStrlen(char s[ ])          //返回串 s 的长度
{
    int i=0;
    while(s[i++]);               //从头开始，逐字符地检验 s[i]，直到遇到了“ \0 ”结束字符
    return(i-1);                 //返回的当前的 i-1 即为串 s 的长度
}

void main(void)
{
    char str[80];
    cout<<"Input a string:"<<endl;          //提示输入
    cin.getline(str, 80);                    //从键盘输入一行放入 str 中
    cout<<"str=" " "<<str<<" " "<<endl;
    cout<<"MyStrlen(str)="<<MyStrlen(str)<<endl;
    //以数组名 str 为实参调用自定义函数 MyStrlen，返回字符串 str 的长度
}
```

[输出结果]

```
Input a string:
ABCD 123!
str=" ABCD 123!"
MyStrlen(str)=9
```


5.27 二进制数转化为十进制数的函数

编写一个将二进制数转化为十进制数的函数，具有如下原型：

```
void Chg2To10(char a[ ]);
```

参数为字符串 a，其中存放的是一个二进制数(非“0”即“1”的串)。函数功能为：将 a 中的二进制数转化为十进制数并输出。并编写主函数，对所编函数进行调用。

[分析提示]

(1) Chg2To10 函数中使用的转换方法与范例 4.2 的方法相同。

(2) main 函数中输入多个二进制数字符串，并通过调用 Chg2To10 将它们转化为十进制数。

[解答程序]

```
#include <iostream.h>
#include <string.h>
#include <process.h>
void Chg2To10(char a[ ])          //将二进制数转化为十进制数并输出
{
    int i, value=0, w=1, bitv;
    //value 放累加和, w 为“ 位权” (2 的 0 次方、2 的 1 次方、...)
    for (i=strlen(a)-1; i>=0; i--)    //从后往前逐一处理每一个二进制位
    {
        bitv=a[i];                  //一个二进制位放 bitv
        if (bitv == '1')
            value += w;              //计算“ 位权” 乘以“ 位值”
                                     //并累加到一个初值为 0 的变量 value 上
        else
            if (bitv != '0')         //输入的某位既非“ 0” 又非“ 1” 时，提示出错并退出
            {
                cout<<"Input error!"<<endl;
                exit(0);
            }
        w*=2;                        //使 w 变为下一位的“ 权值”
    }
    cout<<a<<"(2) = "<<value<<"(10)"<<endl; //最后输出该 value
}
```

```
void main(void)
```

```
{
    char a[20];
    cout<<"Input a bit binary number (less than 20 bit, 'q' or 'Q' will ending!): "<<endl;
    cin>>a;          //用字符数组 a 存放刚输入的二进制数(非“ 0” 即“ 1” 的串),
                    //输入“ Q” 时结束程序
    while ( a[0]!='q' && a[0]!='Q')          //非结束字符“ Q” 时继续处理
    {
        Chg2To10(a); //以 a 为实参调用自定义函数 Chg2To10, 并输出转化结果
        cout<<"Input a bit binary number (less than 20 bit, 'q' or 'Q' will ending!): "<<endl;
        cin>>a;          //输入下一个二进制数(非“ 0” 即“ 1” 的串)
    }
}
```

[输出结果]

```
Input a bit binary number (less than 20 bit, 'q' or 'Q' will ending!):
1011
1011(2) = 11(10)
Input a bit binary number (less than 20 bit, 'q' or 'Q' will ending!):
1100
1100(2) = 12(10)
Input a bit binary number (less than 20 bit, 'q' or 'Q' will ending!):
Q
```

5.28 查看数组中满足某条件之数据的个数

编写具有如下原型的函数: `int lessThan(int a[], int n, int value);`

它负责查看数组 `a` 的前 `n` 个元素中共有多少个小于 `value`, 并返回小于 `value` 的数据的个数。而后编写主函数, 对 `lessThan` 进行调用, 以验证其正确性。

[分析提示]

(1) `lessThan` 函数中通过循环查找出数组 `a` 的前 `n` 个元素中共有多少个数小于 `value`。

(2) 主函数中使用不同的实参对 `lessThan` 函数进行调用, 注意第一个实参必须为数组名。

[解答程序]

```
#include <iostream.h>
int lessThan(int a[], int n, int value)
{
```

```

//负责查看数组 a 的前 n 个元素中共有多少个小于 value，并返回小于 value
//的数据的个数
int count=0;                //在 count 上进行累加
for(int i=0; i<n; i++)
    if(a[i]<value) count++;    //判断数据是否小于 value
return count;                //返回 count
}

void main(void)
{
    int a[8]={2, -4, 8, 99, -57, 0, 100, 62}, cnt;
    cnt=lessThan(a, 8, 88);    //查看数组 a 的前八个元素中共有多少个小于 88
    cout<<"cnt=lessThan(a, 8, 88)="<<cnt<<endl;
    cnt=lessThan(a, 8, 0);     //查看数组 a 的前八个元素中共有多少个小于 0
    cout<<"cnt=lessThan(a, 8, 0)="<<cnt<<endl;
}

```

[输出结果]

```

cnt=lessThan (a, 8, 88)=6
cnt=lessThan (a, 8, 0)=2

```

5.29 在升序数组中折半查找某元素序号的函数

编写一个在升序数组中折半查找某元素的序号(下标号)的函数，具有如下原型：

```
int BinSearch(int A[ ], int n, int key);
```

功能：用折半查找方法，在元素呈升序排列的具有 n 个元素的数组 A 中查找值为 key 的那一元素的序号(下标号)并返回该序号，若数组中没有值为 key 的元素时，则返回 -1。

并编写主函数，对所编函数进行调用。

[分析提示]

(1) BinSearch 中使用如下查找方法：先用 key 值与处于 n 个元素之“中间”的那一元素进行比较，若恰好相等，则返回那一“中间”元素的序号而结束处理；若 key 值小于那一“中间”元素，则将查找范围缩小到数组的前一半去继续查找(基于数组已按升序排列)，否则(key 值大于那一“中间”元素)，则将查找范围缩小到数组的后一半去继续查找。

(2) main 中说明具体数组：int a[10]={ -99, -34, -3, 0, 11, 22, 39, 55, 66, 100}; 并使用数组名 a 作为第一实参对 BinSearch 函数进行调用。

[解答程序]

```
#include <iostream.h>

int BinSearch(int A[ ], int n, int key)
{
    //功能：用折半查找方法，在元素呈升序排列的具有 n 个元素的数组 A 中
    //查找值为 key 的那一元素的序号(下标号)并返回该序号
    //若数组中没有值为 key 的元素时，则返回-1
    int low, high, mid;
    //low、high 代表当前查找范围的低端与高端下标值(数组元素序号)
    //mid 表示“ 中间” 元素位置(下标值)
    int midValue;           //midValue 为“ 中间” 元素的值
    low=0;                  //最初要在整个数组范围内查找，查找范围低端为 0 下标号
    high=n-1;               //最初的查找范围高端为 n-1
    while (low<=high)       //low<=high 表示整个数组尚未查找完
    {
        mid=(low+high)/2;   //求中间元素的下标
        midValue=A[mid];   //取出中间元素的值
        if (key==midValue)  //key 值与那一“ 中间” 元素恰好相等
            return mid;    //则返回那一“ 中间” 元素的序号而结束处理
        else
        {
            if (key<midValue) //若 key 值小于那一“ 中间” 元素
                high=mid-1;
                //将查找范围缩小到数组的前一半去继续查找(基于数组已按升序排列)
            else              //否则(key 值大于那一“ 中间” 元素)
                low=mid+1;    //将查找范围缩小到数组的后一半去继续查找
        }
    }
    return -1;              //查找完而没有找到时，返回-1
}

void main( )
{
    int ind, a[10]={ -99, -34, -3, 0, 11, 22, 39, 55, 66, 100};
    ind=BinSearch(a, 10, -3); //调用自定义函数
    //在元素呈升序排列的具有 10 个元素的数组 a 中查找值为-3 的那一元素的序号
    cout<<"element="<<-3;
    if(ind>=0)                //查到时，显示下标号
        cout<<" ind="<<ind<<endl;
    else                       //返回值为负数时，表示没查到
        cout<<" No such element!"<<endl;
```

```
ind=BinSearch(a, 10, 99);  
    //在具有 10 个元素的数组 a 中查找值为 99 的那一元素  
cout<<"element="<<99;  
if(ind>=0)                                //查到时，显示下标号  
    cout<<"  ind="<<ind<<endl;  
else                                        //没查到  
    cout<<"  No such element!"<<endl;  
}
```

[输出结果]

element=-3, ind=2

element=99, No such element!

5.30 显示数组某下标范围内所有元素的递归函数

编制下述的递归函数 printArr，它负责将 a 数组的从下标 firstInd 到下标 lastInd 的所有元素显示出来：void printArr(double a[], int firstInd, int lastInd);

[分析提示]

递归函数 printArr 中使用如下的实现方法：若 firstInd 大于 lastInd，则递归结束；否则，显示出 a[firstInd]后，再进行递归调用（代入实参 a，firstInd+1，以及 lastInd，负责显示出从下标 firstInd+1 到下标 lastInd 的所有元素）。

[解答程序]

```
#include <iostream.h>  
void printArr(double a[], int firstInd, int lastInd)  
{  
    //递归函数 printArr，负责将数组 a 的从下标 firstInd  
    //到下标 lastInd 的所有元素显示出来  
    if(firstInd<=lastInd)  
    {  
        cout<<a[firstInd]<<"  ";          //显示出 a[firstInd]  
        printArr(a, firstInd+1, lastInd);    //而后再递归调用  
        //负责显示从下标 firstInd+1 到下标 lastInd 的所有元素  
    }  
}  
  
void main(void)
```

```
{
    double a[10]={1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7, 8.8, 9.9, 11};
                                //说明数组 a 并置初值
    cout<<"--- Calling 'printArr(a, 0, 9);' ---"<<endl;
    printArr(a, 0, 9);           //调用递归函数 printArr,
                                //显示数组 a 从下标 0 到下标 9 的所有元素

    cout<<endl;
    cout<<"--- Calling 'printArr(a, 2, 7);' ---"<<endl;
    printArr(a, 2, 7);           //调用递归函数 printArr,
                                //显示数组 a 从下标 2 到下标 7 的所有元素

    cout<<endl;
}
```

[输出结果]

```
----- Calling 'printArr(a, 0, 9);' -----
1.1  2.2  3.3  4.4  5.5  6.6  7.7  8.8  9.9  11
----- Calling 'printArr(a, 2, 7);' -----
3.3  4.4  5.5  6.6  7.7  8.8
```

5.31 判断两数组前 n 个元素值是否完全相同的递归 与非递归函数

使用递归与非递归两种方法编制出具有如下原型的函数：

```
bool arrEqual(int a[ ], int b[ ], int n);
```

它负责判断数组 a 与 b 的前 n 个元素值是否按下标对应完全相同，是则返回 true，否则返回 false。并编写主函数对它们进行调用，以验证其正确性。

[分析提示]

(1) 非递归函数中只需逐个地判断各 a[i]是否均与 b[i]相同(i = 0, 1, ..., n-1)。

(2) 递归函数中将问题分解处理为：先判断最后一个元素是否相同，不同则返回 false；相同则看 n 是否等于 1，是则返回 true，否则进行递归调用(代入实参 a、b 与 n-1，去判断前 n-1 个元素的相等性)，并返回递归调用的结果(与前 n-1 个元素的是否相等性相同)。

[解答程序]

```
#include <iostream.h>
bool arrEqual1(int a[ ], int b[ ], int n)    //非递归函数 arrEqual1
{
```

```

//负责判断数组 a 与 b 的前 n 个元素值是否按下标对应完全相同
//是则返回 true, 否则返回 false
for(int i=0; i<n; i++)
    if(a[i]!=b[i])                //若有一个不相同, 则返回 false
        return false;
return true;                      //前 n 个元素全相同时, 返回 true
}

bool arrEqual2(int a[ ], int b[ ], int n)    //递归函数 arrEqual2
{
    //负责判断数组 a 与 b 的前 n 个元素值是否按下标对应完全相同
    //是则返回 true, 否则返回 false
    if (a[n-1]!=b[n-1]) return false;    //若最后一个元素不相同, 则返回 false
    else                                  //最后一个元素相同时
        if(n==1) return true;            //若 n=1(即只判 1 个元素时), 则返回 true
        else                             //最后一个元素相同且 n 不等于 1 时
            return arrEqual2(a, b, n-1);
    //递归调用(只需判断前 n-1 个元素的相等性)
}

void main(void)
{
    int a[6]={2,4,8,9,7,12}, b[6]={2,4,8,9,7,12}, c[6]={2,4,8,9,8,12};
    if(arrEqual1(a,b,6))                //调用非递归函数, 判断 a、b 数组是否完全相同
        cout<<"Yes! arrEqual1(a,b,6)!"<<endl;
    else
        cout<<"No! not arrEqual1(a,b,6)!"<<endl;
    if(arrEqual2(a,b,6))                //调用递归函数, 判断 a、b 数组是否完全相同
        cout<<"Yes! arrEqual2(a,b,6)!"<<endl;
    else
        cout<<"No! not arrEqual2(a,b,6)!"<<endl;
    if(arrEqual1(a,c,6))                //调用非递归函数, 判断 a、c 数组是否完全相同
        cout<<"Yes! arrEqual1(a,c,6)!"<<endl;
    else
        cout<<"No! not arrEqual1(a,c,6)!"<<endl;
    if(arrEqual2(a,c,6))                //调用递归函数, 判断 a、c 数组是否完全相同
        cout<<"Yes! arrEqual2(a,c,6)!"<<endl;
    else
        cout<<"No! not arrEqual2(a,c,6)!"<<endl;
}

```

```
}
```

[输出结果]

Yes! arrEqual1(a,b,6)!

Yes! arrEqual2(a,b,6)!

No! not arrEqual1(a,c,6)!

No! not arrEqual2(a,c,6)!

5.32 判断某数组前 n 个元素是否完全有序的递归与非递归函数

使用递归与非递归两种方法编写出具有如下原型的函数：

```
bool arrIsSorted(int a[ ], int n);
```

它负责判断数组 a 的前 n 个元素是否已经从小到大完全有序了，是则返回 true，否则返回 false。并编写主函数对它们进行调用，以验证其正确性。

[分析提示]

(1) 非递归函数中只需逐对地判断各 a[i] 与 a[i+1] 是否已有序(i = 0, 1, ..., n-2)。

(2) 递归函数中将问题分解处理为：若 n=1(即只有 1 个元素)时，则返回 true；n>1 时，若最后一对元素不顺序则返回 false，否则进行递归调用(代入实参 a 与 n-1，去判断前 n-1 个元素的顺序性)，并返回递归调用的结果(与前 n-1 个元素的是否顺序性相同)。

[解答程序]

```
#include <iostream.h>

bool arrIsSorted1(int a[ ], int n)           //非递归函数 arrIsSorted1
{
    //负责判断数组 a 的前 n 个元素是否已经从小到大完全有序了
    //是则返回 true，否则返回 false
    for(int i=0; i<n-1; i++)                 //从前往后共比较 n-1 对
        if(a[i]>a[i+1]) return false;        //若有某“一对”前大后小时，则返回 false
    return true;                             //所有 n-1 对都从小到大有序时，返回 true
}

bool arrIsSorted2(int a[ ], int n)           //递归函数 arrIsSorted2
{
    //负责判断数组 a 的前 n 个元素是否已经从小到大完全有序了
    //是则返回 true，否则返回 false
    if(n==1) return true;                   //若 n=1(即只有 1 个元素) 时，则返回 true
```



```

else //至少有两个元素时
    if (a[n-2]>a[n-1]) return false; //若最后一对元素不顺序, 则返回 false
    else return arrIsSorted2(a, n-1);
    //最后一对元素已顺序时, 递归调用(只需判断前 n-1 个元素的顺序性)
}

void main(void)
{
    int a[6]={2,4,8,8,10,12}, b[6]={2,4,8,7,10,12};
    if(arrIsSorted1(a,6)) //调用非递归函数, 判断 a 数组的前六个元素是否已经从小到大有序
        cout<<"Yes! arrIsSorted1(a,6)!"<<endl;
    else
        cout<<"No! not arrIsSorted1(a,6)!"<<endl;
    if(arrIsSorted2(a,6)) //调用递归函数, 判断 a 数组的前六个元素是否已经从小到大有序
        cout<<"Yes! arrIsSorted2(a,6)!"<<endl;
    else
        cout<<"No! not arrIsSorted2(a,6)!"<<endl;
    if(arrIsSorted1(b,6)) //调用非递归函数, 判断 b 数组的前六个元素是否已经从小到大有序
        cout<<"Yes! arrIsSorted1(b,6)!"<<endl;
    else
        cout<<"No! not arrIsSorted1(b,6)!"<<endl;
    if(arrIsSorted2(b,6)) //调用递归函数, 判断 b 数组的前六个元素是否已经从小到大有序
        cout<<"Yes! arrIsSorted2(b,6)!"<<endl;
    else
        cout<<"No! not arrIsSorted2(b,6)!"<<endl;
}

```

[输出结果]

```

Yes! arrIsSorted1(a,6)!
Yes! arrIsSorted2(a,6)!
No! not arrIsSorted1(b,6)!
No! not arrIsSorted2(b,6)!

```

5.33 判断 s 字符串是否为“回文”的递归函数

编写具有如下函数原型的递归函数 `myfunc`, 它负责判断 s 字符数组的从 `firstInd` 下标到 `lastInd` 下标所构成的那一段字符是否为“回文”(是否正读与反读完全相同), 是则返回 `true`, 否则返回 `false`。

```
bool myfunc(char s[ ], int firstInd, int lastInd);
```

并编制主函数，由用户任意输入一些字符串，而后通过调用 myfunc 来判断它们是否为“回文”。

[分析提示]

递归函数 myfunc 中将问题分解处理为：若“lastInd<=firstInd”（即 s 串只有一个字符或根本没字符）时，则返回 true 而递归出口；s 串至少有两个字符时，若“最两头”的两个字符不相同，则整个 s 串非“回文”，要返回 false，否则进行递归调用（传入实参 s、firstInd+1 与 lastInd-1，去判断甩掉“最两头”两个字符后的“小串”字符是否为“回文”），并返回递归调用的结果（与原串的是否性相同）。

[解答程序]

```
#include <iostream.h>
#include <string.h>

bool myfunc(char s[ ], int firstInd, int lastInd)
{
    //递归函数 myfunc，它负责判断 s 字符数组的从 firstInd 下标到 lastInd 下标
    //所构成的那一段字符是否为“回文”（是否正读与反读完全相同），
    //是则返回 true，否则返回 false

    if(lastInd<=firstInd) return true;
        //s 串只有一个字符或根本没字符时，返回 true，递归出口
    else //s 串含有两个以上字符时
    {
        if( s[firstInd] != s[lastInd] ) return false;
        //若“最两头”的两字符不相同，整个 s 串非“回文”，返回 false
        else //“最两头”的两字符相同时，甩掉它们后，
            //剩余“小串”的是否性与“大串”相同
            return myfunc(s, firstInd+1, lastInd-1);
        //递归调用，负责判断 s 字符数组的从 firstInd+1 下标到 lastInd-1 下标所构成
        //的那一“小串”字符是否为“回文”若是，则“大串”也是，
        //则返回 true，否则返回 false
    }
}

void main(void)
{
    char s[81]; //s 存放由用户任意输入的字符串
    cout<<"Input a string:"; //提示输入一个字符串
    cin>>s;
    int lastInd=strlen(s)-1; //lastInd 为 s 串中所含最后一个字符的下标
```

```

bool res=myfunc(s, 0, lastInd);
    //调用递归函数 myfunc, 它负责判断 s 字符数组的从下标 0
    //到下标 lastInd 所构成的那一段字符是否为“回文”
    //(是否正读与反读完全相同), 是则返回 true, 否则返回 false
if(res)                                //调用结果为 true 时
    cout<<s<<" is a palindrome!"<<endl;
else                                    //返回 false 时
    cout<<s<<" is not a palindrome!"<<endl;
}

```

[输出结果]

```

Input a string:asdfsa
asdfsa is not a palindrome!

```

再一次执行:

```

Input a string:abc1221cba
abc1221cba is a palindrome!

```

5.34 函数的间接递归

读下面的程序, 其中出现了函数的间接递归调用, 请写出程序执行后的屏幕显示结果。进一步, 若把函数 f1 的引用型参数“int& x”改为赋值参数“int x”的话, 其执行结果又会如何?

```

#include <iostream.h>

int times;                                //注 1
void f1(int& x);                          //注 2
void f2(int& y);                          //注 3

void main( )
{
    int a=88;
    times=0;                              //注 4
    f1(a);                                //注 5
    cout<<"times="<<times<<endl;         //注 6
    cout<<"a="<<a<<endl;                //注 7
}

void f1(int& x)

```

```
{
    times++;                //注 8
    if (x<100)              //注 9
    {
        x+=9;              //注 10
        f2(x);              //注 11
    }                        //注 12
}

void f2(int& y)
{
    y++;                    //注 13
    f1(y);                  //注 14
}
```

[分析提示]

(1) f1 函数调用 f2 函数，而 f2 函数又调用 f1 函数，从而，形成间接递归。注意，当调用 f1 时带去的实参 x 不小于 100 时，将不再进行间接递归调用，结束递归。

(2) 函数的引用参数是其对应实参变量的“替身”(注意，引用参数的对应实参必须是变量!)，从而主调函数中可以通过实参变量来获取被调函数内对引用形参的更改值。

[解答程序]

具体程序见题目处。

注 1: 全局变量 times，记录函数 f1 被调用的次数。

注 2: f1 的函数原型，使用的是引用参数。

注 3: f2 的函数原型，使用引用参数。

注 4: 全局变量 times 的初值置为 0。

注 5: 变量 a 作实参调用 f1。由于 f1 使用的是引用参数，f1 中对形参 x 值的更改将通过 a 被“带回来”。

注 6: 输出全局变量 times 的值。

注 7: 输出通过引用参数更改后的 a 值。

注 8: 记录 f1 被调用次数的全局变量 times 增加 1。

注 9: 若 x<100 调用 f2，而 f2 又调用 f1，将形成间接递归。

注 10: 对引用参数 x 进行更改。

注 11: 以更改后的 x 作为实参调用 f2，而 f2 又调用 f1。

注 12: 若 x 大于等于 100 时，则什么事也不做而结束 f1，递归结束。

注 13: 对引用参数 y 进行更改。

注 14: 以更改后的 y 为实参调用 f1。

[输出结果]

```
times=3
```

```
a=108
```

把 f1 的引用型参数改为赋值参数的话，其执行结果如下：

```
times=3
```

```
a=88
```

5.35 两个二进制正整数的四则运算

输入两个二进制正整数(两个非“0”即“1”的串)，而后输入一个运算符(“+”、“-”、“*”、“/”)，按 int 型整数运算方式求出这两个二进制整数的运算结果并输出。

例如，程序执行后的屏幕显示结果可以呈现如下的形式：

```
Input first binary number(less than 20 bit):101
```

```
101(2) = 5(10)
```

```
Input second binary number(less than 20 bit):110
```

```
110(2) = 6(10)
```

```
Input a operator('+', '-', '*', '/):+
```

```
101(2) + 110(2) = 1011(2)
```

```
5(10) + 6(10) = 11(10)
```

[分析提示]

(1) 改造原有十进制数转化为二进制数以及二进制数转化为十进制数的函数，使其具有如下原型：

```
int Chg2To10(char a[ ]);
```

//将 a 字符串中的二进制数转化为十进制数并输出，且返回该十进制数

```
void Chg10To2(int n, char result[ ]);
```

//将十进制数 n 化为二进制数并通过 result 返回该二进制数

(2) 编写主函数，通过调用 Chg2To10 先将两个输入串转化为十进制数，而后对这两个十进制数进行指定运算，再将运算结果通过 Chg10To2 转化为二进制数后输出。

[解答程序]

```
#include <iostream.h>
```

```
#include <string.h>
```

```
#include <process.h>
```

```
#include <math.h>
```

```
int Chg2To10(char a[ ])
```

```
{
```

```

//将 a 字符串中的二进制数转化为十进制数并输出，且返回该十进制数
int i, value=0, w=1, bitv;
//value 放累加和, w 为位权(2 的 0 次方、2 的 1 次方、...)
for (i=strlen(a)-1; i>=0; i--) //从后往前逐一处理每一个二进制位
{
    bitv=a[i]; //一个二进制位放 bitv
    if (bitv == '1') value += w;
    //计算位权乘以位值，并累加到一个初值为 0 的变量 value 上
    else
        if(bitv != '0')
        {
            //输入的某位既非“ 0” 又非“ 1” 时，提示出错并退出
            cout<<"Input error!"<<endl;
            exit(0);
        }
    w*=2; //使 w 变为下一位的权值
}
cout<<a<<"(2) = "<<value<<"(10)"<<endl; //最后输出该 value
return value; //返回该十进制数 value
}

void Chg10To2(int n, char result[ ])
{
    //将十进制数 n 化为二进制数并通过 result 返回该二进制数
    //十进制数转化为二进制数方法：“逐次除以 2，倒者排余数”。
    //用 int 型数组 res 存放逐次得到的各余数
    //最后将数组 res 中的各余数倒着存入到 result 结果数组中
    if(n==0)
    {
        result[0]='0';
        result[1]='\0';
        return;
    }
    int res[20], ind=0, n0=n; //用 int 型数组 res 存放逐次得到的各余数
    //ind 初值为 0(从 0 下标开始往 res 中放各余数)
    while (n) //逐次除以 2，将余数放在数组 res 中(按 ind 下标位置放)
    {
        res[ind++]=n%2+48; //每次下推 ind 下标位置
        n=n/2; //将 n 除以 2(取整)仍放回 n
    }
}

```

```

    }
    res[ind]='\0';           //放一个串结束符
    int i=0;
    while (--ind>=0)         //将数组 res 中的各余数倒着存入到 result 结果数组中
        result[i++]=res[ind];
    result[i]='\0';          //放一个串结束符
}

void main(void)              //主函数
{
    char a1[20], a2[20], op;
    cout<<"Input first binary number(less than 20 bit):";
    cin>>a1;                  //用字符数组 a1 存放输入的第二个二进制数(非“ 0” 即“ 1” 的串)
    int i1=Chg2To10(a1);
        //以 a1 为实参调用自定义函数 Chg2To10, 将其转化为十进制数
    cout<<"Input second binary number(less than 20 bit):";
    cin>>a2;                  //用字符数组 a2 存放输入的第二个二进制数(非“ 0” 即“ 1” 的串)
    int i2=Chg2To10(a2);
        //以 a2 为实参调用自定义函数 Chg2To10, 将其转化为十进制数
    cout<<"Input a operator('+', '-', '*', '/):"; //提示输入运算符
    cin>>op;                  //运算符为 op
    int res10;                //两数的十进制数运算结果
    char res2[20];            //两数的二进制数运算结果
    switch (op)
    {
        case '+':             //两数相加
        {
            res10=i1+i2;       //两数的十进制数运算结果 res10
            Chg10To2(res10, res2);
            //将十进制数 res10 化为二进制数, 并通过 res2 返回该二进制数
            cout<<a1<<"(2) + "<<a2<<"(2) = "<<res2<<"(2)"<<endl;
            cout<<i1<<"(10) + "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
            break;
        }
        case '-':             //两数相减
        {
            res10=i1-i2;       //两数的十进制数运算结果 res10
            Chg10To2(abs(res10), res2);
            //将十进制数 res10 的绝对值转化为二进制数, 并通过 res2 返回该

```

```

//二进制数
cout<<a1<<"(2) — "<<a2<<"(2) = ";
if(res10>=0) cout<<res2<<"(2)"<<endl;
else      cout<<"—"<<res2<<"(2)"<<endl;
cout<<i1<<"(10) — "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
break;
}
case '*':          //两数相乘
{
    res10=i1*i2;          //两数的十进制数运算结果 res10
    Chg10To2(res10, res2);
    //将十进制数 res10 转化为二进制数，并通过 res2 返回该二进制数
    cout<<a1<<"(2) * "<<a2<<"(2) = "<<res2<<"(2)"<<endl;
    cout<<i1<<"(10) * "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
    break;
}
case '/':          //两数相除
{
    res10=i1/i2;          //两数的十进制数运算结果 res10
    Chg10To2(res10, res2);
    //将十进制数 res10 化为二进制数，并通过 res2 返回该二进制数
    cout<<a1<<"(2) / "<<a2<<"(2) = "<<res2<<"(2)"<<endl;
    cout<<i1<<"(10) / "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
    break;
}
} // switch 结束
} //main 结束

```

[输出结果]

```

Input first binary number(less than 20 bit):101
101(2) = 5(10)
Input second binary number(less than 20 bit):110
110(2) = 6(10)
Input a operator('+', '-', '*', '/):+
101(2) + 110(2) = 1011(2)
5(10) + 6(10) = 11(10)

```

再一次执行：

```

Input first binary number(less than 20 bit):101

```



```

101(2) = 5(10)
Input second binary number(less than 20 bit):110
110(2) = 6(10)
Input a operator('+', '-', '*', '/):*
101(2) * 110(2) = 11110(2)
5(10) * 6(10) = 30(10)

```

5.36 两个罗马数据的四则运算

输入两个罗马数据(两个使用如下七个基值字母——M, D, C, L, X, V, I——组成的串, 各字母分别用来表示 1 000、500、100、50、10、5、1), 而后输入一个运算符(“ +”、“ -”、“ *”、“ /”), 按 int 型整数运算方式求出这两个罗马数据的运算结果并输出。

例如, 程序执行后的屏幕显示结果可以呈现如下的形式:

```

Input first Roman_number(only use--M,D,C,L,X,V,I):XXI
XXI(Rom) = 21(10)
Input second Roman_number(only use--M,D,C,L,X,V,I):VII
VII(Rom) = 7(10)
Input a operator ('+', '-', '*', '/):*
XXI(Rom) * VII(Rom) = CXXXXVII(Rom)
21(10) * 7(10) = 147(10)

```

[分析提示]

(1) 构造一个将罗马数据转化为十进制数以及将十进制数转化为罗马数据的函数, 使其具有如下原型:

```

int ChgRomTo10(char a[ ]);
    //将 a 数组中的罗马数据转化为十进制数并输出, 且返回该十进制数
void Chg10ToRom(int n, char result[ ]);
    //将十进制数 n 转化为罗马数据并通过 result 返回该罗马数据

```

(2) 编制主函数, 通过调用 ChgRomTo10 先将两个罗马数据转化为十进制数, 而后对这两个十进制数进行指定运算, 再将运算结果通过 Chg10ToRom 转化为罗马数据后输出。

[解答程序]

```

#include <iostream.h>
#include <math.h>
int ChgRomTo10(char a[ ])
{

```

```
//将 a 中的罗马数据转化为十进制数并输出，且返回该十进制数
int i=0, value=0;           //value 放累加和
char baseCh;                //baseCh 放基值字母
//字符数组 a 中存放的是罗马数据(使用七个基值字母组合成的字符串)
while (a[i])                //从前往后逐一处理每一个基值字母
{
    baseCh=a[i++];          //baseCh 为基值字母，使用完 i 后下推 1
    switch(baseCh)
    {
        case 'M':            //基值字母 M 表示 1 000
            value += 1000;    //将基值字母所表示的数量累加到变量 value 上
            break;
        case 'D':            //基值字母 D 表示 500
            value += 500;     //将基值字母所表示的数量累加到变量 value 上
            break;
        case 'C':            //基值字母 C 表示 100
            value += 100;
            break;
        case 'L':            //基值字母 L 表示 50
            value += 50;
            break;
        case 'X':            //基值字母 X 表示 10
            value += 10;
            break;
        case 'V':            //基值字母 V 表示 5
            value += 5;
            break;
        case 'I':            //基值字母 I 表示 1
            value += 1;
            break;
        default:              //非七个基值字母时提示出错
            cout<<"Wrong! baseCh="<<baseCh<<endl;
            break;
    }
}
cout<<a<<"(Rom) = "<<value<<"(10)"<<endl;
//最后输出化好的十进制数 value
return value;               //返回该十进制数 value
}
```



```

int i2=ChgRomTo10(a2);
    //以 a2 为实参调用自定义函数 ChgRomTo10，将其转化为十进制数
cout<<"Input a operator ('+', '-', '*', '/):"; //提示输入运算符
cin>>op; //运算符为 op
int res10; //两数的十进制数运算结果
char resRom[30]; //两数的罗马数据运算结果
switch (op)
{
    case '+': //两数相加
    {
        res10=i1+i2; //两数的十进制数运算结果 res10
        Chg10ToRom(res10, resRom);
        //将十进制数 res10 转化为罗马数据，并通过 resRom 返回该罗马数据
        cout<<a1<<"(Rom) + "<<a2<<"(Rom) = "<<resRom<<"(Rom)"<<endl;
        cout<<i1<<"(10) + "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
        break;
    }
    case '-': //两数相减
    {
        res10=i1-i2; //两数的十进制数运算结果 res10
        Chg10ToRom(abs(res10), resRom);
        //将十进制数 res10 转化为罗马数据，并通过 resRom 返回该罗马数据
        cout<<a1<<"(Rom) - "<<a2<<"(Rom) = ";
        if(res10>=0) cout<<resRom<<"(Rom)"<<endl;
        else cout<<"-"<<resRom<<"(Rom)"<<endl;
        cout<<i1<<"(10) - "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
        break;
    }
    case '*': //两数相乘
    {
        res10=i1*i2; //两数的十进制数运算结果 res10
        Chg10ToRom(res10, resRom);
        //将十进制数 res10 转化为罗马数据，并通过 resRom 返回该罗马数据
        cout<<a1<<"(Rom) * "<<a2<<"(Rom) = "<<resRom<<"(Rom)"<<endl;
        cout<<i1<<"(10) * "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
        break;
    }
    case '/': //二数相除
    {

```

```

        res10=i1/i2;                //二数的十进制数运算结果 res10
        Chg10ToRom(res10, resRom);
        //将十进制数 res10 化为罗马数据并通过 resRom 返回该罗马数据
        cout<<a1<<"(Rom) / "<<a2<<"(Rom) = "<<resRom<<"(Rom)"<<endl;
        cout<<i1<<"(10) / "<<i2<<"(10) = "<<res10<<"(10)"<<endl;
        break;
    }
}
}

```

[输出结果]

Input first Roman_number(only use--M,D,C,L,X,V,I):XXI

XXI(Rom) = 21(10)

Input second Roman_number(only use--M,D,C,L,X,V,I):VII

VII(Rom) = 7(10)

Input a operator ('+', '-', '*', '/):*

XXI(Rom) * VII(Rom) = CXXXXVII(Rom)

21(10) * 7(10) = 147(10)

再一次执行:

Input first Roman_number(only use--M,D,C,L,X,V,I):DCLXXVIII

DCLXXVIII(Rom) = 678(10)

Input second Roman_number(only use--M,D,C,L,X,V,I):MDCCCCLXXVI

MDCCCCLXXVI(Rom) = 1976(10)

Input a operator ('+', '-', '*', '/):+

DCLXXVIII(Rom) + MDCCCCLXXVI(Rom) = MMDCLIII(Rom)

678(10) + 1976(10) = 2654(10)

5.37 将 ASCII 码数字字符串转化为整数的函数

编写一个函数，将 ASCII 码的可形成一个无符号正整数的数字字符串（如“13286”等）转化为一个 int 型整数并返回。其函数原型如下：int AscToInt(char a[]);

参数 a 为字符数组，其中存放的是一个 ASCII 码的数字字符串(可形成一个无符号正整数的串)。函数功能为：将 a 中的 ASCII 码的数字字符串“拼装”成一个 int 型整数并返回。

并编写主函数，对所编函数进行调用，而后将结果数据输出。

[分析提示]

(1) 在 AscToInt 函数中, 从 a 数组的末位数字字符(也即个位对应的数字字符)开始, 依次往左进行处理, 最终“拼装”出一个所需的 int 型整数 num 并返回。

(2) 往初值为 0 的 num 上进行“拼装”时所使用的主要操作为: “num+=(a[lastInd]-‘0’)*weight;”, 其中的 lastInd 总代表 a 数组当前要被处理的那一末位数字字符的下标位置, 而 weight 代表位权(依次取值 1、10、100、..., 表示个位、十位、百位、...), 从后往前每处理一个字符后, 执行一次操作“lastInd--; weight*=10;”。

[解答程序]

```
#include <iostream.h>
#include <string.h>
int AscToInt(char a[ ])
{
    //将 a 中的 ASCII 码的数字字符串“拼装”成一个 int 型整数并返回
    int lastInd=strlen(a)-1;
    //lastInd 为被“拼装”数的最末一个数字字符位置(下标值)
    int num=0, weight=1;          //“拼装”起一个整数放 num
    //weight 为位权(依次取值 1、10、100、..., 表示个位、十位、百位、...)
    //从字符数组 a 中, 由后往前逐字符地取出数字字符并“拼装”它们
    while (lastInd>=0 && a[lastInd]>='0' && a[lastInd]<='9')
    {
        //尚未“拼装”完所有数字字符时要继续处理
        //逐一从 lastInd 下标处取出数字字符并“拼装”到 num 变量中
        num+=(a[lastInd]-'0')*weight;
        //注意 a[lastInd]-'0'为数字字符对应的数值
        lastInd--;                //准备处理前一字符
        weight*=10;               //前一数字字符的位权增大 10 倍
    }
    return num;                  //返回“拼装”结果 num
}

void main( )
{
    char a[15];                  //用字符数组 a 存放刚输入的 ASCII 码的数字字符串
    cout<<"Input a ASCII-positive-integer('q' or 'Q' will ending!):";
    cin>>a;                      //输入 ASCII 码的数字字符串, 输入“Q”时结束程序
    while ( a[0]!='q' && a[0]!='Q' ) //非结束字符“Q”时继续处理
    {
        int res=AscToInt(a);
```

```

        //以 a 为实参调用自定义函数 AscToInt, 并返回“ 拼装” 结果
        cout<<" "<<a<<"\"(ASCII) = "<<res<<"(int)"<<endl; //输出相关结果
        cout<<"Input a ASCII-positive-integer('q' or 'Q' will ending!):";
        cin>>a;                //输入下一个 ASCII 码的数字字符串
    }
}

```

[输出结果]

```

Input a ASCII-positive-integer('q' or 'Q' will ending!):123
"123"(ASCII) = 123(int)
Input a ASCII-positive-integer('q' or 'Q' will ending!):31970
"31970"(ASCII) = 31970(int)
Input a ASCII-positive-integer('q' or 'Q' will ending!):q

```

5.38 将整数转化为 ASCII 码数字字符串的函数

编写函数, 将一个整数转化为与其等值的 ASCII 码的数字字符串 (例如要将整数“ 13286”转化为数字字符串“ 13286”, 而将负整数“ -1892”转化为数字字符串“ -1892”等), 并将结果字符串放于参数数组中返回。其函数原型如下: void IntToAsc(int n, char a[]);

参数 n 为欲转化的整数, 而字符数组 a 为存放结果 ASCII 码数字字符串的数组。函数功能为: 将整数 n 转化为与其等值的 ASCII 码的数字字符串放于参数数组 a 中返回。

并编写主函数, 对所编函数进行调用, 而后将结果数据输出。

[分析提示]

在 IntToAsc 函数中, 设一个 int 型数组 res, 用于存放依次计算出的 n 的个位、十位、百位、... (将 n 逐次除以 10, 将余数放在数组 res 中)。而后再从后往前把 res 中的诸数据一一放入参数数组 a 之中 (n 的高位处于 a 的前面, 而个位则处于最后, 这正是准备返回去的那一转化成功的 ASCII 码数字字符串)。

[解答程序]

```

#include <iostream.h>

void IntToAsc(int n, char a[ ])
{
    //将整数 n 转化为与其等值的 ASCII 码的数字字符串放于参数数组 a 中返回
    if (n==0)                //n=0 的处理
    {
        a[0]='\0'; a[1]='\0'; return;
    }
}

```

```

    }
    char sign='+';           //认为无符号数均为正数
    if (n<0)                 //负数时，sign 值置为“ - ”
    {
        sign='-';  n=-n;
    }
    int res[15], ind=0, n0=n;
        //用 int 型数组 res 依次存放 n 的个位、十位、百位、...
        //ind 初值为 0(从 0 下标开始往 res 中放置数据 -- n 的个位、十位、百位、...)
    while (n) //逐次除以 10,将余数放在数组 res 中(按 ind 下标位置从 0 依次
        //往后存放)数组 res 中依次存放 n 的个位、十位、百位、...
    {
        res[ind++]=n%10;      //每次下推 ind 下标位置
        n=n/10;              //将 n 除以 10(取整)仍放回 n
    }
    int j=0;
    if (sign=='-') a[j++]='-'; //负数时，结果串的第一个符号为‘ - ’
    while (--ind>=0)
        a[j++]=res[ind]+'0';
        //将数组 res 中的各余数(n 的个位、十位、百位、...)倒着放入结果数组 a 中
    a[j]='\0';                //往结果数组 a 中放一个结束符
}

void main(void)
{
    int n;  char a[15];
    cout<<"Input a integer:"; //提示输入一个整数
    cin>>n;                    //输入的整数放 n 中
    IntToAsc(n, a);
        //以 n 为实参调用自定义函数 IntToAsc，并将转化结果放入 a 中
    cout<<n<<"(int) = "<<"\"<<a<<"\"(ASCII)"<<endl; //输出相关结果
}

```

[输出结果]

```

Input a integer:31690
31690(int) = "31690"(ASCII)

```

再一次执行:

```

Input a integer:-13825

```


— 13825(int) = "— 13825"(ASCII)

5.39 实现插入排序的 InsertionSort 函数

读下述程序，写出执行结果。

```
#include <iostream.h>

void InsertionSort(int A[ ], int n)           //注 1
{
    int i, ind, temp;                         //注 2
    for (i=1; i<n; i++)                      //注 3
    {                                         //注 4
        ind=i;                               //注 5
        temp=A[i];                          //注 6
        while (ind>0 && temp<A[ind-1])      //注 7
        {
            A[ind]=A[ind-1];                //注 8
            ind--;
        }
        A[ind]=temp;                        //注 9
    }
}

void main( )
{
    int a[10]={22, 11, 39, 0, -34, 100, -3, 66, 39, -99};
    InsertionSort(a, 10);                   //注 10
    for (int i = 0; i < 10; i++)
        cout<<a[i]<<" ";                  //注 11
    cout<<endl;
}
```

[分析提示]

(1) InsertionSort 函数的功能为：用直接插入排序法按从小到大的顺序对数组 a 中的前 n 个元素进行排序，并将结果通过数组参数 a 返回到主调函数中。

(2) 直接插入排序的思想是：总在已有序的前 n 个数据中（n 最初为 1、而后是 2、...）找到第 n+1 个数据应插入的位置，而后将那一位置腾空（通过将元素逐个后移的方法），再把第 n+1 个数据插入进去（从而保障了前 n+1 个数据的有序性）。

[解答程序]

具体程序见题目处。

注 1: 该函数实现的是用直接插入排序法对数组 A 中的前 n 个元素进行升序排列。

注 2: 将下标为 1 到 n-1 的元素逐个插入到已排序序列中的适当位置, ind 将记录该位置(注意: 插入下标为 1 的元素时, 认为 0 下标位置处的 1 个元素已有序)。

注 3: i=1 的循环使 0、1 下标的两个元素排好序; i=2 的循环使 0、1、2 下标的三个元素排好序; ...。

注 4: 从 A[i-1] 开始“倒着”向 A[0] 方向扫描各元素(元素值会越来越小), 寻找插入 A[i] 的适当位置。

注 5: ind 将记录插入 A[i] 的适当位置, 先认为该位置就是 i。

注 6: temp 即为要插入的元素 A[i]。

注 7: 逐个比较, 直到要插入元素 temp>=A[ind-1] 时, ind 便是应插入的位置。若到达 ind 等于 0, 则 0 是应插入的位置。

注 8: 将“大个”元素逐个后移, 以便找到插入位置时(该位置已腾空)可立即插入。

注 9: 已找到插入位置 ind, 立即将 temp 插入到该位置。

注 10: 用直接插入排序法对数组 a 中的 10 个元素进行升序排列。

注 11: 输出排序后的 a 数组各元素。

[输出结果]

-99 -34 -3 0 11 22 39 39 66 100

5.40 分析递归函数的执行结果

读下述具有递归函数的程序, 分析并给出执行结果。

```
#include <iostream.h>
void func(int A[ ], int count);           //注 1
void main(void)
{
    int A[10]={22, 1, 5, 3, 99, 28, -2, -9, 10, 6};
    func(A, 8);                           //注 2
    for (int i=0; i<10; i++)
        cout<<A[i]<<" ";                //注 3
    cout<<endl;
}

void func(int A[ ], int count)            //注 4
{
```

```
        if (count>1)
            func(A, count-1);           //注 5
        int i=count-1,
        LastElem=A[i];                  //注 6
        while (i>0)                     //注 7
            if(LastElem<A[i-1])
                A[i]=A[i-1];            //注 8
            else
                break;                  //注 9
        A[i]=LastElem;                  //注 10
    }
```

[分析提示]

(1) func 函数为一个递归函数,它的功能为:按从小到大的顺序对数组 A 中的前 count 个元素进行排序,并将结果通过数组参数 A 返回到主调函数中。

(2) 在 func 函数中,首先通过“ func(A, count-1); ”的自递归调用来完成将 A 数组中前 count-1 个元素从小到大排好序的任务;而后再将最后一个元素(第 count 个元素)插入到前 count-1 个已有序数据的某个适当位置处(从后往前找该位置)。

[解答程序]

具体程序见题目处。

注 1: 函数原型。

注 2: 调用自定义函数 func,代入实参 A 与 8,将 A 数组中的前八个元素进行排序(升序排放)。

注 3: 输出经过排序后的 A 数组的各元素。

注 4: 将 A 数组中的前 count 个元素进行排序(升序排放)。

注 5: 若 count>1 时,先处理 A 的前 count-1 个元素。即是说,通过递归调用,先将 A 数组中的前 count-1 个元素进行排序(升序排放)。该 if 语句执行结束后,前 count-1 个元素已按升序排放好了。剩下要做的是,将第 count 个元素插入到某个适当的位置(从后往前找该位置)。

注 6: LastElem 即第 count 个元素。

注 7: 从后往前找第 count 个元素 LastElem 应插入的位置。

注 8: 若 LastElem 小,前一个大者 A[i-1]往后“平移”,腾出一个空位。注意 A[i-1]等同于 A[(i--)-1],也即等同于 A[i-1],i--。

注 9: 已找到 LastElem 应插入的位置 i。

注 10: 将第 count 个元素 LastElem 插入到该位置。

[输出结果]

-9 -2 1 3 5 22 28 99 10 6

5.41 对两个字符数组实现某些统计功能的函数

编写具有如下原型的函数：void myFunc(char str1[], char str2[]);

实现下述的统计功能，并输出各统计结果：

- (1) 数组 str1 中出现了哪些字母(不区分大小写)？
 - (2) 数组 str1 中哪一个字母(不区分大小写)出现次数最多(注，若多个字母次数都相同时，输出任一个均可)？共出现了几次？
 - (3) 在两个数组中同时出现的有哪些字母(不区分大小写)？
- 并编写主函数，调用 myFunc，以验证其正确性。

[分析提示]

- (1) 在 myFunc 函数中说明两个数组：int count1[26], count2[26];

使用 count1 记录 str1 各字母的出现次数，使用 count2 记录 str2 各字母的出现次数。具体地说，count1[0]与 count2[0]分别用来记录两个不同串中“ A ”字母(不分大小写)的出现次数，count1[1]与 count2[1] 分别用来记录“ B ”字母(不分大小写)的出现次数，...

- (2) 本题所要求出的三种统计结果数据，都是在上述 count1 以及 count2 的数据基础上而形成的。如，当 count1[i]非 0 时，意味着 str1 中出现了相应的字母“ a ”+i；又如，当 count1[i]与 count2[i]都非 0 时，意味着两个数组中同时具有 i 对应的那一字母“ a ”+i 等。

[解答程序]

```
#include <iostream.h>

bool isLetter(char ch)           //判断 ch 是否为字母，若是返回 true，否则返回 false
{
    if(ch>='A' && ch<='Z' || ch>='a' && ch<='z') return true;
    return false;
}

void myFunc(char str1[ ], char str2[ ]) //实现指定的统计功能，并输出各统计结果
{
    cout<<"-----"<<endl;
    cout<<"str1="<<str1<<endl;
    cout<<"str2="<<str2<<endl;
    int count1[26], count2[26];
    //count1 中记录 str1 各字母的出现次数，count2 中记录 str2 各字母的
    //出现次数具体说，count1[0]与 count2[0]都用来记录“ A ”字母(不分大小写)
    //的出现次数 count1[1]与 count2[1]都用来记录“ B ”字母的出现次数...
    for (int i=0; i<26; i++)
```

```

{
    count1[i]=0;                //作为累加和的各 count1[i]均先置为 0
    count2[i]=0;                //作为累加和的各 count2[i]均先置为 0
}
i=0;
while (char ch=str1[i++])      //将 str1 数组的各字符从头到尾遍写一遍
    if(isLetter(ch))           //若字符为字母
        if(ch>='a')            //小写字母
            count1[ch-'a']++;   //count1 数组中记录 str1 各字母的出现次数
        else count1[ch-'A']++;  //对大写字母的处理
i=0;
while (char ch=str2[i++])      //将 str2 数组的各字符从头到尾遍写一遍
{
    if(isLetter(ch))           //若字符为字母
    {
        if(ch>='a') count2[ch-'a']++; //小写字母
        else count2[ch-'A']++;       //大写字母
    }
}
//(1) 数组 str1 中出现了哪些字母(不区分大小写)
cout<<"-----"<<endl;
cout<<"The letters in str1:"<<endl;
for (i=0; i<26; i++)
    if(count1[i])              //count1[i]非 0 时, 意味着 str1 中出现了相应字母
        cout<<char('a'+i)<<" ";    //输出统计结果
cout<<endl;
//(2) 数组 str1 中哪一个字母(不区分大小写)出现次数最多
//(注, 若多个字母次数都相同时, 输出任一个均可)? 共出现了几次
cout<<"-----"<<endl;
cout<<"The max_letters in str1:"<<endl;
int max=0, ind;
for (i=0; i<26; i++)
    if(count1[i]>max)
    {
        max=count1[i];          //count1[i]中记录着 str1 中各字母的出现次数
        ind=i;
    }
cout<<char('a'+ind)<<"", " "<<max<<endl; //输出统计结果

```

```

// (3) 在两个数组中同时出现的有哪些字母(不区分大小写)
cout<<"-----"<<endl;
cout<<"The letters in str1 and str2:"<<endl;
for (i=0; i<26; i++)
    if(count1[i] && count2[i])           //两数组中同时具有 i 对应的那一字母
        cout<<char('a'+i)<<" ";       //输出统计结果
cout<<endl;
}

void main( )
{
    const int n=100;
    char a[n],b[n];                      //字符数组 a 与 b
    cout<<"Input 2 strings:"<<endl;
    cin.getline(a, 100);                 //输入字符串 a
    cin.getline(b, 100);                 //输入字符串 b
    myFunc(a, b);                        //以 a、b 为实参调用 myFunc
}

```

[输出结果]

```

Input 2 strings:
abcABCD 123 ok!
I am a teacher. OK!
-----
str1=abcABCD 123 ok!
str2=I am a teacher. OK!
-----
The letters in str1:
a  b  c  d  k  o
-----
The max_letters in str1:
a,  2
-----
The letters in str1 and str2:
a  c  k  o

```

5.42 不同作用域中同名标识符的处理

读下述程序，其中在同一函数的不同作用域中使用了同名标识符，试给出程序执行

后的显示结果。

```
#include <iostream.h>
void main( )
{
    int i=111;                                //注 1
    char ch='1';                              //注 2
    cout<<"main1: i,ch=>"<<i<<" "<<ch<<endl;
    {                                          //注 3
        int i=222;                            //注 4
        char ch='2';                          //注 5
        cout<<"local11: i,ch=>"<<i<<" "<<ch<<endl;
        if(i>0)
        {                                    //注 6
            double i=300.3;                  //注 7
            int ch=333;                       //注 8
            cout<<"local2: i,ch=>"<<i<<" "<<ch<<endl; //注 9
        }                                    //注 10
        cout<<"local12: i,ch=>"<<i<<" "<<ch<<endl; //注 11
    }                                         //注 12
    cout<<"main2: i,ch=>"<<i<<" "<<ch<<endl; //注 13
}
```

[分析提示]

本程序在同一个 **main** 函数的不同作用域中使用了同名标识符。例如，出现有函数级作用域的变量 **i** 与 **ch**；又具有外层块级作用域的同名变量 **i** 与 **ch** 以及被外层块级作用域所包含的内嵌块级作用域中的同名变量 **i** 与 **ch**。这些不同作用域的同名变量具有各自不同的存储空间，虽然同名，但它们的使用含义各不相同，取值毫不相干。

[解答程序]

具体程序见题目处。

注 1：整型变量 **i**，具有函数级作用域。

注 2：字符型变量 **ch**，具有函数级作用域。

注 3：被函数级作用域所包含的外层块级作用域。

注 4：整型变量 **i**，具有外层块级作用域。

注 5：字符型变量 **ch**，具有外层块级作用域。

注 6：被外层块级作用域所包含的内嵌块级作用域。

注 7：双精度实型变量 **i**，具有内嵌块级作用域。

注 8：整型变量 **ch**，具有内嵌块级作用域。

注 9：输出内嵌块级作用域中的 **i**、**ch**。

注 10: 退出内嵌块级作用域。

注 11: 输出外层块级作用域中的 i、ch。

注 12: 退出外层块级作用域。

注 13: 输出函数级作用域中的 i、ch。

[输出结果]

```
main1: i,ch=>111, 1
local11: i,ch=>222, 2
local2: i,ch=>300.3, 333
local12: i,ch=>222, 2
main2: i,ch=>111, 1
```

5.43 文件级作用域以及函数级作用域的变量

读下述程序，其中使用了文件级作用域以及函数级作用域的变量，请写出程序执行结果。

```
#include <iostream.h>
int x=111;                                //注 1
char ch='1';                              //注 2
void func1(int ipara1);                  //注 3
void func2( )
{
    int i=2222;                           //注 4
    double ch=2020.2;                      //注 5
    cout<<"func2: x,ch=>"<<x<<" "<<ch<<endl; //注 6
    cout<<"func2: i=>"<<i<<endl;           //注 7
}

void main( )
{
    cout<<"main:  x,ch=>"<<x<<" "<<ch<<endl; //注 8
    func1(x);
    func2( );
}

void func1(int ii)
{
    int i=2111;                            //注 9
```



```

        int x=2001;                                //注 10
        cout<<"func1: ii=>"<<ii<<endl;           //注 11
        cout<<"func1: x,ch=>"<<x<<","<<ch<<endl;   //注 12
        cout<<"func1: i=>"<<i<<endl;               //注 13
    }

```

[分析提示]

本程序具有三个不同的函数。除在函数中说明有局部于那一函数作用域的变量之外，还在不处于任一函数的外面说明了具有文件级作用域的两个全局变量：`int x=111`；`char ch='1'`；这两个变量在该程序的任一函数中都可被使用。但注意：若在某函数内又一次重定义了这些变量的话，则当进入那一函数后，将屏蔽同名的文件级全局变量（而使用那一局部变量）。

[解答程序]

具体程序见题目处。

注 1：`int` 型文件级全局变量 `x`，具有文件级作用域。

注 2：`char` 型文件级全局变量 `ch`，具有文件级作用域。

注 3：`ipara1` 仅具有函数原型级作用域，它与函数定义处的参数名可以不同。

注 4：`int` 型函数级局部变量 `i`，虽与 `func1` 中的 `i` 重名，但毫不相干。

注 5：`double` 型函数级局部变量 `ch`，与 `char` 型文件级全局变量 `ch` 同名(屏蔽文件级全局变量 `ch`)。

注 6：输出全局变量 `x` 及局部变量 `ch` 的值。

注 7：输出局部变量 `i` 的值。

注 8：输出全局变量 `x` 及全局变量 `ch` 的值。

注 9：`int` 型函数级局部变量 `i`，虽与 `func2` 中的 `i` 重名，但毫不相干。

注 10：`int` 型函数级局部变量 `x`，与 `int` 型文件级全局变量 `x` 同名(屏蔽文件级全局变量 `x`)。

注 11：输出参数 `ii` 的值。

注 12：输出局部变量 `x` 及全局变量 `ch` 的值。

注 13：输出局部变量 `i` 的值。

[输出结果]

```

main:  x,ch=>111, 1
func1: ii=>111
func1: x,ch=>2001, 1
func1: i=>2111
func2: x,ch=>111, 2020.2
func2: i=>2222

```

5.44 全局变量以及具有 static 存储类别变量的使用

下述程序中使用了全局变量以及具有 static 存储类别的变量，请仔细阅读，而后给出程序的执行结果。

```
#include<iostream.h>

int i=5; //注 1

void func(void); //注 2

void main( )
{
    static int a; //注 3
    int b=-22; //注 4
    int c=0;
    cout<<"----- main( ) -----\\n";
    cout<<"i="<<i<<"", a="<<a<<"", b="<<b<<"", c="<<c<<endl; //注 5
    c=c+10;
    func( ); //注 6
    cout<<"----- main( ) -----\\n";
    cout<<"i="<<i<<"", a="<<a<<"", b="<<b<<"", c="<<c<<endl; //注 7
    i=i+12;
    func( ); //注 8
}

void func(void)
{
    static int a=3;
    static int b; //注 9
    int c=10; //注 10
    a=a+2;
    i=i+21;
    c=c+6;
    cout<<"----- func( ) -----\\n";
    cout<<"i="<<i<<"", a="<<a<<"", b="<<b<<"", c="<<c<<endl;
    b=a;
}
```

[分析提示]

要注意静态(static)局部变量与自动(auto)局部变量的使用区别：它们的作用域

都是局部的，作用域外都是不可见的；但静态局部变量在其作用域外仍是存在的（尚且“活着”，其存储单元中的值并不被破坏，仍可在下次进入该作用域后接着使用！即具有全局变量取值“连续性”的特征），而自动局部变量在其作用域外则不复存在（已经“死亡”，其存储单元已经被系统回收！当然不可接着使用该变量的原有值，下次进入该作用域后重新分配存储空间）。另外注意：静态变量仅在首次“遇到”时分配存储空间并进行初始化（若没显式提供初始化值时，系统自动将其初始化为系统隐含值，即将数值量初始化为 0，将字符变量初始化为空格）。

[解答程序]

具体程序见题目处。

注 1: i 为全局变量，具有静态生存期（全局寿命），在各函数中均可使用。

注 2: 函数原型。

注 3: a 为静态局部变量，具有全局寿命，但仅局部可见。

注 4: b、c 为自动局部变量，具有动态生存期（局部寿命），仅局部可见。

注 5: 输出 i、a、b、c。

注 6: 调用 func()。

注 7: 输出 i、a、b、c。

注 8: 调用 func()。

注 9: a、b 为静态局部变量，具有全局寿命，但仅局部可见，只第一次进入函数时被初始化。

注 10: c 为自动局部变量，具有动态生存期（局部寿命），每次进入函数时都初始化，仅局部可见。

[输出结果]

```
----- main() -----
i=5,  a=0,  b=-22,  c=0
----- func() -----
i=26,  a=5,  b=0,  c=16
----- main() -----
i=26,  a=0,  b=-22,  c=10
----- func() -----
i=59,  a=7,  b=5,  c=16
```

5.45 extern 型外部变量的使用

读下述由两个源程序文件构成的程序，其中使用了 extern 型外部变量，请给出程序的执行结果。注意，上机调试运行具有多个源程序文件的同一个程序时，要将它们组织（添加）到同一个工程（project）之中。

```
//extern 型外部变量的使用“ 1.cpp” (源程序文件 1, 含有 2 个函数) //注 1
#include <iostream.h>
int i=1111; //注 2
char ch='+'; //注 3
void func1( ); //注 4
void func2( ); //注 5

void main( )
{
    func1( ); //注 6
    cout<<"file1_main: i, ch=>"<<i<<" "<<ch<<endl; //注 7
    func2( ); //注 8
    cout<<"file1_main: i, ch=>"<<i<<" "<<ch<<endl; //注 9
}

void func1( ) //注 10
{
    cout<<"file1_func1: i, ch=>"<<i<<" "<<ch<<endl; //注 11
    i=2222; //注 12
    ch='-' ;
}

//extern 型外部变量的使用“ 2.cpp” (源程序文件 2, 含有 1 个函数) //注 13
#include <iostream.h>
extern int i; //注 14
extern char ch; //注 15
void func2( ) //注 16
{
    cout<<"file2_func2: i, ch=>"<<i<<" "<<ch<<endl; //注 17
    i=3333; //注 18
    ch='*'; //注 19
}
```

[分析提示]

外部变量的使用, 可以使得在不同的源程序文件之间进行变量值的相互传递。通过 **extern** 关键字来对外部变量进行说明, 系统仅在其中某一文件的外部变量定义处为其分配存储空间, 而在其他文件中则不再分配, 但与定义处的那一同名变量共享同一存储空间, 以达到多文件之间变量值相互传递的目的。

[解答程序]

具体程序见题目处。

注 1: 本文件中对外部变量 `i` 及 `ch` 进行定义(约定没使用存储类关键字 `extern` 的这种定义性出现, 它通常出现于文件开头处且处于所有的函数之外, 格式与单文件级全局变量的说明完全相同, 系统将在此处为其分配存储空间), 并通过调用处于另一文件中的函数来传递与使用这些外部变量的具体值。

注 2: 外部(`extern`) 变量 `i` 的定义性出现(格式与单文件级全局变量的说明完全相同)。

注 3: 外部变量 `ch` 的定义性出现(格式与单文件级全局变量的说明相同), 系统在静态数据区(也称全局数据区)为 `i`、`ch` 分配两个存储单元。

注 4: 函数 `func1` 的原型说明, `func1` 位于本文件中 `main` 函数的后面。

注 5: 函数 `func2` 的原型说明, `func2` 位于另一源程序文件中。

注 6: 调用本文件中的 `func1` 函数, 将输出外部变量 `i`、`ch` 的初始值(`i`, `ch=1111`, +)。

注 7: 输出被本文件中的 `func1` 函数改变后的 `i`、`ch` 值(`i`, `ch=2222`, -)。

注 8: 调用处于另一文件中的 `func2` 函数, 外部变量 `i`、`ch` 的当前值将被“传递”过去, 在 `func2` 函数中将输出传递过去的 `i`、`ch` 值(`i`, `ch=2222`, -)。

注 9: 输出被另一文件中 `func2` 函数改变后的 `i`、`ch` 值(`i`, `ch=3333`, *)。

注 10: 函数 `func1` 的定义。

注 11: 输出当前 `i`、`ch` 的变量值。

注 12: 改变外部变量 `i`、`ch` 的值。

注 13: 在本源程序文件中通过 `extern` 关键字来对外部变量 `i` 及 `ch` 进行说明(不再分配存储空间, 与另一文件定义处的那一同名变量共享同一存储空间, 以达到多文件之间变量值的相互“传递”), 并通过使用与修改这些外部变量的值来达到相互通信的目的。

注 14: 说明外部整型变量 `i`, 必须与另一文件的 `i` 同名且具有相同的数据类型。

注 15: 说明外部字符型变量 `ch`, 必须与另一文件的 `ch` 同名且具有相同的数据类型。此处不再分配存储空间, 将与文件 1 中的 `i`、`ch` 共用一套存储, 从而使文件 1 中定义的这两个同名变量的作用域扩展到了该文件 2 的各函数中, 而变成为多文件级全局变量。

注 16: 函数 `func2` 的定义。

注 17: 输出文件 1 中已赋于外部变量 `i`、`ch` 的当前值。

注 18: 改变外部变量 `i`, 并在下一句改变 `ch` 的值。

注 19: 改变后的 `i`、`ch` 值(`i`, `ch=3333`, *) 将被传送到另一个文件中(去接着使用)。

[输出结果]

```
file1_func1: i, ch=>1111, +
file1_main : i, ch=>2222, -
file2_func2: i, ch=>2222, -
file1_main:  i, ch=>3333, *
```

5.46 与全局变量以及静态局部变量相关的使用

读下述程序，写出执行结果(其中使用了全局变量以及静态局部变量等)。

```
#include <iostream.h>

int i=11, j=22;                                //注 1

long f1(int n)
{
    static long res=1;                          //注 2
    cout<<"---- In f1: i,j="<<i++<<" "<<j++<<endl;    //注 3
    res=res*n;                                  //注 4
    return res;
}

long f2(int n)
{
    static long res=1;                          //注 5
    cout<<"---- In f2: i,j="<<i++<<" "<<j++<<endl;    //注 6
    res=res*n;                                  //注 7
    return res;
}

void main(void)
{
    int i=2, j=5;                                //注 8
    cout<<"main: f1("<<i<<")="<<f1(i)<<endl;        //注 9
    cout<<"main: f2("<<j<<")="<<f2(j)<<endl;        //注 10
    cout<<"main: i,j="<<i<<" "<<j<<endl;          //注 11
    i++;                                          //注 12
    cout<<"main: f1("<<i<<")="<<f1(i)<<endl;        //注 13
    i++;                                          //注 14
    cout<<"main: f1("<<i<<")="<<f1(i)<<endl;        //注 15
    cout<<"main: i,j="<<i<<" "<<j<<endl;          //注 16
}
```

[分析提示]

本程序中使用了全局变量以及静态局部变量。全局变量的作用域为整个程序；而静态局部变量的作用域则是局部的(出了作用域则不可见)，但其生命期却是全程的(一直占用其存储空间，直到程序结束)。

[解答程序]

具体程序见题目处。

注 1: 全局变量 i、j。

注 2: 静态局部变量 res, 具有初值 1, 取值具有“连续性”。对静态局部变量 res, 仅在首次遇到时分配存储空间并进行初始化(一直占据所占空间)。

注 3: 全局变量 i、j。

注 4: 静态局部变量 res 中总保持上次调用结束时的 原有值(具有取值 连续性”)。

注 5: 静态局部变量 res, 具有初值 1(与 f1 函数中的 res 同名, 但毫不相干)。对局部静态变量 res, 仅在首次遇到时分配存储空间并进行初始化(一直占据所占空间)。

注 6: 全局变量 i、j。

注 7: 静态局部变量 res 中总保持上次调用结束时的 原有值(具有取值 连续性”)。

注 8: 局部于 main 的变量 i、j。

注 9: main: f1(2)=2。

注 10: main: f2(5)=5。

注 11: main: i,j=2 5。

注 12: 局部于 main 的变量 i。

注 13: main: f1(3)=6。

注 14: 局部于 main 的变量 i。

注 15: main: f1(4)=24。

注 16: main: i,j=4 5。

[输出结果]

----- In f1: i,j=11 22

main: f1(2)=2

----- In f2: i,j=12 23

main: f2(5)=5

main: i,j=2 5

----- In f1: i,j=13 24

main: f1(3)=6

----- In f1: i,j=14 25

main: f1(4)=24

main: i,j=4 5

5.47 求解骑士巡游问题——为骑士求解巡游“路线图”

在 $n \times n$ 的棋盘上(如 $n=5$), 假设一位骑士 按象棋中“马”的行走法——“马”走“日”) 从初始坐标位置(x_1, y_1)出发, 要“遍访”(巡游)棋盘中的每一个位置一次。请编写一个程序, 为骑士求解巡游“路线图”(或告诉骑士, 从某位置出发时, 无法遍访整个棋盘

的每一个位置一次则问题无解)。

当 $n=5$ 时, 意味着要在 $5*5$ 的棋盘的 25 个“点”处, 按骑士行走规则, 依次将 1~25 这 25 个“棋子”(数码) 分别摆放到棋盘上(摆满 25 个位置则成功, 否则失败, 问题无解)。

例如, 当 $n=5$ 且初始坐标位置定为 (1, 1) —— 即最左上角的那个点时, 如下是一种巡游“路线图”。程序执行后的输出结果为:

```

1   6  15  10  21
14  9   20   5  16
19  2   7   22  11
8   13  24  17   4
25  18   3  12  23

```

[分析提示]

(1) 进行说明: `const int n=5; int a[n+1][n+1];`

当前要在 $n*n$ 的棋盘上进行处理。`a` 数组用于存放棋盘数据, 为方便起见, 只使用其中的 1~ n 行、1~ n 列(不使用 0 行 0 列)。整个任务是要在 `a` 数组的 1~5 行、1~5 列的各位置处摆放上 25 个“棋子”(1~25 的数码), 尚未摆放棋子的位置处均置为 0。

(2) 编写一个具有如下原型的递归函数 `DoIt`, 用于完成任务: 从 (i,j) 点出发, 做第 k ~第 $n*n$ 次的移动(将 k 直到 $n*n$ 这些“棋子”(数码) 分别摆放到棋盘上), 若成功, 则通过引用参数 `q` 返回 `true`(否则返回 `false`)。

```
void DoIt(int i, int j, int k, bool& q);
```

(3) 编写第一种解法的主函数: 用户输入作为巡游起点的初始坐标位置(x1, y1), 程序负责求解并输出巡游“路线图”(无解时输出——“No solution!”)。

(4) 编写第二种解法的主函数: 将棋盘上的 $n*n$ 个点依次作为巡游起点的初始坐标位置(x1, y1), 判断从每一位置出发是否有解或无解 输出“OK!”或“NO!”, 但并不输出“路线图”。

(5) 若更改第一步所说明的“`const int n=5;`”中的 n 值(如改为 4 或 6 等), 便可求解其他阶数的巡游“路线图”。

[解答程序]

```

#include <iomanip.h>

const int n=5;                                //n*n 的棋盘

int l, m, a[n+1][n+1], move[9][3];

/*

```

`a` 数组放棋盘数据, 为方便起见, 只使用其中的 1~ n 行、1~ n 列(不使用 0 行 0 列)。作为全局变量的 `a` 数组, 系统会自动地将其各元素值初始化为 0 (若非全局变量时, 则不享受此待遇)。 `a` 数组的某元素值为 0 意味着该“位置”尚未放上“棋子”(1~25 的数码)。

`move` 数组存放“移动偏移量”数据, 为方便起见, 只使用其中的 1~8 行、1~2 列(不使用 0 行 0 列)。设骑士当前处于 (i,j) 点, 他可朝着八个“方向”迈进, 所到达的新点 (g,h) 与

move 数据有如下关系:

```

    g=i+move[v][1];  h=j+move[v][2];    //沿着 v 方向“ 迈向 ” (g,h) 点
    */

bool q;                                //摆放成功与否的布尔变量

void DoIt(int i, int j, int k, bool& q)    //递归函数
{
    //从(i,j)点出发, 做第 k~第 n*n 次的移动
    //( 将 k 直到 n*n 这些“ 棋子”( 数码) 分别摆放到棋盘上)
    //若成功则通过引用参数 q 返回 true( 否则返回 false)
    int v=0, g, h;    //v 表示“ 方向 ” (与 move 数组配合使用)
    //取值 1~8(表示可朝八个不同方向迈进)
    //从当前的(i,j)点出发, 沿着 v“ 方向 ” 迈一步后, 到达新点(g,h)
do
{
    v++;
    q=false;                //v 增长 1 后意味着去测试下一个新方向
    g=i+move[v][1];
    h=j+move[v][2];        //沿着 v 方向“ 迈向 ” (g,h) 点
    if ( g>=1 && g<=n && h>=1 && h<=n && a[g][h]==0 )
        //(g,h)点处于棋盘内且尚未放“ 棋子”
    {
        a[g][h]=k;        //将“ 棋子 ” k 放在 (g,h) 点
        if (k<n*n)        //若尚未放满整个棋盘时
        {
            DoIt(g, h, k+1, q);    //递归调用
            //从新点(g,h)出发, 做第 k+1~第 n*n 次的移动
            //(将 k+1 直到 n*n 这些“ 棋子 ” (数码)
            //分别摆放到棋盘上), 若成功则通过引用参数 q
            //返回 true(否则返回 false)
            if(!q) a[g][h]=0;
            //若返回的 q 为假时(无法从(g,h)点摆放“ 到底 ” )
            //则去掉(g,h)点的“ 棋子 ”
            //注意: 该赋值语句结束后 =>“ if(!q)” 句结束 =>“ if (k<n*n)”
            //的一个分支子句(复合语句)结束 =>“ if (k<n*n)” 语句结束 =>
            //“ if ( g>=1 && g<=n && h>=1 && h<=n && a[g][h]==0 )”
            //语句结束, 之后转向“ do-while” 语句的条件测试处
            //(准备去“ 试探 ” 下一个 v 方向)
        }
    }
}

```

```

else          //若已放满整个棋盘时返回 q 为 true
    q=true;
}             //if ( g>=1 && g<=n && h>=1 && h<=n && a[g][h]==0 ) 结束
}
while (!q && (v!=8));    //尚未放满 n*n 个“ 棋子” (数码)
                        //且本层次的 v 尚未“ 试探” 到 8 时继续循环
}

void main(void)          //解法一主函数
{
    //用户输入作为巡游起点的初始坐标位置(x1, y1), 程序负责
    //求解并输出巡游“ 路线图” (无解时输出——“ No solution!” )
    //move 数组存放“ 移动偏移量” 数据, 使用下述给定的各常量
    //其含义如下: 设骑士当前处于(i,j)点, 他可朝着八个“ 方向”
    //迈进, 所到达的新点(g,h)与 move 数据有关
    //其中 g=i+move[v][1]; h=j+move[v][2]; //沿着 v 方向“ 迈向” (g,h)点
    move[1][1]=2;  move[1][2]=1; move[2][1]=1;  move[2][2]=2;
    move[3][1]=-1; move[3][2]=2; move[4][1]=-2;   move[4][2]=1;
    move[5][1]=-2;   move[5][2]=-1;   move[6][1]=-1;   move[6][2]=-2;
    move[7][1]=1;   move[7][2]=-2;   move[8][1]=2;   move[8][2]=-1;
    cout<<"n="<<n<<endl;          //当前处理的是 n*n 的棋盘
    cout<<"The first position:";    //提示输入初始坐标位置
    int x1,y1;
    cin>>x1>>y1;                  //输入初始坐标位置(x1, y1)
    a[x1][y1]=1;                  //在棋盘的(x1, y1)处摆放“ 棋子” (数码)1
    DoIt(x1, y1, 2, q);
        //从(x1, y1)点出发, 做第 2~第 25 次的移动
        //(将 2~25 这些“ 棋子” (数码)分别摆放到棋盘上),
        //若成功, 则通过引用参数 q 返回 true(否则返回 false)
    if(q)    //若摆放成功, 输出“ 棋盘” ——a 数组中各“ 棋子” (数码)的
        //摆放位置(巡游“ 路线图” )
        for(l=1; l<=n; l++)
        {
            for(m=1; m<=n; m++)
                cout<<setw(4)<<a[l][m];
            cout<<endl;
        }
    else    //若摆放不成功, 无法遍访整个棋盘——问题无解
        cout<<"No solution!"<<endl;
}

```

```

        cout<<endl;
    }

```

[输出结果]

```

n=5
The first position:1 1
  1   6  15  10  21
14   9  20   5  16
19   2   7  22  11
  8  13  24  17   4
25  18   3  12  23

```

又一次执行:

```

n=5
The first position:1 2
No solution!

```

再一次执行:

```

n=5
The first position:1 3
25  14   1   8  19
 4   9  18  13   2
15  24   3  20   7
10   5  22  17  12
23  16  11   6  21

```

[解法二主函数]

```

void main(void)                                //解法二主函数
{
    //将棋盘上的 n*n 个点依次作为巡游起点的初始坐标位置(x1, y1)，判断从
    //每一位置出发是否有解或无解(输出"OK!"或"NO!"，但并不输出“路线图”)
    //move 数组存放“ 移动偏移量”数据，使用下述给定的各常量
    //其含义如下：设骑士当前处于(i,j)点，他可朝着八个“ 方向”
    //迈进，所到达的新点(g,h)与 move 数据有关
    //其中 g=i+move[v][1]; h=j+move[v][2]; //沿着 v 方向“ 迈向” (g,h)点
    move[1][1]=2; move[1][2]=1; move[2][1]=1; move[2][2]=2;
    move[3][1]=-1; move[3][2]=2; move[4][1]=-2; move[4][2]=1;
    move[5][1]=-2; move[5][2]=-1; move[6][1]=-1; move[6][2]=-2;
    move[7][1]=1; move[7][2]=-2; move[8][1]=2; move[8][2]=-1;

```

```

cout<<"n="<<n<<endl;           //当前处理的是 n*n 的棋盘
int x1,y1;                       //初始坐标位置(x1, y1)
for(x1=1; x1<=n; x1++)
{
    for(y1=1; y1<=n; y1++)
    {
        //初始坐标位置(x1, y1)要“穷尽”棋盘上的所有位置
        for(l=1; l<=n; l++)
            for(m=1; m<=n; m++)
                a[l][m]=0;       //将棋盘清空(所有位置均置为 0)
        a[x1][y1]=1;           //第一个棋子摆放在(x1, y1)处
        DoIt(x1, y1, 2, q);
        //从(x1, y1)点出发, 做第 2 至第 25 次的移动
        //(将 2~25 这些“棋子”(数码)分别摆放到棋盘上)
        //若成功, 则通过引用参数 q 返回 true(否则返回 false)
        if(q)                   //若摆放成功,
            //输出第一个棋子的摆放位置(坐标值)以及"OK!"字样
            cout<<"("<<x1<<","<<y1<<)--OK! ";
        else                    //若摆放不成功
            //输出第一个棋子的摆放位置以及"NO!"字样
            cout<<"("<<x1<<","<<y1<<)--NO! ";
        cout.flush();
    }                          // for--y1 结束
    cout<<endl;
}                              // for--x1 结束
}

```

[解法二主函数输出结果]

n=5

```

(1,1)--OK! (1,2)--NO! (1,3)--OK! (1,4)--NO! (1,5)--OK!
(2,1)--NO! (2,2)--OK! (2,3)--NO! (2,4)--OK! (2,5)--NO!
(3,1)--OK! (3,2)--NO! (3,3)--OK! (3,4)--NO! (3,5)--OK!
(4,1)--NO! (4,2)--OK! (4,3)--NO! (4,4)--OK! (4,5)--NO!
(5,1)--OK! (5,2)--NO! (5,3)--OK! (5,4)--NO! (5,5)--OK!

```

5.48 求解八皇后问题

在 8*8 的棋盘上放置八个皇后, 使任一个皇后都不能吃掉其他的七个皇后(注, 皇

后可吃掉与她处于同行或同列或同一对角线上的其他棋子),并将结果以某种方式显示出来。例如,当求出下述的一个解时,可输出如下信息表示该解(输出了表示摆放皇后的坐标位置以及“棋盘状态”——棋盘中有皇后位置为“Q”字符,其他位置为“.”字符)。

(1,1) (5,2) (8,3) (6,4) (3,5) (7,6) (2,7) (4,8)

Q.....

.....Q.

....Q...

.....Q

.Q.....

...Q....

.....Q..

..Q.....

[分析提示]

(1) 进行说明: `int LineNum[9];` `bool a[9], b[15], c[15];`

其中的 `LineNum[i]` 表示第 i 列的皇后要放的行位置(有效列号从 1~8, 0 号不用); `a[i]` 为 `true` ($i=1,2,\dots,8$) 表示第 i 行上尚未放皇后; `b[i]` 为 `true` ($i=0,1,2,\dots,14$) 表示第 i 条斜对角线上尚未放皇后(斜对角线指的是“/”状对角线,该对角线上各点的行、列号之和 $i+j$ 为一个常数); `c[i]` 为 `true` ($i=0,1,2,\dots,14$) 表示第 i 条反斜对角线上尚未放皇后(反斜对角线指的是“\”状对角线,该对角线上各点的行、列号之差 $i-j$ 为一个常数)。

(2) 编写一个具有如下原型的递归函数 `DoIt`, 它负责往第 i 列开始的连续 $8-i+1$ 列上均放上皇后, 若成功, 则通过引用参数 `q` 返回 `true` (否则返回 `false`)。

```
void DoIt(int i, bool& q);
```

(3) 编制主函数, 首先将各 `a[i]`、`b[i]` 以及 `c[i]` 均置为 `true`, 表示当前棋盘的 8 个行、15 条斜对角线以及 15 条反斜对角线上都尚未摆放皇后。而后执行调用语句 `DoIt(1, q);`, 它负责往第 1 列开始的连续 8 列上均放上皇后, 若成功, 则通过引用参数 `q` 返回 `true` (否则返回 `false`)。接着判断 `q` 的返回值, 若为 `true`, 则根据 `LineNum` 数组中计算出的数据来显示出所需结果; 若 `q` 为 `false` (无解时), 输出“ No solution!” 字符串。

[解答程序]

```
#include <iostream.h>

int i, LineNum[9];           //LineNum[i]表示第 i 列的皇后要放的行位置
bool q;                     //放置成功与否的布尔变量
bool a[9], b[15], c[15];

//a[i]为 true(i=1,2,...,8)表示第 i 行上尚未放皇后
//b[i]为 true(i=0,1,2,...,14)表示第 i 条斜对角线上尚未放皇后
//c[i]为 true(i=0,1,2,...,14)表示第 i 条反斜对角线上尚未放皇后

void DoIt(int i, bool& q)    //递归函数
```

```

//负责往第 i 列开始的连续 8-i+1 列上均放上皇后
//若成功, 则通过引用参数 q 返回 true(否则返回 false)
{
    int j=0;                //j 表示应放的行位置(从第 1 行找起, 直到找到第 8 行)
    do
    {
        j++; q=false;        //j 值从 1, 每次增 1 直到 8
        if ( a[j] && b[i+j-2] && c[i-j+7] ) //如果在第 j 行放置皇后安全
        {
            LineNum[i]=j;
            //LineNum[i]表示第 i 列的皇后要放的行位置(在 j 行处放)
            a[j]=false; b[i+j-2]=false; c[i-j+7]=false;
            //在第 j 行放置皇后之后, 要标记其“ 影响面”
            // --以后不可再往第 j 行上放置皇后
            //不可再往第 i+j-2 条斜对角线上放置皇后
            //也不可再往第 i-j+7 条反斜对角线上放置皇后
            if (i<8)          //尚未放满八个皇后时
            {
                DoIt(i+1, q); //递归调用
                //负责往第 i+1 列开始的连续 8-i+1 列上均放上皇后
                //若成功, 则通过引用参数 q 返回 true(否则返回 false)
                if(!q) //若不成功, 要去掉 j 行的皇后, 接着去“ 试探” j+1 行
                //注意, 该 if 句结束进而“ 上层” 以及“ 上上层” 的
                //if 结束而后接着执行 do-while
                {
                    a[j]=true; b[i+j-2]=true; c[i-j+7]=true;
                    //去掉 j 行的皇后, 就要去掉原来标记的各“ 影响面”
                }
            }
            else              //已放满八个皇后时, 将 q 置为 true
                q=true;
        }
        //if ( a[j] && b[i+j-2] && c[i-j+7] )句体结束
    }
    while (!q && (j!=8));
    //尚未放满八个皇后且本列的八个行尚未“ 试探” 完时继续循环
}

void main(void) {
    for(i=1; i<=8; i++)      //最初, 八个行上都尚未摆放皇后

```

```

        a[i]=true;           //a[i]=true 表示第 i 行上尚无皇后
    for(i=0; i<=14; i++)      //最初, 15 条斜对角线(“ /” 状对角线)都尚未摆放皇后
        b[i]=true;           //b[i]=true 表示第 i 条斜对角线上尚无皇后
    for(i=0; i<=14; i++)      //最初, 15 条反斜对角线(“ \” 状对角线)都尚未摆放皇后
        c[i]=true;           //c[i]=true 表示第 i 条反斜对角线上尚无皇后
    DoIt(1, q);                //负责往第 1 列开始的连续 8 列上均放上皇后
                                //若成功, 则通过引用参数 q 返回 true(否则返回 false)
    if(q)                      //若放置成功
    {
        for(i=1; i<=8; i++)    //输出放置 8 个皇后各自的“ (行号, 列号)” 位置
            cout<<"("<<LineNum[i]<<","<<i<<") ";
        cout<<endl;
        for(i=1; i<=8; i++)    //共处理 8 行(i 为当前行号)
        {
            //输出成功时的“ 结果棋盘状态” ——皇后位置为“ Q” 字符,
            //其他位置为“ .” 字符
            int j, col;
            for(j=1; j<=8; j++)    //找出第 i 行皇后应放置的列位置 col
                if(LineNum[j]==i)    //i 为当前行号
                    col=j;          //皇后应放置的列位置 col
            for(j=1; j<=8; j++)
                //输出第 i 行的结果状态(有皇后位置为“ Q”, 其他位置为“ .” )
                if(j==col) cout<<"Q";
                else cout<<".";
            cout<<endl;
        }
    }
    else                        //若放置不成功
        cout<<"No solution!";
    cout<<endl;
}

```

[输出结果]

与题目处相同, 略。

第六章 指针、结构体及引用

[语法要点]

详细内容可参阅教材第六章。

(1) 指针类型 C++语言为用户对于变量所占空间的地址提供了进行操作的机制,这就是指针。指针不是一种独立的数据类型,而是一种导出数据类型。

指针类型的变量说明格式为:

〈 类型名〉*〈 指针变量名〉=〈 初值〉;

其中的“=〈 初值〉”部分可以缺省,例如:

```
int *ptr;
float a=3.0, b, c[4]; float *pa=&a;
float *p[2]={ &a,&b};
float *pn=NULL;
float *pc=c;
float *pf=new float;
point *pp;
```

指针变量可有三种状态:

- ① 未赋任何值即悬空状态。
- ② 被赋予 NULL 值, 指向任一变量。
- ③ 指向某一变量。

(2) 指针变量的运算

- ① 取地址运算 & 和取内容运算 * , 例如:

```
int a=3, *pa=&a;
*pa=5;
cout<<a<<*pa;
```

- ② 数组指针的算术运算

如果指针变量是指向数组的, 例如:

```
int a[20];
int *pa=a;
```

则指针 pa 可以有限制地进行加减(+、-) 运算。

这时, 可以用三种方式表示该数组的元素:

- 下标变量的形式:
a[0],a[1],a[2],a[3],...,a[19];
- 用数组名 a(一个常量指针):
a, *(a+1), *(a+2), *(a+3), ..., *(a+19) ;

- 用指针变量 `pa( =a)`:
`*pa, *( pa+1), *( pa+2), *( pa+3), ..., *( pa+19);`

③ 指针的关系运算, 例如:

```
float  a, b, c[10];
float  *p1=&a, *p2=NULL, *p3=c, *p4;
if( p1 != p2) p2 = p3+2;
if( p2 > p3) p2+=1;
指针和所有类型一样, 它也可进行赋值运算。
```

(3) 指针与数组 都是导出类型, 它们可以互相导出。

① 指向数组元素的指针, 例如:

```
int  n[ 10];
int *pn=n;
```

指针 `pn` 指向数组 `n` 的首元 `n[0]`。在这里

`pn=n;` 与 `pn=&n[0];`

起相同的效果, 因为数组名就是一个指向其首元的一个指针常量。

② 指向数组的指针 把数组作为整体, 指向这样一个整体的指针被称为指向数组的指针, 其说明语句格式为:

〈 类型名〉(*〈 指针变量名〉) [〈 数组元素数〉];

而指向多维数组的指针说明中, 最后一项[〈 数组元素数〉]可有多个。

例如:

```
int *pa) [4];
```

③ 指针数组 由指针组成的数组, 常常在程序中出现, 其说明格式为:

〈 类型名〉*〈 数组名〉 [〈 元素数〉]

`n` 维指针数组为:

〈 类型名〉*〈 数组名〉 [〈 元素数 1〉] [〈 元素数 2〉] [〈 元素数 `n`〉];

例如:

```
int  a, b, c, d, A[2][4];
int *p1[4]={ &a, &b, &c, &d};
int *p2[2]={ A[0], A[1] };
```

(4) 字符串指针 C++语言对于字符串的操作提供了多种手段, 其中利用字符串指针是最方便的一种。

① 说明与初始化

字符串指针没有自己独立的形式, 其说明语句的格式与字符类型指针相同, 例如:

```
char  ch= 'a';
char  *pc1=&ch;           //字符指针
char  *pc2="world";       //字符串指针
char  as[10]="Chinese";
char  *pc3=as;            //字符串指针
```

② 字符串的整体输入输出把字符串变量用字符数组或字符串指针的形式表示, 然后

直接进行 I/O 操作:

```
cin>>< 字符串);
```

```
cout<<< 字符串);
```

③ 字符串指针数组

以字符串指针组成的数组使用起来比较方便, 例如:

```
char *menu[ ] = {"File", "Edit", "Search", "Help"};
```

④ 利用字符串指针的标准函数

在头文件 `string.h` 中, C++ 系统提供若干个串处理函数的说明, 它们大都利用字符串指针, 下面列出几例:

- 求字符串长度。 `unsigned strlen( const char *str );`

返回字符串 `str` 的长度(不包括串尾符)。

- 字符串拷贝: `char *strcpy( char *str1 , char *str2);`

(5) 指针与函数的关系密切, 主要分三部分: 指针作函数参数, 作函数返回类型和指向函数的指针。函数指针在 Pascal 语言是没有的。

① 指针作函数参数

指针是一种特殊类型的数据, 它的值是另外某种类型的变量的地址。因此, 指针作函数的参数可以起到其他类型参数所起不到的作用。

② 函数返回指针

返回值为指针的函数称为指针型函数, 其返回类型的说明应在指明指针的对象类型后加“*”符号。

③ 函数指针

函数不是数据, 但它与变量还是有两点相通之处: 一个是它有类型(返回类型), 另一个是它也有地址, 称为入口地址, 函数的地址也可作指针的值, 这就是函数指针。

函数指针的说明格式与函数的原型相似, 主要区别是: 原来的“< 函数名>”, 用“* < 函数指针名>”所代替, 例如:

```
int( *pf)( float);
```

(6) 引用(`reference`) 是 C++ 语言特有的(其他语言不具备的) 数据形式。它的存在不仅像数组和指针那样依赖于已有的类型, 而且它还依赖于一个已有的变量。从直观上说, 一个引用变量是一个已定义的变量的别名。

① 引用变量的说明

引用变量的说明格式与指针变量说明相似:

```
< 类型名> &< 变量名> = < 对象变量名>;
```

与指针说明的区别是:

- 用符号“&”代替符号“*”。
- 赋初值部分不可缺省。

② 引用和指针的比较

C++ 语言中增加引用这种数据形式, 主要是用于在一定的范围内, 代替或改进指针的作用。虽然在说明方式上十分相似, 但在概念上却有着明显的不同。其区别最主要有两点:

• 指针表示的是一个对象变量的地址，而引用则表示一个对象变量的别名。因此在程序中表示其对象变量时，前者要通过取内容运算符*，而后者可直接代表：

```
int m, n;  
int *p=&m;  
int &r=m;
```

而在对 m 赋值时，下面三者是等价的：

```
m=5;    *p=5;    r=5;
```

• 指针是可变的，它可以指向变量 m，也可以指向变量 n，而引用变量只能在定义时一次确定，不可改变。

```
int m, n;  
int *p=&m;  
int &r=m;
```

以后执行“ p=&n;”是合法的，而引用变量 r 只可与 m 相联系。

③ 引用型参数

把函数的参数说明为某一类型的引用类型，意味着这个参数为“变量参数”，称为引用调用。

引用型参数在调用时，相应的实参必须是对应类型的变量或对象；在调用函数运行前，生成该实参的引用变量；在整个函数体运行过程中，这个引用变量相当于作为实参的变量或对象的别名。

④ 引用型的函数返回值

当把函数的返回类型说明为引用型时，这个函数返回的不仅仅是某一变量或对象的值，而且返回了它的别名，从而该函数的调用也可以被赋值（引用型返回类型的函数调用是左值表达式）。让我们举例说明：

```
int a=3, b=5, c;  
int &maxr( int &m, int &n )  
{  
    if( m>n ) return m;  
    return n;  
}
```

函数 maxr() 可以有如下用法：

```
c=maxr( a, b );
```

它把变量 a, b 中较大者的值赋给了变量 c。

```
maxr( a, b )=10;
```

它把变量 a, b 中较大者的值改变为 10。

```
maxr( a, b )--;
```

它把变量 a,b 中当前较大者的值减 1。

在有些场合下，以引用代替指针，除了可以实现相同的目标外，同时增加程序的安全性，引用型函数可以作为“左值”，它的这种功能可以说是程序设计语言的一个重要发展，C++ 程序员在程序设计中充分使用引用概念，必然有利于提高程序的水平。

(7) 结构与联合 同样能如类那样实现封装。

结构与类的区别是，在缺省情况下其成员为公有(类为私有)。它同样可以有数据成员，函数成员，有构造函数和析构函数，有 `this` 指针等等，在功能上与类是重复的。

大多数有经验的程序员把结构用来“封装”若干数据的集合，例如，把有关窗口的主要数据组成一个结构，把有关图形文件的数据组成一个结构。

C++语言中的类和对象概念使得结构和联合在程序设计中的重要性变轻，在作为C++语言的衍生语言 Java 语言中，干脆取消了结构和联合这两种数据类型。

6.1 与字符指针相关的使用

读下述程序，写出执行结果。

```
#include <iostream.h>

void main( )
{
    char s[21], *ps=s;                                //注 1
    for (int i=0; i<20; i++)
        s[i]='A'+i;
    s[20]='\0';                                        //注 2
    ps++;                                              //注 3
    cout<<"ps="<<ps<<endl;                          //注 4
    ps+=2;                                            //注 5
    cout<<"ps="<<ps<<endl;                          //注 6
    for (ps=&s[19]; ps>&s[11]; ps-=2)                //注 7
    {
        cout<<"*ps="<<*ps<<endl;                    //注 8
        cout<<"ps="<<ps<<endl;                      //注 9
    }
}
```

[分析提示]

(1) 对字符型指针“ `char*`”值的输出，实际上是输出从该字符指针所指位置开始直到串尾结束符(“ `\0`”字符)为止的那一个字符串。

(2) 本题使用了指针加减一个整数及指针赋值的运算(如， `ps++`; `ps+=2`; `ps-=2`; 等); 也使用了两个指针进行比较以及取地址的运算(如， `ps>&s[11]`)。

[解答程序]

具体程序见题目处。

注 1: `ps` 指向 `s` 的首字符。

- 注 2: s 字符数组中为: ABCDEFGHIJKLMNOPQRST。
- 注 3: ps 指向 s 的第二个字符。
- 注 4: 输出从 s 的第二个字符开始的全体: BCDEFGHIJKLMNOPQRST。
- 注 5: ps 指向 s 的第四个字符。
- 注 6: 输出从 s 的第四个字符开始的全体: DEFGHIJKLMNOPQRST。
- 注 7: 循环共执行四次。
- 注 8: 输出当前 ps 所指向的那一个字符。
- 注 9: 输出从当前 ps 所指字符开始的全体字符(直到字符串的尾部)。

[输出结果]

```
ps=BCDEFGHIJKLMNOPQRST
ps=DEFGHIJKLMNOPQRST
*ps=T
ps=T
*ps=R
ps=RST
*ps=P
ps=PQRST
*ps=N
ps=NOPQRST
```

6.2 指针及其相关运算

读如下的程序, 给出执行结果。

```
#include <iostream.h>
void main(void)
{
    int a[10]={0,1,2,3,4};
    int *p=a, *p1=&a[9];           //注 1
    *(p+5)=15;                      //注 2
    *(p1-3)=*(p+5)+1;               //注 3
    for (int i=7; i<10; i++)         //注 4
        p[i]=10+i;
    cout<<"p1-a="<<p1-a<<endl;     //注 5
    cout<<"a[p1-a]="<<a[p1-a]<<endl;
    cout<<"*p1="<<*p1<<endl;
    cout<<"*(&a[3])="<<*(&a[3])<<endl;
    for (i=0; i<10; i++)             //注 6
```

```
        cout<<a[i]<<" ";
    cout<<endl;
    while (p1>&a[4])                //注 7
        cout<<*p1--<<" ";
    cout<<endl;
}
```

[分析提示]

(1) 本题使用了取指针所指内容的运算(如, *p1, *(p+5)等);还使用了两个指针求差的运算(如, p1-a, 注意它们指向同一个数组, 差为两指针间元素的个数)。

(2) 注意, *(&a[3])就等同于 a[3]——先取 a[3]的地址(为一个指针), 而后又取该指针所指向的内容。

[解答程序]

具体程序见题目处。

注 1: 指针 p 指向 a[0], p1 指向 a[9]。

注 2: 给 a[5]赋值。

注 3: 给 a[6]赋值。

注 4: 给 a[7]、a[8]、a[9]赋值。

注 5: 二指针求差(注意它们指向同一个数组, 差为两个指针间元素的个数)。

注 6: 输出 a 数组的十个元素值。

注 7: 从后往前输出 a 数组的后五个元素值。

[输出结果]

```
p1-a=9
a[p1-a]=19
*p1=19
*(&a[3])=3
0  1  2  3  4  15  16  17  18  19
19  18  17  16  15
```

6.3 判断字符串是否为合法标识符

编写具有如下原型的函数“bool is_Identifier(char* str);”, 它负责判断字符串 str 是否为一个合法的标识符。并编制主函数, 对所编函数进行调用以验证其正确性(注, 标识符是以字母或下划线开头, 后跟任意有限多个字母或下划线或数字的字符串)。

[分析提示]

(1) 函数 `is_Identifier` 的参数 `str` 为字符型指针, 意味着传递过来了一个字符串(字符型数组)。函数中要对以 `str` 指针所指字符为首直到字符串尾(“\0”)的那些符号逐一进行处理(看是否能构成一个有效的标识符)。

(2) 主函数调用时, 传去的实参 `string` 为一个字符数组名(数组首地址), 它等同于一个 `char*` 类型的指针值。

[解答程序]

```
#include<iostream.h>

bool is_Identifier(char* str);           //函数原型

void main( )
{
    char string[80];
    cout<<"Input a string:"<<endl;
    cin>>string;                        //输入一个字符串
    if( is_Identifier(string) )
        //调用自定义函数 is_Identifier 去判断 string 是否为标识符
        //实参 string 为字符数组名, 等同于一个 char* 类型的指针值
        cout<<string<<" -- is a IDENTIFIER!"<<endl;
    else
        cout<<string<<" -- is not a IDENTIFIER!"<<endl;
}

bool is_Identifier(char* str) //负责判断字符串 str 是否为一个合法的标识符
    //标识符是以字母或下划线开头, 后跟任意有限多个字母或下划线或数字
    //的字符串
{
    if( (*str!='_') && !( *str>='A' && *str<='Z') && !( *str>='a' && *str<='z') )
        return false;    //不以字母或下划线开头时, 返回 false(非标识符)
    char *p=str+1;        //使 p 指向字符串 str 的第二个字符
    while( *p )            //当 p 指向的字符并非串结束符 '\0' 时, 继续循环处理
    {
        if( (*p!='_') && !( *p>='A' && *p<='Z') && !( *p>='a' && *p<='z')
            && !( *p>='0' && *p<='9') )
            return false;
        //后跟的字符非字母又非下划线或数字时, 返回 false(非标识符)
        p++;              //p 下推, 指向字符串 str 的下一个字符
    }
}
```

```

        return true;
    //整个字符串处理结束，按标识符语法没有发现错误时，返回 true(是标识符)
}

```

[输出结果]

Input a string:

_ab12x

_ab12x -- is a IDENTIFIER!

再一次执行:

Input a string:

x1.txt

x1.txt -- is not a IDENTIFIER!

6.4 在字符串数组中查找给定串 string 的出现位置

设有如下的对字符串数组 `str` (其各分量 `str[i]` 均为字符串，也即 `str` 为二维字符数组) 的说明：`const int max=15; char str[max][81];`

编写自定义函数：`int SearchString(char s[ ][81], int n, char* string);`

它负责在字符串数组 `s` 的前 `n` 个元素中查找给定串 `string` 的出现位置(下标值)并返回，若 `s` 中不出现 `string` 的话，返回 -1。并编写主函数，输入若干个字符串放入 `str` 中，而后调用 `SearchString` 在 `str` 中查找某个给定串的位置。

[分析提示]

(1) 使用“`gets(str[i]);`”获取从键盘输入的若干个字符串(串中可以有空格，以回车结束串输入)并存放于 `str[i]` 所指向的各字符串中(注意，`str` 为二维字符数组，则 `str[i]` 为一维字符数组，而 `str[i][j]` 则代表一个字符)。

(2) 在 `SearchString` 函数中，通过调用标准库函数 `strcmp` 来进行两个字符串的比较，可使用如下形式的 `if` 语句来进行：`if( strcmp(s[i], string) == 0 ) return (i);`

即是说，用字符串 `string` 与 `s` 数组的元素(也为一个字符串)进行比较，若相等则返回 `s` 元素的下标值 `i`。

[解答程序]

```

#include <iostream.h>
#include <stdio.h>
#include <string.h>

int SearchString(char s[ ][81], int n, char* string);           //SearchString 的原型说明

//在字符串数组 s 的前 n 个元素中，查找给定串 string 的出现位置(下标值)

```


//并返回, 若 s 中不出现 string 的话, 返回-1。

```
void main( )
{
    int i,n;
    const int max=15;
    char str[max][81];          //数组 str, 最多可放 max 个字符串, 每个串不超过 80 字符
    char searchStr[81];         //要查找的字符串
    do
    {
        cout<<"Input a positive number n (0<n<15):";    //最多处理 n 个串
        cin>>n;                                           //输入 n
    } while (n<=0 || n>max);    //必须输入一个小于 max 的正整数 n
    cout<<"Input "<<n<<" STRINGS:"<<endl;               //提示输入 n 个串
    for(i=0; i<n; i++)
    {
        gets(str[i]);                                     //输入一个串(串中可以有空格, 以回车结束串输入)
        cout<<"Searching-str=? ";                       //提示输入 1 个欲查找的串
        cout.flush( );                                   //刷新屏幕
        gets(searchStr);                                  //输入一个串放在 searchStr 中
        int ind = SearchString(str, n, searchStr);        //调用 SearchString
        //在 str 数组的前 n 个串中, 查找 searchStr 串的出现位置
        cout<<"ind="<<ind<<endl; //输出结果(下标位置)
    }
}

int SearchString(char s[ ][81], int n, char* string)
{
    //在字符串数组 s 的前 n 个元素中, 查找给定串 string 的出现位置(下标值),
    //若 s 中不出现 string 的话, 返回-1。
    int i;
    for (i=0; i<n; i++)          //共比较 n 次
        if(strcmp(s[i], string)==0) //用 string 与 s 数组的诸元素一一比较
            return (i);          //比相等后返回下标值
    return(-1);                  //不等时返回-1
}
```

[输出结果]

Input a positive number n (0<n<15):3

Input 3 STRINGS:

123 abc!

```

I am a student.
China,OK!
Searching-str=? China,OK!
ind=2

```

6.5 通过指针参数来实现“双向传值”

编写具有如下函数原型的两个自定义函数：

```
void splitDouble(double x, int *intPart, double *fracPart);
```

它负责“分离”出 x 的整数部分与小数部分，分别放于 $*intPart$ 与 $*fracPart$ 处，由于形参 $intPart$ 与 $fracPart$ 都是指针，从而可实现“双向传值”而将这两个结果同时“带回去”。

```
double processingA(double *p, int n);
```

负责从指针 p 为首地址的位置开始，将最前面的 n 个 $double$ 型数由小到大排序，并求出这 n 个数的累加和作为函数结果而返回。

并编写主函数，对如下的数组 a ，通过调用上述两个函数，实现下述的功能：

```
double a[6]={2.2, 1.1, 44.44, 33.33, 666.666, 555.555};
```

(1) 执行 $processingA(a,6)$ ；而后将排完序的 a 数组数据输出；再将函数返回值(a 数组 6 个元素的累加和)输出；

(2) 通过多次调用 $splitDouble$ “分离”出 a 数组六个元素的整数与小数部分，并分别累加出 a 数组六个元素的整数部分之和 1301、以及六个元素的小数部分之和 2.291，而后输出它们。

[分析提示]

当被调函数的形参是指针，而又在该被调函数中更改此形参指针所指向的变量值的话(如， $splitDouble$ 函数中具有指针形参 $intPart$ ，而且在函数体中使用“ $*intPart=int(x)$ ；”的语句)，就意味着是更改主调函数中相应实参指针所指向的变量值，从而可实现所谓的“双向传值”。

[解答程序]

```

#include <iostream.h>

void splitDouble(double x, int *intPart, double *fracPart)
{
    //分离出整数与小数部分，分别放于*intPart 与*fracPart 处
    *intPart=int(x); //取 x 的整数部分存入 intPart 指针所指向的变量(空间)中
    //注意，通过修改指针参数所指向的内容更改了主调函数中
    //实参指针所指向的变量值，从而实现了“双向传值”的目的。
    *fracPart=x-*intPart;      //取 x 的小数部分存入*fracPart 中
}

```

```

    }

double processingA(double *p, int n)
{
    //负责从指针 p 为首地址的位置开始，将最前面的 n 个 double 型数
    //由小到大排序，并求出这 n 个数的累加和作为函数结果返回。
    int i,j;
    //从指针 p 为首地址，将最前面的 n 个 double 型数按升序排序
    for(i=0; i<n-1; i++)
        for (j=i+1; j<n; j++)
        {
            if(p[j]<p[i])           //按升序排序
            {
                double tmp=p[j];
                p[j]=p[i];
                p[i]=tmp;
            }
        }
    double s=0;                    //求出这 n 个数的累加和 s，并作为函数结果返回
    for (i=0; i<n; i++)
        s+=*p++;
    return s;
}

void main( )
{
    int i,intPt;
    double fracPt, a[6]={2.2, 1.1, 44.44, 666.666, 33.33, 555.555};
    double sum=0, sumInt=0, sumFrac=0;
    sum=processingA(a, 6);        //以数组名 a(为一个常量指针)作为实参，
                                //其对应形参为 double 型指针。
    //将 a 数组的前六个 double 型数按升序排序，并返回这六个数的累加和
    cout<<"Result a[6] = ";
    for(i=0; i<6; i++)           //将排完序的数组 a 的数据输出
        cout<<a[i]<<" ";
    cout<<endl;
    for(i=0; i<6; i++)           //六次调用 splitDouble 函数进行分离
    {
        splitDouble(a[i], &intPt, &fracPt);
    }
}

```

```

//第 2 与第 3 个实参均为变量地址，
//其对应形参分别为 int 型以及 double 型指针变量
sumInt+=intPt;    //往 sumInt 上累加通过参数“带回来”的结果 intPt
sumFrac+=fracPt;  //往 sumFrac 上累加通过参数“带回来”的结果 fracPt
}
cout<<"sum="<<sum<<endl;    //输出三个累加和
cout<<"sumInt="<<sumInt<<endl;
cout<<"sumFrac="<<sumFrac<<endl;
}

```

[输出结果]

```

Result a[6] = 1.1  2.2  33.33  44.44  555.555  666.666
sum=1303.29
sumInt=1301
sumFrac=2.291

```

6.6 字符指针与其他类型指针的使用区别

读下面的程序，并上机进行运行测试，看执行后会显示什么样的结果。注意字符指针与其他类型的指针（如 int 型指针）的使用区别（字符型指针具有特殊性!）。

```

#include <iostream.h>

void main( )
{
    char str[10]="abcdefghi";           //注 1
    int a[10]={0,1,2,3,4,5,6,7,8,9};    //注 2
    cout<<"cout<<str'=>"<<str<<endl;    //注 3
    cout<<"cout<<a'=>"<<a<<endl;        //注 4
    cout<<"-----"<<endl;
    cout<<"cout<<&str'=>"<<&str<<endl;    //注 5
    cout<<"cout<<&a'=>"<<&a<<endl;        //注 6
    cout<<"-----"<<endl;
    cout<<"cout<<(int*)str'=>"<<(int*)str<<endl;    //注 7
    cout<<"cout<<(double*)a'=>"<<(double*)a<<endl;    //注 8
    cout<<"-----"<<endl;
    cout<<"cout<<str+1'=>"<<str+1<<endl;    //注 9
    cout<<"cout<<a+1'=>"<<a+1<<endl;        //注 10
    cout<<"-----"<<endl;
    cout<<"cout<<&str[1]'=>"<<&str[1]<<endl;    //注 11
}

```

```

        cout<<"cout<<&a[1]'=>"<<&a[1]<<endl;           //注 12
        cout<<"-----"<<endl;
        cout<<"cout<<(int*)(str+1)'=>"<<(int*)(str+1)<<endl;   //注 13
        cout<<"cout<<(double*)(a+1)'=>"<<(double*)(a+1)<<endl; //注 14
        cout<<"-----"<<endl;
        cout<<"cout<<(double*)(&str[1])'=>"<<(double*)(&str[1])<<endl; //注 15
        cout<<"cout<<(double*)(&a[1])'=>"<<(double*)(&a[1])<<endl; //注 16
        cout<<"-----"<<endl;
        //cout<<"&(str+1)='"<<&(str+1)<<endl; //error C2102: '&' requires l-value
                                                    //注 17
        //cout<<"&(a+1)='"<<&(a+1)<<endl;    //'&' requires l-value    //注 18
    }

```

[分析提示]

字符指针与其他类型的指针(如 `int` 型指针)在使用上是有所区别的(字符指针具有其特殊性)。如,输出字符数组名即字符指针,就是输出字符指针所指向的那一字符串;而输出 `int` 型数组名即 `int` 型指针,就是输出当前的指针值(一个地址)。但输出字符数组名取地址,或输出 `int` 型数组名取地址,都可以输出当前的指针值(一个地址)。另外,若将字符数组名即字符指针的类型进行转换后,输出的将是当前的指针值(一个地址)。

[解答程序]

具体程序见题目处。

注 1: 字符数组名 `str`, 它为一个字符指针。

注 2: `int` 型数组名 `a`, 它为一个 `int` 型指针。

注 3: 输出字符数组名即字符指针, 就是输出字符指针所指向的那一字符串。

注 4: 输出 `int` 型数组名即 `int` 型指针, 就是输出当前的指针值(一个地址)。

注 5: 输出字符数组名取地址, 可以输出当前的指针值(一个地址)。

注 6: 输出 `int` 型数组名取地址, 可以输出当前的指针值(一个地址)。

注 7: 将字符数组名即字符指针的类型进行转换后, 输出的将是当前的指针值(一个地址)。

注 8: 将 `int` 型数组名即 `int` 型指针的类型进行转换后, 输出的仍将是当前的指针值(一个地址)。

注 9: 输出字符指针, 就是输出字符指针所指向的那一字符串(从第二个字符开始的那一个串)。

注 10: 输出 `int` 型指针, 就是输出当前的指针值(第二个数组元素的地址)。

注 11: 输出某字符的地址即指针, 就是输出字符指针所指向的那一字符串(从第二个字符开始的那个串)。

注 12: 输出某 `int` 型分量的地址即 `int` 型指针, 就是输出当前的指针值(第 2 个数组元素的地址)。

注 13: 将字符指针的类型进行转换后, 输出的将是当前的指针值 (一个地址)。

注 14: 将 int 型指针的类型进行转换后, 输出的仍将是当前的指针值 (一个地址)。

注 15: 将某字符的地址即指针的类型进行转换后, 输出的将是当前的指针值 (一个地址)。

注 16: 将某 int 型分量的地址即 int 型指针的类型进行转换后, 输出的仍将是当前的指针值 (一个地址)。

注 17: str+1 为一个常量地址, 不可以作为左值, 而取地址算符&的运算对象必须为左值。

注 18: a+1 为一个常量地址, 不可以作为左值, 而取地址算符&的运算对象必须为左值。

[输出结果]

```
'cout<<str'=>abcdefghi
'cout<<a'=>0x0065FDC4
-----
'cout<<&str'=>0x0065FDEC
'cout<<&a'=>0x0065FDC4
-----
'cout<<(int*)str'=>0x0065FDEC
'cout<<(double*)a'=>0x0065FDC4
-----
'cout<<str+1'=>bcdefghi
'cout<<a+1'=>0x0065FDC8
-----
'cout<<&str[1]'=>bcdefghi
'cout<<&a[1]'=>0x0065FDC8
-----
'cout<<(int*)(str+1)'=>0x0065FDED
'cout<<(double*)(a+1)'=>0x0065FDC8
-----
'cout<<(double*)(&str[1])'=>0x0065FDED
'cout<<(double*)(&a[1])'=>0x0065FDC8
-----
```

6.7 输入 10 个整数而后反序输出的实现方式五

读下述程序, 并上机进行验证, 看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意, 本程序使用 new 运算符动态申请数据空间存放输入的数据。

```
#include <iostream.h>

void main( )
{
    int i, *b, *p;                //注 1
    b=new int[10];                //注 2
    cout<<"input 10 integers:"<<endl;
    for(i=0; i<10; i++)           //注 3
        cin>>*(b+i);             //注 4
    cout<<"---- The result ----"<<endl;
    for(p=b+9; p>=b; p--)         //注 5
        cout<<*p<<" ";          //注 6
    cout<<endl;
}
```

[分析提示]

首先通过“ new int[10]”的方式动态申请可存放 10 个 int 数据的数组空间，而后通过指针(定位)将输入数据放入动态申请到的数组分量空间中，之后再按反序输出它们。

[解答程序]

具体程序见题目处。

注 1: 说明指针变量 b 及指针变量 p。

注 2: 使用 new 运算符动态申请可存放 10 个 int 数据的数组 b(用于存放数据)。

注 3: 输入数据放入动态申请到的数组 b 中。

注 4: *(b+i) 全同于 b[i]。

注 5: 循环过程中 p 值总在变化(通过指针加减运算), p>=b 为指针比较运算。

注 6: 输出各*p, 也即各不同的 b[i]。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
----- The result -----
10 9 8 7 6 5 4 3 2 1
```

6.8 测试并运行含有 new 与 delete 的指定程序

上机运行下述含有 new 与 delete 的程序,通过分析所显示的结果,来理解运算符 delete 的功能。进一步,将 delete 语句删除掉后再执行一遍,看显示结果有什么不同。再试一下看若删除 delete 且将 for 语句改成为无限循环的话,程序是否就不会停止了昵?

```
#include <iostream.h>
void main( )
{
    struct stru                                //注 1
    {
        int a[10000];
    }
    stru* tmp;                                  //注 2
    int count=0;
    cout<<"sizeof(stru)="<<sizeof(stru)<<endl;    //注 3
    for (int i=0; i<10; i++)
    {
        count++;
        tmp=new stru[100];                      //注 4
        cout<<"count="<<count<<"  tmp="<<tmp<<endl;
        if (tmp==NULL) break;                  //注 5
        delete tmp;                            //注 6
    }
}
```

[分析提示]

(1) 本程序中的 for 语句使得其循环体要执行 10 遍，而每一次循环总是先通过“ tmp = new stru[100];”来申请一块动态存储空间(并将该块空间的首地址赋给了 tmp 变量)，而后又通过其后的“ delete tmp;”来释放掉此块空间(归还给了系统，使得下次还可以对此块空间进行重新分配使用)。所以，10 次循环中输出的 tmp 值都大小一样。

(2) 将 delete 语句删除掉后再执行，屏幕显示的结果则出现不同——因为只分配，不归还，10 次循环中输出的 tmp 值将完全不相同。

(3) 若删除 delete 且将 for 语句改成为无限循环的话，程序还是会停止的。其原因是，由于总通过 new 分配新内存空间，但又从不归还它们(因为没有 delete!)，而可供编译器分配的动态存储空间又是有限的，从而当再没有动态存储空间可供分配的时候(此时通过 new 返回的指针值为 NULL)，将会跳出 for 循环而终止程序。

[解答程序]

具体程序见题目处。

注 1：自定义结构体类型 stru。

注 2：指向 stru 的指针 tmp。

注 3：输出 stru 结构体的大小。

注 4：通过 new 申请动态空间。

注 5：若分配失败则跳出循环而结束程序。

注 6：通过 delete 释放动态空间。

[输出结果]

```
sizeof(stru)=40000
count=1  tmp=0x0088002C
count=2  tmp=0x0088002C
count=3  tmp=0x0088002C
count=4  tmp=0x0088002C
count=5  tmp=0x0088002C
count=6  tmp=0x0088002C
count=7  tmp=0x0088002C
count=8  tmp=0x0088002C
count=9  tmp=0x0088002C
count=10 tmp=0x0088002C
```

将 delete 语句删除掉后再执行，屏幕显示结果为：

```
sizeof(stru)=40000
count=1  tmp=0x0088002C
count=2  tmp=0x00C8002C
count=3  tmp=0x0108002C
count=4  tmp=0x0148002C
count=5  tmp=0x0188002C
count=6  tmp=0x01C8002C
count=7  tmp=0x0208002C
count=8  tmp=0x0248002C
count=9  tmp=0x0288002C
count=10 tmp=0x02C8002C
```

6.9 实现具有动态大小的矩阵乘法

编程序，实现具有动态大小（矩阵行列数由用户任意指定）的矩阵乘法。具体做法如下：

(1) 通过二重指针及 new 运算符，动态申请三个行列数有关系的二维数组用于表示矩阵 A、B、C（假设要计算 $C=A*B$ ，即矩阵乘），其中 A 为 m 行 n 列、B 为 n 行 m 列、C 为 m 行 m 列（A 矩阵的列数要等于 B 矩阵的行数），而且正整数 m 和 n 也是由用户从键盘输入而任意指定的。

(2) 按如下简单方式给 A 矩阵赋以初值：第一行元素全赋值为 1，第二行元素全赋值为 2，...，第 m 行元素全赋值为 m；并将 B 矩阵设置为 A 的转置矩阵。

(3) 按下述方法计算 C 矩阵的每一个元素 $C[i][j]$ (其中 $i=0, 1, \dots, m-1$; $j=0, 1, \dots, m-1$):

```
{
    s=0;           //A 矩阵的第 i 行与 B 矩阵的第 j 列诸元素相乘后累加到 s 上
    s+=A[i][k]*B[k][j];    //(k=0, 1, ..., n-1)
    C[i][j]=s;      //s 值即为 C[i][j]
}
```

[分析提示]

通过二重指针及 new 运算符, 来动态申请所需大小的二维数组存储空间(注意, 使用类似于“`int A[m][n];`”的数组说明只能分配到固定大小的存储空间, 因为语法上要求数组说明处用来指定维数大小的 m 与 n 都必须为常量!)。可以采用如下方法来生成具有 m 行 n 列的动态大小的二维数组(m 与 n 为变量, 其值要在运行时确定, 如通过输入来确定):

```
int **A;           //二重指针, 指向 A 数组( 矩阵) 的“ 头” 位置
A = new int*[m];   //生成动态数组, 该数组具有 m 个 int 类型的指针分量
for(i=0; i<m; i++)
    A[i]=new int[n]; //为上述 m 个 int 类型的指针分量空间赋上指针值
// 意味着为每行分配了 n 个 int 型存储空间, 并将各空间首地址
//赋给了 A[i]
```

[解答程序]

```
#include <iostream.h>
#include <iomanip.h>
void main(void)
{
    int m, n;           //m, n 表示矩阵的行列数
    int i, j, k;        //用作循环变量
    cout<<"Input m, n ( A -- m*n;  B -- n*m ):";    //提示输入
    cin>>m>>n;        //输入 m, n
    int **A, **B, **C; //三个二重指针, 分别指向矩阵 A、B、C 的“ 头” 位置
    A=new int*[m];      //动态生成 m 行 n 列的矩阵 A 所需的存储空间 A[m][n]
    for(i=0; i<m; i++)
        A[i]=new int[n];
    B=new int*[n];      //动态生成 n 行 m 列的矩阵 B 所需的存储空间 B[n][m]
    for(i=0; i<n; i++)
        B[i]=new int[m];
    C=new int*[m];      //动态生成 m 行 m 列的矩阵 C 所需的存储空间 C[m][m]
    for(i=0; i<m; i++)
```

```

        C[i]=new int[m];
    for(i=0; i<m; i++)
        //给 A 矩阵置初值, 第一行元素全赋值为 1, 第二行元素全赋值为 2, ...
        for(j=0; j<n; j++)
            A[i][j]=i+1;
    for(i=0; i<n; i++)                //给矩阵 B 置初值, 它为矩阵 A 的转置矩阵
        for(j=0; j<m; j++)
            B[i][j]=A[j][i];
    //计算 C=A*B( 即矩阵乘)
    for(i=0; i<m; i++)                //数组 C 共 m 行
        for(j=0; j<m; j++)            //每行 m 个元素(共 m 列)
        {
            int s=0;                  //矩阵 A 的第 i 行与矩阵 B 的第 j 列诸元素相乘后累加到 s 上
            for(k=0; k<n; k++)        //共 n 对元素相乘
                s+=A[i][k]*B[k][j];  //累加到 s 上
            C[i][j]=s;                //s 值即为 C[i][j]
        }
    cout<<"----- A -----"<<endl;
    for (i=0; i<m; i++)                //输出数组 A 的元素, 共 m 行
    {
        for (j=0; j<n; j++)            //每行 n 个元素(共 n 列)
            cout<<setw(4)<<A[i][j];
        cout << endl;
    }
    cout<<"----- B -----"<<endl;
    for(i=0; i<n; i++)                //输出数组 B 的元素, 共 n 行
    {
        for(j=0; j<m; j++)            //每行 m 个元素(共 m 列)
            cout<<setw(4)<<B[i][j];
        cout<<endl;
    }
    cout<<"----- C=A*B -----"<<endl;
    for(i=0; i<m; i++)                //输出数组 C 元素, 共 m 行
    {
        for(j=0; j<m; j++)            //每行 m 个元素(共 m 列)
            cout<<setw(4)<<C[i][j];
        cout<<endl;
    }
}

```

[输出结果]

Input m, n (A -- m*n; B -- n*m):3 4

----- A -----

```
1  1  1  1
2  2  2  2
3  3  3  3
```

----- B -----

```
1  2  3
1  2  3
1  2  3
1  2  3
```

----- C=A*B -----

```
4  8 12
8 16 24
12 24 36
```

6.10 求解线性方程组

求解具有 `lines` 元的线性方程组(假设方程组具有惟一解, 消解时不选主元)。有如下几点要求:

- (1) 方程个数(即维数或行数) `lines` 之值由用户任意输入;
- (2) 方程组放于增广矩阵 `MatrA` 之中, 而 `MatrA`(`lines` 行 `lines+1` 列) 的存储空间通过 `new` 来动态分配, 其各元素值也由用户任意输入(从而使指定的线性方程组也具有任意性);
- (3) 方程组的解放在向量 `solution` 之中, 而 `solution`(具有 `lines` 个元素) 的存储空间也通过 `new` 来动态分配。

下面给出实现方式之一的程序核心代码。

```
int lines;           //矩阵的维数(行数), 全局变量
double** MatrA;     //全局变量, 数组首地址,
                    //存放增广矩阵( 系数矩阵 + 方程组右端向量 )

double *solution;   //存放方程组的解
void Linequ(int Lns); //构造函数, 形参为维数(行数),
                    //动态分配 MatrA 的存储空间(Lns 行 Lns+1 列的增广矩阵空间)

void _Linequ();     //析构函数, 释放 MatrA 的存储空间
void InMatrix();    //输入增广矩阵 MatrA 的初值
void printL();      //显示线性方程组(增广矩阵 MatrA 的数据)
void showX();       //输出方程组的解(solution 数据)
void Solve();       //不选主元消去法
```

//(对增广矩阵 MatrA 进行同解变换, 将求出的解放于 solution 之中)

```
void main( )           //主函数
{
    cout<<"Input LINES:";    //屏幕提示输入矩阵行数
    cin>>lines;            //输入行数
    Linequ(lines);        //动态生成一个具有 lines 行的增广矩阵 MatrA
                          //(形参为维数, 动态分配 MatrA 的存储空间)

    cout<<"Input "<<lines*(lines+1)<<" numbers that will form the
        Line_equation:"<<endl;

                          //提示输入矩阵数据
    InMatrix( );          //输入增广矩阵 MatrA 的数据
    printL( );            //输出方程组(以增广矩阵形式)
    Solve( );             //求解方程组(对增广矩阵 MatrA 进行同解变
                          //换, 将求出的解放于 solution 之中)

    showX( );             //输出方程组的解(solution 中的数据)
    ~Linequ( );           //析构函数, 释放所动态分配的存储空间
}
```

进一步, 在上述实现的基础上(消解时不选主元), 而改进成为列主元消去法的求解方式。

[分析提示]

(1)使用与上一范例相同的方法, 通过指针与运算符 new 来生成所需的多个动态数组存储空间(如存放增广矩阵 MatrA 以及方程组解向量的 solution 等存储空间)。

(2) 将整个求解任务进行分解, 设计出上述的各负其责的多个自定义函数。上述所提到的构造函数以及析构函数是类与对象处理方法中的重要概念, 将在后面的章节中频繁使用。此例的函数及其功能设计是今后使用类与对象方法进行处理的基础。

(3) 使用列主元消去法进行求解时, 只需在解答程序一(消解时不选主元)的实现基础上改进其中的 Solve 函数, 使它能够首先选取列主元, 而后进行消元求解。

[解答程序一]

```
#include <iostream.h>
#include <iomanip.h>

int lines;                //矩阵的维数 (行数), 全局变量
double** MatrA;           //全局变量, 数组首地址,
                          //存放增广矩阵 (系数矩阵 + 方程组右端向量)

double *solution;         //存放方程组的解

void Linequ(int Lns);      //构造函数, 形参为维数 (行数),
                          //动态分配 MatrA 的存储空间(Lns 行 Lns+1 列的增广矩阵空间)
```

```

void ~Linequ( );           //析构函数释放 MatrA 的存储空间
void InMatrix( );         //输入增广矩阵 MatrA 的初值
void printL( );           //显示线性方程组 (增广矩阵 MatrA 的数据)
void showX( );            //输出方程组的解 (solution 数据)
void Solve( );            //不选主元消去法 (对增广矩阵 MatrA 进行同解变换,
                          //将求出的解放于 solution 之中)

```

```

void main( )              //主函数
{
    ...                   //与题目处相同,略
}

```

//各函数的实现

```

void Linequ(int Lns)      //构造函数, 形参为维数(行数), 动态分配 MatrA 的存储空间
{
    lines=Lns;            //矩阵维数(行数)
    MatrA=new double*[lines]; //动态内存分配(存放增广矩阵), 共 lines 行
    for(int i=0; i<lines; i++)
        MatrA[i]=new double[lines+1]; //每行 lines+1 列
    solution=new double[Lns]; //solution 存放方程组的解
}

```

```

void ~Linequ( )          //析构函数, 释放 solution 存储空间以及 MatrA 的存储空间
{
    delete[ ] solution;  //释放内存
    for(int i=0; i<lines; i++)
        delete[ ] MatrA[i]; //释放内存
    delete[ ] MatrA;      //释放内存
}

```

```

void InMatrix( )         //输入增广矩阵初值, 它表示欲求解的线性方程组
{
    int i,j;
    for(i=0; i<lines; i++) //共 lines 行
        for(j=0; j<lines+1; j++) //每行 lines+1 列
            cin>>MatrA[i][j];
}

```

```

void printL( )           //显示线性方程组(以增广矩阵形式)

```

```

{
    cout<<"The Line_equation is:"<<endl;
    cout<<"-----"<<endl;
    for(int i=0; i<lines; i++) {                //共显示 lines 行
        for(int j=0; j<lines+1; j++)            //每行显示 lines+1 个数(增广矩阵数据)
            cout<<setw(10)<<MatrA[i][j]<<" ";    //每数占 10 格(宽度)
        cout<<endl;
    }
}

void showX( )                                //输出方程组的解
{
    cout<<"-----"<<endl;
    cout<<"The Result is:"<<endl;
    cout<<"-----"<<endl;
    for(int i=0; i<lines; i++)                //显示 solution[i]的各值
        cout<<"X["<<i<<"]= "<<solution[i]<<endl;
}

void Solve( )                                //不选主元消去法
{
    int k,i,j;
    double d,t;
    for (k=1; k<lines; k++)                    //消去过程——将系数矩阵消为上三角阵
    {
        for (i=k; i<lines; i++)
        {
            double factor=-MatrA[i][k-1]/MatrA[k-1][k-1];
            for (j=k-1; j<lines+1; j++)
                MatrA[i][j]+=MatrA[k-1][j]*factor;
        }
    }                                           //消去过程结束
    d=MatrA[lines-1][lines-1];
    solution[lines-1]=MatrA[lines-1][lines]/d;    //回代过程

    for (i=lines-2; i>=0; i--)
    {
        t=0.0;
        for (j=i+1; j<=lines-1; j++)

```

```

        t=t+MatrA[ i ][ j ]*solution[ j ];
        solution[i]=(MatrA[ i ][lines]-t)/MatrA[ i ][ i ];    //求出各解放于 solution[ i ]之中
    }
}

```

[输出结果]

Input LINES:2

Input 6 numbers that will form the Line_equation:

1 2 3 4 5 6

The Line_equation is:

```

-----
          1          2          3
          4          5          6
-----

```

The Result is:

X[0]=-1

X[1]=2

再一次执行:

Input LINES:4

Input 20 numbers that will form the Line_equation:

0.2815 0.2271 0.2018 1.4396 12.1126

0.1754 0.1958 1.5211 0.5893 25.5612

0.1788 1.3366 0.1832 0.9235 26.8501

1.9876 0.1234 0.2628 0.8821 52.2683

The Line_equation is:

```

-----
    0.2815    0.2271    0.2018    1.4396    12.1126
    0.1754    0.1958    1.5211    0.5893    25.5612
    0.1788    1.3366    0.1832    0.9235    26.8501
    1.9876    0.1234    0.2628    0.8821    52.2683
-----

```

The Result is:

X[0]=23.9054

X[1]=15.5057

X[2]=12.2142

X[3]=-0.418835

[解答程序二]

只需在解答程序一(消解时不选主元)的实现基础上改进其中的 Solve 函数,使它
能够首先选取列主元,而后进行消元求解。下面只给出改进后的 Solve 函数。

```
void Solve( )                //使用列主元消去法求解
{
    int k,i,j;
    double d,t;
    for (k=1; k<lines; k++)    //消去过程
    {
        //先在第 k-1 行到 lines-1 行的范围内进行行互换,
        //使 MatrA[k-1][k-1] 在第 k-1 列中为最大元, 后再进行消元过程
        double max=MatrA[k-1][k-1];
        int ind=k-1;
        for (i=k; i<lines; i++)    //找列中最大元所在行 ind
            if(MatrA[i][k-1]>max)
            {
                max=MatrA[ i ][k-1];
                ind=i;
            }
        if(ind!=k-1)            //第 k-1 行与第 ind 行进行整行互换
            for (j=k-1; j<lines+1; j++)
            {
                double tmp=MatrA[k-1][ j ];
                MatrA[k-1][ j ]=MatrA[ind][ j ];
                MatrA[ind][j]=tmp;
            }
        for (i=k; i<lines; i++)    //消去过程——将系数矩阵消为上三角阵
        {
            double factor=-MatrA[i][k-1]/MatrA[k-1][k-1];
            for (j=k-1; j<lines+1; j++)
                MatrA[i][j]+=MatrA[k-1][j]*factor;
        }
    }                            //消去过程结束
    d=MatrA[lines-1][lines-1];
    solution[lines-1]=MatrA[lines-1][lines]/d;    //回代过程
    for (i=lines-2; i>=0; i--)
    {
        t=0.0;
        for (j=i+1; j<=lines-1; j++)
```



```

        pi22++;                                //注 11
        ri3++;    (*str)++;                    //注 12
        cout<<"----- In myFunc position 2 -----"<<endl;
        cout<<"i1,*pi21,*pi22,ri3,str = "<<i1<<" "<<*pi21<<" ";
        cout<<*pi22<<" "<<ri3<<" "<<str<<endl;    //注 13
        i1++;(*pi21)++;                        //注 14
        (*pi22)++;    ri3++;    str++;        //注 15
        cout<<"----- In myFunc position 3 -----"<<endl;
        cout<<"i1,*pi21,*pi22,ri3,str = "<<i1<<" "<<*pi21<<" ";
        cout<<*pi22<<" "<<ri3<<" "<<str<<endl;    //注 16
        return i1;
    }

```

[分析提示]

系统对赋值参数、指针参数以及引用参数各自的处理是不相同的。设参数 **a** 为赋值参数(其对应实参可为表达式),被调函数中对形参 **a**(所拥有值)的改变不影响对应实参;设参数 **a** 为指针参数(其对应实参为具有指针值的表达式),被调函数中对形参指针 **a**(所拥有值)的改变不影响对应实参,但被调函数中对形参指针 **a** 所指变量***a** 值的改变将影响对应实参指针所指变量的值(双向传递的含义);设参数 **a** 为引用参数(其对应实参必须为同类型的变量),被调函数中对形参 **a**(所拥有值)的改变将直接影响对应的实参变量(引用形参是对应实参变量的“替身”,对引用形参值的改变就意味着对其对应实参变量值的直接改变)。

[解答程序]

具体程序见题目处。

注 1: **i1** 为赋值参数, **pi21** 为指针参数, **pi22** 为指针引用参数, **ri3** 为引用参数, **str** 为指针参数。

注 2: **i, *pa, *pb, r, s = 1 21 21 3456 ABCDEFG**

注 3: **a[0],a[1], b[0],b[1]= 21 202 21 202**

注 4: **i** 对应于赋值参数 **i1**,被调函数中对形参 **i1** 值的改变不影响 **i**; **pa** 对应于指针参数 **pi21**,被调函数中对形参 **pi21** 值的改变不影响 **pa**,但被调函数中对形参 **pi21** 所指变量***pi21** 值的改变将影响***pa**; **pb** 对应于指针引用参数 **pi22**,被调函数中对形参 **pi22** 值的改变将直接改变 **pb**,被调函数中对形参 **pi22** 所指变量***pi22** 值的改变也将影响***pb**; **r** 对应于引用参数 **ri3**,被调函数中对形参 **ri3** 值的改变将直接改变 **r**; **s** 对应于指针参数 **str**,被调函数中对形参 **str** 值的改变不影响 **s**,但被调函数中对形参 **str** 所指变量***str** 值的改变将影响***s**。

注 5: **i, *pa, *pb, r, s = 1 21 203 3458 BBCDEFG**

注 6: **a[0],a[1], b[0],b[1]= 21 203 21 203**

注 7: **tmp = 3**

注 8: i1 为赋值参数, pi21 为指针参数, pi22 为指针引用参数, ri3 为引用参数, str 为指针参数。

注 9: i1,*pi21,*pi22,ri3,str = 1 21 21 3456 ABCDEFG

注 10: 被调函数中对形参 pi21 值的改变不影响对应实参的值。

注 11: 被调函数中指针引用参数 pi22 值的改变将直接改变对应实参的值。

注 12: 被调函数中对形参 str 所指变量*str 值的改变将影响实参指针所指变量的值。

注 13: i1,*pi21,*pi22,ri3,str = 2 202 202 3457 BBCDEFG

注 14: 被调函数中对形参 pi21 所指变量*pi21 值的改变将影响实参指针所指变量的值。

注 15: 被调函数中对形参 str 值的改变不影响对应实参的值。

注 16: i1,*pi21,*pi22,ri3,str = 3 203 203 3458 BCDEFG

[输出结果]

```

----- In main position 1 -----
i, *pa, *pb, r, s = 1 21 21 3456 ABCDEFG
a[0],a[1], b[0],b[1]= 21 202 21 202
----- In myFunc position 1 -----
i1,*pi21,*pi22,ri3,str = 1 21 21 3456 ABCDEFG
----- In myFunc position 2 -----
i1,*pi21,*pi22,ri3,str = 2 202 202 3457 BBCDEFG
----- In myFunc position 3 -----
i1,*pi21,*pi22,ri3,str = 3 203 203 3458 BCDEFG
----- In main position 2 -----
i, *pa, *pb, r, s = 1 21 203 3458 BBCDEFG
a[0],a[1], b[0],b[1]= 21 203 21 203
tmp = 3

```

6.12 求满秩矩阵的逆矩阵

求具有 lines 行的满秩矩阵的逆矩阵。给出以下几个要点:

- (1) 矩阵行数(阶数) lines 之值由用户任意输入;
- (2) 先将欲求逆的原始矩阵另加一个单位矩阵放于 MatrA 之中, 而 MatrA(lines 行、2*lines 列)的存储空间通过 new 来动态分配, 而原始矩阵的各元素值也由用户任意输入;
- (3) 设法将 MatrA 左半的原始矩阵化为单位矩阵 (利用行初等变换), 此时, 右半的原单位矩阵则变成了欲求的结果矩阵(即原始矩阵的逆矩阵)。

下面给出实现方式之一的程序核心代码。

```

int lines;                //矩阵的行数(阶数), 全局变量
double** MatrA;

```

```

//全局变量，数组首地址，存放矩阵。初始时为原始矩阵+单位矩阵
void Matrix(int Lns);
//构造函数语义，形参为行数(阶数)，动态分配 MatrA 的存储空间
void ~Matrix( );           //析构函数语义，释放 MatrA 的存储空间
void InMatrix( );          //输入原始矩阵初值
void printMatrix( );       //显示整体矩阵
void printSourceMatrix( ); //显示原始矩阵
void printResultMatrix( ); //显示结果矩阵
void Solve( );             //求解矩阵的逆

void main( )               //主函数
{
    cout<<"Input LINES:"; //屏幕提示输入矩阵行数(阶数)
    cin>>lines;           //输入行数
    Matrix(lines);        //动态生成一个具有 lines 行 2*lines 列的矩阵 MatrA
    cout<<"Input "<<lines*lines<<" numbers that will form the matrix:"<<endl;
                                //提示输入原始矩阵数据
    InMatrix( );          //输入原始矩阵数据
    cout<<"The source Matrix is:"<<endl;
    printSourceMatrix( ); //输出原始矩阵
    Solve( );             //求解矩阵的逆 (解处于 MatrA 矩阵的右半部分)
    cout<<"The result Matrix is:"<<endl;
    printResultMatrix( ); //输出结果矩阵 (MatrA 矩阵右半部分的数据)
    _Matrix( );           //释放动态空间
}

```

[分析提示]

(1) 通过指针与 new 运算符来生成所需的动态数组存储空间(如存放原始矩阵另加一个单位矩阵的 MatrA 存储空间)。

(2) 将整个求解任务进行分解，设计出上述的各负其责的多个自定义函数。其中也提到了构造函数以及析构函数的使用含义。此例的函数及其功能设计也为今后使用类与对象的处理方法奠定了基础。

(3) 求取逆矩阵的具体做法为：利用行初等变换，设法将 MatrA 左半的原始矩阵化为单位矩阵，此时，右半的原单位矩阵则变成了欲求的逆矩阵。

[解答程序]

```

#include <iostream.h>
#include <iomanip.h>
int lines;                //矩阵的维数 (行数)，全局变量

```

```

double** MatrA;
    //全局变量，数组首地址，存放矩阵。初始时为原始矩阵+单位矩阵
void Matrix(int Lns);
    //构造函数形参为维数（行数），动态分配 MatrA 的存储空间
void ~Matrix( );
    //析构函数，释放 MatrA 的存储空间
void InMatrix( );
    //输入原始矩阵初值
void printMatrix( );
    //显示整体矩阵
void printSourceMatrix( );
    //显示原始矩阵
void printResultMatrix( );
    //显示结果矩阵(MatrA 矩阵右半部分的数据)
void Solve( );
    //求解矩阵的逆(解处于 MatrA 矩阵的右半部分)

void main( )
    //主函数
{
    ...
    //与题目处相同,略
}

//各函数的实现
void Matrix(int Lns)
    //构造函数语义，形参为维数(行数)，动态分配 MatrA 的存储空间
{
    lines=Lns;
    //矩阵维数(行数)
    MatrA=new double*[lines];
    //动态内存分配(存放矩阵)，共 lines 行
    for(int i=0; i<lines; i++)
        MatrA[i]=new double[2*lines];
    //每行 2*lines 列
}

void ~Matrix( )
    //析构函数，释放 MatrA 的存储空间
{
    for(int i=0; i<lines; i++)
        delete[ ] MatrA[i];
    //释放内存
    delete[ ] MatrA;
    //释放内存
}

void InMatrix( )
    //输入原始矩阵初值，并形成右半部分的单位矩阵
{
    int i,j;
    for(i=0; i<lines; i++)
        //输入原始矩阵初值，放于左半部分
        for(j=0; j<lines; j++)
            cin>>MatrA[ i ][ j ];
    for(i=0; i<lines; i++)
        //右半部分的单位矩阵先全置为 0

```



```

int k,i,j;
for (k=1; k<lines; k++) //左半部分消为上三角阵
{
    for (i=k; i<lines; i++)
    {
        double factor=-MatrA[ i ][ k-1 ]/MatrA[ k-1 ][ k-1 ];
        for (j=k-1; j<2*lines; j++)
            MatrA[i][j]+=MatrA[ k-1 ][ j ]*factor;
    }
}
//printMatrix( ); //输出矩阵(左半部分为上三角阵)
for (i=0; i<lines; i++) //左半部分主对角线消为 1
{
    double factor=MatrA[ i ][ i ];
    for (j=i; j<2*lines; j++)
        MatrA[i][j]/=factor;
}
//printMatrix( ); //输出矩阵(左半部分主对角线为 1)
for (k=lines-2; k>=0; k--) //左半部分消为单位矩阵，此时右半部分成为结果
{
    for (i=k; i>=0; i--)
    {
        double factor=-MatrA[ i ][ k+1 ];
        for (j=k+1; j<2*lines; j++)
            MatrA[i][j]+=(factor*MatrA[ k+1 ][ j ]);
    }
}
}

```

[输出结果]

Input LINES:2

Input 4 numbers that will form the matrix:

1 2 3 4

The source Matrix is:

1	2
3	4

The result Matrix is:

-2	1
1.5	-0.5

再一次执行:

Input LINES:4

Input 16 numbers that will form the matrix:

1 1 1 2 1 2 2 1 1 2 1 2 1 2 1 1

The source Matrix is:

1	1	1	2
1	2	2	1
1	2	1	2
1	2	1	1

The result Matrix is:

2	-1	-3	3
-1	0	1	0
0	1	0	-1
0	0	1	-1

6.13 有关指针数组(各分量均为指针)的使用

读下面的程序, 其中使用了指针数组(各分量均为指针), 请写出执行结果。

```
#include <iostream.h>
void main( )
{
    int i,j;
    char s1[ ]="aa 111", s2[ ]="bbb 2222", s3[ ]="cccc 333333";    //注 1
    char *pc[3];                                                    //注 2
    pc[0]=s1;                                                        //注 3
    pc[1]=s2;
    pc[2]=s3;
    for(i=0; i<3; i++)                                              //注 4
        cout<<pc[ i ]<<endl;                                       //注 5
    cout<<"-----"<<endl;
    for(i=0; i<3; i++)
    {
        j=0;
        char ch;
        while(ch=pc[ i ][ j++ ])
            cout<<ch;                                                //注 6
        cout<<endl;
    }
}
```

```

    }
    cout<<"-----"<<endl;
    int a[ ]={1,2,3}, b[ ]={4,5,6}, c[ ]={7,8,9};           //注 7
    int *pi[ 3 ];                                           //注 8
    pi[0]=a;                                                //注 9
    pi[1]=b;
    pi[2]=c;
    for(i=0; i<3; i++)                                     //注 10
    {
        for(j=0; j<3; j++)                                 //注 11
            cout<<*( pi[ i ]+j )<<" ";                  //注 12
        cout<<endl;
    }
}

```

[分析提示]

(1) 使用“ char *pc[3];”来说明指针数组 pc，其 3 个分量均为指向 char 型数据的指针，从而可通过使用指针数组的各元素 pc[i](均为指针)来进行字符串的有关处理。

(2) 使用“ int *pi[3]; ”来说明指针数组 pi，其 3 个分量均为指向 int 型数据的指针，从而可通过使用指针数组的各元素 pi[i](均为指针)来对相关联的 int 型数组 a、b、c 的数据进行处理。

[解答程序]

具体程序见题目处。

注 1：说明 3 个 char 型数组，并赋了初值。

注 2：指针数组 pc，其各分量均为指向 char 型数据的指针。

注 3：数组名 s1 为常量指针，等同于&s1[0]。

注 4：通过使用指针数组的各元素(均为指针)来进行字符串输出。

注 5：等同于: cout<<*(pc+i)<<endl;

注 6：注意，pc[i][j++] 等同于 *(pc[i]+j++)，也等同于 (*(pc+i)+j++)。

注 7：说明 3 个 int 型数组。

注 8：指针数组 pi，其各分量均为指向 int 型数据的指针。

注 9：数组名 a 为常量指针，等同于&a[0]。

注 10：通过使用指针数组的各元素 pi[i](均为指针)来进行 a、b、c 数组数据的输出。

注 11：对矩阵每一元素循环。

注 12：等同于: cout<<pi[i][j]<<" "; 也等同于: cout<<*(pi+i+j)<<" ";

[输出结果]

aa 111

```

bbb 2222
cccc 333333
-----
aa 111
bbb 2222
cccc 333333
-----
1  2  3
4  5  6
7  8  9

```

6.14 指向一维数组的指针及其使用

读程序 (其中使用了指向一维数组的指针 p), 写出执行结果。

```

#include <iostream.h>
void main(void)
{
    int a[ 3 ][ 3 ]={1,2,3,4,5,6,7,8,9};
    int (*p)[ 3 ], i=2, j=0;                                //注 1
    cout<<"int a[ 3 ][ 3 ]={1,2,3,4,5,6,7,8,9};"<<endl;
    cout<<"int (*p)[ 3 ], i=2, j=0;"<<endl;
    cout<<"-----"<<endl;
    cout<<"a[ i ][ j ]="<<a[ i ][ j ]<<endl;                //注 2
    cout<<"*(a[ i ]+j)="<<*(a[ i ]+j)<<endl;
    cout<<"*(*(a+i)+j)="<<*(*(a+i)+j)<<endl;
    p=&a[1];                                                  //注 3
    cout<<"-- After 'p=&a[ 1 ];' --"<<endl;
    cout<<"(*p)[ 0 ]="<<(*p)[ 0 ]<<endl;                    //注 4
    cout<<"*(p+0)[ 0 ]="<<*(p+0)[ 0 ]<<endl;                //注 5
    cout<<"p[ 0 ][ 0 ]="<<p[ 0 ][ 0 ]<<endl;                 //注 6
    cout<<"(*p)[ 1 ]="<<(*p)[ 1 ]<<endl;                    //注 7
    cout<<"(*p+1)[ 0 ]="<<(*p+1)[ 0 ]<<endl;                //注 8
    cout<<"*(p+1)[ 0 ]="<<*(p+1)[ 0 ]<<endl;                //注 9
    cout<<"p[ 1 ][ 0 ]="<<p[ 1 ][ 0 ]<<endl;                 //注 10
    p++;                                                       //注 11
    cout<<"-- After 'p++;' --"<<endl;
    cout<<"(*p)[ 2 ]="<<(*p)[ 2 ]<<endl;                    //注 12
    cout<<"p[ 0 ][ 2 ]="<<p[ 0 ][ 2 ]<<endl;

```

```

        cout<<"*(p[ 0 ]+2)="<<*(p[ 0 ]+2)<<endl;
        cout<<"*( *p+2)="<<*( *p+2)<<endl;
    }

```

[分析提示]

通过“`int (*p)[3];`”，说明 `p` 为指针，指向一维数组，数组包含三个 `int` 型分量。可理解为 `*p` 为具有 3 个 `int` 型分量的一维数组。注意 `p+1` 也是一个指针，它指向的新位置为：从当前 `p` 位置下推定义 `p` 时那一个一维数组的元素个数，即下推 3 个 `int` 型元素后的那一位置。

[解答程序]

具体程序见题目处。

注 1：说明一个指向一维数组的指针 `p`，一维数组由 3 个元素构成。

注 2：`a[ i ][ j ]`等价于`*(a[ i ]+j)`，同时也等价于`*(*(a+i)+j)`。

注 3：使 `p` 指向数组 `a` 的第二行(`a[0]`表示第一行)。

注 4：`(*p)[0]`为第二行第一元素。

注 5：`*(p+0))[0]`也为第二行第一元素。

注 6：`p[0][0]`仍为第二行第一元素。

注 7：`(*p)[1]`表示第二行第二元素。

注 8：`(*p+1)[0]`也表示第二行第二元素。

注 9：`*(p+1))[0]`代表第三行第一元素(`p+1` 指向第三行)。

注 10：`p[1][0]`也代表第三行第一元素(`p[1]`指向第三行)。

注 11：使 `p` 指向数组 `a` 的第三行(注意：`p++`使得 `p` 指针一下子下推定义 `p` 时那一个一维数组的元素个数，即下推三个 `int` 型元素，从而指向了数组 `a` 的下一行)。

注 12：`(*p)[2]`、`p[0][2]`、`*(p[0]+2)`、`*( *p+2)`都表示第三行第三元素。

[输出结果]

```

int a[3][3]={ 1,2,3,4,5,6,7,8,9};
int (*p)[3], i=2, j=0;
-----
a[ i ][ j ]=7
*(a[ i ]+j)=7
*(*(a+i)+j)=7
-- After 'p=&a[1];' --
(*p)[0]=4
*(p+0))[0]=4
p[0][0]=4
(*p)[1]=5
(*p+1)[0]=5

```

```
(*p+1)[0]=7
p[1][0]=7
-- After 'p++;' --
(*p)[2]=9
p[0][2]=9
*(p[0]+2)=9
*(*p+2)=9
```

6.15 结构体与数组两种数据结构的结合使用

通过对结构体与数组两种数据结构的结合使用来实现从键盘输入 10 个 `int` 型数，而后按输入的相反顺序输出它们的反序输出问题。

- (1) 使用 `a` 结构体，其分量为 `int` 型数组，使用分量数组来存放欲处理的数据：

```
struct stru1
{
    int dat[10];
}
stru1 a;
```

- (2) 使用 `a` 结构体，其分量为字符型数组(字符串)，使用该分量数组来存放欲处理的数据：

```
struct stru1
{
    char str[80];
}
stru1 a;
```

- (3) 使用结构体数组(其分量为结构体)，而每一数组元素(结构体)可以存放一个 `int` 型数据：

```
struct stru1
{
    int dat;
}
stru1 a[10];
```

[分析提示]

上述给出的三种相互配合使用的数据结构，实质上仍然是通过 `int` 型数组或者字符型数组(字符串)来存放数据的处理方式。当通过字符型数组(字符串)来存放数据时，同样需要从后往前逐一地“拼装”出所需的那十个整数而后再输出它们。

[解答程序一]

```
//使用 a 结构体, 其分量为 int 型数组( 使用分量数组来存放欲处理的数据):
#include <iostream.h>
void main( )
{
    struct stru1
    {
        int dat[10];           //分量为 int 型数组(用来存放欲处理的数据)
    };
    stru1 a;                  //结构体 a
    cout<<"input 10 integers:"<<endl;    //提示用户输入十个整数
    for(int i=0; i<10; i++)
        cin>>a.dat[i];           //输入数据放入结构体 a 的 dat 分量数组中
    cout<<"---- The result ----"<<endl;
    for(i=9; i>=0; i--)
        cout<<a.dat[i]<<" ";       //按反序输出结构体 a 的 dat 分量数组中的各数据
    cout<<endl;
}
```

[输出结果]

```
input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```

[解答程序二]

```
//使用 a 结构体, 其分量为字符型数组( 字符串, 使用该分量数组来存放欲处理的数据):
#include <iostream.h>
#include <stdio.h>
#include <string.h>
void main( )
{
    struct stru1
    {
        char str[80];        //分量为字符型数组(字符串, 用来存放欲处理的数据)
    }
    stru1 a;                  //结构体 a
    cout<<"input 10 integers(a string ended with ENTER):"<<endl;
    //提示用户输入十个整数(为一个以换行结束的字符串)
```

```

gets(a.str);                //放入结构体 a 的 str 分量数组(字符串)中
cout<<"---- The result ----"<<endl;
//从结构体 a 的分量字符数组 str 中, 由后往前逐一“ 拼装” 出所需的 10 个整数并输出
int lastInd=strlen(a.str)-1;
    //lastInd 为当前被“ 拼装” 数的最末一个数字字符位置(下标值)
while (lastInd>=0)          //尚未“ 拼装” 完所有数据时要继续处理
{
    int num=0, weight=1;     //“ 拼装” 起一个整数放入 num
    //weight 为位权(依次取值 1、10、100、...、表示个位、十位、百位、...)
    while(lastInd>=0 && a.str[lastInd]>='0' && a.str[lastInd]<='9')
    {
        //将从 lastInd 往前的那一段数字字符“ 拼装” 为一个整数放入 num 中
        num+=(a.str[lastInd]-'0')*weight;
                                //注意 a.str[lastInd]- '0' 为数字字符对应的数值
        lastInd--;              //准备处理前一字符
        weight*=10;             //前一数字字符的位权增大 10 倍
    }
    //内层 while 结束
    cout<<num<<" ";           //将“ 拼装” 好的 1 个整数 num 显示到屏幕上
    if(lastInd<0) break;        //已经“ 拼装” 完所有数据时跳出循环结束处理
    while(lastInd>=0 && a.str[lastInd]==' ')    //甩掉数据间的空格
        lastInd--;
    //外层 while 结束
}
cout<<endl;
}

```

注意：上述程序没有处理数的符号位，也没有对输入数据的语法格式进行检查。

[输出结果]

```

input 10 integers(a string ended with ENTER):
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1

```

[解答程序三]

//使用结构体数组(其分量为结构体)，而每一数组元素(结构体)可以存放一个 int 型数据

```
#include <iostream.h>
```

```
void main()
```

```
{
```

```
    struct stru1
```

```
    {
```

```

        int dat;                                //结构体的 dat 分量中可以存放一个 int 型数据
    };
    struct a[10];                                //具有十个结构体分量的数组 a
    cout<<"input 10 integers:"<<endl;           //提示用户输入十个整数
    for(int i=0; i<10; i++)
        cin>>a[i].dat;                          //输入数据放入数组 a 的 dat 分量中
    cout<<"---- The result ----"<<endl;
    for(i=9; i>=0; i--)
        cout<<a[i].dat<<" ";                    //按反序输出数组 a 的 dat 分量中的各数据
    cout<<endl;
}

```

[输出结果]

```

input 10 integers:
1 2 3 4 5 6 7 8 9 10
----- The result -----
10 9 8 7 6 5 4 3 2 1

```

6.16 添加函数完成与结构体有关的对复数的某些指定运算

下述与结构体有关的程序框架用于完成五对复数的加法以及乘法运算并输出计算结果。将程序补充完整(添加 addCom、mulCom 以及 outCom 函数的具体实现),并上机验证各添加函数的正确性。

```

struct complex
{
    double real;
    double imag;
};

complex addCom(complex c1, complex c2);
complex mulCom (complex com1, complex com2);
void outCom (complex *com);

void main(void)
{
    complex c1={1,1}, cArr[ 5 ]={ {1,2}, {1,-2}, {2,-1}, {3,5}, {-3,9} };
    complex resA[ 5 ], resM[ 5 ], *pa=resA, *pm=resM;
    for(int i=0; i<5; i++)

```



```

    {
        resA[ i ]=addCom(c1, cArr[ i ]);
        resM[ i ]=mulCom(c1, cArr[ i ]);
    }
    while (pa<=&resA[ 4 ])
        outCom (pa++);
    cout<<"-----"<<endl;
    while (pm<=&resM[ 4 ])
        outCom (pm++);
}

```

[分析提示]

本题自定义了一个结构体类型变量 **complex**，并说明且使用了这种自定义类型的结构体变量、结构体数组(数组分量为结构体)、指向结构体变量的指针。另外，还对结构体进行了如下的一些操作与使用：结构体数据作为函数参数(如，结构体变量作形参、结构体指针作函数参数)；函数的返回值类型为结构体；同类型结构体变量间的相互赋值等。

[解答程序]

```

#include<iostream.h>

struct complex                                //自定义的结构体类型 complex
{
    double real;                               //复数实部
    double imag;                               //复数虚部
};

complex addCom(complex c1, complex c2); //实现复数 c1+c2，返回结构体
//结构体变量作形参(可“单向传入值”)

complex mulCom (complex com1, complex com2);
//实现复数 c1*c2，返回结构体。结构体变量作形参(可“单向传入值”)

void outCom (complex *com);                 //输出一个结构体的有关数据(各分量)
//结构体指针作函数参数(只对结构体数据进行使用而不改变它的值)

void main(void)
{
    complex c1={1,1}, cArr[ 5 ]={ {1,2}, {1,-2}, {2,-1}, {3,5}, {-3,9} };
    //结构体变量 c1，分量为结构体的数组 cArr，均被赋了初值

    complex resA[ 5 ], resM[ 5 ], *pa=resA, *pm=resM;    //pa 与 pm 为指向结构体的指针
    for(int i=0; i<5; i++)

```

```

    {
        resA[ i ]=addCom(c1, cArr[ i ]); //调用 addCom 实现复数加法,
                                      //实参为结构体
                                      //返回值为结构体, 赋给同类型的结构体变量
        resM[ i ]=mulCom(c1, cArr[ i ]); //调用 mulCom 实现复数乘法,
                                      //实参为结构体
    }
    while (pa<=&resA[ 4 ]) //指针比较, 输出 resA 数组中的五个加法结果
        outCom (pa++);
    cout<<"-----"<<endl;
    while (pm<=&resM[ 4 ]) //指针比较, 输出 resM 数组中的五个乘法结果
        outCom (pm++);
}

complex addCom(complex c1, complex c2) //实现复数 c1+c2, 返回结构体
                                      //结构体变量作形参(可“ 单向传入值” )
{
    complex c;
    c.real=c1.real+c2.real;
    c.imag=c1.imag+c2.imag;
    return c;
}

complex mulCom (complex com1, complex com2)
{
    //实现复数 c1*c2, 返回结构体。结构体变量作形参(可“ 单向传入值” )
    complex temp;
    temp.real=(com1.real*com2.real)-(com1.imag*com2.imag);
    temp.imag=(com1.real*com2.imag)+(com1.imag*com2.real);
    return temp;
}

void outCom (complex* com) //输出结构体的一个数据(各分量)
    //结构体指针作函数参数(只对结构体数据进行使用而不改变它的值)
{
    cout<<"r="<<com->real<<" , i="<<com->imag<<endl;
}

```

[输出结果]

```
r=2, i=3
r=2, i=-1
r=3, i=0
r=4, i=6
r=-2, i=10
-----
r=-1, i=3
r=3, i=-1
r=3, i=1
r=-2, i=8
r=-12, i=6
```

6.17 输入十个整数而后反序输出的实现方式六

读下述程序，并上机进行验证，看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意，本程序使用指针与链表，按栈式用法将输入数据存放在动态生成的链表项中。

```
#include <iostream.h>
void main(void)
{
    struct item                //注 1
    {
        int dat;               //注 2
        item* next;           //注 3
    };
    item *head=NULL, *tmp;     //注 4
    cout<<"Input 10 integers:"<<endl;
    for(int i=0; i<10; i++)    //注 5
    {
        int x;
        cin>>x;                //注 6
        tmp=new item;          //注 7
        tmp->dat=x;             //注 8
        tmp->next=head;         //注 9
        head=tmp;              //注 10
    }
    cout<<"---- The result ----"<<endl;
```

```
tmp=head;
while(tmp!=NULL)                //注 11
{
    cout<<tmp->dat<<" ";
    tmp=tmp->next;                //注 12
}
cout<<endl;
}
```

[分析提示]

通过使用如下的含有指针的结构体形式“串联”起一个可以存放数据的动态链表。

```
struct item                      //结构体类型，用于形成链表项
{
    int dat;                     //存放数据
    item* next;                 //指针，“串联”起后项
};
```

而后将输入数据依次放入动态生成的新链表项中，并总将新链表项插入到原链表的头位置处，从而保障越靠后输入的数据越处在链表的前端（栈式用法）。最后再从头到尾输出链表中各项的数据（即为输入的反顺序）。

[解答程序]

具体程序见题目处。

注 1：结构体类型，用于形成链表项。

注 2：数据域。

注 3：指针域。

注 4：指针 head 指向链表首项，初值为 NULL，表示空链表。

注 5：输入十个数据，依次放入动态生成的新链表项中，并总将新链表项插入到原链表的头位置处，从而保障越靠后输入的数据越处在链表的前端（栈式用法）。

注 6：输入数据。

注 7：动态生成新的链表项。

注 8：数据放入链表项中。

注 9：将链表项插入到原链表的头位置处。

注 10：更新头指针 head。

注 11：从头到尾输出链表中各项的数据（即为输入的反顺序）。

注 12：使 tmp 指向下一链表项。

[输出结果]

Input 10 integers:

1 2 3 4 5 6 7 8 9 10

---- The result ----

10 9 8 7 6 5 4 3 2 1

6.18 输入十个整数而后反序输出的实现方式七

读下述程序，并上机进行验证，看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意，本程序使用指针与链表，按队列式用法将输入数据存放在动态生成的链表项中。

```
#include <iostream.h>

void main(void)
{
    struct item                //注 1
    {
        int dat;                //注 2
        item* next;            //注 3
    };
    item *head=NULL, *tail=NULL, *tmp;    //注 4
    cout<<"Input 10 integers:"<<endl;
    for(int i=0; i<10; i++)            //注 5
    {
        int x;
        cin>>x;                        //注 6
        tmp=new item;                  //注 7
        tmp->dat=x;                    //注 8
        if(i==0)                      //注 9
        {
            head=tmp;
            tail=tmp;
        }
        else                            //注 10
        {
            tail->next=tmp;            //注 11
            tail=tmp;                  //注 12
        }
    }
    cout<<"---- The result ----"<<endl;
    for(i=9; i>=0; i--)                //注 13
    {
```

```
        tmp=head;
        for(int j=1; j<=i; j++)           //注 14
            tmp=tmp->next;
        cout<<tmp->dat<<" ";           //注 15
    }
    cout<<endl;
}
```

[分析提示]

通过使用与实现方式六相同的含有指针的结构体形式“串联”起一个可以存放数据的动态链表。而后将输入数据依次放入动态生成的新链表项中，并总将新链表项插入到原链表的尾部，从而保障越靠前输入的数据越处在链表的前端(队列式用法)。最后再设法找到每一个所需的数据并显示出来。

[解答程序]

具体程序见题目处。

注 1：结构体类型，用于形成链表。

注 2：数据域。

注 3：指针域。

注 4：指针 head 指向链表首项、tail 指向链表末项，初值均为 NULL，表示空链表。

注 5：输入 10 个数据，依次放入动态生成的新链表项中，并总将新链表项插入到原链表的尾部，从而保障越靠前输入的数据越处在链表的前端(队列式用法)。

注 6：输入数据。

注 7：动态生成新的链表项。

注 8：数据放入链表项中。

注 9：链表首项的处理，tmp 项既为链表首项、又为链表末项。

注 10：非链表首项的处理。

注 11：将链表项插入到原链表的尾部。

注 12：tail 指向新的链表末项(tmp 项)。

注 13：使循环体执行 10 遍，每遍找到并显示出一个所需数据。

注 14：找出欲显示数据的链表项位置 tmp。

注 15：输出找到的数据。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```

6.19 使用由结构体形成的链表来实现数据排序

按下面的方法设计一个排序程序：使用由结构体形成的链表来存放数据，总保障在把第 i 个数据插入链表以前，链表中的前 $i-1$ 个数据已经是有序的（即是说，每次总是在原有有序链表的适当位置插入新数据，而保障插入后的链表仍有序）。

可定义如下的数据结构以及全局变量和自定义函数，并编出主函数对它们进行使用：

```
struct item                                //使用 item 结构体来形成链表存放数据
{
    int num;                               //数据域
    item* next;                            //指针域
};

item* head=NULL, *tail=NULL;              //全局变量，head 指向链表首项，
//tail 指向链表末项，赋予初值 NULL 意味着起始时链表为空。
void insertItem(item*& head, item*& tail, int number);
/*
将数据 number 插入到以 head 为首以 tail 为末的当前的有序链表的适当位置，而保障插入后
的链表仍有序。注意，将 head 与 tail 设置成引用型指针，为的是在被调函数中可以改变主
调函数处对应的实参指针的值（比如实参总传递全局变量 head 与 tail）。
*/
```

[分析提示]

(1) insertItem 函数中，首先生成一个链表表项存放欲插入的新数据 number，而后将该表项插入到当前已有序的、以 head 为首以 tail 为末的链表的适当位置。

(2) main 函数中，输入若干个数据，并通过调用 insertItem 函数依次将它们插入到同一个链表中（并已排好了序），最后再将有序链表中的 number 数据逐一输出来。

[解答程序]

```
#include <iostream.h>

struct item                                //使用 item 结构体来形成链表存放数据
{
    int num;                               //数据域
    item* next;                            //指针域
};

item* head=NULL, *tail=NULL;              //全局变量
//head 指向链表首项，tail 指向链表末项。
```

```
void insertItem(item*& head, item*& tail, int number)
{
    //将数据 number 插入到以 head 为首以 tail 为末的当前的有序链表的
    //适当位置，而保障插入后的链表仍有序。

    item* temp;
    temp=new item;           //为新数据生成一个链表表项
    temp->num=number;         //num 域放入新数据 number
    temp->next=NULL;         //next 域置为 NULL
    if (head == NULL)        //若当前链表为空
    {
        head=temp;
        tail = head;
        return;
    }
    if(temp->num <= head->num)  //插入链首
    {
        temp->next=head;
        head=temp;
    }
    else
        if(temp->num >= tail->num) //插入链末
        {
            tail->next=temp;
            tail=temp;
        }
        else                    //插入链中适当位置
        {
            item* insp=head;
            while(!(temp->num > insp->num && temp->num <= insp->next->num))
                insp=insp->next;    //找到插入位置 insp
            temp->next=insp->next;
            insp->next=temp;
        }
    }

void main( )
{
    int n, number;
    do
```



```

{
    cout<<"Input a positive number n (n>0):";    //提示输入
    cin>>n;                                       //输入 n
} while (n<=0);                                  //n 必须为正整数
cout<<"input "<<n<<" INTEGERS:"<<endl;         //提示输入 n 个数据
for(int i=1; i<=n; i++)                          //共处理 n 个数据
{
    cin>>number;                                //输入数据 number
    insertItem(head, tail, number);
    //将数据 number 插入到以 head 为首以 tail 为末的当前的
    //有序链表的适当位置，而保障插入后的链表仍有序
}
cout<<"The sorted results are:"<<endl;
item* temp=head;
while(temp!=NULL)    //输出以 head 为首的有序链表中的所有数据
{
    cout<<temp->num<<" ";
    temp=temp->next;
}
cout<<endl;
}

```

[输出结果]

```

Input a positive number n (n>0):8
input 8 INTEGERS:
4 -2 33 28 9 0 4 12
The sorted results are:
-2 0 4 4 9 12 28 33

```

6.20 将某个有序链表合并(插入)到另一个有序链表之中

利用上一题所编制的自定义函数 `insertItem`，来实现如下的指定工作。

- (1) 从键盘输入 n_1 个数据，形成一个有序的链表 1 (n_1 大小也由用户通过键盘输入)；
- (2) 从键盘输入 n_2 个数据，形成一个有序的链表 2 (n_2 大小也由用户通过键盘输入)；
- (3) 将链表 2 中的所有数据合并到(插入到)链表 1 中(使链表 1 仍有序)；
- (4) 将链表 1 中的所有数据(已有序)显示到屏幕上。

例如，所编程序执行后所显示的结果可以呈现如下形式：

```
Input a positive number n1 (n1>0):2
```

input 2 INTEGERS that will form LINK1:

4 2

The sorted results of LINK1 are:

2 4

Input a positive number n2 (n2>0):3

input 3 INTEGERS that will form LINK2:

5 3 1

The sorted results of LINK2 are:

1 3 5

Merge LINK2 to LINK1, and the sorted results of LINK1 are:

1 2 3 4 5

[分析提示]

- (1) 第一项工作只需使用类似于如下的一个循环来完成:

```
for(int i=1; i<=n1; i++)          //共处理 n1 个数据
{
    cin>>number;                  //输入数据 number
    insertItem(head1, tail1, number);
    //将数据 number 插入到以 head1 为首以 tail1 为末的
    //当前的有序链表 1 的适当位置。而保障插入后的链表 1 仍有序
}
```

- (2) 第二、第三项工作的处理类似于第一项工作, 而第四项工作只需对链表进行一次从头到尾的遍历。

[解答程序]

```
#include <iostream.h>

struct item                      //使用 item 结构体来形成链表存放数据
{
    int num;                     //数据域
    item* next;                 //指针域
};

item* head1=NULL, *tail1=NULL;  //head1 指向链表首项, tail1 指向链表末项
item* head2=NULL, *tail2=NULL;  //head2 指向链表首项, tail2 指向链表末项

void insertItem(item*& head, item*& tail, int number)
{
    //将数据 number 插入到以 head 为首以 tail 为末的当前的有序链表的
    //适当位置, 从而保障插入后的链表仍有序。
```

```

    item* temp;
    temp=new item;           //为新数据生成一个链表表项
    temp->num=number;         //num 域放入新数据 number
    temp->next=NULL;         //next 域置为 NULL
    if (head == NULL)       //若当前链表为空
    {
        head=temp;
        tail = head;
        return;
    }
    if(temp->num <= head->num) //插入链首
    {
        temp->next=head;
        head=temp;
    }
    else
        if(temp->num >= tail->num) //插入链末
        {
            tail->next=temp;
            tail=temp;
        }
        else //插入链中适当位置
        {
            item* insp=head;
            while(!(temp->num > insp->num && temp->num <= insp->next->num))
                insp=insp->next; //找到插入位置 insp
            temp->next=insp->next;
            insp->next=temp;
        }
    }

void main( )
{
    //(1) 从键盘输入 n1 个数据，形成一个有序的链表 1( n1 大小也由用户通过输入指定)
    int n1, n2, number;
    do
    {
        cout<<"Input a positive number n1 (n1>0):"; //提示输入
        cin>>n1; //输入 n1
    }
}

```

```

} while (n1<=0);                                //n1 必须为正整数
cout<<"input "<<n1<<" INTEGERS that will form LINK1:"<<endl;
                                                //提示输入 n1 个数据
for(int i=1; i<=n1; i++)                        //共处理 n1 个数据
{
    cin>>number;                                //输入数据 number
    insertItem(head1, tail1, number);
        //将数据 number 插入到以 head1 为首以 tail1 为末的
        //有序链表 1 的适当位置，从而保障插入后的链表 1 仍有序
}
cout<<"The sorted results of LINK1 are:"<<endl;
item* temp=head1;
while(temp!=NULL)                                //输出以 head1 为首的有序链表 1 中的所有数据
{
    cout<<temp->num<<" ";
    temp=temp->next;
}
cout<<endl;

//(2) 从键盘输入 n2 个数据，形成一个有序的链表 2( n2 大小也由用户通过输入指定)
do
{
    cout<<"Input a positive number n2 (n2>0):"; //提示输入
    cin>>n2;                                    //输入 n2
} while (n2<=0);                                //n2 必须为正整数
cout<<"input "<<n2<<" INTEGERS that will form LINK2:"<<endl;
                                                //提示输入 n2 个数据
for(i=1; i<=n2; i++)                            //共处理 n2 个数据
{
    cin>>number;                                //输入数据 number
    insertItem(head2, tail2, number);
        //将数据 number 插入到以 head2 为首以 tail2 为末的
        //有序链表 2 的适当位置。而保障插入后的链表 2 仍有序
}
cout<<"The sorted results of LINK2 are:"<<endl;
temp=head2;
while(temp!=NULL)                                //输出以 head2 为首的有序链表 2 中的所有数据
{
    cout<<temp->num<<" ";

```

```

        temp=temp->next;
    }
    cout<<endl;

    //(3) 将链表 2 中的所有数据合并到(插入到)链表 1 中使链表 1 仍有序
    cout<<"Merge LINK2 to LINK1, and the sorted results of LINK1 are:"<<endl;
    temp=head2;                //temp 指向链表 2 的表头
    while(temp!=NULL)          //处理以 head2 为首的
    {
        //有序链表 2 中的所有数据, 合并到 (插入到) 链表 1 中
        number=temp->num;        //从链表 2 中提取数据
        insertItem(head1, tail1, number);
        //将数据 number 插入到以 head1 为首以 tail1 为末的
        //有序链表 1 的适当位置, 从而保障插入后的链表 1 仍有序
        temp=temp->next;        //使 temp 指向链表 2 的下一项
    }

    //(4) 将有序链表 1 中的所有数据显示到屏幕上。
    temp=head1;
    while(temp!=NULL)          //输出以 head1 为首的有序链表 1 中的所有数据
    {
        cout<<temp->num<<" ";
        temp=temp->next;
    }
    cout<<endl;
}

```

[输出结果]

与题目处相同, 略。

6.21 合并两个有序链表形成一个新的有序结果链表

通过结构体、指针以及运算符 new 来生成并合并成所需链表, 并实现如下指定的处理工作。

- (1) 使用数组 a1 中的 n1 个有序数据, 形成一个有序的链表 1 —— LINK1;
- (2) 使用数组 a2 中的 n2 个有序数据, 形成一个有序的链表 2 —— LINK2;
- (3) 将链表 1 与链表 2 中的所有数据取出合并到一个新生成的链表 3 —— LINK3 之中(但必须使链表 3 仍有序, 而且并不破坏原来的链表 1 与链表 2);

(4) 将有序链表 3 中的所有数据显示到屏幕上。

现假设数组 a1 与数组 a2 中的数据是通过如下说明来确定的：

```
const int n1=6, n2=8;
```

```
int a1[n1]={-5, 0, 18, 66, 66, 100}; //数组 a1 已经有序(升序排列)
```

```
int a2[n2]={3, 4, 10, 18, 50, 59, 62, 66}; //数组 a2 已经有序(升序排列)
```

那么，本练习处理后的屏幕显示结果可为：

The sorted results of LINK1 are:

—5 0 18 66 66 100

The sorted results of LINK2 are:

3 4 10 18 50 59 62 66

The sorted results of LINK3 are:

—5 0 3 4 10 18 18 50 59 62 66 66 66 100

[分析提示]

(1) 前面已经做过类似于第一、第二项的工作，只要逐次生成新的链表项并插入到原链表的末尾处即可。第四项工作也是要对链表进行一次从头到尾的遍历。

(2) 第三项工作要求不破坏原链表 1 与链表 2, 可从它们取数据来生成一个有序的新链表 3。这是一个二路归并问题。当链表 1 与链表 2 都没结束时, 每次从两个链表的当前位置找一个小者, 并取来其数据生成新表项加入到链表 3 的尾部; 当二链表中至少有一个已结束时, 将尚未结束的那一链表的剩余数据取来生成新表项依次加入到链表 3 的尾部。注意, 与范例 4.11 的“合并两个有序数组”有类似之处。

[解答程序]

[illegible]

//(1) 使用 a1 数组中的 n1 个有序数据, 形成一个有序的链表 1 —— LINK1

```
item* temp;
temp=new item;           //动态生成链表的一个新表项
temp->num=a1[0];          //num 域放入 a1 数组的首元素(数据)
temp->next=NULL;          //next 域置为 NULL
head1=temp;
tail1 = head1;           //此时已形成有序链表 1 的首项
for(int i=1; i<n1; i++)    //形成有序链表 1 的第 2 至第 n1 项
{
    temp=new item;        //动态生成链表的一个新表项
    temp->num=a1[i];       //num 域放入数组 a1 的元素
    temp->next=NULL;       //next 域置为 NULL
    tail1->next=temp;      //新表项作为链表 1 的末项
    tail1 = temp;         //改变链表 1 的末项指针位置
}
cout<<"The sorted results of LINK1 are:"<<endl;
temp=head1;
while(temp!=NULL)         //输出以 head1 为首的有序链表 1 中的所有数据
{
    cout<<temp->num<<" ";
    temp=temp->next;
}
cout<<endl;
```

//(2) 使用数组 a2 中的 n2 个有序数据, 形成一个有序的链表 2 —— LINK2

```
temp=new item;           //动态生成链表的一个新表项
temp->num=a2[0];          //num 域放入数组 a2 的首元素
temp->next=NULL;          //next 域置为 NULL
head2=temp;
tail2 = head2;           //此时已形成有序链表 2 的首项
for(i=1; i<n2; i++)      //形成有序链表 2 的第 2 至第 n2 项
{
    temp=new item;        //动态生成链表的一个新表项
    temp->num=a2[i];       //num 域放入数组 a2 的元素
    temp->next=NULL;       //next 域置为 NULL
    tail2->next=temp;      //新表项作为链表 2 的末项
    tail2 = temp;         //改变链表 2 的末项指针位置
}
cout<<"The sorted results of LINK2 are:"<<endl;
```

```

temp=head2;
while(temp!=NULL)      //输出以 head2 为首的有序链表 2 中的所有数据
{
    cout<<temp->num<<" ";
    temp=temp->next;
}
cout<<endl;

//(3) 将链表 1 与链表 2 中的所有数据取出合并到一个新生成的链表
//3——LINK3 之中
//(但必须使链表 3 仍有序, 而且并不破坏原来的链表 1 与链表 2);
item* head3=NULL, *tail3=NULL;
        //head3 指向链表 3 首项, tail3 指向链表 3 末项
item *tmp1=head1, *tmp2=head2, *tmp3;
        //tmp1 为取链表 1 数据的当前位置,
        //tmp2 为取链表 2 数据的当前位置
        //而 tmp3 指向为链表 3 新生成的数据项
while(tmp1!=NULL && tmp2!=NULL)
        //当链表 1 与链表 2 都没结束时(二者都没被取光时)
    if( tmp1->num <= tmp2->num )
    {
        //当链表 1 当前位置的数据小于链表 2 当前位置数据时, 链表 1 数据=>链表 3
        tmp3=new item;      //动态生成链表的一个新表项
        tmp3->num=tmp1->num;
                                //取出链表 1 当前位置的数据作为 num 域值
        tmp3->next=NULL;      //next 域置为 NULL
        if(head3==NULL)      //链表 3 为空时
        {
            head3=tmp3;      //新表项即为链表 3 的首项又为末项
            tail3=tmp3;
        }
        else                  //链表 3 非空时, 将新表项加到该链的尾部
        {
            tail3->next = tmp3;
            tail3 = tmp3;      //改变链表 3 的末项指针位置
        }
        tmp1=tmp1->next;      //下推取链表 1 数据的当前位置
    }
    else

```



```

{
//当链表 1 当前位置的数据大于等于链表 2 当前位置
//数据时，链表 2 数据=>链表 3
    tmp3=new item;           //动态生成链表的一个新表项
    tmp3->num=tmp2->num; //取出链表 2 当前位置的数据作为
                           //num 域值
    tmp3->next=NULL;         //next 域置为 NULL
    if(head3==NULL)         //链表 3 为空时
    {
        head3=tmp3;         //新表项即为链表 3 的首项又为末项
        tail3=tmp3;
    }
    else                     //链表 3 非空时，将新表项加到该链的尾部
    {
        tail3->next = tmp3;
        tail3 = tmp3;       //改变链表 3 的末项指针位置
    }
    tmp2=tmp2->next ;        //下推取链表 2 数据的当前位置
}
while(tmp1!=NULL)           //当链表 1 尚未结束时 (此时链表 2 已结束),
                             //链表 1 剩余数据=>链表 3
{
    tmp3=new item;           //动态生成链表的一个新表项
    tmp3->num=tmp1->num;      //取出链表 1 当前位置的数据作为 num 域值
    tmp3->next=NULL;
    tail3->next = tmp3;       //将新表项加到链表 3 的尾部
    tail3 = tmp3;
    tmp1=tmp1->next ;        //下推取链表 1 数据的当前位置
}
while(tmp2!=NULL)           //当链表 2 尚未结束时(此时链表 1 已结束),
                             //链表 2 剩余数据=>链表 3
{
    tmp3=new item;           //动态生成链表的一个新表项
    tmp3->num=tmp2->num;      //取出链表 2 当前位置的数据作为 num 域值
    tmp3->next=NULL;
    tail3->next = tmp3;       //将新表项加到链表 3 的尾部
    tail3 = tmp3;
    tmp2=tmp2->next ;        //下推取链表 2 数据的当前位置
}
}

```

```

// (4) 将链表 3 中的所有数据(已有序)显示到屏幕上。
cout<<"The sorted results of LINK3 are:"<<endl;
temp=head3;
while(temp!=NULL)    //输出以 head3 为首的有序链表 3 中的所有数据
{
    cout<<temp->num<<" ";
    temp=temp->next;
}
cout<<endl;
}

```

[输出结果]

与题目处相同，略。

6.22 利用链表存储字符串并实现相关的字符串处理

自定义结构体 node 如下：

```

struct node
{
    char c;           //存放一个字符
    node* next;       //串接后继项
};

```

试利用这种结构存储字符串，并编制具有如下功能的各函数，对此种结构中的字符串进行处理。

(1) void putString(char* a, node* p1);

将 a 字符串(数组)中的所有字符串接到以 p1 指向其首的链串的后面；

(2) void printString(node* p1);

输出以 p1 指向其首的链串中的所有字符(一个字符串)；

(3) void getString(char *b, node* p1);

将以 p1 指向其首的链串中的所有字符(一个字符串)取出放入字符数组 b 中；

(4) char SearchMax(node* p1, int& max);

找出以 p1 指向其首的链串中出现次数最多的某个字母(不区分大小写)；并由 SearchMax 函数返回该字母，另外还通过引用参数 max 返回该字母的出现次数。

并编写如下的示例性主函数，对上述各函数进行调用以验证它们的正确性。

```

void main()           //主函数
{
    char a[30]="abcd1AA2345aabc", b[80];

```

```

node* p1;
p1=new node;           //生成一个由 p1 指向其链首的空链串
p1->c='\0';             //空链串首项的字符为“ \0”
p1->next=NULL;         //空链串没有后继字符，所以其 next 域值为 NULL
putString(a, p1);      //将 a 字符串(数组)中的所有字符
                        //串接到以 p1 指向其首的“ 链串” 的后面
                        //但不改变 p1 的值(p1 仍指向整个“ 链串” 的首部)
cout<<"printString(p1)'=>";    //屏幕提示
printString(p1);        //输出以 p1 指向其首的链串中的所有字符(一个字符串)
cout<<"getString(b, p1)'=>";    //屏幕提示
getString(b, p1);      //将以 p1 指向其首的链串中的
                        //所有字符(一个字符串)取出放入字符数组 b 中
cout<<"b="<<b<<endl;          //输出 b(字符串)
int count;
char Ch_CountMax=SearchMax(p1, count);
                        //找出以 p1 指向其首的链串中出现次数最多的某个字母
                        //(不区分大小写)，并由 SearchMax 函数返回该字母，
                        //另外还通过 count 返回该字母的出现次数
cout<<"Ch_CountMax="<<Ch_CountMax<<"", count="<<count<<endl;
                        //输出出现最多的那一字母以及该字母的出现次数
}

```

[分析提示]

(1) 通过自定义的 `node` 结构体类型，可以串接成一个链表。通过使用这样的链表来存放一个字符串(串中的每一个字符占用一个链表项，字符本身存放在链表项结构体的 `c` 分量中，而使用链表项结构体的 `next` 分量去串接后继项)。

(2) 要对存储在上述这种链表结构中的字符串进行处理，总要涉及到对链表的遍历过程(因为只有从链首遍历链尾才能找出那一个字符串的所有各字符)，而后再对找出的各字符进行规定的处理。

[解答程序]

```

#include <iostream.h>
struct node           //自定义结构体 node
{
    char c;           //存放一个字符
    node* next;       //串接后继项
};

void putString(char* a, node* p1)

```

```
{
    //将 a 字符串(数组)中的所有字符串接到以 p1 指向其首的链串的后面
    node* last=p1;
    while(*a)                //处理 a 字符串(数组)中的所有字符(直到“ \0” )
    {
        last->c=*a;           //将字符放于当前项中
        last->next=new node;  //新生成一个项作为结束项
        last=last->next;
        last->c='\0';
        last->next=NULL;
        a++;                 //处理 a 字符串(数组)的下一字符
    }
}

void printString(node* p1)
{
    //输出以 p1 指向其首的链串中的所有字符(一个字符串)
    node* last=p1;
    while(last)              //对链串进行遍历
    {
        cout<<last->c;       //输出当前项中的字符
        last=last->next;     //准备访问下一项
    }
    cout<<endl;
}

void getString(char *b, node* p1)
{
    //将以 p1 指向其首的链串中的所有字符 (一个字符串) 取出放入字符数组 b 中
    node* last=p1;
    while(last)              //对链串进行遍历
    {
        *b++=last->c;        //从链串取字符放入字符数组 b 中
        last=last->next;     //准备访问下一项
    }
}

char SearchMax(node* p1, int& max)
{

```

```

//找出以 p1 指向其首的链串中出现次数最多的某个字母(不区分大小写)
//并由 SearchMax 函数返回该字母，另外还通过引用参数 max 返回该字母
//的出现次数
node* last=p1;
int letter[26];           //记录 26 个字母各自的出现次数
    //用 letter[0]记录“ a” 的出现次数，letter[1]记录“ b” 的出现次数，...
char cc;
for(int i=0; i<26; i++)
    letter[i]=0;
while(last)               //对链串进行遍历
{
    cc=last->c;
    if(cc>='A' && cc<='Z')    //检查大写字母
        letter[cc-'A']++;
    if(cc>='a' && cc<='z')    //检查小写字母
        letter[cc-'a']++;
    last=last->next;
}
max=0;                    //注意 max 为引用参数
int ind=0;
for(i=0; i<26; i++)       //找出出现次数最多的那个字母
    if(letter[i]>max)
    {
        max=letter[i];
        //对引用参数 max 值的改变将返回到主调函数对应实参中
        //(双向传值)
        ind=i;
    }
return ind+'a';           //返回出现次数最多的那个字母
}

void main( )               //主函数
{
    ...                    //详见题目处,略。
}

```

[输出结果]

'printString(p1)'=>abcd1AA2345aabc

'getString(b, p1)'=>b=abcd1AA2345aabc

```
Ch_CountMax='a', count=5
```

6.23 对函数指针的使用

读下面的程序(其中使用了函数指针), 写出执行结果。

```
#include <iostream.h>
#include <math.h>
double func1(double x);
double func2(double y);
double func3(double z);
void processing(double x, double (*fpt)(double));           //注 1

void main( )
{
    double x,res;
    double (*fp)(double);                                   //注 2
    cout<<"Input a real number:";
    cin>>x;
    fp=func1;                                               //注 3
    res=(*fp)(x);                                           //注 4
    cout<<"res=(*fp)(x)="<<res<<endl;
    fp=func2;                                               //注 5
    res=(*fp)(x);                                           //注 6
    cout<<"res=(*fp)(x)="<<res<<endl;
    processing(x, func3);                                    //注 7
    fp=func3;                                               //注 8
    processing(x, fp);                                       //注 9
}

double func1(double x)                                     //注 10
{
    cout<<" ---- Call func1:  1/x="<<1/x<<endl;
    return (1/x);
}

double func2(double x)                                     //注 11
{
    cout<<" ---- Call func2:  x*x="<<x*x<<endl;
    return (x*x);
}
```

```
    }

    double func3(double x)                                //注 12
    {
        cout<<" ---- Call func3:  sqrt(x)="<<sqrt(x)<<endl;
        return sqrt(x);
    }

    void processing(double x, double (*fpt)(double))      //注 13
    {
        cout<<" ---- Call processing"<<endl;
        double result=(*fpt)(x);                        //注 14
        cout<<"  result=(*fpt)(x)="<<result<<endl;
    }
```

[分析提示]

(1) 可通过“ `double (*fp)(double);`”形式的说明来指出 `fp` 为函数指针，该指针所指向的那一函数具有一个 `double` 型参数，且函数的返回值类型也为 `double`。可知，若 `fp` 为函数指针的话，则 `*fp` 即为指针所指向的那一函数，它等同于一个具体的函数名。

(2) 函数的形参可以为函数指针，在调用这样的函数时，形参函数指针所对应的实参必须是一个同类型的具体函数指针（譬如一个具体的函数名，注意函数名与数组名一样，是一个常量指针，代表函数的入口首地址）。

[解答程序]

具体程序见题目处。

注 1：函数参数为函数指针 `fpt`。

注 2：函数指针 `fp`。

注 3：函数指针指向 `func1`。

注 4：函数指针调用（将调用 `func1`）。

注 5：函数指针指向 `func2`。

注 6：函数指针调用（将调用 `func2`）。

注 7：注意第二个实参为函数名（常量型函数指针）。

注 8：函数指针指向 `func3`。

注 9：第二个实参为函数指针 `fp`。

注 10：计算 `1/x` 并返回。

注 11：计算 `x` 平方并返回。

注 12：计算 `x` 的平方根并返回。

注 13：函数指针 `fpt` 作为函数参数。

注 14：函数指针调用（将调用实参指向的具体函数）。

[输出结果]

```
Input a real number:5.5
---- Call func1:  1/x=0.181818
res=(*fp)(x)=0.181818
---- Call func2:   x*x=30.25
res=(*fp)(x)=30.25
---- Call processing
---- Call func3:  sqrt(x)=2.34521
result=(*fpt)(x)=2.34521
---- Call processing
---- Call func3:  sqrt(x)=2.34521
result=(*fpt)(x)=2.34521
```

6.24 使用返回指针值的函数

编写如下返回指针值的函数 `search`，由它寻找并返回串 `str` 中第一个 `ch` 字符出现的位置(地址值，即指向字符的指针)，若串 `str` 中不出现 `ch` 字符的话，则返回 `NULL`。

```
char * search(char ch, char * str);
```

`search` 为具有两个参数的函数，其返回值类型为 `char*`，即指针类型。

而后编写主函数，提示用户输入任意一个字符串和一个字符 `c`，通过调用 `search` 寻找输入串中第一个 `c` 字符出现的位置(若有的话)，并输出从 `c` 字符开始的子串以及 `c` 字符在输入串中的下标号；若输入串中不出现 `c` 字符的话，输出 “No match found”。

[分析提示]

`search` 函数中，从头至尾逐个地查看各 `str[i]` 是否等于 `ch`，若相等的话，要返回那一 `str[i]` 字符的地址(即 `&str[i]`)。

[解答程序]

```
#include <iostream.h>
#include <stdio.h>

char * search(char ch, char * str);    //search 函数原型，返回值类型为 char *
//search 函数返回串 str 中第一个 ch 字符出现的位置(地址值，即指向字符的指针)

void main(void)
{
    char s[80], *p, c;
    cout<<"Input a string:"<<endl;    //提示用户输入一个字符串
    gets(s);                            //输入串放 s(数组)中
```



```

    cout<<"Input a character:";           //提示用户输入一个字符
    cin>>c;                               //输入字符放 c 中
    p=search(c, s);
        //调用 search, 返回串 s 中第一个字符 c 出现的位置(地址值, 即指向字符的指针)
    if(p)                                  //s 中含有字符 c 时, 返回的结果指针 p 值为非 0
    {
        cout<<"Sub_string ==>"<<p<<endl;    //输出从字符 c 开始的子串
        cout<<"Index ==>"<<p-s<<endl;        //输出字符 c 在 s 串中的下标号
    }
    else                                  //s 中不含有 c 字符
        cout<<"No match found"<<endl;

}

char * search(char ch, char * str)
{
    //寻找 str 串中第一个 ch 字符出现的位置(地址值, 即指向字符的指针)并返回
    int i=0;
    while(ch!=str[i] && str[i]!='\0')        //从 str 串中找第一个 ch 字符出现的位置 i
        i++;
    if(ch==str[i])
        return (&str[i]);                  //若找到, 返回该 ch 字符在 str 串的地址
    else
        return(NULL);                      //没找到时, 返回 NULL
}

```

[输出结果]

```

Input a string:
Hello world!
Input a character:o
Sub_string ==>o world!
Index ==>4

```

6.25 求取源串中首次出现某子串的头字符位置

编写具有如下原型的函数 FindPlace:

```
char * FindPlace(char * sourceStr, char * subStr);
```

它具有两个指针参数 sourceStr 与 subStr(其对应实参可为两个字符型数组即字符串), 其返回值类型为 char*, 即也为指针类型。该函数的功能为: 返回源串 sourceStr 中

第一次出现 subStr 子字符串的头字符位置(地址值,即指向字符的指针)。如果源串 sourceStr 的长度小于 subStr 子字符串的长度,或者源串 sourceStr 中根本就不出现(不包含) subStr 子字符串的话,返回一个空指针值 NULL。

并编写主函数,任意输入两个字符串,将它们用作实参来调用 FindPlace,以验证其函数功能的正确性。

[分析提示]

FindPlace 函数中,首先把 subStr 子串的头位置 j0=0 (j0 表示 subStr 字符数组的下标)与 sourceStr 的头位置 j1=0 (j1 表示 sourceStr 字符数组的下标)对齐,而后逐字符地对两个串进行比较,若一直比到了子串的结束各字符均相同时,则意味着在源串中找到了子串,此时返回子串在源串中的头字符位置(地址值);若逐字符进行比较时发现不相同,则要进一步将 subStr 子串的头位置 j0=0 与 sourceStr 的新头位置 j1=1 对齐,而后又一次逐字符地对两个串进行比较(相当于子串整体右平移一个字符后再一次比较),如此等等。

[解答程序]

```
#include <iostream.h>
#include <string.h>
char * FindPlace(char * sourceStr, char * subStr);
//FindPlace 函数返回串 sourceStr 中第一个 subStr 子字符串出现的位置
//(地址值,即指向字符的指针)

void main(void)
{
    char s0[81], s1[81], *p;
    cout<<"Input a source_string:"<<endl;    //提示用户输入一个源字符串
    cin.getline(s0, 80);    //遇换行才结束的输入方式,输入串放 s0(数组)中
    cout<<"Input a sub_string:"<<endl;    //提示用户输入一个子字符串
    cin.getline(s1, 80);    //遇换行才结束的输入方式,输入串放 s1(数组)中
    p=FindPlace(s0, s1);    //以 s0, s1 为实参,调用自定义函数 FindPlace
    //将返回串 s0 中第一次出现 s1 子字符串的头字符位置(地址值,
    //即指向字符的指针)
    if(p)    //s0 中含有 s1 子串时,返回的结果 p 为非 0
        cout<<"Sub_str from first '"<<s1<<"'=>"<<p<<endl;
        //输出以子串开始的后半部分
    else    //s0 中不含有 s1 子串时
        cout<<"No match found"<<endl;
}
```

```

char * FindPlace(char * sourceStr, char * subStr)
{
    //找出源串 sourceStr 中第一次出现 subStr 子字符串的头字符位置
    //(地址值, 即指向字符的指针)并返回。如果源串 sourceStr 的长度
    //小于 subStr 子字符串的长度, 或者源串 sourceStr 中根本就不出现
    //subStr 子字符串的话, 返回一个空指针值 NULL
    int i=0, sourceLen, subLen;
    sourceLen=strlen(sourceStr);           //源串长度
    subLen=strlen(subStr);                  //子字符串长度
    int count=sourceLen-subLen;
    if (count<0)
    {
        //如果源串 sourceStr 的长度小于 subStr 子字符串的长度(肯定找不到)
        cout<<"ERROR! -- sourceLen<subLen!"<<endl;
        return(NULL);                      //返回 NULL
    }
    for (i=0; i<=count; i++)
    {
        int j0=0, j1=i;
        //把 subStr 子串的头位置 j0=0 与 sourceStr 的 j1 位置
        //(初始时为 i) 对齐, 逐字符往下比较
        while (subStr[j0] && subStr[j0]==sourceStr[j1])
        {
            //若 subStr 未结束, 且当前二字符相同, 下推一位置, 去比下一字符
            j0++;      //注意 j0 的值恰为二串已经具有了多少相同字符
            j1++;
        }
        if (j0==subLen) //j0 的值(二串已经具有的相同字符个数)恰为子串长度时
            return (&sourceStr[j1-subLen]);
            //找到了子串, 返回头字符位置(地址值)
        }
        return(NULL);      //没找到时, 返 NULL
    }
}

```

[输出结果]

Input a source_string:

abcde 12345

Input a sub_string:

e 1

Sub_str from first 'e' =>e 12345

再一次执行:

Input a source_string:

123a 123 bc abc 12

Input a sub_string:

bc

Sub_str from first 'bc' =>bc abc 12

6.26 通过带参数的 main 函数实现指定的四则运算

编写如下形式的带参数的 main 函数: `void main(int argc, char* argv[ ]){...}`

它接收如下格式的三个命令行参数: <数字串> <运算符> <数字串>。而后完成由<运算符>指定的四则运算(+、-、*、/), 并输出运算结果。

例如: 假设设置了下述以空格分割的三个命令行参数: 12、+、34, 程序输出结果应为: 12+34=46。

注意: argv 数组中存放的是字符串(如: "12"、"+"、"34"), 若要将字符串的“ 12 ”转化为 int 型的“ 12 ”, 可通过<stdlib.h>头文件中包含的“ atoi ”函数, 如可通过使用“ `int i=atoi(argv[1]);` ”来完成这种转化(当然也可通过用户程序将连续数字串“ 拼装 ”成所需的整数)。

[分析提示]

(1) 在 VC6.0 集成开发环境下, 运行这种 main 带参数的程序之前, 要通过如下步骤来设置命令行参数(程序参数): Project => Settings... => Debug =>在 Program arguments: 框内, 输入以空格分割的各参数, 如: “ 12 + 34 ”(该框空时, 表示没有命令行参数) => OK。

(2) 若通过用户程序将连续数字串“ 拼装 ”成所需整数的话, 要将程序中的“ `i=atoi(argv[1]);` ”替换为如下的程序段:

```
int i=0, ind=0;
while (argv[1][ind]!='\0')
{
    i=i*10+(argv[1][ind]-'0');
    ind++;
}
```

同理, 对程序中的“ `j=atoi(argv[3]);` ”也要做相同的处理。

[解答程序]

// 使用 atoi 函数(将连续数字字符串转化为 int 型数据)。

```
#include <iostream.h>
#include <stdlib.h>          //使用 atoi 函数(将连续数字字符串转化为 int 型数据)
void main(int argc, char* argv[ ])
{
    if(argc!=4)              //提供的参数个数不对时
        cout<<"ERROR! You must supply 3 argements!"<<endl;
    else
    {
        int i,j;
        char ch;
        i=atoi(argv[1]);
        //使用 atoi 函数将 argv[1]中的连续数字字符串转化为 int 型数据并赋值给 i
        ch=argv[2][0];       //将 argv[2]中的第一个字符(运算符)赋值给 ch
        j=atoi(argv[3]);
        //使用 atoi 函数将 argv[3]中的连续数字字符串转化为 int 型数据并赋值给 j
        switch (ch)          //根据 ch 来进行分支处理
        {
            case '+':
            {
                cout<<i<<"+"<<j<<"="<<i+j<<endl;
                break;
            }
            case '-':
            {
                cout<<i<<"-"<<j<<"="<<i-j<<endl;
                break;
            }
            case '*':
            {
                cout<<i<<"*"<<j<<"="<<i*j<<endl;
                break;
            }
            case '/':
            {
                cout<<i<<"/"<<j<<"="<<i/j<<endl;
                break;
            }
        }                    //switch 句结束
    }                        //if 句结束
}
```

```
}
```

[输出结果]

假设设置了下述以空格分割的三个参数：12 、+、23：

12+34=46

再一次执行，假设设置了下述以空格分割的三个参数：17、/、3：

17/3=5

6.27 引用的建立及简单使用

读下述程序，其中涉及引用的建立及简单使用，请给出程序执行后的输出结果。

```
#include <iostream.h>

void main( )
{
    int i;
    int &ri=i;                //注 1
    int *pi=&i;                //注 2
    cout<<"&i ="<<&i<<endl;    //注 3
    cout<<"&ri="<<&ri<<endl;    //注 4
    cout<<"&pi="<<&pi<<endl;    //注 5
    i=1011;                    //注 6
    cout<<"i="<<i<<endl, *pi="<<*pi<<endl, ri="<<ri<<endl;
    *pi=2022;                  //注 7
    cout<<"i="<<i<<endl, *pi="<<*pi<<endl, ri="<<ri<<endl;
    ri=3033;                    //注 8
    cout<<"i="<<i<<endl, *pi="<<*pi<<endl, ri="<<ri<<endl;
}
```

[分析提示]

引用是某个变量或对象的别名。建立引用时，必须用某个变量名或对象名来对它进行初始化，从而将该引用绑定在那一个变量或对象上。即是说，建立引用并不重新为其分配内存空间，引用只是另一变量或对象的别名，任何对引用的使用与改变都是对该引用所绑定的那一变量或对象的直接使用与改变。

[解答程序]

具体程序见题目处。

注 1：ri 为引用，它是变量 i 的别名。

注 2：指针 pi 指向变量 i。

注 3: `i` 为独立变量, 具有其独立的变量地址。

注 4: `ri` 不是一个独立的变量, 它本身不具有独立的变量地址, 使用的是与其绑定的那一变量 `i` 的地址。

注 5: `pi` 为独立变量, 具有其独立的变量地址。

注 6: 对 `i` 赋值 1011 后, 使 `ri` 及 `*pi` 的值都成为 1011。

注 7: 对 `*pi` 赋值 2022 后, 使 `i` 及 `ri` 的值都成为 2022。

注 8: 对 `ri` 赋值 3033 后, 使 `i` 及 `*pi` 的值都成为 3033。

[输出结果]

`&i =0x0065FDF4`

`&ri=0x0065FDF4`

`&pi=0x0065FDEC`

`i=1011, *pi=1011, ri=1011`

`i=2022, *pi=2022, ri=2022`

`i=3033, *pi=3033, ri=3033`

第七章 类 与 对 象

[语法要点]

详细内容可参阅教材第七章。

(1) 类 (class) 是面向对象程序设计的核心概念。从程序的组织角度, C++通过 class 把数据以及对这些数据进行处理和运算的函数封装为互相关联的程序模块, 从数据类型的角度, C++通过 class 引入了抽象数据类型的概念, 一个由数据成员和函数成员组成的类就是一种新的数据类型。

(2) 类说明和对象说明

类说明的常用格式为:

```
class <类名>
{
private:
    < 私有成员表 >
public:
    < 公有成员表 >
};
例如:
class point
{
private:
    float Xcoord,Ycoord;
public:
    void SetX(float x)
    {
        Xcoord=x;
    }
    void SetY(float y)
    {
        Ycoord=y;
    }
    float GetX(void)
    {
        return Xcoord;
    }
}
```



```
float GetY(void)
{
    return Ycoord;
}

};
```

对象说明的格式为:〈类名〉〈对象表〉;

例如: point p1,p2;
stack s1,s2;

(3) 对象初始化:对象是类的一个实例,它也可以为其数据成员赋初值。不过对象的初始化情况比较复杂,有多种方式,其中最重要的方式是构造函数。

① 公有数据成员:

类的公有数据成员的初始化与一般变量,结构变量或变量数组的初始化没有什么区别。例如:

```
class address
{
    public:
        long telenum;
        char addr[30];
};

class person
{
    public:
        char name[15];
        int age;
        address paddr;
};

person p1={ "Zhang Hua", 23, {2475096,"NanKai University"} };
```

② 公有的初始化函数:

在类中设置公有的初始化函数完成此项任务,在上一段的类 point 中,函数 SetX(float x)和 SetY(float y)可以完成初始化的任务。

③ 构造函数:

为了更好地解决对象的初始化问题,C++规定在类的说明中可以包含一个或多个特殊的公有函数成员——构造函数。例如在 class point 中可有构造函数。

```
point(void)
{
    Xcoord=0.0;
    Ycoord=0.0;
}

point(float ix,float iy)
```

```
{  
    Xcoord=ix;  
    Ycoord=iy;  
}
```

前一个构造函数把每个新的对象，即一个平面上的点初始化为(0.0,0.0)，而第二个构造函数则将新的对象点赋予由用户指定的初值。如果这两个构造函数同时存在，程序员在说明新的对象时可任意选择：

```
point p(2.0,1.5), q;
```

上面的对象说明语句建立了两个对象 p 和 q，对于点 p，用参数 2.0 和 1.5 通过第二个构造函数进行初始化，而点 q 没有提供初始化的值，它将自动选择第一个构造函数，把 Xcoord, Ycoord 初始化为 0.0。

④ 析构函数：

与构造函数在对象生成时自动执行相对应，析构函数是专门用来在对象的生存期结束时做善后工作的。

(4) 类的静态成员：由关键字 static 修饰的类成员说明，称为静态成员。

类的静态成员为其所有对象共享，不管有多少对象，静态成员只有一份存于公用内存中。

静态数据成员实际上是该类所有对象的共享变量，静态函数成员被说明为静态的，同样将与该类的不同对象无关。

(5) 友元：在类的说明语句中出现，用关键字 friend 说明，其作用是：

① 位于一个函数说明语句之前，指出该函数为这个类的友元函数。

② 位于一个类名之前，指出该类是这个类的友元类。

例如：

```
class A  
{  
    ...  
    friend int f(int a);  
    ...  
    friend class B;  
    ...  
};
```

在类 A 中说明的友元函数 f:

- 它不是 A 的函数成员。
- f 的定义可以在类 A 的说明内，也可以在类外。
- 函数 f 虽不是 A 的成员，但有权访问和调用 A 的所有私有及保护成员。

在类 A 中说明的友元类 B:

- 它可能是与 A 无关的另外一个类。
- 类 B 在类 A 外说明。
- 类 B 有权访问和调用类 A 的所有成员，包括私有及保护成员。

(6) this 指针: 相当于在类 C 的每一成员函数中隐含定义了一个抽象的指针常量: `const C* this`; 其特点是:

① this 是类 C 的指针类型, 指向类 C 的对象地址。

② this 是在该类的对象 `cobj` 被创建后, 其成员函数 `f( )` 被调用时, this 指针也同时被说明和创建, 即

`C cobj;`

相当于在 `cobj.f( )` 中有一说明

`const C* this=&cobj;`

7.1 自定义类(类型)及其相关的构造与析构函数

读下面的程序, 写出执行结果。

```
#include<iostream.h>
int count=0;                                     //注 1

class myCla
{
    char cc;                                     //注 2
public:
    myCla(char ch)                             //注 3
    {
        ++count; cc=ch;                       //注 4
        cout<<"constructor: count="<<count<<" , ch="<<ch<<endl;
    }

    ~myCla( )                                  //注 5
    {
        --count;                               //注 6
        cout<<"destructor: count="<<count<<" , cc="<<getcc( )<<endl;
    }

    char getcc( ){return cc;}                  //注 7
};

myCla globalG('G');                             //注 8

void main(void)
{
```

```
myCla autoA('A'); //注 9
for(int i=1; i<=2; i++)
{
    cout<<"---- begin block"<<endl;
    myCla autoB('B'); //注 10
    static myCla staticS('S'); //注 11
    cout<<"---- end block"<<endl;
}
cout<<"---- end main"<<endl;
}
```

[分析提示]

(1) 类的构造函数与析构函数都是特殊的类成员函数。构造函数主要完成对象的初始化工作，而析构函数主要完成对象撤消前的善后工作。当一个新的对象被建立时(通过对象说明语句或通过 new)，系统将自动调用其相应的构造函数；当对象退出其说明区域，或使用 delete 释放动态对象时，系统将自动调用其析构函数。形象地说，当对象诞生和死亡时，将分别调用该对象所属类的构造和析构函数。

(2) 与“小级别类型变量”的作用域及生存期类似，对象(即“大级别”的类类型的变量)也有各自的作用域及它们的生存期。如全局性对象将在执行 main 前被构造，在 main 结束时。各函数内的局部对象(auto 属性者)，将在说明时被构造，在函数出口时。块级别的局部对象 auto 属性者)，在每次入块后都被重新构造(重新分配空间并初始化)，出块时结束生命期(调用析构函数去析构它们)。块级别的局部静态对象 static 属性者)，其作用域局部于块，但其生命期与全局性对象相同，只在第一次遇见时分配空间并初始化。

[解答程序]

具体程序见题目处。

注 1: count 记录当前具有对象的个数。

注 2: 私有数据成员。

注 3: 构造函数。

注 4: 每进入构造函数，count 加 1。

注 5: 析构函数。

注 6: 每进入析构函数，count 减 1。

注 7: 获取私有数据成员。

注 8: 全局性 MyCla 对象 globalG 在执行 main 前被构造，main 结束时死亡。

注 9: main 中的函数级别(auto 局部性)对象 autoA，在说明时被构造，在函数出口时死亡。

注 10: 块级别(auto 局部性)对象 autoB，每次入块后都被重新构造(重新分配空间并初始化)，出块时结束生命期(调用析构函数去析构 autoB)。

注 11: 块级别局部静态对象 i3, 其作用域局部于块, 但其生命期与全局性对象相同, 只在第一次遇见时分配空间并初始化。

[输出结果]

```
constructor: count=1, ch=G
constructor: count=2, ch=A
----- begin block
constructor: count=3, ch=B
constructor: count=4, ch=S
----- end block
destructor: count=3, cc=B
----- begin block
constructor: count=4, ch=B
----- end block
destructor: count=3, cc=B
----- end main
destructor: count=2, cc=A
destructor: count=1, cc=S
destructor: count=0, cc=G
```

7.2 动态对象的构造与析构

分析程序执行后的显示结果。

```
#include <iostream.h>
class myCla
{
    int a, b;                                //注 1
public:
    void SetXY(int x, int y)                 //注 2
    {
        a=x;  b=y;
    }

    myCla()                                  //注 3
    {
        cout<<"constructor!\n";
        a=0;  b=0;
    }
}
```

```
~myCla( )                                //注 4
{
    cout<<"destructor! "<<"a ="<<a;
    cout<<"  b ="<<b<<endl;
}

void show( )                             //注 5
{
    cout<<"a="<<a<<"  b="<<b<<endl;
}

};

void main( )
{
    myCla *pa=new myCla[3];              //注 6
    for(int i=0; i<3; i++)
        pa[i].show( );                  //注 7
    myCla *p1=new myCla;                 //注 8
    p1->SetXY(66,88);                    //注 9
    for(i=0; i<3; i++)
        pa[i].SetXY(i*i+1, i*i+1);      //注 10
    delete p1;                           //注 11
    for(i=0; i<3; i++)
        pa[i].show( );                  //注 12
    delete [ ]pa;                        //注 13
}
```

[分析提示]

使用 new 来动态生成 myCla 对象时要调用构造函数，此时对象诞生；使用 delete 来动态释放 myCla 对象时要调用析构函数，此时对象死亡。

[解答程序]

具体程序见题目处。

注 1：类的私有数据成员。

注 2：设置 myCla 对象的值。

注 3：构造函数。

注 4：析构函数。

注 5：显示 myCla 对象的 a，b 值。

注 6: 动态生成三个 myCla 对象(动态数组), 将三次调用构造函数进行初始化。

注 7: 显示三个 myCla 对象的初始化值。

注 8: 再动态生成一个 myCla 对象, 并由指针 p1 指向它。

注 9: 为 p1 所指向的对象设置新值。

注 10: 重新为动态数组中的三个 myCla 对象设置新值。

注 11: 释放 p1 所指向的对象, 系统将自动去调用析构函数 “~myCla()”。

注 12: 显示动态数组中的三个 myCla 对象的新设置值。

注 13: 释放由 pa 所指向的对象数组, 系统将三次自动调用析构函数 “~myCla()”。

注意, 若无此句的话, 系统不会自动去调用析构函数“~myCla()”。

[输出结果]

```

constructor!
constructor!
constructor!
a=0    b=0
a=0    b=0
a=0    b=0
constructor!
destructor! a =66,  b =88
a=1    b=1
a=2    b=2
a=5    b=5
destructor! a =5,  b =5
destructor! a =2,  b =2
destructor! a =1,  b =1

```

7.3 自定义的 Circle 类(类型)以及对它的使用

完成如下的自定义 Circle 类(类型), 它具有表示半径的私有数据成员 radius; 并提供构造函数、计算圆周长以及圆面积的公有成员函数。并编制主函数对 Circle 类进行使用: 说明 Circle 类对象, 输入其半径与单位面积造价, 而后计算出周长 circ、面积 are、以及整体造价 cost 并输出。

```

class Circle                                //自定义类 Circle
{
    double    radius;                        //私有数据成员 —— 半径
public:
    Circle ( double r );                    //构造函数 —— 参数为半径值
    double Circum( );                       //计算圆周长的公有成员函数

```

```
double Area( );           //计算圆面积的公有成员函数
};
```

[分析提示]

(1) 在成员函数 Circum 和 Area 中可直接使用 Circle 类的私有数据成员 radius, 进而按公式计算出各调用者对象的圆周长和圆面积。

(2) 在主调函数 main 中, 必须通过类对象来对公有成员函数 Circum 和 AreaArea 进行调用。

[解答程序]

```
#include <iostream.h>
const double PI = 3.1415926;

class Circle                //自定义类 Circle
{
    double    radius;       //私有数据成员 —— 半径
public:
    Circle ( double r );    //构造函数 —— 参数为半径值
    double Circum( );       //计算圆周长的公有成员函数
    double Area( );         //计算圆面积的公有成员函数
};

Circle::Circle(double r)    //成员函数的类外说明
{
    radius=r;               //构造函数初始化数据成员 radius
}

double Circle::Circum( )    //成员函数的类外说明
{
    return 2 * PI * radius; //计算圆的周长
}

double Circle::Area( )      //成员函数的类外说明
{
    return PI * radius * radius; //计算圆的面积
}

//主函数对 Circle 类进行使用: 说明 Circle 类对象, 输入其半径与单位面积造价,
//计算出周长 circ、面积 are、以及整体造价 cost 并输出
```



```

void main ( )
{
    double radius, price;
    cout<<"Enter the radius and the price: "; //提示用户输入半径与单位面积造价
    cin>>radius>>price;                    //输入 radius 与 price
    Circle obj1(radius);                    //定义 Circle 类对象 obj1
    double circ, are, cost;
        //要计算对象 obj1 的周长 circ、面积 are、以及整体造价 cost
    circ=obj1.Circum( );                    //通过类对象 obj1 对公有成员函数 Circum 进行调用
    are=obj1.Area( );                       //通过类对象 obj1 对公有成员函数 Area 进行调用
    cost=price*are;
    cout << "circ=" << circ<< endl;        //输出结果
    cout << "are=" << are<< endl;
    cout << "cost=" << cost<< endl;
}

```

[输出结果]

```

Enter the radius and the price: 22.5 11.2
circ=141.372
are=1590.43
cost=17812.8

```

7.4 输入十个整数而后反序输出的实现方式八

读下述程序，并上机进行验证，看是否实现了输入 10 个整数而后将它们反序输出的预期目标。注意，本程序使用自定义类(类型)，在类的私有数据成员中存放输入数据而后输出。

```

#include <iostream.h>
class myCla                                //注 1
{
    int arr[10];                           //注 2
public:
    void myIn( );                           //注 3
    void myOut( );                          //注 4
};

void myCla::myIn( )                        //注 5
{

```

```
        cout<<"input 10 integers:"<<endl;
        for(int i=0; i<10; i++)                //注 6
            cin>>arr[i];
    }

    void myCla::myOut( )                        //注 7
    {
        cout<<"---- The result ----"<<endl;
        for(int i=9; i>=0; i--)                //注 8
            cout<<arr[i]<<" ";
        cout<<endl;
    }

    void main( )
    {
        myCla obj1;                            //注 9
        obj1.myIn( );                          //注 10
        obj1.myOut( );                         //注 11
    }
```

[分析提示]

首先自定义类(类型) myCla, 而后通过面向对象方法说明类对象并通过对象调用其负责输入的成员函数输入数据(放入类对象的数据成员 arr 中), 而后再通过对象调用另一成员函数将数据成员 arr 中的输入数据按反序输出。

[解答程序]

具体程序见题目处。

注 1: 自定义类(类型)myCla。

注 2: 类的私有数据成员 arr, 在其中存放数据。

注 3: 输入数据的成员函数。

注 4: 将数据按输入的相反顺序输出的成员函数。

注 5: 成员函数 myIn, 负责输入 10 个数据。

注 6: 将输入数据放入 arr 中。

注 7: 成员函数 myOut, 负责将数据按输入的相反顺序输出来。

注 8: 逆序输出 arr 中的数据。

注 9: 类对象 obj1。

注 10: 通过类对象 obj1 调用成员函数 myIn, 输入 10 个数据。

注 11: 通过类对象 obj1 调用成员函数 myOut, 将数据按输入的相反顺序输出来。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
----- The result -----
10 9 8 7 6 5 4 3 2 1
```

7.5 自定义类及其有关类成员函数的使用

读下述程序，写出执行结果(假设执行时输入了：2 4 6 8 10 五个数据)。

```
#include<iostream.h>
class myCla //注 1
{
    double dat;
public:
    myCla(double d=0) { dat=d; } //注 2
    void inDat( ) { cin>>dat; } //注 3
    myCla addDat(myCla d2); //注 4
    myCla subDat(myCla d2); //注 5
    myCla mulDat(myCla d2); //注 6
    void outDat( ) { cout<<"d="<<dat<<" "; } //注 7
};

myCla myCla::addDat(myCla d2) //注 8
{
    myCla d;
    d.dat=dat+d2.dat;
    return d;
}

myCla myCla::subDat(myCla d2) //注 9
{
    myCla d;
    d.dat=dat-d2.dat;
    return d;
}

myCla myCla::mulDat(myCla d2) //注 10
{
```

```
        myCla temp;
        temp.dat=dat*d2.dat;
        return temp;
    }

void main(void)
{
    myCla obj1(5), dArr[5]; //注 11
    cout<<"Input 5 myCla(5 integers):";
    for(int i=0; i<5; i++) //注 12
        dArr[i].inDat( );
    myCla resA[5], resS[5], resM[5], *pa=resA, *ps=resS, *pm=resM; //注 13
    for(i=0; i<5; i++) //注 14
    {
        resA[i]=obj1.addDat(dArr[i]);
        resS[i]=obj1.subDat(dArr[i]);
        resM[i]=obj1.mulDat(dArr[i]);
    }
    while (pa<=&resA[4]) //注 15
    {
        pa->outDat( );
        pa++;
    }
    cout<<endl;
    while (ps<=&resS[4]) //注 16
    {
        ps->outDat( );
        ps++;
    }
    cout<<endl;
    while (pm<=&resM[4]) //注 17
    {
        pm->outDat( );
        pm++;
    }
    cout<<endl;
}
```

[分析提示]

(1) 类成员函数内(定义处)可处理与使用本类的私有成员,而且总以当前调用者对象(*this)作为该成员函数的隐式第一运算分量。若所定义的运算多于一个运算对象时,才将其余运算对象显式地列在该成员函数的参数表中。如,本范例的 addDat、subDat 以及 mulDat 虽然都只含有一个 myCla 类的对象参数,但这三个成员函数都要处理两个运算对象--调用者对象以及参数对象。

(2) 可以说明对象的数组,其每一个数组分量都为同一种类类型的类对象,而数组名则仍为一个常量地址,也即指向数组首分量的一个常量指针。

[解答程序]

具体程序见题目处。

注 1: 自定义类类型 myCla。

注 2: 构造函数,将私有数据成员 dat 的值设置为 d,而参数 d 的隐含值(默认值)为 0。

注 3: 输入对象的有关数据放入私有数据成员 dat 中。

注 4: 实现两个 myCla 类对象的加法运算(调用者对象与对象 d2 相加)。注意,参数为 myCla 类对象,且返回结果类型仍为 myCla。

注 5: 实现两个 myCla 类对象的减法运算(调用者对象与对象 d2 相减)。

注 6: 实现两个 myCla 类对象的乘法运算(调用者对象与对象 d2 相乘)。

注 7: 输出 myCla 类对象的有关数据。

注 8: addDat 成员函数的具体实现。

注 9: subDat 成员函数的具体实现。

注 10: mulDat 成员函数的具体实现。

注 11: 类对象 obj1,其初值为 5;以及具有五个分量的类对象数组 dArr。

注 12: 为类对象数组 dArr 输入数据(五个整数,数间以空格分割)。

注 13: resA 数组中存放 obj1 对象与 dArr 数组的五个对象相加后的结果;resS 数组中存放 obj1 对象与 dArr 数组的五个对象相减后的结果;resM 数组中存放 obj1 对象与 dArr 数组的五个对象相乘后的结果。

注 14: 执行 5 次循环,每次都要调用类成员函数来实现对象的加法、减法与乘法运算。

注 15: 输出 resA 数组中存放的加法结果。

注 16: 输出 resS 数组中存放的减法结果。

注 17: 输出 resM 数组中存放的乘法结果。

[输出结果]

```
Input 5 myCla(5 integers):2 4 6 8 10
d=7 d=9 d=11 d=13 d=15
d=3 d=1 d=-1 d=-3 d=-5
d=10 d=20 d=30 d=40 d=50
```

7.6 一个简单的复数类型 complex

自定义具有如下核心程序代码的一个简单的复数类型 `complex`，其中说明了多个类成员函数，使用该类可以完成复数的输入、加法、减法、乘法、以及对复数的输出。请完成各成员函数，并编制主函数，说明 `complex` 类对象，对定义的各成员函数进行调用，以验证它们的正确性。

```
class complex                                //自定义的复数类型 complex
{
    double real;                             //复数实部
    double imag;                             //复数虚部
public:
    complex(double r=0, double i=0);         //构造函数
    void inCom( );                           //输入调用者对象的有关数据(各分量)
    complex addCom(complex c2);              //调用者对象与对象 c2 相加，
                                              //返回 complex 类对象
    complex subCom(complex c2);              //调用者对象与对象 c2 相减，
                                              //返回 complex 类对象
    complex mulCom(complex c2);              //调用者对象与对象 c2 相乘，
                                              //返回 complex 类对象
    void outCom ( );                         //输出调用者对象的有关数据(各分量)
};
```

[分析提示]

类成员函数内(定义处)可处理与使用本 `complex` 类的私有成员，而且总以当前调用者对象作为该成员函数的隐式第一运算分量。当所处理的运算对象多于一个时，才将其余的对象显式地列在成员函数的参数表中。注意 `addCom`、`subCom` 以及 `mulCom` 三个成员函数的返回值均为 `complex` 类对象。

[解答程序]

```
#include<iostream.h>
class complex                                //自定义的复数类型 complex
{
    ...                                     //与题目处相同，略。
};

complex::complex(double r, double i)
{
    //构造函数，为调用者对象的私有数据成员赋初值
```

```
        real=r;
        imag=i;
    }

    void complex::inCom( )           //输入调用者对象的有关数据(各分量)
    {
        cin>>real>>imag;
    }

    complex complex::addCom(complex c2)
    {
        //调用者对象与对象 c2 相加, 返回 complex 类对象
        complex c;
        c.real=real+c2.real;
        c.imag=imag+c2.imag;
        return c;
    }

    complex complex::subCom(complex c2)
    {
        //调用者对象与对象 c2 相减, 返回 complex 类对象
        complex c;
        c.real=real-c2.real;
        c.imag=imag-c2.imag;
        return c;
    }

    complex complex::mulCom (complex com2)
    {
        //调用者对象与对象 c2 相乘, 返回 complex 类对象
        complex temp;
        temp.real=(real*com2.real)-(imag*com2.imag);
        temp.imag=(real*com2.imag)+(imag*com2.real);
        return temp;
    }

    void complex::outCom ( )         //输出调用者对象的有关数据(各分量)
    {
        cout<<"("<<real<<", "<<imag<<")";
    }
```

```

    }

    void main(void)
    {
        complex c1, c2, res;           //说明类对象 c1, c2, res
        cout<<"Input complex1 ( 2 integers ):";
        c1.inCom( );                   //为类对象 c1 输入数据
        cout<<"Input complex2 ( 2 integers ):";
        c2.inCom( );                   //为类对象 c2 输入数据
        res=c1.addCom(c2);              //调用 addCom 实现复数加法, 完成 c1+c2=>res
        c1.outCom( );                  //输出类对象 c1
        cout<<" + ";
        c2.outCom( );                  //输出类对象 c2
        cout<<" = ";
        res.outCom( );                 //输出类对象 res
        cout<<"\n-----"<<endl;
        res=c1.subCom(c2);              //调用 subCom 实现复数减法, 完成 c1-c2=>res
        c1.outCom( );                  //输出类对象 c1
        cout<<" - ";
        c2.outCom( );                  //输出类对象 c2
        cout<<" = ";
        res.outCom( );                 //输出类对象 res
        cout<<"\n-----"<<endl;
        res=c1.mulCom(c2);              //调用 mulCom 实现复数乘法, 完成 c1*c2=>res
        c1.outCom( );                  //输出类对象 c1
        cout<<" * ";
        c2.outCom( );                  //输出类对象 c2
        cout<<" = ";
        res.outCom( );                 //输出类对象 res
        cout<<"\n-----"<<endl;
    }

```

[输出结果]

Input complex1 (2 integers):1 1

Input complex2 (2 integers):2 3

(1, 1) + (2, 3) = (3, 4)

(1, 1) - (2, 3) = (-1, -2)

```

float student::sum(void)                //求三门课总成绩的函数 sum
{
    return(math+eng+comp);
}

float student::average(void)           //求三门课平均成绩的函数 average
{
    return((math+eng+comp)/3);
}

void student::print(void)              //显示学生数据信息的函数 print
{
    cout<<reg_num<<" "<<name<<" "<<math<<" "<<eng<<" "<<comp<<endl;
}

int student::get_reg_num( )            //获取学生注册号的函数 get_reg_num
{
    return(reg_num);
}

void student::set_stu_inf(int r,char* n,float m,float e,float c)
{
    //设置学生数据信息的函数 set_stu_inf

    reg_num=r;    strcpy(name,n);
    math=m;       eng=e;       comp=c;
}

/*
    编制主函数，说明一个 student 类对象的数组并进行全班学生信息的输入与设置，而后
    求出每一学生的总成绩、平均成绩、全班学生总成绩最高分、全班学生总平均分，并在输
    入一个注册号后，输出该学生有关的全部数据信息。
*/

void main( )
{
    const int maxsize=100;
    student stu[maxsize];           //说明一个 student 类对象的数组
    int reg_num;
    char name[30];

```

```

float math, eng, comp;
int total;                                //共 total 个学生
do
{
    cout<<"integer TOTAL = ? ";
    cin>>total;                            //total 之值由用户输入
    if(total>maxsize||total<1)
        cout<<"ERROR: 0 < TOTAL <= "<<maxsize<<" (The maxsize)!"<<endl;
}
while (total>maxsize||total<1); //输入的 total 必须处于 1 与 maxsize 之间
cout<<"Input student inf (->reg_num ->name ->math ->eng ->comp) "<<endl;
for (int i=0;i<total;i++)
{
    //进行全班学生信息的输入与设置, 共 total 个学生
    cout<<"student "<<i+1<<":";
    cin>>reg_num>>name>>math>>eng>>comp;    //输入一个学生的有关信息
    stu[i].set_stu_inf(reg_num,name,math,eng,comp); //存入对象数组中
}
cout<<"-----"<<endl;
for (i=0;i<total;i++)
    stu[i].print( );
float sum_max=0.0, class_ave=0.0;
//求出每一学生的总成绩、平均成绩、全班学生总成绩最高分、全班学生总
//平均分
for (i=0; i<total; i++)                //共 total 个学生
{
    float tmp_sum, tmp_ave;
    tmp_sum=stu[i].sum( );    //总成绩
    tmp_ave=stu[i].average( ); //平均成绩
    cout<<"student"<<i+1<<".sum="<<tmp_sum;
    cout<<", student"<<i+1<<".average="<<tmp_ave<<endl;
    if(tmp_sum>sum_max)
        sum_max=tmp_sum; //全班学生总成绩最高分为 sum_max
    class_ave+=tmp_ave;    //为计算全班学生总平均分做累加
}
class_ave/=total;        //全班学生总平均分 class_ave
cout<<"class_sum_max="<<sum_max<<endl;
cout<<"class_average="<<class_ave<<endl;
cout<<"input searching reg_number:";

```

```

int reg_n;
//输入一个注册号后，输出该学生有关的全部数据信息
cin>>reg_n;                //输入注册号
bool found=false;
for (i=0; i<total; i++)
if( stu[i].get_reg_num( )==reg_n)    //查找具有该注册号的学生
{
    cout<<"The infomation of the student with the reg_number:";
    stu[i].print( );                //输出该学生有关的全部数据信息
    found=true;
    break;
}
if(!found)                    //没查到具有该注册号的学生时提示信息
    cout<<"No student with the reg_number!"<<endl;
}

```

[输出结果]

```

integer TOTAL = ? 3
Input student inf (->reg_num ->name ->math ->eng ->comp)
student 1:1001 ma 77 86 90
student 2:1002 li 87 88 95
student 3:1003 hu 92 89 96
-----
1001  ma  77  86  90
1002  li  87  88  95
1003  hu  92  89  96
student1.sum=253,  student1.average=84.3333
student2.sum=270,  student2.average=90
student3.sum=277,  student3.average=92.3333
class_sum_max=277
class_average=88.8889
input searching reg_number:1002
The infomation of the student with the reg_number:1002  li  87  88  95

```

7.8 自定义的 queue(队列) 类型

队列(queue) 是一种程序设计中常用的数据结构。它的结构和栈(stack) 类似，但队列中的数据是先进先出。仿照教材 7.1 节中对栈(stack)的定义，来设计并完成下述队列

类 queue 的定义，其核心代码程序如下(注意，要求对存放队列数据的动态数组 data 进行循环使用)：

```
class queue
{
    int size; //队列的大小(数据项数)，即是说，每一个队列类对象的大小可以不同
    double * data;
    //动态数组 data，在构造函数中，将通过 new 来生成该动态数组
    //将它用来存放队列数据。动态数组 data 的大小由私有数据成员
    //size 之值来确定，而且要循环使用数组空间。
    int front,rear,num;
    //front,rear 为队列数据首尾下标位置；num 记录当前队列中具有的数据个数
public:
    queue (int maxsize);
    //构造函数，负责生成一个 double 类型的、maxsize 大小的动态数组
    //(将用于存放队列数据)，并对 front、rear、num 进行初始化(意味着空队列)
    ~queue(void);          //析构函数，释放动态数组 data 占用的空间
    bool empty(void);      //成员函数 empty，判断队列是否为空
    bool full(void);       //成员函数 full，判断队列是否已满
    void add (double a);   //成员函数 Add，用于往队列尾部加入元素 a
    double Delete (void);  //用于删取队列首部的那一元素
    //也即返回队列首部的那一元素，并下推队列首位置 front
};
```

并使用下述主函数对 queue 类型进行应用，以验证其正确性。

```
void main( )
{
    queue s1(6), s2(4);
    int i=1;
    do{
        s1.add(2*i);
        i++;
    } while(!s1.full( ));
    //往队列 s1 中加入了 6 个数据，此时 s1 队列已满 -- 为：2，4，6，8，10，12
    for (i=1; i<=4; i++)
        cout<<s1.Delete( )<<" ";
    //删取并输出 s1 的前 4 个数据 -- 2，4，6，8( 此时 s1 队列中仅剩
    //两个数据：10，12，且队列的前 4 个位置已空，可以重新用来放数据)
    for (i=1; i<=6; i++)
        s1.add(3.5*i);
```

```

//又往 s1 中加入 6 个数据( 实际加入 4 个数据后队列满, 后两个没加入)
//由于循环使用数组空间, 使得往前面加入新数据 -- 3.5, 7,
//10.5, 14, 成为可能。此时队列中的数据顺序为: 10, 12, 3.5, 7,
//10.5, 14( 注意: 总由 front 指向首数据)
cout<<endl;
for (i=1; i<=3; i++)
    cout<< s1.Delete()<<" ";
//删取并输出 s1 的前 3 个数据 --: 10, 12, 3.5
cout<<s1.Delete()<<endl; //又删取并输出 s1 的 1 个数据 --: 7
//此时 s1 队列中仅剩两个数据: 10.5, 14)
while(!s1.empty()) //将 s1 队列中剩余的两个数据取出加入到 s2 队列中
    s2.add(s1.Delete()); //往队列 s2 中加入两个数据 -- 10.5, 14
do
    cout<<s2.Delete()<<" "; //输出队列 s2 中的两个数据 -- 10.5, 14
while (! (s2.empty()));
cout <<endl;
}

```

[分析提示]

(1) 使用“ class queue { ...}”的方式定义出所需的队列类, 通过其中的数据成员 data (一个动态数组) 来盛放队列所拥有的数据, 并设立 front 与 rear 数据成员来记录队列之当前数据的首尾下标位置。每次总从 data[front]处也即队列首部取用数据, 而总往队列尾部也即 data[rear]位置处添加数据。

(2) 本范例要求对存放队列数据的动态数组 data 进行循环使用, 为能实现这一要求, 在成员函数 add 中, 每当往队列尾部加入元素 a 时, 并不能直接通过“ data[rear]=a;”来完成, 而要首先进行“ rear=(rear+1)%size;”的运算, 而后再通过使用“ data[rear]=a;”来实现(其中的 rear 表示当前队列之末尾数据的下标位置, 而 size 则表示队列的大小、也即动态数组 data 最多可以盛放数据的个数多少)。另外在成员函数 Delete 中(用于删取队列首部的那一元素, 也即返回队列首部的那一元素, 并下推队列首位置 front), 也要首先进行“ front=(front+1)%size;”的运算, 而后再通过“ return data[front];”返回首部元素。

[解答程序]

```

#include <iostream.h>
#include <process.h>
class queue
{
    ...
    //与题目处相同, 略!
};

```

```
queue::queue(int maxsize)
{
    //构造函数，负责生成一个 double 类型的、maxsize 大小的动态数组
    //(将用于存放队列数据)，并对 front,rear,num 进行初始化(意味着空队列)
    size=maxsize;
    data=new double[size];
    front=-1; rear=-1; num=0;      //意味着队列为空(其元素个数 num=0)
    cout<<"queue initialized."<<endl;
}

queue::~queue(void)
{
    //析构函数，释放动态数组 data 占用的空间
    delete [ ]data;
    cout<<"queue destroyed."<<endl;
}

bool queue::empty(void)
{
    //成员函数 empty，判断队列是否为空
    return((num==0)?true:false);
}

bool queue::full(void)
{
    //成员函数 full，判断队列是否已满
    return ((num==size)?true:false);
}

void queue::add (double a)
{
    //成员函数 Add，用于往队列尾部加入元素 a
    if (full( ))                //若队列已满
        cout<<"\nThe queue is full!";    //显示信息后，不再加入数据
    else
    {
        rear=(rear+1)%size; //为能循环使用数组空间而进行模 size 的运算
        data[rear]=a;        //往队列尾部加入元素 a
    }
}
```

```

        num++;           //记录当前队列中所具有数据个数的 num 加 1
    }
}

double queue::Delete (void)
{
    //用于删取队列首部的那一元素
    //也即返回队列首部的那一元素，并下推队列首位置 front
    if ( empty( ) )
    {
        cout<<"\nThe queue is empty!";
        exit(0);
    }
    front=(front+1)%size;    //为能循环使用数组空间而进行模 size 的运算
    num--;
    return data[front];
}

void main( )
{
    ...                    //与题目处相同，略!
}

```

[输出结果]

```

queue initialized.
queue initialized.
2, 4, 6, 8,
The queue is full!
The queue is full!
10, 12, 3.5, 7
10.5, 14,
queue destroyed.
queue destroyed.

```

7.9 在平面上用直线、矩形和圆画出一个粗略的汽车图形

自定义表示直线、矩形和圆的 Line、Rectang、Circle 类(类型)，而后在平面上用直线、矩形和圆画出一个粗略的汽车图形轮廓线。

[分析提示]

(1) 涉及到绘图的程序, 在 VC6.0 环境下, 可按照如下的 4 个步骤来处理、来实现。

步 1: 使用 VC6.0 的 AppWizard 自动生成一个典型的“ Hello World” 程序框架。

生成这一种框架程序的操作步骤为: “ File ” => “ New ” => “ Projects ” => “ Win32 Application ” => (右上处) “ Project name ” 域中填入如 “ 7_7 drawing ”, “ Location ” 框内, 添入你自用的文件夹名 => “ OK ” => 选第 3 种类型: “ A Typical ‘ Hello World ’ application ” => “ Finish ” => “ OK ”。

步 2: 在所生成的框架程序基础上, 增加相关的绘图处理类、以及处理 WM_PAINT 消息的重画处理函数 MyPaintProFunc, 并将它们组织在同一个头文件, 如 “ 7_7 drawing2.h ” 之中。

步 3: 在所生成的那一主要文件 “ 7_7 drawing.cpp ” 的开头处(最初的两个 #include 行之下), 加入如下的一行:

```
#include "7_7 drawing2.h" //新增头文件
```

//内容为本例相关的绘图处理类、以及处理 WM_PAINT 消息的重画处理函数 MyPaintProFunc

步 4: 在所生成的那一主要文件 “ 7_7 drawing.cpp ” 的 WndProc 函数的 switch 语句中, 将 case WM_PAINT: 处理分支内的 GetClientRect(hWnd, &rt); 与 EndPaint(hWnd, &ps); 之间的 “ DrawText(hdc, szHello, strlen(szHello), &rt, DT_CENTER); ” 语句替换为如下的一个函数调用句:

```
MyPaintProFunc(hWnd, hdc, rt);
```

```
//调用本例的绘图处理函数 MyPaintProFunc(处于"7_7 drawing2.h"文件中)
```

(2) 下面再给出一些有关的解释与说明:

通过上述的步 1 所自动生成的 “ Hello World ” 程序框架, 它含有十多个文件, 这些文件包括 “ Source Files ”、“ Header Files ”、以及 “ Resource Files ” 等, 它已经是一个可执行程序, 执行该程序后将在 “ Windows 窗口模式 ” 下显示出(确切地说是绘制或画出)字符串 “ Hello World! ”。可以看出, 通过使用 AppWizard, 用户只需进行一些框架程序类型以及其他一些相关的具体指定, 而不必亲自编写任何一条程序代码, 就获得了一个可执行的程序。

另一点要注意的是, 执行所生成程序后进入的是 “ Windows 窗口模式 ”(与 “ Dos 窗口模式 ” 不同, 该模式支持鼠标操作, 是以消息进行驱动执行的), 而且还自动为我们提供了可用的 File 和 Help 菜单以及一些简单的菜单命令项, 以及可用于进行窗口极大化、极小化、和关闭的鼠标按钮。这正是 VC6.0 之 AppWizard 的诱人之处, 它为开发 Windows 程序提供了方便, 使程序员不必再去为每一应用编写那些基本相同的框架式底层支撑程序也即上述自动生成的那样的所谓框架程序), 而把注意力放在进行逻辑处理的所谓 “ 上层程序部分 ” 的编写上。

VC6.0 的所有绘图功能, 都要在上述 “ Windows 窗口模式 ” 下来实现, 而 Windows 编程技术已超出了本课程的教学范围。本课作为示例的几个具有绘图功能的与自定义类相关的程序, 都是在上述自动生成的框架程序的基础上、通过添加某些处于上层的与绘图逻辑有关的程序代码而完成的。读者应把注意力集中在这些 “ 上层 ” 程序代码的编写

与添加上, 上述的步 2 到步 4 正是要在自动生成那一框架程序的基础上, 主要通过与其中的“.cpp”文件进行“衔接交融”, 进而把那些与绘图逻辑有关的程序代码添加进去。至于 Windows 编程技术则还需要通过学习一些专门的有关书籍与资料方可掌握与应用。通过上述步 1 所生成的框架程序的“.cpp”文件主要由以下 5 个函数来构成: WinMain、MyRegisterClass、InitInstance、WndProc、About。

其中的 WinMain 等同于 Dos 窗口模式下的 main 函数, 它首先通过调用 InitInstance 来进行本应用的初始化工作, 而后进入一个消息循环, 循环检查、发送、并处理本应用的各种消息(实际上仅负责调用相应的消息处理函数而指派它们去处理各消息), 直至用户对此应用进行了关闭为止。MyRegisterClass 用于注册窗口类。InitInstance 用于对本应用进行初始化。About 对应于 Help 菜单的 About 命令项, 将负责显示 About 对话框并处理其相应消息。

WndProc 是我们最关心的一个函数, 它负责处理主窗口的各种消息。实际上, 它是通过一个 switch 语句先分辨出不同的消息, 而后大多情况下再调用相应的消息处理函数并指派它们去进行具体处理。其 switch 语句的格式如下(其中有所省略并加入了一些注释):

```
switch (message)
{
    case WM_COMMAND:
        ... //对 Windows 的“ WM_COMMAND” 消息(如, 各种菜单命令消息)的处理
        break;
    case WM_PAINT: //对 Windows 的“ WM_PAINT” 消息的处理分支
        hdc = BeginPaint(hWnd, &ps);
        // TODO: Add any drawing code here...
        RECT rt;
        GetClientRect(hWnd, &rt);
        DrawText(hdc, szHello, strlen(szHello), &rt, DT_CENTER);
        EndPaint(hWnd, &ps);
        break;
    case WM_DESTROY:
        ... //对 Windows 的“ WM_DESTROY” 消息的处理代码
        break;
    default:
        ... //缺省情况的处理代码
}
```

实际上, 绘图逻辑程序代码的添加位置可以放在上述 WndProc 函数的 switch 语句中的“ case WM_PAINT:”分支内。因为凡要进行“绘图”或对窗口进行“重画”时都产生该“ WM_PAINT”消息, 最终都要进入该“ case WM_PAINT:”分支内进行处理。现将该分支下所用几个语句的功能简述如下:

```
case WM_PAINT: //对 Windows 的“ WM_PAINT” 消息的处理分支
```

```

hdc = BeginPaint(hWnd, &ps);    //hWnd 对应的窗口将被“重画”
    //获取设备描述表(Device Context, 也称设备上下文或设备环境)
    //句柄送 hdc TODO: Add any drawing code here...
RECT rt;                        //rt 代表一个窗口区域
GetClientRect(hWnd, &rt);       //使 rt 区域与 hWnd 对应的窗口关联起来
DrawText(hdc, szHello, strlen(szHello), &rt, DT_CENTER);
//具体的“重画”程序代码, 框架程序仅“画出”一个“Hello World!”字符串。
//若要“画”其他内容, 则应在此处给出相应的代码以取代该 DrawText 语句。
//本实现总将此语句替换为如下的一个函数调用句: MyPaintProFunc(hWnd, hdc, rt);
//而总在 MyPaintProFunc 函数之中“放置”本实现的具体“重画”程序代码。
EndPaint(hWnd, &ps);
//与上述“BeginPaint”呼应的“EndPaint”, 结束本次“重画”
break;

```

[解答程序]

```

/*
    下面给出的是, 经过上述的步 1 至步 4 之后, “7_7 drawing.cpp”文件的大致核心内容;
    以及另一个新增头文件“7_7 drawing2.h”的内容, 该头文件与“7_7 drawing.cpp”相呼应,
    内容为本例相关的“绘图”处理类 -- Line、Rectang、Circle 类(类型)、以及处理 WM_PAINT
    消息的“重画”处理函数 MyPaintProFunc。利用它们就可以在平面上画出一个粗略的汽车
    图形。
*/

// 7_7 drawing.cpp : Defines the entry point for the application.
//

#include "stdafx.h"
#include "resource.h"
#include "7_7 drawing2.h"          //新增头文件

//内容为本例相关的“绘图”处理类、以及处理 WM_PAINT 消息的“重画”处理函数
MyPaintProFunc (HWND hwnd,HDC hde,RECT rt)
{
    ...
}

int APIENTRY WinMain(HINSTANCE hInstance,
                     HINSTANCE hPrevInstance,
                     LPSTR     lpCmdLine,
                     int        nCmdShow)
{

```

```
...
}

ATOM MyRegisterClass(HINSTANCE hInstance)
{
    ...
}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow)
{
    ...
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam,
LPARAM lParam)
{
    ...

    switch (message)
    {
        case WM_COMMAND:
            ...
            break;

        case WM_PAINT:
            hdc = BeginPaint(hWnd, &ps);
            // TODO: Add any drawing code here...
            RECT rt;
            GetClientRect(hWnd, &rt);

            MyPaintProFunc(hWnd, hdc, rt);
            //调用本例的“绘图”处理函数 MyPaintProFunc(处于“7_7
            //drawing2.h”文件中)

            EndPaint(hWnd, &ps);
            break;
        case WM_DESTROY:
            ...
            break;
```

```

        default:
            ...
    }
    return 0;
}

LRESULT CALLBACK About(HWND hDlg, UINT message, WPARAM wParam, LPARAM
lParam)
{
    ...
}

//7_7 drawing2.h, 新增头文件, 与“ 7_7 drawing.cpp” 相呼应
//内容为本例相关的“ 绘图” 处理类、以及处理 WM_PAINT 消息的“ 重画”
//处理函数 MyPaintProFunc。利用自定义的 Line、Rectang、Circle 类(类型),
//在平面上用直线、矩形和圆画出一个粗略的汽车图形。
//自定义 Line、Rectang、Circle 类(类型)

class Line //自定义类 Line(类型)
{
    int startx,starty,endx,endy; //用于保存线的始点及终点坐标
public:
    Line(int sx,int sy,int ex,int ey) //构造函数
    {
        startx=sx;
        starty=sy;
        endx=ex;
        endy=ey;
    }
    void draw(HWND hWnd); //函数 draw, 负责在与 hWnd 关联的窗口中画出 Line(对象)
};

void Line::draw( HWND hWnd) //Line 类的成员函数 draw
{
    //Line 类的 draw, 负责在与 hWnd 关联的窗口中画出 Line(对象)
    HDC hdc=GetWindowDC(hWnd); //获取设备描述表 DC
    HPEN myPen; //说明画笔对象
    myPen=CreatePen( PS_SOLID, 2, RGB(0,0,0) ); //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );

```

```

        //添加画笔对象(到设备描述表 DC 中)
        MoveToEx(hdc,startx,starty,NULL);    //移至线始点处
        LineTo(hdc,endx,endy);              //画至线终点处(使用新设置的画笔属性)
        // Restore the old pen to the device context
        (HPEN)SelectObject( hdc,pOldPen );    //恢复原画笔(各属性)
        ReleaseDC(NULL,hdc);                //释放所获取到的那一设备描述表
    }

class Rectang                                //自定义类 Rectang(类型)
{
    int uplx,uply,lowrx,lowry;              //矩形左上角及右下角的点坐标
public:
    Rectang(int ulx,int uly,int lrx,int lry)
    {
        uplx=ulx;
        uply=uly;
        lowrx=lrx;
        lowry=lry;
    }
    void draw( HWND hWnd);                  //函数 draw，负责画出 Rectang(对象)
};

void Rectang::draw( HWND hWnd)              //Rectang 类的 draw，负责画出 Rectang(对象)
{
    HDC hdc=GetWindowDC(hWnd);
    HPEN myPen;
    myPen=CreatePen( PS_SOLID, 2, RGB(255,0,0) );    //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
    POINT p4[5];                                    //将矩形按多边形处理
    p4[0].x=uplx;  p4[0].y=uply;
    p4[1].x=lowrx;  p4[1].y=uply;
    p4[2].x=lowrx;  p4[2].y=lowry;
    p4[3].x=uplx;  p4[3].y=lowry;
    p4[4].x=uplx;  p4[4].y=uply;
    Polyline(hdc,p4,5);                            //画出该多边形
    // Restore the old pen to the device context
    (HPEN)SelectObject( hdc,pOldPen );
    ReleaseDC(NULL,hdc);
};

```

```

class Circle                                //自定义类 Circle(类型)
{
    int centx,centy,radius;                //保存圆的圆心坐标及半径
public:
    Circle(int cx,int cy,int r)
    {
        centx=cx;
        centy=cy;
        radius=r;
    }
    void draw( HWND hWnd);                 //函数 draw，负责画出 Circle(对象)
};

void Circle::draw( HWND hWnd)              //Circle 类的 draw，负责画出 Circle(对象)
{
    HDC hdc=GetWindowDC(hWnd);
    HPEN myPen;
    myPen=CreatePen( PS_SOLID, 2, RGB(0,0,255) );           //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
    Ellipse(hdc,centx-radius,centy-radius,centx+radius,centy+radius); //画出该圆
    // Restore the old pen to the device context
    (HPEN)SelectObject( hdc,pOldPen );
    ReleaseDC(NULL,hdc);
}

//本例处理 WM_PAINT 消息的“重画”处理函数 MyPaintProFunc
void MyPaintProFunc(HWND hWnd, HDC hdc, RECT rt)
{
    short ulx=100,uly=70;
    //对象说明部分 -- 共说明了两个矩形、两个圆、五条线(表示一个粗略的汽车图形)
    Rectang rec1(ulx+200,uly+0,ulx+240,uly+30);
    Line lin1(ulx+200,uly+30,ulx+200,uly+80);
    Rectang rec2(ulx+0,uly+80,ulx+400,uly+260);
    Circle cir1(ulx+100,uly+300,40);
    Circle cir2(ulx+300,uly+300,40);
    Line lin2(ulx+100,uly+260,ulx+100,uly+340);
    Line lin3(ulx+60,uly+300,ulx+140,uly+300);
    Line lin4(ulx+300,uly+260,ulx+300,uly+340);
}

```

```
Line lin5(ulx+260,uly+300,ulx+340,uly+300);  
//将上述 9 个图元(表示一个粗略的汽车图形)画出来  
rec1.draw(hWnd);  
lin1.draw(hWnd);  
rec2.draw(hWnd);  
cir1.draw(hWnd);  
cir2.draw(hWnd);  
lin2.draw(hWnd);  
lin3.draw(hWnd);  
lin4.draw(hWnd);  
lin5.draw(hWnd);  
}
```

[输出结果]

为一个粗略的汽车图形轮廓线，略。

7.10 含有静态数据成员及静态函数成员的定义类

读下述程序，写出执行结果(自定义类中含有静态数据及静态函数成员)。

```
#include <iostream.h>  
class Point  
{  
public:  
    double x,y;  
    static int count; //注 1  
    Point(double a=0, double b=0) //注 2  
    {  
        x=a;    y=b;  
        count++; //注 3  
    }  
    static void staticFunc( ) //注 4  
    {  
        cout<<"current_count="<<count<<endl;  
    }  
};  
  
int Point::count=0; //注 5
```



```
void main( )
{
    Point A(2.2, 5.5), *p; //注 6
    cout<<"Point A: ("<<A.x<< ", "<<A.y<<")\n";
    A.staticFunc( ); //注 7
    cout<<"A.count="<<A.count<<endl;
    Point B; //注 8
    cout<<"Point B: ("<<B.x<< ", "<<B.y<<")\n";
    Point::staticFunc( ); //注 9
    cout<<"Point::count="<<Point::count<<endl;
    p=new Point(12.1, 27); //注 10
    cout<<"Point *p: ("<<p->x<< ", "<<p->y<<")\n";
    Point::staticFunc( ); //注 11
    cout<<"Point::count="<<Point::count<<endl;
}
```

[分析提示]

(1) 类的静态数据成员是指由关键字 `static` 修饰的类中的数据成员。类的静态数据成员为该类的所有对象所共享。对类的静态数据成员的访问通常使用“<类名>::<静态数据成员名>”的方式。另外注意：在类中说明的静态数据成员属于引用性说明，即是说，还必须在类外文件作用域中的某个地方对静态数据成员按如下格式进行定义性说明(且仅能说明一次)：

<类型> <类名>::<静态数据成员> = <初值>;

(2) 类的静态函数成员是指由关键字 `static` 修饰的类中的函数成员。对类的静态函数成员的引用通常使用“<类名>::<静态函数成员调用>”的方式。类的静态函数成员没有 `this` 指针，从而无法处理不同调用者对象的各自数据成员值。通常情况下，类的静态函数只处理类的静态数据成员值。若要访问类中的非静态成员时，必须借助对象名或指向对象的指针。

[解答程序]

具体程序见题目处。

注 1：公有的静态数据成员 -- 对所有的类对象来说，只有一份拷贝，所有对象共享。用于记录已通过构造函数生成了多少个对象。

注 2：构造函数，提供了参数 `a`、`b` 的隐含值(默认值)。

注 3：每生成一个对象，静态数据成员 `count` 增加 1。

注 4：静态函数成员，输出静态数据成员 `count` 的当前值。既可使用对象名调用静态函数成员，也可使用类名调用静态函数成员。

注 5：必须在类外使用类名初始化静态数据成员。

注 6：说明对象 `A`，将调用构造函数一次。

- 注 7: 可使用对象名调用静态函数成员。
注 8: 说明对象 B, 将再一次调用构造函数。
注 9: 通常使用类名调用静态函数成员。
注 10: 说明动态对象 *p, 将第三次调用构造函数。
注 11: 使用类名调用静态函数成员。

[输出结果]

```
Point A: (2.2, 5.5)
current_count=1
A.count=1__
Point B: (0, 0)
current_count=2
Point::count=2
Point *p: (12.1, 27)
current_count=3
Point::count=3
```

7.11 自定义类中含有常量数据成员

读下述程序, 写出执行结果(自定义类中含有常量数据成员)。

```
#include<iostream.h>
class ClaA
{
    const int const_v;                //注 1
    static const int static_const_v;  //注 2
public:
    ClaA(int i);
    void print( );                    //注 3
};

const int ClaA::static_const_v=8;    //注 4

ClaA::ClaA(int i):const_v(i)         //注 5
{
}

void ClaA::print( )
{
```

```
        cout<<"const_v = "<<const_v<<endl;
        cout<<"static_const_v = "<<static_const_v<<endl;
    }

    void main( )
    {
        ClaA obj1(123), obj2(66);                //注 6
        obj1.print( );
        obj2.print( );
    }
```

[分析提示]

类的常量数据成员是指由关键字 `const` 修饰的类中的数据成员。类的常量数据成员必须进行初始化，而且只能通过构造函数的成员初始化列表的方式来进行。由于经过构造函数可对常量数据成员进行初始化，从而可在说明对象时给常量数据成员赋初值。一旦对象被创建，其常量数据成员的值就不允许被修改，任何类内外函数只可读取其值，但不可改变它。

[解答程序]

具体程序见题目处。

注 1：常量数据成员。

注 2：静态常量数据成员。

注 3：显示出成员值的成员函数。

注 4：静态常量数据成员在类外说明和初始化。

注 5：常量数据成员只能通过初始化列表来获得初值。

注 6：建立对象 `obj1` 和 `obj2`，并以 123 和 66 作为初值分别调用构造函数，通过构造函数的初始化列表给对象的常量数据成员 `const_v` 赋初值。

[输出结果]

```
const_v = 123
static_const_v = 8
const_v = 66
static_const_v = 8
```

7.12 友元函数的使用

读下述使用了友元函数的程序，写出执行结果(假设执行时输入了：2 4 6 8 10 五个数据)。

```
#include<iostream.h>

class myCla                                     //注 1
{
    double dat;
public:
    myCla(double d=0) { dat=d; }
    friend void inDat(myCla& d) { cin>>d.dat; }
    friend myCla addDat(myCla d1, myCla d2);
    friend myCla subDat(myCla d1, myCla d2);
    friend myCla mulDat(myCla d1, myCla d2);
    friend void outDat(myCla* pd) { cout<<"d="<<pd->dat<<" "; }
};

myCla addDat(myCla d1, myCla d2)               //注 2
{
    myCla d;
    d.dat=d1.dat+d2.dat;
    return d;
}

myCla subDat(myCla d1, myCla d2)
{
    myCla d;
    d.dat=d1.dat-d2.dat;
    return d;
}

myCla mulDat(myCla d1, myCla d2)
{
    myCla temp;
    temp.dat=d1.dat*d2.dat;
    return temp;
}

void main(void)
{
    myCla obj1(5), dArr[5];                    //注 3
    cout<<"Input 5 myCla(5 integers):";
    for(int i=0; i<5; i++)                     //注 4
```

```

        inDat(dArr[i]);
myCla resA[5], resS[5], resM[5], *pa=resA, *ps=resS, *pm=resM; //注 5
for(i=0; i<5; i++) //注 6
{
    resA[i]=addDat(obj1, dArr[i]);
    resS[i]=subDat(obj1, dArr[i]);
    resM[i]=mulDat(obj1, dArr[i]);
}
while (pa<=&resA[4]) //注 7
    outDat(pa++);
cout<<endl;
while (ps<=&resS[4])
    outDat (ps++);
cout<<endl;
while (pm<=&resM[4])
    outDat(pm++);
cout<<endl;
}

```

[分析提示]

(1) 本范例与范例 7.5 实现的是同一个问题,但本例使用了多个友元函数去取代类成员函数。

(2) 注意,友元函数内(定义处)可处理与使用自定义类类型 myCla 的私有成员,但参加运算的所有运算分量必须显式地列在友元函数的参数表中(由于友元函数中没有 this 指针,从而没有当前调用者对象的概念,这一点与类成员函数是不同的)。另外,友元函数并非类成员,在类外说明(定义)友元函数时不能进行类限定——“myCla::”;对友元函数进行调用时也不需要通过类对象。

[解答程序]

具体程序见题目处。

注 1: 自定义类类型 myCla。除构造函数外,还说明了该类的如下五个友元函数。友元函数 inDat: 负责输入 myCla 类对象的有关数据放入引用参数 d 中。友元函数 addDat: 负责实现两个 myCla 类对象的相加运算(注意参数为 myCla 类对象),返回结果类型仍为 myCla。友元函数 subDat: 负责实现两个 myCla 类对象的相减运算。友元函数 mulDat: 负责实现两个 myCla 类对象的相乘运算。友元函数 outDat, 负责输出 myCla 类对象的有关数据(注意参数为指向 myCla 类对象的指针)。

注 2: 下面给出了友元函数 addDat、subDat 以及 mulDat 的具体实现(友元函数并非类成员,在类外定义时不进行类限定)。

注 3: 类对象 obj1, 其初值为 5; 以及具有五个分量的类对象数组 dArr。

注 4: 为类对象数组 dArr 输入数据(五个整数, 数间以空格分割)。

注 5: resA 数组中存放 obj1 对象与 dArr 数组的五个对象相加后的结果; resS 数组中存放 obj1 对象与 dArr 数组的五个对象相减后的结果; resM 数组中存放 obj1 对象与 dArr 数组的五个对象相乘后的结果。

注 6: 执行五次循环, 每次都要调用友元函数来实现对象的加法、减法与乘法运算。但注意: 对友元函数进行调用时并不需要通过类对象 而对类成员函数调用时则必须!。

注 7: 以下程序段输出 resA、resS、resM 各数组中存放的运算结果。

[输出结果]

```
Input 5 myCla(5 integers):2 4 6 8 10
d=7 d=9 d=11 d=13 d=15
d=3 d=1 d=-1 d=-3 d=-5
d=10 d=20 d=30 d=40 d=50
```

7.13 在复数类型 complex 中使用多个友元函数

自定义具有如下框架的一个简单的复数类型 complex, 其中说明了多个友元函数, 使用该类型可以完成复数的输入、加法、减法、乘法、以及对复数的输出。请完成各函数, 并编制主函数, 说明 complex 类对象, 对定义的各函数进行调用。

```
class complex                                //自定义的复数类型 complex
{
    double real;                             //复数实部
    double imag;                             //复数虚部
public:
    complex(double r=0, double i=0);         //构造函数
    friend void inCom(complex& com);          //输入 complex 类对象的有关数据(各分量)
    friend complex addCom(complex c1, complex c2);
                                                //实现复数加法 c1+c2, 返回 complex 类对象
    friend complex subCom(complex c1, complex c2);
                                                //实现复数减法 c1-c2, 返回 complex 类对象
    friend complex mulCom(complex c1, complex c2);
                                                //实现复数乘法 c1*c2, 返回 complex 类对象
    friend void outCom (complex c);          //输出 complex 类对象的有关数据(各分量)
};
```

[分析提示]

(1) 本范例与范例 7.6 实现的是同一个问题, 但本例使用了多个友元函数去取代类成员函数。

(2) 友元函数内(定义处)可处理与使用自定义类类型 `complex` 的私有成员,但参加运算的所有运算分量必须显式地列在友元函数的参数表中(友元函数中没有当前调用者对象的概念)。

[解答程序]

```
#include<iostream.h>

class complex                //自定义的复数类型 complex
{
    ...                      //与题目处相同,略
};

complex::complex(double r, double i) //构造函数, 为对象的私有数据成员赋初值
{
    real=r;          imag=i;
}

void inCom (complex& com)        //输入 complex 类对象的有关数据(各分量)
{
    cin>>com.real>>com.imag;
}

complex addCom(complex c1, complex c2) //实现复数加法 c1+c2,
                                        //返回 complex 类对象
{
    complex c;
    c.real=c1.real+c2.real;
    c.imag=c1.imag+c2.imag;
    return c;
}

complex subCom(complex c1, complex c2) //实现复数减法 c1-c2,
                                        //返回 complex 类对象
{
    complex c;
    c.real=c1.real-c2.real;
    c.imag=c1.imag-c2.imag;
    return c;
}
```

```

complex mulCom(complex c1, complex c2) //实现复数乘法 c1*c2,
                                     //返回 complex 类对象
{
    complex temp;
    temp.real=(c1.real*c2.real)-(c1.imag*c2.imag);
    temp.imag=(c1.real*c2.imag)+(c1.imag*c2.real);
    return temp;
}

void outCom (complex com)           //输出 complex 类对象的有关数据(各分量)
{
    cout<<"("<<com.real<<", "<<com.imag<<")";
}

void main(void)
{
    complex c1, c2, res;             //说明类对象 c1, c2, res
    cout<<"Input complex1 ( 2 integers ):";
    inCom (c1);                      //为类对象 c1 输入数据
    cout<<"Input complex2 ( 2 integers ):";
    inCom (c2);                      //为类对象 c2 输入数据
    res=addCom(c1, c2); //调用 addCom 实现复数加法, 实参为类对象 c1 与 c2
    outCom(c1);                      //输出类对象 c1
    cout<<" + ";
    outCom(c2);                      //输出类对象 c2
    cout<<" = ";
    outCom(res);                     //输出类对象 res
    cout<<"\n-----"<<endl;
    res=subCom(c1, c2); //调用 subCom 实现复数减法, 实参为类对象 c1 与 c2
    outCom(c1);                      //输出类对象 c1
    cout<<" - ";
    outCom(c2);                      //输出类对象 c2
    cout<<" = ";
    outCom (res);                    //输出类对象 res
    cout<<"\n-----"<<endl;
    res=mulCom(c1, c2); //调用 mulCom 实现复数乘法, 实参为类对象 c1 与 c2
    outCom(c1);                      //输出类对象 c1
    cout<<" * ";
    outCom(c2);                      //输出类对象 c2
}

```



```

        cout<<" = ";
        outCom (res);           //输出类对象 res
        cout<<"\n-----"<<endl;
    }

```

[输出结果]

```

Input complex1 ( 2 integers ):1 1
Input complex2 ( 2 integers ):2 3
(1, 1) + (2, 3) = (3, 4)
-----
(1, 1) - (2, 3) = (-1, -2)
-----
(1, 1) * (2, 3) = (-1, 5)
-----

```

7.14 简单的分数类(类型)fraction

自定义如下形式的一个简单的分数类类型 fraction，其中说明了多个类成员函数，使用该类型可以完成分数的输入、分数的加减乘除二目运算和一目减运算、分数的倒数运算、对两个分数进行六种比较运算、以及对分数进行输出等操作。请完成各函数，并编制主函数，说明 fraction 类对象，对定义的各函数进行调用，以验证它们的正确性。

下面给出 fraction 的类定义，并给出一个示意性的主函数以及程序执行结果样本。

```

class fraction           //自定义的分数类型 fraction
{
    int above;           //分子 above
    int below;           //分母 below
    void reduction( );   //私有成员函数 reduction，负责对分数进行约分
    void makeCommonD(fraction&);
                        //私有成员函数 makeCommonD，负责对两个分数进行通分
public:
    fraction(int,int);    //二参构造函数，带来分子分母
    fraction(int);        //一参构造函数，带来分子，分母隐含为 1
    fraction( );          //无参构造函数，隐含分子为 0，分母为 1
    fraction add(fraction); //两个分数相加
    fraction sub(fraction); //两个分数相减
    fraction mul(fraction); //两个分数相乘
    fraction div(fraction); //两个分数相除
    fraction sub( );      //一目减运算

```

```

        fraction reciprocal( );           //求倒数
        bool equal(fraction);             //等于运算
        bool notEqual(fraction);          //不等于运算
        bool greaterThan(fraction);       //大于运算
        bool lessThan(fraction);          //小于运算
        bool greaterOrEqual(fraction);    //大于等于运算
        bool lessOrEqual(fraction);       //小于等于运算
        void display( );                  //显示对象数据
        void input( );                   //输入对象数据
    };

    void main( )                          //主函数 main
    {
        //说明 fraction 类对象，对定义的各函数进行调用，以验证它们的正确性
        fraction f1(-2,-3),f2(-2,3),f3(3,-5),f4,f5(6);
        cout<<"f1=";    f1.display( );
        cout<<"f2=";    f2.display( );
        cout<<"f3=";    f3.display( );
        cout<<"f4=";    f4.display( );
        cout<<"f5=";    f5.display( );
        if(f1.greaterThan(f2)) cout<<"f1>f2"<<endl;
        if(f2.greaterThan(f3)) cout<<"f2>f3"<<endl;
        if(f1.equal(f1)) cout<<"f1==f1"<<endl;
        if(f4.notEqual(f5)) cout<<"f4!=f5"<<endl;
        cout<<"f4=? ";  f4.input( );
        cout<<"f4=";    f4.display( );
        f4=f4.reciprocal( );    cout<<"f4=f4.reciprocal( ); f4=>";    f4.display( );
        f1=f2.add(f4);    cout<<"f1=f2+f4; f1=>";    f1.display( );
        f1=f2.sub(f4);    cout<<"f1=f2-f4; f1=>";    f1.display( );
        f1=f2.mul(f4);    cout<<"f1=f2*f4; f1=>";    f1.display( );
        f1=f2.div(f4);    cout<<"f1=f2/f4; f1=>";    f1.display( );
        f1=f4.sub( );      cout<<"f1=-f4; f1=>";    f1.display( );
        f5=(f1.add(f2)).sub(f3);    cout<<"f5=f1+f2-f3; f5=>";    f5.display( );
    }

```

程序执行后，屏幕显示结果为：

f1=2/3

f2=-2/3

f3=-3/5

```

f4=0
f5=6
f1>f2
f1==f1
f4!=f5
f4=? (above below):3 -6
f4=-1/2
f4=f4.reciprocal(); f4=>-2
f1=f2+f4; f1=>-8/3
f1=f2-f4; f1=>4/3
f1=f2*f4; f1=>4/3
f1=f2/f4; f1=>1/3
f1=-f4; f1=>2
f5=f1+f2-f3; f5=>29/15

```

[分析提示]

(1) 题目处已通过“`class fraction { ... }`”的方式对所需的分数类 `fraction` 进行了定义，并设定了各成员函数的参数界面以及要实现的功能，用户只需编制出各成员函数的类外实现部分即可。之所以将负责对分数进行约分以及通分的两个函数 `reduction` 和 `makeCommonD` 设计为私有成员函数，是因为只允许在类内对它们进行使用而禁止类外调用它们(注意，本类提供的所有运算执行完后，都对运算结果进行了约分)。

(2) 负责对分数进行约分的私有成员函数 `reduction` 中，首先求出分子分母的最大公约数 `c`，而后分子分母同时除以 `c` 而完成对分数的约分功能。

[解答程序]

```

#include <iostream.h>
#include <math.h>
#include <process.h>

class fraction                                //自定义的分数类型 fraction
{
    ...                                       //与题目处相同，略
};

fraction::fraction(int a, int b)             //二参构造函数，带来分子分母
{
    above=a;        below=b;
}

fraction::fraction(int a)                   //一参构造函数，带来分子，分母隐含为 1

```

```

{
    above=a;        below=1;
}

fraction::fraction( )           //无参构造函数，隐含分子为 0，分母为 1
{
    above=0;        below=1;
}

void fraction::makeCommonD(fraction& b)
{
    //私有成员函数 makeCommonD，负责对两个分数进行通分
    //二分数进行通分，且保障通分后两个分数的分母均为正数
    (*this).reduction( );       //第一分数约分(经约分后的分母总被化成了正数)
    b.reduction( );             //第二分数约分(经约分后的分母总被化成了正数)
    above*=b.below;            //改变调用者对象的分子
    b.above*=below;            //改变参数对象的分子
    int temp=b.below*below;
    b.below=below=temp;        //改变两个对象的分母为同一个正数
}

void fraction::reduction( )     //私有成员函数 reduction，负责对分数进行约分
{
    //先求出分子分母的最大公约数 c，而后分子分母同时除以 c
    //且保障约分后两个分数的分母均为正数
    if(below<0) //为保证负号在分子前，且保证分子分母都为负数时消去负号
    {
        //即是说，经约分后的分母总被化成了正数
        above=-above;
        below=-below;
    }
    //先求出分子分母的最大公约数 c
    int a=abs(above), b=abs(below), c, m, n; //说明五个 int 型变量
    m=a; n=b;                                //m 与 n 的初值为 a 和 b
    while (m%n)                               //当 m%n 不等于 0 时继续循环
    {
        int tmp=m;
        m=n;      n=tmp%n;
        //注意，若 m%n 不等于 0 时，最大公约数 gcd(m,n)=gcd(n, m%n)
    }
}

```

```

        //即是说, 将 n 当作新 m, 将 m%n 当作新的 n 后, 接着去判断
        //m%n 是否等于 0
    }
    c=n;           //此时 m%n 等于 0, 所求最大公约数 c 等于 n
    above/=c;      //而后分子除以 c
    below/=c;      //分母除以 c
}

fraction fraction::reciprocal() //求倒数, 分子分母互换后再约分
{
    fraction r;           //临时对象 r
    r.above=below;        //分子为原分母
    r.below=above;        //分母为原分子
    r.reduction();         //对分数进行约分
    return r;             //返回 r
}

fraction fraction::add(fraction b) //两个分数相加
{
    fraction r;           //临时对象 r
    makeCommonD(b);        //先将两个分数通分
    r.above=b.above+above; //再分子相加
    r.below=b.below;       //分母变为同分母
    r.reduction();         //对分数进行约分
    return r;             //返回 r
}

fraction fraction::sub(fraction b) //两个分数相减
{
    fraction r;
    makeCommonD(b);        //先将两个分数通分
    r.above=above-b.above; //再分子相减
    r.below=b.below;       //分母变为同分母
    r.reduction();         //对分数进行约分
    return r;
}

fraction fraction::mul(fraction b) //两个分数相乘
{

```

```

        fraction r;
        r.above=b.above*above;    //分子乘分子
        r.below=b.below*below;    //分母乘分母
        r.reduction( );           //对分数进行约分
        return r;
    }

fraction fraction::div(fraction b) //两分数相除
{
    fraction r;
    if(b.below==0)                //若分母为 0 时提示出错并按隐含值 0 处理
    {
        cout<<"ERROR! below=0! set default: 0/1"<<endl;
        r.above=0;    r.below=1;
    }
    else                          //若分母非 0 时
    {
        r=b.reciprocal( );        //将 b 求倒数
        r=r.mul(*this);           //调用者对象乘以 b 的倒数
    }
    r.reduction( );               //对分数进行约分
    return r;
}

fraction fraction::sub( )         //一目减运算
{
    fraction r;
    r.above=-above;               //将分子的符号取反
    r.below=below;                //分母不改变
    r.reduction( );               //对分数进行约分
    return r;
}

bool fraction::equal(fraction b) //等于运算
{
    (*this).makeCommonD(b); //先通分且保障通分后两个分数的分母均为正数
    return(above==b.above);    //结果转化为分子等于运算
}

```

```
bool fraction::notEqual(fraction b)    //不等于运算
{
    (*this).makeCommonD(b);           //先通分且保障通分后两个分数的分母均为正数
    return(above!=b.above);            //结果转化为分子不等于运算
}

bool fraction::greaterThan(fraction b) //大于运算
{
    (*this).makeCommonD(b);           //先通分且保障通分后两个分数的分母均为正数
    return(above>b.above);             //结果转化为分子大于运算
}

bool fraction::lessThan(fraction b)    //小于运算
{
    (*this).makeCommonD(b);           //先通分且保障通分后两个分数的分母均为正数
    return(above<b.above);            //结果转化为分子小于运算
}

bool fraction::greaterOrEqual(fraction b) //大于等于运算
{
    (*this).makeCommonD(b);           //先通分且保障通分后两个分数的分母均为正数
    return(above>=b.above);           //结果转化为分子大于等于运算
}

bool fraction::lessOrEqual(fraction b) //小于等于运算
{
    (*this).makeCommonD(b);           //先通分且保障通分后两个分数的分母均为正数
    return(above<=b.above);           //结果转化为分子小于等于运算
}

void fraction::display( )              //显示对象数据
{
    this->reduction( );
    //保障通过构造函数形成的对象也进行了约分(其他从输入到运算都已随时约分过了)
    if(below==1||above==0)             //对整数及 0 的处理
        cout<<above<<endl;
    else
        //既有分子又有分母
    {
        cout<<above<<"/";
    }
}
```

```

        cout<<below<<endl;
    }
}

void fraction::input()           //输入对象数据
{
    cout<<"(above below):";     //提示输入
    cin>>above>>below;
    if(below==0)                 //若输入的分母为 0，提示出错信息
    {
        cout<<"ERROR! below=0! set default: 0/1"<<endl;
        above=0;        below=1;
    }
    if(below<0)
    {
        //为保证负号在分子前，且保证分子分母都为负数时消去负号
        above=-above;
        below=-below;
    }
}

void main()                     //主函数 main
{
    //说明 fraction 类对象，对定义的各函数进行调用，以验证它们的正确性
    ...                          //与题目处相同，略
}

```

[输出结果]

与题目处相同，略。

7.15 “后缀式简单单步计算器”类(类型)Calculator

自定义一个 Calculator 类(类型)，即“后缀式简单单步计算器”类型。它具有如下一些功能：

(1) 从键盘输入欲处理的后缀式表达式 str(为一个字符串)，或通过一参构造函数设置该表达式。

(2) 表达式中可含有加、减、乘、除、乘方、等号运算符(“ +”、“ -”、“ *”、“ /”、“ ^”、“ =”)以及实型数据的运算对象，表达式不限长度，遇等号结束表达式并计算出

结果显示出来。

(3) 运算符没有优先级,即是说,总按“ 后缀式简单单步”形式执行。如:“ $1\ 3\ +\ 5\ *\ 2\ ^\ =$ ”,意味着先取运算对象 1 与 3 进行后缀的加法运算得结果 4,而后再将 4 与后面的 5 进行后缀的乘法运算得结果 20,而后再将 20 与后面的 2 进行后缀的乘方运算得结果 400,再往后遇到等号时将运算结果 400 输出来而结束该表达式的处理。

(4) 具有一定的排错功能,当用户输入错误的表达式时提示用户。

例如,程序执行后的输入输出对应关系举例如下:

输入: $1\ 2\ +\ =$	输出: 3	对应于: $1+2$
输入: $2\ 5\ +\ 3\ *\ =$	输出: 21	对应于: $(2+5)*3$
输入: $1.23\ 4.5\ -\ =$	输出: -3.27	对应于: $1.23-4.5$
输入: $1.1+2.2+\ -3.3\ *\ =$	输出: -10.89	对应于: $(1.1+(2.2))*(-3.3)$
输入: $1\ 3\ +\ 5\ *\ 2\ ^\ =$	输出: 400	对应于: $((1+3)*5)^2$
输入: $-1.23\ 3\ ^\ 100\ +\ =$	输出: 98.1391	对应于: $(-1.23^3)+100$
输入: $72.9\ -3\ /\ 2.2\ *\ 60\ +\ =$	输出: 6.54	对应于: $(72.9/(-3))*2.2+60$

下面给出实现方式之一的程序核心代码。

```
const int MaxStackSize = 100;           //运算对象栈的大小,即栈中元素的最大个数
class Stack                             //类 Stack 的定义(用于存放运算对象)
{
private:
    double stacklist[MaxStackSize];    //数组,用于存放栈的元素(运算对象)
    int top;                            //栈顶位置(数组下标)
public:
    Stack (void);                       //构造函数,初始化栈
    void Push (double item);            //将元素 item 压入栈
    double Pop (void);                  //将栈顶元素弹出栈并返回该元素
    bool StackEmpty(void);              //测试栈是否为空
    bool StackFull(void);               //测试栈是否已满
};

class Calculator //自定义的 Calculator 类(类型),即“ 后缀式简单单步计算器”类型
{
    char str[100];                       //私有数据成员 str 中存放欲计算的表达式
    int ind;                             //表达式已被处理到的当前位置(下标值)
    Stack S;                             //运算对象栈 S
    void PushNum(double num);            //将运算对象 num 压入栈
    bool Get2Operands(double& oprd1, double& oprd2);
        //连续从栈中弹出两个运算对象,放在 oprd1 和 oprd2 中
    void Compu(char op); //执行由运算符 op 指定的运算,并将结果存入栈 S 中
public:
```

```

Calculator( ); //无参构造函数
Calculator(char * s); //一参构造函数(带来欲计算的表达式 s)
void setStr(char * s); //设置欲计算的表达式 s
void printExp( ); //显示出当前欲计算的表达式
bool myInFromStr(char* str, float& x, char& ch, char& f_OR_c);
/*
    从欲处理表达式 str 的当前位置 ind 开始, 拼装出一个运算对象 x 或一个运算符
    ch。若拼装的是运算对象 x 时, 通过引用参数 f_OR_c 返回“ f”, 否则返回“ c” (为运
    算符时)。拼装过程中可以调用标准函数 atof 来转换形成运算对象 x, 也可靠程序本身
    来形成 x。
*/
void Read_processing(void);
/*
    对欲计算的后缀表达式 str 进行从头到尾的处理(直至输出运算结果而结束)。具体
    地说, 通过调用 myInFromStr 从欲处理的表达式 str 处获取运算对象与运算符, 每遇运
    算对象时, 总先将其放入运算对象栈 S, 一旦遇运算符时, 从运算对象栈 S 中取出两个
    运算对象进行指定的运算, 并立刻将结果又放入栈 S 中, 如此重复, 直到遇见等号(“ =” )
    时输出表达式结果而结束。
*/
};

void main( )
{
    //主函数, 说明 Calculator 类对象, 并通过对象调用 Read_processing 处理其后缀表达式
    Calculator Obj1("1 2 + =");
    Obj1.printExp( ); Obj1.Read_processing( );
    //输出结果: 1 2 + = --> 3
    Calculator Obj2("2 5 + 3 * =");
    Obj2.printExp( ); Obj2.Read_processing( );
    //输出: 2 5 + 3 * = --> 21
    Calculator Obj3("1.23 4.5 - =");
    Obj3.printExp( ); Obj3.Read_processing( );
    //输出: 1.23 4.5 - = --> -3.27
    Calculator Obj4("1.1 +2.2 + -3.3 * =");
    Obj4.printExp( ); Obj4.Read_processing( );
    //输出: 1.1 +2.2 + -3.3 * = --> -10.89
    Calculator Obj5("-1.23 3 ^ 100 + =");
    Obj5.printExp( ); Obj5.Read_processing( );
    //输出: -1.23 3 ^ 100 + = --> 98.1391

```

```

Calculator Obj6("72.9 - 3 / 2.2 * 60 + =");
Obj6.printExp(); Obj6.Read_processing();
//输出: 72.9 - 3 / 2.2 * 60 + = --> 6.54

Calculator Obj7; char str0[100];
cout<<"Input EXPRESSION (ending with '='), if only 'ENTER' then ending
program."<<endl;
cin.getline(str0, 100);
//从键盘输入欲计算的任意表达式(为一个符号串)放入 str0
while(strlen(str0)>0) //当输入串非空时继续处理(入空串时循环结束)
{
    Obj7.setStr(str0); //将 str0 设置为欲处理的表达式
    Obj7.Read_processing(); //计算表达式并输出结果
    cout<<"Input EXPRESSION (ending with '='), if only 'ENTER' then ending
program."<<endl;
    cin.getline(str0, 100);
    //从键盘输入下一个欲计算的任意表达式(符号串)放入 str0
}
}

```

[分析提示]

本示例所处理的表达式中, 运算符没有优先级, 即是说, 总按“ 后缀式简单单步”形式执行。如, “ 2 4 + 5 * =”, 意味着先取运算对象 2 与 4 进行后缀的加法运算得结果 6, 而后再将 6 与后面的 5 进行后缀的乘法运算得结果 30, 再往后遇到等号时将运算结果 30 输出出来而结束该表达式的处理。主要处理过程靠 Calculator 的类成员函数 Read_processing 来完成, 由它负责对欲计算的后缀表达式 str(在 Calculator 类对象的私有数据成员 str 中存放欲计算的表达式) 进行从头到尾的处理(直至输出运算结果而结束)。具体地说, Read_processing 首先通过调用另一个类成员函数 myInFromStr 从欲处理的表达式 str 处获取运算对象与运算符, 每遇运算对象时, 总先将其放入运算对象栈 S (该 S 为 Stack 类型的一个对象, 它为 Calculator 类中的一个私有数据成员), 一旦遇运算符时, 从运算对象栈 S 中取出两个运算对象进行指定的运算, 并立刻将结果又放入栈 S 中, 如此重复, 直到遇见等号(“ =”) 时输出表达式结果而结束。

[解答程序]

```

#include <iostream.h>
#include <math.h>
#include <string.h>
#include <process.h>
const int MaxStackSize = 100; //运算对象栈的大小, 即栈中元素的最大个数

```

```
class Stack                                //类 Stack 的定义
{
    ...                                    //与题目处相同,
};

                                           //Stack 类各成员函数的类外实现
Stack::Stack (void)                       //构造函数, 栈顶位置初始化为-1
{
    top=-1;
}

void Stack::Push (double item)            //将元素 item 压入栈
{
    if (top == MaxStackSize-1)            //如果栈满, 程序终止
    {
        cerr << "Stack overflow!" << endl;
        exit(1);
    }
    top++;                                //栈顶指针增 1
    stacklist[top] = item;                //将新元素压入栈顶
}

double Stack::Pop (void)                  //将栈顶元素弹出栈并返回该元素
{
    double temp;
    if (top == -1)                        //如果栈空, 程序终止
    {
        cerr << "Attempt to pop an empty stack!" << endl;
        exit(1);
    }
    temp = stacklist[top];                //取出栈顶元素
    top--;                                //栈顶指针减 1
    return temp;                          //返回栈顶元素
}

bool Stack::StackEmpty(void)              //测试是否栈空
{
    return (top==-1);                    //如果栈空则返回 true, 否则返回 false
}
```

```

bool Stack::StackFull(void){           //测试是否栈满
    return (top == MaxStackSize-1);    //如果栈满则返回 true, 否则返回 false
}

class Calculator //自定义的 Calculator 类(类型), 即“ 后缀式简单单步计算器” 类型
{
    ...                                //与题目处相同,
};

Calculator::Calculator( )               //无参构造函数
{
    strcpy(str, "0 =");                //隐含表达式为“ 0 =”
    ind=0;                             //表达式已被处理到的当前位置为 0(下标值)
}

Calculator::Calculator(char * s)        //一参构造函数(带来欲计算的表达式 s)
{
    strcpy(str, s);                    //将欲计算的表达式 s 拷贝到私有数据成员 str 中
    ind=0;                             //表达式已被处理到的当前位置为 0(下标值)
}

void Calculator::setStr(char * s)       //设置欲计算的表达式 s
{
    strcpy(str, s);                    //将欲计算的表达式 s 拷贝到私有数据成员 str 中
    ind=0;                             //表达式已被处理到的当前位置为 0(下标值)
}

void Calculator::printExp( )            //显示出当前欲计算的表达式
{
    cout<<str;
}

void Calculator::PushNum(double num)    // 将运算对象 num 压入栈 S
{
    S.Push(num);
}

bool Calculator::myInFromStr(char* str, float& x, char& ch, char& f_OR_c)

```

```

{
//从欲处理表达式 str 的当前位置 ind 开始, 拼装出一个运算对象 x 或一个运算符 ch
//若拼装的是运算对象 x 时, 通过引用参数 f_OR_c 返回“ f”, 否则返回“ c”
//(为运算符时)
//本拼装过程使用了系统预定义的标准函数 atof, 靠它来转换形成运算对象 x

    if(ind>=strlen(str))
        return false;

    char cc=str[ind++];           //从 str 的当前位置 ind 开始取字符
    while(cc==' ')                //甩掉空格
        cc=str[ind++];

    char sign='+';                //数的正、负号将记录在 sign 中, 无符号时认为是正数
    if(cc=='+'||cc=='-'||cc=='*'||cc=='/'||cc=='^'||cc=='=') //可能为运算符时
    {
        if( (cc=='+'||cc=='-') && str[ind]!='=') //为数的正、负号(并非运算符)
        {
            sign=cc;                //将数的正、负号记录在 sign 中, 接着去处理数
            cc=str[ind++];
        }
        else                        //为运算符时
        {
            f_OR_c=c';
            //将标志变量 f_OR_c 的值置为“ c”, 表示返回了一个 char 型的
            //运算符 ch
            ch=cc;    return true;    //立即返回
        }
    }

//若非运算符时, 要从 str 的当前位置 ind 开始, 拼装出一个运算对象 x 并返回
f_OR_c=" f"; //将标志变量 f_OR_c 的值置为“ f”, 表示将返回一个 float
//型的运算对象 x

    char s0[20];                //准备存放一个数的连续各字符
    int i=0;
    if(cc<'0' || cc>'9')        //第一个字符应为数字字符
    {
        cout<<endl<<"Wrong_0000!";
        //提示出:“ Wrong!” 字符串, 表示输入的非数字字符

        exit(0);
    }

    while (cc!=' ')                //将一个数的连续各字符放入 s0 数组中
    {

```

```

        s0[i++]=cc;
        cc=str[ind++];
    }
    s0[i]='\0';           //放一个结尾符
    x=atof(s0);           //通过标准函数 atof 来拼装出运算对象 x
    if(sign=='-')         //若有负号
        x=-x;
    return true;
};

```

```

bool Calculator::Get2Operands(double& oprd1, double& oprd2)
{
    //连续将两个运算对象弹出栈 S，放在 oprd1 和 oprd2 中
    //如果栈 S 中没有两个运算对象，则返回 false 并输出相关出错信息
    if (S.StackEmpty())    //如果栈为空，返回 false 并输出相关出错信息
    {
        cout << "Missing operand!" << endl;
        return false;
    }
    oprd1 = S.Pop();       //将右运算对象弹出栈
    if (S.StackEmpty())    //如果栈为空，返回 false 并输出相关出错信息
    {
        cout << "Missing operand!" << endl;
        return false;
    }
    oprd2 = S.Pop();       //将左运算对象弹出栈
    return true;
}

```

```

void Calculator::Compu(char op)
    //执行由运算符 op 指定的运算，并将结果存入栈 S 中
{
    double operd1, operd2;
    bool result = Get2Operands(operd1, operd2);
    // 将两个运算对象弹出栈，放入 operd1 与 operd2 之中
    if (result)
    {
        //如果成功，执行运算并将运算结果压入栈，否则提示出错信息并退出程序
        switch(op)

```

```

{
    case '+':    S.Push(operd2+operd1);          //加后将和入栈
                break;
    case '-':    S.Push(operd2-operd1);          //减后将差入栈
                break;
    case '*':    S.Push(operd2*operd1);          //乘后将积入栈
                break;
    case '/':    if (operd1 == 0)                //当除数为 0 时
                {                               //除数为 0 时，提示出错信息并退出程序
                    cout << "ERROR -- Divided by 0! exited!" << endl;
                    exit(0);
                }
                else
                    S.Push(operd2/operd1);        //除数非 0 时，除后将商入栈
                break;
    case '^':    S.Push(pow(operd2,operd1));
                //计算乘方后将结果入栈，pow(operd2,operd1)计算出 operd2 的 operd1 次方
                break;
}                //switch 句结束
}
else            //运算对象不够，提示出错信息并退出程序
{
    cout << "ERROR -- less operand! exited!" << endl;
    exit(0);
}
}

```

```
void Calculator::Read_processing(void)
```

```
{
```

```
    /*
```

对欲计算的后缀表达式 str 进行从头到尾的处理(直至输出运算结果而结束)。具体地说，通过调用 myInFromStr 从欲处理的表达式 str 处获取运算对象与运算符，每遇运算对象时，总先将其放入运算对象栈 S，一旦遇运算符时，从运算对象栈 S 中取出两个运算对象进行指定的运算，并立刻将结果又放入栈 S 中，如此重复，直到遇见等号(“=”)时输出表达式结果而结束。

```
    */
```

```
    float x;
```

```
    char ch, f_OR_c;
```



```

while( myInFromStr(str,x,ch,f_OR_c) )
{
//从头到尾读入表达式 str 的各语法单位(或为运算对象或为运算符)并加以处理
    if(f_OR_c=='f')                //若读入的是运算对象，则压入栈
        PushNum(x);
    else                            //若读入的是运算符时
        switch(ch)                 //分支进行各种运算
        {
            case '+':
            case '-':
            case '*':
            case '/':
            case '^':
                Compu(ch);          //执行由运算符 ch 指定的运算，并将结果存入栈中
                break;
            case '=':                //遇见等号(“ = ”)时，输出表达式结果
                cout<<"  --> "<<S.Pop( )<<endl;
                break;
            default:
                // 若读入的既非运算对象又非运算符时，提示出错并退出
                cout << "ERROR -- !!!  exited!" << endl;
                exit(0);
        } //switch(ch)
    } //while( myInFromStr(str,x,ch,f_OR_c) )
}

void main( )
{
//主函数，说明 Calculator 类对象，并通过对象调用 Read_processing 处理其后缀
//表达式
    ... //与题目处相同，略
}

```

[输出结果]

```

1 2 += --> 3
2 5 + 3 *= --> 21
1.23 4.5 - = --> -3.27
1.1 +2.2 + -3.3 *= --> -10.89
-1.23 3 ^ 100 += --> 98.1391

```

72.9 - 3 / 2.2 * 60 + = → 6.54

Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.

1 3 + 5 * 2 ^ =

→ 400

Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.

5 - 2.2 - 3 * 10 / =

→ 2.16

Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.

Press any key to continue

7.16 自定义时间类(类型)TimeType

根据如下定义编制一个简单的时间类型 `TimeType`，其中说明了多个类成员函数，使用该类型可以完成任一时刻的输入、对时分秒的增加、计算任意两个时刻的时间间隔、对两个时刻进行六种比较、以及对时刻的输出等操作。请完成各函数，并编制主函数，说明 `TimeType` 类对象，对定义的各函数进行调用，以验证它们的正确性。

```
class TimeType                                //自定义的时间类型 TimeType
{
    int h,m,s;                                //私有数据成员，表示当前时间的时、分、秒
public:
    TimeType(int h0, int m0, int s0);          //三参构造函数，设定时、分、秒
    TimeType();                                //无参构造函数，将时、分、秒设为 0:0:0
    void setTime(int h0, int m0, int s0);      //按指定的时、分、秒来设置时间
    int getHou() { return h; }                 //获取当前时刻的“时”——私有数据 h 之值
    int getMin() { return m; }                 //获取当前时刻的“分”——私有数据 m 之值
    int getSec() { return s; }                 //获取当前时刻的“秒”——私有数据 s 之值
    void incrementSec(int sec);                 //增加若干秒，sec>0
    void incrementMin(int min);                 //增加若干分，min>0
    void incrementHou(int hrs);                 //增加若干小时，hrs>0
    void incrementAll(int hrs, int min, int sec); //增加一定的时、分、秒，sec>=0, min>=0, hrs>=0

    long difs2Time(TimeType t2);                //二时刻的间隔秒数(差的绝对值)
    bool equal(TimeType t2);                    //判断二时刻是否相等
    bool notEqual(TimeType t2);                 //判断二时刻是否不相等
    bool lessThan(TimeType t2);                 //判断时刻 1 是否小于时刻 2
    bool lessOrEqual(TimeType t2);              //判断时刻 1 是否小于等于时刻 2
    bool greaterThan(TimeType t2);              //判断时刻 1 是否大于时刻 2
```

```

bool greaterOrEqual(TimeType t2);    //判断时刻 1 是否大于等于时刻 2
void inputTime( );                  //键盘输入时刻对象的有关数据(时、分、秒)
void printTime( );                  //屏幕输出时刻对象的有关数据(时、分、秒)
void printAmPm( );    //以 Am(上午)或 Pm(下午)的格式在屏幕上输出时间对象
};

```

[分析提示]

(1) 类成员函数内(定义处)可处理与使用本 TimeType 类的私有成员,而且总以当前调用者对象(*this)作为该成员函数的隐式第一运算分量。若所定义的运算多于一个运算对象时,才将其余运算对象显式地列在该成员函数的参数表中。

(2) 对时间进行增加的 incrementSec、incrementMin、incrementHou 以及 incrementAll 成员函数中,要注意所谓的进位处理——超过 60,进位到分;超过 60 分,进位到小时;超过 24 时,甩掉 24 以上的部分等。

(3) 编制主函数,说明 TimeType 类对象,并通过类对象对 TimeType 类的各个成员函数进行各种可能情况的调用,以验证每一成员函数的正确性。

[解答程序]

```

#include<iostream.h>
#include<math.h>
class TimeType                                //自定义的时间类型 TimeType
{
    ...                                       //与题目处相同,略!
};

TimeType::TimeType(int h0, int m0, int s0)    //三参构造函数,设定时、分、秒
{
    //构造函数,为调用者对象的私有数据成员赋初值
    h=h0;s=s0; m=m0;
}

TimeType::TimeType( )                        //无参构造函数,将时、分、秒设为 0:0:0
{
    h=0;    s=0;    m=0;
}

void TimeType::setTime(int h0, int m0, int s0) //按指定的时、分、秒来设置时间
{
    h=h0;s=s0; m=m0;
}

```

```
void TimeType::incrementSec(int sec)    //增加若干秒, sec>0
{
    s+=sec;
    if(s>=60)                          //秒数超过 60, 进位到分
    {
        m+=s/60;    s=s%60;
    }
    if(m>=60)                          //超过 60 分, 进位到时
    {
        h+=m/60;    m=m%60;
    }
    if(h>=24)                          //超过 24 时, 甩掉 24 以上的部分
    {
        h=h%24;
    }
}

void TimeType::incrementMin(int min)    //增加若干分, min>0
{
    m+=min;
    if(m>=60)                          //超过 60 分, 进位到时
    {
        h+=m/60;    m=m%60;
    }
    if(h>=24)                          //超过 24 时, 甩掉 24 以上的部分
    {
        h=h%24;
    }
}

void TimeType::incrementHou(int hrs)    //增加若干小时, hrs>0
{
    h+=hrs;
    if(h>=24) h=h%24;                  //超过 24 时, 甩掉 24 以上的部分
}

void TimeType::incrementAll(int hrs, int min, int sec)
{
    //增加一定的时、分、秒, sec>=0, min>=0, hrs>=0
    (*this).incrementSec(sec);         //调用本类成员函数增加秒
```

```

        (*this).incrementMin(min);           //调用本类成员函数增加分
        (*this).incrementHou(hrs);          //调用本类成员函数增加小时
    }

long TimeType::difs2Time(TimeType t2)      //两个时刻的间隔秒数(差的绝对值)
{
    long m1, m2, difm;
    m1=h*3600+m*60+s;                      //调用者对象的 h:m:s 距 0:0:0 的秒数
    m2=t2.h*3600+t2.m*60+t2.s;             //对象 t2 的 h:m:s 距 0:0:0 的秒数
    difm=abs(m1 - m2);                     //两个时刻的间隔总秒数(差的绝对值)
    return difm;                           //返回间隔总秒数
}

bool TimeType::equal(TimeType t2)          //判断两个时刻是否相等
{
    if(h==t2.h && m==t2.m && s==t2.s)
        return true;
    return false;
}

bool TimeType::notEqual(TimeType t2)       //判断二时刻是否不相等
{
    if(h!=t2.h || m!=t2.m || s!=t2.s)
        return true;
    return false;
}

bool TimeType::lessThan(TimeType t2){      //判断时刻 1 是否小于时刻 2
    if(h<t2.h) return true;                //时刻 1 的 h 小
    if(h>t2.h) return false;
    if(m<t2.m) return true;                //时刻 1 的 m 小
    if(m>t2.m) return false;
    if(s<t2.s) return true;                //时刻 1 的 s 小
    else return false;
}

bool TimeType::lessOrEqual(TimeType t2)    //判断时刻 1 是否小于等于时刻 2
{
    if( (*this).lessThan(t2) || (*this).equal(t2) ) //调用本类成员函数

```

```

        return true;
    else
        return false;
}

bool TimeType::greaterThan(TimeType t2)    //判断时刻 1 是否大于时刻 2
{
    if( !(*this).lessOrEqual(t2) )          //调用本类成员函数
        return true;
    else
        return false;
}

bool TimeType::greaterOrEqual(TimeType t2) //判断时刻 1 是否大于等于时刻 2
{
    if( (*this).greaterThan(t2) || (*this).equal(t2) ) //调用本类成员函数
        return true;
    else
        return false;
}

void TimeType::inputTime( )                //键盘输入时间对象的有关数据(时、分、秒)
{
    cout<<"Input time(h(0~23) m(0~59) s(0~59)).:";
    cin>>h>>m>>s;
}

void TimeType::printTime( )                //屏幕输出时间对象的有关数据(时、分、秒)
{
    cout<<h<<":"<<m<<":"<<s<<endl;
}

void TimeType::printAmPm( )
//以 Am(上午)或 Pm(下午)的格式在屏幕上输出时刻对象
{
    if(h<12)
        cout<<h<<":"<<m<<":"<<s<<"(AM)"<<endl;
    else
        cout<<h-12<<":"<<m<<":"<<s<<"(PM)"<<endl;
}

```

```

    }

    void printDifs(long difs)    //并非类成员函数，只是 main 中使用的函数
    {
        //以“ difs=...s=...m...s=...h...m...s” 的格式输出两个时刻间隔

        long ds, dm, dh;
        cout<<"difs="<<difs<<"s=";
        dm=difs/60;
        ds=difs%60;
        cout<<dm<<"m"<<ds<<"s=";
        dh=dm/60;
        dm=dm%60;
        cout<<dh<<"h"<<dm<<"m"<<ds<<"s"<<endl;
    }

    void main(void)
    {
        TimeType t1, t2(8, 15, 10), t3;    //说明类对象 t1, t2, t3
        cout<<"t1=";
        t1.printTime();    //输出类对象 t1
        cout<<"t2=";
        t2.printTime();    //输出类对象 t2
        long difs=t1.difs2Time(t2);    //t1 与 t2 两个时刻的间隔(差的绝对值)
        printDifs(difs);    //printDifs 并非类成员函数
        cout<<"t1: ";
        t1.inputTime();    //为类对象 t1 输入数据
        cout<<"t2: ";    t2.inputTime();    //为类对象 t2 输入数据
        difs=t1.difs2Time(t2);    //t1 与 t2 两个时刻的间隔(差的绝对值)
        printDifs(difs);    //printDifs 并非类成员函数
        t3.setTime(15, 28, 0);    //为类对象 t3 设置时间
        cout<<"t3=";
        t3.printTime();    //输出类对象 t3
        if(t1.greaterThan(t2))    //t1>t2?
            cout<<"t1>t2"<<endl;
        if(t1.equal(t2))    //t1==t2?
            cout<<"t1==t2"<<endl;
        if(t1.greaterOrEqual(t2))    //t1>=t2?
            cout<<"t1>=t2"<<endl;
        if(t1.lessThan(t2))    //t1<t2?

```

```

        cout<<"t1<t2"<<endl;
    if(t1.lessOrEqual(t2))           //t1<=t2?
        cout<<"t1<=t2"<<endl;
    if(t1.notEqual(t2))             //t1!=t2?
        cout<<"t1!=t2"<<endl;
    t1.incrementSec(40);            //增加若干秒
    cout<<"t1.incrementSec(40);'=> ";
    cout<<"t1=";  t1.printTime( );    //输出类对象 t1
    cout<<"t1=";  t1.printAmPm( );    //以 AM-PM 格式输出类对象 t1
    t2.incrementMin(50);            //增加若干分
    cout<<"t2.incrementMin(50);'=> ";
    cout<<"t2=";  t2.printTime( );    //输出类对象 t2
    cout<<"t2=";  t2.printAmPm( );    //以 AM-PM 格式输出类对象 t2
    t2.incrementHou(11);            //增加若干小时
    cout<<"t2.incrementHou(11);'=> ";
    cout<<"t2=";  t2.printTime( );    //输出类对象 t2
    cout<<"t2=";  t2.printAmPm( );    //以 AM-PM 格式输出类对象 t2
    t2.incrementAll(6, 45, 55);      //增加一定的时、分、秒
    cout<<"t2.incrementAll(6, 45, 55);'=> ";
    cout<<"t2=";  t2.printTime( );    //输出类对象 t2
    cout<<"t2=";  t2.printAmPm( );    //以 AM-PM 格式输出类对象 t2
}

```

[输出结果]

```

t1=0:0:0
t2=8:15:10
difs=29710s=495m10s=8h15m10s
t1: Input time(h(0~23) m(0~59) s(0~59)):11 30 45
t2: Input time(h(0~23) m(0~59) s(0~59)):22 18 30
difs=38865s=647m45s=10h47m45s
t3=15:28:0
t1<t2
t1<=t2
t1!=t2
't1.incrementSec(40);'=> t1=11:31:25
t1=11:31:25(AM)
't2.incrementMin(50);'=> t2=23:8:30
t2=11:8:30(PM)
't2.incrementHou(11);'=> t2=10:8:30

```



```
t2=10:8:30(AM)
t2.incrementAll(6, 45, 55);'=> t2=16:54:25
t2=4:54:25(PM)
```

7.17 在自定义时间类(类型)TimeType 中使用多个友元函数

自定义具有如下核心代码的一个简单的时间类型 TimeType，其中说明了多个友元函数，使用该类型可以完成时刻的输入、对时分秒的增加、计算两个时刻的间隔、对两个时刻进行六种比较、以及对时刻的输出等操作。请完成各函数，并编写主函数，说明 TimeType 类对象，对定义的各函数进行调用，以验证它们的正确性。

```
class TimeType                                //自定义的时间类型 TimeType
{
    int h,m,s;                                //私有数据成员，表示当前时刻的时、分、秒
public:
    TimeType(int h0, int m0, int s0); //三参构造函数，设定时、分、秒
    TimeType();                        //无参构造函数，将时、分、秒设为 0:0:0
    friend void setTime(TimeType& t1, int h0, int m0, int s0);
        //按指定的时、分、秒来设置时间
    friend int getHou(TimeType t1){ return t1.h; }
        //获取当前时间的“ 时 ” ——私有数据 h 之值
    friend int getMin(TimeType t1){ return t1.m; }
        //获取当前时间的“ 分 ” ——数据 m 之值
    friend int getSec(TimeType t1){ return t1.s; }
        //获取当前时间的“ 秒 ” ——私有数据 s 之值
    friend void incrementSec(TimeType& t1, int sec); //增加若干秒， sec>0
    friend void incrementMin(TimeType& t1, int min); //增加若干分， min>0
    friend void incrementHou(TimeType& t1, int hrs); //增加若干小时， hrs>0
    friend void incrementAll(TimeType& t1, int hrs, int min, int sec);
        //增加一定的时、分、秒， sec>=0, min>=0, hrs>=0
    friend long difs2Time(TimeType t1, TimeType t2);
        //二时刻的间隔秒数(差的绝对值)
    friend bool equal(TimeType t1, TimeType t2); //判断两个时刻是否相等
    friend bool notEqual(TimeType t1, TimeType t2); //判断两个时刻是否不相等
    friend bool lessThan(TimeType t1, TimeType t2); //判断时间 1 是否小于时间 2
    friend bool lessOrEqual(TimeType t1, TimeType t2);
        //判断时间 1 是否小于等于时间 2
    friend bool greaterThan(TimeType t1, TimeType t2);
        //判断时间 1 是否大于时间 2
```

```

friend bool greaterOrEqual(TimeType t1, TimeType t2);
    //判断时间 1 是否大于等于时间 2
friend void inputTime(TimeType& t1);
    //键盘输入时间对象的有关数据(时、分、秒)
friend void printTime(TimeType t1);
    //屏幕输出时间对象的有关数据(时、分、秒)
friend void printAmPm(TimeType t1);
    //以 Am(上午)或 Pm(下午)的格式在屏幕上输出时间对象
};

```

[分析提示]

该范例处理与实现的问题与上一范例相同，只是改用了友元函数来替代类成员函数。注意：友元函数内（定义处）可处理与使用自定义类类型 `TimeType` 的私有成员，但参加运算的所有运算分量必须显式地列在友元函数的参数表中。另外对友元函数的调用并不需要通过类对象。

[解答程序]

```

#include<iostream.h>
#include<math.h>
class TimeType                                //自定义的时间类型 TimeType
{
    ...                                        //与题目处相同，略
};

TimeType::TimeType(int h0, int m0, int s0)    //三参构造函数，设定时、分、秒
{
    //构造函数，为调用者对象的私有数据成员赋初值
    h=h0;s=s0; m=m0;
}

TimeType::TimeType( )                        //无参构造函数，将时、分、秒设为 0:0:0
{
    h=0;      s=0;      m=0;
}

void setTime(TimeType& t1, int h0, int m0, int s0)
{
    //按指定的时、分、秒来设置时间
    t1.h=h0;      t1.m=m0;      t1.s=s0;
}

```

```
}

void incrementSec(TimeType& t1, int sec)    //增加若干秒, sec>0
{
    t1.s+=sec;
    if(t1.s>=60)                          //秒数超过 60, 进位到分
    {
        t1.m+=t1.s/60;
        t1.s=t1.s%60;
    }
    if(t1.m>=60)                          //分超过 60 分, 进位到时
    {
        t1.h+=t1.m/60;
        t1.m=t1.m%60;
    }
    if(t1.h>=24)                          //超过 24 时, 甩掉 24 以上的部分
        t1.h=t1.h%24;
}

void incrementMin(TimeType& t1, int min)  //增加若干分, min>0
{
    t1.m+=min;
    if(t1.m>=60)                          //超过 60 分, 进位到时
    {
        t1.h+=t1.m/60;
        t1.m=t1.m%60;
    }
    if(t1.h>=24)                          //超过 24 时, 甩掉 24 以上的部分
        t1.h=t1.h%24;
}

void incrementHou(TimeType& t1, int hrs)  //增加若干小时, hrs>0
{
    t1.h+=hrs;
    if(t1.h>=24)                          //超过 24 时, 甩掉 24 以上的部分
        t1.h=t1.h%24;
}

void incrementAll(TimeType& t1, int hrs, int min, int sec)
```

```

{
    //增加一定的时、分、秒, sec>=0, min>=0, hrs>=0
    incrementSec(t1, sec);           //调用友元函数增加秒
    incrementMin(t1, min);          //调用友元函数增加分
    incrementHou(t1, hrs);          //调用友元函数增加小时
}

long difs2Time(TimeType t1, TimeType t2) //两个时刻的间隔秒数(差的绝对值)
{
    long m1, m2, difm;
    m1=t1.h*3600+t1.m*60+t1.s;      //对象 t1 的 h:m:s 距 0:0:0 的秒数
    m2=t2.h*3600+t2.m*60+t2.s;      //对象 t2 的 h:m:s 距 0:0:0 的秒数
    difm=abs(m1 - m2);              //两个时刻的间隔总秒数(差的绝对值)
    return difm;                    //返回间隔总秒数
}

bool equal(TimeType t1, TimeType t2) //判断两个时刻是否相等
{
    if(t1.h==t2.h && t1.m==t2.m && t1.s==t2.s)
        return true;
    return false;
}

bool notEqual(TimeType t1, TimeType t2) //判断两个时刻是否不相等
{
    if(t1.h!=t2.h || t1.m!=t2.m || t1.s!=t2.s)
        return true;
    return false;
}

bool lessThan(TimeType t1, TimeType t2) //判断时刻 1 是否小于时间 2
{
    if(t1.h<t2.h) return true;      //时刻 1 的 h 小
    if(t1.h>t2.h) return false;
    if(t1.m<t2.m) return true;      //时刻 1 的 m 小
    if(t1.m>t2.m) return false;
    if(t1.s<t2.s) return true;      //时刻 1 的 s 小
    else return false;
}

```

```
bool lessOrEqual(TimeType t1, TimeType t2) //判断时刻 1 是否小于等于时刻 2
{
    if( lessThan(t1, t2) || equal(t1, t2) )    //调用友元函数
        return true;
    else return false;
}
```

```
bool greaterThan(TimeType t1, TimeType t2)    //判断时刻 1 是否大于时刻 2
{
    if( !lessOrEqual(t1, t2) )                //调用友元函数
        return true;
    else return false;
}
```

```
bool greaterOrEqual(TimeType t1, TimeType t2) //判断时刻 1 是否大于等于时刻 2
{
    if( greaterThan(t1, t2) || equal(t1, t2) )    //调用友元函数
        return true;
    else return false;
}
```

```
void inputTime(TimeType& t1) //键盘输入时刻对象的有关数据(时、分、秒)
{
    cout<<"Input time(h(0~23) m(0~59) s(0~59)):";
    cin>>t1.h>>t1.m>>t1.s;
}
```

```
void printTime(TimeType t1) //屏幕输出时间对象的有关数据(时、分、秒)
{
    cout<<t1.h<<":"<<t1.m<<":"<<t1.s<<endl;
}
```

```
void printAmPm(TimeType t1) //以 Am(上午)或 Pm(下午)的格式在屏幕上输出时间对象
{
    if(t1.h<12)
        cout<<t1.h<<":"<<t1.m<<":"<<t1.s<<"(AM)"<<endl;
    else
        cout<<t1.h-12<<":"<<t1.m<<":"<<t1.s<<"(PM)"<<endl;
}
```

```

}

void printDifs(long difs)    //非类成员函数也非友元函数，只是 main 中使用的函数
{
    //以“ difs=...s=...m...s=...h...m...s” 的格式输出两个时刻间隔

    long ds, dm, dh;
    cout<<"difs="<<difs<<"s=";
    dm=difs/60;          ds=difs%60;
    cout<<dm<<"m"<<ds<<"s=";
    dh=dm/60;          dm=dm%60;
    cout<<dh<<"h"<<dm<<"m"<<ds<<"s"<<endl;
}

void main(void)
{
    TimeType t1, t2(8, 15, 10), t3;    //说明类对象 t1、t2、t3
    cout<<"t1=";
    printTime(t1);                    //输出类对象 t1
    cout<<"t2=";
    printTime(t2);                    //输出类对象 t2
    long difs=difs2Time(t1, t2);      //t1 与 t2 两个时刻的时间间隔(差的绝对值)
    printDifs(difs);                  //printDifs 并非类成员函数也非友元函数
    cout<<"t1: ";
    inputTime(t1);                    //为类对象 t1 输入数据
    cout<<"t2: ";
    inputTime(t2);                    //为类对象 t2 输入数据
    difs=difs2Time(t1, t2);           //t1 与 t2 两个时刻的时间间隔(差的绝对值)
    printDifs(difs);                  //printDifs 并非类成员函数也非友元函数
    setTime(t3, 15, 28, 0);           //为类对象 t3 设置时间
    cout<<"t3=";
    printTime(t3);                    //输出类对象 t3
    if(greaterThan(t1, t2))            //t1>t2?
        cout<<"t1>t2"<<endl;
    if(equal(t1, t2))                  //t1==t2?
        cout<<"t1==t2"<<endl;
    if(greaterOrEqual(t1, t2))         //t1>=t2?
        cout<<"t1>=t2"<<endl;
    if(lessThan(t1, t2))               //t1<t2?
        cout<<"t1<t2"<<endl;
}

```

```

        if(lessOrEqual(t1, t2))           //t1<=t2?
            cout<<"t1<=t2"<<endl;
        if(notEqual(t1, t2))             //t1!=t2?
            cout<<"t1!=t2"<<endl;
        incrementSec(t1, 40);             //增加若干秒
        cout<<"t1.incrementSec(40);'=> ";
        cout<<"t1="; printTime(t1);       //输出类对象 t1
        cout<<"t1="; printAmPm(t1);       //以 AM-PM 格式输出类对象 t1
        incrementMin(t2, 50);            //增加若干分
        cout<<"t2.incrementMin(50);'=> ";
        cout<<"t2="; printTime(t2);       //输出类对象 t2
        cout<<"t2="; printAmPm(t2);       //以 AM-PM 格式输出类对象 t2
        incrementHou(t2, 11);            //增加若干小时
        cout<<"t2.incrementHou(11);'=> ";
        cout<<"t2="; printTime(t2);       //输出类对象 t2
        cout<<"t2="; printAmPm(t2);       //以 AM-PM 格式输出类对象 t2
        incrementAll(t2, 6, 45, 55);     //增加一定的时、分、秒
        cout<<"t2.incrementAll(6, 45, 55);'=> ";
        cout<<"t2="; printTime(t2);       //输出类对象 t2
        cout<<"t2="; printAmPm(t2);       //以 AM-PM 格式输出类对象 t2
    }

```

[输出结果]

```

t1=0:0:0
t2=8:15:10
difs=29710s=495m10s=8h15m10s
t1: Input time(h(0~23) m(0~59) s(0~59)):11 30 45
t2: Input time(h(0~23) m(0~59) s(0~59)):22 18 30
difs=38865s=647m45s=10h47m45s
t3=15:28:0
t1<t2
t1<=t2
t1!=t2
't1.incrementSec(40);'=> t1=11:31:25
t1=11:31:25(AM)
't2.incrementMin(50);'=> t2=23:8:30
t2=11:8:30(PM)
't2.incrementHou(11);'=> t2=10:8:30
t2=10:8:30(AM)

```

```
t2.incrementAll(6, 45, 55);'=> t2=16:54:25
t2=4:54:25(PM)
```

7.18 自定义日期类(类型)DateType

自定义具有如下原型的一个简单的日期类型 `DateType`, 其中说明了多个类成员函数, 使用该类可以完成日期的输入、对天的增加、计算两个日期的间隔天数、计算某日期为星期几、打印对象当前数据的 `y` 年 `m` 月的月历、对两个日期进行 6 种比较、以及对日期的输出等操作。请完成各函数, 并编制主函数, 说明 `DateType` 类对象, 对定义的各函数进行调用, 以验证它们的正确性。

```
class DateType                                //自定义的日期类型 DateType
{
    int y,m,d;                                //私有数据成员, 表示当前日期的年、月、日
public:
    DateType(int y0, int m0, int d0);          //三参构造函数, 设定年、月、日
    DateType();                                //无参构造函数, 将年、月、日设为 1.1.1(1 年 1 月 1 日)
    void setDate(int y0, int m0, int d0);      //按指定的年、月、日来设置日期
    int getYear(){ return y; }                 //获取当前日期的“年” —— 私有数据 y 之值
    int getMonth(){ return m; }                //获取当前日期的“月” —— 私有数据 m 之值
    int getDay(){ return d; }                  //获取当前日期的“日” —— 私有数据 d 之值
    void incrementDay();                        //增加 1 天
    long difsTwoDate(DateType dt2);            //两个日期的间隔天数(差的绝对值)
    int whatDay();                              //某日期为星期几(公元的元年元月元日即 1 年 1 月 1 日为星期一)
    void printMonthCalen();                     //打印对象当前数据的 y 年 m 月的月历
    bool equal(DateType dt2);                  //判断两个日期是否相等
    bool notEqual(DateType dt2);               //判断两个日期是否不相等
    bool lessThan(DateType dt2);               //判断日期 1 是否小于日期 2
    bool lessOrEqual(DateType dt2);            //判断日期 1 是否小于等于日期 2
    bool greaterThan(DateType dt2);            //判断日期 1 是否大于日期 2
    bool greaterOrEqual(DateType dt2);         //判断日期 1 是否大于等于日期 2
    void inputDate();                           //键盘输入日期对象的有关数据(年、月、日)
    void printDate();                           //屏幕输出日期对象的有关数据(年、月、日)
}
```

[分析提示]

(1) 类成员函数内(定义处)可处理与使用本 `DateType` 类的私有成员, 而且总以当前调用者对象(`*this`)作为该成员函数的隐式第一运算分量。若所定义的运算多于一个运算对象时, 才将其余运算对象显式地列在该成员函数的参数表中。

(2) 对日期增加 1 天的 `incrementDay` 成员函数中, 要注意进位处理 —— 首先算出本“日”所在的月份具有的天数 `daysInMonth`, 若加 1 之后的“日”数值超过所在的月份具有的天数时, 进位到月, 而月份若超过 12 时, “进位”到年。

(3) 成员函数 `difsTwoDate` 用来计算二日期的间隔天数(差的绝对值), 在实现中调用了存放于“`difs2Date.h`”的头文件中的 `difs2Date` 函数 注意, `difs2Date` 并非类成员函数)。

(4) 编制主函数, 说明 `DateType` 类对象, 并通过类对象对 `DateType` 类的各个成员函数进行各种可能实参情况数据的调用, 以验证每一成员函数的正确性。

[解答程序]

```
#include <iostream.h>
#include <iomanip.h>
#include <math.h>
#include "difs2Date.h"
//计算两个日期间隔总天数的 difs2Date 函数放在"difs2Date.h"的头文件中
int whatD(int y, int m, int d);          //计算出由(y,m,d)指定的日期为星期几(0~6)并返回

class DateType                          //自定义的日期类型 DateType
{
    ...                                //与题目处相同, 略
};

DateType::DateType(int y0, int m0, int d0)  //三参构造函数, 设定年、月、日
                                           //构造函数, 为私有数据成员赋初值
{
    y=y0;d=d0;m=m0;
}

DateType::DateType()  //无参构造函数, 将年、月、日设为 1.1.1(1 年 1 月 1 日)
{
    y=1;    d=1;    m=1;
}

void DateType::setDate(int y0, int m0, int d0)  //按指定的年、月、日来设置日期
{
    y=y0;d=d0;m=m0;
}

void DateType::incrementDay()                //增加 1 天
{
```

```

d+=1;
int daysInMonth;           //本日所在的月份具有多少天
if(m==1||m==3||m==5||m==7||m==8||m==10||m==12)
    daysInMonth=31;
else
    if(m==4||m==6||m==9||m==11)
        daysInMonth=30;
    else
        daysInMonth=28;
if( (y%4==0 && y%100!=0) || y%400==0 )
    daysInMonth++;
if(d>daysInMonth)         //d 值超过所在的月份具有的天数时，进位到月
{
    d=1;      m++;
    if(m>12)   //月份超过 12 时，进位到年
    {
        m=1;
        y++;   //年份增加一
    }
}
}

long DateType::difsTwoDate(DateType dt2)  //两个日期的间隔天数(差的绝对值)
{
    long difd;
    difd=difs2Date(y,m,d, dt2.y,dt2.m,dt2.d);
    //通过 difs2D 函数计算日期(y,m,d)与日期(dt2.y,dt2.m,dt2.d)的间隔天数并返回
    return difd;           //返回 difd
}

int DateType::whatDay( )
{
    //某日期为星期几(公元的元年元月元日即 1 年 1 月 1 日为星期一)
    int wD=whatD(y,m,d);
    //通过 whatD 函数计算出由(y,m,d)指定的日期为星期几(0~6)并返回
    return wD;             //返回 wD
}

int whatD(int y, int m, int d)

```

```

{
    //公元的元年元月元日即 1 年 1 月 1 日为星期一
    long difsD=difs2Date(1, 1, 1, y, m, d);
    //算出从日期(1,1,1)到日期(y,m,d)的间隔总天数
    return (difsD+1)%7;          //间隔总天数加 1 模 7 即为所求结果
}

void DateType::printMonthCalen()    //打印对象当前数据的 y 年 m 月的月历
{
    int MthDays[13]={0,31,28,31,30,31,30,31,31,30,31,30,31};
    //MthDays[i]表示月份为 i 的那一个月的天数(若闰年的话, 2 月份要再增加 1 天)
    int ans=whatD(y,m,1);          //ans 为欲处理月的 1 号是星期几(0 表示星期日)
    cout<<y<<" "<<m<<" "<<1<<" => weekday: "<<ans<<endl;
    //显示欲处理月的 1 号是星期几
    cout<<"-----"<<endl;
    cout<<" Sun Mon Tue Wed Thu Fri Sat"<<endl;
    cout<<"-----"<<endl;
    int wday=ans;                  //wday 现在为欲处理月的 1 号是星期几
    cout<<setw(4*(wday+1))<<1;    //将当月 1 号显示在适当位置
    for (int i=2; i<=MthDays[m]; i++) //显示那一月的其他日期数 i(i=2,3,4,...)
    {
        wday++;                  //每处理一天, 星期数增加一
        if (wday % 7==0){        //处理到周末时
            cout<<endl;          //下一周的日期数要换行显示
            wday=0;              //星期数从头来
        }
        cout<<setw(4)<<i;         //在适当位置显示出日期数 i
    }
    cout<<endl;
    cout<<"-----"<<endl;
}

bool DateType::equal(DateType dt2) //判断两个日期是否相等
{
    if(y==dt2.y && m==dt2.m && d==dt2.d)
        return true;
    return false;
}

```

```
bool DateType::notEqual(DateType dt2)    //判断两个日期是否不相等
{
    if(y!=dt2.y || m!=dt2.m || d!=dt2.d)
        return true;
    return false;
}

bool DateType::lessThan(DateType dt2)    //判断日期 1 是否小于日期 2
{
    if(y<dt2.y) return true;             //日期 1 的 y 小
    if(y>dt2.y) return false;
    if(m<dt2.m) return true;             //日期 1 的 m 小
    if(m>dt2.m) return false;
    if(d<dt2.d) return true;             //日期 1 的 d 小
    else return false;
}

bool DateType::lessOrEqual(DateType dt2) //判断日期 1 是否小于等于日期 2
{
    if( (*this).lessThan(dt2) || (*this).equal(dt2) ) //调用本类成员函数
        return true;
    else
        return false;
}

bool DateType::greaterThan(DateType dt2)    //判断日期 1 是否大于日期 2
{
    if( !(*this).lessOrEqual(dt2) )          //调用本类成员函数
        return true;
    else
        return false;
}

bool DateType::greaterOrEqual(DateType dt2) //判断日期 1 是否大于等于日期 2
{
    if( (*this).greaterThan(dt2) || (*this).equal(dt2) ) //调用本类成员函数
        return true;
    else
        return false;
}
```

```

    }

    void DateType::inputDate( )      //键盘输入日期对象的有关数据(年、月、日)
    {
        cout<<"Input date(y(1->) m(1->12) d(1->31)):";
        cin>>y>>m>>d;
    }

    void DateType::printDate( )      //屏幕输出日期对象的有关数据(年、月、日)
    {
        cout<<y<<":"<<m<<":"<<d<<<<endl;
    }

    void main(void)
    {
        DateType dt1, dt2(2000, 1, 1), dt3;      //说明类对象 dt1, dt2, dt3
        cout<<"dt1=";
        dt1.printDate( );                        //输出类对象 dt1
        cout<<"dt2=";
        dt2.printDate( );                        //输出类对象 dt2
        long dif=dt1.difsTwoDate(dt2);           //dt1 与 dt2 两个日期的间隔天数(差的绝对值)
        cout<<"dif=dt1.difsTwoDate(dt2);' => dif="<<dif<<"(days)"<<endl;
        cout<<"dt1: ";
        dt1.inputDate( );                        //为类对象 dt1 输入数据
        cout<<"whatDay of dt1="<<dt1.whatDay( )<<endl;      //显示 dt1 为星期几
        dt1.printMonthCalen( );                  //打印对象 dt1 当前数据的的月历
        cout<<"dt2: ";
        dt2.inputDate( );                        //为类对象 dt2 输入数据
        cout<<"whatDay of dt2="<<dt2.whatDay( )<<endl;      //显示 dt2 为星期几
        dif=dt1.difsTwoDate(dt2);               //dt1 与 dt2 两个日期的间隔天数(差的绝对值)
        cout<<"dif=dt1.difsTwoDate(dt2);' => dif="<<dif<<"(days)"<<endl;
        dt3.setDate(1998,11,28);                //为类对象 t3 设置日期
        cout<<"dt3=";
        dt3.printDate( );                        //输出类对象 dt3
        if(dt1.greaterThan(dt2))                 //dt1>dt2?
            cout<<"dt1>dt2"<<endl;
        if(dt1.equal(dt2))                      //dt1==dt2?
            cout<<"dt1==dt2"<<endl;
        if(dt1.greaterOrEqual(dt2))             //dt1>=dt2?

```

```

        cout<<"dt1>=dt2"<<endl;
    if(dt1.lessThan(dt2))                //dt1<dt2?
        cout<<"dt1<dt2"<<endl;
    if(dt1.lessOrEqual(dt2))            //dt1<=dt2?
        cout<<"dt1<=dt2"<<endl;
    if(dt1.notEqual(dt2))                //dt1!=dt2?
        cout<<"dt1!=dt2"<<endl;
    dt1.incrementDay( );                //增加若干天
    cout<<"dt1.incrementDay( );'=> ";
    cout<<"dt1="; dt1.printDate( );    //输出类对象 dt1
}

```

[输出结果]

```

dt1=1:1:1
dt2=2000:1:1
'dif=dt1.difsTwoDate(dt2);'=> dif=730119(days)
dt1: Input date(y(1->...) m(1->12) d(1->31)):2002 5 10
whatDay of dt1=5
2002.5.1 => weekday: 3
-----
Sun Mon Tue Wed Thu Fri Sat
-----
          1   2   3   4
5   6   7   8   9  10  11
12  13  14  15  16  17  18
19  20  21  22  23  24  25
26  27  28  29  30  31
-----
dt2: Input date(y(1->...) m(1->12) d(1->31)):2008 8 18
whatDay of dt2=1
'dif=dt1.difsTwoDate(dt2);'=> dif=2292(days)
dt3=1998:11:28
dt1<dt2
dt1<=dt2
dt1!=dt2
'dt1.incrementDay( );'=> dt1=2002:5:11

```

7.19 自定义的计算器类(类型)compu

自定义一个 compu 类，即计算器类型。它具有如下一些功能：

(1) 从键盘输入欲处理的表达式 str(为一个字符串)，或通过一参构造函数设置该表达式。

(2) 表达式中只可含有加、减、乘、除、等运算符以及实型数据的运算对象，表达式不限长度，遇等号结束表达式并计算出结果显示出来。

(3) 运算符具有不同优先级，乘、除优先于加、减。

(4) 具有一定的排错功能，当用户输入错误的表达式时提示用户。

例如，程序执行后的输入输出对应关系举例如下：

输入：1-2*3=	输出：-5
输入：1+2*3-4=	输出：3
输入：1+2*3-25/5=	输出：2
输入：1-2*3-4*2/3-1=	输出：-8.66667
输入：55-2.5*3/2-22.75=	输出：28.5
输入：1.1+2.2*2-3+4*2.1=	输出：10.9
输入：100/2/2/5+5*2*2-20=	输出：5

下面给出实现方式之一的程序核心代码。

```
const int maxsize=100;    //运算对象栈及运算符栈的大小
class Num_Stack           //自定义的 Num_Stack 类类型，即栈类型
{
    float data[maxsize]; //data 中存放栈的实际数据(运算对象)，大小为 maxsize
    int top;             //top 为栈顶位置
public:                  //五个公有成员函数的原型
    Num_Stack (void);    //构造函数
    ~Num_Stack(void);    //析构函数
    bool empty(void);    //判断栈是否为空
    void push (float a); //将数据 a (运算对象)压入栈顶
    float pop (void);    //将栈顶数据(运算对象)弹出并返回
};

class Op_Stack           //自定义的 Op_Stack 类类型，即栈类型
{
    char data[maxsize];  //data 中存放栈的实际数据(运算符)，大小为 maxsize
    int top;             //top 为栈顶位置
public:                  //五个公有成员函数的原型
    Op_Stack (void);    //构造函数
```

```

~Op_Stack(void);    //析构函数
bool empty(void);   //判断栈是否为空
void push (char a); //将数据 a(运算符)压入栈顶
char pop (void);    //将栈顶数据(运算符)弹出并返回
};

class compu          //自定义的 compu 类(类型), 即计算器类型
{
    Num_Stack numStack; //存放运算对象的栈 numStack
    Op_Stack opStack;   //存放运算符的栈 opStack
    Num_Stack numStack2;
        //存放运算对象的第二个栈 numStack2(遇等号后进行处理时临时使用)
    Op_Stack opStack2;
        //存放运算符的第二个栈 opStack2(遇等号后进行处理时临时使用)
    char str[100];      //存放欲计算的表达式(为一个符号串)
    int ind;            //表达式已被处理到的当前位置(下标值)
public:
    compu( );          //无参构造函数
    compu(char * s);    //一参构造函数(带来欲计算的表达式 s)
    void setStr(char * s); //设置欲计算的表达式 s
    void printExp( );   //显示出当前欲计算的表达式
    void myInFromStr(char* str, float& x, char& op);
        //从欲处理表达式 str 的当前位置 ind 开始,
        //拼装出一个运算对象 x 以及一个运算符 op
    void myDo( );
        //通过调用 myInFromStr 从欲处理的表达式 str 处获取运算对象与运算符,
        //并利用四个私有数据的栈对象, 对欲计算的表达式 str 进行从头到尾的
        //处理(直至输出运算结果而结束)
};

void main( ) //主函数, 说明 compu 对象并通过调用 myDo 函数来计算各种表达式
{
    compu cmp1("1-2*3=");
    cmp1.printExp( );  cmp1.myDo( );    //输出结果: 1-2*3=-5
    compu cmp2("1+2*3-4=");
    cmp2.printExp( );  cmp2.myDo( );    //输出结果: 1+2*3-4=3
    compu cmp3("1+2*3-25/5=");
    cmp3.printExp( );  cmp3.myDo( );    //输出结果: 1+2*3-25/5=2
    compu cmp4("1-2*3-4*2/3-1=");

```



```

    cmp4.printExp();  cmp4.myDo();    //输出结果: 1-2*3-4*2/3-1=-8.66667
    compu cmp5("55-2.5*3/2-22.75=");
    cmp5.printExp();  cmp5.myDo();    //输出结果: 55-2.5*3/2-22.75=28.5
    compu cmp6("1.1+2.2*2-3+4*2.1=");
    cmp6.printExp();  cmp6.myDo();    //输出结果: 1.1+2.2*2-3+4*2.1=10.9
    compu cmp7;
    cmp7.printExp();  cmp7.myDo();    //输出结果: 0=0
    compu cmp8;  char str0[100];
    cout<<"Input EXPRESSION (ending with '='), if only 'ENTER' then ending
program."<<endl;
    cin.getline(str0, 100);
        //从键盘输入欲计算的任意表达式(为一个符号串)放入 str0
    while(strlen(str0)>0)                //当输入串非空时继续处理(入空串时结束)
    {
        cmp8.setStr(str0);                //将 str0 设置为欲计算的表达式
        cmp8.myDo();                    //计算表达式并输出结果
        cout<<"Input EXPRESSION (ending with '='), if only 'ENTER' then ending
program."<<endl;
        cin.getline(str0, 100);
            //从键盘输入下一个欲计算的任意表达式(符号串)放入 str0
    }
}

```

[分析提示]

本示例所处理的表达式中, 不同的运算符具有不同的优先级(乘除优先于加减), 而且使用的是中缀运算符。主要处理过程依靠 `compu` 的类成员函数 `myDo` 来完成, 由它负责对欲计算的中缀表达式 `str`(在 `compu` 类对象的私有数据成员 `str` 中存放欲计算的表达式)进行从头到尾的处理(直至输出运算结果而结束)。具体地说, `myDo` 首先通过调用另一个类成员函数 `myInFromStr` 从欲处理的表达式 `str` 处获取运算对象与运算符, 并利用 `compu` 类对象中的运算对象栈以及运算符栈进行临时存储。而后总是在获取到后一个运算符时, 才与运算符栈中的前一个运算符进行比较, 若后一运算符的优先级高(前一运算符为加减, 而后一运算符为乘除时), 则不可马上处理, 要将当前的运算对象及运算符再一次放入栈中, 如此等等。而只有在后一运算符的优先级低于等于前一运算符时, 方可利用前一运算符对它所关联的运算对象栈内的两个运算对象进行立即计算, 而后又将计算结果重新放入运算对象栈中, 如此重复, 直到遇见等号时, 要对当前栈中的剩余部分进行连续运算后输出计算结果而结束。

[解答程序]

```
#include <iostream.h>
```

```
#include <process.h>
#include <string.h>
const int maxsize=100;           //运算对象栈及运算符栈的大小

class Num_Stack                  //自定义的 Num_Stack 类类型，即栈类型
{
    ...                          //与题目处相同，略!
};

Num_Stack::Num_Stack(void)       //构造函数
{
    top=0;                      //将当前调用者对象(变量)的私有数据 top 置为 0
}

Num_Stack::~Num_Stack(void)     //析构函数
{
}

bool Num_Stack::empty(void)
{
    return top==0?true:false;    //当当前调用者对象(变量)的 top=0 时，栈为空
}

void Num_Stack::push (float a)  //将(运算对象)数据 a 压入栈顶
{
    if (top==maxsize)           //栈满时，不再压入数据
    {
        cout << "Num_Stack is full!"<< endl;
        return;
    }
    data[top]=a;                //栈未满时，将 a 放入存放栈实际数据的 data 数组中
    top ++;                     //下推栈顶位置
}

float Num_Stack::pop (void)     //将栈顶数据(运算对象)弹出并返回
{
    if (top==0)                 //若栈为空，返回 0
    {
        cout << "Num_Stack is underflow!"<< endl;
```

```

        return 0;
    }
    top--;
    return data[top];           //栈非空时，返回栈顶数据
}

class Op_Stack                 //自定义的 Op_Stack 类类型，即栈类型
{
    ...                         //与题目处相同，略!
};

Op_Stack::Op_Stack(void)       //构造函数
{
    top=0;                     //将当前调用者对象(变量)的私有数据 top 置为 0
}

Op_Stack::~Op_Stack(void)      //析构函数
{
}

bool Op_Stack::empty(void)
{
    return top==0?true:false;   //当当前调用者对象(变量)的 top=0 时，栈为空
}

void Op_Stack::push (char a)    //将数据 a(运算符)压入栈顶
{
    if (top==maxsize)           //栈满时，不再压入数据
    {
        cout << "Num_Stack is full!"<< endl;
        return;
    }
    data[top]=a;                 //栈未滿时，将 a 放入存放栈实际数据的 data 数组中
    top ++;                     //下推栈顶位置
}

char Op_Stack::pop (void)       //将栈顶数据(运算符)弹出并返回
{
    if (top==0)                 //若栈为空，返回 0

```

```

    {
        cout << "Num_Stack is underflow!" << endl;
        return 0;
    }
    top--;
    return data[top];           //栈非空时，返回栈顶数据
}

float opFunc(float x, char op, float y)
{
    //将 x 与 y 进行指定的“ op ”运算并返回运算结果
    float res;
    switch (op)                 //根据运算符 op 的值来进行多分支处理
    {
        case '+':               //当 op 为“ + ”符号时的处理分支
            res=x+y;           break;
        case '-':               //当 op 为“ - ”符号时的处理分支
            res=x-y;           break;
        case '*':               //当 op 为“ * ”符号时的处理分支
            res=x*y;           break;
        case '/':               //当 op 为“ / ”符号时的处理分支
            if(y==0)            //分母为 0 时的处理
            {
                cout<<endl<<"overflow!"; //提示出：“ overflow!” 字符串，表示运算溢出
                exit(0);
            }
            res=x/y;           break;
        default:                //当 op 不为上述任一符号时的处理
            cout<<endl<<"Wrong_001!";    //提示出：“ Wrong!” 字符串，
            //表示输入的运算符有错
            exit(0);
    }
    //switch 语句结束
    return res;
}

class compu                    //自定义的 compu 类(类型)，即“ 计算器 ”类型
{
    ...                        //与题目处相同，略!
};

```

```

compu::compu( ){                                //无参构造函数
    strcpy(str, "0=");
    ind=0;                                       //表达式已被处理到的当前位置为 0(下标值)
}

compu::compu(char * s)                          //一参构造函数(带来欲计算的表达式 s)
{
    strcpy(str, s);
    ind=0;                                       //表达式已被处理到的当前位置为 0(下标值)
}

void compu::setStr(char * s)                   //设置欲计算的表达式 s
{
    strcpy(str, s);
    ind=0;                                       //表达式已被处理到的当前位置为 0(下标值)
}

void compu::printExp( )                       //显示出当前欲计算的表达式
{
    cout<<str;
}

void compu::myDo( )
{
    //通过调用 myInFromStr 从欲处理的表达式 str 处获取运算对象与运算符,
    //并利用四个私有数据的栈对象, 对欲计算的表达式 str 进行从头到尾的
    //处理(直至输出运算结果而结束)
    float x, y, res;
    char op1, op2;
    myInFromStr(str,x,op1);
        //从欲处理表达式 str 的当前位置 ind 开始,
        //拼装出一个运算对象 x 以及一个运算符 op1
    numStack.push(x);                          //将 x 压入运算对象栈
    opStack.push(op1);                        //将 op1 压入运算符栈
    while(!opStack.empty( ))                 //当运算符栈非空时循环处理
    {
        x=numStack.pop( );                  //从栈顶取来运算对象 x
        op1=opStack.pop( );                //取来运算符 op1
    }
}

```

```

if(op1!='=')                //若 op1 非结束符号“=”时
    myInFromStr(str,y,op2);
    //从 str 的当前位置 ind 开始,
    //拼装出后一运算对象 y 以及后一运算符 op2
switch (op1)                //根据前一运算符 op1 的值来进行多分支处理
{
case '+':                    //当 op1 为“+”符号时的处理分支
    if (op2=='*'||op2=='/') //后一运算符 op2 优先级高时
    {
        //不可马上处理, 将运算对象及运算符放入栈中
        numStack.push(x);    numStack.push(y);
        opStack.push(op1);   opStack.push(op2);
        break;
    }
    else //后一运算符 op2 的优先级与 op1 相同时, 处理前一运算
    {
        res=opFunc(x,op1,y);
        //将 x 与 y 进行指定的”op1”运算并返回运算结果
        numStack.push(res);   //将运算结果放入栈中
        opStack.push(op2);
        break;
    }
case '-':                    //当 op1 为“-”符号时的处理分支
    if (op2=='*'||op2=='/') //后一运算符 op2 优先级高时
        //不可马上处理, 将运算对象及运算符放入栈中
    {
        numStack.push(x);    numStack.push(-y);
        opStack.push('+');    opStack.push(op2);
        break;
    }
    else //后一运算符 op2 的优先级与 op1 相同时, 处理前一运算
    {
        res=opFunc(x,op1,y);
        //将 x 与 y 进行指定的”op1”运算并返回运算结果
        numStack.push(res);   //将运算结果放入栈中
        opStack.push(op2);
        break;
    }
case '*':                    //当 op1 为“*”符号时的处理分支

```



```

    }                                     //while(!opStack.empty())循环结束
    cout<<res<<endl;
}

void compu::myInFromStr(char* str, float& x, char& op)
{
    //从欲处理表达式 str 的当前位置 ind 开始,
    //拼装出一个运算对象 x 以及一个运算符 op
    float num=0;
    char cc=str[ind++];                 //从 str 的当前位置 ind 开始取字符
    if(cc<'0' || cc>'9')                 //第一个字符应为数字字符
    {
        cout<<endl<<"Wrong_0000!";    //提示输入的不是数字
        exit(0);
    }
    while (cc>='0' && cc<='9')           //处理连续的各数字字符(小数点前)
    {
        num=num*10+(cc-'0');
        cc=str[ind++];
    }
    x=num;
    if(cc!='+' && cc!='-' && cc!='*' && cc!='/' && cc!='=' && cc!='.')
    {
        //数后符号
        cout<<endl<<"Wrong_0001!";    //错误提示, 输入的数后符号有误
        exit(0);
    }
    op=cc;
    if(cc!='.') return;                 //整数则返回
    float num2=0, fac=0.1;             //处理点后部分
    cc=str[ind++];
    if(cc<'0' || cc>'9')                 //第一个字符应为数字字符
    {
        cout<<endl<<"Wrong_0000!";    //错误提示: 输入的不是数字
        exit(0);
    }
    while (cc>='0' && cc<='9')           //处理点后数据
    {
        num2+=fac*(cc-'0');

```



```

        cc=str[ind++];
        fac/=10;
    }
    x+=num2;
    op=cc;                //运算符 op
}

void main() //主函数，说明 compu 对象并通过调用 myDo 函数来计算各种表达式
{
    ...                    //与题目处相同，略!
}

```

[输出结果]

```

1-2*3=-5
1+2*3-4=3
1+2*3-25/5=2
1-2*3-4*2/3-1=-8.66667
55-2.5*3/2-22.75=28.5
1.1+2.2*2-3+4*2.1=10.9
0=0
Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.
0-55+6/2.2*3+10=
-36.8182
Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.
100/2/2/5+5*2*2-20=
5
Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.

Press any key to continue

```

7.20 求满秩矩阵逆矩阵的自定义类 Matrix

通过自定义类 Matrix，求具有 lines 行的满秩矩阵的逆矩阵。给出以下几个要点：

- (1) 矩阵行数(阶数)lines 之值由用户任意输入(lines 为类的私有数据成员)。
- (2) 先将欲求逆的原始矩阵另加一个单位矩阵放于 MatrA 之中(MatrA 为类的私有数据成员，被说明为：double** MatrA；它所具有的 lines 行、2*lines 列的存储空间通过 new 来动态分配)，而原始矩阵的各元素值也由用户任意输入。
- (3) 设法将 MatrA 左半的原始矩阵化为单位矩阵(利用行初等变换)，此时，右半的原

单位矩阵则变成了欲求的结果矩阵(即原始矩阵的逆矩阵)。

下面给出实现方式之一的核心程序代码。

```
class Matrix                                //类 Matrix 说明
{
    int lines;                               //矩阵的维数(行数)
    double** MatrA; //数组首地址, 存放矩阵, 初始时为原始矩阵+单位矩阵
    //它所具有的 lines 行、2*lines 列的存储空间通过 new 语句来动态分配
public:
    Matrix(int Lns); //构造函数, 形参为行数(阶数), 动态分配 MatrA 的存储空间
    ~Matrix( );      //析构函数, 释放 MatrA 的存储空间
    void InSourceMatrix( );
    //输入原始矩阵初值, 并形成 MatrA 的、右半部分的单位矩阵
    void printMatrix( );
    //显示整体矩阵(lines 行、2*lines 列, 左半部分+右半部分数据)
    void printSourceMatrix( ); //按格式显示原始矩阵(MatrA 矩阵的左半部分的数据)
    void printResultMatrix( ); //按格式显示结果矩阵(MatrA 矩阵的右半部分的数据)
    void Solve( );            //求解矩阵的逆
};

void main( )                                //主函数
{
    cout<<"Input LINES:";                  //屏幕提示输入矩阵行数(阶数)
    int lines;
    cin>>lines;                             //输入行数
    Matrix matr1(lines);                     //定义一个具有 lines 行的矩阵对象 matr1
    cout<<"Input "<<lines*lines<<" numbers that will form the matrix:"<<endl;
    //提示输入原始矩阵数据
    matr1.InSourceMatrix( );
    //输入原始矩阵数据, 并形成 Matr 的右半部分的单位矩阵
    cout<<"The source Matrix is:"<<endl;
    matr1.printSourceMatrix( );
    //按格式输出原始矩阵(MatrA 矩阵左半的数据)
    matr1.Solve( );                         //求解矩阵的逆(解处于 MatrA 矩阵的右半部分)
    cout<<"The result Matrix is:"<<endl;
    matr1.printResultMatrix( );             //按格式输出结果矩阵(MatrA 矩阵的右半部分的数据)
}
```

程序执行后的显示结果如下:

Input LINES:2

Input 4 numbers that will form the matrix:

1 2 3 4

The source Matrix is:

1	2
3	4

The result Matrix is:

-2	1
1.5	-0.5

[分析提示]

自定义类 **Matrix** 中的一个最主要的成员函数是 **Solve** 函数, 通过它将 **MatrA** 左半的原始矩阵化为单位矩阵(利用行初等变换), 那时 **MatrA** 右半的原单位矩阵则变成了欲求的结果矩阵(即原始矩阵的逆矩阵)。更具体的做法是: 利用行初等变换, 先将 **MatrA** 的 左半部分消为上三角阵(要多次利用“ 某行诸元素乘上一个适当的倍数加到另一行的相应各元素上去” 这种变换操作。另外注意, 右半部分矩阵也要同时进行上述那种相同的初等变换操作); 而后进一步将左半部分化为单位矩阵(当然部分右半也进行相同的变换), 结束后右半部分将成为所要求出的逆矩阵。

[解答程序]

```
#include <iostream.h>
#include <iomanip.h>

class Matrix                                //类 Matrix 说明
{
    ...                                    //与题目处相同, 略!
};

//类 Matrix 的成员函数的实现(类外说明)
Matrix::Matrix(int Lns)                    //矩阵 Matrix 类的构造函数
{
    lines=Lns;                             //矩阵维数(行数)
    MatrA=new double*[lines];              //动态内存分配(盛放矩阵), 共 lines 行
    for(int i=0; i<lines; i++)
        MatrA[i]=new double[2*lines]; //每行 2*lines 列
}

Matrix::~Matrix()                          //矩阵 Matrix 类的析构函数
{
    for(int i=0; i<lines; i++)
```

```

        delete[ ] MatrA[i];           //内存释放
    delete[ ] MatrA;                 //内存释放
}

void Matrix::InSourceMatrix( )
{
    //输入原始矩阵初值，并形成 MatrA 的右半部分的单位矩阵
    int i,j;
    for(i=0; i<lines; i++)           //输入原始矩阵初值，放于 MatrA 的左半部分、
        for(j=0; j<lines; j++)
            cin>>MatrA[i][j];
    for(i=0; i<lines; i++)           //MatrA 的右半部分的单位矩阵先全置为 0
        for(j=lines; j<2*lines; j++)
            MatrA[i][j]=0;
    int ind=lines;
    for(i=0; i<lines; i++)           //形成 Matr 右半部分、的单位矩阵
        MatrA[i][ind++]=1;
}

void Matrix::printMatrix( )          //显示“整体”矩阵 MatrA
{
    for(int i=0; i<lines; i++)        //共显示 lines 行
    {
        for(int j=0; j<2*lines; j++)  //每行显示 2*lines 个数
            cout<<setw(8)<<MatrA[i][j]<<" ";    //每数占 8 格(域宽)
        cout<<endl;
    }
}

void Matrix::printSourceMatrix( )    //显示原始矩阵(处于 MatrA 数组的左半部分)
{
    for(int i=0; i<lines; i++)        //共显示 lines 行
    {
        for(int j=0; j<lines; j++)    //每行显示 lines 个数
            cout<<setw(8)<<MatrA[i][j]<<" ";    //每数占 8 格(域宽)
        cout<<endl;
    }
}

```

```

void Matrix::printResultMatrix( )           //显示结果矩阵(处于 MatrA 数组的右半部分)
{
    for(int i=0; i<lines; i++)              //共显示 lines 行
    {
        for(int j=lines; j<2*lines; j++)    //每行显示 lines 个数
            cout<<setw(8)<<MatrA[i][j]<<" "; //每数占 8 格(域宽)
        cout<<endl;
    }
}

void Matrix::Solve( )                      //求解矩阵的逆, 结果放在 MatrA 数组的右半部分
{
    int k,i,j;
    for (k=1; k<lines; k++)                //左半部分消为上三角阵
    {
        for (i=k; i<lines; i++)
        {
            double factor=-MatrA[i][k-1]/MatrA[k-1][k-1];
            for (j=k-1; j<2*lines; j++)
                MatrA[i][j]+=MatrA[k-1][j]*factor;
        }
    }
    //printMatrix( );                      //输出矩阵(左半部分为上三角阵)
    for (i=0; i<lines; i++)                //左半部分主对角线消为 1
    {
        double factor=MatrA[i][i];
        for (j=i; j<2*lines; j++)
            MatrA[i][j]/=factor;
    }
    //printMatrix( );                      //输出矩阵(左半部分主对角线为 1)
    for (k=lines-2; k>=0; k--) //左半部分消为单位矩阵, 此时的右半部分成为结果
        for (i=k; i>=0; i--)
        {
            double factor=-MatrA[i][k+1];
            for (j=k+1; j<2*lines; j++)
                MatrA[i][j]+=(factor*MatrA[k+1][j]);
        }
    }
}

```

```
void main( )                //主函数
{
    ...                      //与题目处相同，略
}
```

[输出结果]

Input LINES:4

Input 16 numbers that will form the matrix:

1 1 1 2 1 2 2 1 1 2 1 2 1 2 1 1

The source Matrix is:

1	1	1	2
1	2	2	1
1	2	1	2
1	2	1	1

The result Matrix is:

2	-1	-3	3
-1	0	1	0
0	1	0	-1
0	0	1	-1

第八章 继承与派生

[语法要点]

详细内容可参阅教材第八章。

(1) 派生类说明 也是类说明，不过它要指明它所继承的基类，同时在类的成员中可增加一类保护成员。其格式为：

```
class<类名>:<基类说明表>
{
private:
    < 私有成员表 >
protected:
    < 保护成员表 >
public:
    < 公有成员表 >
};
```

其中：

基类说明表 列出所给类的基类；

每个基类说明格式为：<派生方式><基类名>

派生方式 公有派生，保护派生或私有派生。派生方式决定了从基类继承过来的成员在派生类中的封装属性。其规定为：

① 无论何种派生方式，基类的私有成员不能成为派生类继承的成员，基类的保护成员和公有成员可继承为派生类的成员。

② 私有派生使基类的保护成员和公有成员都成为派生类的私有成员。

③ 保护派生使基类的保护成员和公有成员都成为派生类的保护成员。

④ 公有派生使基类的保护成员和公有成员在派生类中，仍然是保护成员和公有成员。

继承与派生关系，有下面几点应注意：

① 一个类可以派生出多个派生类。

② 一个类可有一个或多个基类，称为单一继承和多重继承。

③ 派生类又可有派生类，称为多级继承。

④ 继承关系不可循环。例如：类 A 继承类 B，类 B 继承类 C，类 C 又 继承类 A，这是非法的。

⑤ 基类的友元关系，基类的构造函数和析构函数都不能被派生类所继承。

(2) 派生类对象的创建和初始化

虽然类 B 的派生类 A 只从类 B 继承了保护成员和公有成员，但是，派生类 A 的对

象的创建,则必须包含着基类 B 的对象创建。换句话说,派生类 A 的对象包含了基类 B 的对象,这里面基类 B 的私有成员也在其中!

(3) 构造函数和析构函数

创建派生类对象时系统按下列步骤工作:

- ① 先调用基类的构造函数。
- ② 再调用对象成员的构造函数(如果该类有对象成员的话)。
- ③ 最后调用派生类自己的构造函数。

释放派生类对象时系统的工作步骤则相反:

- ① 先调用派生类自己的析构函数。
- ② 再调用对象成员的析构函数。
- ③ 最后调用基类的析构函数。

应注意:

- ① 派生类构造函数的参数不仅要为自己的数据成员提供初始化数据,还要为基类,以及基类的基类提供初始化数据。
- ② 由初始化符表指明哪些参数用于本类,哪些参数用于基类。
- ③ 在多数多重继承情况下,初始化工作先基类,多个基类则按自左至右的顺序,再对象成员,最后是自身。

(4) 友元关系 基类的友元不会被其派生类继承。如果基类或基类的某成员函数是某类的友元,则这种友元关系则是被其派生类继承的。

(5) 赋值 派生类对象间的赋值操作依据下面的原则:

- ① 如果派生类有自己的赋值运算符的重载定义,即按该重载函数处理。
- ② 派生类未定义自己的赋值操作,而基类定义了赋值操作,则系统自动定义派生类赋值操作,其中基类成员的赋值按基类的赋值操作进行。

③ 二者都未定义专门的赋值操作,系统自动定义缺省赋值操作。

④ 基类对象和派生类对象之间允许有赋值关系:

允许派生类对象赋值给基类对象;

允许派生类对象为基类的引用型对象赋初值。

(6) 静态成员的继承 如果基类中被派生类继承的成员是静态成员,则其静态属性也随静态成员被派生类继承过来。

(7) 多态性 是 C++语言的重要特征。面向对象程序设计支持多态性,从现实世界的实际情况和人们的习惯出发,C++程序中允许大量的同名函数出现。

例如:同一个函数名 `abs()` 可以计算不同类型数据的绝对值;同一运算符“`<<`”可有完全不同的运算与之对应(可以是整数的移位,也可以是数据的输出)。

函数重载和运算符重载(overload)使用同样的函数名和同样的运算符来完成不同的数据处理与操作。

函数的超载或过载(overriding)若干函数,有完全相同的函数原型,却可以有多种多样的相异的函数体。

静态联编(static bandory)在编译时就已确定函数调用语句对应的函数体代码。

动态联编(dynamic bandory)在编译阶段,系统无法判断此次调用应执行哪一段函

数代码，只有到了运行过程中执行到此时，才能临时判断应执行哪一函数代码。

(8) 指针 指向基类对象的指针和指向其派生类对象的指针有一定的关系：

① 基类指针可以指向其派生类的对象，可以用此指针访问派生类对象中从基类继承过来的成员。

② 派生类指针不可指向其基类的对象。

③ 如果要以基类指针指向其派生类对象，且用来引用派生类自己的成员，则须在把派生类对象的地址赋予该指针时，对该指针作强制类型转换（转换为派生类指针）。

(9) 虚函数 一个类的非静态成员函数可以说明为虚函数：

`virtual <返回类型><函数名>(<参数表>)`

被 `virtual` 说明的函数成员称为虚函数。

虚函数的机制要点为：

① 在基类 `CB` 中说明某一函数成员 `f()` 为虚函数，方法是在说明前加关键字“`virtual`”。如 `virtual void print()`；

② 如在 `CB` 的各个派生类 `CD1`, `CD2`, ..., `CDn` 中定义与 `f()` 的原型完全相同（但函数体可以各异）的函数成员 `f()`，则无论是否用关键字 `virtual` 来说明它们，它们自动地定义为虚函数。

③ 当在程序中采用以 `CB` 类指针 `pb` 引用的形式调用函数 `f()`，`pb->f()` 时，系统对它将采用动态联编的方式处理。

采用虚函数的优点在于：

- ① 可使程序简单易读。
- ② 它使得程序模块间的独立性加强。
- ③ 增加了程序的易维护性。
- ④ 提高了程序中信息隐藏的等级。

(10) 纯虚函数与抽象基类

没有函数体的虚函数称为纯虚函数。纯虚函数用下面的形式表示：

`virtual<函数原型>=0;`

例如：

`virtual void print()=0;`

只能用作别的类的基类，而本身不能直接创建对象的类称为抽象基类。包含有纯虚函数的基类为抽象基类。

许多引入虚函数的程序，把基类的虚函数说明为纯虚函数，从而使基类成为一种抽象基类，可以更自然的反映实际应用问题中对象之间的关系。

8.1 继承与派生关系中相关成员的存取权限

仔细分析下述程序中的继承与派生关系，找出所有违反存取权限的使用情况（语句），并上机进行测试验证，以加深对如何正确使用理解。

```
class baseClass                //基类 baseClass
{
    int privateData;           //私有成员
protected:
    int protectData;           //保护成员
public:
    int publicData;            //公有成员
};

class publicDerivedClass : public baseClass        //公有继承
{
public:
    void usebaseClassData( )
    {
publicData=11;                // publicData 仍具有 public 存取权限(公有继承)
    protectData=12;           // protectData 仍具有 protected 存取权限(公有继承)
    privateData=13;           //ERROR! 不可存取基类的 private 成员
    }
};

class claD21 : public publicDerivedClass          //公有继承
{
public:
    void usebaseClassData( )
    {
    publicData=111;            //仍为 public
    protectData=121;          //仍为 protected
    privateData=131;          //ERROR! 不可存取基类的 private 成员
    }
};

class protectedDerivedClass : protected baseClass //保护继承
{
public:
    void usebaseClassData( )
    {
    publicData=21;             //publicData 变为 protected 存取权限(保护继承)
    protectData=22;            //protectData 仍为 protected 存取权限(保护继承)
    privateData=23;           //ERROR! 不可存取基类的 private 成员
    }
```

```

    }
};

class claD22 : public protectedDerivedClass    //公有继承
{
public:
    void usebaseClassData( )
    {
        publicData=211;        //仍为 protected
        protectData=221;       //仍为 protected
        privateData=231;       //ERROR! 不可存取基类的 private 成员
    }
};

class privateDerivedClass : private baseClass    //私有继承
{
public:
    void usebaseClassData( )
    {
        publicData=31;        //publicData 变为 private 存取权限(私有继承)
        protectData=32;       //protectData 变为 private 存取权限(私有继承)
        privateData=33;       //ERROR! 不可存取基类的 private 成员
    }
};

class claD23 : public privateDerivedClass    //公有继承
{
    //可以看出，若以 private 派生方式派生出的类(如上述 privateDerivedClass
    //作为基类)
    //再去派生它的派生类(子类，如 claD23)时，则在其派生类(子类，如 claD23)中，
    //根本无法存取其基类(父类，如 privateDerivedClass)中的任一个成员!
public:
    void usebaseClassData( )
    {
        publicData=311;        //ERROR! 不可存取基类的 private 成员
        protectData=321;       //ERROR! 不可存取基类的 private 成员
        privateData=331;       //ERROR! 不可存取基类的 private 成员
    }
};

```

```
void main( )
{
    baseClass aBase;
    aBase.publicData=1;      //OK!  可以存取公有成员
    aBase.protectData=2;     //ERROR!  不可存取 protected 成员(并非在派生类之中)
    aBase.privateData=3;     //ERROR!  不可存取 private 成员(并非在类定义之中)
    claD21 d21;
    claD22 d22;
    claD23 d23;
    d21.publicData=4;        //OK!  可以存取公有成员
    d21.protectData=5;       //ERROR!  不可存取 protected 成员
    d21.privateData=6;       //ERROR!  不可存取 private 成员
    d22.publicData=7;        //ERROR!  不可存取 protected 成员
    d22.protectData=8;       //ERROR!  不可存取 protected 成员
    d22.privateData=9;       //ERROR!  不可存取 private 成员
    d23.publicData=7;        //ERROR!  不可存取 private 成员
    d23.protectData=8;       //ERROR!  不可存取 private 成员
    d23.privateData=9;       //ERROR!  不可存取 private 成员
}
```

[分析提示]

继承与派生关系中，基类及其派生类的相关成员的存取权限必须遵照如下的规定。

派生方式(基类的被继承方式)	在基类中的存取权限	在派生类中的存取权限
public	public	public
public	protected	protected
public	private	(inaccessible)
protected	public	protected
protected	protected	protected
protected	private	(inaccessible)
private	public	private
private	protected	private
private	private	(inaccessible)

注意，虽然派生类继承了基类的所有成员(确切地说是除掉构造和析构函数之外的所有成员)，但基类的 private 成员在派生类中根本不可被访问(即是说，要想让某成员

在派生类中能够被访问的话,就必须在基类中将其说明为 `protected` 或 `public` 属性的)。派生方式的不同(由程序员来选择、来控制),也使得被继承过来的那些成员在派生类中有了新的存取权限。可以看出,若以 `private` 派生方式派生出的类(作为基类)再去派生它的派生类(子类)时,则在其派生类(子类)中,根本无法存取其基类(父类)中的任何一个成员!

另外注意,通常是要在派生类中添加本派生类的特有成员的,因为只有靠这种添加才使得派生类有区别于其基类,从而使派生类的功能得到进一步扩展与提高。

[解答程序]

具体程序见题目处。

注意,凡出现违反存取权限的使用情况(语句),其后的注释处均给出了“**ERROR!**”标识,并简单指出了出错原因。请查看基类成员在派生类中的存取权限表,以加深对如何进行正确使用的理解。

[输出结果]

由于程序中具有许多处语法错误,无法通过编译,只有将所有的注有“**ERROR!**”的语句全部改正后,方可运行该程序得到某种结果。

8.2 学生教师档案的管理

仿照教材 8.1 节“公司雇员档案的管理”的类定义方法,由 `person`(人员)类出发(作为基类),派生出 `student`(学生)及 `teacher`(教师)类;而后又由 `student`(学生)类出发(作为基类),派生出 `graduateStudent`(研究生)类。并编制主函数,对这些类进行使用。

注意,可假定这几个类各自具有的数据成员为:

`person`(人员)类: 姓名、性别、年龄。

`student`(学生)类: 姓名、性别、年龄、学号、系别。

`teacher`(教师)类: 姓名、性别、年龄、职称、担任课程。

`graduateStudent`(研究生)类: 姓名、性别、年龄、学号、系别、导师。

下面给出实现方式之一的程序的核心代码。

```
class person                                //自定义的 person 类(类型), 它将作为其他几个类的基类
{
    char * name;                            //私有数据成员 name, 表示姓名
    char sex;                               //私有数据成员 sex, 表示性别
    short age;                              //私有数据成员 age, 表示年龄
public:
    person (char * na, char sx, short ag);
```

```
//构造函数，其三个参数分别对应于类的数据成员 name、sex、以及 age；
//要分配动态空间盛放 name
void print ( ) const;           //显示出 person 类对象的三个数据成员的当前值
//由 'const' 指出，在本 print 函数内不可改变调用者对象(即*this)的任何
//成员值
~person ( ) ;                  //析构函数，释放 name 所指向的动态空间
};

class student:public person
{
    //自定义的 student 类(类型)，它由基类 person 所派生，使用的是 public 派生方式
    //除自动包含基类 person 的 name、sex 以及 age 数据项外
    //另添加如下的 Reg_Number 项以及 department 项
    int Reg_Number;             //代表学号
    char department[50];        //代表系别
public:
    student(char* na, char sx, short ag, int num, char* dep);
    //派生类的构造函数，与其基类的初始化相关
    void print ( ) const;       //显示本 student 类的相关数据
};

class teacher:public person
{
    //自定义的 teacher 类(类型)，它由基类 person 所派生，使用的是 public 派生方式
    char post[20],course[30];    //另添加成员：职称，担任课程
    //除自动包含基类 person 的 name、sex 及 age 数据项外
    //另添加两个特性成员
public:
    teacher(char* na,char sx,short ag,char* po,char* co);
    //派生类的构造函数，与其基类的初始化相关
    void print ( ) const;       //显示本 teacher 类的相关数据
};

class graduateStudent:public student
{
    //自定义的 graduateStudent 类(类型)，它由基类 student 所派生
    //使用的是 public 派生方式
    char advisor[30];           //添加一个特性成员 advisor —— 导师
public:
```

```

graduateStudent(char* na, char sx, short ag, int num, char* dep, char* ad);
    //派生类的构造函数，与其基类的初始化相关
void print ( ) const;           //显示本 graduateStudent 类的相关数据
};

void main( )                    //主函数，对所定义类进行使用
{
    person per1("zhang", 'F', 23), per2("chen", 'M', 37);
    //person 类对象，用三个参数对对象进行初始化
    student stu1("li", 'M', 22, 1001, "comp"), stu2("cui", 'F', 21, 1002, "math");
    //前三个参数为基类 person 准备，最后一个参数为本派生类 student 的
    //特性成员准备。实际上，创建派生类对象时，先执行基类 person 的
    //构造函数，完成基类对象的初始化；而后再返回来执行本派生类的
    //构造函数体，完成本派生类对象的初始化。
    teacher teach1("meng", 'M', 46, "professor", "english");
    graduateStudent gra_stu("zhou", 'M', 25, 1011, "comp", "liu");
    cout<<"=== per1.print( ) ==> name,age,sex"<<endl;
    per1.print( );
    cout<<"=== per2.print( ) ==> name,age,sex"<<endl;
    per2.print( );
    cout<<"=== stu1.print( ) ==> name,age,sex,Reg_Number,department"<<endl;
    stu1.print( );
    cout<<"=== stu2.person::print( ) ==> name,age,sex"<<endl;
    stu2.person::print( ); //显示 stu2 的 name, age, sex
    cout<<"=== teach1.print( ) ==> name,age,sex,post,course"<<endl;
    teach1.print( );
    cout<<"=== gra_stu.print( ) ==> name,age,sex,Reg_Number,department,advisor"<<endl;
    gra_stu.print( );
}

```

程序执行后的显示结果如下：

```

=== per1.print( ) ==> name,age,sex
    zhang: 23 : F
=== per2.print( ) ==> name,age,sex
    chen: 37 : M
=== stu1.print( ) ==> name,age,sex,Reg_Number,department
    li: 22 : M
    Reg_Number: 1001
    department: comp

```

```

=== stu2.person::print( ) ==> name,age,sex
    cui: 21 : F
=== teach1.print( ) ==> name,age,sex,post,course
    meng: 46 : M
    post: professor
    course: english
=== gra_stu.print( ) ==> name,age,sex,Reg_Number,department,advisor
    zhou: 25 : M
    Reg_Number: 1011
    department: comp
    advisor: liu

```

[分析提示]

(1) 上面给出了本范例程序的核心代码，说明了基类及其派生类之间的相互关系并规定了每一个类的成员函数的具体功能，剩余要做的只是在类外将各个类的成员函数补充完整即可。

(2) 上述三个派生类都采用了公有派生的方式，即是说，每一派生类都继承了其基类的所有成员（包括数据成员和函数成员），而且还进一步说明了某些本派生类的特有成员。

(3) 注意，派生类的构造函数与其基类的初始化密切相关。具体地说，执行某派生类的构造函数时，总是要先执行其基类的构造函数，完成基类对象的初始化，而后再返回来执行本派生类的构造函数体。

[解答程序]

```

#include <iostream.h>
#include <string.h>
class person          //自定义的 person 类(类型)，它将作为其他几个类的基类
{
    ...                //与题目处相同,略
}

person::person (char * na, char sx, short ag)
{
    //构造函数，其三个参数分别对应于类的数据成员 name，sex，以及 age；
    //要分配动态空间存放 name
    age=ag;
    sex=sx;
    name=new char[strlen(na)+1];    //分配动态空间存放 name
    strcpy(name,na);
}

```



```

}

void person::print ( ) const           //显示出 person 类对象的三个数据成员的当前值
{
    //由“ const”指出，在本 print 函数内不可改变调用者对象(即*this)的任何成员值
    cout<<"    "<<name<<": ";
    cout<<age<<": ";
    cout<<sex<<endl;
}

person::~person ( )                   //析构函数，释放 name 所指向的动态空间
{
    delete[ ]name;
}

class student:public person           //自定义 student 类(类型)，它由基类 person 所派生
{
    ...                               //与题目处相同,略
};

student::student(char* na, char sx, short ag, int num, char* dep)
{
    //派生类的构造函数，与其基类的初始化相关
    :person (na,sx,ag){
        //指明形参中的前三个是用来对基类 person 部分的共性成员进行初始化的，
        //既是说，先执行基类 person 的构造函数，完成基类对象的初始化，而后再
        //返回来执行本派生类的构造函数体。
        Reg_Number=num;
        //形参 num 带来 student 类中特性成员 Reg_Number 的初始值
        strcpy(department, dep);
        //形参 dep 带来 student 类中特性成员 department 的初始值
    }

void student::print ( ) const         //显示本 student 类的相关数据
{
    person::print( );                 //调用 person 类的公有函数成员 print(显示共性数据)
    cout <<"    Reg_Number: "<<Reg_Number<<endl;    //显示特性数据
    cout <<"    department: "<<department<<endl;    //显示特性数据
}

```

```

class teacher:public person          //自定义的 teacher 类(类型), 它由基类 person 所派生
{
    ...                              //与题目处相同,略
};

teacher::teacher(char* na,char sx,short ag,char* po,char* co)
    //派生类的构造函数, 与其基类的初始化相关
:person (na,sx,ag)
{
    //形参中的前三个, 对基类 person 部分的共性成员进行初始化
    strcpy(post,po);                  //职称
    strcpy(course,co);                //担任课程
    //后二参数 po 及 co 用于对本类的特性成员初始化
}

void teacher::print ( ) const        //显示本 teacher 类的相关数据
{
    person::print( );                //调用 person 类的公有函数成员 print(显示共性数据)
    cout << "    post: "<<post<<endl;
    cout << "    course: "<<course<<endl;    //显示本类特性成员
}

class graduateStudent:public student
{
    //自定义的 graduateStudent 类(类型), 它由基类 student 所派生
    ...                              //与题目处相同,略
};

graduateStudent::graduateStudent(char* na, char sx, short ag, int num, char* dep, char* ad)
:student(na,sx,ag,num,dep)
{
    //形参中的前五个, 对基类 student 部分的共性成员进行初始化
    strcpy(advisor,ad);              //第六参数 ad 用于对本类的特性成员 advisor 初始化
}

void graduateStudent::print ( ) const
{
    student::print( );               //调用 student 类的公有函数成员 print(显示共性数据)
}

```

```
        cout << "    advisor: "<<advisor<<endl;
    }

    void main()                //主函数，对所定义类进行使用
    {
        ...                    //与题目处相同，略
    }
```

[输出结果]

与题目处相同，略。

8.3 派生类的构造函数及其对象的初始化

读下面的程序，给出程序的执行结果（注意为派生类的构造函数及其对象的初始化）。

```
#include <iostream.h>

class Base1                                //注 1
{
    int d1;
public:
    Base1(int i)                            //注 2
    {
        d1=i;
        cout<<"constructing Base1 "<<d1<<endl;
    }
};

class Base2                                //注 3
{
    int d2;
public:
    Base2(int j)                            //注 4
    {
        d2=j;
        cout<<"constructing Base2 "<<d2<<endl;
    }
};

class Base3                                //注 5
```

```
{
    int d3;
public:
    Base3(int k=0)                                //注 6
    {
        d3=k;
        cout<<"constructing Base3 "<<d3<<endl;
    }
};

class Derived: public Base3, public Base1, public Base2    //注 7
{
    int d4;
    Base1 memberObj1;                                    //注 8
    Base3 memberObj3;
    Base2 memberObj2;
public:
    Derived(int a, int b, int c, int d, int e)
        :memberObj2(d),memberObj3(b),memberObj1(c),Base2(a),Base1(b)
                                                //注 9
    {
        d4=e;
        cout<<"constructing Derived "<<d4<<endl;
    }
};

void main( ) {
    Derived obj(1,2,3,4,5);                            //注 10
}
```

[分析提示]

派生类 `Derived` 由三个基类(`Base3`、`Base1` 和 `Base2`) 公有派生, 而且该派生类中还含有三个不同基类的类对象(`memberObj1`、`memberObj3` 和 `memberObj2`) 作为其数据成员。在这种情况下, 派生类的构造函数要负责对其基类数据成员、对其所含对象成员的数据成员以及该派生类新增的其他特有数据成员一块进行初始化 —— 均通过处于派生类构造函数体前的成员初始化符表来指定完成。而且总按照如下的一般次序来进行这种初始化: 先“ 父母”(基类, 并严格按照基类名的说明顺序 —— `Base3`、`Base1`、`Base2`, 依次去调用基类构造函数), 再“ 对象”(对象成员, 并严格按照各对象的说明顺序 —— `memberObj1`、`memberObj3`、`memberObj2`, 依次去调用基类构造函数), 后“ 自己”(本

派生类的构造函数)。注意,成员初始化符表中的书写顺序可以与上述的初始化执行次序不相同。

[解答程序]

具体程序见题目处。

注 1: 基类 Base1。

注 2: 基类构造函数, 有一个参数, 为数据成员置初值。

注 3: 基类 Base2。

注 4: 基类构造函数, 有一个参数, 为数据成员置初值。

注 5: 基类 Base3。

注 6: 基类构造函数, 有一个可缺省参数, 为数据成员置初值。

注 7: 派生类 Derived, 注意基类名的说明顺序(将依此顺序调用基类构造函数)。

注 8: 派生类的私有对象成员, 注意它们的说明顺序(将依此顺序调用基类构造函数)。

注 9: 派生类构造函数。构造派生类的对象时, 要对其基类数据成员、对其所含对象成员的数据成员以及本派生类的新增数据成员一块进行初始化。这种初始化工作是由派生类的构造函数来完成的——靠构造函数体前的成员初始化符表来指定具体初始化信息。而且按照如下的一般次序来进行: 先“ 父母”(基类), 再“ 对象”(成员对象), 后“ 自己”(本派生类构造函数)。另外注意, 对基类以及成员对象的初始化顺序是严格按照它们在程序中的说明顺序依次去进行的, 并不按照派生类的初始化符表中的顺序进行(本例的初始化符表顺序依次为: memberObj2、memberObj3、memberObj1、Base2、Base1, 缺少的 Base3 也进行了初始化, 只不过使用的是可缺省参数“ int k=0”)。

注 10: 说明派生类对象, 将去调用派生类构造函数。

[输出结果]

```
constructing Base3 0
constructing Base1 2
constructing Base2 1
constructing Base1 3
constructing Base3 2
constructing Base2 4
constructing Derived 5
```

8.4 派生类的构造与析构函数及其对象的初始化

读下面的程序, 给出程序的执行结果(注意点为派生类的构造与析构函数及其对象的初始化。在析构一个派生类对象时, 系统将按与构造派生类对象时的相反次序进行工作!)。

```
#include <iostream.h>

class Base1
{
    int d1;
public:
    Base1(int i)
    {
        d1=i;
        cout<<"constructing Base1 "<<d1<<endl;
    }
    ~Base1( ) //注 1
    {
        cout<<"destructing Base1 "<<endl;
    }
};

class Base2
{
    int d2;
public:
    Base2(int j)
    {
        d2=j;
        cout<<"constructing Base2 "<<d2<<endl;
    }
    ~Base2( ) //注 2
    {
        cout<<"destructing Base2 "<<endl;
    }
};

class Base3
{
    int d3;
public:
    Base3(int k=0)
    {
        d3=k;
        cout<<"constructing Base3 "<<d3<<endl;
```

```
    }
    ~Base3( )                                //注 3
    {
        cout<<"destructing Base3 "<<endl;
    }
};

class Derived: public Base3, public Base1, public Base2    //注 4
{
    int d4;
    Base1 memberObj1;                                    //注 5
    Base3 memberObj3;
    Base2 memberObj2;
public:
    Derived(int a, int b, int c, int d, int e)
        :memberObj2(d),memberObj3(b),memberObj1(c),Base2(a),Base1(b)
                                                //注 6
    {
        d4=e;
        cout<<"constructing Derived "<<d4<<endl;
    }
    ~Derived( )                                        //注 7
    {
        cout<<"destructing Derived "<<endl;
    }
};

void main( )
{
    Derived obj(1,2,3,4,5);                            //注 8
}
```

[分析提示]

派生类 **Derived** 由三个基类(**Base3**、**Base1** 和 **Base2**) 公有派生, 而且该派生类中还含有三个不同基类的类对象(**memberObj1**、**memberObj3** 和 **memberObj2**) 作为其数据成员。在这种情况下, 派生类的构造函数要负责对其基类数据成员、对其所含对象成员的数据成员以及该派生类新增的其他数据成员一块进行初始化。而且总按照如下的一般次序来进行初始化: 先“ 父母”(基类), 再“ 对象”(对象成员), 后“ 自己”(本派生类的构造函数)。注意, 在析构一个派生类对象时, 其工作次序恰与构造派生类对象时的次

序相反——先“自己”(本派生类构造函数), 再“对象”(成员对象), 后“父母”(基类)。

[解答程序]

具体程序见题目处。

注 1: Base1 的析构函数, Base1 的类对象“死亡”时对它进行调用。

注 2: Base2 的析构函数, Base2 的类对象“死亡”时对它进行调用。

注 3: Base3 的析构函数, Base3 的类对象“死亡”时对它进行调用。

注 4: 派生类 Derived, 注意基类名的说明顺序。

注 5: 派生类的私有对象。

注 6: 派生类构造函数。构造派生类的对象时, 要对其基类数据成员、对其所含对象成员的数据成员以及其他的新增数据成员一块进行初始化。这种初始化工作是由派生类的构造函数来完成的。而且按照如下的一般次序: 先“父母”(基类), 再“对象”(成员对象), 后“自己”(本派生类构造函数)。注意, 在析构一个派生类对象时, 其工作次序恰与构造派生类对象时的次序相反!

注 7: 派生类 Derived 的析构函数, Derived 的类对象“死亡”时对它进行调用。

注 8: 说明派生类 Derived 之对象 obj, 此时要调用派生类的构造函数。当对象 obj “死亡”时, 要调用派生类 Derived 的析构函数。

[输出结果]

```
constructing Base3 0
constructing Base1 2
constructing Base2 1
constructing Base1 3
constructing Base3 2
constructing Base2 4
constructing Derived 5
destructing Derived
destructing Base2
destructing Base3
destructing Base1
destructing Base2
destructing Base1
destructing Base3
```

8.5 使用类定义方式求解线性方程组

求解具有 lines 元的线性方程组(假设方程组具有惟一解, 使用列主元消去法)。有

如下几点要求:

- (1) 通过自定义类 `Linequ` (它派生于另一个自定义类 `Matrix`) 来实现求解问题。
- (2) 方程个数(即维数或行数) `lines` 之值由用户任意输入。
- (3) 方程组放于增广矩阵 `MatrA` 之中, 而 `MatrA`(`lines` 行 `lines+1` 列)的存储空间通过 `new` 语气来动态分配, 其各元素值也由用户任意输入(从而使指定的线性方程组也具有任意性)。
- (4) 方程组的解放在向量 `solution` 之中, 而 `solution`(具有 `lines` 个元素)的存储空间也通过 `new` 语句来动态分配。

下面给出实现方式之一的程序的核心代码(注意, 该程序的求解方法与范例 6.10 有关联性)。

```
class Matrix                                //基类 Matrix 声明
{
protected:                                //保护数据成员
    int    lines;                          //矩阵的维数(行数)lines, 列数为 lines+1
    double** MatrA;                        //数组首地址, 存放增广矩阵(系数矩阵+方程组右端向量)
        //而 MatrA(lines 行 lines+1 列)的存储空间是通过 new 语句来动态分配的
public:
    Matrix(int Lns);                       //构造函数, 动态分配存放增广矩阵 MatrA 的内存空间
    ~Matrix();                             //析构函数, 释放 MatrA 的动态空间
    void InMatrix();                       //输入增广矩阵初值
    void printMatrix();                    //显示增广矩阵
};

class Linequ: public Matrix                //公有派生类 Linequ 声明
{
    double *solution;                      //私有数据 solution, 方程组的解
public:
    Linequ(int Lns);                       //构造函数
    ~Linequ();                             //析构函数
    void InLinequ();                       //输入方程组初值
        //调用基类成员函数 InMatrix 来输入增广矩阵(即方程组)初值
    void printL();                          //显示方程组
        //调用基类成员函数 printMatrix 来显示增广矩阵(即方程组)
    void Solve();                          //列主元消去法
    void showX();                          //显示方程组的解
}

void main()                               //主函数
```

```

{
    int lines;
    cout<<"Input LINES:";    //屏幕提示输入矩阵行数(方程个数)
    cin>>lines;              //输入行数
    Linequ equ1(lines);
        //动态生成一个具有 lines 行的增广矩阵 MatrA(形参为维数)
        //并且动态分配 solution 空间存放方程组的解
    cout<<"Input"<<lines*(lines+1)<<"numbers that will form the Line_equation:"<<endl;
        //提示输入增广矩阵数据

    equ1.InLinequ( );        //输入增广矩阵数据
    equ1.printL( );          //输出方程组(以增广矩阵形式)
    equ1.Solve( );          //求解方程组
    equ1.showX( );          //输出方程组的解
}

```

[分析提示]

(1) 在自定义的基类 **Matrix** 之中,设有存放增广矩阵 系数矩阵 + 方程组右端向量) 的数据成员 **MatrA**(一个 **lines** 行 **lines+1** 列的动态数组); 在 **Matrix** 的构造函数中, 动态分配该 **MatrA** 的内存空间, 而在析构函数中, 释放该 **MatrA** 的动态空间; **Matrix** 的类成员函数 **InMatrix**, 负责输入增广矩阵 **MatrA** 的初值; 而另一个类成员函数 **printMatrix** 则用来显示出增广矩阵 **MatrA** 的各元素之值。

(2) 派生类 **Linequ** 继承自其基类 **Matrix**, 新增数据成员 **solution**(用来存放方程组的解), 并通过派生类构造函数为 **solution** 分配动态存储空间; 在派生类的成员函数 **InLinequ** 中, 通过调用其基类成员函数 **InMatrix** 来输入增广矩阵(即方程组) 的初值; 在 **printL** 中, 则通过调用基类成员函数 **printMatrix** 来显示出增广矩阵等; 派生类的 **Solve** 成员函数包含着列主元消去法的整个求解过程, 具体处理方法与范例 6.10 相类似。

[解答程序]

```

#include <iostream.h>
#include <iomanip.h>

class Matrix                                //基类 Matrix 声明
{
    ...                                    //与题目处相同, 略!
};

class Linequ: public Matrix                 //公有派生类 Linequ 声明
{
    ...                                    //与题目处相同, 略!
};

```

//基类 Matrix 的成员函数的实现(类外说明)

```
Matrix::Matrix(int Lns)                                //矩阵 Matrix 类的构造函数
{
    lines=Lns;                                         //保护数据赋值, 矩阵维数(行数)
    MatrA=new double*[lines];                         //动态内存分配(存放增广矩阵 MatrA)
    for(int i=0; i<lines; i++)
        MatrA[i]=new double[lines+1];
}
```

```
Matrix::~Matrix( )                                     //矩阵 Matrix 类的析构函数
{
    for(int i=0; i<lines; i++)
        delete[ ] MatrA[i];                          //释放内存
    delete[ ] MatrA;                                  //释放内存
}
```

```
void Matrix::InMatrix( ) //输入增广矩阵初值, 它表示欲求解的线性方程组
{
    int i,j;
    for(i=0; i<lines; i++)                            //共 lines 行
        for(j=0; j<lines+1; j++)                      //每行 lines+1 列
            cin>>MatrA[i][j];                        //输入数据
}
```

```
void Matrix::printMatrix( )                           //显示增广矩阵的元素
{
    for(int i=0; i<lines; i++)                         //共显示 lines 行
    {
        for(int j=0; j<lines+1; j++)                  //每行显示 lines+1 个数
            cout<<setw(10)<<MatrA[i][j]<<" ";        //每数占 10 格(宽度)
        cout<<endl;
    }
}
```

//派生类 Linequ 的成员函数的实现(类外说明)

```
Linequ::Linequ(int Lns):Matrix(Lns)                  //派生类 Linequ 的构造函数
{
    //使用参数 Lns 作为实参调用基类构造函数, 动态分配盛放增广矩阵 MatrA 的空间
```

```

        solution=new double[Lns];                //动态内存分配，solution 存放方程组的解
    }

    Linequ::~~Linequ( )                          //派生类 Linequ 的析构函数
                                                //会自动调用基类析构函数
    {
        delete[ ] solution;                      //释放内存
    }

    void Linequ::InLinequ( )                    //输入方程组初值
    {
        InMatrix( );                            //调用基类成员函数 InMatrix 来输入增广矩阵(即方程组)初值
    }

    void Linequ::printL( )                      //显示线性方程组
    {
        cout<<"The Line_equation is:"<<endl;
        cout<<"-----"<<endl;
        printMatrix( );                        //调用基类成员函数 printMatrix 来显示增广矩阵(即方程组)
    }

    void Linequ::showX( )                      //输出方程组的解
    {
        cout<<"-----"<<endl;
        cout<<"The Result is:"<<endl;
        cout<<"-----"<<endl;
        for(int i=0; i<lines; i++)              //显示 solution[i]的各值
            cout<<"X["<<i<<"]="<<solution[i]<<endl;
    }

    void Linequ::Solve( )                      //使用列主元消去法求解
    {
        int k,i,j;
        double d,t;
        for (k=1; k<lines; k++)                //消去过程
        {
            //先在第 k-1 行到 lines-1 行的范围内进行行互换，使 MatrA[k-1][k-1]
            //在第 k-1 列中为最大元，而后再进行消元过程
            double max=MatrA[k-1][k-1];

```

```

int ind=k-1;
for (i=k; i<lines; i++)           //找列中最大元所在行 ind
    if(MatA[i][k-1]>max)
    {
        max=MatA[i][k-1];
        ind=i;
    }
if(ind!=k-1)                       //第 k-1 行与第 ind 行进行整行互换
    for (j=k-1; j<lines+1; j++)
    {
        double tmp=MatA[k-1][j];
        MatA[k-1][j]=MatA[ind][j];
        MatA[ind][j]=tmp;
    }
for (i=k; i<lines; i++)           //将系数矩阵消为上三角阵
{
    double factor=-MatA[i][k-1]/MatA[k-1][k-1];
    for (j=k-1; j<lines+1; j++)
        MatA[i][j]+=MatA[k-1][j]*factor;
}
}                                   //消去过程结束
d=MatA[lines-1][lines-1];
solution[lines-1]=MatA[lines-1][lines]/d; //回代过程
for (i=lines-2; i>=0; i--)
{
    t=0.0;
    for (j=i+1; j<=lines-1; j++)
        t=t+MatA[i][j]*solution[j];
    solution[i]=(MatA[i][lines]-t)/MatA[i][i];
    //方程解存放于 solution[i]之中
}
}

void main( )                       //主函数
{
    ...                             //与题目处相同，略!
}

```

[输出结果]

Input LINES:2

Input 6 numbers that will form the Line_equation:

1 2 3 4 5 6

The Line_equation is:

```
-----
          1          2          3
          4          5          6
-----
```

The Result is:

```
-----
X[0]=-1
X[1]=2
```

再一次执行:

Input LINES:4

Input 20 numbers that will form the Line_equation:

0.2815 0.2271 0.2018 1.4396 12.1126

0.1754 0.1958 1.5211 0.5893 25.5612

0.1788 1.3366 0.1832 0.9235 26.8501

1.9876 0.1234 0.2628 0.8821 52.2683

The Line_equation is:

```
-----
    0.2815    0.2271    0.2018    1.4396    12.1126
    0.1754    0.1958    1.5211    0.5893    25.5612
    0.1788    1.3366    0.1832    0.9235    26.8501
    1.9876    0.1234    0.2628    0.8821    52.2683
-----
```

The Result is:

```
-----
X[0]=23.9054
X[1]=15.5057
X[2]=12.2142
X[3]=-0.418835
```

8.6 通过组合方式自定义日期时间类(类型)DateTimeType

自定义一个如下的日期时间类型 `DateTimeType`，它含有类 `DateType`(参看范例 7.18

的具体定义)与类 `TimeType`(参看范例 7.16 的具体定义)的类对象作为其数据成员,并具有所列的其他几个成员函数。而后编制主函数,说明 `DateTimeType` 的类对象,对该类的成员函数以及两个对象成员所属类的成员函数(例如对日期的输入输出、计算两个日期的间隔天数、计算某日期为星期几、打印对象当前数据的 y 年 m 月的月历、对两个日期进行六种比较、对时间的输入输出、对时分秒的增加、计算两个时刻的间隔等)进行使用。

```
class DateTimeType      //自定义的日期时间类型 DateTimeType
{
    //它含有类 DateType 与类 TimeType 的类对象作为其数据成员
    DateType date;      //类 DateType 的类对象 date 作为其数据成员
    TimeType time;      //类 TimeType 的类对象 time 作为其另一个数据成员
public:
    DateTimeType(int y0, int m0, int d0, int hr0, int mi0, int se0);
        //六参构造函数, 设定 DateTimeType 类对象的日期时间
        //(年、月、日、时、分、秒)

    DateTimeType();
        //无参构造函数, 将 DateTimeType 类对象的日期时间设为
        //1 年 1 月 1 日的 0 时 0 分 0 秒

    DateType& getDate() { return date; } //通过该函数返回本类的私有数据对象 data
        //而后在类外如主函数处才可通过返回对象去使用
        //DateType 类的公有成员函数(或公有数据成员)

    TimeType& getTime() { return time; } //通过该函数返回本类的私有数据对象 time
        //而后在类外如主函数处才可通过返回对象去使用
        //TimeType 类的公有成员函数(或公有数据成员)

    void incrementSecond(int s);
        //增加若干秒, 注意进位问题(可能涉及到天的数据)

    void inputDateTime();
        //键盘输入日期时间对象的有关数据(年、月、日、时、分、秒)

    void printDateTime();
        //屏幕输出日期时间对象的有关数据(年、月、日、时、分、秒)

}
```

[分析提示]

当类的数据成员为另一个类的类对象时,称为类的组合(关系)。组合意味着包含,是指 A 类类对象(组合类对象)中总含有一个 B 类类对象(对象成员)。有时也称被包含的对象为子对象,它们属于整体与部分的关系。

若类间具有组合关系,使用它们时有两点需要注意。一是注意构造和析构函数的执行次序以及组合类构造函数应负有的责任——要对所包含的每一个对象成员的初始化负责。二是注意,通过组合类对象调用其对象成员的公有成员函数(或公有数据成员)

时, 必须使用如下的调用方式: <组合类对象>.<对象成员>.<对象成员所属类的公有成员>。

另外, 如下的两种赋值操作都是不允许的, 因为它们的类型不匹配, 属于整体与部分的关系, 不可相互赋值:

<对象成员所属类的对象> = <组合类对象>;

<组合类对象> = <对象成员所属类的对象>;

[解答程序]

```
#include "DateType.h"
```

```
/*
```

该头文件中自定义一个简单的日期类型 `DateType`, 其中说明了多个类成员函数, 使用该头文件可以完成日期的输入、对天的增加、计算两个日期的间隔天数、计算某日期为星期几、打印对象当前数据的 y 年 m 月的月历、对两个日期进行六种比较、以及对日期的输出等操作。该头文件与范例 7.18 的头文件相同, 本题略。

```
*/
```

```
#include "TimeType.h"
```

```
/*
```

该头文件中自定义简单的时间类型 `TimeType`, 其中说明了多个类成员函数, 使用该头文件可以完成时间的输入、对时分秒的增加、计算两个时间的间隔、对两个时间进行六种比较、以及对时间的输出等操作。该头文件与范例 7.16 的头文件相同, 本题略。

```
*/
```

```
class DateTimeType          //自定义的日期时间类型 DateTimeType
```

```
{
```

```
    ...                    //与题目处相同, 略
```

```
};
```

```
DateTimeType::DateTimeType(int y0, int m0, int d0, int hr0, int mi0, int se0)
```

```
:date(y0,m0,d0), time(hr0, mi0, se0) { }
```

//六参构造函数, 设定 `DateTimeType` 对象的日期时间(年、月、日、时、分、秒)

//注意, 本类中包含有对象成员 `date` 与 `time`, 构造函数处要对其所包含的每一个对象

//成员的初始化负责。实际上是通过基类的构造函数将参数数据设置到各数据成员处的。

```
DateTimeType::DateTimeType( )
```

```
:date(1,1,1), time(0,0,0) { }
```

//无参构造函数, 将 `DateTimeType` 类对象的 `date` 与 `time` 的日期时间设为 1 年 1 月

//1 日的 0 时 0 分 0 秒。若某类中包含有对象成员的话, 则该类的构造函数就要对其

//所包含的每一个对象成员的初始化负责。实际上是通过基类的构造函数将 1 年 1 月

//1 日的 0 时 0 分 0 秒的对应数据设置到各数据成员处的。

```

void DateTimeType::inputDateTime( )
{
    //键盘输入日期时间对象的有关数据(年、月、日、时、分、秒)
    date.inputDate( );           //输入类对象的日期数据(年、月、日)
    //必须通过本类的私有数据对象 data
    //才可以去调用 DateTimeType 类的公有成员函数 inputDate
    time.inputTime( );           //输入类对象的时间数据(时、分、秒)
    //必须通过本类的私有数据对象 time 才可去调用
    //TimeType 类的公有成员函数 inputTime
}

void DateTimeType::printDateTime( )
{
    //屏幕输出日期时间对象的有关数据(年、月、日、时、分、秒)
    cout<<"(year:month:day) -- ";
    date.printDate( );           //输出类对象的日期数据(年、月、日)
    //必须通过本类的私有数据对象 data
    //才可去调用 DateTimeType 类的公有成员函数 printDate
    cout<<"(hour:minute:second) -- ";
    time.printTime( );           //输出类对象的时间数据(时、分、秒)
    //必须通过本类的私有数据对象 time 才可去调用
    //TimeType 类的公有成员函数 printTime
}

void DateTimeType::incrementSecond(int s) //增加若干秒, 注意进位问题
{
    int oldHour=time.getHou( );
    //通过 TimeType 类提供的公有函数 getHou 来获取其私有成员 h 的值
    time.incrementSec(s);        //调用 TimeType 类的公有成员函数 incrementSec
    if(oldHour==23 && time.getHou( )==0)
        date.incrementDay( ); //进位到天
    //通过数据对象 data 调用 DateTimeType 类的公有成员函数 incrementDay
}

void main(void) //主函数, 说明 DateTimeType 的类对象, 对各成员函数进行使用
{
    DateTimeType dttm1, dttm2(1999,12,31,23,59,59), dttm3;

```

```

cout<<"(dtm1.getDate( )).printDate( );=>";
//dtm1.date.printDate( );
//语法错误! 通过 dtm1.date 来存取 DateTimeType 类的私有数据成员!
(dtm1.getDate( )).printDate( ); //输出类对象 dtm1 的日期数据(年、月、日)
//必须先通过 dtm1.getDate( )调用返回 DateTimeType 类的私有数据对象 data
//而后在类外(如主函数处)才可通过返回对象去使用 DateType 类的
//公有成员函数(或公有数据成员)
cout<<"(dtm1.getTime( )).printAmPm( );=>";
(dtm1.getTime( )).printAmPm( );
//调用 TimeType 类的公有成员函数 printAmPm
//以 Am(上午)或 Pm(下午)的格式在屏幕上输出类对象 dtm1 的时间数据
//(时、分、秒)。必须先通过 dtm1.getTime( )调用返回 DateTimeType
//类的私有数据对象 time, 而后在类外如主函数处才可通过返回对象
//去使用 TimeType 类的公有成员函数(或公有数据成员)
cout<<"dtm1.printDateTime( );=>"<<endl;
dtm1.printDateTime( ); //调用本派生类的 printDateTime
//输出类对象 dtm1 的时间数据(年、月、日、时、分、秒)
cout<<"dtm2.printDateTime( );=>"<<endl;
dtm2.printDateTime( ); //调用本派生类的 printDateTime
//输出类对象 dtm2 的时间数据(年、月、日、时、分、秒)
cout<<"dtm2.incrementSecond(30); dtm2.printDateTime( );=>"<<endl;
dtm2.incrementSecond(30); //调用本派生类的 incrementSecond, 增加 30 秒
dtm2.printDateTime( );
//输出类对象 dtm2 的日期时间数据(年、月、日、时、分、秒)
cout<<"(dtm3.getDate( )).inputDate( );=>"<<endl;
(dtm3.getDate( )).inputDate( );
//调用 DateTimeType 类的 inputDate 为类对象 dtm3 输入日期数据(年、月、日)
//必须先通过 dtm3.getDate( )调用返回 DateTimeType 类的私有数据对象 data
cout<<"whatDay of dtm3.date="<<((dtm3.getDate( )).whatDay( ))<<endl;
//显示 dtm3 的日期为星期几, 通过 dtm3.getDate( )调用返回私有数据对象 data
}

```

[输出结果]

```

(dtm1.getDate( )).printDate( );=>1:1:1
(dtm1.getTime( )).printAmPm( );=>0:0:0(AM)
dtm1.printDateTime( );=>
(year:month:day) -- 1:1:1
(hour:minute:second) -- 0:0:0
dtm2.printDateTime( );=>

```

```

(year:month:day) -- 1999:12:31
(hour:minute:second) -- 23:59:59
dttm2.incrementSecond(30); dttm2.printDateTime( ); ==>
(year:month:day) -- 2000:1:1
(hour:minute:second) -- 0:0:29
(dtmm3.getDate( )).inputDate( ); ==>
Input date(y(1->...) m(1->12) d(1->31)):2003 1 1
whatDay of dttm3.date=3

```

8.7 通过继承方式自定义日期时间类(类型)DateTimeType

自定义一个如下的日期时间类型 `DateTimeType`，它由基类 `DateType`（参看范例 7.18 的具体定义）与 `TimeType`（参看范例 7.16 的具体定义）二者公有派生。并编制主函数，说明派生类对象，并对派生类的成员函数以及二基类的成员函数（例如对日期的输入输出、计算二日期的间隔天数、计算某日期为星期几、打印对象当前数据的 *y* 年 *m* 月的月历、对两个日期进行 6 种比较、对时间的输入输出、对时分秒的增加、计算二时刻的间隔等）进行使用。

```

class DateTimeType : public DateType, public TimeType
{
public:
    DateTimeType(int y0, int m0, int d0, int hr0, int mi0, int se0);
    //六参构造函数，设定派生类对象的日期时间(年、月、日、时、分、秒)
    DateTimeType( );
    //无参构造函数，将派生类对象的日期时间设为 1 年 1 月 1 日的 0 时 0 分 0 秒
    void incrementSecond(int s);
    //增加若干秒，注意进位问题(可能涉及到天的数据)
    void inputDateTime( );
    //键盘输入日期时间对象的有关数据(年、月、日、时、分、秒)
    void printDateTime( );
    //屏幕输出日期时间对象的有关数据(年、月、日、时、分、秒)
};

```

[分析提示]

(1) 虽然没在派生类 `DateTimeType` 中说明数据成员，但由于公有继承了基类 `DateType` 与 `TimeType`，所以本派生类中包含了二基类的所有数据成员（虽然不能直接存取它们——因为它们都为基类的私有数据成员，但可通过基类提供的公有函数如 `getHou` 等来获取它们的值，也可通过像 `setDate` 的函数为它们设置新值。另外注意，如果在基类中将数据成员说明为 `protected` 类型的，则派生类中就可以直接存取它们了）。

(2) 注意, 通过继承产生的派生类对象中总具有基类的类对象(的所有成员), 意味着派生类类对象必然是一个基类类对象。

若类间具有继承关系, 使用它们时有两点需要注意。一是注意构造和析构函数的执行次序以及派生类构造函数应负有的责任——要对其直接基类的初始化负责。二是注意, 由于派生类类对象必然是一个基类类对象, 通过派生类对象, 也就可以直接调用 (或存取) 其基类的公有或保护成员函数 (或公有及保护数据成员)。如最常用的调用方式为: <派生类对象>.<基类的公有或保护成员>。又比如, 如下的赋值操作是允许的: <基类对象> = <派生类对象>; 因为派生类对象必然是一个基类类对象, 它包含着基类对象所需要的一切数据(另外还有富余, 但富余部分被甩掉不进行赋值)。但反方向的赋值则不被允许。

[解答程序]

```
#include "DateType.h"

/*
    该头文件中自定义一个简单的日期类型 DateType, 其中说明了多个类成员函数, 使用该
    类可以完成对日期的输入、对天的增加、计算二日期的间隔天数、计算某日期为星期几、
    打印对象当前数据的 y 年 m 月的月历、对两个日期进行 6 种比较、以及对日期的输出等操
    作。该头文件与范例 7.18 的头文件相同, 本题略。
*/

#include "TimeType.h"

/*
    该头文件中自定义一个简单的时间类型 TimeType, 其中说明了多个类成员函数, 使用
    该类可以完成时间的输入、对时分秒的增加、计算二时刻的间隔、对两个时刻进行 6 种比
    较、以及对时间的输出等操作。该头文件与范例 7.16 的头文件相同, 本题略。
*/

class DateTimeType : public DateType, public TimeType
{
    ...                //与题目处相同, 略
};

DateTimeType::DateTimeType(int y0, int m0, int d0, int hr0, int mi0, int se0)
:DateType(y0,m0,d0), TimeType(hr0, mi0, se0) { }

    //六参构造函数, 设定派生类对象的日期时间(年、月、日、时、分、秒)
    //派生类的构造函数要对其直接基类 DateType 和 TimeType 的初始化负责
    //实际上是通过基类的构造函数将参数数据设置到各数据成员处的

DateTimeType::DateTimeType( ) DateType( ), TimeType( ){ }

    //无参构造函数, 将派生类对象的日期时间设为 1 年 1 月 1 日的 0 时 0 分 0 秒
```

//实际上是通过基类的无参构造函数将 1 年 1 月 1 日的 0 时 0 分 0 秒的
//对应数据设置到各数据成员处的

```
void DateTimeType::inputDateTime( )
{
    //键盘输入日期时间对象的有关数据(年、月、日、时、分、秒)
    inputDate( );    //调用基类的 inputDate, 输入类对象的日期数据(年、月、日)
    //通过派生类对象, 可以调用其基类的公有成员函数
    inputTime( );    //调用基类的 inputTime, 输入类对象的时间数据(时、分、秒)
}

void DateTimeType::printDateTime( )
{
    //屏幕输出日期时间对象的有关数据(年、月、日、时、分、秒)
    cout<<"(year:month:day) -- ";
    printDate( );    //调用基类的 printDate, 输出类对象的日期数据(年、月、日)
    //通过派生类对象, 可以调用其基类的公有成员函数
    cout<<"(hour:minute:second) -- ";
    printTime( );    //调用基类的 printAmPm, 输出类对象的时间数据(时、分、秒)
}

void DateTimeType::incrementSecond(int s)    //增加若干秒, 注意进位问题
{
    int oldHour=getHou( );
    //通过基类提供的公有函数 getHou 来获取基类私有成员 h 的值
    incrementSec(s); //调用其基类的公有成员函数
    if(oldHour==23 && getHou( )==0)
        incrementDay( ); //进位到天, 调用其基类的公有成员函数
    /*
    注意, 如果在基类 TimeType 中将数据成员 h 说明成 protected 类型的, 则上述的
    几行代码也可进行如下的改写(从而可对 h 进行直接存取)。
    int oldHour=h;
    incrementSec(s);
    if(oldHour==23 && h==0)
        incrementDay( );
    */
}

void main(void)    //主函数, 说明派生类对象, 对各成员函数进行使用
```

```

{
    DateTimeType dttm1, dttm2(1999,12,31,23,59,59), dttm3;
    cout<<"dttm1.printDate( );=>";
    dttm1.printDate( );
        //调用基类的 printDate, 输出类对象 dttm1 的日期数据(年、月、日)
    cout<<"dttm1.printAmPm( );=>";
    dttm1.printAmPm( );        //调用基类的 printAmPm
        //以 Am(上午)或 Pm(下午)的格式在屏幕上输出
        //类对象 dttm1 的时间数据(时、分、秒)
    cout<<"dttm1.printDateTime( );=>"<<endl;
    dttm1.printDateTime( );    //调用本派生类的 printDateTime
        //输出类对象 dttm1 的日期时间数据(年、月、日、时、分、秒)
    cout<<"dttm2.printDateTime( );=>"<<endl;
    dttm2.printDateTime( );    //调用本派生类的 printDateTime
        //输出类对象 dttm2 的日期时间数据(年、月、日、时、分、秒)
    dttm2.incrementSecond(30);
        //调用本派生类的 incrementSecond, 增加 30 秒
    cout<<"dttm2.incrementSecond(30); dttm2.printDateTime( );=>"<<endl;
    dttm2.printDateTime( );
        //输出类对象 dttm2 的日期时间数据(年、月、日、时、分、秒)
    cout<<"dttm3.inputDate( );=>"<<endl;
    dttm3.inputDate( );
        //调用基类的 inputDate 为类对象 dttm3 输入日期数据(年、月、日)
    cout<<"whatDay of dttm3="<<dttm3.whatDay( )<<endl;
        //显示 dttm3 的日期为星期几
}

```

[输出结果]

```

dttm1.printDate( );=>1:1:1
dttm1.printAmPm( );=>0:0:0(AM)
dttm1.printDateTime( );=>
(year:month:day) -- 1:1:1
(hour:minute:second) -- 0:0:0
dttm2.printDateTime( );=>
(year:month:day) -- 1999:12:31
(hour:minute:second) -- 23:59:59
dttm2.incrementSecond(30); dttm2.printDateTime( );=>
(year:month:day) -- 2000:1:1
(hour:minute:second) -- 0:0:29

```

```
dtm3.inputDate( ); =>
Input date(y(1->...) m(1->12) d(1->31)):2003 1 1
whatDay of dtm3=3
```

8.8 虚函数的使用

下面的程序中使用了虚函数，请给出程序的执行结果（假设执行时对 i 变量的输入依次为：11， 2， 12， 0， 3）。

```
#include <iostream.h>

class B0
{
public:
    virtual void print( )                //注 1
    {
        cout<<"B0::print( )" << endl;
    }
};

class B11: public B0
{
public:
    virtual void print( )                //注 2
    {
        cout<<"B11::print( )" << endl;
    }
};

class B12: public B0
{
public:
    virtual void print( )                //注 2
    {
        cout<<"B12::print( )" << endl;
    }
};

class B2: public B0
{

```

```
public:
    virtual void print( )                //注 2
    {
        cout<<"B2::print( )"<<endl;
    }
};

void main( )
{
    B0 b0, *p;                          //注 3
    B11 b11;                             //注 4
    B12 b12;      B2 b2;      int i;
    cout<<"Input i ( 0, 11, 12, 2, else ending! ):";
    cin>>i;                              //注 5
    while( i==0 || i==11 || i==12 || i==2 ) //注 6
    {
        switch (i)
        {
            case 0:
                p=&b0;                    //注 7
                break;
            case 11:
                p=&b11;                   //注 8
                break;
            case 12:
                p=&b12;                   //注 9
                break;
            case 2:
                p=&b2;                    //注 10
                break;
        }    //switch 结束
        p->print( );                     //注 11
        cout<<"Input i ( 0, 11, 12, 2, else ending! ):";
        cin>>i;                          //注 12
    }    //while 结束
}
```

[分析提示]

动态联编(dynamic binding)是与虚函数以及程序中使用指向基类的指针(变量)密切

相关的。注意，C++规定，基类指针可以指向其派生类的对象（即可将派生类对象的地址赋给其基类指针变量），但反过来不可以。允许基类指针指向其派生类对象——这一点正是函数重载及虚函数用法的基础。本范例程序中，说明了一个指向基类的指针 `p`，而后通过 `p = 某派生类的类对象地址;` 使 `p` 指向了不同的派生类，紧接着使用 `p->print();` 去对虚函数 `print()` 进行调用，系统将视基类指针 `p` 所指对象的不同而调用不同类的 `print`。

实际上，在编译阶段，系统发现要调用的是一个虚函数 `print`，而此时它又无法确定，程序究竟要调用哪一个派生类的重载函数（虚函数）`print`（因为依赖于运行时 `p` 指针的动态取值！）。此种情况下，系统将采用动态联编方式来处理：在运行阶段，通过 `p` 指针的当前值，去动态地确定出它所指向的那一个对象所属的类（基类或某一派生类），而后，再去找到该类的那一个虚函数（的函数体代码，进而去执行）。

[解答程序]

具体程序见题目处。

注 1：在基类中说明了一个虚函数 `print`。

注 2：派生类中也使用虚函数 `print`，其关键字 `virtual` 可省。

注 3：基类对象 `b0`，及指向基类的指针 `p`。

注 4：派生类对象 `b11`，`b12` 以及 `b2`。

注 5：输入 `i`（根据不同的 `i`，使基类指针 `p` 指向不同的类对象）。

注 6：`i=0` 要使用类对象 `b0`，`i=11` 要使用类对象 `b11`，`i=12` 要使用类对象 `b12`，`i=2` 要使用类对象 `b2`。

注 7：指向基类的指针 `p = 基类对象 b0 的地址;`

注 8：指向基类的指针 `p = 派生类对象 b11 的地址;`

注 9：指向基类的指针 `p = 派生类对象 b12 的地址;`

注 10：指向基类的指针 `p = 派生类对象 b2 的地址;`

注 11：对虚函数 `print()` 进行调用，系统将视基类指针 `p` 所指对象的不同而调用不同类的 `print`。

注 12：输入下一个 `i`，继续 `while` 循环。

[输出结果]

```
Input i ( 0, 11, 12, 2, else ending! ):11
```

```
B11::print()
```

```
Input i ( 0, 11, 12, 2, else ending! ):2
```

```
B2::print()
```

```
Input i ( 0, 11, 12, 2, else ending! ):12
```

```
B12::print()
```

```
Input i ( 0, 11, 12, 2, else ending! ):0
```

```
B0::print()
```

```
Input i ( 0, 11, 12, 2, else ending! ):3
```

8.9 通过虚函数手段按不同方法来计算定积分

修改教材 8.4.1 节定义的 `inte_algo` 类和它的几个派生类，对于下面的几个函数，采用矩形法、梯形法以及辛甫生算法来计算同一函数的定积分值并输出这些积分值：

$f_1(x)=\sin(x)+2*\cos(x)$; 积分区间(0~PI/2)

$f_2(x)=2.5*(1+x*x)/(1+x*x*x)$; 积分区间(0~1)

$f_3(x)=x*x/2.0$; 积分区间(0~1)

下面给出实现方式之一的程序的核心代码。

```
class inte_algo          //自定义的基类 inte_algo
{
protected:
    double a,b;          //a, b 为积分区间的左右边界
    int n;                //n 表示把[a, b]区间划分成多少个小小区段进行积分
    double (*f)(double x); //函数指针 f, 表示欲积分的函数
    double h,sum;         //h 表示步长, sum 表示积分结果值
public:
    inte_algo (double left, double right, int steps, double (*func)(double x))
    {
        //inte_algo 类的构造函数, 由参数带来的 left, right 将赋给
        //表示积分区间左右边界的 a 与 b, 而将 steps 赋给表示划分多少个
        //小区段的 n, 将 func 赋给表示积分函数的指针 f
        a=left;          b=right;          n=steps;          f=func;
        h=(b-a)/n;        //算出步长 h
        sum=0.0;          //sum 初值为 0
    }
    virtual void integrate(void); //基类 inte_algo 中说明了一个虚函数 integrate
};

class rectangle:public inte_algo
{
    //由基类 inte_algo 派生出的 rectangle 类, 其中也说明同一个虚函数 integrate
public:
    rectangle(double left, double right, int steps, double (*func)(double x))
        //rectangle 的构造函数, 要对基类 inte_algo 的初始化负责
        :inte_algo (left,right,steps,func)
    {
    }
}
```

```

        virtual void integrate(void); //派生类中说明同一个虚函数 integrate
    };

    .....                                //其他部分略

void f(rectangle rec, ladder lad, simpson sim)
{
    //自定义函数 f, 具有三个不同的类对象作为形参, 将对不同类对象进行积分
    inte_algo *p1, *p2, *p3;           //设三个指向基类 inte_algo 的指针变量
    p1=&rec, p2=&lad, p3=&sim;          //三个基类指针(变量)指向了三个不同派生
        //类的类对象使随后的通过基类指针(变量)对 integrate 函数的调用
        //将对应于不同派生类的 integrate
    cout<<"rectangle method:  sum=>";
    p1->integrate( );                  //矩形法
    cout<<"ladder method:  sum=>";
    p2->integrate( );                  //梯形法
    cout<<"simpson method:  sum=>";
    p3->integrate( );                  //辛甫虫法
}

void main( )
{
    int steps;
    cout<<"steps=? ";
    cin>>steps;
    const double PI=3.1415926;
    rectangle rec1(0.0, PI/2.0, steps, f1);
        //rectangle 类对象 rec1, [0.0, PI/2.0]区间,
        //分为 steps 等份算积分, 对 f1 函数积分
    ladder lad1(0.0, PI/2.0, steps, f1);
        //ladder 类对象 lad1, [0.0, PI/2.0]区间,
        //分为 steps 等份算积分, 对 f1 函数积分
    simpson sim1(0.0, PI/2.0, steps, f1);
        //simpson 类对象 sim1, [0.0, PI/2.0]区间,
        //分为 steps 等份算积分, 对 f1 函数积分
    cout<<endl<<"==== f1(x)=sin(x)+2*cos(x); (0=>PI/2) ===== "<<endl;
    f(rec1, lad1, sim1);
        //调用自定义函数 f, 代入三个不同的类对象(作为实参),

```

```

//将对不同类对象进行积分
rectangle rec2(0.0, 1.0, steps, f2);
ladder lad2(0.0, 1.0, steps, f2);
simpson sim2(0.0, 1.0, steps, f2);
cout<<endl<<"===== $f_2(x)=2.5*(1+x*x)/(1+x*x*x)$ ; (0=>1)====="<<endl;
f(rec2, lad2, sim2);
rectangle rec3(0.0, 1.0, steps, f3);
ladder lad3(0.0, 1.0, steps, f3);
simpson sim3(0.0, 1.0, steps, f3);
cout<<endl<<"===== $f_3(x)=x*x/2.0$ ; (0=>1)====="<<endl;
f(rec3, lad3, sim3);
}

```

程序执行后的显示结果如下:

steps=? 1000

===== $f_1(x)=\sin(x)+2*\cos(x)$; (0=>PI/2)=====

rectangle method: sum=>3.00078

ladder method: sum=>3.00157

simpson method: sum=>3

===== $f_2(x)=2.5*(1+x*x)/(1+x*x*x)$; (0=>1)=====

rectangle method: sum=>2.66674

ladder method: sum=>2.66799

simpson method: sum=>2.66674

===== $f_3(x)=x*x/2.0$; (0=>1)=====

rectangle method: sum=>0.166417

ladder method: sum=>0.166667

simpson method: sum=>0.166667

[分析提示]

(1) 与教材 8.4.1 节定义的 `inte_algo` 类相比, 本范例对 `inte_algo` 类的构造函数功能进行了扩充——多设立了一个参数“`double (*func)(double x)`”, 从而在创建类对象时可以任意指定不同的被积分函数 `func`。

(2) 在基类 `inte_algo` 所派生出的 `rectangle`、`ladder` 以及辛甫生算各派生类之中, 均说明了同一个虚函数 `integrate`, 通过它们来实现三种不同的积分方法(矩形法、梯形法、辛甫生法)。并在程序的 `f` 函数中说明了三个指向基类 `inte_algo` 的指针 `p1`、`p2` 与 `p3`, 而后通过赋值操作, 使这三个基类指针(变量)指向了三个不同的派生类对象(由函数

f 的实参带来欲进行积分的不同派生类对象), 这将使随后的通过基类指针(变量) 对 `integrate` 函数的调用分别对应于不同派生类的 `integrate` 积分函数。

[解答程序]

```
#include <iostream.h>
#include <math.h>
double f1(double x)                //欲积分的函数 f1
{
    return (sin(x)+2*cos(x));
}

double f2(double x)                //欲积分的函数 f2
{
    return 2.5*(1+x*x)/(1+x*x*x);
}

double f3(double x)                //欲积分的函数 f3
{
    return (x*x/2.0);
}

class inte_algo                    //自定义的基类 inte_algo
{
    ...                            //与题目处相同,略
};

class rectangle:public inte_algo
{
    ...                            //与题目处相同,略
};

class ladder:public inte_algo
{
    //由基类 inte_algo 派生出的 ladder 类, 其中也说明同一个虚函数 integrate
public:
    ladder(double left, double right, int steps, double (*func)(double x))
        //ladder 的构造函数, 要对基类 inte_algo 的初始化负责
        :inte_algo (left,right,steps,func)
    {
```

```

    }
    virtual void integrate(void);    //派生类中说明同一个虚函数 integrate
};

class simpson:public inte_algo
{
    //由基类 inte_algo 派生出的 simpson 类，其中也说明同一个虚函数 integrate
public:
    simpson(double left, double right, int steps, double (*func)(double x))
        //simpson 的构造函数，要对基类 inte_algo 的初始化负责
        :inte_algo (left,right,steps,func)
    {
    }
    virtual void integrate(void);    //派生类中说明同一个虚函数 integrate
};

void inte_algo::integrate( )
{
    //基类 inte_algo 之虚函数 integrate 的类外定义
    cout<<sum<<endl;
}

void rectangle::integrate( )
{
    //派生类 rectangle 之虚函数 integrate 的类外定义，
    //采用矩形法来计算函数的定积分。
    //计算公式为: sum=( f(a)+f(a+h)+f(a+2h)+...+f(a+(n-1)h) )h
    double al=a; //al 为调用 f 函数时的实参值，依次取值 a, a+h, a+2h, ... , a+(n-1)h
    for(int i=0; i<n; i++)    //共在 n 个点处计算函数值
    {
        sum+=f(al);
        al+=h;    //al 每次增加一个步长 h
    }
    sum*=h;
    cout<<sum<<endl;    //显示积分结果 sum
}

void ladder::integrate( )
{

```

//派生类 ladder 之虚函数 integrate 的类外定义, 采用梯形法来计算函数的定积分

//计算公式为: $\text{sum}=(f(a)+2f(a+h)+2f(a+2h)+\dots+2f(a+(n-1)h)+f(b))h/2$

double al=a; //al 为调用 f 函数时的实参值, 依次取值 a, a+h, a+2h, ... , a+(n-1)h

sum=(f(a)+f(b))/2.0; //对边界点 a、b 特殊处理

for(int i=1;i<n;i++) //还要在 n-1 个点处计算函数值

{

al+=h; //al 每次增加一个步长 h

sum+=f(al);

}

sum*=h;

cout<<sum<<endl; //显示积分结果 sum

}

void simpson::integrate()

{

//派生类 simpson 之虚函数 integrate 的类外定义,

//采用 simpson 法来计算函数的定积分

//计算公式为: $\text{sum}=(f(a)+4f(a+h)+2f(a+2h)+4f(a+3h)+2f(a+4h)+\dots$

$+2f(a+(n-2)h)+4f(a+(n-1)h)+f(b))h/3$

sum=f(a)+f(b); //对边界点 a、b 特殊处理

double s=1.0, al=a;

//s 依次取值 1, -1; 1, -1; ...。从而可使(3.0+s)依次成为公式中的

//系数 4,2, 4,2, 4,2, ...; al 为调用 f 函数时的实参值, 依次

//取值 a, a+h, a+2h, ..., a+(n-1)h

for(int i=1;i<n;i++) //还要在 n-1 个点处计算函数值

{

al+=h; //al 每次增加一个步长 h

sum+=(3.0+s)*f(al); //(3.0+s)依次为系数 4,2, 4,2, 4,2, ...

s=-s; //s 依次取值 1, -1; 1, -1; ...

}

sum*=h/3.0;

cout<<sum<<endl; //显示积分结果 sum

}

void f(rectangle rec, ladder lad, simpson sim)

{

...

//与题目处相同,略

}

```
void main()  
{  
    ...                               //与题目处相同,略  
}
```

[输出结果]

与题目处相同，略。

8.10 增加梯形图元类后在平面上画出简单图形

在教材 8.4.2 节的“利用图元类画图”的程序基础上，增加梯形图元类(类型)，并使用这些类来画一个简单的图形(如，画一个具有五官线条的人的头型图形)。

[分析提示]

(1) 涉及到绘图的程序，在 VC6.0 环境下，可按照如下的四个步骤来处理、来实现。

步 1: 使用 VC6.0 的 AppWizard 自动生成一个典型的“Hello World”框架程序(程序框架)。生成这一种框架程序的操作步骤为:“File”=>“New”=>“Projects”=>“Win32 Application”=>(右上处)“Project name”域中填入如“9_12_MyDrawing”,“Location”框内，添入你自用的文件夹名=>“OK”=>选第 3 种类型:“A Typical “Hello World” application”=>“Finish”=>“OK”。

步 2: 在所生成的框架程序基础上，增加相关的“绘图”处理类、以及处理 WM_PAINT 消息的“重画”处理函数 MyPaintProFunc，并将它们组织在同一个头文件 -- 如:“9_12_MyDrawing2.h”之中。

步 3: 在所生成的那一主要文件“9_12_MyDrawing.cpp”的开头处(最初的两个#include 行之下)，加入如下的一行:

```
#include "9_12_MyDrawing2.h" //新增头文件  
//内容为本例相关的“绘图”处理类、以及处理 WM_PAINT 消息的“重画”  
//处理函数 MyPaintProFunc
```

步 4: 在所生成的那一主要文件“9_12_MyDrawing.cpp”的 WndProc 函数的 switch 语句中，将“case WM_PAINT:”处理分支内的“GetClientRect(hWnd, &rt);”与“EndPaint(hWnd, &ps);”之间的“DrawText(hdc, szHello, strlen(szHello), &rt, DT_CENTER);”语句替换为如下的一个函数调用句:

```
MyPaintProFunc(hWnd, hdc, rt);  
//调用本例的“绘图”处理函数 MyPaintProFunc(位于“9_12_MyDrawing2.h”文件中)
```

(2) 其他一些涉及在 VC6.0 环境下进行绘图的进一步的有关解释与说明可参看范例 7.9 的分析提示的第 2 点。

(3) 在新增头文件“9_12_MyDrawing2.h”中(该头文件与上述自动生成的“9_12_MyDrawing.cpp”相呼应)，包含着与本例相关的“绘图”处理类、以及处理

WM_PAINT 消息的“重画”处理函数 `MyPaintProFunc`。其中的“绘图”处理类具体为：自定义的 `pixel`、`graphelem`、`Line`、`Rectang`、`Triangle`、`Circle`、`Square`、`Ladder`、`figure` 等 9 个类(类型)。

上述九个类之间的相互关系以及本程序的实现要点如下：

① 本程序设立并处理以下图元：直线 `Line` 类)，矩形 `Rectang` 类)，三角形 `Triangle` 类)，圆(`Circle` 类)，正方形(`Square` 类)，梯形(`Ladder` 类)。

② 将每一种图元设计成一个类，在每一个类的定义中，除含有其构造函数外，还包含一个可将本类的图元画出来(显示出来)的公有函数 `draw`。

③ 由于每一个图元都要用到颜色(`color`) 数据成员，所以设立一个基类 `graphelem`，它含有 `protected` 型数据成员 `color`，以及一个虚函数 `draw`(由于每一个类都要“`draw`”出属于它的类对象的图形，所以可利用函数重载的手段，在基类 `graphelem` 及其派生类中，共用同一个虚函数 `draw`)。

④ 由于不准备在基类 `graphelem` 的虚函数 `draw` 中做任何事情，所以在其原型后加上“`=0`”字样(而替掉函数定义体)，这样的虚函数称为纯虚函数。纯虚函数不能被直接调用，它只为其派生类的各虚函数规定了一个一致的函数原型。

⑤ 含有纯虚函数的基类称为抽象基类(如，本程序定义的基类 `graphelem` 即为抽象基类)。注意，不可用抽象基类来说明并创建它自己的对象(如，不可通过语句 `graphelem gra1, gra2;` 来创建对象 `gra1`，`gra2`)，只有在创建其派生类对象时，才有抽象基类自身的实例伴随而生。

本例中的抽象基类 `graphelem`：它是各种具体图元 `Line`、`Rectang`、`Triangle`、`Circle` 等)之共同点的一个抽象综合，通过它，再加上各派生类的特有成员，方可构成一个具体的图元类型。

⑥ 程序中至少要设立具有以下关系的七个类：抽象基类 `graphelem`(含有数据成员 `color`，以及一个纯虚函数 `draw`)；由抽象基类 `graphelem` 直接派生出的五个类：`Line`、`Rectang`、`Triangle`、`Circle`、`Ladder`；由 `Rectang` 直接派生出的一个类：`Square`。

⑦ 由于画以上图元时，都要用到“点”的概念与位置，所以设立的第八个类为：`pixel`。

⑧ 为了通过虚函数进行动态联编处理，需说明并使用指向基类 `graphelem` 的指针(注：该程序中说明了 12 个这种指针，放在一个称为 `pg` 的数组中，既是说，`pg[0]`、`pg[1]`、...，`pg[11]` 均为这种指向基类 `graphelem` 的指针)。而后通过这些指针的动态取值(使它们指向不同的派生类)，进而利用函数调用“`pg[i] -> draw()`”(`i=0, 1, ..., 11`) 来画出组成一个图形的不同图元来。

本程序中的第九个类 `figure` 中的 `paint` 成员函数)正是用来完成上述画图元功能的。

[解答程序]

/*

下面给出的是，经过上述的步 1 至步 4 之后，“9_12_MyDrawing.cpp”文件的核心代码内容；以及另一个新增头文件“9_12_MyDrawing2.h”的内容，该头文件与“9_12_MyDrawing.cpp”相呼应，内容为本例相关的“绘图”处理类以及处理 WM_PAINT

消息的“重画”处理函数 `MyPaintProFunc`。

`*/`

`// 9_12_MyDrawing.cpp : Defines the entry point for the application.`

`#include "stdafx.h"`

`#include "resource.h"`

`#include "9_12_MyDrawing2.h" //新增头文件`

`//内容为本例相关的“绘图”处理类、以及处理 WM_PAINT 消息的“重画”`

`//处理函数 MyPaintProFunc`

`...`

```
int APIENTRY WinMain(HINSTANCE hInstance,
                    HINSTANCE hPrevInstance,
                    LPSTR      lpCmdLine,
                    int         nCmdShow)
```

`{`

`...`

`}`

`ATOM MyRegisterClass(HINSTANCE hInstance)`

`{`

`...`

`}`

`BOOL InitInstance(HINSTANCE hInstance, int nCmdShow)`

`{`

`...`

`}`

`LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)`

`{`

`...`

`switch (message)`

`{`

`case WM_COMMAND:`

```

        ...
        break;

    case WM_PAINT:
        hdc = BeginPaint(hWnd, &ps);
        // TODO: Add any drawing code here...
        RECT rt;
        GetClientRect(hWnd, &rt);

        MyPaintProFunc(hWnd, hdc, rt);
        //调用本例的“绘图”处理函数 MyPaintProFunc
        //(处于"9_12_MyDrawing2.h"文件中)

        EndPaint(hWnd, &ps);
        break;
    case WM_DESTROY:
        ...
        break;
    default:
        ...
}
return 0;
}

LRESULT CALLBACK About(HWND hDlg, UINT message, WPARAM wParam, LPARAM
lParam)
{
    ...
}

//9_12_MyDrawing2.h --新增头文件，与“ 9_12_MyDrawing.cpp”相呼应，内容
//为本例相关的“绘图”处理类、以及处理 WM_PAINT 消息的“重画”处理函
//数 MyPaintProFunc“绘图”处理类包括：自定义的 pixel、graphelem、Line、
//Rectang、Triangle、Circle、Square、Ladder、figure 等 9 个类(类型)。

enum colort                //枚举类型，用于定义颜色常量(名字)
{
    black,blue,green,cyan,red,magenta,brown,
    lightgray,darkgray,lightblue,lightgreen,

```

```
lightcyan,lightred,lightmagenta,yellow,white,blink
};

COLORREF get_rgb(int col)
{
    //函数 get_rgb, 将颜色常量 col 转换成 VC 绘图时所需的 RGB 颜色值并返回
    COLORREF rgb;
    switch (col)
    {
        case red:
            rgb =RGB(255,0,0);      break;  //红色 red 对应于 RGB(255,0,0)
        case green:
            rgb =RGB(0,255,0);      break;  //绿色 green 对应于 RGB(0,255,0)
        case blue:
            rgb =RGB(0,0,255);      break;
        case cyan:
            rgb =RGB(0,255,255);    break;
        case white:
            rgb =RGB(255,255,255);  break;
        case magenta:
            rgb =RGB(255,0,255);    break;
        case yellow:
            rgb =RGB(255,255,0);    break;
        case lightgray:
            rgb =RGB(192,192,192);  break;
        case darkgray:
            rgb =RGB(128,128,128);  break;
        default:
            rgb =RGB(0,0,0);        break;
    }
    return (rgb);
}

class pixel
{
    //该 pixel 类的对象表示屏幕上的一个像素点, 由私有数据 x、y 表示点的位置
    int x,y;
public:
    pixel()                //构造函数一, 无参
```

```

    {
        x=0;        y=0;
    }
    pixel(int a, int b)        //构造函数二，参数 a、b 表示点的位置
    {
        x=a;        y=b;
    }
    pixel(const pixel& p)        //构造函数三，引用参数 p 为某个已存在对象
    {
        x=p.x;        y=p.y;
    }
    int getx( ){return x;}        //获取对象的 x 值
    int gety( ){return y;}        //获取对象的 y 值
};

class graphelem
{
//自定义类 graphelem，将作为其他图元类的基类——含有纯虚函数的基类称为抽象基类
protected:
    colort color;        //颜色 color
public:
    graphelem(colort col)
    {
        color=col;
    }
    virtual void draw(HWND hWnd)=0;
        //纯虚函数 draw( 原型后加上“=0”字样而替掉函数定义体)，
        //为该基类 graphelem 及其派生类中的超载函数。因为不在基类
        //graphelem 的虚函数 draw 中做任何事情，所以不需要函数体。
};

class Line:public graphelem        //自定义类 Line，为基类 graphelem 的派生类
{
    pixel start, end;        //成员为自定义 pixel 类(类型)的对象
public:
    Line(pixel sta, pixel en, colort col)        //初始化含有类对象的派生类对象
        :graphelem(col),start(sta),end(en) { };
        //先“ 父母”，再“ 对象”，后“ 自己” ——
        //先初始化基类 graphelem 对象，再初始化对象成员 start 与 end，

```

```

        //最后执行派生类的构造函数体, 初始化其“特性”成员(本派生类无此部分)
virtual void draw(HWND hWnd); //虚函数 draw
};

void Line::draw( HWND hWnd)          //类 Line 之虚函数 draw 的类外定义
{
    HDC hdc=GetWindowDC(hWnd);        //获取设备描述表 DC
    HPEN myPen;                        //说明画笔对象
    myPen=CreatePen( PS_SOLID, 2, get_rgb(graphelem::color) ); //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
                                    //添加画笔对象(到设备描述表 DC 中)
    MoveToEx(hdc,start.getx( ),start.gety( ),NULL);          //移至线始点处
    LineTo(hdc,end.getx( ),end.gety( ));                      //画至线终点处(使用新设置的画笔属性)
    (HPEN)SelectObject( hdc,pOldPen );                          //恢复原画笔(各属性)
    ReleaseDC(NULL,hdc);          //释放所获取到的那一设备描述表
}

class Rectang:public graphelem    //自定义类 Rectang, 为基类 graphelem 的派生类
{
    pixel ulcorner,lrcorner;
public:
    Rectang(pixel ul,pixel el,colort col)    //初始化含有类对象的派生类对象
        :graphelem(col),ulcorner(ul),lrcorner(el)
    { };
    virtual void draw(HWND hWnd);          //虚函数 draw
};

void Rectang::draw(HWND hWnd)          //类 Rectang 之虚函数 draw 的类外定义
{
    HDC hdc=GetWindowDC(hWnd);
    HPEN myPen;
    myPen=CreatePen( PS_SOLID, 2, get_rgb(graphelem::color) ); //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
    Rectangle(hdc, ulcorner.getx( ),ulcorner.gety( ),
    lrcorner.getx( ),lrcorner.gety( ));          //画出所需矩形来
    (HPEN)SelectObject( hdc,pOldPen );
    ReleaseDC(NULL,hdc);
}

```

```

class Circle:public graphelem           //自定义类 Circle，为基类 graphelem 的派生类
{
    pixel center;
    int radius;
public:
    Circle(pixel cen,int rad,colort col)    //初始化含有类对象的派生类对象
        :graphelem(col),center(cen)
        //先“ 父母”，再“ 对象”，后“ 自己” ——
        //先初始化基类 graphelem 对象，再初始化对象成员 center，
        //最后执行派生类的构造函数体，初始化其特性成员 radius
    {    radius=rad; }
    virtual void draw(HWND hWnd);        //虚函数 draw
};

void Circle::draw(HWND hWnd)            //类 Circle 之虚函数 draw 的类外定义
{
    HDC hdc=GetWindowDC(hWnd);
    HPEN myPen;
    myPen=CreatePen( PS_SOLID, 2, get_rgb(graphelem::color) ); //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
    Ellipse(hdc,center.getx( )-radius,center.gety( )-radius,
            center.getx( )+radius,center.gety( )+radius);    //画出所需圆来
    (HPEN)SelectObject( hdc,pOldPen );
    ReleaseDC(NULL,hdc);
}

class Triangle:public graphelem         //自定义类 Triangle，为基类 graphelem 的派生类
{
    pixel pointa,pointb,pointc;
public:
    Triangle(pixel pa,pixel pb,pixel pc,colort col)
        //初始化含有类对象的派生类对象
        :graphelem(col),pointa(pa),pointb(pb),pointc(pc)
    { }
    virtual void draw(HWND hWnd);        //虚函数 draw
};

void Triangle::draw(HWND hWnd)          //类 Triangle 之虚函数 draw 的类外定义
{

```

```

HDC hdc=GetWindowDC(hWnd);
HPEN myPen;
myPen=CreatePen( PS_SOLID, 2, get_rgb(graphelem::color) ); //设置画笔属性
HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
MoveToEx(hdc,pointa.getx( ),pointa.gety( ),NULL);
LineTo(hdc,pointb.getx( ),pointb.gety( ));
LineTo(hdc,pointc.getx( ),pointc.gety( ));
LineTo(hdc,pointa.getx( ),pointa.gety( ));    //画出所需三角形来
// Restore the old pen to the device context
(HPEN)SelectObject( hdc,pOldPen );
ReleaseDC(NULL,hdc);
}

class Square:public Rectang           //自定义类 Square， 为基类 Rectang 的派生类
{
public:
    Square(pixel ul,int lh,colort col)
        :Rectang(ul,pixel(ul.getx( )+lh,ul.gety( )+lh),col)
    { }
    virtual void draw(HWND hWnd); //虚函数 draw
};

void Square::draw(HWND hWnd)         //类 Square 之虚函数 draw 的类外定义
{
    HDC hdc=GetWindowDC(hWnd);
    HPEN myPen;
    myPen=CreatePen( PS_SOLID, 2, get_rgb(graphelem::color)); //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
    Rectang::draw(hWnd);             //画出所需正方形来
    // Restore the old pen to the device context
    (HPEN)SelectObject( hdc,pOldPen );
    ReleaseDC(NULL,hdc);
}

class Ladder:public graphelem        //自定义类 Ladder， 为基类 graphelem 的派生类
{
    pixel center;
    int height,upWid,lowWid;
public:
    Ladder(pixel cen,int hght,int upW,int lowW,colort col)

```



```

        :graphelem(col)
        {
            center=cen;          height=hght;
            upWid=upW;           lowWid=lowW;
        }
        void draw(HWND hWnd);    //虚函数 draw
};

void Ladder::draw(HWND hWnd)    //类 Ladder 之虚函数 draw 的类外定义
{
    HDC hdc=GetWindowDC(hWnd);
    HPEN myPen;
    myPen=CreatePen( PS_SOLID, 2, get_rgb(graphelem::color)); //设置画笔属性
    HPEN pOldPen = (HPEN)SelectObject( hdc,myPen );
    int centx,centy;
    centx=center.getx( );
    centy=center.gety( );
    //通过画出梯形的 4 条线来完成图形的绘制
    MoveToEx(hdc,centx-upWid/2,centy-height/2,NULL);
    LineTo(hdc,centx+upWid/2,centy-height/2);
    LineTo(hdc,centx+lowWid/2,centy+height/2);
    LineTo(hdc,centx-lowWid/2,centy+height/2);
    LineTo(hdc,centx-upWid/2,centy-height/2);
    // Restore the old pen to the device context
    (HPEN)SelectObject( hdc,pOldPen );
    ReleaseDC(NULL,hdc);
}

```

/*

为了通过虚函数进行动态联编处理，需说明并使用指向基类 `graphelem` 的指针(注：该程序中说明了 12 个这种指针，放在一个称为 `pg` 的数组中，既就是说，`pg[0]`，`pg[1]`，...，`pg[11]` 均为这种指向基类 `graphelem` 的指针)。而后通过这些指针的动态取值(使它们指向不同的派生类)，进而利用函数调用“`pg[i] ->draw( )`” (`i=0, 1, ... , 11`) 来画出组成一个图形的不同图元来。

本程序中的第九个类 `figure` 中的 `paint` 成员函数正是用来完成上述画图元功能的。

*/

```

class figure
{
    //自定义类 figure，通过它的 paint 成员函数可画出组成一个图形的各图元；

```

```

    graphelem *pg[12]; //pg 为指针数组, 含有 12 个指向基类 graphelem 的指针
public:

```

```

    figure(graphelem *pg1=0,   graphelem *pg2=0,   graphelem *pg3=0,
    graphelem *pg4=0,   graphelem *pg5=0,   graphelem *pg6=0,
    graphelem *pg7=0,   graphelem *pg8=0,   graphelem *pg9=0,
    graphelem *pg10=0,  graphelem *pg11=0,  graphelem *pg12=0 )
    /*

```

figure 类的构造函数, 含有 12 个参数, 且该 12 个参数均为可缺省参数(被赋了缺省值的参数, 调用时, 相应实参可缺省)。说明 figure 类对象时, 其实参可为 12 个(或少于 12 个)指向 graphelem 不同派生类对象的具体指针(注意, C++ 允许基类指针指向其派生类对象; 而此处的各派生类对象正是准备画出的那些不同图元)。注意如下语法: 函数定义处, 若有可缺省参数的话, 必须放于参数表的最右边, 且要连续出现。

```

    */
    {
        pg[0]=pg1;    pg[1]=pg2;    pg[2]=pg3;    pg[3]=pg4;
        pg[4]=pg5;    pg[5]=pg6;    pg[6]=pg7;    pg[7]=pg8;
        pg[8]=pg9;    pg[9]=pg10;   pg[10]=pg11;   pg[11]=pg12;
    }

    void paint(HWND hWnd)        //paint 成员函数画出图形的各图元
    {
        for(int i=0;i<12;i++)
            if(pg[i]!=0)
                pg[i] ->draw(hWnd);
                //由于 pg[i] 为各派生类对象的地址(由构造函数的各实参带来),
                //从而使此处的 i 循环可画出所设计图形的各图元
    }
};

```

//处理 WM_PAINT 消息的“重画”处理函数 MyPaintProFunc

```

void MyPaintProFunc(HWND hWnd, HDC hdc, RECT rt)
{
    //说明 11 个类对象(图元)
    Square sq(pixel(0,0),410,magenta);        //一个正方形(对象)
    Circle ce1(pixel(200,160),150,green);      //三个圆(对象)
    Circle ce2(pixel(145,75),25,blue);
    Circle ce3(pixel(255,75),25,blue);
    Triangle tr1(pixel(200,120),pixel(180,180),pixel(220,180),red);
                                                //两个三角形(对象)
    Triangle tr2(pixel(200,310),pixel(0,407),pixel(400,407),brown);
}

```

```

Rectang re1(pixel(160,240),pixel(240,255),magenta);    //一个长方形(对象)
Ladder ladR(pixel(350,180),50,100,60,lightblue);      //两个梯形(对象)
Ladder ladL(pixel(50,180),50,100,60,lightblue);
Line lin1(pixel(115,25),pixel(175,35),lightred);      //两条直线(对象)
Line lin2(pixel(225,35),pixel(285,25),lightred);
    //一个由 11 个图元构成的 figure 类对象，这 11 个图元由
    //调用构造函数时带去的那 11 个实参所确定，各实参均为指向
    //graphelem 不同派生类对象的具体指针(也即，对象地址)
figure fig(&sq,&ce1,&ce2,&ce3,&tr1,&re1,&tr2,&ladL,&ladR,&lin1,&lin2);
fig.paint(hWnd);    //调用 paint 成员函数，画出 fig 对象的 11 个图元
}

```

[输出结果]

为一个具有五官线条的人的头型图形，略。

8.11 派生关系中的赋值兼容性以及重名成员处理等

读下面的程序，给出执行结果（涉及到基类对象和派生类对象相关的赋值兼容性问题，以及单继承时父类与子类间重名成员的处理问题等）。

```

#include <iostream.h>

class B0                                //基类 B0
{
public:
    void print( )                        //公有成员函数 print
    {
        cout<<"B0::print( )"<<" ";
    }
};

class B1: public B0                      //公有派生类 B1(由 B0 派生)
{
public:
    void print( )                        //同名的公有成员函数 print
    {
        cout<<"B1::print( )"<<" ";
    }
};

```

```

class B2: public B1                //公有派生类 B2(由 B1 派生)
{
public:
    void print()                  //同名的公有成员函数 print
    {
        cout<<"B2::print()"<<" ";
    }
};

void main()
{
    B0 b0;    B1 b1;    B2 b2;
    B0 *p0;    B1 *p1;    B2 *p2;
    b0=b1;                //基类对象 = 派生类对象;(注: 只赋共性成员部分)
    b0=b2;                //基类对象 = 派生类对象;
                        //b1=b0; //error! 派生类对象 = 基类对象;
    b1=b2;                //基类对象 = 派生类对象;
                        //b2=b0; //error! 派生类对象 = 基类对象;
                        //b2=b1; //error! 派生类对象 = 基类对象;
    b0.print();            //调用 B0 类的 print()
    cout<<endl;
    b0.B0::print();        //调用 B0 类的 print(), 也可通过类名限定(虽然多余)
                        //b0.B1::print(); //error C2039: " B1" : is not a member of " B0"
                        //b0.B2::print(); //error C2039: " B2" : is not a member of " B0"
    cout<<"\n-----1-----"<<endl;
    b1.print();            //调用 B1 类的 print()
    cout<<endl;
    b1.B0::print();        //调用 B0 类的 print(), 要通过类名限定
    b1.B1::print();        //调用 B1 类的 print(), 也可通过类名限定(虽然多余)
                        //b1.B2::print(); //error C2039: " B2" : is not a member of " B1"
    cout<<"\n-----2-----"<<endl;
    b2.print();            //调用 B2 类的 print()
    cout<<endl;
    b2.B0::print();        //调用 B0 类的 print(), 要通过类名限定
    b2.B1::print();        //调用 B1 类的 print(), 要通过类名限定
    b2.B2::print();        //调用 B2 类的 print(), 也可通过类名限定(虽然多余)
    cout<<"\n-----3-----"<<endl;
    p0=&b0;                //指向基类型的指针 = 基类对象的地址;
    p0->print();            //调用 B0 类的 print()
}

```

```

cout<<endl;
p0->B0::print( );    //调用 B0 类的 print( ), 也可通过类名限定(虽然多余)
                    //p0->B1::print( ); //error C2039: " B1 " : is not a member of " B0"
                    //p0->B2::print( ); //error C2039: " B2 " : is not a member of " B0"

cout<<endl;
((B0*)p0) ->print( );
                    //调用 B0 类的 print( ), 也可加上指针强制类型转换(虽然多余)
((B1*)p0) ->print( ); //调用 B1 类的 print( ), 要通过指针强制类型转换
((B2*)p0) ->print( ); //调用 B2 类的 print( ), 要通过指针强制类型转换
cout<<"\n-----4-----"<<endl;

                    //p1=&b0; //error! B1 类指针指向 B0 类对象
                    //p2=&b0; //error! B2 类指针指向 B0 类对象
p1=&b1;              //B1 类指针指向 B1 类对象——具有类型的一致性
p1->print( );        //调用 B1 类的 print( )
cout<<endl;
p1->B0::print( );    //调用 B0 类的 print( ), 要通过类名限定
p1->B1::print( );    //调用 B1 类的 print( ), 也可通过类名限定(虽然多余)
                    //p1->B2::print( ); //error C2039: " B2 " : is not a member of " B1 "
cout<<endl;
((B0*)p1) ->print( ); //调用 B1 类的 print( ), 要通过指针强制类型转换
((B1*)p1) ->print( );
                    //调用 B1 类的 print( ), 也可加上指针强制类型转换(虽然多余)
((B2*)p1) ->print( ); //调用 B2 类的 print( ), 要通过指针强制类型转换
cout<<"\n-----5-----"<<endl;
p2=&b2;              //B2 类指针指向 B2 类对象——具有类型的一致性
p2->print( );        //调用 B2 类的 print( )
cout<<endl;
p2->B0::print( );    //调用 B0 类的 print( ), 要通过类名限定
p2->B1::print( );    //调用 B1 类的 print( ), 要通过类名限定
p2->B2::print( );    //调用 B2 类的 print( ), 也可通过类名限定(虽然多余)
cout<<endl;
((B0*)p2) ->print( ); //调用 B0 类的 print( ), 要通过指针强制类型转换
((B1*)p2) ->print( ); //调用 B1 类的 print( ), 要通过指针强制类型转换
((B2*)p2) ->print( );
                    //调用 B2 类的 print( ), 也可加上指针强制类型转换(虽然多余)
cout<<"\n-----6-----"<<endl;
}

```

[分析提示]

(1) 本范例程序涉及到基类对象和派生类对象相关的赋值兼容性问题, 例如, 允许

进行下述两种情况的赋值：

① 基类对象 = 派生类对象；

允许进行这种赋值，但只赋共性成员部分（冗余部分被甩掉）。注意，反方向的赋值则不被允许，即是说，不可将基类对象赋值给派生类对象。

② 指向基类型的指针 = 派生类对象的地址；

允许进行这种赋值，它正是函数重载及虚函数用法的基础。通过指向基类型的指针可以直接访问基类成员部分，但访问非基类成员部分时，要经过指针类型的强制转换。注意，反方向的赋值则不被允许，即是说，不可进行下述赋值：指向派生类类型的指针 = 基类对象的地址。

(2) 派生关系中产生的二义性（如，单继承时父类与子类间的成员重名），要按照如下的规则来进行处理：对子类而言在子类定义范围内以及通过子类对象访问重名成员时，不加类名限时默认为是处理子类成员；而要访问父类重名成员时，则要通过类名限定（通过“<类名>::”进行限定）。

[解答程序]

具体程序以及对它们的注释见题目处。

[输出结果]

```

B0::print()
B0::print()
-----1-----
B1::print()
B0::print() B1::print()
-----2-----
B2::print()
B0::print() B1::print() B2::print()
-----3-----
B0::print()
B0::print()
B0::print() B1::print() B2::print()
-----4-----
B1::print()
B0::print() B1::print()
B0::print() B1::print() B2::print()
-----5-----
B2::print()
B0::print() B1::print() B2::print()
B0::print() B1::print() B2::print()
-----6-----

```

第九章 运算符重载

[语法要点]

详细内容可参阅教材第五章和第七章。

(1) C++允许“借用”老算符，对自定义类对象实现某种自定义的新功能，这被称为运算符重载。运算符重载的定义是一个函数的定义过程（其函数名处为：`operator <运算符>`）。允许使用如下两种不同方式来定义运算符重载函数：以类的友元函数方式定义（也称为友元运算符）；以类的公有成员函数方式定义（也称为类运算符）。以类的友元函数方式定义时的特征为：友元函数内（定义处）可处理与使用本类的私有成员；所有运算分量必须显式地列在本友元函数的参数表中（由于友元函数中没有 `this` 指针），而且这些参数类型中至少要有一个应该是说明该友元的类类型或是对该类的引用。若以类的公有成员函数方式定义时，则类成员函数内（定义处）可处理与使用本类的私有成员，且总以当前调用者对象（`*this`）作为该成员函数的隐式第一运算分量，当所定义的运算多于一个运算对象时，才将其余运算对象显式地列在该成员函数的参数表中。请注意如下几条对重载运算符的限制：

① 被用户重定义（重载）的运算符，其优先级、运算顺序（结合性）、以及运算分量个数都必须与系统中的原运算符相一致。

② 如下五个运算符不可重载：“`、`”、“`::`”、“`?:`”、“`.*`”、“`sizeof`”。

③ 只能以类成员而不能以友元身份重载者：“`=`”、“`()`”、“`[]`”、“`->`”、“`type`”。

④ 不可自创新的运算符。

通过对运算符进行重定义（重载），可以方便用户对运算符的使用，提高程序的易读性、易维护性。

(2) 对增量运算符“`++`”及减量运算符“`--`”重载时，要按如下的格式进行。

① 以类成员方式重载前缀运算符“`++`”：`<类型> <类名>::operator++ ();`

② 以类成员方式重载后缀运算符“`++`”：`<类型> <类名>::operator++ (int);`

注意，后缀算符多给出一个 `int` 参数只是表明调用该函数时运算符应放在操作数的后面，参数本身在函数体中并不使用，因此也没必要给出参数名字。以类成员方式重载运算符“`——`”的函数原型一般格式与运算符“`++`”相同。

③ 以友元方式重载前缀运算符“`++`”：`<类型> operator++ (<类名>&);`

④ 以友元方式重载后缀运算符“`++`”：`<类型> operator++ (<类名>&, int);`

其中的`<类名>`为重载运算符“`++`”的自定义类的类名。注意，多给出一个 `int` 参数的理由与上面相同。以友元方式重载运算符“`——`”的函数原型一般格式与“`++`”相同。

(3) 可以且只能以类成员方式重载下标运算符“`[]`”。若在某自定义类中重载了下标运算符“`[]`”的话，则可将该类的类对象当作一个“数组”，从而可对该类对象通过使用下标变量的方式来访问其中的数据成员（但各下标变量的取值与类对象数据成员间的对

应关系完全由程序员第九章 运算符重载在该重载函数体中来规定)。重载下标运算符“[]”的函数原型一般格式如下:

<类型> <类名>::operator[] (<形参类型> <形参名>);

注意,重载下标运算符“[]”时,必须且只能带一个形参,其参数值相当于下标值(规定从0开始),而函数返回值正是对应于参数下标值的那一下标变量的具体取值。

(4) 可以对赋值运算符进行重载且只能重载为类成员函数。若没重载赋值运算符时,系统将按隐含的“浅拷贝”方式进行赋值处理(只是按位进行原样拷贝)。若对象占有系统资源(如,通过运算符 new 对指针型数据成员分配有堆空间)时,则应考虑重载赋值运算符,以提供按“深拷贝”方式进行赋值的处理过程(要重新分配一块相同大小的动态空间,并拷贝指针型数据成员所指堆空间中的具体内容。否则的话,两个指针型数据成员将指向完全相同的那一块系统的堆空间,而容易产生析构对象时两次释放动态空间的错误)。

要注意拷贝构造函数(原型为“<类名>(<类名>&);”的构造函数,它只含一个参数,且为本类对象的引用)与赋值运算符重载(函数)的使用区别:拷贝构造函数是用已存在对象(的各成员的当前值)来创建一个相同的新对象(尚不存在,正在创建)。在下述三种情况下系统将自动调用所属类的拷贝构造函数(若程序中没有定义显式的拷贝构造函数的话,系统将提供隐含的只进行“浅拷贝”的拷贝构造函数):当说明新的类对象的同时要给它赋值另一个已存在对象(的各成员当前值);当对象作为函数的赋值参数而对函数进行调用要进行实参和形参的结合时;当函数的返回值是类的对象而在执行被调函数的返回语句后(也即在函数调用完成返回时)。而赋值运算符重载(函数)则要把一个已存在对象(的各成员当前值)赋值给另一个已存在的同类对象(已经存在,不需再创建)。例如,假设 matrix 为自定义的矩阵类类型,则:

```
matrix matr1(2,3);           //调用二参构造函数,创建出对象 matr1
matrix matr2;                //调用无参构造函数,创建出对象 matr2
matr2 = matr1;
//调用赋值运算符重载(函数),把一个已存在对象 matr1(的各成员当前值)
//赋值给另一个已存在的同类对象 matr2(对象 matr2 已经存在,不需再创建)。
```

注意与如下使用情况进行区分:

```
matrix matr1(2,3);           //调用二参构造函数,创建出对象 matr1
matrix matr2 = matr1;        //调用拷贝构造函数
//创建新的类对象 matr2 的同时要给它赋值另一个已存在对象 matr1 的当前值
```

(5) 重载运算符 new 与 delete 时有以下几点需要注意。

① 重载运算符 new 与 delete 时,可以选择重载系统预定义的缺省全局运算符 new 和 delete,也可以只为某个(或某几个)自定义类重载运算符 new 与 delete。

② 对系统预定义的任一基本数据类型、对于没有在其类中自定义 operator new 函数的那些类类型以及对任何类型的数组来说,当使用 new 来创建这些类型的动态对象(变量)时,将使用系统预定义的缺省全局运算符 new 或者将使用被用户重载了的全局运算符 new(若用户重载了全局运算符 new 后,此时系统预定义的缺省全局运算符 new 将完全不可再被访问)。

③ 对某个自定义的类类型来说,如若该类中自定义了 `operator new` 函数的话,那么当使用运算符 `new` 来创建该类类型的动态对象时,则将屏蔽全局的运算符 `new` 而去使用本类中所定义的那一个 `operator new` 函数。对用户自定义类(类型)重载运算符 `new` 与 `delete` 时,只能将它们重载为类的成员函数,而且,无论是否使用了关键字 `static` 进行修饰,它们均被重载为 `static` 特性的成员函数。

④ 按如下的一般形式来重载运算符 `new` 与 `delete`:

```
void* operator new(unsigned bytes);
void operator delete(void* pvMem);
```

重载运算符 `new` 时,它含有一个无符号整型的参数 `bytes` (也经常说明为 `size_t` 类型,而 `size_t` 为 `stdio.h` 头文件中的一个常量类型名),用于指出本次需要分配多少字节的存储空间。但注意,调用时对应的实参值由编译器产生并传递过来,并不需要程序员来直接提供(因为编译器通过运算符 `new` 后的类型可以获得该值)。被重载的 `new` 函数的返回值类型必须为 `void*` 类型,通常在调用后将它转换为所需的其它指针类型再去使用。重载运算符 `delete` 时,由 `void*` 类型的参数 `pvMem` 带来欲释放空间的首地址(指针值)。

⑤ 当重载运算符 `new` 后,只改变原有的动态内存分配方法而不是完成一个对象的建立。即是说,编译器首先调用重载的 `new` 函数按照用户规定的方法去分配动态内存,而后还将立即去调用构造函数。当重载运算符 `delete` 后,也只改变原有的动态内存释放的方法,即是说,编译将首先调用析构函数,而后按用户规定的方法释放动态内存。

⑥ 若重载了全局运算符 `new` 与 `delete` 后,通常不再使用 `cin` 与 `cout` 进行输入输出操作,其原因是:`cin` 与 `cout` 流类对象的使用会涉及到缺省的全局运算符 `new` 与 `delete`,而若用户重载它们后,则屏蔽了缺省的全局运算符 `new` 与 `delete`,从而容易使系统产生死锁状态。此时的输入输出可用 `stdio.h` 中的 `scanf` 与 `printf` 来完成。

⑦ 重载的 `new` 函数要按照用户设定的方法去分配动态内存,比如可“委托”处于 `malloc.h` 头文件中的系统预定义函数 `malloc` 来进行具体分配。而重载的 `delete` 函数要按照用户设定的方法去释放动态内存,比如也可“委托”处于 `malloc.h` 头文件中的系统预定义函数 `free` 来进行具体的释放工作。这样做的话,所用动态空间的分配与释放工作实质上仍然是由系统来完成的。另一具有全权的处理方法是:由程序事先申请并掌握一大块所谓的动态存储空间(如一个静态数组空间 `MemArea`),而后在重载的 `new` 与 `delete` 函数中,总使用自己已经掌握的那一大块所谓的动态存储空间来进行分配与释放。即是说,与分配和释放有关的一切工作(包括分配和回收策略等)都由程序自己来控制、来管理。

(6) 类型转换运算符(函数)也称转换函数,只可按类成员方式进行重载(定义)。它是一类特殊的运算符重载(函数)。其声明格式只能为:

```
operator <类型名>();
```

功能:将本类对象(的数据)转换为“<类型名>”所指定的类型(数据),具体转换过程由函数体描述。

在进行运算或函数参数传递的过程中,当系统发现自定义类对象的类型不匹配时,若程序中对该自定义类定义了转换函数的话,则系统会自动调用该类的转换函数去对类对象进行类型转换,而后再进行相应的运算或处理(否则的话,不进行这种类型转换处

理)。此种由系统自动调用用户自定义转换函数而进行的类型转换称为隐式转换。

另外, 如果需要, 程序中还可以利用自定义转换函数进行显式的强制类型转换。这实际上是系统强制类型转换功能对自定义类型的扩充。否则的话, 强制类型转换只对系统预定义的基本数据类型有效。强制类型转换使用如下两种格式之一:

(<类型名>) <对象名> 或 <类型名> (<对象名>)

例如, `double(comObj)` 或 `(double)comObj`, 就是显式地让系统将类对象 `comObj` 的类型强制转换为 `double` 类型。此处假设 `comObj` 为自定义类 `complex` 的类对象, 而且在自定义类 `complex` 中定义了将 `complex` 类对象(的数据)转换为 `double` 类型(数据)的转换函数, 而具体的转换过程在该转换函数体中进行了描述(定义)。

9.1 通过友元方式重载运算符

自定义一个称为 `point3` 的类型, 其对象由空间一个点的三个坐标(x,y,z)组成, 并通过友元方式重载二目运算符“==”、“+”、“-”和一目运算符“-”, 用来判断两个对象是否相等, 完成两个对象的和与差运算以及完成对一个对象的数值取反。

例如, (1.1, 2.2, 3.3)只与它本身即(1.1, 2.2, 3.3)相等;

$(1.1, -2.2, 3.3) + (1, 3, -5.5) = (2.1, 0.8, -2.2);$

$(8, 8, 12.2) - (1, 3, -5.5) = (7, 5, 17.7);$

$-(1.1, -2.2, 3.3) = (-1.1, 2.2, -3.3)。$

并编制主函数, 对所定义的类型进行使用, 对类对象进行相关运算(以验证其正确性)。

如, 自定义的 `point3` 类(类型)可按如下格式定义(只通过友元方式重载各运算符):

```
class point3                                //自定义的 point3 类(类型)
{
    double x,y,z;                          //数据成员, 表示空间的一个点(x,y,z)
public:
    point3 ();                             //无参构造函数, 默认为(0,0,0)点
    point3 (point3& pt);                   //拷贝构造函数
    point3 (double x0, double y0, double z0); //三参构造函数, 表示(x0,y0,z0)点
    friend point3 operator - (point3 pt);
                                           //一目运算符“-”, 将组成一个对象的数值取反
    friend point3 operator + (point3 pt1, point3 pt2); //两个对象的和运算
    friend point3 operator - (point3 pt1, point3 pt2); //两个对象的差运算
    friend bool operator == (point3 pt1, point3 pt2);  //比较两个对象是否相等
    void print(void);                          //以“(x,y,z)”的格式显示对象的当前值
};
```

[分析提示]

本范例以类的友元函数方式对运算符进行重载。友元函数内(定义处)可处理与使用相关类的私有成员,但要求将所有运算分量都显式地列在友元函数的参数表中(由于友元函数中没有 **this** 指针),而且这些参数类型中至少要有一个应该是说明该友元的类类型或是对该类的引用。

[解答程序]

```
#include <iostream.h>

class point3                                //自定义的 point3 类(类型)
{
    ...                                    //与题目处相同, 略
};

point3::point3 (void)                       //无参构造函数, 默认为(0,0,0)点
{
    x=0.0;      y=0.0;      z=0.0;
}

point3::point3 (point3& pt)                 //拷贝构造函数
{
    x=pt.x;      y=pt.y;      z=pt.z;
}

point3::point3 (double x0, double y0, double z0) //三参构造函数, 表示(x0,y0,z0)点
{
    x=x0;y=y0;z=z0;
}

point3 operator - (point3 pt)              //一目运算符“ -”, 将组成一个对象的数值取反
{
    point3 temp;      temp.x=-pt.x;
    temp.y=-pt.y;    temp.z=-pt.z;
    return temp;
}

point3 operator + (point3 pt1, point3 pt2) //二目运算符“ +”, 两个对象的和运算
{
    point3 temp;
    temp.x=pt1.x+pt2.x;
```

```

        temp.y=pt1.y+pt2.y;
        temp.z=pt1.z+pt2.z;
        return temp;
    }

point3 operator-(point3 pt1, point3 pt2)    //二目运算符“ - ”，两个对象的差运算
{
    point3 temp;
    temp.x=pt1.x-pt2.x;
    temp.y=pt1.y-pt2.y;
    temp.z=pt1.z-pt2.z;
    return temp;
}

bool operator==(point3 pt1, point3 pt2)    //二目运算符“ ==”，比较两个对象是否相等
{
    return( (pt1.x==pt2.x) && (pt1.y==pt2.y) && (pt1.z==pt2.z) );
}

void point3::print ( )                    //以“ (x,y,z)” 的格式显示对象的当前值
{
    cout << "( "<<x<< ", "<<y<< ", "<<z<< )" << endl;
}

void main( )                            //主函数，对所定义的类进行使用，对类对象进行相关运算(以验证其正确性)
{
    //说明八个类对象，并对其中的四个指定显式初值(其他四个被赋予隐含初值(0,0,0))
    point3 s0, s1(1.1, -2.2, 3.3), s2(1, 3, -5.5), s3(8, 8, 12.2);
    point3 s4(s2), s5, s6, s7;
    cout<<"s0=";  s0.print( );           //显示前五个对象的值
    cout<<"s1=";  s1.print( );
    cout<<"s2=";  s2.print( );
    cout<<"s3=";  s3.print( );
    cout<<"s4=";  s4.print( );
    s5=s1+s2;                            //二对象“ + ” 运算
    cout<<"s5=s1+s2=";  s5.print( );
    cout<<"(s1+s2).print( ): ";
    (s1+s2).print( );                    //二对象“ + ” 运算
    s6=s3-s2;                            //二对象“ - ” 运算
}

```



```

point3 ( );           //无参构造函数，默认为(0,0,0)点
point3 (point3& pt);  //拷贝构造函数
point3 (double x0, double y0, double z0); //三参构造函数，表示(x0,y0,z0)点
point3 operator - ( ); //一目运算符“ - ”，将组成一个对象的数值取反
point3 operator + (point3 pt2); //两个对象的和运算
point3 operator - (point3 pt2); //两个对象的差运算
bool operator == (point3 pt2); //比较两个对象是否相等
void print(void);      //以“ (x,y,z)” 的格式显示对象的当前值
};

```

[分析提示]

本范例与上一范例实现相同的问题，但要类以类的公有成员函数方式定义运算符重载函数。注意，类成员函数内（定义处）可处理与使用本类的私有成员，且总以当前调用者对象（*this）作为该成员函数的隐式第一运算分量，若所定义的运算多于一个运算对象时，才将其余运算对象显式地列在该成员函数的参数表中。

[解答程序]

```

#include <iostream.h>

class point3           //自定义的 point3 类(类型)
{
    ...                //与题目处相同,略
};

point3::point3 (void)   //无参构造函数，默认为(0,0,0)点
{
    x=0.0;    y=0.0;    z=0.0;
}

point3::point3 (point3& pt) //拷贝构造函数
{
    x=pt.x;    y=pt.y;    z=pt.z;
}

point3::point3 (double x0, double y0, double z0) //三参构造函数，表示(x0,y0,z0)点
{
    x=x0;y=y0;z=z0;
}

point3 point3::operator-( ) //一目运算符“ - ”，将组成一个对象的数值取反
{

```

```

    point3 temp;
    temp.x=-x;
    temp.y=-y;
    temp.z=-z;
    return temp;
}

```

point3 point3::operator + (point3 pt2) //二目运算符“+”，两个对象的和运算

```

{
    point3 temp;
    temp.x=x+pt2.x;
    temp.y=y+pt2.y;
    temp.z=z+pt2.z;
    return temp;
}

```

point3 point3::operator-(point3 pt2) //二目运算符“-”，两个对象的差运算

```

{
    point3 temp;
    temp.x=x-pt2.x;
    temp.y=y-pt2.y;
    temp.z=z-pt2.z;
    return temp;
}

```

bool point3::operator == (point3 pt2) //二目运算符“==”，比较两个对象是否相等

```

{
    return( (x==pt2.x) && (y==pt2.y) && (z==pt2.z) );
}

```

void point3::print () //以“ (x,y,z)” 的格式显示对象的当前值

```

{
    cout <<(" <<x<<","<<y<<","<<z<<")<<endl;
}

```

void main() //主函数，对所定义的类进行使用，对类对象进行相关运算

//(以验证其正确性)

```

{
//说明八个类对象，并对其中的四个指定显式初值(其他四个被赋予隐含初值(0,0,0))
    point3 s0, s1(1.1, -2.2, 3.3), s2(1, 3, -5.5), s3(8, 8, 12.2);
}

```

```

point3 s4(s2, s5, s6, s7);
cout<<"s0="; s0.print( );           //显示前五个对象的值
cout<<"s1="; s1.print( );
cout<<"s2="; s2.print( );
cout<<"s3="; s3.print( );
cout<<"s4="; s4.print( );
s5=s1+s2;                             //两个对象“+”运算
cout<<"s5=s1+s2="; s5.print( );
cout<<"(s1+s2).print( ): ";
(s1+s2).print( );                     //两个对象“+”运算
s6=s3-s2;                             //两个对象“-”运算
cout<<"s6=s3-s2="; s6.print( );
s7=-s1;                               //一目运算符“-”
cout<<"s7=-s1="; s7.print( );
if(s1==s1)                            //二目运算符“==”
    cout<<"s1==s1"<<endl;
else
    cout<<"!(s1==s1)"<<endl;
}

```

[输出结果]

```

s0=( 0, 0, 0 )
s1=( 1.1, -2.2, 3.3 )
s2=( 1, 3, -5.5 )
s3=( 8, 8, 12.2 )
s4=( 1, 3, -5.5 )
s5=s1+s2=( 2.1, 0.8, -2.2 )
(s1+s2).print( ): ( 2.1, 0.8, -2.2 )
s6=s3-s2=( 7, 5, 17.7 )
s7=-s1=( -1.1, 2.2, -3.3 )
s1==s1

```

9.3 自定义简单分数类(类型)fraction 中的运算符重载

改造范例 7.14 的自定义简单分数类类型 `fraction`，将其中使用的普通函数尽量改写成运算符重载函数。如，可重载`+`、`-`、`*`、`/`、`>`、`>=`、`==`、`!=`、`<=`、`<`、`~`，以实现分数的加减乘除二目运算，一目减运算、六种进行比较的运算、以及分数的倒数运算。

下面给出改造后的 `fraction` 的类定义，并给出一个示意性的主函数以及程序执行结果样本。

```
class fraction                                //自定义的分数类型 fraction
{
    int above;                                //分子 above
    int below;                                //分母 below
    void reduction( );                        //私有成员函数 reduction，负责对分数进行约分
    void makeCommonD(fraction&);             //私有成员函数 makeCommonD，负责对两个分数进行通分

public:
    fraction(int,int);                        //二参构造函数，带来分子分母
    fraction(int);                             //一参构造函数，带来分子，分母隐含为 1
    fraction( );                               //无参构造函数，隐含分子为 0，分母为 1
    fraction operator +(fraction);             //两个分数相加
    fraction operator -(fraction);             //两个分数相减
    fraction operator *(fraction);             //两个分数相乘
    fraction operator /(fraction);             //两个分数相除
    fraction operator -( );                     //一目减运算
    fraction operator ~( );                     //求倒数
    bool operator ==(fraction);                //等于运算
    bool operator !=(fraction);                //不等于运算
    bool operator >(fraction);                 //大于运算
    bool operator <(fraction);                 //小于运算
    bool operator >=(fraction);                //大于等于运算
    bool operator <=(fraction);                //小于等于运算
    void display( );                           //显示对象数据
    void input( );                             //输入对象数据
};

void main( )                                //主函数 main
{
    //说明 fraction 类对象，对定义的各函数进行调用，以验证它们的正确性
    fraction f1(-2,-3), f2(-2,3), f3(3,-5), f4, f5(6);
    cout<<"f1=";    f1.display( );
    cout<<"f2=";    f2.display( );
    cout<<"f3=";    f3.display( );
    cout<<"f4=";    f4.display( );
    cout<<"f5=";    f5.display( );
}
```

```

if(f1>f2) cout<<"f1>f2"<<endl;
if(f2>f3) cout<<"f2>f3"<<endl;
if(f1==f1) cout<<"f1==f1"<<endl;
if(f4!=f5) cout<<"f4!=f5"<<endl;
cout<<"f4=? "; f4.input( );
cout<<"f4="; f4.display( );
f4=~f4; cout<<"f4=~f4; f4=>"; f4.display( );
f1=f2+f4; cout<<"f1=f2+f4; f1=>"; f1.display( );
f1=f2-f4; cout<<"f1=f2-f4; f1=>"; f1.display( );
f1=f2*f4; cout<<"f1=f2*f4; f1=>"; f1.display( );
f1=f2/f4; cout<<"f1=f2/f4; f1=>"; f1.display( );
f1=-f4; cout<<"f1=-f4; f1=>"; f1.display( );
f5=f1+f2-f3; cout<<"f5=f1+f2-f3; f5=>"; f5.display( );
}

```

程序执行后，屏幕显示结果为：

```

f1=2/3
f2=-2/3
f3=-3/5
f4=0
f5=6
f1>f2
f1==f1
f4!=f5
f4=? (above below):3 -6
f4=-1/2
f4=~f4; f4=>-2
f1=f2+f4; f1=>-8/3
f1=f2-f4; f1=>4/3
f1=f2*f4; f1=>4/3
f1=f2/f4; f1=>1/3
f1=-f4; f1=>2
f5=f1+f2-f3; f5=>29/15

```

[分析提示]

实现与范例 7.14 相同的功能，只是将 `fraction` 自定义类中使用的普通函数改写成了运算符重载函数，而且使用的是类成员方式（而非友元方式）。与使用普通函数相比，改写成运算符重载函数后，调用处只需像使用一般运算符那样来对它们进行使用，提高了程序的易读性、易维护性与易使用性。

[解答程序]

```

#include <iostream.h>
#include <math.h>
#include <process.h>

class fraction          //自定义的分数类型 fraction
{
    ...                //内容与题目处相同，略
}

fraction::fraction(int a, int b)    //二参构造函数，带来分子分母
{
    above=a;
    below=b;
}

fraction::fraction(int a)          //一参构造函数，带来分子，分母隐含为 1
{
    above=a;
    below=1;
}

fraction::fraction( )              //无参构造函数，隐含分子为 0，分母为 1
{
    above=0;
    below=1;
}

void fraction::makeCommonD(fraction& b)
{
    //私有成员函数 makeCommonD，负责对两个分数进行通分
    //两个分数进行通分，且保障通分后两个分数的分母均为正数
    (*this).reduction( );          //第一分数约分(经约分后的分母总被化成了正数)
    b.reduction( );                //第二个分数约分(经约分后的分母总被化成了正数)
    above*=b.below;                //改变调用者对象的分子
    b.above*=below;                //改变参数对象的分子
    int temp=b.below*below;
    b.below=below=temp;            //改变两个对象的分母为同一个正数
}

void fraction::reduction( )        //私有成员函数 reduction，负责对分数进行约分
{

```

```

        //先求出分子分母的最大公约数 c, 而后分子分母同时除以 c
        //且保障约分后二分数的分母均为正数
if(below<0)    //为保证负号在分子前, 且保证分子分母都为负数时消去负号
{
    //即是说, 经约分后的分母总被化成了正数
    above=-above; below=-below;
}

        //先求出分子分母的最大公约数 c
int a=abs(above), b=abs(below), c,m,n;    //说明五个 int 型变量
m=a; n=b;    //m 与 n 的初值为 a 和 b
while (m%n)    //当 m%n 不等于 0 时继续循环
{
    int tmp=m;
    m=n;    n=tmp%n;
    //注意, 若 m%n 不等于 0 时, gcd(m,n)=gcd(n, m%n)
    //即是说, 将 n 当作新 m, 将 m%n 当作新的 n 后, 接着去判断 m%n 是否等于 0
}
c=n;    //此时 m%n 等于 0, 所求最大公约数 c 等于 n
above/=c;    //而后分子除以 c
below/=c;    //分母除以 c
}

fraction fraction::operator ~( )    //求倒数,分子分母互换后再约分
{
    fraction r;    //临时对象 r
    r.above=below;    //分子为原分母
    r.below=above;    //分母为原分子
    r.reduction( );    //对分数进行约分
    return r;    //返回 r
}

fraction fraction::operator +(fraction b)    //两个分数相加
{
    fraction r;    //临时对象 r
    makeCommonD(b);    //先将两个分数通分
    r.above=b.above+above;    //再分子相加
    r.below=b.below;    //分母变为同分母
    r.reduction( );    //对分数进行约分
    return r;    //返回 r
}

```



```
fraction fraction::operator -( )           //一目减运算
{
    fraction r;
    r.above=-above;                        //将分子的符号取反
    r.below=below;                        //分母不改变
    r.reduction( );                       //对分数进行约分
    return r;
}

bool fraction::operator ==(fraction b)     //等于运算
{
    (*this).makeCommonD(b);              //先通分且保障通分后两个分数的分母均为正数
    return(above==b.above);               //结果转化为“ 分子等于” 运算
}

bool fraction::operator !=(fraction b)
{
    (*this).makeCommonD(b);              //先通分且保障通分后两个分数的分母均为正数
    return(above!=b.above);               //结果转化为“ 分子不等于” 运算
}

bool fraction::operator >(fraction b)
{
    (*this).makeCommonD(b);              //先通分且保障通分后两个分数的分母均为正数
    return(above>b.above);                //结果转化为“ 分子大于” 运算
}

bool fraction::operator <(fraction b)
{
    (*this).makeCommonD(b);              //先通分且保障通分后两个分数的分母均为正数
    return(above<b.above);                //结果转化为“ 分子小于” 运算
}

bool fraction::operator >=(fraction b)
{
    (*this).makeCommonD(b);              //先通分且保障通分后两个分数的分母均为正数
    return(above>=b.above);               //结果转化为“ 分子大于等于” 运算
}
```

```

bool fraction::operator <=(fraction b)
{
    (*this).makeCommonD(b);           //先通分且保障通分后两个分数的分母均为正数
    return(above<=b.above);           //结果转化为“ 分子小于等于” 运算
}

void fraction::display( )              //显示对象数据
{
    this->reduction( );
    //保障通过构造函数形成的对象也进行了约分(其他从输入到运算都已随时约分过了)
    if(below==1||above==0)             //对整数及 0 的处理
        cout<<above<<endl;
    else
    {
        //既有分子又有分母
        cout<<above<<"/";
        cout<<below<<endl;
    }
}

void fraction::input( )
{
    //输入对象数据
    cout<<"(above below):";           //提示输入
    cin>>above>>below;
    if(below==0)
    {
        //若输入的分母为 0，提示出错信息
        cout<<"ERROR! below=0! set default: 0/1"<<endl;
        above=0;        below=1;
    }
    if(below<0)
    {
        //为保证负号在分子前，且保证分子分母都为负数时消去负号
        above=-above; below=-below;
    }
}

```

```

void main()                                //主函数 main
{
    //说明 fraction 类对象，对定义的各函数进行调用，以验证它们的正确性
    ...                                    //内容与题目处相同，略
}

```

[输出结果]

与题目处相同，略。

9.4 在处理链表的 link 类中重载下标运算符

在自定义的用于处理链表的 link 类中重载下标运算符“[]”，从而把链表当作数组，而后则可对该 link 类对象通过使用下标变量方式来访问链表元素。所要编制的程序框架如下所示，请将程序补充完整，而后上机进行调试运行。

```

struct item                                //链表元素项为 item 结构体
{
    int dat;                               //链表项数据
    item *next;                            //指向下一链表项的指针
};

class link
{
    item *head, *tail;                     //链表首元素指针 head，末元素指针 tail
    int itemNum;                           //链表实际元素个数
public:
    int& operator[ ](int index);           //对 link 类重载 ‘[]’ 运算符
                                           //获取链表第 index 项的 dat 元素值并返回
    ...
};

void main( )
{
    link LinkObj;                           //说明一个 link 类对象 LinkObj
    ...
    LinkObj[1]=666;                         //对下标运算符(函数)的调用
    ...
}

```



```

        head=tail=temp;
    }
    else //若链表非空时，将新生成项插入链尾
    {
        tail->next=temp;
        tail=temp;
    }
    itemNum++; //链表元素个数增加 1
}

link::~link() //析构函数，释放链表所占动态空间
{
    if(head==NULL) return;
    else
    {
        item *temp;
        while(head!=NULL) //将以 head 为首的链表各项所占动态空间统统释放掉
        {
            temp=head;
            head=head->next;
            delete temp; //释放动态空间
        }
    }
}

int& link::operator [ ] (int index) //获取链表第 index 项的 dat 元素值并返回
{
    //重载“ [ ] ”运算符，把链表当作数组，从而可使用下标方式访问链表元素
    if(index<0||index>itemNum-1) //检验下标范围不得超界(也规定下标从 0 开始)
    {
        cout<<"beyond the limits!"<<endl;
        exit(0);
    }
    item* temp=head; //使 temp 指向链首
    if(index==0) //若下标 index 为 0 时
        return temp->dat; //返回首项的 dat 元素值
    for(int i=0; i<index; i++) //从链表首项开始，直到下推到第 index 项
    {
        temp=temp->next;
    }
}

```

```

    }
    return temp->dat;           //返回第 index 项的 dat 元素值
}

void main( )
{
    link LinkObj;              //说明一个 link 类对象 LinkObj
    for (int i=1; i<=5; i++)
        LinkObj.add_to_tail(i*i);    //往链表 LinkObj 中加入五个数
    for (i=0; i<5; i++)
        cout<<setw(5)<<LinkObj[i];
                                   //使用下标方式，依次取出链表中的数并显示出来
    cout<<endl;
    LinkObj[1]=82;
                                   //自定义类中的下标变量——LinkObj[1]可作为左值出现
                                   //(由于重载“[]”运算符时返回的是引用)

    LinkObj[3]++;
                                   //同理自定义类中的“下标变量”——LinkObj[3]也可作为左值出现
    LinkObj[0]=LinkObj[2]+LinkObj[4];
    for (i=0; i<5; i++)
        cout<<setw(5)<<LinkObj[i];    //显示出链表中的各数
    cout<<endl;
}

```

[输出结果]

```

1    4    9    16   25
34   82    9    17   25

```

9.5 重载全局运算符 new 与 delete

下述程序中重载了全局运算符 new 与 delete，即是说，本程序中用到的所有 new 与 delete 函数均对应于被用户重载的运算符 new 和 delete，请阅读程序后给出其执行结果。

```

#include <stdio.h>
#include <malloc.h>
void* operator new(unsigned bytes)           //注 1
{
    printf("SELF-GLOBAL—operator new! bytes=%d\n", bytes);
}

```

```

        void* pvTemp=malloc(bytes);
        return pvTemp;
    }

    void operator delete(void *pvMem)                                //注 2
    {
        printf("SELF—GLOBAL—operator delete!\n");
        free(pvMem);
    }

    class claA                                                        //注 3
    {
        int dat[5];
    public:
        claA( ) { printf("claA::claA( )!\n"); }
        ~claA( ) { printf("claA::~claA( )!\n"); }
    };

    void main( )                                                      //注 4
    {
        int* pi=new int(123);                                         //注 5
        delete pi;                                                    //注 6
        claA* pmyCla=new claA;                                       //注 7
        delete pmyCla;                                                //注 8
        claA* pmyClaA=new claA[2];                                   //注 9
        delete [ ]pmyClaA;                                           //注 10
    }

```

[分析提示]

本范例的程序中重载了全局运算符 `new` 与 `delete`；还自定义了一个 `claA` 类(类型)，但在该类中没有自定义 `operator new` 函数。即是说，本程序中用到的所有 `new` 与 `delete` 函数均对应于被用户重载的 `new` 和 `delete`。本例重载的 `new` 函数委托处于 `malloc.h` 头文件中的系统预定义函数 `malloc` 来完成所需动态内存的具体分配，而重载的 `delete` 函数则委托系统预定义函数 `free` 来进行具体的释放工作。

重载运算符 `new` 后，每当创建一个新的动态对象时，编译器将首先调用重载的 `new` 函数按照用户规定的方法去分配动态内存，而后还将立即去调用构造函数。运算符 `delete` 被重载后，每当动态对象“死亡”时，编译都将首先调用析构函数，而后再按照用户规定的方法释放动态内存。由于重载了全局运算符 `new` 与 `delete`，此程序中的输入输出要改用 `stdio.h` 中的 `scanf` 与 `printf` 来完成。

[解答程序]

具体程序见题目处。

注 1: 被重载的全局运算符 new, 通过系统函数 malloc 分配 bytes 大小的空间。

注 2: 被重载的全局 delete 运算符, 通过系统函数 free 将 pvMem 所指向的动态空间释放掉。

注 3: 自定义类类型 claA, 在其类中没有自定义 operator new 函数。

注 4: 主函数, 其中使用的 new 与 delete 均为用户自定义的(被重载的)全局运算符 new 和 delete。

注 5: 输出 SELF-GLOBAL—operator new! bytes=4

注 6: 输出 SELF-GLOBAL—operator delete!

注 7: 输出 SELF-GLOBAL—operator new! bytes=20

输出 claA::claA()!

注 8: 输出 claA::~~claA()!

输出 SELF-GLOBAL—operator delete!

注 9: 输出 SELF-GLOBAL—operator new! bytes=44

输出 claA::claA()!

输出 claA::claA()!

注 10: 输出 claA::~~claA()!

输出 claA::~~claA()!

输出 SELF-GLOBAL—operator delete!

[输出结果]

SELF-GLOBAL—operator new! bytes=4

SELF-GLOBAL—operator delete!

SELF-GLOBAL—operator new! bytes=20

claA::claA()!

claA::~~claA()!

SELF-GLOBAL—operator delete!

SELF-GLOBAL—operator new! bytes=44

claA::claA()!

claA::claA()!

claA::~~claA()!

claA::~~claA()!

SELF-GLOBAL—operator delete!

9.6 对自定义类重载运算符 new 与 delete

下述程序对自定义的类类型 myCla 重载了运算符 new 与 delete, 但没有重载全局的

运算符 `new` 与 `delete`。即是说,本程序中用到的与自定义类类型 `myCla` 相关的 `new` 和 `delete` 均为对该类重载的 `new` 与 `delete` (但动态数组除外),而其他的 `new` 和 `delete` 均使用系统预定义的缺省全局运算符 `new` 和 `delete`。请阅读该程序后写出执行结果。

```
#include <iostream.h>
#include <malloc.h>                                //注 1

class myCla
{
    int dat;
public:
    myCla( )
        cout<<"CONSTRUCTOR—myCla( )!"<<endl;
    }
    ~myCla( )
        cout<<"DESTRUCTOR—~myCla( )!"<<endl;
    }
    void *operator new(unsigned bytes);              //注 2
    void operator delete(void *pvMem);              //注 3
};

void* myCla::operator new(unsigned bytes)           //注 4
{
    cout<<"myCla::operator new! bytes="<<bytes<<endl;
    void* pvTemp=malloc(bytes);
    return pvTemp;
}

void myCla::operator delete(void *pvMem)            //注 5
{
    cout<<"myCla::operator delete!"<<endl;
    free(pvMem);
}

void main( )                                        //注 6
{
    int *p;
    p=new int;                                       //注 7
    delete p;                                       //注 8
```

```

        myCla *p1, *p2;
        p1=new myCla[2];                                //注 9
        delete [ ]p1;                                    //注 10
        cout<<"-----"<<endl;
        p2=new myCla;                                    //注 11
        delete p2;                                        //注 12
    }

```

[分析提示]

本范例的程序中对自定义的类类型 `myCla` 重载了 `new` 与 `delete`，但没有重载全局的运算符 `new` 与 `delete`。即是说，除动态数组外，本程序中用到的与自定义类类型 `myCla` 相关的 `new` 和 `delete` 均使用对该类重载的 `new` 与 `delete`，而其他的 `new` 和 `delete` 则仍要使用系统预定义的缺省全局运算符 `new` 和 `delete`。由于缺省的全局运算符 `new` 和 `delete` 没被重载，所以本程序中仍可使用 `cout` 进行屏幕输出。

与上一范例一样，本例重载的 `new` 与 `delete` 函数中也委托系统的预定义函数 `malloc` 和 `free` 来进行具体的分配与释放工作。每当创建新的 `myCla` 类动态对象时（动态数组除外），编译器也将首先调用为 `myCla` 类重载的 `new` 函数按照用户规定的方法去分配动态内存，而后还将立即去调用构造函数。每当 `myCla` 类动态对象“死亡”时（动态数组除外），编译将首先调用析构函数，而后再按照用户规定的方法释放动态内存。

[解答程序]

具体程序见题目处。

注 1：专为 `myCla` 类重载的 `new`，负责分配动态空间。

注 2：专为 `myCla` 类重载的 `delete`，负责释放动态空间。

注 3：专为 `myCla` 类重载的 `new`，通过系统函数 `malloc` 分配 `bytes` 大小的空间。

注 4：专为 `myCla` 类重载的 `delete`，通过系统函数 `free` 将 `pvMem` 所指向的动态空间释放掉。

注 5：主函数，其中既使用缺省的全局运算符 `new` 与 `delete` 也使用为 `myCla` 类重载的 `new` 与 `delete`。

注 6：对系统预定义的基本数据类型 `int`，使用缺省的全局运算符 `new`。

注 7：缺省的全局运算符 `delete`。

注 8：对任何类型的数组，使用缺省的全局运算符 `new`。

注 9：缺省的全局运算符 `delete`。

注 10：使用 `myCla` 类中所定义的 `operator new` 函数。

注 11：使用 `myCla` 类中所定义的 `operator delete` 函数。

[输出结果]

```

CONSTRUCTOR—myCla( )!
CONSTRUCTOR—myCla( )!

```

```

DESTRUCTOR—~myCla( )!
DESTRUCTOR—~myCla( )!
-----
myCla::operator new! bytes=4
CONSTRUCTOR—myCla( )!
DESTRUCTOR—~myCla( )!
myCla::operator delete!

```

9.7 全权控制运算符 new 与 delete 的分配与释放细节

读程序，写结果。该程序重载了全局的运算符 new 与 delete，但并不使用系统函数 malloc 和 free。实际上，程序事先申请并掌握一大块所谓的动态存储空间（如本程序中的静态数组空间 MemArea），而后在重载的 new 与 delete 函数中，总使用自己已经掌握的那一大块所谓的动态存储空间来进行分配与释放（回收）。即是说，与分配和释放（回收）有关的一切工作（包括分配和回收策略等）都由程序自己来控制、来管理。

```

#include <stdio.h>
#include <process.h>
#include <string.h>
#define MAX_SIZE 10000 //注 1
static char MemArea[MAX_SIZE]; //注 2
static char* pm=MemArea; //注 3

void* operator new(unsigned bytes) //注 4
{
    printf("SELF—GLOBAL—operator new! bytes=%d\n", bytes);
    if(pm+bytes<=MemArea+MAX_SIZE){ //注 5
        {
            pm+=bytes; //注 6
            return (void*)(pm-bytes); //注 7
        }
    }
    else { //注 8
        printf("My—new—fail! I only have %d bytes memory now!", MemArea+MAX_SIZE—pm);
        printf(" but you need %d bytes!\n", bytes);
        return NULL; //注 9
    }
}

```



```

void operator delete(void *pvMem)                                //注 10
{
    printf("SELF—GLOBAL—operator delete!\n");
    if((char*)pvMem>=MemArea && (char*)pvMem<MemArea+MAX_SIZE)
        pm=(char*)pvMem;                                         //注 11
}

void main( )                                                     //注 12
{
    char* pc = (char*)new char[MAX_SIZE];                        //注 13
    strcpy(pc, "1234567abcdefg");
    printf("pc=>%s\n", pc);
    delete pc;                                                    //注 14
    double* pd = (double*)new double(862.13);                   //注 15
    printf("(pd)=%f\n", *pd);                                     //注 16
    double* pdA1 = (double*)new double[10];                     //注 17
    delete [ ]pdA1;                                              //注 18
    double* pdA2 = (double*)new double[1250];                   //注 19
    if(!pdA2)                                                     //注 20
    {
        delete pd;        exit(0);
    }
    delete [ ]pdA2;                                              //注 21
    delete pd;                                                  //注 22
}

```

[分析提示]

该程序重载了全局的运算符 `new` 与 `delete`，但与上两个范例不同的是，本程序并不使用系统预定义函数 `malloc` 和 `free`，而是由程序自己在重载的 `new` 与 `delete` 函数体中来控制管理属于程序自己的一大块所谓的动态存储空间（如本程序中的静态数组空间 `MemArea`）。即是说，重载的 `new` 函数要负责在自己的动态存储空间中进行分配，而重载的 `delete` 函数则要负责回收所释放的空间（仍由自己掌管，而后可再一次被分配使用等）。由于重载了全局运算符 `new` 与 `delete`，所以本程序中使用 `printf` 来替代 `cout` 进行屏幕输出。

[解答程序]

具体程序见题目处。

注 1：动态存储空间的最大容量(字节数)。

注 2： `MemArea` 为由程序事先申请并掌握的那一大块所谓的动态存储空间。

注 3: 总使 `pm` 指向下次可进行分配的动态存储空间首地址。

注 4: 被重载的全局运算符 `new`, 分配 `bytes` 字节的动态空间。总通过自己掌握的那一大块所谓的动态存储空间 `MemArea` 来进行分配与释放(回收), 即是说, 与分配和释放(回收)有关的一切工作(包括分配和回收策略等)都由程序自己来控制、来管理。

注 5: 如果自己掌握的那块动态存储空间尚有 `bytes` 字节的空闲空间时。

注 6: 总使 `pm` 指向下次可进行分配的动态存储空间首地址。

注 7: 返回本次所分得的空间首地址。

注 8: 若自己掌握的那块动态存储空间已不足 `bytes` 字节的空闲空间时。

注 9: 提示信息后返回 `NULL`。

注 10: 被重载的全局 `delete` 运算符, 释放掉 `pvMem` 所指向的动态空间, 只简单地将 `pvMem` 所指位置以下的动态空间全部“回收”。

注 11: 只简单地将下次可进行分配的动态存储空间的首地址改为 `pvMem`, `pm` 总指向下次可进行分配的动态存储空间首地址。

注 12: 主函数, 其中使用被重载的全局运算符 `new` 与 `delete`。

注 13: 使用被重载的全局运算符 `new`, `MAX_SIZE` 大小的空间为被重载的 `new` 运算符可给予的最大一块动态空间。

注 14: 使用被重载的全局运算符 `delete` 释放掉 `MAX_SIZE` 大小的动态空间。

注 15: 申请动态空间并带去初值 862.13, 注意, 分配到的 `pd` 空间并没有马上释放。

注 16: 输出 `*pd` 之值即 862.13。

注 17: 申请动态数组。

注 18: 释放动态数组。

注 19: 试图分配超过 `new` 之给予能力的空间量时会提示信息并返回 `NULL`。

注 20: 如若没申请到所需的动态空间(即返回 `NULL`)时, 释放以前分配到的 `pd` 空间, 而后退出程序。

注 24: 释放 `pdA2` 动态空间。

注 25: 释放以前分配到的 `pd` 空间。

[输出结果]

```
SELF-GLOBAL-operator new! bytes=10000
pc=>1234567abcdefg
SELF-GLOBAL-operator delete!
SELF-GLOBAL-operator new! bytes=8
(*pd)=862.130000
SELF-GLOBAL-operator new! bytes=80
SELF-GLOBAL-operator delete!
SELF-GLOBAL-operator new! bytes=10000
My-new-fail! I only have 9992 bytes memory now! but you need 10000 bytes!
SELF-GLOBAL-operator delete!
```

9.8 自定义的类型转换运算符(函数)

读下述程序并写出执行后的屏幕显示结果。该程序在 `complex` 类中自定义了类型转换运算符(函数),从而可将本 `complex` 类对象(的数据)转换为程序所指定的类型(数据),具体转换过程由自定义函数体来描述。

例如, `double(comobj)`或`(double)comobj`,就是显式地让系统将类对象 `comobj` 的类型强制转换为 `double` 类型。此处假设 `comobj` 为自定义类 `complex` 的类对象,而且在自定义类 `complex` 中定义了将 `complex` 类对象(的数据)转换为 `double` 类型(数据)的转换函数,具体的转换过程在该转换函数体中描述(定义)。

```
#include <iostream.h>
class complex                                //注 1
{
    double real, image;
    public:
    complex(double r, double i)
    {
        real=r;        image=i;
    }
    operator double ( )                        //注 2
    {
        return real;                            //注 3
    }
};

void main( )
{
    double d;
    complex com1(1010,111), com2 (2020,222), com3(3030,333);
    d=com1;                                    //注 4
    cout<<"d="<<d<<endl;                    //注 5
    d=double(com2);                            //注 6
    cout<<"d="<<d<<endl;                    //注 7
    d=(double)com3;                            //注 8
    cout<<"d="<<d<<endl;                    //注 9
}
```

[分析提示]

在自定义的 `complex` 类中, 通过“ `operator double(){ return real; }`”的形式说明(定义)了一个类型转换运算符(函数, 也称转换函数)。用于实现如下功能: 将本 `complex` 类对象(之数据)转换为系统预定义的 `double` 类型(数据), 具体如何转换由用户程序通过函数体来描述来定义(此例通过函数体中的“ `return real;`”就规定了一种具体的转换关系)。

在进行运算或函数参数传递的过程中, 当系统发现自定义类对象的类型不符或不匹配时, 若程序中对该自定义类定义了转换函数的话, 则系统会自动调用该类的转换函数去对类对象进行类型转换。另外, 如果需要, 程序中还可以利用自定义转换函数进行显式的强制类型转换, 例如本程序 `main` 函数中出现的“ `double(com2)`”以及“ `(double)com3`”就属于这种使用方式。

[解答程序]

具体程序见题目处。

注 1: 自定义复数 `complex` 类(类型), 其 `double` 型数据成员 `real` 与 `image` 表示复数的实部与虚部。

注 2: 将本 `complex` 类对象(之数据)转换为 `double` 类型(数据)的自定义转换函数。

注 3: 具体转换过程由函数体描述(定义), 此处只取复数的实部作为转换结果。当然, 如果需要, 也可返回诸如 `real+image` 之值作为转换结果。

注 4: 左部为 `double` 型, 右部为 `complex` 型, 赋值运算时类型不符。由于定义了从 `complex` 转换为 `double` 的类型转换函数, 此时系统将自动调用转换函数去进行隐式的类型转换。

注 5: 输出转换结果, 即对象 `com1` 的 `real` 值 1010。

注 6: 利用自定义的那一转换为 `double` 的函数, 进行显式的强制类型转换(格式一), 将 `com2` 强制转换为 `double` 类型后, 再赋值给变量 `d`。

注 7: 输出转换结果, 即对象 `com2` 的 `real` 值 2020。

注 8: 利用自定义的那一转换为 `double` 的函数, 进行显式的强制类型转换(格式二), 将 `com3` 强制转换为 `double` 类型后, 再赋值给变量 `d`。

注 9: 输出转换结果, 即对象 `com3` 的 `real` 值 3030。

[输出结果]

`d=1010`

`d=2020`

`d=3030`

9.9 成员指针运算符的使用

下述程序使用了成员指针运算符(“`*`”与“`->*`”), 试给出该程序执行后的屏幕显

示结果。

```
#include <iostream.h>

class myCla
{
public:
    int x; //注 1
    void SetX(int a){x=a;}
    myCla(int a=0):x(a){}
    int fx(int b){return x*x+b;} //注 2
};

void main( )
{
    myCla ob1(20), *cp=&ob1;
    int myCla::*iptr; //注 3
    iptr=&myCla::x; //注 4
    cout<<"ob1.*iptr="<<ob1.*iptr; //注 5
    cout<<"    ob1.x="<<ob1.x<<endl; //注 6
    cout<<"cp->*iptr="<<cp->*iptr; //注 7
    cout<<"    cp->x="<<cp->x<<endl; //注 8
    int (myCla::*fptr)(int); //注 9
    fptr=myCla::fx; //注 10
    cout<<"(ob1.*fptr)(35)="<<(ob1.*fptr)(35); //注 11
    cout<<"    ob1.fx(35)="<<ob1.fx(35)<<endl; //注 12
    cout<<"(cp->*fptr)(12)="<<(cp->*fptr)(12); //注 13
    cout<<"    (cp->fx)(12)="<<(cp->fx)(12)<<endl; //注 14
}
```

[分析提示]

(1) 注意，成员指针是一种特殊的指针，它并非类的成员，要在类外对它进行说明，用来指向指定类中的公有数据成员或公有函数成员。成员指针的值实际上只是成员在各自类对象中的偏移量，也即成员距离类的所有对象首地址的字节数。尽管一个类的各对象在内存中的首地址各不相同，但其每一个成员在对象中的偏移量却总是一个固定数。

(2) 说明一个与指定类相关的数据成员指针的一般格式为：“<类型> <类名>::*<数据成员指针名>”，与普通指针相比，说明数据成员指针时要指定与其相关的类。

若类中某公有数据成员的类型与数据成员指针所指对象的类型相同，则可以使用如下的方式将该数据成员的地址赋给该数据成员指针：“<数据成员指针> = &<类名>::<公有数据成员名>”。要访问一个数据成员指针所指向的内容时，一是通过与数据成员指针相关的类对象，二是通过指向类对象的指针。通过类对象来进行访问时，使用如下格式：

“<类对象>.*<数据成员指针>”，而通过指向类对象的指针来访问时，使用如下格式：
“<类对象>->*<数据成员指针>”。说明与使用与指定类相关的函数成员指针的方法基本相同。

[解答程序]

具体程序见题目处。

注 1: `int` 型的公有数据成员 `x`。

注 2: 返回值类型为 `int`、具有一个 `int` 型参数的公有函数成员 `fx`。

注 3: 在类外说明一个指向类数据成员的指针 `iptr` (简称为数据成员指针 `iptr`，它并非类的成员)。

注 4: 为数据成员指针 `iptr` 赋值，使其指向类的公有数据成员 `x` (可看成是对象首地址偏移量)。 `myCla` 中的 `x` 必须为 `public` 型的! 否则为语法错!

注 5: `ob1` 对象的数据成员指针 `iptr` 所指向的成员——即成员 `x`。

注 6: `ob1` 对象的数据成员 `x`。

注 7: `cp` 所指对象的数据成员指针 `iptr` 所指向的成员——即成员 `x`。

注 8: `cp` 所指对象的数据成员 `x`。

注 9: 在类外说明一个指向类公有函数成员的指针 `fptr` (简称为函数成员指针 `fptr`，它并非类的成员)。

注 10: 为函数成员指针 `fptr` 赋值，使其指向类公有函数成员 `fx`。 `myCla` 中的 `fx` 必须为 `public` 型的! 否则为语法错!

注 11: `ob1` 对象的函数成员指针所指向的函数成员——即成员 `fx`。

注 12: `ob1` 对象的函数成员 `fx`。

注 13: `cp` 所指对象的函数成员指针所指向的函数成员——即成员 `fx`。

注 14: `cp` 所指对象的函数成员 `fx`。

[输出结果]

```
ob1.*iptr=20,   ob1.x=20
cp->*iptr=20,   cp->x=20
(ob1.*fptr)(35)=435,   ob1.fx(35)=435
(cp->*fptr)(12)=412,   (cp->fx)(12)=412
```

9.10 自定义矩阵 `matrix` 类(类型)中的运算符重载

自定义矩阵 `matrix` 类型，在其中重载如下几个可用来实现矩阵加、矩阵减、矩阵乘、矩阵转置、矩阵赋值的运算符 “+”、“-”、“*”、“~”、“=”，以及用于完成矩阵输入输出的运算符“>>”和“<<”(其中涉及到的各输入矩阵的大小均由用户动态决定)。进一步，参照 7.20 的自定义类 `Matrix` 的实现方法(该类用来求出具有 `lines` 行的满秩矩阵的逆矩阵)，再将求逆运算添加进去。即是说，可进一步增加用来实现矩阵求逆的重载运算

符“!”。如, void operator! ();(用于求出矩阵的逆, 并使解处于对象的 MatrA 矩阵的右半部分。并编制主函数, 通过不同对象对各种重载运算符进行使用(以验证它们的正确性)。

下面给出实现方式之一的程序核心代码。

```
class matrix
{
    int r,c;                //r 行 c 列的矩阵
    float** mem;            //数据放于 mem 动态数组
    int lines;              //矩阵的维数(行数, 仅当求逆时使用该值)
    double** MatrA;
                            //数组首地址, 仅当求逆时使用, 存放矩阵
                            //初始时为原始矩阵+单位矩阵
                            //它所具有的 lines 行、2*lines 列的存储空间通过 new 来动态分配
    float getElem(int a, int b);        //获取(a,b)元之值
    void setElem(int a, int b, float f); //将(a,b)元置为值 f
    friend istream& operator>>(istream& in, matrix& matr);
        //从流 in 上输入 matr 矩阵数据——输入行数 r 及列数 c
        //而后分配 mem 动态数组并输入 r*c 个矩阵数据
    friend ostream& operator<<(ostream& out, matrix& matr);
        //在流 out 上输出矩阵 matr
public:
    int getr( );            //获取行数 r
    int getc( );            //获取列数 c
    matrix(int a, int b);    //二参构造函数, a 行 b 列的矩阵, 负责分配 mem 动态数组
    matrix( );              //无参构造函数, 不做任何事情(“ operator>>”
                            //函数处定大小, 输入数据)
    matrix(int Lns);
        //一参构造函数, 仅当求逆时使用, 形参为行数(阶数),
        //动态分配 MatrA 的存储空间
    matrix(matrix& matr);    //拷贝构造函数
    matrix::~matrix( );
        //析构函数, 释放 mem 动态数组或动态分配的 MatrA 的存储空间
    matrix operator+ (matrix &);    //矩阵加
    matrix operator- (matrix &);    //矩阵减
    matrix operator* (matrix &);    //矩阵乘
    void operator= (matrix &);      //矩阵赋值
    matrix operator~ ( );           //矩阵转置
    void operator! ( );             //求解矩阵的逆(解处于对象的 MatrA 矩阵“ 右半” )
```

```

void InSourceMatrix( );
//输入原始矩阵初值, 并形成 MatrA 的右半部分的单位矩阵

void printMatrix( );
//显示“ 整体” 矩阵(lines 行、2*lines 列, 左半部分+右半部分数据)

void printSourceMatrix( ); //按格式显示原始矩阵
// (MatrA 矩阵的左半部分的数据)

void printResultMatrix( ); //按格式显示结果矩阵( MatrA 矩阵的右半部分的数据)
};

void main( ) //主函数, 对 matrix 类(对象)及各种重载运算符进行使用
{
    char choose;
    matrix x,y;
    do
    {
        cout<<"      矩阵运算, 请选择"<<endl;
        cout<<"-----"<<endl;
        cout<<"1. 求和; 2. 求差; 3. 求积; 4. 转置; 5. 求逆; 6. 退出"<<endl;
        cin>>choose;
    }
    while (choose<'1' || choose>'6'); //用户所做选择必须处于 1~6 之内
    if(choose=='6') //6.退出
        exit (0);
    if(choose=='5') //5.矩阵求逆
    {
        cout<<"Input LINES:"; //屏幕提示输入矩阵行数(阶数)
        int lines;
        cin>>lines; //输入行数
        matrix matr1(lines); //定义一个具有 lines 行(2*lines 列)的矩阵对象 matr1
        cout<<"Input "<<lines*lines<<" numbers that will form the matrix:"<<endl;
        //提示输入原始矩阵数据

        matr1.InSourceMatrix( );
        //输入原始矩阵数据, 并形成 MatrA 的右半部分的单位矩阵
        cout<<"--- The source Matrix is ---"<<endl;
        matr1.printSourceMatrix( );
        //按格式输出原始矩阵(MatrA 矩阵的左半部分的数据)
        !matr1; //求解矩阵的逆(解处于 MatrA 矩阵的右半部分)
        cout<<"--- The result Matrix is ---"<<endl;
        matr1.printResultMatrix( );
    }
}

```



```

//按格式输出结果矩阵(Matrx 矩阵的右半部分的数据)
}
else //choose='1'-'4' (1. 求和; 2. 求差; 3. 求积; 4. 转置;)
{
    cout<<"please input the first matrix—x"<<endl;
    cin>>x; //输入第一个矩阵
    cout<<"----- x= -----"<<endl;
    cout<<x; //按格式显示出第一个矩阵
    if(choose<'4') //二目运算时(1. 求和; 2. 求差; 3. 求积;)
    {
        cout<<"please input the second matrix—y"<<endl;
        cin>>y; //输入第二个矩阵
        cout<<"----- y= -----"<<endl;
        cout<<y; //按格式显示出第二个矩阵
    }
    switch(choose)
    {
        //根据用户选择去分支处理
        case'1': //矩阵加
        {
            matrix r; //说明一个 matrix 类对象 r, 此时并无分配动态空间
            r=x+y; //矩阵加, 结果放 r
            //加完返回临时对象时用到拷贝构造函数, 赋值时调用赋值算符重载函数
            //在赋值算符重载函数处为类对象 r 分配动态空间, 并进行赋值处理
            cout<<"----- r=x+y= -----"<<endl;
            cout<<r; //输出结果矩阵 r
            break;
        }
        case'2': //矩阵减
        {
            matrix r; //说明一个 matrix 类对象 r, 此时并无分配动态空间
            r=x-y; //矩阵减, 结果放 r
            //减完返回临时对象时用到拷贝构造函数, 赋值时调用赋值算符重载函数
            //在赋值算符重载函数处为类对象 r 分配动态空间, 并进行赋值处理
            cout<<"----- r=x-y= -----"<<endl;
            cout<<r; //输出结果矩阵 r
            break;
        }
        case'3': //矩阵乘
        {

```

```

        matrix r;           //说明一个 matrix 类对象 r，此时并无分配动态空间
        r=x*y;              //矩阵乘， 结果放 r
//乘完返回临时对象时用到拷贝构造函数，赋值时调用赋值算符重载函数
//在赋值算符重载函数处为类对象 r 分配动态空间，并进行赋值处理
        cout<<"----- r=x*y= -----"<<endl;
        cout<<r;            //输出结果矩阵 r
        break;
    }
    case'4':                //矩阵转置
    {
        matrix r;           //说明一个 matrix 类对象 r，此时并无分配动态空间
        r=~x;               //矩阵转置，结果放 r
//转置完返回临时对象时用到拷贝构造函数，赋值时调用赋值算符重载函数
//在赋值算符重载函数处为类对象 r 分配动态空间，并进行赋值处理
        cout<<"----- r=~x= -----"<<endl;
        cout<<r;            //输出结果矩阵 r
        break;
    }
}                            //switch 结束
}                            //if 结束
}                            //main 结束

```

程序执行后的显示结果如下(结果之一):

矩阵运算，请选择

```

-----
1. 求和; 2. 求差; 3. 求积; 4. 转置; 5. 求逆; 6. 退出
1
please input the first matrix—x
Please input r (row) and c (column) : 2 2
Please input 4 numbers (矩阵数据，逐行输入):
1 2 3 4
----- x= -----
{      1.00,      2.00}
{      3.00,      4.00}
please input the second matrix—y
Please input r (row) and c (column) : 2 2
Please input 4 numbers (矩阵数据，逐行输入):
5 6 7 8
----- y= -----

```

```

{      5.00,      6.00}
{      7.00,      8.00}
----- r=x+y= -----
{      6.00,      8.00}
{     10.00,     12.00}

```

程序执行后的显示结果如下(结果之二):

矩阵运算, 请选择

```

-----
1. 求和; 2. 求差; 3. 求积; 4. 转置; 5. 求逆; 6. 退出
3
please input the first matrix—x
Please input r (row) and c (column) : 2 3
Please input 6 numbers (矩阵数据, 逐行输入):
1 2 3 4 5 6
----- x= -----
{      1.00,      2.00,      3.00}
{      4.00,      5.00,      6.00}
please input the second matrix—y
Please input r (row) and c (column) : 3 2
Please input 6 numbers (矩阵数据, 逐行输入):
1 2 3 4 5 6
----- y= -----
{      1.00,      2.00}
{      3.00,      4.00}
{      5.00,      6.00}
----- r=x*y= -----
{     22.00,     28.00}
{     49.00,     64.00}

```

程序执行后的显示结果如下(结果之三):

矩阵运算, 请选择

```

-----
1. 求和; 2. 求差; 3. 求积; 4. 转置; 5. 求逆; 6. 退出
5
Input LINES:3
Input 9 numbers that will form the matrix:
1 1 1 1 2 1 2 3 3
--- The source Matrix is ---

```

```

1      1      1
1      2      1
2      3      3
--- The result Matrix is ---
3      0      -1
-1     1      0
-1     -1     1

```

[分析提示]

有关矩阵加、矩阵乘、矩阵转置、矩阵求逆等与矩阵相关的求解方法以及相关的程序都已经在前面的有关单元进行过讨论与具体实现，本范例只是利用运算符重载的方式又一次将这些运算组合到一起以形成一个自定义的类 `matrix`，而且对这些运算符均使用类成员的方式进行重载（而不使用友元方式）。另外，本例还要重载用于完成矩阵输入/输出的运算符“>>”和“<<”。重载这两个运算符有些特殊，通常使用友元方式来进行，而且大都使用类似于如下的重载格式：

```

friend istream& operator>>(istream& in, matrix& matr);
friend ostream& operator<<(ostream& out, matrix& matr);

```

其中的“`operator>>`”用于完成从 `istream` 类的流类对象 `in` 上（如对应实参可为 `cin`，即指定从键盘上）输入 `matr` 矩阵的数据，而“`operator<<`”则用于实现在 `ostream` 类的流类对象 `out` 上（如对应实参可为 `cout`，即指定在屏幕上）输出矩阵 `matr` 的有关数据。有关 `istream` 类以及 `ostream` 类的进一步解释与讨论可参看本书第十一单元的“流类库与输入/输出”。

[解答程序]

```

#include<iostream.h>
#include<iomanip.h>
#include<process.h>

class matrix                                //自定义类 matrix
{
    ...                                    //与题目处相同，略
};

matrix::matrix(int a, int b)                //a 行 b 列的矩阵，分配 mem 动态数组
{
    r=a;      c=b;
    mem=new float*[r];                    //分配 mem 动态数组空间(a 行 b 列)
    for(int i=0; i<r; i++)
        *(mem+i)=new float[c];
}

```

```

        MatrA=NULL;
    }

matrix::matrix()
{
    //无参构造函数，不做任何事情(“ operator>>” 函数处定大小，输入数据)
    r=1;      c=1;
    mem=NULL;           //不分配 mem 动态数组空间
    MatrA=NULL;
}

matrix::matrix(matrix& matr)           //拷贝构造函数
{
    r=matr.r;  c=matr.c;
    mem=new float*[r];                 //分配 mem 动态数组空间(r 行 c 列)
    for(int i=0; i<r; i++)
        *(mem+i)=new float[c];
    for(i=0;i<r;i++)
        for(int j=0;j<c;j++)           //调用者对象的(i,j)元赋值 matr 的(i,j)元
            setElem(i, j, matr.getElem(i,j));
    MatrA=NULL;
}

matrix::~matrix()           //析构函数，释放 mem 动态数组
{
    if(mem)
    {
        for(int i=0; i<r; i++)
            delete [ ]mem[i];
        delete [ ]mem;
    }
    if(MatA)                   //原矩阵 Matrix 类的析构函数
    {
        for(int i=0; i<lines; i++)
            delete[ ] MatrA[i];    //内存释放
        delete[ ] MatrA;           //内存释放
    }
}

```

```
float matrix::getElem(int a, int b)           //获取(a,b)元之值
{ return mem[a][b]; }

int matrix::getr( )                          //获取行数 r
{ return r; }

int matrix::getc( )                         //获取列数 c
{ return c; }

void matrix::setElem(int a, int b, float f)   //将(a,b)元置为值 f
{ mem[a][b]=f; }

istream& operator>>(istream& in, matrix & matr) //从流 in 上输入 matr 矩阵数据
{
    //输入行数 r 及列数 c，生成 r 行 c 列的 mem 动态数组，
    //而后输入 r*c 个矩阵数据放入 mem 中
    cout<<"Please input r (row) and c (column) : ";      //提示输入行数 r 及列数 c
    in>>matr.r>>matr.c;                                //输入行数 r 及列数 c
    matr.mem=new float*[matr.r];                        //生成 r 行 c 列的 mem 动态数组
    for(int i=0; i<matr.r; i++)
        *(matr.mem+i)=new float[matr.c];
    cout<<"Please input "<<matr.r*matr.c<<" numbers (矩阵数据，逐行输入):"<<endl;
    for(i=0; i<matr.r; i++)
        for(int j=0; j<matr.c; j++)
            in>>matr.mem[i][j]; //输入 r*c 个矩阵数据，放入新生成的 mem 动态数组中
    return in;
}

ostream& operator<<(ostream& out, matrix& matr) //在流 out 上输出矩阵 matr
{
    out.precision(2);
    out<<setiosflags(ios::fixed);
    for(int i=0; i<matr.r; i++)                      //共处理矩阵的 r 行
    {
        out<<"{" ;                                     //每行数据以一个“ { ”开始
        for(int j=0; j<matr.c; j++)                  //每行具有 c 个数据，逐一输出它们
        {
            out.width(10);                             //每个数据宽度为 10(10 字符宽)
            out<<matr.mem[i][j];                       //显示出 i 行 j 列的矩阵元素
```

```

        if(j<matr.c-1)
            out<<" ";                //非本行末元素时，输出一个“ ,”
        }                            //for j ... ending
        out<<"}";                    //每行数据以一个“ }” 结束
        out<<endl;
    }                                //for i ... ending
    return out;
}

matrix matrix::operator+(matrix& matr)    //矩阵加
{
    if(r!=matr.getr( )||c!=matr.getc( ))    //矩阵加，要求行数列数全同
    {
        cout<<endl<<"WRONG! -- row1!=row2 || col1!=col2"<<endl;
        exit(0);
    }
    matrix res(r,c);                    //申请一个 r 行 c 列的结果矩阵 res
    for(int i=0;i<r;i++)
        for(int j=0;j<c;j++)            //调用者对象的(i,j)元加 matr 的(i,j)元
            res.setElem(i, j, getElem(i,j)+matr.getElem(i,j));
    return res;                        //返回结果矩阵 res
}

matrix matrix::operator-(matrix& matr)    //矩阵减
{
    if(r!=matr.getr( )||c!=matr.getc( ))    //矩阵减，要求行数列数全同
    {
        cout<<endl<<"WRONG! -- row1!=row2 || col1!=col2"<<endl;
        exit(0);
    }
    matrix res(r,c);                    //申请一个 r 行 c 列的结果矩阵 res
    for(int i=0;i<r;i++)
        for(int j=0;j<c;j++)            //调用者对象的(i,j)元减 matr 的(i,j)元
            res.setElem(i, j, getElem(i,j)-matr.getElem(i,j));
    return res;                        //返回结果矩阵 res
}

matrix matrix::operator*(matrix& matr)    //矩阵乘
{

```

```

if(c!=matr.getr( ))                //矩阵乘，要求第一阵的列数等于第二阵的行数
{
    cout<<endl<<"WRONG!—col!=row2"<<endl;
    exit(0);
}
matrix res(r,matr.getc( ));        //结果阵 res 为第一阵的行数，第二阵的列数
for(int i=0; i<r; i++)              //第一阵的行数 r 次循环
    for(int j=0; j<matr.getc( ); j++) //第二阵的列数 getc( )次循环
    {
        float temp=0;
        for(int k=0; k<c; k++)      //第一阵的列数 c 次循环
            temp+=mem[i][k]*matr.getElem(k,j);
        res.setElem(i,j,temp);      //形成 res 的(i,j)处的一个元素，值为 temp
    }
return res;                        //返回结果矩阵 res
}

void matrix::operator= (matrix & matr) //矩阵赋值(赋值运算符重载)
{
    r=matr.r;
    c=matr.c;
    mem=new float*[r];              //分配 mem 动态数组空间(r 行 c 列)
    for(int i=0; i<r; i++)
        *(mem+i)=new float[c];
    for(i=0;i<r;i++)
        for(int j=0;j<c;j++)        //调用者对象的(i,j)元赋值 matr 的(i,j)元
            setElem(i, j, matr.getElem(i,j));
}

matrix matrix::operator ~ ( )        //矩阵转置
{
    if(c!=r)                        //矩阵转置，要求矩阵的列数等于其行数
    {
        cout<<endl<<"WRONG! -- col!=row"<<endl;
        exit(0);
    }
    matrix res(c,r);                //申请一个 c 行 r 列的结果矩阵 res
    for(int i=0;i<c;i++)
        for(int j=0;j<r;j++)

```



```

        res.setElem(i,j,mem[j][i]);
    return res;                //返回结果矩阵 res
}

matrix::matrix(int Lns)
{
    //矩阵求逆时的构造函数(动态分配 MatrA 空间——具有 lines 行、2*lines 列)
    lines=Lns;                //矩阵维数(行数)
    MatrA=new double*[lines]; //动态内存分配(存放矩阵), 共 lines 行
    for(int i=0; i<lines; i++)
        MatrA[i]=new double[2*lines]; //每行 2*lines 列
    mem=NULL;                 //不分配 mem 动态数组空间
}

void matrix::InSourceMatrix( )
{
    //输入原始矩阵初值, 并形成 MatrA 的右半部分的单位矩阵

    int i,j;
    for(i=0; i<lines; i++) //输入原始矩阵初值, 放于 MatrA 的左半部分
        for(j=0; j<lines; j++)
            cin>>MatrA[i][j];
    for(i=0; i<lines; i++) //MatrA 的右半部分的单位矩阵先全置为 0
        for(j=lines; j<2*lines; j++)
            MatrA[i][j]=0;
    int ind=lines;
    for(i=0; i<lines; i++) //形成 MatrA 的右半部分的单位矩阵
        MatrA[i][ind++]=1;
}

void matrix::printMatrix( ) //显示整体矩阵 MatrA
{
    for(int i=0; i<lines; i++) //共显示 lines 行
    {
        for(int j=0; j<2*lines; j++) //每行显示 2*lines 个数
            cout<<setw(8)<<MatrA[i][j]<<" "; //每数占 8 格(域宽)
        cout<<endl;
    }
}

```

```

void matrix::printSourceMatrix( )           //显示原始矩阵(处于 MatrA 数组的左半部分)
{
    for(int i=0; i<lines; i++)              //共显示 lines 行
    {
        for(int j=0; j<lines; j++)          //每行显示 lines 个数
            cout<<setw(8)<<MatrA[i][j]<<" "; //每数占 8 格(域宽)
        cout<<endl;
    }
}

void matrix::printResultMatrix( )           //显示结果矩阵(处于 MatrA 数组的右半部分)
{
    for(int i=0; i<lines; i++)              //共显示 lines 行
    {
        for(int j=lines; j<2*lines; j++)    //每行显示 lines 个数
            cout<<setw(8)<<MatrA[i][j]<<" "; //每数占 8 格(域宽)
        cout<<endl;
    }
}

void matrix::operator! ( )                  //求解矩阵的逆，结果放在 MatrA 数组的右半部分
{
    int k,i,j;
    for (k=1; k<lines; k++)                 //矩阵的左半部分消为上三角阵
    {
        for (i=k; i<lines; i++)
        {
            double factor=-MatrA[i][k-1]/MatrA[k-1][k-1];
            for (j=k-1; j<2*lines; j++)
                MatrA[i][j]+=MatrA[k-1][j]*factor;
        }
    }
    //printMatrix( );                       //输出矩阵(矩阵的左半部分为上三角阵)
    for (i=0; i<lines; i++)                 //矩阵的左半部分主对角线消为 1
    {
        double factor=MatrA[i][i];
        for (j=i; j<2*lines; j++)
            MatrA[i][j]/=factor;
    }
}

```

```
//printMatrix();           //输出矩阵(矩阵的左半部分主对角线为 1)
for (k=lines-2; k>=0; k--)   //矩阵的左半部分消为单位矩阵, 此时的右半部分成为结果
{
    for (i=k; i>=0; i--)
    {
        double factor=-MatrA[i][k+1];
        for (j=k+1; j<2*lines; j++)
            MatrA[i][j]+=(factor*MatrA[k+1][j]);
    }
}

void main( )                //主函数, 对 matrix 类(对象)及各种重载运算符进行使用
{
    ...                      //与题目处相同, 略!
}
```

[输出结果]

与题目处相同, 略。

第十章 模板及其应用

[语法要点]

详细内容可参阅教材第九章。

(1) 通常设计的算法(如某一个函数体中的那些处理语句)是可以处理多种数据类型的。利用函数模板(带类型参数的函数),就可一次定义出具有共性(除类型参数外,其余全相同)的一组函数——可以处理多种不同类型数据的函数。函数模板定义的格式如下: `template < class 类型形参名 1, ... , class 类型形参名 n > 返回类型 函数模板名 ( 形参表 ) { 函数体 }`

注意,与一般函数定义不同的是,除语法格式上要求在定义开头处加上“`template < class 类型形参名 1, ... , class 类型形参名 n >`”之外,还应在随后的返回类型或形参表或函数体中使用上述在尖括号中列出的那些类型形参名(否则将没有可变性,只能定义出一个具体的函数)。对函数模板的调用则类似于调用一般函数,用户只需给出具体的实参,而系统则根据所提供的实参信息,将其实例化为一个具体的函数(也称模板函数),而后再去调用执行。另外注意,与一般函数调用不同的是,对函数模板的调用不进行实参到形参类型的自动转换。即是说,调用函数的实参表在类型上必须与函数模板的形参表完全匹配。

(2) C++允许出现函数模板与某一函数同名的所谓重载的使用方法。但注意,在这种情况下,每当遇见函数调用时,C++编译器都将首先检查是否存在重载函数,若匹配成功则调用该重载函数,否则再去匹配函数模板。另外,C++也允许出现同名但参数表不相同的函数模板,这即是函数模板重载的使用方法。系统靠调用时的实参表来对它们进行区分。下面给出出现上述两种情况的示例性程序段:

```
template <class T> T min (T a, T b)           //模板 min
{ return (a<b?a:b); }

char* min (char* a, char* b)                 //函数 min
{ return (strcmp(a,b)<0?a:b); }

template <class Type> Type sum (Type* array, int size) //二参的模板 sum
{
    { ... }
}

template <class Type> Type sum (Type* a1, Type* a2, int size) //三参的模板 sum
{ ... }
```

(3) 利用类模板(带类型参数或普通参数的类),一次就可定义出具有共性(除类型参数或普通参数外,其余全相同)的一组类。即是说,通过使用类模板,可使得所定义类中的某些数据成员、某些成员函数的参数、某些成员函数的返回值都可以是任意类型

的(包括系统预定义类型以及用户自定义类型),从而使得同一段程序可用于处理多种不同类型的对象。类模板定义格式如下:

```
template < 类型形参或普通形参 1 的说明, ... , 类型形参或普通形参 n 的说明 >
class 类模板名 { 带上述类型形参或普通形参名的类定义体 }
```

要求说明类型形参时,使用“class 类型形参名”的方式;说明普通形参时,使用“类型 普通形参名”的方式。注意,与一般类定义不同的是,除语法格式上要求在定义开头处加上“template < ... >”之外,还应在随后的类定义体中使用上述尖括号中列出的“类型形参名”及“普通形参名”(否则将没有可变性,只能定义出一个具体的类)。还要注意,定义类模板时允许使用类型形参与普通形参,而定义函数模板时只允许使用类型形参。

(4) 利用类模板说明类对象时,要随类模板名同时给出对应于类型形参与普通形参的具体实参,从而将类模板实例化为某个具体的类(也称为模板类,而后才可通过该具体类来说明对象)。即是说,要使用如下的格式来实例化类:类模板名<类型形参或普通形参 1 的相应实参, ... , 类型形参或普通形参 n 的相应实参>。其中类型形参的相应实参要为类型名,而普通形参的相应实参必须为一个常量。

假设通过“template <class T, int i> class tmplt { ... }”的形式已经定义了一个类模板 tmplt,其中出现了类型形参 T 以及普通形参 i,那么“tmplt<double, 6>”就意味着要以类型实参 double 和整型实参 6 取代类型形参 T 和普通形参 i,进而将该类模板实例化成了一个具体的类。当然,通过如下的形式也就说明了该实例化类的一个具体对象 ClassInstF: tmplt<double, 6> ClassInstF;

(5) 类模板的成员函数既可以在类定义体内进行说明(自动按内联函数处理),也可以在类外进行说明(定义)。若在类外定义类模板的成员函数时(注意,类模板的成员函数实际上是一个函数模板),要使用如下的一般格式:

```
template < 类型形参或普通形参 1 的说明, ... , 类型形参或普通形参 n 的说明 > 函
数返回类型 类模板名< 类型形参或普通形参 1 的名字, ... , 类型形参或普通形参 n 的
名字 >::成员函数名( 形式参数表 )
{
    ... //成员函数的函数体
};
```

上述的“类型形参或普通形参的名字来自于尖括号中“类型形参或普通形参的说明”,由甩掉说明部分的类型而得,是对类型形参或普通形参的使用。而“类模板名< 类型形参或普通形参 1 的名字, ... , 类型形参或普通形参 n 的名字 >::”所起的作用正是在类外定义成员函数时在函数名前所加的类型限定符!

(6) 可以使用不同的方式来派生出类模板。例如常用如下几种派生类模板的方式:

① 一般类(其中不使用类型参数的类)作基类,派生出类模板(其中要使用类型参数)。

② 类模板作基类,派生出新的类模板,但仅基类中使用类型参数 T 而派生的类模板中不使用类型参数 T)。

③ 类模板作基类,派生出新的类模板,且基类与派生类中均使用相同的类型参数 T。

④ 类模板作基类，派生出新的类模板，但基类中使用类型参数 **T2**，而派生类中只使用另一个类型参数 **T1**（而不使用 **T2**）。

10.1 对一维数组进行排序的函数模板

编写一个对具有 **n** 个元素的一维数组进行排序的函数模板。并通过对它调用以验证其正确性。例如，可设计该函数模板的原型为：

```
template <class Type> void sort(Type *A, int n);
```

其中一维数组 **A** 的元素类型为 **Type**，而且要对数组 **A** 的前 **n** 个数据进行排序。

并可通过如下形式的主函数，对函数模板进行调用，以实现不同类型一维数组的排序：

```
void main(void)
{
    int ai[8]={8,10,2,14,4,6,12,16};           //说明 int 型一维数组 ai 并赋初值
    char ac[13]="jdhretancbal";                //说明 char 型一维数组 ac 并赋初值
    double af[10]={1.2, -3, 77, 20.5, 1, 0, -21.66, 1, 38, 60};
                                           //说明 double 型一维数组 af 并赋初值
    sort(ai,8);                                //对函数模板 sort 进行调用，对 ai 数组的 8 个元素进行排序
    sort(ac,12);                               //对函数模板 sort 进行调用，对 ac 数组的 12 个元素进行排序
    sort(af,6);                                //对函数模板 sort 进行调用，仅对 af 数组的前 6 个元素进行排序
    ...                                         //输出经排序后各数组元素的值
}
```

[分析提示]

利用函数模板一次就可定义出具有共性的一组函数（可以处理多种不同类型数据的函数）。本范例通过“`template <class Type> void sort(Type *A, int n){ ... }`”的形式定义了一个函数模板 **sort**，它含有一个类型参数 **Type**，而 **Type** 可以对应于多种不同的具体类型，从而使 **sort** 可以处理具有不同数据类型的一维数组 **A** 中的各数据（数据类型均为 **Type**）。另外注意，对函数模板的调用则类似于调用一般函数，用户只需给出具体的实参，而系统则根据所提供的实参信息，将其实例化为一个具体的函数（也称模板函数），而后再去调用执行。

[解答程序]

```
#include <iostream.h>

template <class Type> void sort(Type *A, int n);

//函数模板 sort 的原型，其中数组 A 的元素类型为 Type，
//将要对数组 A 的前 n 个数据进行排序。
```

```

void main(void)      //主函数，对模板进行调用，以实现对不同类型数组的排序
{
    int ai[8]={8,10,2,14,4,6,12,16};          //说明 int 型一维数组 ai 并赋初值
    char ac[13]="jdhetancbal";               //说明 char 型一维数组 ac 并赋初值
    double af[10]={1.2, -3, 77, 20.5, 1, 0, -21.66, 1, 38, 60};
                                                //说明 double 型一维数组 af 并赋初值 int i

    cout<<"The initial array ai is: ";        //屏幕提示
    for(i=0;i<8;i++)
        cout<<ai[i]<<" ";                  //输出排序前的 int 型数组 ai 的各元素值
    cout<<endl;
    sort(ai,8);                               //对函数模板 sort 进行调用，对 ai 数组的八个元素进行排序
    cout<<" The sorted array ai is: ";        //屏幕提示
    for(i=0;i<8;i++)
        cout<<ai[i]<<" ";                  //输出排序后的 int 型数组 ai 的各元素值
    cout<<"\n-----\n";
    cout<<"The initial array ac is: ";        //屏幕提示
    for(i=0;i<12;i++)
        cout<<ac[i]<<" ";                  //输出排序前的 char 型数组 ac 的各元素值
    cout<<endl;
    sort(ac,12);                             //对函数模板 sort 进行调用，对 ac 数组的十二个元素进行排序
    cout<<" The sorted array ac is: ";        //屏幕提示
    for(i=0;i<12;i++)
        cout<<ac[i]<<" ";                  //输出排序后的 char 型数组 ac 的各元素值
    cout<<"\n-----\n";
    cout<<"The initial array af is: ";        //屏幕提示
    for(i=0;i<10;i++)
        cout<<af[i]<<" ";                  //输出排序前的 double 型数组 af 的各元素值
    cout<<endl;
    sort(af,6);                              //对函数模板 sort 进行调用，对 af 数组的十个元素进行排序
    cout<<" The sorted array af is: ";        //屏幕提示
    for(i=0;i<10;i++)
        cout<<af[i]<<" ";                  //输出排序后的 double 型数组 af 的各元素值
    cout<<"\n-----\n";
}

```

//对具有 n 个元素的数组 A 进行排序的函数模板 sort，数组元素类型为 Type

```

template <class Type> void sort(Type *A, int n )
{
    for (int i=0; i<n-1; i++)

```

```

        for(int j=i+1; j<n; j++)
            if(A[i]>A[j])                //若顺序不对, 则二元素值互换
            {
                Type tmp=A[i];          //设一个 Type 类型的临时变量 tmp
                A[i]=A[j];
                A[j]=tmp;
            }
    }

```

[输出结果]

```

The initial array ai is: 8  10  2  14  4  6  12  16
The sorted array ai is: 2  4  6  8  10  12  14  16
-----
The initial array ac is: j  d  h  r  e  t  a  n  c  b  a  l
The sorted array ac is: a  a  b  c  d  e  h  j  l  n  r  t
-----
The initial array af is: 1.2  -3  77  20.5  1  0  -21.66  1  38  60
The sorted array af is: -3  0  1  1.2  20.5  77  -21.66  1  38  60
-----

```

10.2 二维数组中找最大元和最小元的函数模板

编写一个在具有 m 行 n 列的二维数组各元素中找出最大元和最小元并显示出来的函数模板。并通过主函数对它调用以验证其正确性。例如, 可设计该函数模板的原型为:

```
template <class Type> void MaxMin (Type *A, int m, int n );
```

其中二维数组 A 的元素类型为 $Type$ 。由第一参数传递过来 $Type *$ 类型的首元素指针, 而后可按一维数组来对待处理后继的 m 乘以 n 个数据), 数组 A 具有 m 行 n 列, 要在数组 A 的各元素中找出最大元和最小元并把它们显示出来。

并可通过如下形式的主函数, 对函数模板进行调用, 以找出不同类型、不同大小数组中的最大元和最小元。

```

void main(void)
{
    int ai[2][3]={ ... };           //说明 int 型 2 行 3 列的数组 ai 并赋初值
    char ac[3][5]={ ... };          //说明 char 型 3 行 5 列的数组 ac 并赋初值
    double af[3][4]={ ... };        //说明 double 型 3 行 4 列的数组 af 并赋初值
    MaxMin((int*)ai,2,3);
    //对函数模板 MaxMin 进行调用, 求 2 行 3 列的 ai 数组中的最大元和最小元
    MaxMin((char*)ac,3,5);
}

```



```

//对函数模板 MaxMin 进行调用, 求 3 行 5 列的 ac 数组中的最大元和最小元
MaxMin((double*)af,3,4);
//对函数模板 MaxMin 进行调用, 求 3 行 4 列的 af 数组中的最大元和最小元
...
}

```

[分析提示]

函数模板 MaxMin 中要处理二维数组 A 的 m 行 n 列的诸元素, 但设计第一参数传递过来的是 Type *类型的首元素指针, 所以具体处理时可以按照如下的一维数组方式进行(共处理 m 乘以 n 个数据, 也即二维数组 A 的 m 行 n 列的诸元素。注意, 二维数组的诸元素在内存中是按行连续存放的, 所以才能够这样来处理)。

```

for (int i=0;i<m;i++)
    for(int j=0;j<n;j++)
    {
        对 A[i*m+j]进行处理; //按一维数组来对待处理 A 中第 i 行第 j 列的数据
    }

```

[解答程序]

```

#include <iomanip.h>
template <class Type> void MaxMin (Type *A, int m, int n )
{
    /*
        定义函数模板 MaxMin, 其中二维数组 A 的元素类型为 Type( 由第一参数传递过来
        Type *类型的首元素指针, 而后可按一维数组来对待处理后继的 m 乘以 n 个数据), 数组 A
        具有 m 行 n 列, 要在数组 A 的各元素中找出最大元和最小元并把它们显示出来。
    */
    Type max=A[0],min=A[0];          //设两个 Type 类型的变量 max 和 min
    for (int i=0;i<m;i++)
        for(int j=0;j<n;j++)
        {
            if(A[i*m+j]>max)           //按“一维数组”来对待处理 A 中第 i 行第 j 列的数据
                max=A[i*m+j];         //发现更大者后更新 max
            if(A[i*m+j]<min)           //按一维数组来对待处理 A 中的数据
                min=A[i*m+j];         //发现更小者后更新 min
        }
    cout<<"max="<<max;               //输出所求出的最大元和最小元
    cout<<" min="<<min<<endl;
}

```

```

void main(void)
{
    //主函数，对模板进行调用，以找出不同类型、不同大小数组中的最大元和最小元
    int i;
    int m=2,n=3;
    int ai[2][3]={ {8,10,2},{14,4,6}};           //说明 int 型数组 ai 并赋初值
    char ac[3][5]={                               //说明 char 型数组 ac 并赋初值
        {'a','s','d','e','h'},
        {'r','t','u','m','g'},
        {'l','k','a','s','e'}
    };
    double af[3][4]=                             //说明 double 型数组 af 并赋初值
    {
        {12.1, 23, 3.56, -38.5},
        {-2.1, -0.63, 3.656, -1},
        {127.1, 66, 8.56, 8.5}
    };
    cout<<"the initial array ai is: "<<endl;      //屏幕提示
    for (i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
            cout<<ai[i][j]<<" ";                //按格式输出 int 型矩阵 ai 的各元素值
        cout<<endl;
    }
    cout<<"the MaxMin of array ai is: ";          //屏幕提示
    MaxMin((int*)ai,2,3);
    //对函数模板 MaxMin 进行调用，求 2 行 3 列的 ai 数组中的最大元和最小元
    cout<<"-----";
    m=3; n=5;
    cout<<endl<<"the initial array ac is: "<<endl; //屏幕提示
    for (i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
            cout<<ac[i][j]<<" ";                //按格式输出 char 型矩阵 ac 的各元素值
        cout<<endl;
    }
    cout<<"the MaxMin of array ac is: ";          //屏幕提示
    MaxMin((char*)ac,3,5);
    //对函数模板 MaxMin 进行调用，求 3 行 5 列的 ac 数组中的最大元和最小元

```

```

cout<<"-----";
m=3; n=4;
cout<<endl<<"the initial array ai is: "<<endl;          //屏幕提示
for (i=0;i<m;i++)
{
    for(int j=0;j<n;j++)
        cout<<setw(6)<<ai[i][j]<<" ";          //按格式输出 double 型矩阵 ai 的各元素值
    cout<<endl;
}
cout<<"the MaxMin of array ai is: ";          //屏幕提示
MaxMin((double*)ai,3,4);
    //对函数模板 MaxMin 进行调用, 求 3 行 4 列的 ai 数组中的最大元和最小元
cout<<"-----\n";
}

```

[输出结果]

```

the initial array ai is:
8  10  2
14  4  6
the MaxMin of array ai is: max=14  min=2
-----
the initial array ac is:
a  s  d  e  h
r  t  u  m  g
l  k  a  s  e
the MaxMin of array ac is: max=u  min=a
-----
the initial array af is:
12.1      23      3.56    -38.5
-2.1     -0.63    3.656     -1
127.1     66      8.56     8.5
the MaxMin of array af is: max=127.1  min=-38.5
-----

```

10.3 类模板的使用

读下面的程序(其中使用了类模板), 写出执行后的显示结果。

```
#include <iostream.h>
```

```
#include <process.h>

template <class T> class valueAccess //注 1
{
    T value; //注 2
    bool haveValue; //注 3
public:
    valueAccess(void); //注 4
    T GetValue(void); //注 5
    void PutValue(T x); //注 6
};

template <class T>
valueAccess<T>::valueAccess(void) //注 7
{
    haveValue=false;
}

template <class T>
T valueAccess<T>::GetValue(void) //注 8
{
    if (haveValue)    return value;
    else
    {
        cout << "No value present!" << endl;
        exit(1);
    }
}

template <class T>
void valueAccess<T>::PutValue(T x) //注 9
{
    haveValue=true;
    value = x;
}

struct Student //注 10
{
    char  name[20];
    long  id;
```

```

        float score;
    };

    void main(void)
    {
        valueAccess<int> iObj1, iObj2;           //注 11
        iObj1.PutValue(321);                     //注 12
        iObj2.PutValue(-876);
        cout << "iObj1.GetValue( )=>" << iObj1.GetValue( ) << endl;    //注 13
        cout << "iObj2.GetValue( )=>" << iObj2.GetValue( ) << endl;
        Student maji = {"ma ji", 1022, 89};      //注 14
        valueAccess<Student> sObj;               //注 15
        sObj.PutValue(maji);
        cout << "sObj.GetValue( ).id=>" << sObj.GetValue( ).id << endl;    //注 16
        valueAccess<double> dObj;               //注 17
        cout << "dObj.GetValue( )=>" ;
        cout << dObj.GetValue( ) << endl;        //注 18
    }

```

[分析提示]

(1) 利用类模板(带类型参数或普通参数的类),一次就可定义出具有共性(除类型参数或普通参数外,其余全相同)的一组类。本范例通过“`template <class T> class valueAccess { ... }`”的形式定义了一个类模板 `valueAccess`,它含有一个类型参数 `T`,而 `T` 可以对应于多种不同的具体类型,从而使 `valueAccess` 可以处理具有不同数据类型的各种数据。

(2) 通过“`valueAccess<int>`”的方式将类模板实例化为一个具体的类,而通过“`valueAccess<int> iObj1, iObj2;`”来说明该实例化类的两个具体类对象 `iObj1` 与 `iObj2`。

(3) 类模板中的成员函数在类外说明时,相当于是在说明一个函数模板。如本例中要通过“`template <class T> void valueAccess<T>::PutValue(T x) { ... }`”的格式在类模板的体外定义 `PutValue` 成员函数。

[解答程序]

具体程序见题目处。

注 1: 类模板 `valueAccess`: 实现对任意 `T` 类型的数据 `value` 进行存取。

注 2: `value` 用于存放任意类型的数据。

注 3: `haveValue` 标记 `value` 是否已被存入内容。

注 4: 构造函数,将 `haveValue` 置为 `false`。

注 5: 获取数据成员 `value` 之值。

注 6: 向数据成员 `value` 存入值。

注 7: 以下为三个成员函数的类外说明。注意, 模板类的成员函数在类外说明时, 相当于一个模板函数。此处的第一个为构造函数(模板)。

注 8: 第二个为 GetValue 函数(模板), 用来获取数据成员 value 中存放的数据。但若试图提取未初始化的数据, 则终止程序。

注 9: 第三个为 PutValue 函数(模板), 用来向数据成员 value 存入值 x。并将 haveValue 置为 TRUE, 表示 value 中已存入了数值。

注 10: 结构体 Student 的定义, 具有三个分量: 姓名、学号、成绩。

注 11: 用 int 类型实例化 valueAccess 类模板, 并定义两个此种类型的类对象。

注 12: 向对象 iObj1 中存入数据 321。

注 13: 输出对象 iObj1 的数据成员。

注 14: 定义 Student 结构体变量 maji 的同时赋给其初值。

注 15: 用 Student 类型实例化 valueAccess 类模板, 并定义此种类型的类对象 sObj。注意, sObj 对象的数据成员 value 为 Student 类型。

注 16: 输出对象 sObj 的数据成员 value(为 Student 类型)的分量 id 值。

注 17: 用 double 类型实例化 valueAccess 类模板, 并定义此种类型的类对象 dObj。

注 18: 输出对象 dObj 的数据成员。由于 dObj 未经初始化, 在执行函数 dObj.GetValue() 过程中导致程序终止。

[输出结果]

```
iObj1.GetValue() =>321
iObj2.GetValue() =>-876
sObj.GetValue().id =>1022
dObj.GetValue() =>No value present!
```

10.4 既使用类型参数又使用普通参数的类模板

读下述程序, 写出该程序执行后的屏幕显示结果。注意, 本程序的类模板定义中使用了类型参数以及普通参数(非类型参数)。

```
#include "iostream.h"
#include "string.h"
template <class T, int i> class tmplt           //注 1
{
public:
    T dat[i];                                   //注 2
    T getDat(int j);                           //注 3
};
```

```
template <class T, int i>
T tmplt<T,i>::getDat(int j)           //注 4
{
    return *(dat+j);                 //注 5
};

void main( )                          //注 6
{
    tmplt<char, 5> ClassInstA;         //注 7
    strcpy(ClassInstA.dat,"rsxyz");    //注 8
    for(int i=0; i<5; i++)             //注 9
    {
        char res=ClassInstA.getDat(i);
        cout<<res<<" ";
    }
    cout<<endl;
    tmplt<double, 6> ClassInstF;       //注 10
    for(i=0; i<6; i++)                 //注 11
        ClassInstF.dat[i]=2*i+0.12;
    for(i=0; i<6; i++)                 //注 12
    {
        double res=ClassInstF.getDat(i);
        cout<<res<<" ";
    }
    cout<<endl;
}
```

[分析提示]

本程序中以“`template <class T, int i> class tmplt { ... }`”的样式定义了类模板 `tmplt`，其中要使用类型形参 `T` 以及普通形参 `i`。并在类定义体中使用了“`T dat[i];`”形式的说明语句，来说明一个 `T` 类型的公有数据成员 `dat`，它为一个数组，分量类型为可变类型 `T`，数组大小也随普通形参 `i` 的值而变化。利用该类模板来实例化一个具体的类类型时，对应于类型形参 `T` 的相应实参要为类型名，而普通形参 `i` 的相应实参必须为一个常量。如“`tmplt<char, 5> ClassInstA;`”，就以类型实参 `char` 和整型常量实参 `5` 取代类型形参 `T` 和普通形参 `i` 后，实例化为一个具体的类（其 `dat` 数组的类型为 `char`，大小为 `5`），并说明了一个此具体类的对象 `ClassInstA`。

[解答程序]

具体程序见题目处。

注 1: 类模板 `tmplt`, 使用类型形参 `T` 以及普通形参 `i`。

注 2: `T` 类型的公有数据成员 `dat`, 其大小也随普通形参 `i` 的值变化(灵活性大!)

注 3: `T` 类型的公有函数成员 `getDat`。

注 4: `T` 类型的公有函数成员 `getDat` 在类外定义, 函数名前要加类限定符 `tmplt<T,i>::`”。

注意, 在类外定义类模板的函数成员时, 其格式类似于函数模板的定义。

注 5: 返回 `T` 类型公有数据成员 `dat`(数组) 的第 `j` 个分量。

注 6: `main` 函数, 创建类模板 `tmplt` 的实例对象并对它们进行使用。

注 7: 以类型实参 `char` 和整型实参 `5` 取代类型形参 `T` 和普通形参 `i` 后, 则实例化为一个具体的类 其 `dat` 数组的类型为 `char`, 大小为 `5`), 说明一个此具体类的对象 `ClassInstA`。

注 8: 为类对象 `ClassInstA` 的 `dat` 数组赋值。

注 9: 调用类成员函数 `getDat` 取出 `dat` 数组各元素值并输出。将输出: `r s x y z`。

注 10: 以类型实参 `double` 和整型实参 `6` 取代类型形参 `T` 和普通形参 `i` 后, 则实例化为一个具体的类(其 `dat` 数组的类型为 `double`, 大小为 `6`), 说明一个此具体类的对象 `ClassInstF`。

注 11: 为类对象 `ClassInstF` 的 `dat` 数组赋值。

注 12: 调用类成员函数 `getDat` 取出 `dat` 数组各元素值并输出。输出值将为: `0.12 2.12 4.12 6.12 8.12 10.12`。

[输出结果]

```
r s x y z
0.12 2.12 4.12 6.12 8.12 10.12
```

10.5 为链表类模板 `list` 补充成员函数

对教材 9.3 节“ 程序实例: 链表模板的设计” 中的类模板 `list`, 补充具有如下原型的两个成员函数:

```
int count();           //返回当前链表的项数
```

```
void htot();          //把链首项移到链尾
```

并在主函数中对这两个函数进行使用, 以检验它们的正确性。

[分析提示]

(1) 在类模板 `list` 之外说明 `count` 时, 要使用如下的格式:

```
template <class T> int list<T>::count() { ... }
```

在 `count` 中数出当前链表项数的过程实际上是一个遍历链表的过程: 若 `head` 等于 `NULL` 时, 意味着链表为空, 返回 `n=0`; 否则, 通过“ `p=head;`” 使 `p` 指向链表首项, 而后再通过循环使用“ `p=p->next;`” 来对链表逐项下推检验处理, 进而可完成从头到尾“ 数” 出链表项数的任务。

(2) 在类模板 `list` 之外说明 `htot` 时, 要使用如下的格式:


```
template <class T> void list<T>::htot() { ... }
```

当链表为空时无须移动；非空时，将链表首项链接到链表的尾项之后，并且要及时调整新链首指针 head 以及新链尾指针 tail。

[解答程序]

```
#include <iostream.h>
#include <process.h>

template <class T> class list          //类模板 list，使用类型形参 T
{
    struct node                        //结构体 node 类型用来定义链表表项(的具体数据项目)
    {
        T data;                        //每一表项的 data 数据域的类型由类型形参 T 所指定
        node * next;                  //通过指针 next，将多个表项链接成一个链表
    } *head, *tail;

    /*
        两个私有数据成员 head 与 tail，它们均为指向链表项(结构体)的指针。其中的 head
        总指向链表的首项，而 tail 总指向链表的尾项。每当准备往链表中加入一个表项(及其表项
        data 数据)时，程序中首先使用“new node”来动态生成一个新的表项空间，并填入该表
        项的 data 数据，而后通过指针的改变与关联，将该表项加入到以 head 为首以 tail 为尾的当
        前链表结构中(以形成一个更新后的链表)。
    */

public:
    list();                            //构造函数，将 head 与 tail 均置为 NULL，意味着创建一个空链表
    void Insert (T * item);             //成员函数 Insert，参数为指向 T 类型数据的指针
                                        //动态生成一块链表项空间，并将 item 所指向的 T 类型的数据放入该链表表项的
                                        //data 域，而后将新生成的该链表项插入到原链的链首(链表的栈式用法)
    void Append (T * item);             //成员函数 Append，参数为指向 T 类型数据的指针
                                        //动态生成一块链表项空间，并将 item 所指向的 T 类型的数据放入该链表表项的
                                        //data 域，而后将新生成的该链表项附加到原链的链尾(链表的队列式用法)
    T get ();                           //成员函数 get，将返回 T 类型的结果数据
    int count();                        //新补充成员函数，返回当前链表的项数
    void htot();                        //新补充成员函数，把链首项移到链尾
};

                                        //类模板 list 定义结束
template <class T> list<T>::list ()
{
    //构造函数，将 head 与 tail 均置为 NULL，意味着创建一个空链表
```

```

        head=tail=NULL;
    };

template <class T> void list<T>::Insert (T * item)
{
    /*
        成员函数 Insert，参数为指向 T 类型数据的指针。动态生成一块链表表项空间，并将
        item 所指向的 T 类型的数据放入该表项的 data 域，而后将新生成的该表项插入到原链的链
        首(链表的栈式用法)。
    */

    node * pn;
    pn=new node;           //动态生成一块链表表项空间
    pn->next=head;         //将新生成的该链表表项插入到原链的链首
    pn->data=*item;         //将 item 所指向的 T 类型的数据放入该表项的 data 域
    head=pn;               //将新生成的表项作为新的链表项
    if (tail==NULL)        //若处理链表第一项，使 tail 也指向它
        tail=pn;
};                          //成员函数 Insert 结束

template <class T> void list<T>::Append (T * item)
{
    /*
        成员函数 Append，参数为指向 T 类型数据的指针。动态生成一块链表表项空间，并将
        item 所指向的 T 类型的数据放入该链表项的 data 域，而后将新生成的该链表项附加到原链
        的链尾(链表的队列式用法)。
    */

    node * pn;
    pn=new node;           //动态生成一块链表表项空间
    pn->next=NULL;         //该项将作为链表末项
    pn->data=*item;         //将 item 所指向的 T 类型的数据放入该链表项的 data 域
    if (tail==NULL)        //若处理链表第一项，使 head 与 tail 都指向该项
        head=tail=pn;
    else
    {
        //处理链表非首项时，将新生成表项链接到末项之后
        tail->next=pn;
        tail=pn;
    }
};                          //成员函数 Append 结束

```

```

template <class T> T list<T>::get ( )//成员函数 get, 将返回 T 类型的结果数据
{
    //取出链表首项的数据(data 域值), 并将该首项从链表中删去
    if (head==NULL)          //链表为空, 退出程序
        exit (0);
    T temp=head->data;        //temp 即为要返回的链表首项数据(data 域值)
    node * pn=head;
    if (head->next==NULL)
        head=tail=NULL;      //若链表中仅此一项, 删除后链表应变为空链
    else
        head=head->next;      //链表项多于 1 时, 下推首指针 head
    delete (pn);              //删除链表首项
    return temp;               //返回链表首项数据(data 域值)
};                             //成员函数 get 结束

```

```

template <class T> int list<T>::count( ) //成员函数 count, 返回当前链表的项数
{
    node *p;
    int n=0;                   //统计链表项数的 n 初值为 0
    if(head==NULL)
        return (n);           //空链时返回 n=0
    p=head;    n++;
    while(p!=tail)             //从头到尾“数”出链表的项数
    {
        p=p->next; n++;
    }
    return (n);                //返回“数”出的项数 n
};                             //成员函数 count 结束

```

```

template <class T> void list<T>::htot( ) //成员函数 htot, 把链首项移到链尾
{
    node *p;
    if(head==NULL||head==tail) //空链时无须移动
        return;
    p=head;
    head=head->next;            //调整新链首指针 head
    p->next=NULL;               //原链首项将充当链尾
    tail->next=p;               //将原链首项接到链接到现链尾项之后
    tail=p;                     //调整新链尾指针 tail
}

```

```

}                                     //成员函数 htot 结束

class person                          //自定义 person 类(类型), 仅含有三个公有的数据成员
{
public:
    char name[20];                    //注, 若说明为“ char* name;”, 则主函数处
                                     //要通过 new 为每一对象的 name 域申请动态空间
    int age;
    float hight;
};

void main( )                          //创建类模板 list 的实例对象并对它们进行使用
{
    list<int> link1;
        //以类型实参 int 取代类型形参 T 后, 则成为一个具体的类
        //说明一个此具体类的对象 link1(通过构造函数, 被初始化为空链)
    list<person> link2;
        //以类型实参 person 取代类型形参 T 后, 则成为一个具体的类
        //说明一个此具体类的对象 link2(通过构造函数, 被初始化为空链)
    person ps;
    cout<<" --- Input 5 person's information ---"<<endl;
    for (int i=0;i<5;i++)             //输入 5 个人的有关信息并进行处理
    {
        cout<<"input "<<i<<" inf(name,age,hight):";          //提示进行输入
        cin>>ps.name>>ps.age>>ps.hight; //输入一个人的 name, age 与 hight
        link2.Insert(&ps);              //实参为指向 person 类型的指针,
        //将 ps 对象(当前人的信息)插入到 link2 链表的链首
        link1.Append(&i);                //实参为指向 int 类型的指针,
        //将当前对象的序号附加到 link1 链表的链尾
    }

    /*
        上述循环结束后, link1 和 link2 链表的逻辑结构及内容概示如下:
    link1 链表: head -> 0 -> 1 -> 2 -> 3 -> 4
    link2 链表: head -> 4 号人员信息 -> 3 号人员信息 -> 2 号人员信息 -> 1 号人员信息
               -> 0 号人员信息
    */

    cout<<"link1.count="<<link1.count( )<<endl; //输出链表 link1 当前的项数

```

```

cout<<"link2.count="<<link2.count()<<endl; //输出链表 link2 当前的项数
link1.htot( ); //把 link1 链表首项移到链表尾
link2.htot( ); //把 link2 的链表首项移到链表尾
cout<<" ---- The result ----"<<endl;
for (i=0;i<5;i++)
{
    ps=link2.get( ); //取出 link2 链表首项的人员信息
    link2.Append(&ps);
    //将刚从 link2 首取来的人员信息，再一次附加到 link2 链表的链尾
    cout<<ps.name<<" "<<link1.get( )<<endl;
    //输出 link2 链表人员信息的 name，以及 link1 链表表项中的对象的顺序号
}
} //main 函数结束

```

[输出结果]

```

--- Input 5 person's information ---
input 0 inf(name,age,hight):zhang 20 1.68
input 1 inf(name,age,hight):wang 22 1.76
input 2 inf(name,age,hight):zhou 21 1.72
input 3 inf(name,age,hight):qian 20 1.81
input 4 inf(name,age,hight):chen 21 1.78
link1.count=5
link2.count=5
---- The result ----
qian 1
zhou 2
wang 3
zhang 4
chen 0

```

10.6 实现队列类模板 queue

参照教材 9.1 节“一个队列模板”的定义方法，将范例 7.8 中对类 queue 的定义改造成一个类模板 queue(当然仍具有存取数据时首先通过对下标进行模 size 的运算，从而可实现对存放队列数据的动态数组 data 空间进行循环使用的目的)。

[分析提示]

(1) 使用“template <class keytype> class queue { ...}”的方式定义出所需的队列类模

板, 通过其中的具有 `keytype` 可变类型的数据成员 `data` (一个动态数组) 来存放队列所拥有的数据, 并设立 `front` 与 `rear` 数据成员来记录队列之当前数据的首尾下标位置。每次总从 `data[front]` 处也即队列首部取用数据, 而总往队列尾部也即 `data[rear]` 位置处添加数据。

(2) 为了对动态数组 `data` 进行循环使用, 在成员函数 `add` 中, 每当往队列尾部加入元素 `a` 时, 要首先进行“`rear=(rear+1)%size;`”的运算, 而后再使用“`data[rear]=a;`”。另外在成员函数 `Delete` 中, 也要首先进行“`front=(front+1)%size;`”的运算, 而后再通过“`return data[front];`”返回首部元素。

[解答程序]

```
#include <iostream.h>
#include <process.h>

template <class keytype> class queue    //类模板 queue, 使用类型形参 keytype
{
    int size; //队列的大小(数据项数), 即是说每一个队列类对象的大小可以不同
    keytype * data;
        //动态数组 data, 为 keytype 可变类型, 在构造函数中, 将通过 new
        //来生成该动态数组, 将用它来存放队列数据。data 的大小由私有
        //成员变量 size 之值来确定, 而且要循环使用该数组空间。

    int front, rear, num;
        //front, rear 为队列数据首尾下标位置;
        //num 记录当前队列中具有的数据个数

public:
    queue (int maxsize);
        //构造函数, 负责生成一个 keytype 类型的、maxsize 大小的动态数组
        //(将用于存放队列数据), 并对 front, rear, num 进行初始化
        //(意味着创建空队列)

    ~queue(void);           //析构函数, 释放动态数组 data 占用的空间
    bool empty(void);       //成员函数 empty, 判断队列是否为空
    bool full(void);        //成员函数 full, 判断队列是否已满
    void add (keytype a);
        //成员函数 Add, 用于往队列尾部加入 keytype 可变类型的元素 a
    keytype Delete (void);  //用于删取队列首部的那一元素(为 keytype 可变类型的)
        //也即返回队列首部的那一元素, 并下推队列首位置 front
};

        //类模板 Queue 的各成员函数在类外的实现
template <class keytype> queue<keytype>::queue(int maxsize)
```

```

{
    //构造函数，负责生成一个 keytype 类型的、maxsize 大小的动态数组
    //(将用于存放队列数据)，并对 front, rear, num 进行初始化(意味着创建空队列)
    size=maxsize;
    data=new keytype[size];
    front=-1; rear=-1; num=0;           //意味着队列为空(其元素个数 num=0)
    cout<<"queue initialized."<<endl;
}

template <class keytype> queue<keytype>::~~queue(void)
{
    //析构函数，释放动态数组 data 占用的空间
    delete [ ]data;
    cout<<"queue destroyed."<<endl;
}

template <class keytype> bool queue<keytype>::empty(void) ]
{
    //成员函数 empty，判断队列是否为空
    return((num==0)?true:false);
}

template <class keytype> bool queue<keytype>::full(void)
{
    //成员函数 full，判断队列是否已满
    return ((num==size)?true:false);
}

template <class keytype> void queue<keytype>::add (keytype a)
{
    //成员函数 Add，用于往队列尾部加入 keytype 可变类型的元素 a
    if (full( ))           //若队列已满
        cout<<"\nThe queue is full!";    //显示信息后，不再加入数据
    else
    {
        rear=(rear+1)%size; //为能循环使用数组空间而进行模 size 的运算
        data[rear]=a;       //往队列尾部加入元素 a
        num++;              //记录当前队列中所具有数据个数的 num 加 1
    }
}

```

```

}

template <class keytype>  keytype queue<keytype>::Delete (void)
{
    //用于删取队列首部的那一元素(为 keytype 可变类型的)
    //也即返回队列首部的那一元素，并下推队列首位置 front
    if ( empty() )
    {
        cout<<"\nThe queue is empty!";
        exit(0);
    }
    front=(front+1)%size;    //为能循环使用数组空间而进行模 size 的运算
    num--;
    return data[front];
}

void main( )                //主函数对 queue 类模板进行应用，以验证其正确性
{
    int maxsize=6;
    queue<double> s1(maxsize), s2(maxsize); //double 类型的队列 s1 与 s2(大小均为 6)
    int i=1;
    do
    {
        s1.add(2*i);  i++;
    } while(!s1.full( ));
    //往队列 s1 中加入 maxsize=6 个数据，此时 s1 队列已满，为：2，4，6，8，10，12
    for (i=1; i<=4; i++)
        cout<<s1.Delete( )<<" ";
    //取出并输出 s1 的前四个数据——2，4，6，8(s1 队列中仍剩两个数据：10，12)
    for (i=1; i<=maxsize; i++)
        s1.add(2.5*i);
    //又往 s1 中加入 maxsize=六个数据(实际加入四个数据后队列满)。由于循环
    //使用数组空间，使得往前面加入新数据 2.5，5，7.5，10，成为可能
    cout<<endl;
    for (i=1; i<=3; i++)
        cout<< s1.Delete( )<<" ";    //输出 10, 12, 2.5
    cout<<s1.Delete( )<<endl;        //输出 5
    while(!s1.empty( ))
        s2.add(s1.Delete( ));        //往队列 s2 中加入两个数据，为 7.5，10
}

```



```

do
    cout << s2.Delete( )<<" ";    //输出队列 s2 中全部也即两个数据，为 7.5，10
while (! (s2.empty( )));
cout <<endl;
queue<char> s3(26);                //char 类型的队列 s3(大小为 26)
i=1;
do
{
    s3.add(64+i);    i++;
} while(!s3.full( ));              //将队列 s3 加满数据(共 26 个字符)
                                    //此时 s1 队列为：“ A”，“ B”，“ C”，“ D”，...，“ Z”

for (i=1; i<=20; i++)
    cout<<s3.Delete( )<<" ";      //取出并输出 s3 的前 20 个数据(字符)
cout<<endl;
}

```

[输出结果]

```

queue initialized.
queue initialized.
2  4  6  8
The queue is full!
The queue is full!
10 12 2.5 5
7.5 10
queue initialized.
A B C D E F G H I J K L M N O P Q R S T
queue destroyed.
queue destroyed.
queue destroyed.

```

10.7 实现数组类模板 array

编写具有如下形式的数组类模板 `array`，其中要重载下标“`[]`”运算符以及赋值“`=`”运算符。而后编制下述所示的主函数，将类模板 `array` 实例化为多种不同的具体类并进行具体的使用。

```

template <class T> class array                //含有类型参数 T 的数组类模板 array
{

```

```

T *arr;                                //array 对象数据的首元素指针 arr
int arrSize;                            //array 对象数据的个数 arrSize
public:
    array(int size);
        //一参构造函数，指定对象数据个数为 size，并通过 new 分配动态空间
    array(int size, T initValue);
        //二参构造函数，指定对象数据个数为 size，以及初始化值 initValue，
        //并通过 new 分配动态空间
    ~array();                            //析构函数，释放 array 对象所占动态空间
    T& operator[ ](int index);
        //对 array 类重载“ [ ]” 运算符，把 array 对象当做数组
        //( 数据放于 arr 所指向的动态空间中)，
        //从而可对该 array 类对象通过使用下标方式来访问 arr 的诸数据元素。
    array<T>& operator=(array<T>& otherArrObj);
        //对 array 类重载“ =” 运算符，将参数对象 otherArrObj 的值赋给
        //当前调用者对象，而且要求二对象的数据个数 arrSize 之大小完全相同
}

void main() //主函数，将类模板 array 实例化为多种不同的具体类并进行具体的使用
{
    array<int> arrInt(10), arrInt1(10);
        //说明 array<int>类对象 arrInt 及 arrInt1(它们含有的数据个数都为 10)
    for(int i=0; i<10; i++)
        arrInt[i]=i;
        //由于对 array 类重载了“ [ ]” 运算符，从而可把 array 对象当作数组使用
        //注意，自定义类中的下标变量 arrInt[i]可作为左值出现
        //这是由于重载“ [ ]” 运算符时返回的是引用
    cout<<"----- arrInt[i]: -----"<<endl;
    for(i=0; i<10; i++)                //显示出 arrInt 数组的各数据元素
        cout<<arrInt[i]<<" ";
    cout<<endl;
    arrInt1=arrInt;
        //由于对 array 类重载了“ =” 运算符，此处将调用那一自定义函数
    //arrInt1.operator=(arrInt);        //该句功能与上句等同
    cout<<"-- 'arrInt1=arrInt;' arrInt1[i]: --"<<endl;
    for(i=0; i<10; i++)                //显示出 arrInt1 数组的各数据元素
        cout<<arrInt1[i]<<" ";
    cout<<endl;
    array<int> arrInt2(8, 26);

```

```

        //说明 array<int>类对象 arrInt2(它含有的数据个数为 8, 且数据初值为 26)
arrInt2[6]=987;           //按“下标变量”方式使用
arrInt2[7]++;
cout<<"----- arrInt2[i]: -----"<<endl;
for(i=0; i<8; i++)        //显示出 arrInt2 数组的各数据元素
    cout<<arrInt2[i]<<" ";
cout<<endl;
array<double> arrDouble(6, 1.2);
    //说明 array<double>类对象 arrDouble
    //(它含有的数据个数为 6, 且数据初值为 1.2)
arrDouble[4]=567.89;      //按下标变量方式使用
arrDouble[5]*=3;
cout<<"----- arrDouble[i]: -----"<<endl;
for(i=0; i<6; i++)        //显示出 arrDouble 数组的各数据元素
    cout<<arrDouble[i]<<" ";
cout<<endl;
array<char> arrChar(10, 'A');
//说明 array<char>类对象 arrChar(它含有的数据个数为 10, 且数据初值为“ A” )
arrChar[8]='+';           //按下标变量方式使用
arrChar[9]+=3;
cout<<"----- arrChar[i]: -----"<<endl;
for(i=0; i<10; i++)       //显示出 arrChar 数组的各数据元素
    cout<<arrChar[i]<<" ";
cout<<endl;
arrChar[10]='T';          //下标范围超界(注意 arrChar 对象的下标范围为 0~9)
                           //将在重载函数中提示出“ beyond the limits!” 信息串
    }

```

[输出程序]

程序执行后的显示结果如下:

```

----- arrInt[i]: -----
0  1  2  3  4  5  6  7  8  9
-- 'arrInt1=arrInt;' arrInt1[i]: --
0  1  2  3  4  5  6  7  8  9
----- arrInt2[i]: -----
26 26 26 26 26 26 987 27
----- arrDouble[i]: -----
1.2 1.2 1.2 1.2 567.89 3.6
----- arrChar[i]: -----

```

A A A A A A A A + D

beyond the limits!

[分析提示]

本范例通过“`template <class T> class array {...}`”所定义的数组类模板 `array` 仅仅是一个示例性的数组类型，其中的类型参数 `T` 指明了数组元素的类型，而类成员函数中只设计并提供被重载的下标“`[]`”运算符以及赋值“`=`”运算符。通过下标运算符对 `array` 类的数组元素进行操作，而通过赋值运算符来实现对两个 `array` 类的数组（整体）进行赋值也即拷贝的功能。

[解答程序]

```
#include<iostream.h>
#include<process.h>

template <class T> class array          //含有类型参数 T 的数组类模板 array
{
    ...                                //与题目处相同，略
};

template <class T> array<T>::array(int size)
{
    //一参构造函数，指定对象数据个数为 size，并通过 new 语句分配动态空间
    arrSize=size;
    arr=new T[arrSize];
}

template <class T> array<T>::array(int size, T initValue)
{
    //二参构造函数，指定对象数据个数为 size，以及初始化值 initValue，
    //并通过 new 分配动态空间
    arrSize=size;    arr=new T[arrSize];
    for(int i=0; i<arrSize; i++)        //将对象数据的初值置为 initValue
        *(arr+i)=initValue;
}

template <class T> array<T>::~~array() //析构函数，释放 array 对象所占动态空间
{
    delete [] arr;
}
```

```

template <class T> T& array<T>::operator [ ](int index)
{
    //获取 array 对象第 index 下标处的元素值并返回
    //对 array 类重载 “ [ ] ” 运算符, 把 array 对象当作数组
    //(数据放于 arr 所指向的动态空间中),
    //从而可对该 array 类对象通过使用下标方式来访问 arr 的诸数据元素。
    if(index<0||index>arrSize-1)          //检验下标范围不得超界(也规定下标从 0 开始)
    {
        cout<<"beyond the limits!"<<endl; //若下标范围超界则显示信息后退出
        exit(0);
    }
    return *(arr+index);                  //返回第 index 下标处的元素值
}

```

```

template <class T> array<T>& array<T>::operator=(array<T>& otherArrObj)
{
    //对 array 类重载“ = ” 运算符, 将参数对象 otherArrObj 的值赋给当前调用者对象
    //而且要求二对象的数据个数 arrSize 之大小完全相同
    if (arrSize!=otherArrObj.arrSize)
    {
        /*
        若被赋值的调用者对象的数据个数与参数对象的数据个数不不同时, 本处理方案认为
        是一种错误(实际上也可允许数据个数不相同, 那就要设计另外的处理方案——如, 可以
        为作为左值的调用者对象重新分配与参数对象的数据个数相同大小的存储空间等)。
        */
        cout<<"arrSize' not equal!"<<endl; //若下标范围超界则显示信息后退出
        exit(0);
    }
    for(int i=0; i<arrSize; i++)
    {
        //将参数对象 otherArrObj 的个元素值赋给当前调用者对象
        arr[i]=otherArrObj.arr[i];
    }
    return *this;                          //返回当前调用者对象
}

```

```

void main() { //主函数, 将类模板 array 实例化为多种不同的
              //具体类并进行具体的使用

```

```

...          //与题目处相同，略!
}

```

[输出结果]

与题目处相同，略!

10.8 使用了栈类模板的计算器类(类型) compu

自定义一个 `compu` 类(类型)，即计算器类型。它具有如下一些功能(注意，本范例与范例 7.19 处理实现的是同一个问题)：

(1) 从键盘输入欲处理的表达式 `str` (为一个字符串)，或通过一参构造函数设置该表达式。

(2) 表达式中只可含有加、减、乘、除、等运算符以及实型数据的运算对象，表达式不限长度，遇等号结束表达式并计算出结果显示出来。

(3) 运算符具有优先级，乘、除优先于加、减。

(4) 具有一定的排错功能，当用户输入错误的表达式时提示用户。

例如，程序执行后的输入输出对应关系举例如下：

输入：1-2*3=	输出：-5
输入：1+2*3-4=	输出：3
输入：1+2*3-25/5=	输出：2
输入：1-2*3-4*2/3-1=	输出：-8.66667
输入：55-2.5*3/2-22.75=	输出：28.5
输入：1.1+2.2*2-3+4*2.1=	输出：10.9
输入：100/2/2/5+5*2*2-20=	输出：5

注意，与 7.19 的范例相比，此范例要求将存放运算对象的 `Num_Stack` 类(类型)和存放运算符的 `Op_Stack` 类(类型)合并成为一个称为 `Num_Stack` 的类模板，即栈类模板，并利用这种栈类模板，来实例化出所需的运算对象栈以及运算符栈。

下面给出实现方式之一的核心程序代码。

```

const int maxsize=100;          //运算对象栈及运算符栈的大小
template <class T> class Num_Stack //自定义的 Num_Stack 类模板，即栈类模板
{
    T data [maxsize];           //data 中存放栈的实际数据(T 类型的数据)
    int top;                     //top 为栈顶位置
public:                          //五个公有成员函数的原型
    Num_Stack (void);            //构造函数
    ~Num_Stack(void);           //析构函数

```

```

    bool empty(void);           //判断栈是否为空
    void push (T a);           //将 T 类型的数据 a 压入栈顶
    T pop (void);              //将 T 类型的栈顶数据弹出并返回
};

class compu                    //自定义的 compu 类(类型), 即计算器类型
{
    Num_Stack<float> numStack,numStack2;
    //通过 Num_Stack 类模板来实例化出所需的运算对象栈 numStack 与 numStack2
    //存放运算对象的第二个栈 numStack2 是在遇等号后进行处理时临时使用的
    Num_Stack<char> opStack,opStack2;
    //通过 Num_Stack 类模板来实例化出所需的运算符栈 opStack 与 opStack2
    //存放运算符的第二个栈 opStack2 是在遇等号后进行处理时临时使用的
    char str[100];             //存放欲计算的表达式(为一个符号串)
    int ind;                   //表达式已被处理到的当前位置(下标值)
public:
    compu( );                 //无参构造函数
    compu(char * s);          //一参构造函数(带来欲计算的表达式 s)
    void setStr(char * s);     //设置欲计算的表达式 s
    void printExp( );          //显示出当前欲计算的表达式
    void myInFromStr(char* str, float& x, char& op);
                                //从欲处理表达式 str 的当前位置 ind 开始,
                                //“ 拼装 ” 出一个运算对象 x 以及一个运算符 op
    void myDo( );
                                //通过调用 myInFromStr 从欲处理的表达式 str 处
                                //获取运算对象与运算符, 并利用四个栈对象,
                                //对欲计算的表达式 str 进行从头到尾的处理(直至输出运算结果而结束)
};

void main( )                  //主函数, 说明 compu 对象并通过调用 myDo 函数来计算各种表达式
{
    compu cmp1("1-2*3=");
    cmp1.printExp( );  cmp1.myDo( );    //输出结果: 1-2*3=-5
    compu cmp2("1+2*3-4=");
    cmp2.printExp( );  cmp2.myDo( );    //输出结果: 1+2*3-4=3
    compu cmp3("1+2*3-25/5=");
    cmp3.printExp( );  cmp3.myDo( );    //输出结果: 1+2*3-25/5=2
    compu cmp4("1-2*3-4*2/3-1=");
    cmp4.printExp( );  cmp4.myDo( );    //输出结果: 1-2*3-4*2/3-1=-8.66667
}

```

```

compu cmp5("55-2.5*3/2-22.75=");
cmp5.printExp( ); cmp5.myDo( );    //输出结果: 55-2.5*3/2-22.75=28.5
compu cmp6("1.1+2.2*2-3+4*2.1=");
cmp6.printExp( ); cmp6.myDo( );    //输出结果: 1.1+2.2*2-3+4*2.1=10.9
compu cmp7;
cmp7.printExp( ); cmp7.myDo( );    //输出结果: 0=0
compu cmp8; char str0[100];
cout<<"Input EXPRESSION (ending with '='), if only 'ENTER' then ending
program."<<endl;
cin.getline(str0, 100);
//从键盘输入欲计算的任意表达式(为一个符号串)放入 str0
while(strlen(str0)>0)                //当输入串非空时继续处理(入空串时结束)
{
    cmp8.setStr(str0);                //将 str0 设置为欲计算的表达式
    cmp8.myDo( );                    //计算表达式并输出结果
    cout<<"Input EXPRESSION (ending with '='), if only 'ENTER' then ending
program."<<endl;
    cin.getline(str0, 100);
    //从键盘输入下一个欲计算的任意表达式(符号串)放入 str0
}
}

```

[分析提示]

本范例所自定义的 `compu` 类(类型)即“计算器”类型与范例 7.19 处理实现的是同一个问题。只是此范例要求将存放运算对象的 `Num_Stack` 类(类型)和存放运算符的 `Op_Stack` 类(类型)合并成为一个称为 `Num_Stack` 的类模板,即栈类模板,并利用这种栈类模板,来实例化出所需的运算对象栈以及运算符栈。主要处理过程也是靠 `compu` 的类成员函数 `myDo` 来完成,由它负责对欲计算的中缀表达式 `str` (在 `compu` 类对象的私有数据成员 `str` 中存放欲计算的表达式)进行从头到尾的处理(直至输出运算结果而结束)。可参看范例 7.19 的有关分析提示获取进一步的提示信息。

[解答程序]

```

#include <iostream.h>
#include <process.h>
#include <string.h>
const int maxsize=100;                //运算对象栈及运算符栈的大小
template <class T> class Num_Stack    //自定义的 Num_Stack 类模板,即栈类模板
{
    T data [maxsize];                //data 中存放栈的实际数据(T 类型的数据)

```



```

        int top;                //top 为栈顶位置
public:                        //五个公有成员函数的原型
    Num_Stack (void);          //构造函数
    ~Num_Stack(void);          //析构函数
    bool empty(void);          //判断栈是否为空
    void push (T a);           //将 T 类型的数据 a 压入栈顶
    T pop (void);              //将 T 类型的栈顶数据弹出并返回
};

template <class T> Num_Stack<T>::Num_Stack(void)    //构造函数
{
    top=0;                //将当前调用者对象(变量)的私有数据 top 置为 0
}

template <class T> Num_Stack<T>::~~Num_Stack(void)    //析构函数
{
}

template <class T> bool Num_Stack<T>::empty(void)
{
    return top==0?true:false; //当当前调用者对象(变量)的 top=0 时，栈为空
}

template <class T> void Num_Stack<T>::push (T a) //将 T 类型的数据 a 压入栈顶
{
    if (top==maxsize)        //栈满时，不再压入数据
    {
        cout << "Num_Stack is full!"<< endl;
        return;
    }
    data[top]=a;              //栈未滿时，将 a 放入存放栈实际数据的数据数组中
    top ++;                  //下推栈顶位置
}

template <class T> T Num_Stack<T>::pop (void)        //将 T 类型的栈顶数据弹出并返回
{
    if (top==0)                //若栈为空，返回 0
    {
        cout << "Num_Stack is underflow!"<< endl;
    }
}

```

```

        return 0;
    }
    top--;
    return data[top];           //栈非空时，返回栈顶数据
}

float opFunc(float x, char op, float y) //将 x 与 y 进行指定的“ op ”运算并返回运算结果
{
    float res;
    switch (op)                 //根据运算符 op 的值来进行多分支处理
    {
        case '+':               //当 op 为“ + ”符号时的处理分支
            res=x+y;            break;
        case '-':               //当 op 为“ - ”符号时的处理分支
            res=x-y;            break;
        case '*':               //当 op 为“ * ”符号时的处理分支
            res=x*y;            break;
        case '/':               //当 op 为“ / ”符号时的处理分支
            if(y==0)             //分母为 0 时的处理
            {
                cout<<endl<<"overflow!"; //错误提示!
                //表示运算溢出
                exit(0);
            }
            res=x/y;            break;
        default:                 //当 op 不为上述任一符号时的处理
            cout<<endl<<"Wrong_001!"; //错误提示!
            //表示输入的运算符有错
            exit(0);
    }                            //switch 语句结束
    return res;
}

class compu                     //自定义的 compu 类(类型)，即计算器类型
{
    ...                           //与题目处相同，略!
};

compu::compu()                  //无参构造函数

```

```

{
    strcpy(str, "0=");
    ind=0;                                //表达式已被处理到的当前位置为 0(下标值)
}

compu::compu(char * s)                    //一参构造函数(带来欲计算的表达式 s)
{
    strcpy(str, s);
    ind=0;                                //表达式已被处理到的当前位置为 0(下标值)
}

void compu::setStr(char * s)              //设置欲计算的表达式 s
{
    strcpy(str, s);
    ind=0;                                //表达式已被处理到的当前位置为 0(下标值)
}

void compu::printExp( )                   //显示出当前欲计算的表达式
{
    cout<<str;
}

void compu::myDo( )
{
    //通过调用 myInFromStr 从欲处理的表达式 str 处获取运算对象与运算符,
    //并利用四个私有数据的栈对象, 对欲计算的表达式 str 进行从头到尾的
    //处理(直至输出运算结果而结束)
    float x, y, res;
    char op1, op2;
    myInFromStr(str,x,op1);                //从欲处理表达式 str 的当前位置 ind 开始,
                                           //“ 拼装 ” 出一个运算对象 x 以及一个运算符 op1
    numStack.push(x);                      //将 x 压入运算对象栈
    opStack.push(op1);                     //将 op1 压入运算符栈
    while(!opStack.empty( ))               //当运算符栈非空时循环处理
    {
        x=numStack.pop( );                 //从栈顶取来运算对象 x
        op1=opStack.pop( );                //取来运算符 op1
        if(op1!='=')                       //若 op1 非结束符号“ = ” 时
            myInFromStr(str,y,op2);        //从 str 的当前位置 ind 开始,

```

```

//“ 拼装” 出后一运算对象 y 以及后一运算符 op2
switch (op1)           //根据前一运算符 op1 的值来进行多分支处理
{
case '+':              //当 op1 为“ +” 符号时的处理分支
    if (op2=='*'||op2=='/') //后一运算符 op2 优先级高时
    {
        //不可马上处理，将运算对象及运算符放入栈中
        numStack.push(x);    numStack.push(y);
        opStack.push(op1);    opStack.push(op2);
        break;
    }
else //后一运算符 op2 的优先级与 op1 相同时，处理前一运算
{
    res=opFunc(x,op1,y);
    //将 x 与 y 进行指定的“ op1” 运算并返回运算结果
    numStack.push(res);      //将运算结果放入栈中
    opStack.push(op2);
    break;
}
case '-':              //当 op1 为“ -” 符号时的处理分支
    if (op2=='*'||op2=='/') //后一运算符 op2 优先级高时
    {
        //不可马上处理，将运算对象及运算符放入栈中
        numStack.push(x);    numStack.push(-y);
        opStack.push('+');    opStack.push(op2);
        break;
    }
else //后一运算符 op2 的优先级与 op1 相同时，处理前一运算
{
    res=opFunc(x,op1,y);
    //将 x 与 y 进行指定的“ op1” 运算并返回运算结果
    numStack.push(res);      //将运算结果放入栈中
    opStack.push(op2);
    break;
}
case '*':              //当 op1 为“ *” 符号时的处理分支
    res=opFunc(x,op1,y);      //处理前一运算
    numStack.push(res);        //运算结果进栈
    opStack.push(op2);

```

```

        break;
    case '/': //当 op1 为“ /” 符号时的处理分支
        if(y==0) //分母为 0 时的处理
        {
            cout<<endl<<"overflow!";
            //错误提示，表示运算溢出

            exit(0);
        }
        res=opFunc(x,op1,y); //处理前一运算
        numStack.push(res); //运算结果进栈
        opStack.push(op2);
        break;
    case '=': //遇等号后，numStack 栈中的剩余部分可以进行
              //从前往后(从栈底到栈顶)的连续运算
        numStack2.push(x);
            //利用 numStack2 与 opStack2 将 numStack 与 opStack
            //栈中的剩余部分给反序压入栈中
        while(!opStack.empty( ))
        {
            numStack2.push(numStack.pop( ));
            opStack2.push(opStack.pop( ));
        }
        //连续处理 numStack2 与 opStack2 中已经反序的
        //原 numStack 与 opStack 栈中的剩余部分
        res=numStack2.pop( );
        while(!opStack2.empty( ))
        {
            y=numStack2.pop( );
            op1=opStack2.pop( );
            res=opFunc(res,op1,y);
        }
        break;
    default: //当 op 不为上述任一符号时的处理
        cout<<endl<<"Wrong_002!";
            //错误提示！表示输入的运算符有错
        exit(0);
    } //switch 语句结束
    op1=op2;
} //while(!opStack.empty( ))循环结束

```

```

        cout<<res<<endl;
    };

void compu::myInFromStr(char* str, float& x, char& op)
{
    //从欲处理表达式 str 的当前位置 ind 开始,
    //“ 拼装” 出一个运算对象 x 以及一个运算符 op
    float num=0;
    char cc=str[ind++];          //从 str 的当前位置 ind 开始取字符
    if(cc<'0' || cc>'9')          //第一个字符应为数字字符
    {
        cout<<endl<<"Wrong_0000!";
        //错误提示! 表示输入的非数字

        exit(0);
    }
    while (cc>='0' && cc<='9') //处理连续的各数字字符(小数点前)
    {
        num=num*10+(cc-'0');
        cc=str[ind++];
    }
    x=num;
    if(cc!='+' && cc!='-' && cc!='*' && cc!='/' && cc!='=' && cc!='.')
    {
        //数后符号
        cout<<endl<<"Wrong_0001!";
        //错误提示! 表示输入的数后符号有错

        exit(0);
    }
    op=cc;
    if(cc!='.') return;          //整数则返回
    float num2=0, fac=0.1;      //处理点后部分
    cc=str[ind++];
    if(cc<'0' || cc>'9')          //第一个字符应为数字字符
    {
        cout<<endl<<"Wrong_0000!";
        //错误提示! 表示点后输入的非数字

        exit(0);
    }
    while (cc>='0' && cc<='9')    //处理点后数据
    {

```

```

        num2+=fac*(cc-0);
        cc=str[ind++];
        fac/=10;
    }
    x+=num2;    op=cc;
};

void main( )    //主函数, 说明 compu 对象并通过调用 myDo 函数来计算各种表达式
{
    ...        //与题目处相同, 略!
}

```

[输出结果]

```

1-2*3=-5
1+2*3-4=3
1+2*3-25/5=2
1-2*3-4*2/3-1=-8.66667
55-2.5*3/2-22.75=28.5
1.1+2.2*2-3+4*2.1=10.9
0=0
Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.
0-55+6/2.2*3+10=
-36.8182
Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.
100/2/2/5+5*2*2-20=
5
Input EXPRESSION (ending with '='), if only 'ENTER' then ending program.

Press any key to continue

```

10.9 类模板作基类派生出类模板

读下面的程序, 而后给出执行后的屏幕显示结果。该程序使用类模板作为基类派生出新的类模板。注意, 基类中用到类型参数 T2, 而派生类中用到另一个类型参数 T1。

```

#include <iostream.h>
template <class T2> class baseCla                                //注 1
{

```

```
int d; char ch;
T2 x2;
public:
    baseCla( )
    {
        d=25;ch='a';    x2=100;
    }
    void putx2(T2 tt)
    {
        x2=tt;
    }
    T2 getx2( )
    {
        return x2;
    }
    void print( )
    {
        cout<<"baseCla: d="<<d<<"  ch="<<ch<<"  x2="<<x2<<endl;
    }
} //类模板 baseCla 结束
```

```
template <class T1, class T2> class derivedCla:public baseCla<T2> //注 2
{
    T1 x1;
public:
    derivedCla( )
    {
        x1=0;
    }
    void putx1(T1 tt)
    {
        x1=tt;
    }
    T1 getx1( )
    {
        return x1;
    }
    void print( )
    {
```



```

        baseCla<T2>::print( );
        cout<<"derivedCla: x1="<<x1<<endl;
    }
}          //类模板 derivedCla 结束

void main( )‘                                     //注 3
{
    derivedCla <double, char> der1;                //注 4
    der1.print( );                                //注 5
    cout<<"-----"<<endl;
    der1.putx1(321.5);                             //注 6
    der1.putx2('H');                               //注 7
    cout<<"der1.getx1( )="<<der1.getx1( )<<endl;    //注 8
    cout<<"der1.getx2( )="<<der1.getx2( )<<endl;    //注 9
    der1.print( );                                 //注 10
}

```

[分析提示]

可以使用多种不同的方式来派生出类模板。如可使用一般类作基类而派生出类模板，也可使用类模板作基类而派生出新的类模板。本范例的 `derivedCla` 是被派生出的类模板，而其基类 `baseCla` 也为一个类模板。`derivedCla` 类模板中具有两个类型参数 `T1` 与 `T2`，其中的类型参数 `T2` 将被传递给基类 `baseCla`，而在派生类 `derivedCla` 中将只使用另一个类型参数 `T1`（而不使用 `T2`）。

[解答程序]

具体程序见题目处。

注 1: `baseCla` 为类模板（其中使用了类型参数 `T2`），它将作为类模板 `derivedCla` 的基类。该类模板中，含有一个私有数据成员 `x2`，为 `T2` 类型的（其中 `T2` 为类型参数）。公有成员函数中，除构造函数外，还有 `putx2` 以及 `getx2`，它们中都用到类型参数 `T2`。另外还设有基类的公有成员函数 `print`，负责显示出基类的三个私有数据。

注 2: `derivedCla` 为类模板，其基类 `baseCla` 也为类模板。注意，`derivedCla` 类模板中具有两个类型参数，其中的类型参数 `T2` 将被传递给基类 `baseCla`，而本派生类中将使用另一个类型参数 `T1`。该类模板中，含有一个私有数据成员 `x1`，为 `T1` 类型的（其中 `T1` 为类型参数）。公有成员函数中，除构造函数外，还有 `putx1` 以及 `getx1`，它们中都用到类型参数 `T1`。另外还设有派生类的公有成员函数 `print`，它首先调用基类 `baseCla<T2>` 的成员函数 `print`，负责显示出基类的三个私有数据，而后再输出本派生类的特有成员 `x1`。

注 3: `main` 函数中，将创建类模板 `derivedCla` 的实例对象并对它进行使用。

注 4: 以类型实参 `double` 及 `char` 取代类型形参 `T1` 与 `T2` 后，则成为一个具体的派生类，此处说明一个该具体派生类的类对象 `der1`。注意，通过构造函数，使对象 `der1` 的各

私有数据成员具有了如下初值：d=25; ch=' a' ; x1=0; x2=100; 实际上，按 char 类型看待 x2 的值 100 时则为字符“ d”。

注 5：显示出对象 der1 具有的各数据成员初值。

注 6：调用派生类的成员函数 putx1。注意对象 der1 所属的类为“ derivedCla <double, char>”，故其成员函数 putx1 的实参类型 T1 应为 double 型（此处的实参为 321.5）。

注 7：调用基类成员函数 putx2。注意对象 der1 所属的类为“ derivedCla <double, char>”，故其成员函数 putx2 的实参类型 T2 应为 char 型（派生类 derivedCla 已将类型参数 T2 传递给了基类 baseCla）。

注 8：getx1 为派生类的公有成员函数，可以通过派生类对象 der1 来直接调用。

注 9：getx2 为基类的公有成员函数，可以通过派生类对象 der1 来直接调用。

注 10：显示出对象 der1 当前各数据成员的值。

[输出结果]

```
baseCla: d=25  ch=a  x2=d
derivedCla: x1=0
-----
der1.getx1( )=321.5
der1.getx2( )=H
baseCla: d=25  ch=a  x2=H
derivedCla: x1=321.5
```

第十一章 流类库与输入/输出

[语法要点]

详细内容可参阅教材第十章。

(1) C++语言中的输入/输出操作(功能)是由它所预定义的诸 I/O 流类的一个类库所提供的。流(stream)是一个逻辑概念,它是 C++语言对所有外部设备的逻辑抽象,它代表的是某种流类类型的一个对象(即流类对象)。C++的 IO 系统将每个外部设备都转换成一个称为流的逻辑设备,由流来完成对不同设备的具体操作。C++语言预定义了一批 I/O 流类(类型)以及若干个如 cin 和 cout 这样的标准流(类对象)以方便用户的使用。

流又分为文本流(Text Stream)和二进制流(Binary Stream)。文本流是一串 ASCII 字符。可用文字处理器对文本流进行编辑。二进制流是将数据以其二进制形式存放的,系统不对数据进行任何转换。

(2) 文件(File)是一个物理概念,代表存储着信息集合的某个外部介质,它是 C++语言对具体设备的抽象。如:磁盘文件、显示器、键盘,它们都是 C++的一个具体文件。

所有流(类对象)的行为都是相同的,而不同的文件则可能具有不同的行为。如,磁盘文件可进行写也可进行读操作,显示器文件则只可进行写操作,而键盘文件只可进行读操作。

当程序与一个文件交换信息时,必须通过“打开文件”的操作将一个文件与一个流(类对象)联系起来。一旦建立了这种联系,以后对该流(类对象)的访问就是对该文件的访问,也就是对一个具体设备的访问。可通过关闭文件的操作将一个文件与流(类对象)的联系断开。

(3) 在头文件“ iostream.h”中说明(预定义)了一批基本流类(类型),最常用的几个类(类型)是: ios、istream、ostream 以及 iostream。C++预定义的流(类对象)有: cin、cout、cerr 以及 clog。程序中可直接对上述 4 个预定义流类对象进行读写(只要包含头文件“ iostream.h”),而不必先进行“打开文件”的操作,使用完后也不需要进行“关闭文件”的操作(因为这些流类对象与文件之间的联系是预定义好的,可认为系统已为每一程序都隐含地进行了对它们的打开与关闭的操作)。

(4) 在 ios 类中定义了一批公有的格式控制标志位以及一些相关的成员函数,各标志位的具体取值或为 0 或为 1,系统将通过这些标志位来对数据的输入/输出格式以及某些 I/O 特征实现控制,通常先用某些成员函数来设置标志位,然后再使用另一些成员函数来进行格式输出。ios 类中还设置了一个 long 型的格式控制标志字,该标志字的值可通过程序来“合成” -- 使用位运算符“|”将下述代表格式控制标志位属性的多个枚举常量值连接在一起: ios::skipws、ios::left、ios::right、ios::internal、ios::dec、ios::oct、ios::hex、ios::showbase、ios::showpoint、ios::uppercase、ios::showpos、ios::scientific、ios::fixed、

`ios::unitbuf`、`ios::stdio`。

常用的 `ios` 类的公有成员函数有：`flags`、`setf`、`unsetf`、`fill`、`precision` 以及 `width`。通过成员函数 `flags(IFlags)` 或 `setf(IFlags)` 都可以更改格式控制标志字，但注意，`flags` 函数采用“替换”方式（用实参 `IFlags` 值更换原有标志字）；而 `setf` 则采用“添加”方式，即是说，它并不更改标志字中不涉及到的那些标志位的当前值。成员函数 `precision` 用来设置浮点数的精度，当格式为 `ios::scientific` 或 `ios::fixed` 时，精度是指小数点后的位数，否则指有效数字。`width` 设置当前被显示数据的域宽，但此设置只对随后的一个数据有效。

另外注意，`ios` 类作为诸多 I/O 流类的基类，其公有成员函数当然可被各派生类的对象所直接调用。

(5) 系统预定义了一批常用的输入/输出格式控制符，它们可直接用于提取和插入算符（“>>”和“<<”）之后，而不必像格式控制成员函数那样必须被单独调用。`iostream.h` 中含有的无参格式控制符有：`endl`、`ends`、`flush`、`ws`、`dec`、`hex` 以及 `oct`。`iomanip.h` 中含有的有参格式控制符有：`setbase`、`resetiosflags`、`setiosflags`、`setfill`、`setprecision` 以及 `setw`。

(6) 在头文件“`fstream.h`”中说明（预定义）了一批文件流类（类型），它们是专用于磁盘文件 IO 的流类（类型）。最常用的几个类（类型）是：`ifstream`、`ofstream` 以及 `fstream`。但 C++ 中没有预定义的文件流类对象，即是说，程序中用到的所有文件流类对象都要进行自定义（规定对象名及打开方式，并将该流类对象与一个具体的磁盘文件联系起来等）。另外，对自定义磁盘文件进行读写时，必须遵照如下的一般处理步骤：打开文件 => 读写操作 => 关闭文件。

“打开文件”通常在创建流类对象的同时进行。而“关闭文件”通常通过使用“<流类对象名>.close();”来完成。对文件的读写操作通常使用预定义的类成员函数来进行，但也可使用继承而来的插入和提取运算符“>>”和“<<”来实现。但要注意，使用预定义的算符“<<”来进行写操作时，为了能正确读出，数据间要人为添加分割符（比如空格），这与用算符“>>”来进行读操作时遇空格或换行均结束相呼应。

(7) 与创建文件流类对象相关的三个构造函数的使用格式如下（其中的 `szName` 为必须提供的文件名，`nMode` 为打开文件的方式，`NProt` 用来指定所打开文件的保护方式，对后两个参数通常使用系统已提供的那些默认值）。

① `ifstream(const char* szName, int nMode=ios::in, int nProt=filebuf::openprot);`

其中的 `nMode` 可用如下常量的组合来表示：`ios::in`、`ios::nocreate`、`ios::binary`。

② `ofstream(const char* szName, int nMode=ios::out, int nProt=filebuf::openprot);`

其中的 `NMode` 可用如下常量的组合来表示：`ios::in`、`ios::ou`、`ios::ate`、`ios::app`、`ios::trunc`、`ios::nocreate`、`ios::noreplace`、`ios::binary`。

③ `fstream(const char* szName, int nMode, int nProt=filebuf::openprot);`

其中的 `nMode` 与 `ofstream` 构造函数处完全相同，但此处的 `nMode` 必须提供显式值。

注意，由程序员决定将数据存储为 `text` 文件或者 `binary` 文件两种形式之一。通过将打开文件方式的 `nMode` 设置为“`ios::binary`”，可使打开的文件为 `binary` 文件形式；而在缺省情况下，打开的文件为 `text` 文件形式。通常将纯文本信息（如字符串）以 `text` 文件形式存储，而将数值信息以 `binary` 文件形式存储。

以 `text` 形式存储，优点是具有较高的兼容性，可利用任何一个文字处理程序进行阅

读或编辑修改等。缺点是存储一批纯数值信息时，要在数据之间人为地添加分割符（否则将导致数据无法正确读出）；另外 text 文件通常比 binary 文件所占的磁盘空间大，且不利于对数据实行随机访问（因为每一数据所占磁盘空间的大小通常不相同）。

以 binary 形式存储，优点是便于对数据实行随机访问（每一同类型数据所占磁盘空间的大小均相同，不必在数据之间人为地添加分割符），而且所占的磁盘空间通常比 text 文件要小。缺点是兼容性低，不可利用文字处理程序进行阅读或编辑修改等。

(8) 使用类成员函数 get 与 put 可以对自定义磁盘文件进行读和写操作。ostream::put 与 istream::get 的最常用格式为：“ ostream& put(char ch);” 以及“ istream& get(char& rch);”。从某一文件中读出所有符号进行处理的一般程序框架如下：

```
char ch;          ifstream fin("f1.dat");
//打开用于读的文件“ f1.dat”，并使它与 ifstream 流类对象 fin 相联系 fin.get(ch);
//使用 get 函数进行读操作（读出的字符放 ch）
while(!fin.eof()) //从头读到文件结束（当前符号非文件结束符时继续）
{
    //注，成员函数 eof()在读到文件结束时方返回 true
    处理 ch;        //如将读出的 ch 显示在屏幕上等
    fin.get(ch);    //使用 get 函数进行下一次的读操作（读出的字符放 ch）
}
fin.close();      //关闭与 ifstream 流类对象 fin 相联系的文件“ f1.dat”
```

(9) 使用 istream 类的成员函数 getline 可以从 text 文件中读入一行。

```
istream& getline( char* pch, int nCount, char delim = '\n');
```

功能：从与 istream 流类对象关联的某个文件中读入一行（至多 nCount 个字符）放在由 pch 所指向的缓冲区中，缺省行结束符为“ \n”。

从某一个 text 文件中读出所有行进行处理的一般程序的核心代码如下：

```
char line[81];    ifstream infile("getline_1.cpp");
//打开用于“读”的文件“ getline_1.cpp”（即某一源程序文件）
infile.getline(line, 80); //读出一行（至多 80 个字符）放入 line 中
while(!infile.eof())    //尚未读到文件结束时，继续循环（处理）
{
    //如将读出的 line 显示在屏幕上等
    infile.getline(line,80); //再读出一行（至多 80 个字符）放入 line 中
}
infile.close();      //关闭文件“ getline_1.cpp”
```

(10) 使用类成员函数 read 与 write 可以对文件进行读写操作。通常使用 read 与 write 对二进制文件（binary file）进行读写。istream::read 与 ostream::write 类成员函数的常用格式及功能如下：

```
ostream& write( const char* pch, int nCount );
```

功能：将 pch 缓冲区中的前 nCount 个字符写出到某个文件（ostream 流对象）中。

```
istream& read( char* pch, int nCount );
```

功能：从某个文件(`istream` 流对象) 中读入 `nCount` 个字符放在 `pch` 所指向的缓冲区内(若读至文件结束尚不足 `nCount` 个字符时，也将立即结束本次读取过程)。

(11) 使用类成员函数 `write` 与 `read`，并配合使用类成员函数 `seekp` 和 `seekg`，就可以对文件进行随机性(非顺序性) 的写与读操作。对文件进行随机性的读写操作所涉及到的几个类成员函数的常用格式及功能如下：

① `ostream::seekp` 常用格式：`ostream& seekp( long pos );`

功能：将输出指针的值置成 `pos`(从文件首后推 `pos` 字节处)，使以后的输出从 `pos` 位置开始。

② `istream::seekg` 常用格式：`istream& seekg( long pos );`

功能：将读入指针的值置成 `pos`(从文件首后推 `pos` 字节处)，使以后的读入从 `pos` 位置开始。

③ `istream::tellg` 使用格式：`long tellg( );`

功能：获取读入指针的当前位置值(从文件首到当前位置的字节数)。

④ `ostream::tellp` 使用格式：`long tellp( );`

功能：获取输出指针的当前位置值(从文件首到当前位置的字节数)。

(12) 从标准文件 `cin`(即键盘) 或从自定义磁盘文件 `fin` 读入字符时，常使用 `get` 函数或提取算符“`>>`”。但要注意这两种读入方式的如下使用区别：以 `get` 来读入字符时，总认为“空格”、“`Tab` 键”、“换行”这些前导字符均为一个输入字符；但若使用提取算符“`>>`”来读入字符时，在默认情况下，将甩掉前导的“空格”、“`Tab` 键”、“换行”(不认为它们是一个输入字符!)。如若需要，也可先通过使用成员函数 `unsetf(ios::skipws);`，之后再使用算符“`>>`”进行读入时，将不甩掉上述前导字符。

11.1 插入与提取运算符的两种使用方式

下述程序中使用了插入与提取运算符(也称输入/输出运算符) 的函数调用方式以及运算符调用方式。请阅读该程序后写出其执行结果(假设执行时为 `i1`、`x1` 以及 `i2`、`x2` 输入的数据均为 89 和 531.2)。

```
#include <iostream.h>

void main( )
{
    int i=123; double x=456.78;
    cout<<"i=";      cout<<i; cout<<endl;
    cout.operator<<("i="); //注 1
    cout.operator<<(i);      //注 2
    cout.operator<<(endl);    //注 3
    cout<<"x="<<x<<endl;
    ((cout.operator<<("x=")).operator<<(x)).operator<<(endl); //注 4
```

```
int i1, i2; double x1, x2;
cout<<"Input i1, x1:"<<endl;
cin>>i1>>x1;
cout<<"i1, x1="<<i1<<" "<<x1<<endl;
cout<<"Input i2, x2:"<<endl;
(cin.operator>>(i2)).operator>>(x2);           //注 5
cout<<"i2, x2="<<i2<<" "<<x2<<endl;
}
```

[分析提示]

对基本数据类型的某表达式 x ，使用“`cout<<x;`”的运算符调用方式，完全等同于“`cout.operator<<(x);`”的函数调用方式。系统对基本数据类型自动重载了“`operator<<`”。进一步，“`(cout.operator<<(x)).operator<<(y);`”的函数调用方式，也完全等同于“`cout<<x<<y;`”的运算符调用方式。

同理，“`cin>>x;`”的运算符调用方式完全等同于“`cin.operator>>(x);`”的函数调用方式，而“`(cin.operator>>(x)).operator>>(y);`”的函数调用方式也等同于“`cin>>x>>y;`”的运算符调用方式。

[解答程序]

具体程序见题目处。

注 1：函数调用方式，等同于“`cout<<"i=";`”。

注 2：等同于“`cout<<i;`”。

注 3：等同于“`cout<<endl;`”。

注 4：函数调用方式，等同于“`cout<<"x="<<x<<endl;`”。

注 5：等同于“`cin>>i2>>x2;`”。

[输出结果]

```
i=123
i=123
x=456.78
x=456.78
Input i1, x1:
89 531.2
i1, x1=89 531.2
Input i2, x2:
89 531.2
i2, x2=89 531.2
```

11.2 对自定义 fraction 类重载输入运算符“>>” 与输出运算符“<<”

将范例 9.3 处的自定义简单分数类类型 `fraction` 中的负责输入/输出的类成员函数 `input` 以及 `display` 改写成运算符重载实现形式，即是说，在 `fraction` 类中，重载输入运算符“>>”与输出运算符“<<”以取代 `input` 与 `display` 函数。

下面只给出与其相关的一部分定义，并给出一个示意性主函数以及程序执行结果样本。

```
class fraction                                //自定义的分数类型 fraction
{
    int above;                               //分子 above
    int below;                               //分母 below
public:
    fraction(int,int);                       //二参构造函数，带来分子分母
    fraction( );                             //无参构造函数，隐含分子为0，分母为1
    fraction operator *(fraction);           //两个分数相乘
    friend istream& operator >> (istream& in, fraction& fra);
                                                //重载输入运算符“>>”
                                                //从流 in 处输入 fraction 类对象数据放入引用形参对象 fra 之中
    friend ostream& operator << (ostream& out, fraction& fra);
                                                //重载输出运算符“<<”
                                                //将引用形参对象 fra 之数据输出到流 out 上
};

void main( )                                //主函数 main
{
    //说明 fraction 类对象，对定义的各函数进行调用，以验证它们的正确性
    fraction f1, f2(-2,3), f3(1,5), f4;
    cout<<"f1="<<f1;
    cout<<"f2="<<f2;
    cout<<"f3="<<f3;
    cout<<"f2*f3="<<f2*f3;
    f1=f2*f3;
    cout<<"f1=f2*f3; f1=>";
    cout<<f1;
    cin>>f4;
    cout<<"f4="<<f4;
```



```
    }
```

程序执行后，屏幕显示结果为：

```
f1=0
f2=-2/3
f3=1/5
f2*f3=-2/15
f1=f2*f3; f1=>-2/15
(above below):3 7
f4=3/7
```

[分析提示]

(1) 用户可对任一个自定义类(类型)重载插入与提取运算符“<<”与“>>”，以扩大输入/输出运算符的使用范围(否则的话，不可使用算符“<<”与“>>”直接对该自定义类的对象进行输入与输出操作)。

(2) 本范例以“`ostream& operator<<(ostream& out, fraction& fra){...}`”的形式重载输出运算符“<<”，实现将引用形参对象 `fra` 之数据输出到流 `out` 上的功能，之所以将函数返回类型设为“`ostream&`”，为的是可使用返回结果继续作左值，也即使返回结果能起到一个 `ostream` 类型的独立对象(变量)的作用，从而可使用像“`cout<<fra1<<fra2;`”这样的调用语句(虽然本范例中没按这种方式使用它们)。同理，被重载的输入运算符“>>”的返回类型被设定为“`istream&`”，以使调用语句“`cin>>fra1>>fra2;`”也成为合法的。

[解答程序]

```
#include <iostream.h>

class fraction                //自定义的分数类型 fraction
{
    ...                       //与题目处相同，略
};

fraction::fraction(int a, int b)    //二参构造函数，带来分子分母
{
    above=a;        below=b;
}

fraction::fraction()              //无参构造函数，隐含分子为 0，分母为 1
{
    above=0;        below=1;
}
```

[illegible]

```
void main()                //主函数 main
{   ...                    //与题目处相同，略
}
```

[输出结果]

与题目处相同，略

11.3 使用格式控制符将数据输出到屏幕上

下面的程序中使用了格式控制符，将数据输出到 `cout` 对象——也即屏幕上。请写出屏幕显示结果。

```
#include <iostream.h>
void main(void)
{
    cout<<"6543* *fdja"<<endl;
    cout.flags(ios::showbase);           //注 1
    cout<<hex<<17<<endl;                 //注 2
    cout<<oct<<17<<endl;                 //注 3
    cout<<dec<<17<<endl;                 //注 4
    cout.flags(ios::fixed);               //注 5
    cout.precision(3);                    //注 6
    cout<<9876.54321<<endl;              //注 7
    cout.flags(ios::scientific);          //注 8
    cout.setf(ios::uppercase);            //注 9
    cout.precision(4);                    //注 10
    cout<<2.345678E+03<<endl;           //注 11
}
```

[分析提示]

通过成员函数 `flags` 或 `setf` 都可以更改格式控制标志字(简称标志字)。标志字是由格式控制标志位来合成的，而各标志位信息的具体取值(或为 0，或为 1) 将对数据的输入/输出格式或某些 I/O 特征产生控制作用。但注意，`flags` 函数采用替换方式(用实参值更换原有标志字)；而 `setf` 则采用添加方式，即是说，它并不更改标志字中不涉及到的那些标志位的当前值。成员函数 `precision` 用来设置浮点数的精度，当格式为 `ios::scientific` 或 `ios::fixed` 时，精度是指小数点后的位数，否则指有效数字。

无参格式控制符 `hex`、`oct` 以及 `dec` 用于表示输出数据的进制格式，它们像“ `endl` ”一样可以直接出现在输出算符“ `>>` ”之后。

[解答程序]

具体程序见题目处。

注 1: `ios::showbase`——输出的数值前面带有基指示符(八进时带 0, 十六进时带 0x)。

注 2: 无参格式控制符 `hex` 表示按十六进制格式输出。

注 3: 无参格式控制符 `oct` 表示按八进制格式输出。

注 4: 无参格式控制符 `dec` 表示按十进制格式(默认数制)输出。

注 5: `ios::fixed` —— 以定点方式显示浮点数(无指数部分)。

注 6: 设置浮点数精度为 3。当格式为 `ios::fixed` 时, 精度 3 指小数点后的位数。

注 7: 输出结果为: 9876.543。

注 8: `ios::scientific` —— 以科学表示法显示浮点数(有指数部分)。

注 9: 对十六进制数值显示大写的 A 到 F, 对科学表示法显示大写的 E。

注 10: 设置浮点数精度为 4。当格式为 `ios::scientific` 时, 精度 4 指小数点后的位数。

注 11: 输出结果为: 2.3457E+003。

[输出结果]

```
6543* *fdja
0x11
021
17
9876.543
2.3457E+003
```

11.4 使用格式控制符将数据输出到 text 型磁盘文件中

下面的程序中使用了格式控制符, 将数据输出到 `fout` 对象——也即对应于“ `f0.dat` ”的 text 型磁盘文件中。而后又从 `fin` 对象——也为对应于“ `f0.dat` ”的 text 型磁盘文件中读出这些数据并显示在屏幕上。请写出屏幕显示结果。

```
#include <fstream.h>

void main(void)
{
    ofstream fout("f0.dat");                //注 1
    fout<<"6543* *fdja"<<endl;
    fout.flags(ios::showbase);              //注 2
    fout<<hex<<17<<endl;                    //注 3
    fout<<oct<<17<<endl;                     //注 4
    fout<<dec<<17<<endl;                     //注 5
    fout.flags(ios::fixed);                  //注 6
```

```

        fout.precision(3);                                //注 7
        fout<<9876.54321<<endl;                          //注 8
        fout.flags(ios::scientific);                     //注 9
        fout.setf(ios::uppercase);                       //注 10
        fout.precision(4);                                //注 11
        fout<<2.345678E+03<<endl;                       //注 12
        fout.close( );
        ifstream fin("f0.dat");                          //注 13
        char str1[20], str2[20];
        fin>>str1>>str2;                                  //注 14
        int x11, x12, x13;
        fin>>x11>>x12>>x13;                              //注 15
        double d1, d2;
        fin>>d1>>d2;                                       //注 16
        fin.close( );                                     //注 17
        cout<<"str1, str2="<<str1<<" "<<str2<<endl;
        cout.flags(ios::showbase);                       //注 18
        cout<<"x11="<<hex<<x11<<endl;
        cout<<"x12="<<oct<<x12<<endl;
        cout<<"x13="<<dec<<x13<<endl;
        cout.flags(ios::fixed);
        cout.precision(3);                                //注 19
        cout<<"d1="<<d1<<endl;
        cout.flags(ios::scientific);
        cout.setf(ios::uppercase);
        cout.precision(4);
        cout<<"d2="<<d2<<endl;
    }

```

[分析提示]

(1) 对自定义磁盘文件进行读/写操作时，遵照如下的处理步骤：打开文件 => 读/写操作 => 关闭文件。如本范例中，首先通过“`ofstream fout("f0.dat");`”打开文件(准备写)，而后通过使用算符“`>>`”往与 `fout` 流类对象关联的“f0.dat”文件中写数据，最后使用“`fout.close( );`”关闭文件。随后又通过“`ifstream fin("f0.dat");`”打开文件(准备读)，而后通过使用算符“`<<`”从与 `fin` 流类对象关联的“f0.dat”文件中读数据(并处理)，最后使用“`fout.close( );`”关闭文件。

(2) 成员函数 `flags`、`setf`、`precision` 以及格式控制符 `hex`、`oct` 及 `dec` 等的使用方法与功能与上一范例完全相同，具体可参看那一范例的分析提示。

[解答程序]

具体程序见题目处。

注 1: 打开用于写的 text 文件"f0.dat"(流对象 fout 与其对应)。

注 2: ios::showbase ——输出的数值前面带有基指示符(八进时带 0, 十六进时带 0x)。

注 3: 无参格式控制符 hex 表示按十六进制格式输出。

注 4: 无参格式控制符 oct 表示按八进制格式输出。

注 5: 无参格式控制符 dec 表示按十进制格式(默认数制)输出。

注 6: ios::fixed ——以定点方式显示浮点数(无指数部分)。

注 7: 设置浮点数精度为 3。当格式为 ios::fixed 时, 精度 3 指小数点后的位数。

注 8: 输出结果为: 9876.543。

注 9: ios::scientific ——以科学表示法显示浮点数(有指数部分)。

注 10: 对十六进制数值显示大写的 A 到 F, 对科学表示法显示大写的 E。

注 11: 设置浮点数精度为 4。当格式为 ios::scientific 时, 精度 4 指小数点后的位数。

注 12: 输出结果为: 2.3457E+003。

注 13: 打开用于读的 text 文件"f0.dat"(流对象 fin 与其对应)。

注 14: 从文件"f0.dat"(流对象 fin)中读入两个字符串。

注 15: 从文件"f0.dat"(流对象 fin)中读入三个整数。

注 16: 从文件"f0.dat"(流对象 fin)中读入两个实数。

注 17: 关闭文件, 而后把从文件"f0.dat"(流对象 fin)中读入的那些数据按格式显示到屏幕上。

注 18: 使输出的数值前面带有基指示符。

注 19: 精度 3 指小数点后的位数。

[输出结果]

```
str1, str2=6543* *fdja
x11=0x11
x12=021
x13=17
d1=9876.543
d2=2.3457E+003
```

11.5 从文件 cin 以抽取算符读入各字符的方式一

下述程序想要做的事情为: 以抽取算符“>>”从标准文件 cin 即键盘读入一批字符, 直到遇见换行(“\n”)时结束程序, 并将读入的各字符及其 ASCII 码值一并显示在屏幕上。

但程序中没使用“cin.unsetf(ios::skipws);”, 从而在使用算符“>>”进行输入时将甩掉前导的“空格”、“Tab 键”、以及“换行”(不认为它们是一个输入字符!), 所以在输

入换行(“ \n”)符号后根本无法结束本程序(要靠再按下 `ctrl+c` 来强制程序结束)。

假设程序执行后输入四个字符:“ 空格”、“ Tab 键”、“ a”、“ 换行”(此时程序不结束,再按 `ctrl+c`),请推想程序执行后的屏幕显示结果。

```
#include <iostream.h>

void main( )
{
    char ch; int d;
    cin>>ch; //注 1
    d=ch; //注 2
    cout<<"ch="<<ch<<" d="<<d<<endl; //注 3
    while (ch!='\n')
    { //注 4
        cin>>ch; //注 5
        d=ch; //注 6
        cout<<"ch="<<ch<<" d="<<d<<endl; //注 7
        if(ch=='\n') //注 8
            cout<<"---new line---"<<endl;
    }
}
```

[分析提示]

从标准文件 `cin`(即键盘)或从自定义磁盘文件 `fin` 读入字符时,常使用 `get` 函数或提取算符“`>>`”。但要注意这两种读入方式的如下使用区别:以 `get` 来读入字符时,总认为“空格”、“Tab 键”、“换行”这些前导字符均为一个输入字符;但若使用提取算符“`>>`”来读入字符时,在默认情况下,将甩掉前导的“空格”、“Tab 键”、“换行”(不认为它们是一个输入字符!)。如若需要,也可先通过使用成员函数“`unsetf(ios::skipws);`”,之后再使用算符“`>>`”进行输入时,将不甩掉这些前导字符(“空格”、“Tab 键”、“换行”)。

[解答程序]

具体程序见题目处。

注 1:以抽取算符“`>>`”从键盘读入一字符。默认情况下,将甩掉前导的“空格”、“Tab 键”、“换行”(不认为它们是一个输入字符!)

注 2:把刚输入字符的 ASCII 码值赋给 `d` 变量。

注 3:将读入的字符及其 ASCII 码值一并显示在屏幕上。

注 4:直到遇见“换行”(“ \n”)时结束程序。

注 5:读入下一字符。默认情况下,将甩掉前导的“空格”、“Tab 键”、“换行”。

注 6:把刚输入字符的 ASCII 码值赋给 `d` 变量。

注 7:将读入的字符及其 ASCII 码值一并显示在屏幕上。

注 8:遇换行(“ \n”)符时再显示一些提示信息。

[输出结果]

程序执行后假设输入：“空格”、“Tab 键”、“a”、“换行”(再按 `ctrl+c` 结束程序)，此时的显示结果如下：

```
a
ch=a d=97
```

11.6 从文件 `cin` 以抽取算符读入各字符的方式二

以抽取算符“`>>`”从标准文件 `cin` 即键盘读入一批字符，直到遇见换行(“`\n`”)时结束程序，并将读入的各字符及其 ASCII 码值一并显示在屏幕上。

[分析提示]

本程序中事先使用了“`cin.unsetf(ios::skipws);`”，而后再使用算符“`>>`”进行输入，此时将不再甩掉前导的“空格”、“Tab 键”、以及“换行”(将认为它们是一个输入字符!)。

[解答程序]

```
#include <iostream.h>
void main()
{
    char ch;    int d;
    cin.unsetf(ios::skipws);
    /*
        默认情况下，使用算符“>>”进行输入时，将甩掉前导的“空格”、“Tab 键”、“换行”
        (不认为它们是一个输入字符!)，但若先通过使用“cin.unsetf(ios::skipws);”，之后再使用算
        符“>>”进行输入时，将不用甩掉这些前导字符(“空格”、“Tab 键”、“换行”)。
    */
    cin>>ch;        //以抽取算符“>>”从键盘读入一字符(将不用甩掉前导字符)
    d=ch;            //把刚输入字符的 ASCII 码值赋给 d 变量
    cout<<"ch="<<ch<<" d="<<d<<endl;
                    //将读入字符及其 ASCII 码值一并显示在屏幕上
    while (ch!='\n')    //直到遇见换行(“\n”)时结束程序
    {
        cin>>ch;        //读入下一字符，不再甩掉前导字符
        d=ch;            //把刚输入字符的 ASCII 码值赋给 d 变量
        cout<<"ch="<<ch<<" d="<<d<<endl;
                        //将读入字符及其 ASCII 码值一并显示在屏幕上
        if(ch=='\n')    //遇换行(“\n”)符时再显示一些提示信息
            cout<<"---new line---"<<endl;
```



```
if(ch=='\n')           //遇换行(“ \n ”)符时再显示一些提示信息
    cout<<"---new line---"<<endl;

}
```

[输出结果]

程序执行后假设输入：“空格”、“Tab 键”、“a”、“换行”，此时的显示结果如下：

```

          a
ch=      d=32
ch=          d=9
ch=a    d=97
ch=
          d=10
---new line---
```

11.8 从磁盘文件 fin 以抽取算符读入各字符

先往自定义磁盘文件 `fin` 中写出五个字符：“空格”、“1”、“Tab 键”、“a”、“换行”；而后再以抽取算符“`>>`”从该磁盘文件 `fin` 读入这些字符（直到读至文件结束为止），并将读入的各字符及其 ASCII 码值一并显示在屏幕上。

[分析提示]

程序中事先使用了“`fin.unsetf(ios::skipws);`”，之后再使用算符“`>>`”进行输入，这时将不再甩掉前导字符（“空格”、“Tab 键”、“换行”，否则将甩掉这些前导字符）。

[解答程序]

```
#include <fstream.h>

void main( )
{
    char ch;    int d;

    fstream fin("fin.txt", ios::in|ios::out);

    //打开可用于读/写的与流 fin 关联的文件"fin.txt"

    fin<<' '<<'1'<<'\t'<<'a'<<'\n';

    //往文件中写出五个字符：“ 空格”、“ 1”、“ Tab 键”、“ a”、“ 换行”

    fin.seekg(0);    //将文件的读指针置为 0(随后将从文件首开始读)

    fin.unsetf(ios::skipws);

    //使得今后使用算符“ >>” 进行输入时，不甩掉
    //前导的“ 空格”、“ Tab 键”、以及“ 换行” 符号
```

```

        fin>>ch;                //以抽取算符“>>”从 fin 读入一字符，不甩掉前导字符
    while (!fin.eof())          //读入 fin 文件的所有字符
    {
        d=ch;                    //把刚输入字符的 ASCII 码值赋给 d 变量
        cout<<"ch="<<ch<<" d="<<d<<endl;
                                //将读入字符及其 ASCII 码值一并显示在屏幕上
        if(ch=='\n')            //遇换行(“\n”)符时再显示一些提示信息
            cout<<"---new line---"<<endl;
        fin>>ch;                //读入下一字符，读入时不甩掉前导字符
    }
}

```

[输出结果]

```

ch=    d=32
ch=1   d=49
ch=     d=9
ch=a   d=97
ch=
      d=10
---new line---

```

11.9 使用 get 与 put 对指定文件进行拷贝

编程序，使用类成员函数 get 与 put 对指定文件进行拷贝。被拷贝的源文件以及拷贝到的目的文件的名字与路径均由命令行参数来提供。

[分析提示]

(1) 编制一个 main 带参数的程序，argv[1]中存放被拷贝的源文件的名字与路径，argv[2]中存放拷贝到的目的文件的名字与路径。通过“ ifstream fin(argv[1]);”来打开用于读的源文件，并使它与 ifstream 流类对象 fin 相联系。通过“ ofstream fout(argv[2]);”来打开用于写的目的文件，并使它与 ofstream 流类对象 fout 相联系。而后使用“ fin.get(ch);”逐字符地从源文件读入，又使用“ fout.put(ch);”逐字符地向目的文件写出，从而实现文件的拷贝功能。

(2) 在 VC6.0 集成开发环境下，运行这种 main 带参数的程序之前，要通过如下步骤来设置命令行参数(程序参数)：Project => Settings... => Debug =>在 Program arguments: 框内，输入以空格分割的各参数，如，“aa.cpp aa2.cpp”(该框空时，表示没有命令行参数) => OK。

[解答程序]

```

#include <fstream.h>
#include <process.h>

void main(int argc, char* argv[ ])
{
    if(argc!=3)                //提供的命令行参数个数不对时，提示信息后退出
    {
        cout<<"ERROR !——2 arguments ?!"<<endl;
        exit (1);
    }
    cout<<"Copy file from '"<<argv[1]<<"' to '"<<argv[2]<<"'"<<endl; //屏幕提示
    ifstream fin(argv[1]);      //命令行参数 1 作为文件名
    //打开用于“ 读” 的文件(argv[1]指定)，并使它与 ifstream 流类对象 fin 相联系
    ofstream fout(argv[2]);     //命令行参数 2 作为文件名
    //打开用于“ 写” 的文件(argv[2]指定)，并使它与 ofstream 流类对象 fout 相联系
    char ch;
    fin.get(ch);                //使用 get 函数对流 fin 进行读操作
    while(!fin.eof( ))         //从头读到文件结束(当前符号非文件结束符时继续)

    {
        fout.put(ch);          //使用 put 函数对流 fout 进行写操作
        fin.get(ch);           //使用 get 函数对流 fin 进行读操作
    }
    fin.close( );              //关闭与 ifstream 流类对象 fin 相联系的文件
    fout.close( );             //关闭与 ofstream 流类对象 fout 相联系的文件
}

```

[输出结果]

若提供的命令行参数为:aa.cpp aa2.cpp 时输出

Copy file from 'aa.cpp' to 'aa2.cpp'

若提供的命令行参数个数不对时输出

ERROR ! -- 2 arguments ?!

11.10 对磁盘文件的读/写处理(数值性数据)

假设对于 k=1, 2, 3, ..., 9, 10, 分别按下式计算出了 10 组(i,d,c):

整数 $i=2*k$; 实数 $d=k*k+\text{sqrt}(k)$; 字符 $c='A'+k-1$ 。

(1) 将这 10 组数据保存到一个或多个自定义的磁盘文件中(以备后用);

(2) 将保存到磁盘文件中的那些数据读出并显示到屏幕上。

[分析提示]

可按如下四种方式来实现。

实现方式一(使用一个 text 文件):

使用运算符“<<”将这些数据保存到同一个自定义的 text 型磁盘文件中;而后再使用运算符“>>”从那一 text 文件中读出这些数据并显示到屏幕上。

实现方式二(使用三个 text 文件):

使用运算符“<<”,按不同数据类型将它们保存到三个不同的自定义 text 型磁盘文件中;

而后再使用运算符“>>”从那三个 text 文件中读出这些数据并显示到屏幕上。

实现方式三(使用三个 binary 文件):

使用 write 成员函数,按不同数据类型将这些数据保存到三个不同的自定义 binary 型磁盘文件中;

而后再使用 read 成员函数从那三个 binary 文件中读出这些数据并显示到屏幕上。

实现方式四(使用一个 binary 文件。定义存放一组(i,d,c)数据的结构体类型,而后再在文件中保存十个结构体数据):

使用 write 成员函数,将这些数据保存到同一个自定义的 binary 型磁盘文件中;

而后再使用 read 成员函数从那一 binary 文件中读出这些数据并显示到屏幕上。

[解答程序一]

//实现方式一(使用 1 个 text 文件):

//(1) 使用运算符“<<”将这些数据保存到同 1 个自定义的 text 型磁盘文件中;

//(2) 而后再使用运算符“>>”从那一 text 文件中读出这些数据并显示到屏幕上。

```
#include <math.h>
```

```
#include <fstream.h>
```

```
#include <iomanip.h>
```

```
void main(void)
```

```
{
```

```
    ofstream fout("f0.dat");
```

```
    //打开用于写的 text 文件"f0.dat"(流对象 fout 与其对应)
```

```
    int i, k;    double d;    char c;
```

```
    for(k=1; k<=10; k++)                //计算出 10 组(i,d,c)并写出到文件中
```

```
    {
```

```
        i=2*k;                d=k*k+sqrt(k);                c='A'+k-1;
```

```
        fout<<i<<" "<<d<<" "<<c<<" ";
```

```

//使用运算符“ <<” 将数据保存到同一个自定义的 text 型
//磁盘文件“ f0.dat” 中。数据间以空格相分割。

}
fout.close();
ifstream fin(“ f0.dat” );
//打开用于读的 text 文件“ f0.dat” (流对象 fin 与其对应)
for(k=1; k<=10; k++)
{
    fin>>i>>d>>c;           //将保存到磁盘文件中的那些数据读出
    cout<<"i="<<setw(2)<<i<<“, d="<<setw(7)<<d<<“, c="<<c<<endl;
                                //显示到屏幕上
}
fin.close();
}

```

[输出结果]

```

i= 2, d=      2, c=A
i= 4, d=5.41421, c=B
i= 6, d=10.7321, c=C
i= 8, d=     18, c=D
i=10, d=27.2361, c=E
i=12, d=38.4495, c=F
i=14, d=51.6458, c=G
i=16, d=66.8284, c=H
i=18, d=     84, c=I
i=20, d=103.162, c=J

```

[解答程序二]

```

//实现方式二(使用三个 text 文件):
//(1) 使用运算符“ <<”, 按不同数据类型将它们保存到三个不同的自定义 text
//型磁盘文件中;
//(2) 而后再使用运算符“ >>” 从那三个 text 文件中读出这些数据并显示到屏幕上。
#include <math.h>
#include <fstream.h>
#include <iomanip.h>

void main(void)
{
    //打开用于写的三个 text 文件

```

```

ofstream fouti("fi.dat");
ofstream foutd("fd.dat");
ofstream foutc("fc.dat");
int i, k;    double d;  char c;
for(k=1; k<=10; k++)
{
//计算出 10 组(i,d,c)并按不同数据类型将它们写出到不同的文件中
    i=2*k;          d=k*k+sqrt(k);      c='A'+k-1;
//使用运算符“ << ”来写出数据，数据间以空格相分割
    fouti<<i<<" ";      foutd<<d<<" ";      foutc<<c<<" ";
}
fouti.close();    foutd.close();    foutc.close();
//打开用于读的三个 text 文件
ifstream fini("fi.dat");
ifstream find("fd.dat");
ifstream finc("fc.dat");
for(k=1; k<=10; k++)
{
//将保存到磁盘文件中的那些数据读出并显示到屏幕上
    fini>>i;      find>>d;      finc>>c;
    cout<<"i="<<setw(2)<<i<<"", d="<<setw(7)<<d<<"", c="<<c<<endl;
}
fini.close();    find.close();    finc.close();
}

```

[输出结果]

与解答程序一随后的输出结果相同，略。

[解答程序三]

```

//实现方式三(使用三个 binary 文件):
//使用 write 成员函数，按不同数据类型将这些数据
//保存到三个不同的自定义 binary 型磁盘文件中；
//而后再使用 read 成员函数从那三个 binary 文件中读出这些数据并显示到屏幕上。
#include <math.h>
#include <fstream.h>
#include <iomanip.h>

void main(void)
{

```

```

//打开用于写的三个 binary 文件
ofstream fouti("fi.dat", ios::binary);
ofstream foutd("fd.dat", ios::binary);
ofstream foutc("fc.dat", ios::binary);
int i, k;    double d;  char c;
for(k=1; k<=10; k++)
{
    //计算出 10 组(i,d,c)并按不同数据类型将它们写出到不同的文件中
    i=2*k;          d=k*k+sqrt(k);      c='A'+k-1;
    //使用 write 成员函数来写出数据, 数据间不需要分割符
    fouti.write((char*)&i, sizeof(i));
    foutd.write((char*)&d, sizeof(d));
    foutc.write((char*)&c, sizeof(c));
}
fouti.close( );    foutd.close( );    foutc.close( );

//打开用于读的三个 binary 文件
ifstream fin_i("fi.dat", ios::binary);
ifstream fin_d("fd.dat", ios::binary);
ifstream fin_c("fc.dat", ios::binary);
for(k=1; k<=10; k++)
{
    //将保存到磁盘文件中的那些数据读出并显示到屏幕上
    fin_i.read((char*)&i, sizeof(i));
    fin_d.read((char*)&d, sizeof(d));
    fin_c.read((char*)&c, sizeof(c));
    cout<<"i="<<setw(2)<<i<<"",  d="<<setw(7)<<d<<"",  c="<<c<<endl;
}
fin_i.close( );    fin_d.close( );    fin_c.close( );
}

```

[输出结果]

与解答程序一随后的输出结果相同, 略。

[解答程序四]

```

//实现方式四 (使用一个 binary 文件。定义存放一组(i,d,c)数据的结构体类型,
//而后在文件中保存十个结构体数据):
// (1) 使用 write 成员函数, 将这些数据保存到同一个自定义的 binary 型磁盘文件中;
// (2) 而后再使用 read 成员函数从那一 binary 文件中读出这些数据并显示到屏幕上。
#include <math.h>

```



```

#include <fstream.h>
#include <iomanip.h>

void main(void)
{
    ofstream fout("f0.dat", ios::binary);    //打开用于写的 binary 文件
    struct stru1                               //定义存放一组(i,d,c)数据的结构体类型 stru1
    {
        int i;
        double d;
        char c;
    } ss;                                       //结构体变量 ss
    int k;
    for(k=1; k<=10; k++)
    {
        //计算出十组(i,d,c)——十个结构体，并将它们写出到文件中
        ss.i=2*k;  ss.d=k*k+sqrt(k);  ss.c='A'+k-1;
        fout.write((char*)&ss, sizeof(stru1)); //使用 write 成员函数来写出数据
    }
    fout.close();
    ifstream fin("f0.dat", ios::binary);      //打开用于读的 binary 文件
    for(k=1; k<=10; k++)
    {
        fin.read((char*)&ss, sizeof(stru1));
        //将保存到磁盘文件中的数据读入到结构体 ss 中
        cout<<"i="<<setw(2)<<ss.i<<" d="<<setw(7)<<ss.d<<" c="<<ss.c<<endl;
    }
    fin.close();
}

```

[输出结果]

与解答程序一随后的输出结果相同，略。

11.11 对磁盘文件的读/写处理(文本性数据)

编制具有如下原型的函数：

```
void countLetters(ifstream& fin);
```

它负责读入 text 型文本流 fin 所对应的文件内容(同时显示在屏幕上)，并统计其中

各字母的出现次数(不区分大小写),最后将统计结果显示在屏幕上。并编写主函数,从键盘输入欲处理的文件名,而后打开相应的文件并对 `countLetters` 函数进行调用。

例如,假设已经在 `fa.txt` 文件中准备好了下述的两行内容,程序执行后的屏幕显示结果可以呈现如下形式:

```
---fileName=?fa.txt
abcdefghijklmnopqrstuvwxyz1234567890!
I am a teacher. How do you do!
----- Result -----
a--4  b--1  c--2  d--3  e--3  f--1  g--1
h--3  i--2  j--1  k--1  l--1  m--2  n--1
o--5  p--1  q--1  r--2  s--1  t--2  u--2
v--1  w--2  x--1  y--2  z--1
```

[分析提示]

(1) 在 `countLetters` 函数中,通过“`int count[26];`”说明一个 `count` 数组,用于记录 26 个字母各自出现的次数。而后通过使用“`fin.getline(str, 80);`”逐行地从流类对象 `fin` 所对应的文件中读入有关信息并进行处理,直到将整个文件读入并处理结束(由程序中的“`while(!fin.eof()){ ... }`”的循环来完成)。

(2) 主函数 `main` 中,由用户从键盘输入被处理文件名 `fileName`,而后打开该文件,若该文件不存在时(若“`fin.fail()`”结果为真),意味着操作失败则提示出错信息并退出;否则,通过语句“`countLetters(fin);`”来进行调用,去处理流 `fin` 所对应的文件内容。

[解答程序]

```
#include <fstream.h>
#include <string.h>
#include <process.h>

bool isLetter(char ch)    //判断 ch 是否为字母,若是返回 true, 否则返回 false
{
    if(ch>='A' && ch<='Z' || ch>='a' && ch<='z')
        return true;
    return false;
}

void countLetters(ifstream& fin)
{
    //读出流 fin 所对应的文件内容(同时显示在屏幕上),并统计其中各字母
    //的出现次数 (不区分大小写),最后将统计结果显示在屏幕上
    int count[26];
```

```

//count 中记录各字母的出现次数。具体说, count[0]用来记录“ A” 字母
//的出现次数,
//count1[1]用来记录“ B” 字母的出现次数, ...
for (int i=0; i<26; i++)
    count[i]=0;           //作为累加和的各 count[i]均先置为 0
char str[81];             //认为每行不超过 80 字符, 读入后先放在 str 中
fin.getline(str, 80);     //从文件读入一行放 str
while(!fin.eof( ))       //将文件从头处理到尾
{
    cout<<str<<endl;     //将读入行显示在屏幕上
    i=0;
    while (char ch=str[i++]) //将 str 数组的各字符从头到尾编写一遍
        if(isLetter(ch))    //若字符为字母
            if(ch>='a')     //小写字母
                count[ch-'a']++; //count 数组中记录 str 各字母的出现次数
            else             //大写字母
                count[ch-'A']++;
    fin.getline(str, 80);    //从文件读入下一行放 str
}
//将统计结果显示在屏幕上
cout<<"----- Result -----"<<endl;
for(char ch='a'; ch<='z'; ch++)
{
    cout<<ch<<"——"<<count[ch-'a']<<" ";
    if( (ch-'a'+1)%7==0 )    //七个字母的结果占 1 行
        cout<<endl;
}
cout<<endl;
fin.close( );
}

void main(void)
{
    char fileName[30];      //被处理文件的名称将放于 fileName 之中(由用户从键盘输入)
    cout<<"——fileName=?";
    cin.getline(fileName, 30); //输入被处理文件的名称 fileName
    ifstream fin(fileName, ios::nocreate);
        //ios::nocreate ——仅打开一个已存在文件(文件不存在时操作失败)
        //打开用于读的文件(由 fileName 指定)
        //并使它与 ifstream 流类对象 fin 相联系

```

```

if(fin.fail( ))    //当打开一个不存在的文件时(由 fileName 指定的文件不存在)
{
    cout<<"Can not open file "<<fileName<<"!"<<endl; //提示信息并退出
    exit(1);
}
countLetters(fin);    //调用 countLetters 函数去处理流 fin 所对应的文件内容
}

```

[输出结果]

```

---fileName=?fa.txt
abcdefghijklmnopqrstuvwxyz1234567890!
I am a teacher. How do you do!
----- Result -----
a——4  b——1  c——2  d——3  e——3  f——1  g——1
h——3  i——2  j——1  k——1  l——1  m——2  n——1
o——5  p——1  q——1  r——2  s——1  t——2  u——2
v——1  w——2  x——1  y——2  z——1

```

11.12 使用 getline 读出 text 文件中的“一篇文章”进行处理

使用 getline 成员函数读出某个 text 文件中的一篇文章(如可以是一个 C++ 源程序, 文件名由用户从键盘输入), 而后数出该文件的总行数 totalLine, 并找出且显示其中的最长行 maxLineTxt 以及最短的非空行 minLineTxt, 而且还要求同时显示出它们的行号 maxLineNum、minLineNum 以及行长 maxLineLen、minLineLen。

[分析提示]

由用户从键盘输入欲处理文件的名称 fileName, 而后通过“ ifstream fin(fileName, ios::nocreate);”的方式打开该文件, 并使它与 ifstream 流类对象 fin 相联系, 其中的第二参数“ ios::nocreate”指出仅打开一个已存在文件。而后通过使用如下的循环模式来完成指定的任务。

```

while(!fin.eof( ))    //当没有处理到文件末尾时就继续循环
{
    fin.getline(str, 100);    //从文件读入一新行放入 str 之中
    对刚刚读入的新行 str 进行处理;
}

```

[解答程序]

```
#include <fstream.h>
```

```

#include <string.h>
#include <process.h>

void main(void)
{
    char fileName[30]; //被处理文件的名称将放于 fileName 之中(由用户从键盘输入)
    int totalLine=0, maxLineNum, minLineNum, maxLineLen, minLineLen;
        //文件的总行数 totalLine, 最长行 maxLineTxt,
        //其行号为 maxLineNum, 行长 maxLineLen;
        //最短的非空行 minLineTxt, 行号 minLineNum, 行长 minLineLen。
    char str[100], maxLineTxt[100]="\0", minLineTxt[100];
        //认为每行不超过 99 个字符, 读入后先放在 str 中
    for(int i=0; i<99; i++)
        minLineTxt[i]=' ';
        //初始化存放最短的非空行 minLineTxt(数组, 先置为最长)
    minLineTxt[99]='\0';
    cout<<"fileName=?";
    cin>>fileName; //被处理文件的名称 fileName 由用户从键盘输入
    ifstream fin(fileName, ios::nocreate);
        //ios::nocreate——仅打开一个已存在文件(文件不存在时操作失败)
        //打开用于读的文件(由 fileName 指定)
        //并使它与 ifstream 流对象 fin 相联系
    if(fin.fail()) //当打开一个不存在的文件时(由 fileName 指定的文件不存在)
    {
        cout<<"Can not open file '"<<fileName<<"'!"<<endl; //提示信息并退出
        exit(1);
    }
    while(!fin.eof())
    {
        //数出该文件的总行数 totalLine, 并找出且显示其中的最长行 maxLineTxt
        //以及最短的非空行 minLineTxt, 而且还记录下它们的行号 maxLineNum、
        //minLineNum 以及行长 maxLineLen、minLineLen。
        fin.getline(str, 100);
        totalLine++; //总行数 totalLine 加 1
        if(strlen(str)>strlen(maxLineTxt))
        {
            strcpy(maxLineTxt, str); //最长行 maxLineTxt
            maxLineNum=totalLine; //最长行的行号 maxLineNum
            maxLineLen=strlen(str); //最长行的行长 maxLineLen
        }
    }
}

```

```

    }
    if(strlen(str)!=0 && strlen(str)<strlen(minLineTxt))
    {
        strcpy(minLineTxt, str);    //最短的非空行 minLineTxt
        minLineNum=totalLine;    //最短非空行的行号 minLineNum
        minLineLen=strlen(str);    //最短非空行的行长 minLineLen
    }
}
//输出所有统计结果
cout<<"totalLine="<<totalLine<<endl;
cout<<"maxLineNum="<<maxLineNum<<", maxLineLen="<<maxLineLen<<endl;
cout<<"maxLineTxt="<<maxLineTxt<<endl;
cout<<"minLineNum="<<minLineNum<<", minLineLen="<<minLineLen<<endl;
cout<<"minLineTxt="<<minLineTxt<<endl;
fin.close();
}

```

[输出结果]

程序执行后，屏幕显示结果为(其中假设 tmp.cpp 为某个 C++源程序文件):

```

fileName=?tmp.cpp
totalLine=71
maxLineNum=3, maxLineLen=92
maxLineTxt=    12.4 使用 getline 成员函数读出某个 text 文件中的一篇文章(如可
以是一个 C++源程序，文件名由
minLineNum=13, minLineLen=1
minLineTxt={

```

11.13 使用 getline 读出 text 文件中的 C++源程序并删除注释

使用 getline 成员函数读出某个 text 文件中的一个 C++源程序 文件名由用户从键盘输入，如“ a.cpp”), 而后将该文件的各程序行中的注释均删除掉(该行中从双撇符号“//”开始直至行末的所有符号为注释), 并将删除注释后的结果行写出到与读入文件相呼应的另一个文件中(被写文件名是通过在读入文件名的前面加上“ DEL”而形成的，如通过输入文件名“ a.cpp”而由程序形成被写文件名“ DELa.cpp”)。

[分析提示]

由用户从键盘输入欲处理文件的名称 fileName，并通过“ ifstream fin(fileName, ios::nocreate);”的方式打开一个已存在文件；而后通过在当前文件名 fileName 的前面加

上“ DEL”而形成一个被写文件名 outFName, 之后通过“ ofstream fout(outFName);”的方式打开用于写删除注释后诸结果行的另一个文件。接着通过成员函数 getline 从 fin 流类对象所对应的文件中依次读入每一行进行处理直至结束。另外, 对某一程序行删除注释的任务由自定义的形如“ void delComments(char* str){...}”的函数来完成, 由它负责将参数 str 带来的程序行中的注释删除掉, 并仍通过该指针参数返回结果字符串到调用处。

[解答程序]

```
#include <fstream.h>
#include <string.h>
#include <process.h>

void delComments(char* str)
{
    //将参数 str 带来的程序行中的注释删除掉(仍通过 str 返回结果串)
    //处理方法: 寻找该行中第一次出现双撇符号“ //” 的位置
    //并在第一个撇符号位置处置上一个结束符“ \0”
    int len=strlen(str);
    for (int i=0; i<len-1; i++)
    {
        if (str[i]=='/' && str[i+1]=='/')
            //寻找该行中第一次出现双撇符号“ //” 的位置
            str[i]='\0';    //并在第一个撇符号位置处置上一个结束符“ \0”
    }
}

void main(void)
{
    char fileName[30];
    //被读入进行处理的文件的名字将放于 fileName 之中(由用户从键盘输入)
    cout<<"——fileName=?";
    cin.getline(fileName, 30);    //输入被处理文件的名字 fileName
    ifstream fin(fileName, ios::nocreate);
    //ios::nocreate——仅打开一个已存在文件(文件不存在时操作失败)
    //打开用于读的文件(由 fileName 指定)
    //并使它与 ifstream 流类对象 fin 相联系
    if(fin.fail()) //当打开一个不存在的文件时(由 fileName 指定的文件不存在)
    {
        cout<<"Can not open file“ "<<fileName<<"”  !"<<endl; //提示信息并退出
        exit(1);
    }
}
```

```

    }
    cout<<"Read from "<<fileName<<endl;        //屏幕显示信息
    char outFName[33]; //被写文件的名字将放于 outFName 之中(由程序形成)
    strcpy(outFName, "DEL");
        //被写文件名是通过在读入文件名的前面加上“ DEL” 而形成的
    strcat(outFName, fileName);
    ofstream fout(outFName);                    //打开用于写的文件(由 outFName 指定),
                                                //并使它与 ofstream 流类对象 fout 相联系
    cout<<"Write to "<<outFName<<endl;        //屏幕显示信息
    char str[101];                               //用于存放所读文件的一行
    fin.getline(str, 100);                       //从文件读入一行放 str
    while(!fin.eof( ))                          //将文件从头处理到尾
    {
        int sourceLen=strlen(str);               //删除注释之前 str 行的长度
        delComments(str);
        //调用 delComments 函数将 str 行中的注释删除掉(仍通过 str 返回结果串)
        int resLen=strlen(str);                 //删除注释之后 str 行的长度
        if ( resLen>0  || resLen==0 && sourceLen==0 )
            //删除注释后不为空行者要写出。另外,
            //删除注释前就为空行(行的长度为 0)时, 结果文件中也输出一个空行
            fout<<str<<endl;
        fin.getline(str, 100);                  //从文件读入下一行放 str
    }
}

```

[输出结果]

--fileName=?a.cpp

Read from a.cpp

Write to DELa.cpp

另外注意, 执行完该程序后, 要检查一下“ a.cpp” 以及“ DELa.cpp” 两文件中的具体内容, 以断定本程序所实现功能的正确性。例如: 假设执行该程序前准备的“ a.cpp” 文件中具有如下的几行程序正文:

```

void main(void)           //主函数
{
    int i=1,j=2;           //说明变量 i, j 并赋予初值
    cout<<i<<j;
}

```

则执行程序后的“ DELa.cpp” 文件中会形成删除注释后的如下新程序正文:

```

void main(void)

```



```

{
int i=1,j=2;
cout<<i<<j;
}

```

11.14 读出两个指定文件的各行内容进行比较

使用类成员函数 `getline` 读出两个指定文件的各行内容进行比较。要求如下：

(1) 进行比较的两个文件的名字与路径均由命令行参数来提供。

(2) 具体实现时，每次对两文件各读一行，比较后若发现有不同，则在屏幕上显示出此两行内容；每发现五个不同行后则暂停一下（让用户观看确认），等用户任按一键后继续。若两文件行数不相同（两文件不同时结束时），则还要显示出较长文件的剩余部分的第一行后而结束。

[分析提示]

(1) 自定义返回真假值的函数 `LineEq1`，由它判断行 `line1`（字符串）是否与行 `line2`（字符串）完全相同：“`bool LineEq1(char * line, char * line2){...}`”。

(2) 从 `argv[1]` 及 `argv[2]` 处获取到二文件的名字，之后打开二文件并通过 `getline` 读入它们的各行内容进行比较直至结束。

(3) 在 VC6.0 环境下调试本程序时，若使用如下两个命令行参数：“`f1.cpp f2.cpp`”，则将对处于与本源程序文件同一文件夹（子目录）下的上述两个文件进行比较。

[解答程序]

```

#include <fstream.h>
#include <process.h>
#include <string.h>
#include <conio.h>

//负责读入一字符，但不回显，实现暂停功能
const int MaxChar=200; //假设每行不超过 199 个字符

bool LineEq1(char * line, char * line2)
{
    //判断行 line(字符串)是否与行 line2(字符串)相同

    bool res=true;
    for(int i=0; i<=strlen(line); i++)
        if(line[i]!=line2[i]) //若发现有一个字符不同时则返回 false
        {
            res=false;

```

```

        break;
    }
    return res;          //两串全同时返回 true
}

void main(int argc, char* argv[ ])
{
    if(argc!=3)          //提供的命令行参数个数不对时，提示信息后退出
    {
        cout<<"ERROR !——supplying 2 command-line arguments ?!"<<endl;
        exit(1);
    }
    cout<<" ***** Comparing "<<argv[1]<<" with "<<argv[2]<<" ***** "<<endl<<endl;
    ifstream fin(argv[1], ios::nocreate); //命令行参数 1——argv[1]作为文件名
        //ios::nocreate ——仅打开一个已存在文件(文件不存在时操作失败)
        //打开用于读的文件(argv[1]指定)，并使它与 ifstream 流类对象 fin 联系
    ifstream fin2(argv[2], ios::nocreate); //命令行参数 2——argv[2]作为文件名
        //ios::nocreate——仅打开一个已存在文件(文件不存在时操作失败)
    if(fin.fail( ))      //当打开一个不存在的文件时(argv[1]指定的文件不存在)
    {
        cout<<"Can not open file "<<argv[1]<<"!"<<endl; //提示信息并退出
        exit(2);
    }
    if(fin2.fail( ))     //当打开一个不存在的文件时(argv[2]指定的文件不存在)
    {
        cout<<"Can not open file "<<argv[2]<<"!"<<endl; //提示信息并退出
        exit(3);
    }
    int LineNum=0, errSum=0;
    char line[MaxChar], line2[MaxChar];
    fin.getline(line, MaxChar);
        //从第一文件读出一行(至多 MaxChar 个字符)放入 line 中
    fin2.getline(line2, MaxChar);
        //从第二文件读出一行(至多 MaxChar 个字符)放入 line2 中
    /*

```

具体实现时，每次对两文件各读一行，比较后若发现有不同，则在屏幕上显示出此两行内容；每发现五个不同行后则暂停一下(让用户观看确认)，等用户任按一键后继续。若两文件行数不相同(两文件不同时结束时)，则还要显示出较长文件的剩余部分的第一行而后结束程序的执行。

```

*/

while(!fin.eof() && !fin2.eof())
{
    //从头读到某一文件结束(当前符号非文件结束符时继续)
    LineNum++;
    if(!LineEq1(line, line2)) //若两行不相同时, 则在屏幕上显示出此两行内容
    {
        errSum++; //不相同行的个数(满五后屏幕显示暂停, 并重新清为 0)
        cout<<" ---- DIFFERENT! ----   LineNum="<<LineNum<<endl;
        cout<<line<<endl;
        cout<<line2<<endl;
    }
    if(errSum && errSum%5==0)
    {
        //每发现五个不同行后则暂停一下(让用户观看确认),
        //等用户任按一键后继续

        errSum=0; //重新清为 0
        _getch(); //暂停功能。任意输入一个字符, 不回显, 则继续
    }
    fin.getline(line, MaxChar);
    //从第一文件再读出一行(至多 MaxChar 个字符)放入 line 中
    fin2.getline(line2, MaxChar);
    //从第二文件再读出一行(至多 MaxChar 个字符)放入 line2 中
}
cout<<"Comparing_Total_LineNum="<<LineNum<<endl<<endl;
//显示比较的总行数

//若两文件行数不不同时( 两文件不同时结束时),
//则还要显示出较长文件的剩余部分的第一行而后结束程序的执行。
if (fin.eof() && !fin2.eof()) //若第一文件结束了, 但第二文件没结束时
{
    cout<<"   !!!   File "<<argv[1]<<" ended, but file ";
    cout<<argv[2]<<" dose't not!   !!!"<<endl;
    cout<<" --- The next line of file "<<argv[2]<<" is ---"<<endl;
    cout<<line2<<endl; //第二文件的剩余部分的第一行
}
else
    if (fin2.eof() && !fin.eof()) //若第二文件结束了, 但第一文件没结束时

```

```

    {
        cout<<"  !!!  File "<<argv[2]<<" ended, but file ";
        cout<<argv[1]<<" dose`not!  !!!"<<endl;
        cout<<" --- The next line of file "<<argv[1]<<" is ---"<<endl;
        cout<<line<<endl;    //第一文件的剩余部分的第一行
    }
    fin.close();    //关闭与 ifstream 流类对象 fin 相联系的文件
    fin2.close();    //关闭与 ifstream 流类对象 fin2 相联系的文件
}

```

[输出结果]

程序执行后, 屏幕显示结果为(假设进行比较的文件为“ f1.cpp” 和“ f2.cpp”, 它们间仅一行有差别):

```

***** Comparing 'f1.cpp' with 'f2.cpp' *****

----- DIFFERENT! -----   LineNum=114
    array<T>& operator=(array otherArrObj);
    array<T>& operator=(array<T>& otherArrObj);
Comparing_Total_LineNum=252

```

11.15 使用算符“>>”与“<<”对自定义的 text 型文件进行读/写

通过使用预定义的运算符“>>”与“<<”, 对自定义的 text 型文件进行如下指定的读/写操作。

- (1) 对 $x=1, 2, 3, \dots, 20$, 计算 $y=x*x+0.1$, 并将 20 个结果写出到自定义的 text 文件“ f01.dat” 中;
- (2) 对 $x=1, 2, 3, \dots, 15$, 计算 $y=10*x+0.1$, 并将 15 个结果写出到自定义的 text 文件“ f02.dat” 中, 而后再写出第 16 个数: 579.8;
- (3) 读出文件“ f01.dat” 以及“ f02.dat” 中的数据, 用来形成 text 文件“ f03.dat”, 但要求存放于“ f03.dat” 中的数据仍然是有序的(按照从小到大的顺序);
- (4) 从头到尾读出“ f03.dat” 中的数据显示在屏幕上。

[分析提示]

(1) 要完成第一与第二项工作, 只需使用一重循环生成所需数据, 而后写出到指定文件中; 第四项工作也只需使用一重循环将指定文件中的数据从头到尾读出并显示在屏幕上。

(2) 为完成第三项工作, 自定义一个函数“ `bool readToY(istream& fin, double & y){ ... }`”, 由它负责从输入流 `fin` 中读入一个数据放入引用变量 `y` 中并返回 `true`, 若没读到

数据(已到文件尾)时返回 false; 而后通过调用该函数, 并按照二路归并思想, 从文件“ f01.dat” 以及“ f02.dat” 中读出数据并将读出的各数据按照从小到大的顺序存放到另一个文件中(二路归并的思想可参看本书范例 6.21 的“合并两个有序链表形成一个新的有序结果链表”的处理方法。本范例要考虑两个文件都没结束也即都没被取光的情况, 以及至少有一个已结束而要进一步处理尚未结束的那一文件剩余数据的情况)。

[解答程序]

```
#include <fstream.h>
#include <iomanip.h>

bool readToY(istream& fin, double & y)           //自定义函数 readToY
{
    //从输入流 fin 中读入一个数据放入引用变量 y 中
    //若没读到数据(已到文件尾时)返回 false
    if (!fin.eof( ))           //文件没结束时
    {
        fin>>y;               //从输入流 fin 中读入一个数据放入引用变量 y 中
        return true;          //返回 true
    }
    else                       //若文件已结束
        return false;         //返回 false
}

void main(void)
{
    int x; double y;
    //对 x=1, 2, 3, ..., 20, 计算 y=x*x+0.1
    //并将 20 个结果写出到自定义的 text 文件“ f01.dat” 中
    ofstream fout1("f01.dat"); //打开用于写的 text 文件“ f01.dat”
    for(x=1; x<=20; x++)
    {
        y=x*x+0.1;            //计算 y=x*x+0.1
        if(x==1)              //若第一个数时, 其前不加空格
            fout1<<y;          //将结果写出到自定义的 text 文件“ f01.dat” 中
        else                   //非第一个数时, 其前加一个空格
            fout1<<" "<<y;      //text 文件时, 数间需要以空格作为分割符
    }
    fout1.close( );
}
```

```

//对 x=1, 2, 3, ..., 15, 计算 y=10*x+0.1
//并将 15 个结果写出到自定义的 text 文件“ f02.dat” 中
ofstream fout2("f02.dat"); //打开用于写的 text 文件“ f02.dat”
for(x=1; x<=15; x++)
{
    y=10*x+0.1; //计算 y=10*x+0.1
    fout2<<y<<" ";
    //将结果写出到自定义的 text 文件“ f02.dat” 中, 数间需要以空格作为分隔符
}
fout2<<579.8; //而后再写出第 16 个数: 579.8, 最末一数后不加空格
fout2.close();
//读出文件“ f01.dat” 以及“ f02.dat” 中的数据,
//用来形成 text 文件“ f03.dat”, 但要求存放于“ f03.dat” 中
//的数据仍然是有序的(按照从小到大的顺序);
ifstream fin1("f01.dat"); //打开用于读的 text 文件“ f01.dat”
ifstream fin2("f02.dat"); //打开用于读的 text 文件“ f02.dat”
ofstream fout3("f03.dat"); //打开用于写的 text 文件“ f03.dat”
double y1,y2; bool b1, b2;
b1=readToY(fin1, y1); //调用 readToY 从 fin1 对应的文件中读数据放入 y1
b2=readToY(fin2, y2); //调用 readToY 从 fin2 对应的文件中读数据放入 y2
while ( b1 && b2 ) //当读到了两个真实数据时(两个文件都没结束时)
{
    if(y1<y2) //y1 小, 写出 y1, 再从 fin1 对应的文件中读下一数据放入 y1
    {
        fout3<<" "<<y1; //数间以空格分割
        b1=readToY(fin1, y1);
    }
    else //y2 小, 写出 y2, 再从 fin2 对应的文件中读下一数据放入 y2
    {
        fout3<<" "<<y2; //数间以空格分割
        b2=readToY(fin2, y2);
    }
}
while (b1) //当仅读到了一个真实数据 y1 时(fin2 对应的文件已结束)
{
    fout3<<" "<<y1; //写出 y1
    b1=readToY(fin1, y1); //再从 fin1 对应的文件中读下一数据放入 y1
}
while (b2) //当仅读到了一个真实数据 y2 时(fin1 对应的文件已结束)

```

```

    {
        fout3<<" "<<y2;           //写出 y2
        b2=readToY(fin2, y2);      //再从 fin2 对应的文件中读下一数据放入 y2
    }
    fin1.close( );    fin2.close( );    fout3.close( );

    //从头到尾读出“ f03.dat” 中的数据显示在屏幕上
    ifstream fin3("f03.dat");      //打开用于读的 text 文件“ f03.dat”
    double y3;
    while (!fin3.eof( ))           //连续读，直至文件结束
    {
        fin3>>y3;
        cout<<setw(8)<<y3; //将读出的数据显示在屏幕上(每一数占八个字符宽度)
    }
    fin3.close( );
    cout<<endl;
}

```

[输出结果]

1.1	4.1	9.1	10.1	16.1	20.1	25.1	30.1
36.1	40.1	49.1	50.1	60.1	64.1	70.1	80.1
81.1	90.1	100.1	100.1	110.1	120.1	121.1	130.1
140.1	144.1	150.1	169.1	196.1	225.1	256.1	289.1
324.1	361.1	400.1	579.8				

11.16 使用 write 与 read 对自定义的 binary 型文件进行读/写

通过使用 write 与 read 成员函数，对自定义的 binary 型文件进行如下指定的读/写操作。

- (1) 对 $x=1, 2, 3, \dots, 20$ ，计算 $y=x*x+0.1$ ，并将 20 个结果写出到自定义的 binary 文件“ f01.dat”中；
- (2) 对 $x=1, 2, 3, \dots, 15$ ，计算 $y=10*x+0.1$ ，并将 15 个结果写出到自定义的 binary 文件“ f02.dat”中，而后再写出第 16 个数：579.8；
- (3) 读出文件“ f01.dat”以及“ f02.dat”中的数据，用来形成 binary 文件“ f03.dat”，但要求存放于“ f03.dat”中的数据仍然是有序的（按照从小到大的顺序）；
- (4) 从头到尾读出“ f03.dat”中的数据显示在屏幕上。

[分析提示]

本范例要实现的任务与上一范例具有类似性，实现方法也与上一范例基本相同。所不同的是，本范例要使用 `write` 与 `read` 对 `binary` 型文件进行读/写，而上一范例则使用运算符“`>>`”与“`<<`”对 `text` 型文件进行读/写操作。为完成第三项工作，也自定义了一个函数“`bool readToY(istream& fin, double & y){ ... }`”，而后通过调用该函数，也按照所谓的二路归并思想，从两个文件中读出数据并按序存放到另一个指定文件中。具体可参看上一范例的分析提示。

[解答程序]

```
#include <fstream.h>
#include <iomanip.h>

bool readToY(istream& fin, double & y)    //自定义函数 readToY
{
    //从输入流 fin 中读入一个数据放入引用变量 y 中，
    //若没读到数据(已到文件尾时)返回 false
    fin.read((char*)&y, sizeof(y));
    //从输入流 fin 中读入一个数据放入引用变量 y 中
    if (!fin.eof())                //文件没结束时，读入的 y 为真实数据
        return true;              //返回 true
    else                            //若文件已结束
        return false;             //返回 false
}

void main(void)
{
    int x; double y;
    //对 x=1, 2, 3, ..., 20, 计算 y=x*x+0.1
    //并将 20 个结果写出到自定义的 binary 文件“ f01.dat” 中
    ofstream fout1("f01.dat", ios::binary); //打开用于写的 binary 文件“ f01.dat”
    for(x=1; x<=20; x++)
    {
        y=x*x+0.1;                //计算 y=x*x+0.1
        fout1.write((char*)&y, sizeof(y));
        //将结果写出到自定义的 binary 文件“ f01.dat” 中，数间不需要分隔符
    }
    fout1.close();

    //对 x=1, 2, 3, ..., 15, 计算 y=10*x+0.1
```



```

//并将 15 个结果写出到自定义的 binary 文件“ f02.dat” 中
ofstream fout2("f02.dat", ios::binary); //打开用于写的 binary 文件“ f02.dat”
for(x=1; x<=15; x++)
{
    y=10*x+0.1; //计算 y=10*x+0.1
    fout2.write((char*)&y, sizeof(y));
    //将结果写出到自定义的 binary 文件“ f02.dat” 中，数间不需要分割符
}
y=579.8;
fout2.write((char*)&y, sizeof(y)); //而后再写出第 16 个数： 579.8
fout2.close( );

//读出文件“ f01.dat” 以及“ f02.dat” 中的数据，用来形成 binary 文件“ f03.dat”，
//但要求存放于“ f03.dat” 中的数据仍然是有序的(按照从小到大的顺序)。
ifstream fin1("f01.dat", ios::binary); //打开用于读的 binary 文件“ f01.dat”
ifstream fin2("f02.dat", ios::binary); //打开用于读的 binary 文件“ f02.dat”
ofstream fout3("f03.dat", ios::binary); //打开用于写的 binary 文件“ f03.dat”
double y1,y2; bool b1, b2;
b1=readToY(fin1, y1); //调用 readToY 从 fin1 对应的文件中读数据放入 y1
b2=readToY(fin2, y2); //调用 readToY 从 fin2 对应的文件中读数据放入 y2
while ( b1 && b2 ) //当读到了两个真实数据时(两个文件都没结束时)
{
    if(y1<y2) //y1 小，写出 y1，再从 fin1 对应的文件中读下一数据放入 y1
    {
        fout3.write((char*)&y1, sizeof(y1)); //写出 y1
        b1=readToY(fin1, y1);
    }
    else //y2 小，写出 y2，再从 fin2 对应的文件中读下一数据放入 y2
    {
        fout3.write((char*)&y2, sizeof(y2)); //写出 y2
        b2=readToY(fin2, y2);
    }
}
while (b1) //当仅读到了一个真实数据 y1 时(fin2 对应的文件已结束)
    fout3.write((char*)&y1, sizeof(y1)); //写出 y1
    b1=readToY(fin1, y1); //再从 fin1 对应的文件中读下一数据放入 y1
}
while (b2) //当仅读到了 1 个真实数据 y2 时(fin1 对应的文件已结束)
{

```

```

        fout3.write((char*)&y2, sizeof(y2));           //写出 y2
        b2=readToY(fin2, y2); //再从 fin2 对应的文件中读下一数据放入 y2
    }
    fin1.close();    fin2.close();    fout3.close();

    //从头到尾读出“ f03.dat” 中的数据显示在屏幕上。
    ifstream fin3("f03.dat", ios::binary);    //打开用于读的 binary 文件“ f03.dat”
    double y3;
    fin3.read((char*)&y3, sizeof(y));           //读数据到 y3
    while (!fin3.eof())    //若 y3 为真实数据，也即尚未遇 fin3 文件结束时
    {
        cout<<setw(8)<<y3;           //将读出的数据 y3 显示在屏幕上
        fin3.read((char*)&y3, sizeof(y)); //再读下一数据到 y3
    }
    fin3.close();
    cout<<endl;
}

```

[输出结果]

```

1.1    4.1    9.1    10.1    16.1    20.1    25.1    30.1
36.1   40.1   49.1   50.1   60.1   64.1   70.1   80.1
81.1   90.1  100.1  100.1  110.1  120.1  121.1  130.1
140.1  144.1  150.1  169.1  196.1  225.1  256.1  289.1
324.1  361.1  400.1  579.8

```

11.17 输入 10 整数而后反序输出的实现方式九

读下述程序，并上机进行验证，看是否实现了输入十个整数而后反序输出的预期目标。注意，本程序使用 text 文件，将输入数据存放在文件中。

```

#include <fstream.h>
void main(void)
{
    const int n=10;  int a, i;
    ofstream fout("fa.txt");           //注 1
    cout<<"Input "<<n<<" integers:"<<endl;
    for(i=1; i<=n; i++)               //注 2
    {
        cin>>a;
    }
}

```

```

        fout<<a<<" ";                                //注 3
    }
    fout.close( );                                    //注 4
    cout<<"---- The result ----"<<endl;
    ifstream fin("fa.txt");                            //注 5
    for(i=n; i>=1; i--)                                //注 6
    {
        fin.seekg(0);                                    //注 7
        for(int j=1; j<=i; j++)                        //注 8
            fin>>a;                                    //注 9
        cout<<a<<" ";                                //注 10
    }
    fin.close( );    cout<<endl;
}

```

[分析提示]

按输入数据的顺序, 先将各数据写出到 text 文件“ fa.txt”中(越靠前输入的数据越处在文件的前端), 而后再设法从文件中读出每一个所需的数据并显示出来(第一次读出最靠后的数据、第二次读出倒数第二靠后的、...)。

[解答程序]

具体程序见题目处。

注 1: 打开用于写的 text 文件“ fa.txt”。

注 2: 输入 10 个数据, 并写出到 text 文件“ fa.txt”中。

注 3: 通过算符“ <<”将 a 写出到文件中, 数间以空格分割。

注 4: 关闭文件。

注 5: 打开用于读的 text 文件“ fa.txt”。

注 6: 顺序从头读十次文件, 每一次找出一个所需的数据显示出来。i=n 的循环找出第 n 个数据(最后一个数据), 而后是倒数第二个数据, ..., 最后是文件中的第一个数据。

注 7: 将读指针位置跳转到文件开头处。

注 8: 共读出 i 个数据(i 第一次为 n, 而后为 n-1, ...)。

注 9: 但对前 i-1 个数据不加处理(跳过它们), 而将第 i 个数据读至变量 a 中。

注 10: 将第 i 个数据 a 显示在屏幕上。

[输出结果]

```

Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1

```

11.18 输入 10 个整数而后反序输出的实现方式十

读下述程序，并上机进行验证，看是否实现了输入 10 个整数而后反序输出的预期目标。注意，本程序使用 **binary** 文件，将输入数据存放在文件中，并使用随机访问方式读出。

```
#include <fstream.h>
void main(void)
{
    const int n=10;    int a, i;
    ofstream fout("fb.bin", ios::binary);           //注 1
    cout<<"Input "<<n<<" integers:"<<endl;
    for(i=1; i<=n; i++)                             //注 2
    {
        cin>>a;                                       //注 3
        fout.write((char*)&a,sizeof(int));           //注 4
    }
    fout.close( );
    cout<<"---- The result ----"<<endl;
    ifstream fin("fb.bin", ios::binary);             //注 5
    for(i=n-1; i>=0; i--)                             //注 6
    {
        fin.seekg(i*sizeof(int));                   //注 7
        fin.read((char*)&a,sizeof(int));             //注 8
        cout<<a<<" ";                               //注 9
    }
    fin.close( );    cout<<endl;
}
```

[分析提示]

按输入数据的顺序，先将各数据写出到可进行随机访问的 **binary** 文件“ **fb.bin**”中，而后通过使用“ **seekg**”先定位到最后输入的那一数据处并使用“ **read**”将其读出，再往下是倒数第二靠后的、...，直至全部读入处理完毕。

[解答程序]

具体程序见题目处。

注 1：打开用于写的二进制文件“ **fb.bin**”。

注 2：输入 10 个数据，并写出到二进制文件“ **fb.bin**”中。

注 3：从键盘读入一个数据 **a**。

- 注 4: 通过 write 成员函数将 a 写出到文件中。
 注 5: 打开用于读的二进制文件“ fb.bin”。
 注 6: 从文件中(从后往前)读出数据显示在屏幕上。
 注 7: 读指针位置与 i 的大小有关(从后往前读)。
 注 8: 读一个 int 型数据放入 a。
 注 9: 将读出的数据 a 显示在屏幕上。

[输出结果]

```
Input 10 integers:
1 2 3 4 5 6 7 8 9 10
---- The result ----
10 9 8 7 6 5 4 3 2 1
```

11.19 binary 型学生数据文件的简单管理

采用 binary 文件形式对类对象数据进行存储与读写处理(使用 write 将类对象数据写出到某个自定义二进制磁盘文件中,而后再使用 read 读出这些类对象数据并进行处理)。

自定义一个类(类型) CStudent,它具有如下九个数据成员:学号、姓名、班级、性别、年龄、数学成绩、语文成绩、英语成绩、奖惩记录。并具有三个成员函数,而且对输出运算符“ <<”进行重载,其类定义格式如下:

```
class CStudent //自定义类(类型) Cstudent
{
public:
    long num; //学号,共九个数据成员
    char name [20]; //姓名
    int cla; //班级
    char sex; //性别
    int age; //年龄
    float math, chinese, eng; //数学成绩、语文成绩、英语成绩
    char rec[20]; //奖惩记录
    void Init_DataFile(CStudent* stu, int n); //共三个成员函数
        //将程序(主函数) stu 数组中准备好的 n 个类对象初始数据
        //依次写出到自定义二进制磁盘文件中
    void Read_Append_1Rec();
        //从键盘读入一个类对象数据,使用 write 将该类对象数据
        //追加到自定义二进制磁盘文件尾部
    void Display_allRec(); //使用 read 从文件读出所有对象数据并显示在屏幕上
```

```
friend ostream& operator << (ostream& out, CStudent& stu); //一个友元函数
//重载运算符“ <<”，从而可以使用该运算符直接输出 CStudent 类对象
};
```

进而可编写如下形式的主函数，说明 CStudent 类对象，并通过调用其成员函数完成数据文件的处理。

```
void main() //主函数，对所定义的类进行使用
{
    const int n=3;
    CStudent stu[n] = //事先准备好的 n 个类对象初始数据
    {
        {100001, "li", 1, 'm', 16, 89, 98, 94, "SanHaoSheng"},
        {100002, "ma", 1, 'f', 17, 83, 90, 89, "BanGanBu"},
        {100003, "wu", 2, 'f', 17, 92, 88, 90, "nothing" }
    };
    CStudent ss; //CStudent 类对象 ss
    ss.Init_DataFile(stu, n);
        //将程序(主函数)stu 数组中准备好的 n 个类对象初始数据
        //依次写出到自定义二进制磁盘文件中
    ss.Display_allRec(); //使用 read 从文件读出所有对象数据并显示在屏幕上
    ss.Read_Append_1Rec( );
        //从键盘读入一个类对象数据，使用 write 将该类对象数据
        //追加到自定义二进制磁盘文件尾部
    ss.Display_allRec( ); //使用 read 从文件读出所有对象数据并显示在屏幕上
}
```

程序执行后的显示结果如下：

```
---Init_DataFile( ), 3 RECs will be saved to 'f01.dat'---
-----Display_allRec()-----
100001 li 1 m 16 89 98 94 SanHaoSheng
100002 ma 1 f 17 83 90 89 BanGanBu
100003 wu 2 f 17 92 88 90 nothing
-----Read_Append_1Rec()-----
input REC's inf -- num, name, cla, sex, age, math, chinese, eng, rec:
100004 yu 2 m 18 90 88 84 nothing
-----Display_allRec()-----
100001 li 1 m 16 89 98 94 SanHaoSheng
100002 ma 1 f 17 83 90 89 BanGanBu
100003 wu 2 f 17 92 88 90 nothing
```

100004 yu 2 m 18 90 88 84 nothing

[分析提示]

(1) 本范例要处理类 Cstudent 的对象数据：将一批这种对象数据存储到某个 binary 类型的文件中长期保存起来；从键盘读入一个类对象数据并将它追加到该 binary 文件的尾部；将 binary 文件中保存的各对象数据显示在屏幕上，如此等等。进而可根据上述需要设计出类 Cstudent 的每一个成员函数以及各函数的具体功能。

(2) 往 binary 文件中写数据时，是在 Cstudent 类的某个成员函数中通过使用类似于“ fout.write((char *)(this), sizeof(*this));”形式的语句来完成；而从 binary 文件中读数据时则使用类似于“ fin.read((char *)(this), sizeof(*this));”的调用语句。上述第一参数的 this 指针均指向 Cstudent 类的当前调用者对象，但注意，必须将该参数转换为所要求的“ char *”指针类型的。若准备往“ f01.dat”文件的尾部追加对象数据时，要使用“ ofstream fout("f01.dat", ios::appios::binary);”的方式来打开用于追加的二进制文件“ f01.dat”，其中的“ ios::app”意味着追加，而“ ios:: binary”则指定使用二进制格式。

[解答程序]

```
#include <fstream.h>
#include <string.h>

class CStudent                                //自定义类(类型)Cstudent
{
    ...                                       //与题目处相同，略
};

void CStudent::Init_DataFile(CStudent* stu, int n)
{
    //将程序(主函数)stu 数组中准备好的 n 个类对象初始数据
    //依次写出到自定义二进制磁盘文件中
    cout<<"---Init_DataFile( ), "<<n<<" RECs will be saved to "f01.dat"---"<<endl;
    ofstream fout("f01.dat", ios::binary);
    //打开用于写的二进制文件“ f01.dat”
    for (int i=0; i<n; i++)
        fout.write( (char *)&stu[i], sizeof(CStudent));
        //将程序中准备好的 n 个初始类对象数据依次写出到自定义二进
        //制磁盘文件中
    fout.close();                            //关闭文件“ f01.dat”
}

void CStudent::Read_Append_1Rec( )
```

```

{
    //从键盘读入一个类对象数据，使用 write 将该类对象数据
    //追加到自定义二进制磁盘文件尾部
    cout<<"-----Read_Append_1Rec( )-----"<<endl;
    ofstream fout("f01.dat", ios::appios::binary);
    //打开用于追加的二进制文件“ f01.dat”
    cout<<"input REC's inf——";
    cout<<"num, name, cla, sex, age, math, chinese, eng, rec:"<<endl;
    //屏幕提示
    cin>>num>>name>>cla>>sex>>age>>math>>chinese>>eng>>rec;
    //从键盘输入每一类对象的九个数据成员值
    fout.write( (char *) (this), sizeof(*this));
    //将类对象数据一次性写出到与流对象 fout 相关联的“ f01.dat” 文件中
    fout.close( );           //关闭文件“ f01.dat”
}

void CStudent::Display_allRec( )
{
    //使用“ read” 从文件读出所有对象数据并显示在屏幕上
    cout<<"-----Display_allRec( )-----"<<endl;
    ifstream fin("f01.dat", ios::binary); //打开用于读的二进制文件“ f01.dat”
    fin.read( (char *) (this), sizeof(*this)); //读入一个类对象数据放入 *this
    while (!fin.eof( ))           //若尚未到达文件末时，继续循环处理
    {
        cout<<(*this);           //输出当前对象(的各数据成员值)
        fin.read( (char *) (this), sizeof(*this) );
        //再读出一个类对象数据放入 *this
    }
    fin.close( );                 //关闭文件“ f01.dat”
}

ostream& operator << (ostream& out, CStudent& stu)
{
    //重载运算符“ <<”，从而可以直接输出 CStudent 类对象
    //实际要将对象 stu 的各数据成员值显示在屏幕上
    out<<stu.num<<" "<<stu.name<<" "<<stu.cla<<" "<<stu.sex<<" "<<stu.age<<" ";
    out<<stu.math<<" "<<stu.chinese<<" "<<stu.eng<<" "<<stu.rec<<endl;
    return out;                   //返回 ostream 类对象 out
}

```



```

void main( )                //主函数，对所定义的类型进行使用
{
    ...                      //与题目处相同，略
}

```

[输出结果]

与题目处相同，略。

11.20 对 binary 型学生数据文件管理程序进一步扩充

对范例 11.19 中定义的 CStudent 类(类型)进行改进，增加新的成员函数，以实现新功能。详细描述与要求如下：

采用 binary 文件形式对类对象数据进行存储与读写处理(使用 write 将类对象数据写出到某个自定义二进制磁盘文件中，而后再使用 read 读出这些类对象数据并进行处理)。

自定义一个类(类型) CStudent，它具有如下九个数据成员：学号、姓名、班级、性别、年龄、数学成绩、语文成绩、英语成绩、奖惩记录。并具有八个成员函数，而且对输出运算符“<<”进行重载，其类定义格式如下：

```

class CStudent                //自定义类(类型)Cstudent
{
public:
    long num;                  //学号，共九个数据成员
    char name[20];             //姓名
    int cla;                   //班级
    char sex;                  //性别
    int age;                   //年龄
    float math, chinese, eng;  //数学成绩、语文成绩、英语成绩
    char rec[20];              //奖惩记录
    void Init_DataFile(CStudent* stu, int n); //共八个成员函数
        //将程序(主函数)中准备好的 stu 数组中的 n 个类对象初始数据
        //依次写出到自定义二进制磁盘文件中
    void Read_Append_1Rec( );
        //从键盘读入一个类对象数据，使用 write 将该类对象数据
        //追加到自定义二进制磁盘文件尾部
    void Display_allRec( ); //使用 read 从文件读出所有对象数据并显示在屏幕上
    bool Search_BasedNum(long num0);
        //按学号 num0 从文件检索对象并显示在屏幕上
}

```

```

bool Search_BasedName(char* name0);
    //按姓名 name0 从文件检索对象并显示在屏幕上
int AllStudent_in_Score(char* course, int scr1, int scr2);
    //统计文件中 course 课程成绩处于 scr1 到 scr2 分数段的
    //学生情况(显示于屏幕上), 并返回统计人数
void Delete_BasedNum(long num0);
    //按学号 num0 从文件检索对象并将该对象从文件中删除
void Delete_BasedName(char* name0);
    //按姓名 name0 从文件检索对象并将该对象从文件中删除
friend ostream& operator << (ostream& out, CStudent& stu); //一个友元函数
    //重载运算符“ <<”, 从而可以直接输出 CStudent 类对象
};

```

进而可编写如下形式的主函数, 说明 CStudent 类对象, 并通过调用其成员函数完成数据文件的处理。

```

void main( )                //主函数, 对所定义的类进行使用
{
    const int n=3;
    CStudent stu[n] =        //事先准备好的 n 个类对象初始数据
    {
        {100001, "li", 1, 'm', 16, 89, 98, 94, "SanHaoSheng"},
        {100002, "ma", 1, 'f', 17, 83, 90, 89, "BanGanBu"},
        {100003, "wu", 2, 'f', 17, 92, 88, 90, "nothing" }
    }
    CStudent ss;              //CStudent 类对象 ss
    ss.Init_DataFile(stu, n);
        //将程序(主函数)中准备好的 n 个类对象初始数据
        //依次写出到自定义二进制磁盘文件中
    ss.Display_allRec( );     //使用 read 从文件读出所有对象数据并显示在屏幕上
    ss.Read_Append_1Rec( );
        //从键盘读入一个类对象数据, 使用 write 将该类对象数据
        //追加到自定义二进制磁盘文件尾部
    ss.Search_BasedName("li"); //按名检索
        //按姓名“ li” 从文件检索对象并显示在屏幕上
    ss.Search_BasedNum(100002); //按学号检索
        //按学号 100002 从文件检索对象并显示在屏幕上
    int sum=ss.AllStudent_in_Score("eng", 90, 100);
        //统计文件中“ eng” 课程成绩处于 90 到 100 分数段的
        //学生情况(显示于屏幕上), 并返回统计人数
}

```

```

        cout<<"sum="<<sum<<endl;
        ss.Delete_BasedNum(100003);    //按学号删除
            //按学号 100003 从文件检索对象并将该对象从文件中删除
        ss.Delete_BasedName("li");      //按名删除
            //按姓名“ li” 从文件检索对象并将该对象从文件中删除
        ss.Display_allRec( );
            //使用 read 从文件读出所有对象数据并显示在屏幕上
    }

```

程序执行后的显示结果如下:

```

---Init_DataFile( ), 3 RECs will be saved to “ f01.dat” ---
-----Display_allRec( )-----
100001 li 1 m 16 89 98 94 SanHaoSheng
100002 ma 1 f 17 83 90 89 BanGanBu
100003 wu 2 f 17 92 88 90 nothing
-----Read_Append_1Rec( )-----
input REC's inf -- num, name, cla, sex, age, math, chinese, eng, rec:
100004 yu 2 m 18 90 88 84 nothing
-----Search_BasedName("li")-----
100001 li 1 m 16 89 98 94 SanHaoSheng
-----Search_BasedNum(100002)-----
100002 ma 1 f 17 83 90 89 BanGanBu
-----AllStudent_in_Score("eng",90,100)-----
100001 li 1 m 16 89 98 94 SanHaoSheng
100003 wu 2 f 17 92 88 90 nothing
sum=2
-----Delete_BasedNum(100003)-----
100003 wu 2 f 17 92 88 90 nothing
-----Delete_BasedName("li")-----
100001 li 1 m 16 89 98 94 SanHaoSheng
-----Display_allRec( )-----
100002 ma 1 f 17 83 90 89 BanGanBu
100004 yu 2 m 18 90 88 84 nothing

```

[分析提示]

(1) 本范例对上一范例的功能进行扩充,但仍要处理类 `Cstudent` 的对象数据:将一批这种对象数据存储到某个 `binary` 类型的文件中长期保存起来;从键盘读入一个类对象数据并将它追加到该 `binary` 文件的尾部;将 `binary` 文件中保存的各对象数据显示在屏幕上;按学号或按姓名从文件检索对象(数据)并显示在屏幕上;按学号或按姓名从文件

检索对象(数据)并将该对象(数据)从文件中删除;将某一指定科目课程成绩处于某一指定分数段内的学生情况信息(文件中的对象数据)显示在屏幕上,并统计满足这种条件的人数,如此等等。进而可根据上述的各种操作需求,详细设计出类 Cstudent 的每一个成员函数以及各函数要完成的具体功能。

(2) 注意,对文件的处理都是按顺序进行的(并没配合使用“ seekg”或“ seekp”那样的函数来进行所谓的随机存取处理方式)。即是说,每一成员函数要完成对文件的操作,都要先打开文件,而后从头开始按顺序逐个地对对象数据进行读入或写出处理,直到完成规定的功能后关闭文件而结束。当然,还可进一步按照随机存取处理方式来实现本范例,请读者自己改进并实现。

[解答程序]

```
#include <fstream.h>
#include <string.h>
class CStudent //自定义类(类型)Cstudent
{
    ... //与题目处相同,略
};

void CStudent::Init_DataFile(CStudent* stu, int n)
{
    //将程序(主函数)中准备好的 stu 数组中的 n 个类对象初始数据
    //依次写出到自定义二进制磁盘文件中
    cout<<"---Init_DataFile( ), "<<n<<" RECs will be saved to "f01.dat"---"<<endl;
    ofstream fout("f01.dat", ios::binary);
    //打开用于写的二进制文件“ f01.dat”
    for (int i=0; i<n; i++)
        fout.write( (char *)&stu[i], sizeof(CStudent));
    //将程序中准备好的 n 个初始类对象数据依次写出到自定义二进
    //制磁盘文件中
    fout.close( ); //关闭文件“ f01.dat”
}

void CStudent::Read_Append_1Rec( )
{
    //从键盘读入一个类对象数据,使用 write 将该类对象数据
    //追加到自定义二进制磁盘文件尾部
    cout<<"-----Read_Append_1Rec( )-----"<<endl;
    ofstream fout("f01.dat", ios::appios::binary);
    //打开用于追加的二进制文件“ f01.dat”
```

```

        cout<<"input REC's inf -- ";
        cout<<"num, name, cla, sex, age, math, chinese, eng, rec:"<<endl;
        cin>>num>>name>>cla>>sex>>age>>math>>chinese>>eng>>rec;
        //从键盘输入每一类对象的九个数据成员值
        fout.write( (char *)(this), sizeof(*this));
        //将类对象数据一次性写出到与流对象 fout 相关联的“ f01.dat” 文件中
        fout.close( );           //关闭文件“ f01.dat”
    }

void CStudent::Display_allRec( )
{
    //使用 read 从文件读出所有对象数据并显示在屏幕上
    cout<<"-----Display_allRec( )-----"<<endl;
    ifstream fin("f01.dat", ios::binary);    //打开用于读的二进制文件“ f01.dat”
    fin.read( (char *)(this), sizeof(*this)); //读入一个类对象数据放入 *this
    while (!fin.eof( ))                      //若尚未到达文件末时，继续循环处理
    {
        cout<<(*this);                      //输出当前对象(的各数据成员值)
        fin.read( (char *)(this), sizeof(*this));
        //再读出一个类对象数据放入 *this
    }
    fin.close( );                          //关闭文件“ f01.dat”
}

bool CStudent::Search_BasedNum(long num0)
{
    //按学号 num0 从文件检索对象并显示在屏幕上
    cout<<"-----Search_BasedNum("<<num0<<")-----"<<endl;
    ifstream fin("f01.dat", ios::binary); //打开用于读的二进制文件“ f01.dat”
    //而后再使用 read 读出这些类对象数据并进行处理
    fin.read( (char *)(this), sizeof(*this)); //读入一个类对象数据放入 *this
    while (!fin.eof( ))                      //若尚未到达文件末时，继续循环处理
    {
        if( num==num0 )                    //若找到学号为 num0 的对象，输出信息后返回
        {
            cout<<(*this);                //输出当前对象(的各数据成员值)
            fin.close( );                  //关闭文件“ f01.dat”
            return true;
        }
    }
}

```

```

        fin.read( (char *)(this), sizeof(*this));
                                //再读出一个类对象数据放入 *this
    }
    fin.close();                //关闭文件“ f01.dat”
    cout<<"No such a num!"<<endl;    //没找到时提示信息
    return false;
}

bool CStudent::Search_BasedName(char* name0)
{
    //按姓名 name0 从文件检索对象并显示在屏幕上
    cout<<"-----Search_BasedName(\""<<name0<<"\"")-----"<<endl;
    ifstream fin("f01.dat", ios::binary); //打开用于读的二进制文件“ f01.dat”
        //而后再使用 read 读出这些类对象数据并进行处理
    fin.read( (char *)(this), sizeof(*this)); //读入一个类对象数据放入 *this
    while (!fin.eof())                //若尚未到达文件末时，继续循环处理
    {
        if( strcmp(name, name0)==0 )
        {
            //若找到姓名为 name0 的对象，输出信息后返回
            cout<<(*this);            //输出当前对象(的各数据成员值)
            fin.close();              //关闭文件“ f01.dat”
            return true;
        }
        fin.read( (char *)(this), sizeof(*this));
                                //再读出一个类对象数据放入 *this
    }
    fin.close();                //关闭文件“ f01.dat”
    cout<<"No such a name!"<<endl;
    return false;
}

int CStudent::AllStudent_in_Score(char* course, int scr1, int scr2)
{
    //统计文件中 course 课程成绩处于 scr1 到 scr2 分数段的
    //学生情况(显示于屏幕上)，并返回统计人数
    int sum=0;                    //统计人数
    cout<<"-----AllStudent_in_Score(\""<<course<<"\", "<<scr1<<";<<scr2<<")-----"<<endl;
    ifstream fin("f01.dat", ios::binary); //打开用于读的二进制文件“ f01.dat”

```

```

//而后再使用 read 读出这些类对象数据并进行处理
fin.read( (char *)(this), sizeof(*this));           //读入一个类对象数据放入*this
while (!fin.eof( ))                                //若尚未到达文件末时，继续循环处理
{
    switch(course[0])
    {
        case 'm':
        case 'M':                                    //课程为数学的处理
            if( math>=scr1 && math<=scr2 ) //处于 scr1 到 scr2 分数段的学生
            {                                         //将情况显示于屏幕上，并累加人数
                cout<<(*this);                       //输出当前对象(的各数据成员值)
                sum++;
            }
            break;
        case 'c':
        case 'C':                                    //课程为语文的处理
            if( chinese>=scr1 && chinese<=scr2 )
            {                                         //处于 scr1 到 scr2 分数段的学生
                cout<<(*this);                       //将情况显示于屏幕上，并累加人数
                cout<<(*this);                       //输出当前对象(的各数据成员值)
                sum++;
            }
            break;
        case 'e':
        case 'E':                                    //课程为外语的处理
            if( eng>=scr1 && eng<=scr2 ) //处于 scr1 到 scr2 分数段的学生
            {                                         //将情况显示于屏幕上，并累加人数
                cout<<(*this);                       //输出当前对象(的各数据成员值)
                sum++;
            }
            break;
    }
    fin.read( (char *)(this), sizeof(*this));       //再读出一个类对象数据放入*this
}
fin.close( );                                       //关闭文件“ f01.dat ”
return sum;                                        //返回统计出的人数
}

```

```

void CStudent::Delete_BasedNum(long num0)
{
    //按学号 num0 从文件检索对象并将该对象从文件中删除
    bool delMark=0;                //尚未找到欲删除的对象 num0
    cout<<"-----Delete_BasedNum("<<num0<<")-----"<<endl;
    ofstream fout("tmp.dat", ios::binary);    //打开用于写的二进制文件“ tmp.dat”
    ifstream fin("f01.dat", ios::binary);    //打开用于读的二进制文件“ f01.dat”
        //而后再使用 read 读出这些类对象数据并进行处理
    fin.read( (char *)(this), sizeof(*this));    //读入一个类对象数据放入 *this
    while (!fin.eof( ))                //若尚未到达文件末时，继续循环处理
    {
        if( num==num0 )                //若找到学号为 num0 的对象时，该对象
        {
            //不写出到“ tmp.dat” 中——意味着进行了删除
            cout<<(*this);                //屏幕输出当前对象(的各数据成员值)
            delMark=1;                //标记 delMark 变量
        }
        else                            //将对象写出到“ tmp.dat” 中
            fout.write( (char *)(this), sizeof(*this));
        fin.read( (char *)(this), sizeof(*this));
            //再读出一个类对象数据放入 *this
    }
    fin.close( );                //关闭文件“ f01.dat”
    fout.close( );                //关闭文件“ tmp.dat”
    if(delMark)
    {
        //若找到了欲删除的对象 num0 时，用“ tmp.dat” 文件的内容去充当“ f01.dat”
        ofstream fout("f01.dat", ios::binary);
            //打开用于写的二进制文件“ f01.dat”
        ifstream fin("tmp.dat", ios::binary);
            //打开用于读的二进制文件“ tmp.dat”
            //而后再使用 read 读出类对象数据并进行处理
        fin.read( (char *)(this), sizeof(*this));    //读入一个类对象数据放入 *this
        while (!fin.eof( ))                //若尚未到达文件末时，继续循环处理
        {
            fout.write( (char *)(this), sizeof(*this));    //写出 *this 到“ f01.dat”
            fin.read( (char *)(this), sizeof(*this));
                //再读出一个类对象数据放入 *this
        }
    }
}

```



```

        fin.close( );                //关闭文件“ tmp.dat”
        fout.close( );              //关闭文件“ f01.dat”
    }
    else                             //若没找到欲删除的对象 num0 时
        cout<<"No such a num!"<<endl;
}

void CStudent::Delete_BasedName(char* name0)
{
    //按姓名 name0 从文件检索对象并将该对象从文件中删除
    bool delMark=0;                  //尚未找到欲删除的对象 name0
    cout<<"-----Delete_BasedName(\"" <<name0<<"\" )-----"<<endl;
    ofstream fout("tmp.dat", ios::binary);    //打开用于写的二进制文件“ tmp.dat”
    ifstream fin("f01.dat", ios::binary);      //打开用于读的二进制文件“ f01.dat”
                                           //而后再使用 read 读出类对象数据并进行处理
    fin.read( (char *)(this), sizeof(*this)); //读入一个类对象数据放入*this
    while (!fin.eof( ))                    //若尚未到达文件末时，继续循环处理
    {
        if( strcmp(name, name0)==0 )
        {
            //若找到姓名为 name0 的对象时，该对象
            //不写出到“ tmp.dat” 中——意味着进行了删除
            cout<<(*this); //输出当前对象(的各数据成员值)
            delMark=1;     //标记 delMark 变量
        }
        else                //将对象写出到“ tmp.dat” 中
            fout.write( (char *)(this), sizeof(*this));
        fin.read( (char *)(this), sizeof(*this));
        //再读出一个类对象数据放入*this
    }
    fin.close( );          //关闭文件“ f01.dat”
    fout.close( );         //关闭文件“ tmp.dat”
    if(delMark)
    {
        //若找到了欲删除的对象时，用“ tmp.dat” 文件的内容去充当“ f01.dat”
        ofstream fout("f01.dat", ios::binary);
        //打开用于写的二进制文件“ f01.dat”
        ifstream fin("tmp.dat", ios::binary);
        //打开用于读的二进制文件“ tmp.dat”
    }
}

```

```

        //而后再使用 read 读出类对象数据并进行处理
        fin.read( (char *)(this), sizeof(*this)); //读入一个类对象数据放入*this
        while (!fin.eof()) //若尚未到达文件末时, 继续循环处理
        {
            fout.write( (char *)(this), sizeof(*this)); //写出*this 到“ f01.dat”
            fin.read( (char *)(this), sizeof(*this));
                                                    //再读出一个类对象数据放入*this
        }
        fin.close(); //关闭文件“ tmp.dat”
        fout.close(); //关闭文件“ f01.dat”
    }
    else
        cout<<"No such a name!"<<endl;
}

ostream& operator << (ostream& out, CStudent& stu)
{
    //重载运算符“ <<”, 从而可以直接输出 CStudent 类对象
    //将对象 stu 的各数据成员值显示在屏幕上
    out<<stu.num<<" "<<stu.name<<" "<<stu.cla<<" "<<stu.sex<<" "<<stu.age<<" ";
    out<<stu.math<<" "<<stu.chinese<<" "<<stu.eng<<" "<<stu.rec<<endl;
    return out; //返回 ostream 类对象 out
}

void main() //主函数, 对所定义的类进行使用
{
    ... //与题目处相同, 略
}

```

[输出结果]

与题目处相同, 略。

参 考 文 献

- 1 Bjarne Stroustrup. The C++ Programming Language (Special Edition)(影印版). 北京: 高等教育出版社, 2001
- 2 Nell Dale, Chip Weems & Mark Headington. Programming in C++ (Second Edition)(影印版). 北京: 高等教育出版社, 2001
- 3 James P. Cohoon, Jack W. Davidson. C++ Program Design: An Introduction to Programming and Object-Oriented Design (Third Edition). 北京: 清华大学出版社, 2002
- 4 刘璟. 高级语言 C++程序设计. 北京: 高等教育出版社, 2001
- 5 吕凤翥. C++语言程序设计. 北京: 电子工业出版社, 2001
- 6 钱能. C++程序设计教程. 北京: 清华大学出版社, 2001
- 7 郑莉, 董渊, 傅仕星. C++语言程序设计. 北京: 清华大学出版社, 2000
- 8 和克智. C++程序设计(第2版). 西安: 西安交通大学出版社, 1999
- 9 Kayshav Dattatri 著, C++面向对象高效编程. 潇湘工作室译. 北京: 人民邮电出版社, 2000
- 10 Bruce Eckel 著. C++编程思想 Thinking in C++. 刘宗田, 邢大红, 孙慧杰译. 北京: 机械工业出版社, 2001
- 11 Kate Gregory 著, 前导工作室译. Visual C++6 开发使用手册. 北京: 机械工业出版社, 1999
- 12 林森, 高峰霞, 罗丽琼, 奚红宇. Visual C++ 6.0 使用与开发. 北京: 清华大学出版社, 1998